cadence®

Voltus Stylus Common UI Text Reference Manual

Product Version 25.13
April 2026

© 2026 Cadence Design Systems, Inc. All rights reserved.
Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used solely for personal, informational, and noncommercial purposes;
2. The publication may not be modified in any way;
3. Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and
4. Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence's customer in accordance with, a written agreement between Cadence and its customer. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Cadence is committed to using respectful language in our code and communications. We are also active in the removal and replacement of inappropriate language from existing content. This product

documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

About This Manual 6
Audience 6
Conventions Used in This Manual 6
Related Documents 8
Voltus Product Documentation 8
1 9
Understanding the Common UI Interface 9
Stylus Introduction 9
2 11
Advanced Timing Tcl Scripting Commands 11
Overview 11
get_* Commands 11
Examples 12
get_activity 13
get_cells 18
get_lib_cells 22
get_nets 25
get_pins 27
get_ports 29
get_power 31
3 37
Basic Database Access Tcl Commands and Attributes 37
add_thru_substrate_insts 38
compress_special 41
convert_dbu_to_um 44
convert_um_to_dbu 45
create_cell_obs 47
decompress_special 49
define_attribute 51
delete_cell_obs 56
delete_floating_nets 57

delete_obj 58
delete_thru_substrate_insts 60
get_computed_shapes 61
get_db 65
get_db Category Attributes 83
get_edge 85
get_obj_in_area 87
get_rect 93
get_transform_shapes 96
get_via_pillars 99
init_hier_cell 100
is_attribute 101
report_floating_nets 103
report_metal_stack 106
report_via_def 108
reset_db 110
set_db 113
set_db Category Attribute 116
set_via_pillars 118
sort_db 121
tech Category Attributes 124
uniquify 125
write_preserves 129
4 131
Check Commands and Attributes 131
check_ac_limits 131
check_connectivity 132
check_pg_shorts 136
check_power_vias 137
create_marker 146
delete_markers 148
gui_show_markers 149
read_markers 152
report_markers 155
check_ac_limit Category Attributes 158
5 176

Flowkit Commands and Attributes 176
check_flow 177
create_flow 179
create_flow_step 181
define_feature 183
delete_flow_config 186
edit_flow 187
end_flow 190
flow Category Attributes 190
flowtool Category Attributes 204
flowtool_exit_timeout 205
flowtool_extra_arguments 205
flowtool_metrics_qor_html 205
flowtool_metrics_qor_excel 205
flowtool_metrics_qor_text 205
flowtool_metrics_qor_vivid 206
flowtool_predict_full_names 206
flowtool_summary_tcl 206
get_feature 206
get_flow_config 208
init_flow 209
is_feature 210
is_flow 211
read_flow 212
report_flow 213
reset_flow 215
run_flow 218
schedule_flow 221
set_feature 223
set_flow_config 224
set_flowkit_read_db_args 225
set_flowkit_write_db_args 229
write_flow 234
write_flow_template 236
6 242
General Commands and Attributes 242

alias 244
check_ccr 246
check_netlist 247
convert_legacy_to_common_ui 250
create_metric_page 251
create_metric_table 252
create_metric_table_heading 254
create_snapshot 255
decrypt 257
define_metric 258
define_proc 263
define_variables 269
delete_dangling_ports 270
delete_metric 271
delete_metric_page 272
delete_metric_run 273
delete_metric_table 274
delete_metric_table_heading 275
deselect_obj 276
encrypt 277
eval_legacy 278
get_all_layers 279
get_common_ui_map 280
get_legacy 281
get_message 282
get_metric 283
get_metric_alias 285
get_metric_config 286
get_metric_definition 287
get_metric_header 288
get_preference 289
gui_bind_key 290
gui_move_cursor 291
help 292
is_common_ui_mode 294
man 295
metric Category Attributes 296

pop_snapshot_stack 301
print_message 303
program Category Attributes 305
push_snapshot_stack 306
puts 308
read_metric 309
read_metric_config 310
read_metric_html_package 311
redirect 312
report_area 314
report_dangling_ports 318
report_hidden_usage 319
report_messages 321
report_metric 323
report_netlist_statistics 324
report_obj 325
report_obj Category Attributes 328
report_qor 328
reset_metric_config 330
reset_metric_run_id 331
resume 332
run_metric_category 333
select_obj 334
set_compress_level 335
set_layer_preference 336
set_license_check 338
set_limited_access_feature 341
set_log_post_cmd 342
set_message 343
set_metric 346
set_metric_alias 347
set_metric_header 348
set_preference 349
set_proc_verbose 365
source 366
suspend 367
system Category Attributes 368

uniquify_filename 374
update_inst_name 375
update_metric 376
update_metric_definition 378
view_log 383
write_metric 384
write_metric_config 386
rename_snapshot 387
delete_floating_constants 388
rename_obj 390
7 393
GUI Commands 393
clear_all_rulers 395
create_gui_shape 396
create_ruler 398
gui_add_marker 399
gui_add_ui 400
gui_attach_to_cursor 404
gui_clear_highlight 405
gui_create_user_bundle_net 407
gui_delete_objs 409
gui_delete_ui 410
gui_deselect 411
gui_delete_user_bundle_net 412
gui_dim_foreground 413
gui_write_picture 414
gui_find_ui 417
gui_fit 422
gui_get_ui 423
gui_get_visible_nets 429
gui_hide 430
gui_highlight 431
gui_highlight_pin 435
gui_highlight_pin_connection 438
gui_list_marker 442
gui_open_schematic 443

gui_pan 445
gui_pan_center 446
gui_pan_page 447
gui_redraw 448
gui_remove_marker 449
gui_select 450
gui_select_highlighted 452
gui_set_obj_color 453
gui_set_power_rail_display 456
gui_set_power_rail_layers_nets 468
gui_set_tool 474
gui_set_ui 476
gui_set_visible_nets 483
gui_show 484
gui_show_edge_number 485
gui_show_user_bundle_nets 487
gui_view_box 489
gui_zoom 490
read_power_rail_results 493
read_temperature_map_file 500
report_power_rail_results 500
view_last 510
view_next 511
write_to_gif 512

8 514

Import and Export Commands and Attributes 514

add_route_via_defs 515
add_route_vias Category Attributes 518
check_design 519
create_hier_design 521
design Category Attributes 524
init_design 530
init_hier_design 533
init Category Attributes 535
read_db 538
read_def 543

read_hier_db 548
read_libs 549
read_mmmc 551
read_netlist 554
read_physical 556
read_physical Category Attributes 560
read_sdc 562
read_twf 563
relink_db_files 566
reset_design 568
set_cell_power_domain 569
set_power_def_files 571
set_power_lib_files 572
set_power_spef_files 574
specify_oa 575
write_db 577
write_db Category Attributes 583
write_def 585
write_def Category Attributes 594
write_hier_db 595
write_mmmc 596
write_netlist Category Attributes 597
write_via_defs 598
write_twf 601
9 604
Interactive ECO Commands and Attributes 604
connect 604
connect_hpin 608
connect_pin 610
create_hinst 613
create_hnet 615
create_hport 618
create_inst 620
create_module 623
create_port 624
delete_inst 625

delete_notch_fill 626
disconnect 627
disconnect_hpin 629
disconnect_pin 630
gui_show_buffer_tree 631
update_inst 632
10 635
Low Power Commands and Attributes 635
check_power_domains 636
clone_msv_gate 639
commit_power_intent 641
cut_power_domain_by_overlaps 642
delete_power_intent 643
get_cpf_user_attributes 644
power_intent Category Attributes 645
read_power_intent 645
report_isolation 647
report_partition_boundary_ports 649
report_power_domains 650
report_shifters 654
report_voltage 657
update_power_domain 658
write_power_intent 663
11 666
Multiple-CPU Processing Commands 666
check_multi_cpu_usage 666
get_distributed_hosts 668
get_multi_cpu_usage 670
monitor_distributed_hosts 672
report_resource 674
set_distributed_hosts 678
set_multi_cpu_usage 685
12 690
OpenAccess Commands and Attributes 690
compare_oa_cellview 690

create_oa_lib 692
create_oa_lib_property 695
get_oa_default_rule_lib 697
get_oa_design_data 700
oa_read_write Category Attributes 702
read_oa 714
register_trigger 717
unlock_oa_design 718
update_oa_lib 719
write_lef_library 721
write_oa_techfile 723
13 726
Power Calculation Commands and Attributes 726
encrypt_thermal_file 727
extract_metal_density 729
get_default_switching_activity 730
get_dynamic_power_simulation 731
merge_vtm 732
power Category Attributes 734
propagate_activity 813
query_power_data 815
read_activity_file 818
read_activity_map_file 827
report_annotated_parasitics 830
read_power_db 834
report_inst_power 836
report_power 837
report_thermal 859
report_unannotated_nets 860
reset_power_activity 862
set_default_switching_activity 863
set_dynamic_power_simulation 868
set_instance_temperature_file 870
set_metal_density_options 871
set_power 875
set_power_analysis_temperature 886

set_power_include_file 887
set_power_output_dir 889
set_switching_activity 890
set_thermal_analysis_mode 894
set_twf_attribute 896
set_virtual_clock_network_parameters 900
write_power_constraints 902
write_tcf 903
write_thermal_model 905
14 910
Power Route Category Attributes 910
route_special Category Attributes 910
15 920
Power-Grid Library Commands 920
check_pg_library 921
check_pgv 932
merge_pg_library 939
rename_pg_library 942
set_advanced_pg_library_mode 944
set_pg_library_mode 977
trim_pg_library 1011
write_pg_library 1013
write_tech_pg_lib 1015
write_xpgv 1019
16 1024
Placement Commands and Attributes 1024
add_fillers Category Attributes 1024
gui_clear_scan_chains 1026
gui_clear_spare_cells 1027
gui_show_scan_chains 1028
gui_show_spare_cells 1029
write_physical_context_data 1030
read_physical_context_data 1031
17 1033
Rail Analysis Commands 1033

add_what_if_shapes 1035
analyze_jitter 1040
connect_die_package 1045
convert_stream_to_def 1047
extract_package 1057
generate_esd_rlrp_report 1060
get_ir_diagnostics 1063
get_voltus_diagnostics_mode 1071
map_dies 1073
report_differential_voltage 1077
report_irdrop_regions 1079
report_esd_network 1084
report_esd_voltage 1087
report_joule_heat 1090
report_noise_margin 1093
report_package 1096
report_rail 1098
report_resistance 1100
report_self_heat 1109
report_signal_resistance 1112
scale_what_if_capacitance 1114
scale_what_if_current 1117
scale_what_if_resistance 1120
set_advanced_package_options 1123
set_advanced_rail_options 1124
set_die_model 1126
set_dynamic_rail_simulation 1129
set_multi_die_analysis_mode 1132
set_net_group 1135
set_off_chip_package_trace 1137
set_package 1139
set_pg_nets 1142
set_power_data 1145
set_power_pads 1150
set_rail_analysis_config 1156
set_rail_analysis_domain 1240
set_self_heat_analysis_mode 1243

set_voltage_regulator_module 1247
set_voltus_diagnostics_mode 1249
view_dynamic_movie 1257
view_dynamic_waveform 1258
view_package_results 1262
write_current_region 1263
write_die_model 1268
write_hier_view 1280
write_power_pads 1286
write_power_up_report 1292
18 1296
RC Extraction Commands and Attributes 1296
create_parasitic_coordinate_transform 1296
extract_parasitics 1301
extract_rc Category Attributes 1302
read_parasitics 1311
read_spef 1314
report_net_parasitics 1321
report_rcdb 1323
report_unit_parasitics 1325
write_parasitics 1328
write_rcdb 1330
19 1332
Signal ElectroMigration Commands 1332
create_cell_signal_em_model 1332
create_top_scope 1338
report_freq_violation 1340
set_max_cap_per_freq 1345
set_max_transition_per_freq 1348
20 1352
Unix Commands 1352
flowtool 1352
lef2oa 1355
oa2lef 1360
voltus 1362

Index 1371
A-Z 1371
22 1381
RTL-to-Gate Rule Mapping 1381

About This Manual

The Cadence® Voltus™ IC Power Integrity Solution is a full-chip, cell-level power signoff tool that provides accurate, fast, and high-capacity analysis and optimization technologies. This manual describes the syntax for the Voltus Stylus Common UI (Common UI) text commands.

The chapters are presented alphabetically by general command functionality. Within each chapter, the text commands appear alphabetically.

Audience

This manual is written for experienced designers of digital integrated circuits. Such designers must be familiar with design planning, placement and routing, block implementation, chip assembly, and design verification. Designers must also have a solid understanding of UNIX and Tcl/Tk programming.

Conventions Used in This Manual

This section describes the typographic and syntax conventions used in this manual.

`text`	Indicates text that you must type exactly as shown. For example: `set_message -severity info`
`text`	Indicates information for which you must substitute a name or value. In the following example, you must substitute the name of a specific file for *`file_name`*: `report_area -out_file` *`file_name`*
text	Indicates the following: • Text found in the graphical user interface (GUI), including form names, button labels, and field names • Terms that are new to the manual, are the subject of discussion, or need special emphasis • Titles of manuals

`[ ]`	Indicates optional arguments. In the following example, you can specify none, one, or both of the bracketed arguments: `command [-arg1] [arg2 value]`
`[ | ]`	Indicates an optional choice from a mutually exclusive list. In the following example, you can specify any of the arguments or none of the arguments, but you cannot specify more than one: `command [arg1 | arg2 | arg3 | arg4]`
`{ | }`	Indicates a required choice from a mutually exclusive list. In the following example, you must specify one, and only one, of the arguments: `command {arg1 | arg2 | arg3}`
`{[ ] [ ]}`	Indicates a required choice of one or more items in a list. In the following example, you must choose one argument from the list, but you can choose more than one: `command {[arg1] [arg2] [arg3]}`
`{ }`	Indicates curly braces that must be entered with the command syntax. In the following example, you must type the curly braces: `command arg1 {x y}`
. . .	Indicates that you can repeat the previous argument.
. . .	Indicates an omission in an example of computer output or input.
Command - Subcommand	Indicates a command sequence, which shows the order in which you choose commands and subcommands from the GUI menu. In the following example, you choose *Power* from the menu, then *Power Planning* from the submenu, and then *Add Ring* from the displayed list: *Power - Power Planning - Add Ring* This sequence opens the *Add Rings* form.

Related Documents

For more information about the Voltus family of products, see the following documents. You can access these and other Cadence documents with the Cadence Help documentation system.

Voltus Product Documentation

For more information about the Voltus family of products, see the following documents. You can access these and other Cadence documents with the Cadence Help online documentation system.

- What's New in Voltus
  Provides information about new and changed features in this release of the Voltus family of products.
- Voltus Known Problems and Solutions
  Describes important Cadence Change Requests (CCRs) for Voltus, including solutions for working around known problems.
- Voltus Menu Reference
  Provides information specific to the forms and commands available from the Voltus graphical user interface.
- Voltus User Guide
  Provides information on using various Voltus features.
- Voltus Text Command Reference
  Describes the Voltus text commands, including syntax and examples.
- README file
  Contains installation, compatibility, and other prerequisite information, including a list of Cadence Change Requests (CCRs) that were resolved in this release. You can read this file online at downloads.cadence.com.

For a complete list of documents provided with this release, see the Cadence Help online documentation system.

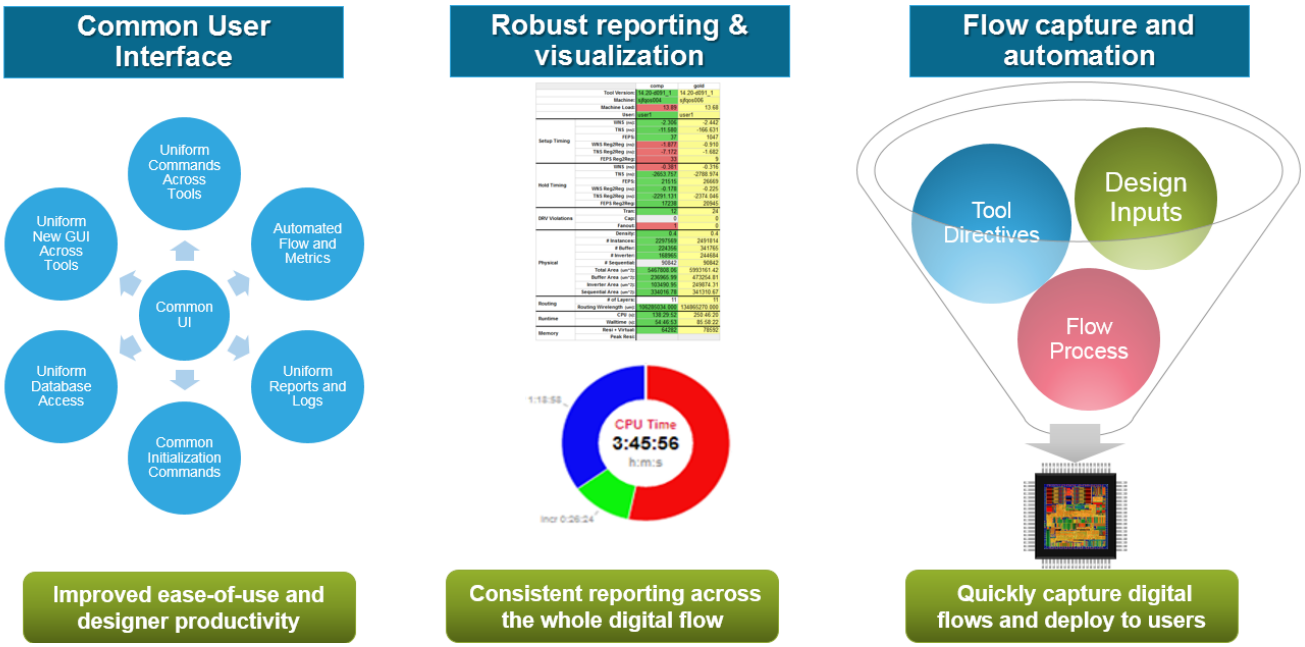
1

Understanding the Common UI Interface

Stylus Introduction

Stylus is an infrastructure that offers 3 significant facilities:

- Stylus Common User Interface offering consistent commands across the whole digital flow
- Stylus Unified Metrics capture and reporting
- Stylus Flow Kit for design flow capture and deployment



The Stylus Common User Interface (Common UI) has been designed to be used across Genus, Joules, Modus, Innovus, Tempus, and Voltus tools. By providing a common interface from RTL to signoff, the Common UI enhances user experience by making it easier to work with multiple Cadence products. The Common UI simplifies command naming and aligns common implementation methods and reporting across Cadence digital and signoff tools. For example, the processes of design initialization, database access, command consistency and timing reports are all the same across the different digital tools.

Stylus Unified Metrics enables you to generate unified and holistic design metrics, which make it easier to summarize and compare results that enables effective debugging. Metric collection has been streamlined and simplified.

Stylus Flow Kit introduces new set of flow commands, including `create_flow` and `create_flow_step`. New `flow` and `flow_step` objects store the state of the flow with the database, enabling you to quickly capture and maintain flows.

These updated Stylus interfaces and reference flow capture increased productivity by delivering a familiar interface across core implementation and signoff products. You can take advantage of consistently robust RTL-to-signoff reporting and management, as well as a customizable environment.

⚠ The Common UI is limited access for early adopters. You should contact Cadence to obtain extra support if you are interested in using the Common UI.

2

Advanced Timing Tcl Scripting Commands

- get_activity
- get_cells
- get_lib_cells
- get_nets
- get_pins
- get_ports
- get_power

Overview

The following lists the Advanced Timing Tcl Scripting commands. Here, we explain what they are used for followed by some simple examples.

get_* Commands

The `get_*` commands return a collection of items filtered by name pattern or object type. Additionally, you can use the `-filter` option for most of the `get_*` commands to filter the collection by property value.

The `filter_expression` for the `-filter` option can be created by combining several object properties using the following relational and logical operators: `>`, `<`, `==`, `!=`, `<=`, `>=`, `&&`, `||`, `=~`, `!~`, `AND`, and `OR`. You also can use parentheses to create expressions. The `get_*` commands include the following:

- `get_activity`
- `get_cells`
- `get_lib_cells`

- `get_nets`
- `get_pins`
- `get_ports`
- get_power

Examples

- To return a collection of all input ports (same as running `[all_inputs]`), use the following command:

  ```
  get_ports -filter "direction==in"
  ```

- To return a collection of pins that match `*/D`, use the following command:

  ```
  get_pins -hier -filter "hierarchical_name =~ */D"
  ```

- To return a collection of the libraries a cell belongs to, use the following command:

  ```
  get_lib_cells INVXL
  ```

get_activity

```
get_activity
[-help]
[-out_file filename]
[-net netname]
[-pin pinname]
[-port portname]
[-summary]
[-tcl_list]
[-list_of_nets_based_on_driver {primary_inputs | sequential | combinational | memory |
ICG | black_box}]
[-list_of_nets_based_on_source_of_activity_info
{activity_file|set_switching_activity|propagated|default}]
[-report_average_switching_activity]
[-report_cell_group_activity_summary]
[-inst inst_name]
```

Determines the source of activity for the specified nets/pins/ports. The power analysis engine has various sources from where activities can be annotated from, such as switching activity specification (input/sequential/global/clock gate), clock definitions/constants through SDC/TWF, activity files (VCD/TCF/SAF), user defined activity and/or propagated values.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_activity`
`-inst inst_name`	
	Specifies the top/hierarchy instance or the leaf level instance name for which average activity is to be reported.
`-list_of_nets_based_on_driver {primary_inputs | sequential | combinational | memory | ICG | black_box}`	

	Specifies to output the list of nets based on the specified net driver. You can specify one or more net drivers at a time, as shown below: `get_activity -list_of_nets_based_on_driver primary_inputs|combinational|sequential | ICG | memory | black_box` To specify multiple drivers, you can use / (means 'or') or **&** (means 'and') between two drivers, without a space.
`-list_of_nets_based_on_source_of_activity_info {activity_file|set_switching_activity|propagated|default}`	
	Specifies to output the list of nets based on the specified activity source information.
`-net` *`netname`*	Specifies the name of the net. **Note:** You can use the `get_nets` command to determine the activity source for a collection of nets.

`-out_file` *`netname`*	Specifies the output file into which the activity sources are to be written. The format of the report is:
	<net_name/pin_name> <activity_source> <duty_cycle> <transition_density>
	For user-specified activity, the transition density is calculated as = switching activity * fastest clock associated with the pin/net.
	Following is a snippet of the output:
	`n_14 user_defined_activity 0.3 3.125e+07`
	Here, 0.3 is the duty cycle, and `3.125e+07` is the transition density. The transition density of 3.125e+07 for the net is calculated by:
	`.25 (user-specified switching activity) * 1/8e-9 (8ns clock)`
	For more information, refer to the "Static Power Calculation Overview" section of the *"Static Power, IRdrop and EM Analysis"* chapter in *Voltus Stylus Common UI User Guide*.

`-pin` *`pinname`*	Specifies the name of the pin. **Note:** You can use the `get_pins` command to determine the activity source for a collection of pins.

`-port` *portname*	Specifies the name of the port. **Note:** You can use the `get_ports` command to determine the activity source for a collection of ports.
`-report_average_switching_activity`	
	Specifies to report the average activity of the given instance/hierarchy/top-level.
`-report_cell_group_activity_summary`	
	Specifies to give a summary of cell group activity.
`-summary`	Generates a detailed activity annotation report of the specified nets, pins, and ports. This report contains the following information: • Primary Inputs • Sequential Outputs • Memory/Macro Outputs • Tristate Outputs **Note:** Use the `get_activity -summary` command to report activity annotation summary without printing information for nets/pins/ports.
`-tcl_list`	Produces the report in the Tcl list format instead of a tabular format. The `-tcl_list` and `-out_file` parameters are mutually exclusive; you cannot specify them together.

Examples

- The following example returns the activity sources of all nets that match the specified pattern and writes the output information to a file named `nets.rpt`:

  ```
  get_activity -net [get_nets n*] -out_file nets.rpt
  ```

  The following is the example output of the nets.rpt file:

  ```
  int3 activity_file 0.5 5e+07
  int2 sequential_activity 0.5 2.5e+07
  int1 user_defined_activity 0.5 1e+07
  nt3 global_activity 0.5 5e+07
  ```

```
nt3 input_activity 0.5 5e+07
int5 clock_gate_activity 0.5 2.5e+07
int1 user_defined_activity 0.5 1e+07out
sequential_activity 0.5 2.5e+07
clk sdc/twf 0.5 5e+08
c_int2 propagation 0.5 5e+08
```

- The following example returns the activity source of the pin `i_4/A1` and writes the output information to a power report file named `pin.rpt`:

  ```
  get_activity -pin i_4/A1 -out_file pin.rpt
  ```

- The following command report the average activity of all nets:

  ```
  get_activity -report_average_switching_activity
  ```

  The following is the example output:

  ```
  Avg Switching activity of all nets 4.727e+17
  ```

- The following command gives a summary of cell group activity:

  ```
  get_activity -report_cell_group_activity_summary
  ```

  The following is the example output:

  ```
  Group Activity Summary Report:
  Source ->                    activity file           set_switching_activity
  propagated              default                  Total
  All nets                     2                        1
  2                       5                        10
  Break-up based on net driver:
  Primary Input                2(40%)                   0(0%)
  0(0%)                   3(60%)                   5
  Sequential                   0(0%)                    1(33.33%)
  0(0%)                   2(66.67%)                3
  Combinational                0(0%)                    0(0%)
  2(100%)                 0(0%)                    2
  Memory                       0(0%)                    0(0%)
  0(0%)                   0(0%)                    0
  ICG                          0(0%)                    0(0%)
  0(0%)                   0(0%)                    0
  Black Box                    0(0%)                    0(0%)
  0(0%)                   0(0%)                    0
  ```

- The following command gives the list of nets based on primary inputs:

  ```
  get_activity -list_of_nets_based_on_driver primary_inputs
  ```

  The following is the example output:

  ```
  D2 D1 CP2 CP1 dummy0
  ```

- The following command gives the list of nets based on primary inputs or sequential driver:
  ```
  get_activity -list_of_nets_based_on_driver primary_inputs|sequential
  ```
  The following is the example output:
  ```
  Q2 Q1 D2 D1 CP2 CP1 n1 dummy0
  ```
- The following command gives the list of nets based on primary inputs or sequential driver for the instance `ff1b`:
  ```
  get_activity -list_of_nets_based_on_driver primary_inputs|sequential -instance ff1b
  ```
- The following command gives the list of nets based on the propagated source for the instance `i2`:
  ```
  get_activity -list_of_nets_based_on_source_of_activity_info propagated -instance i2
  ```
- The following command gives a summary of cell group activity for the instance `i2`:
  ```
  get_activity -instance i2 -report_cell_group_activity_summary
  ```
- The following command gives the list of nets based on the default activity:
  ```
  get_activity -list_of_nets_based_on_source_of_activity default
  ```

  The following is the example output:
  ```
  rst shutdown isolate coef_ld start bypass test_mode se sdi Xn_in[3] Xn_in[2]
  Xn_in[1] Xn_in[0]
  Yn_in[15] Yn_in[14] Yn_in[13] Yn_in[12] Yn_in[11] Yn_in[10] Yn_in[9] Yn_in[8]
  Yn_in[7] Yn_in[6]
  ```
- The following command gives the list of nets based on the activity file:
  ```
  get_activity -list_of_nets_based_on_source_of_activity activity_file
  ```

  The following is the example output:
  ```
  ref_clk test_clk pll_clk
  ```
- The following command gives the list of nets based on the switching activity:
  ```
  get_activity -list_of_nets_based_on_source_of_activity set_switching_activity
  ```

  The following is the example output:
  ```
  rst ref_clk shutdown isolate coef_ld start bypass test_clk test_mode se sdi sdo
  Xn_in[3] Xn_in[2]
  Xn_in[1] Xn_in[0] Yn_in[15] Yn_in[14] Yn_in[13]
  ```

get_cells

```
get_cells
[-help]
[-filter expr]
[-hierarchical]
[-hsc char]
[-leaf]
[-nocase]
[-quiet]
[-regexp]
[patterns | -of_objects object_list]
```

Creates a collection of instances in the current design whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument in another command.

Parameters

`-filter` *expr*	Filters the collection with the specified expression. For any cells that match the pattern, the software evaluates the expression based on the attribute of the cell. You can filter cells using any of the cell properties supported through the `get_property` command.
`-hierarchical`	Specifies that the patterns should be matched hierarchically. If the flat name of an instance at a particular hierarchical level matches the patterns, it is included in the result. For example. if there is top level hierarchical instance `Top` and it contains another sub-instance called `FF1`, then the following command can be used to get the instance `Top/FF1` (since the given pattern will be matched with instance flat name `FF1`): get_cells -hier FF* However, the following command will display an error: get_cells -hier Top/FF*
`-hsc` *char*	Specifies the hierarchical delimiter for patterns. For example, if the hierarchical delimiter is `@`, the `sub/I1@I2` pattern matches instance `I2` within the hierarchy limited at `sub/I1`.

`-leaf`	Returns leaf cells only. When the `-leaf` parameter is not specified, the `get_cells` command returns the hierarchical instances present on a net along with the flat instances at the same hierarchical level as the specified object. **Note:** The `-leaf` parameter can only be specified with the -of_objects parameter.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects` *`object_list`*	Creates a collection of all cells associated with the specified pins or nets. You can specify a list of pins or a list of nets, but you cannot specify a list that contains both pins and nets. You can specify patterns for the pin or net names, a collection of pins or nets, or a combination of patterns and a collection.
`patterns`	Specifies patterns for instance names. Patterns include the `*` and `?` wildcard characters and collections of instances. The `get_cells` command returns a collection of instances that match the patterns. If no such instance is found, an empty collection is returned.
`-quiet`	Suppresses all error and warning messages generated when the `get_cells` command is run.

`-regexp`	Treats the specified patterns as a regular expression patterns.
	When this parameter is specified, the given patterns `start` and `end` are assumed to be matching with the start and end part of the complete object name.
	For example, the following can be used to retrieve the hierarchical instance `Top/block1/block2/FF1`:
	`get_cells Top/*/*/FF1`
	`get_cells -regexp Top/.*/FF1`
	However, the following command will display an error :
	`get_cells Top/*/FF1`
	If this parameter is specified with `-hier` parameter, the given pattern will be matched with the complete hierarchical name of objects instead of only the flat name and all the successful matches are included in the result.
	For example, if there is a top level hierarchical instance `Top` and it contains another sub-instance called `FF1`, then either of these can be used to retrieve `Top/FF1` instance:
	`get_cells -hier FF1`
	`get_cells -hier -regexp Top/FF1`
	However, the following commands will display an error :
	`get_cells -hier Top/FF1`
	`get_cells -hier -regexp FF1`

Example

- The following command searches for pins within the hierarchy limited at `B`:
  `get_cells -hsc @ B@*`
- The following command searches for cells with set pattern and filter:
  `get_cells "o*" -filter "@ref_lib_cell_name != FD2 && @is_black_box==false"`

Related Information

- get_lib_cells
- get_pins

get_lib_cells

```
get_lib_cells
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{pattern_list | -of_objects object_list}
```

Creates a collection of library cells from the loaded libraries whose name matches the supplied pattern list. Assign these library cells to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-filter expr`	Filters the collection with the specified expression. For any library cells that match the pattern, the software evaluates the expression based on the attribute of the library cell. You can filter library cells using any of the library cell properties supported through the `get_property` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of all library cells associated with the specified library pins or cells or libs. You can specify a list of library pins or a list of cells or a list of libs, but you cannot specify a list that contains mixed of library pins and cells and libs. You can specify patterns for the library pin or cell or lib names, a collection of library pins or cells or libs, or a combination of patterns and a collection.

pattern_list	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the *pattern_list* argument. The characters ? and * are wild cards. The ? character matches any one character while the * character matches zero or more characters. If a library matches multiple patterns, it appears multiple times in the collection. A pattern can have only one hierarchical separator character. The left part of the pattern is the library name pattern and the right part is the cell name pattern. If there is no separator character, the pattern is compared to cell names in all the libraries. A pattern can have only one hierarchical separator character, dividing the pattern into one or two parts. If the hierarchical separator is /, the alternatives are: • *cell_name_pattern* Selects all cells whose name matches the *cell_name_pattern* from all libraries. • *lib_name_pattern/cell_name_pattern* Selects all cells whose name matches the *cell_name_pattern* from all libraries whose name matches the *lib_name_pattern*.
-quiet	Suppresses all error and warning messages generated when the get_lib_cells command is run.
-regexp	Treats the specified patterns as a regular expression patterns. *Default:* Treats the specified pattern as a wild card

Examples

- The following command prints all cells for all libraries: get_lib_cells *
- The following command prints a collection with all cells in the lca500kv lib library: get_lib_cells lca500kv/*
- The following command prints a collection with all cells whose names starts with AN in all libraries starting with lca: get_lib_cells lca*/AN*
- The following command disables delay arcs on the EF11L050032A library cell. The set_disable_cell_timing command effects all instantiations of the specified cell, and lets you

disable delay arcs from and to internal pins. set_disable_timing -from WACLK -to RBO1 [get_lib_cells {EF11L050032A}]

Related Information

get_cells

get_nets

```
get_nets
[-hierarchical]
[-hsc char]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
[patterns | -of_objects object_list]
```

Creates a collection of nets in the current design whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to an another command. If no such net is found, an empty collection is returned.

When you use the `get_nets` command to specify hierarchical nets, the software matches the specified pattern and returns the top level net connected to it.

Parameters

`-filter` *expr*	Filters the collection with the specified expression. For any nets that match the pattern, the software evaluates the expression based on the attribute of the net. You can filter nets using any of the net properties supported through the `get_property` command.
`-hierarchical`	Specifies that the patterns should match hierarchically. If the hierarchical name of a net at any hierarchical level matches the patterns, it is included in the result.
`-hsc` *char*	Specifies the hierarchical delimiter for patterns.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects` *object_list*	Creates a collection of all nets associated with the specified cells, pins, or ports. You can specify a list of cells, a list of pins, or a list of ports, but you cannot specify a list that contains cells, pins, and ports together. You can specify patterns for the cell, pin, or port names, a collection of cells, pins, or ports, or a combination of patterns and a collection.

patterns	Specifies patterns for net names. Patterns include the * and ? wildcard characters and collections of nets.
-quiet	Suppresses all error and warning messages generated when the get_nets command is run.
-regexp	Treats the specified patterns as a regular expression patterns. *Default:* Treats the specified pattern as a wild card

Examples

- The following command returns all nets within the hierarchy limited at A: get_nets -hsc @ A@*
- The following command returns all patterns that match hierarchy and identifies nets to be excluded from optimization. get_nets * -hierarchical -filter "@dont_touch== true"

Related Information

get_pins

get_pins

```
get_pins
[-hierarchical]
[-hsc char]
[-filter expr]
[-leaf]
[-regexp]
[-nocase]
[-quiet]
{patterns | -of_objects object_list}
```

Creates a collection of instance pins whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to an another command. If no such pin is found, an empty collection is returned.

Parameters

`-filter` *expr*	Filters the collection with the specified expression. For any pins that match the pattern, the software evaluates the expression based on the attribute of the pin. If the expression is set to true, the pin is included in the result. You can filter pins using any of the pin properties supported through the `get_property` command.
`-hierarchical`	Specifies that patterns should match hierarchically. If the hierarchical name of a pin at any hierarchical level matches the patterns, it is included in the result.
`-hsc` *char*	Specifies the hierarchical delimiter for patterns.
`-leaf`	Returns leaf pins only. When the `-leaf` parameter is not used, the `get_pins` command returns the hierarchical pins present on a net along with the flat pins at the same hierarchical level as the specified object. **Note:** The `-leaf` parameter can only be specified with the `-of_objects` parameter.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.

`-of_objects` *`object_list`*	Creates a collection of all pins associated with the specified cells or nets. You can specify a list of cells or a list of nets, but you cannot specify a list that contains both cells and nets. You can specify patterns for the cell or net names, a collection of cells or nets, or a combination of patterns and a collection.
`patterns`	Specifies patterns for pin names. Patterns include the `*` and `?` wildcard characters and collections of pins.
`-quiet`	Suppresses all error and warning messages generated when the `get_pins` command is run.
`-regexp`	Treats the specified patterns as a regular expression patterns. *Default:* Treats the specified pattern as a wild card

Examples

- The following command returns all instance pins within the hierarchy limited at `A`: get_pins -hsc @ A@*
- The following command filters the collection for `pin_direction` pin attribute. get_pins "in*" -filter "@pin_direction == in"

Related Information

`get_cells`

`get_ports`

get_ports

```
get_ports
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{patterns | -of_objects object_list}
```

Creates a collection of ports whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to an another command.

Note: The `get_ports` command always creates a collection of top level ports irrespective of the current instance set using the `current_instance` command.

Parameters

`-filter` *`expr`*	Filters the collection with the specified expression. For any ports that match the pattern, the software evaluates the expression based on the attribute of the port. If the expression is set to true, the port is included in the result. You can filter ports using any of the port properties supported through the `get_property` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects` *`object_list`*	Creates a collection of all ports associated with the specified nets. You can specify patterns for the net names, a collection of nets, or a combination of patterns and a collection.
`patterns`	Specifies patterns for port names. Patterns include the `*` and `?` wildcard characters and collections of ports.The command returns a collection of ports that match the patterns. If no such port is found, an empty collection is returned.
`-quiet`	Suppresses all error and warning messages generated when the `get_ports` command is run.
`-regexp`	Treats the specified patterns as a regular expression patterns. *Default:* Treats the specified pattern as a wild card

Example

- The following example returns all ports beginning with the name `inb`: get_ports inb*
- The following command filters the collection for `port_direction` port attribute. get_ports "mode*" -filter {port_direction == in}

Related Commands

`get_pins`

get_power

```
get_power
[-help]
[-out_file filename]
[-nets nets_list | -pins pins_list | -insts instances_list]
[-attribute attributes_list]
[-include_unit]
[-tcl_list ]
[-pg_net {pg_net_name_list | all}]
```

Queries various power related properties of design objects like nets, pins, and instances. Using this command, you can retreive power attributes, such as internal power, switching power, leakage power, total power, and toggle rate. This command is very useful for writing custom scripts.

Note: This command outputs toggle rates as N.A. if the clock domain information for a given net/pin/instance is not found.

Use this command after running the `report_power` or `restore_power_database` command.

Note: The `-nets`, `-pins`, and `-insts` parameters are mutually exclusive. You must specify only one of the parameters.

Parameters

`-attribute` *attributes_list*	Specifies a list of power attributes. If an attribute is not available for the specified design object, the software returns its value as NA. See List of Power Attributes for a mapping of the list of available attributes with the design objects. This is a required parameter.
`-help`	Outputs a brief description that includes the type and default information for each `get_power` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_power`
`-include_unit`	Specifies to include the units for the power attributes.

`-insts` *instances_list*	Specifies a list of instance names. The `-nets`, `-pins`, and `-insts` parameters are mutually exclusive. You must specify one, and only one, of the parameters. **Note:** You can use the `get_cells` command to determine the power attributes for a collection of instances.
`-nets` *nets_list*	Specifies a list of net names. The `-nets`, `-pins`, and `-insts` parameters are mutually exclusive. You must specify one, and only one, of the parameters. **Note:** You can use the `get_nets` command to determine the power attributes for a collection of nets.
`-out_file` *filename*	Specifies the output file into which the power related properties are to be written. The `-tcl_list` and `-out_file` parameters are mutually exclusive; you cannot specify them together.
`-pg_net {`*pg_net_name_list* `| all}`	
	Specifies to query various power-related properties of a specified power net for an instance. This parameter supports the argument all to query properties of all power nets connected to the instance. You must use the `-insts` parameter in conjunction with the `-pg_net` parameter. The `-pg_net` parameter supports the following attributes: `switching_power`, `internal_power`, `leakage_power`, `total_power`, and `dynamic_power`.
`-pins` *pins_list*	Specifies a list of pin names. The `-nets`, `-pins`, and `-insts` parameters are mutually exclusive. You must specify one, and only one, of the parameters. **Note:** You can use the `get_pins` command to determine the power attributes for a collection of pins.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. The `-tcl_list` and `-out_file` parameters are mutually exclusive; you cannot specify them together.

The following power attributes can be queried for the design objects:

List of Power Attributes with Attribute Description

- `switching_power (net, inst)`: switching power of a net or instance
- `internal_power (inst)`: internal power of an instance
- `dynamic_power (inst)`: switching and internal power of an instance
- `leakage_power (inst)`: leakage power of an instance
- `total_power (inst)`: total power of an instance
- `toggle_rate (net, pin, inst)`: Toggle rate - The frequency at which the state of net, pin, or instance changes between logic levels during the duration of the VCD/TCF
- `duty_cycle (net, pin)`: Static probability of the signal to stay high
- `ref_clock (net, pin, inst)`: Reference clock name (Fastest clock for multi clock domains)
- `ref_clock_period (net, pin, inst)`: Reference clock period
- `transition_density (net, pin)`: toggle_rate/ref_clock_period or toggle_rate*ref_clock_frequency
- `loading_cap (net, pin, inst)`: Loading capacitance
- `voltage (net, pin, inst)`: Operating voltage for a net or pin. For MSMV instance, this attribute can be used to get breakdown of domain specific power.
- `rise_slew (net, pin)`: Rise slew
- `fall_slew (net, pin)`: Fall slew
- `glitch_rate (net, pin)`: Glitch rate - Number of glitches per duration. Applies only to the vector-driven flow.
- `glitch_power (net, pin, inst)`: Glitch power. Applies only to the vector-driven flow.
- `activity_source (net, pin)`: Source of activity annotation. The options are: propagated, user_annotated, and constant.
- `pin_direction (pin)`: [input | output | inout]
- `cell_type (inst)`:Lists cell type as memory, standard cell, IO, sequential, and so on
- `all_attributes (net, pin, inst)`: Lists all related power attributes for the specified design object.

Examples

- The following command returns the specified power attributes for the net clock1:

  ```
  get_power -nets clock1 -attribute {switching_power internal_power toggle_rate}
  ```

  The following is the sample output:

  ```
  clock1 0.04 NA 2
  ```

- The following command returns the specified power attributes for all nets that match the specified pattern `master*`:

  ```
  get_power -nets [get_nets master*] -attribute {switching_power toggle_rate
  activity_source duty_cycle} -include_unit
  ```

  The following is the sample output:

  ```
  masterin[1] 0.001mW 0.2 user_annotated 0.5
  masterin[2] 0.002mW 0.2 user_annotated 0.5
  masterin[3] 0.003mW 0.1 propagated 0.5
  masterin[4] 0.005mW 1 constant 1
  ```

- The following command returns power-related properties of all power nets connected to the instance `clk_buf_1`:

  ```
  get_power -pg_net all -insts clk_buf_1 -attribute {switching_power internal_power
  leakage_power}
  ```

  The following is the sample output:

  ```
  clk_buf_1 VDD 0.002 0.001 0.000001
  clk_buf_1 VDDL 0.0001 0.0001 0.0000001
  ```

3

Basic Database Access Tcl Commands and Attributes

- add_thru_substrate_insts
- compress_special
- convert_dbu_to_um
- convert_um_to_dbu
- create_cell_obs
- decompress_special
- define_attribute
- delete_cell_obs
- delete_floating_nets
- delete_obj
- delete_thru_substrate_insts
- get_computed_shapes
- get_db
- get_db Category Attributes
- get_edge
- get_obj_in_area
- get_rect
- get_transform_shapes
- get_via_pillars
- init_hier_cell
- is_attribute

- report_floating_nets
- report_metal_stack
- report_via_def
- reset_db
- set_db
- set_db Category Attribute
- set_via_pillars
- sort_db
- tech Category Attributes
- uniquify
- write_preserves

add_thru_substrate_insts

```
[-help]
[-driver_thru_substrate_cell cell1]
-load_thru_substrate_cell cell2
-net net
[-redraw]
[-selected]
[-shared_load_thru_substrate_cell]
```

Adds thru-substrate instances to a net to specify two-sided routing for a critical net (backside routing has lower RC than frontside routing). All pins or ports of the net must be on the front side, and as an exception, one backside port is allowed. A thru-substrate instance is a device that tunnels through the substrate to connect frontside routing with backside routing.

The driver thru-substrate instance is placed on top of the driver and a load thru-substrate instance is placed on top of its load. If a load thru-substrate instance drives multiple loads, it is placed in the center of the group of loads. It means that all instances with a pin on the net must be placed, and if the net has ports, they must also be placed before when the thru-substrate instances can be added to the net. Hence, this command creates unallowed placement that needs to be resolved with the `place_detail` command.

When a net has a backside port with everything else on the front side, this command creates one

thru-substrate instance placed on top of the backside port, so that the net's frontside routing can be brought to the backside, typically to connect to a backside bump. For this usage, specify the same value for the `-load_thru_substrate_cell` and `-driver_thru_substrate_cell` parameters.

Use this command to create one FTV for a net with multiple FS leaf pin, multiple FS port(s), and/or a single BS port.

> FTV is placed on the top of the BS port.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `add_thru_substrate_insts` parameter. For a detailed description of the command and all its parameters, use the man command: `man add_thru_substrate_insts`
`-driver_thru_substrate_cell` *cell1*	Specifies the thru-substrate macro/cell to instantiate a thru-substrate instance, whose front pin is to be routed to the driver.
`-load_thru_substrate_cell` *cell2*	Specifies the thru-substrate macro/cell to instantiate a thru-substrate instance(s), whose front pin(s) is to be routed to load(s).
`-net` *net*	Specifies the net to add thru-substrate instances on both sides, front and back.
`-redraw`	Redraws the graphics window.

`-selected`	Groups the loads where each group will have its own load thru-substrate instance. • When the `-selected` and `-load_thru_substrate_cell` parameters are specified, the only driver thru-substrate instance is first created and the selected loads either share one load thru-substrate instance or get their own load thru-substrate instances. Additionally, the unselected loads are driven directly by the net driver. • In case the `-selected` parameter is specified and the `-load_thru_substrate_cell` parameter is not specified, each selected load driven by the net driver is reconnected to a new load thru-substrate instance. This may be shared through the `-shared_load_thru_substrate_cell` parameter, which is driven by the driver thru-substrate instance. This process repeats until the loads are grouped as desired.
`-shared_load_thru_substrate_cell`	Loads share one thru-substrate instance (one thru-substrate instance for each load, if not specified).

Related Topics

`delete_thru_substrate_insts`

compress_special

```
compress_special
[-help]
[-area {x1 y1 x2 y2}]
[-incremental]
[-layers layer+]
[-nets net+]
```

Compresses Power-Grid (PG) based on the specified area, layer, or net.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `compress_special` parameter. For a detailed description of the command and all its parameters, use the man command: `man compress_special`
`-area {`*`x1 y1 x2 y2`*`}`	Specifies the coordinates for the area from which PG needs to be compressed. The `compress_special` command compresses only those special wires and special vias, which exist within the area specified by this parameter. *Default*: All PG structures are compressed.
`-incremental`	Recompresses the PG after editing the PG wires or vias. This parameter can only be specified when the current design is in compressed mode.
`-layers` *`layer+`*	Specifies the layers on which PG compression is needed. This parameter requires either LEF layer names or a routing layer index value (for example, `1` for the first routing layer, `2` for the second routing layer, and so on).
`-nets` *`net+`*	Specifies the nets from which PG compression is needed. If the specified net does not have any special route on it, it is ignored.

Example

The following command compresses `%s special_wires` and `%s special_vias`:

```
compress_special -area {100 100 120 120} -net VDD -incremental
```

Related Topics

`decompress_special`

convert_dbu_to_um

```
convert_dbu_to_um
[-unit number]
{int | {int_list}}
```

Takes an arbitrary list of ints and returns it in the same form, with each database integer int converted to its user unit floating point equivalent.

Parameters

`-unit` *number*	Specifies the convert factor by which to divide the database integer int. *Value:* One of `100`, `200`, `400`, `800`, `1000`, `2000`, `4000`, `8000`, `10000`, or `20000` *Default:* Uses the LEF `UNITS DATABASE MICRONS` int from the database technology file, or the OpenAccess equivalent.
`{`*int* `|` `{`*int_list*`}}`	
	Specifies the int or int list to convert. You can specify a single int, or an arbitrary list of ints, including points, rectangles and coordinates. *Type:* Integer

Examples

- The following command converts the database int `100` into its equivalent user unit int, based on a convert factor of `2000`:

  ```
  convert_dbu_to_um -unit 2000 100
  ```

  The software returns the following information:

  ```
  0.0500
  ```

- The following command converts the database ints for the list of numbers `{1000 1000 2000 2000}` into their equivalent user unit ints, based on a convert factor of `1000`:

  ```
  convert_dbu_to_um -unit 1000 {1000 1000 2000 2000}
  ```

  The software returns the following information for the list of numbers. The list is implied, but not shown.

  ```
  1.000 1.000 2.000 2.000
  ```

convert_um_to_dbu

```
convert_um_to_dbu
[-unit number]
{double | {double_list}}
```

Takes an arbitrary list of doubles and returns it in the same form, with each user unit floating point double converted to its database integer double equivalent.

Parameters

`-unit number`	Specifies the convert factor by which to multiply the user unit double. *Value:* One of `100`, `200`, `1000`, `2000`, `10000`, or `20000` Default: Uses the LEF `UNITS DATABASE MICRONS` double from the database technology file, or the OpenAccess equivalent.
`{double | {double_list}}`	
	Specifies the double or double list to convert. You can specify a single double, or an arbitrary list of doubles, including points, rectangles and coordinates. *Type:* Float

Examples

- The following command converts the user unit double `30` into its equivalent database integer double, based on a convert factor of `2000`:

  ```
  convert_um_to_dbu -unit 2000 30
  ```

  The software returns the following information:

  ```
  60000
  ```

- The following command converts the user unit doubles for the list of numbers {`30 40`} into their equivalent database unit doubles, based on a convert factor of `200`:

  ```
  convert_um_to_dbu -unit 200 {30 40}
  ```

  The software returns the following information for the list of numbers. The list is implied, but not shown.

  ```
  6000 8000
  ```

create_cell_obs

```
create_cell_obs
[-help]
-cell cell_name_or_pointer
[-ignore_pg_net]
-layer layer_name_or_pointer
[-mask int_value]
{-rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}...} | -polygon {x1 y1 x2 y2 x3
y3 ...}}
[-spacing value | -design_rule_width value]
```

Helps in adjusting the database. The command works in memory and hence, must be used in each session before other commands are run.
It adds an obstruction geometry to a cell on the specified layer and is helpful when you want to add a temporary cell obstruction without changing the cell's LEF and reloading the software.

Parameters

`-help`	Prints a brief description that includes type and default information for each `create_cell_obs` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_cell_obs`
`-cell` *`cell_name_or_pointer`*	Specifies the cell name or pointer.
`-design_rule_width` *`value`*	Specifies the design rule width.
`-ignore_pg_net`	Ignores the Power/Ground routing while checking for DRC violations (including shorts) involving the added obs. This parameter is equivalent to `LEF MACRO OBS LAYER # EXCEPTPGNET`.
`-layer` *`layer_name_or_pointer`*	Specifies the layer name or pointer.
`-mask` *`int_value`*	Specifies the integer value of the mask (within the mask range) for the created OBS. *Default:* `0`, which means that the mask is uncolored.
`-polygon {`*`x1 y1 x2 y2 x3 y3 ...`*`}`	Specifies the polygon shape.
`-rects {{`*`llx1 lly1 urx1 ury1`*`} {`*`llx2 lly2 urx2 ury2`*`}...}`	Specifies the Obs rect list for adding obstruction geometries.
`-spacing` *`value`*	Specifies the spacing value.

decompress_special

```
decompress_special
[-help]
[-area {x1 y1 x2 y2}]
[-layers layer+]
[-nets net+]
```

Decompresses Power-Grid (PG) objects within the specified area, layer, or net. This command decompresses only the overlapped part in case of partial overlapping. The rest of the part is regrouped.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `decompress_special` parameter. For a detailed description of the command and all its parameters, use the man command: `man decompress_special`
`-area {x1 y1 x2 y2}`	Specifies the coordinates for the area from which PG needs to be decompressed. The `decompress_special` command decompresses only those special wires and special vias, which exist within the area specified by this parameter. *Default*: All PG structures are decompressed.
`-layers layer+`	Specifies the layers on which PG decompression is needed. This parameter requires either LEF layer names or a routing layer index value (for example, `1` for the first routing layer, `2` for the second routing layer, and so on).
`-nets net+`	Specifies the nets from which PG decompression is needed.

Example

The following command decompresses `%s special_wires` and `%s special_vias`:

```
decompress_special -area {100 100 120 120} -layer Via56
```

Related Topics

compress_special

define_attribute

```
define_attribute
[-help]
attribute_name
-obj_type object_type
[-obsolete]
{-alias existing_attribute | {-category category [-data_type {bool | int | double | area
| coord | string | point | rect | line | delay | resistance | capacitance | voltage |
in_dir | out_dir | in_file | out_file  | object | object*} [-enum_values values]]
[-check_function check_function]
[-compute_function compute_function]
[-default default]
[-help_string description]
[-hidden]
[-set_function set_function]
[-skip_in_db]}}
```

Creates a new, user-defined attribute with the specified characteristics.

The attribute value can be reset only if a default value is specified. For example, the following command defines a new instance attribute of integer type with a default value equal to 1:

```
define_attribute myattr -default 1 -obj_type inst -data_type int -help_string "my
attribute" -category mycat
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `define_attribute` parameter. For a detailed description of the command and all of its parameters, use the man command: `man define_attribute`.
`-alias existing_attribute`	Makes *`attribute_name`* a hidden alias for *`existing_attribute`*, which must have been defined. This parameter is used along with the `-obj_type` parameter. It could also be used with the `-obsolete`parameter to generate an obsolete warning.

attribute_name	Specifies the name of the attribute to be defined.
-category *category*	Specifies the category of the attribute. Categories group attributes that perform similar functions, whereas object types describe where in the design an attribute is valid. You can specify any category name, both new and existing category names are allowed.
-check_function *check_function*	Specifies a previously defined Tcl procedure's name to ensure that the newly defined attribute is valid. The Tcl procedure should be of the form `proc {object value}`.
-compute_function *compute_function*	Specifies a previously defined Tcl procedure's name to get the newly defined attribute's value later (with the `get_attribute` command). The Tcl procedure must be of the form `proc {object}`. When you use this option, the attribute becomes a read-only attribute because its value is always computed.
-data_type {bool int double area coord string point rect line delay resistance capacitance voltage in_dir out_dir in_file out_file}	
	Defines the data type of the attribute. Possible data types are basic only, no objects.
-default *default*	Specifies a default value for the attribute. In case this parameter is not specified while creating an attribute, the default value will be: • `no_value` for simple numeric types such as `int`, `double`, `delay`, `capacitance`, `coord`, and `boolean` • "" (empty string) for all other types, including `enum`, `string`, `in_file`, `out_file`, `in_dir`, `out_dir`, `point`, `rect`, `polygon`, `line`, and `obj_type`
-enum_values *values*	Specifies a list of all the possible values of the enumerated type. For example, `-enum_values {a b c}`.

`-help_string` *`description`*	Specifies the help text for the attribute.
`-hidden`	Specifies whether the defined attribute is a hidden attribute.
`-obj_type` *`object_type`*	Specifies the object type of the attribute. All valid object types can be found by typing `get_db obj_types .name` at the prompt. The attribute is assigned to the specified type.
`-obsolete`	Specifies if the defined attribute is obsolete, and gives a warning if used.
`-set_function` *`set_function`*	Specifies a previously defined Tcl procedure's name. This option allows you to override user-defined values provided it conforms to the parameters in the Tcl procedure you created. The Tcl procedure should be of the form `proc {object new_value current_value}`. Returns new value assigned to the attribute.
`-skip_in_db`	Prevents the `write_db` command to write out the defined attribute.

Examples

- The following command creates a Tcl procedure, `check_fxn`, and then creates an attribute named test_check. The `-check_function` parameter is specified so that the test_check attribute can be tested for validity when it is specified later:

```
proc check_fxn {obj val} {
+ if {$val >0} { return 0}
+ return 1
+ }
define_attribute test_check1 -obj_type inst -data_type int -category test -
help_string "test_check_function" -check_function  check_fxn
```

  Based on the check_function parameter, the checking criteria attribute value can be set only to a negative value. As a result the following command sets the attribute value to -1:

```
set_db insts .test_check1 -1
```

  but the following command displays an error and keeps the previous value.

```
set_db insts .test_check1 2
```

- The following command creates a Tcl procedure, `set_fxn`, and then creates an attribute named `test_set`. The `-set_function` parameter is specified so that you can change the value of the `test_set` attribute (provided it is valid):

```
{proc set_fxn {obj new_val cur_val} {
+ if {$new_val > $cur_val} {
+ return $new_val
+ }
+ return $cur_val
+ }
+ }
define_attribute test_set -obj_type root -data_type int -category test -
help_string "test set function" -set_function set_fxn
```

  Now, the following command sets the attribute value to 1 because the previous value was undefined

```
set_db test_set 1
```

  The following command changes the Attribute value to 4 as the new value is greater than the current value

```
set_db test_set 4
```

But with the following command the value is not changed as the new value is less than the current value (4)

```
set_db test_set 2
```

- The following command uses the compute_function to set attribute values. Note that the attribute becomes read-only if `compute_function` is applied.

```
proc comp_func {obj} { return 5}
define_attribute test_compute -obj_type root -data_type int -category test -
help_string "test set function" -compute_function comp_func
get_db test_compute 5
```

Now, the following command will issue an error as attribute value cannot be changed.

```
set_db test_compute 3
```

Related Topics

- For information on how to query DB attributes, see the `get_db` command.
- For information on how to set DB attributes, see the `set_db` command.

delete_cell_obs

```
delete_cell_obs
[-help]
-cells {base_cell | design | module}+
{-obs_shapes shape+ | [{-rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}...} | -polygon {x1 y1 x2 y2 x3 y3 ...}] -layers layer+}
```

Helps in deleting an obstruction geometry for a cell on the specified layer. This command deletes only the obstruction geometry added by the `add_cell_obs` command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `delete_cell_obs` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_cell_obs`
`-cells` *cell_name_or_pointer*	Specifies the cell name or pointer.
`-layers` *layer_name_or_pointer*	Specifies the layer name or pointer.
`-rects {{`*llx1 lly1 urx1 ury1*`} {`*llx2 lly2 urx2 ury2*`}...}`	Specifies the Obs rect list for deleting obstruction geometries.
`-obs_shapes` *shape+*	Specifies the cell obstruction shape pointers. **Note:** This parameter must be used with the `-cells` parameter, and cannot be specified with any other parameter except than `-help`.
`-polygon {`*x1 y1 x2 y2 x3 y3 ...*`}`	Specifies the polygon shape.

delete_floating_nets

```
delete_floating_nets
```

The `delete_floating_nets` command deletes all the nets with zero terms except for assign nets and p/g nets. It supports zero-term net deletion in multiple instantiated cells.
This command takes no parameters and can be used whenever the design is read in.

After the command is executed, it reports the following:

- Total number of dangling nets `nrDnet`
- Total number of deleted dangling nets `nrDdnet`
- Total number of nets `ntNet`
- Ratio of `nrDdnet` over `nrNet`
- Ratio of `nrDnet` over `nrNet`
- `nrDdnet` over `nrDnet`

delete_obj

```
delete_obj any_object+
[-help]
```

Deletes one or more existing objects from the database. The command can delete `netlist` objects and physical data like `routing` and `floorplan` objects, but not the `clock` or `timing` objects (such as `clock_tree`, and `rc_corner`).

The following `obj_types` can be deleted through this command: `attribute`, `bump`, `bus_guide`, `flow`, `flow_step`, `group`, `gui_text`, `gui_line`, `gui_rect`, `hinst`, `hnet`, `hpin`, `hport`, `inst`, `marker`, `module`, `net`, `net_group`, `patch_wire`, `pin_guide`, `pin_group`, `pin_blockage`, `place_blockage`, `port`, `port_shape`, `resize_blockage`, `route_blockage`, `route_type`, `row`, `sdp`, `special_wire`, `special_via`, `text`, `via`, `virtual_wire`, `what_if_via`, `what_if_wire`, and `wire`.

Parameters

`-help`	Provides a brief description of the command that includes the type and default information for parameters. For a detailed description of the command and all its parameters, use the man command: `man delete_obj`
`any_object+`	Specifies the object or the list of objects to be deleted.

Examples

- The following command deletes all the `place_blockages`:
  ```
  delete_obj [get_db place_blockages]
  ```
- The following command deletes the specified `place_blockage` object:
  ```
  delete_obj place_blockage:0x2b77dfbf0048
  ```
- The following command deletes all the special routes from the VDD net:
  ```
  set my_net [get_db pg_nets VDD]
  delete_obj [get_db $my_net .special_wires]
  delete_obj [get_db $my_net .special_vias]
  ```
- The following command deletes a group of SDPs:
  ```
  delete_obj sdp:DTMF_CHIP/wCardInstGrp_1
  ```

Related Topics

- For information on querying the database objects, see the `get_db` command.
- For a list of objects that can be queried for information or changed, see the Common UI Database Object Information document.

delete_thru_substrate_insts

```
delete_thru_substrate_insts
[-help]
{-net net_name}
```

Deletes thru-substrate instances for the specified net. A thru-substrate instance is a device that tunnels through a chip to connect frontside routing with backside routing.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `delete_thru_substrate_insts` parameter. For a detailed description of the command and all its parameters, use the man command: `man delete_thru_substrate_insts`
`-net` *net_name*	Specifies the net whose thru-substrate instances are to be deleted.

Related Topics

`add_thru_substrate_insts`

get_computed_shapes

```
get_computed_shapes
[-help]
[-dbu]
[-step step]
[-output {polygon|rect|hrect|area}]
shape_list [AND shape_list | ANDNOT shape_list | OR shape_list | XOR shape_list
| INSIDE shape_list | OUTSIDE shape_list | ENCLOSE shape_list | STRADDLE shape_list
| BBOX | HOLES | NOHOLES | MOVE {dx  | SIZE value | SIZEX value | SIZEY value] ...
```

Performs geometric (SIZE and BBOX) and logical (such as AND and ANDNOT) operations on shape (polygon or a rectangle or a list of rectangles and polygons).

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_computed_shapes` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_computed_shapes`
`-dbu`	Specifies values in database units, specified in integers. All `shape_list` values as well as values for MOVE, SIZE, and step should be using units consistent with the `-dbu` setting. The output units are the same as the input units. *Default:* Specifies values in user units, in microns.
`-step`	Specifies step size if output format is rect, required to convert non-orthogonal shapes into series of rectangles. *Default*: minwidth value of first routing layer. The option is coord, and is optional.
`-output` *polygon*|*rect*|*hrect*|*area*	

	Specifies the output format. • `polygon`: Polygon shape • `rect`: Vertical rectangles • `hrect`: Horizontal rectangles • `area`: Total area of final shape *Default:* `rect` (string)
`shape_list`	A polygon or rectangle or a list of polygon and rectangle. • polygon: `{{x y} {x y} {x y} ... }`; • rect: `{x1 y1 x2 y2}` This option is a required list.
`AND` *`shape_list`*	Is a binary operator that is the intersection of `shape_lists`
`ANDNOT` *`shape_list`*	Is a binary operator which defines the initial `shape_list` minus `shape_list`
`OR` *`shapelist`*	Is a binary operator of the union of shapelists
`XOR` *`shape_list`*	Is a binary operator of the `shape_list`: OR minus AND. It is a string and is optional.
`INSIDE` *`shape_list`*	Is a binary operator for initial shapes that are inside (including touching) of one of the shapes from the second list.
`OUTSIDE` *`shape_list`*	Is a binary operator for initial shapes that are outside (including touching) of one of the shapes from the second list.
`ENCLOSE` *`shape_list`*	Is a binary operator for initial shapes that completely enclose at least one shape from the second list.
`STRADDLE` *`shape_list`*	Is a binary operator for initial shapes that are partially inside and partially outside at least one shape from the second list.
`MOVE {`*`dxdy`*`}`	Is a unary operator that moves the by {`dx dy`}
`BBOX`	Is a unary operator that computes the bounding box of `shape_list`. This is a string and optional.

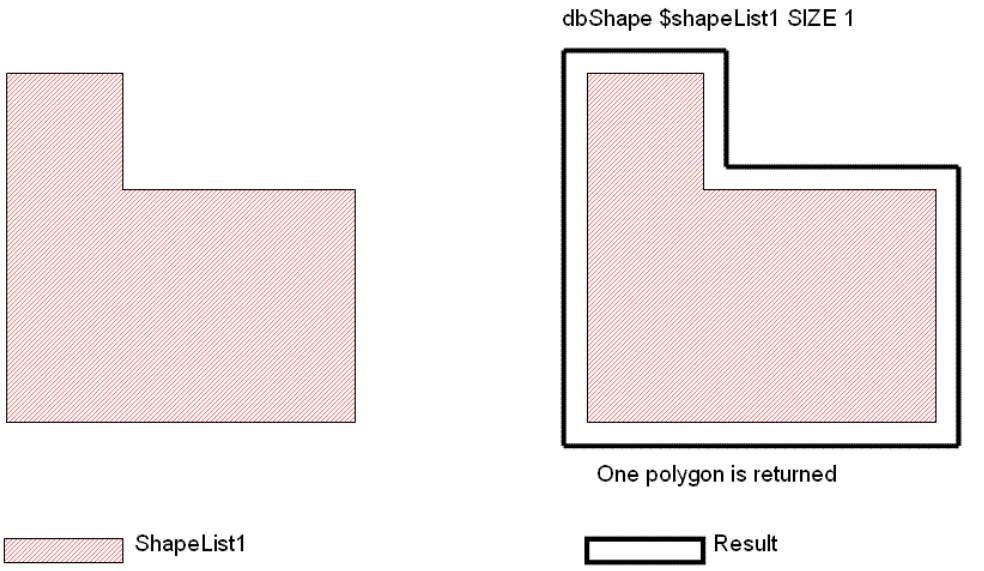
`HOLES`	Is a unary operator that gives list of new shapes that fill holes in the original `shape_list`.
`NOHOLES`	Is a unary operator that gives list of new shapes that include filled holes in the original `shape_list`. This is same as `get_computed_shapes` *`shape_list`* `HOLES` OR *`shape_list`*
`SIZE` *`value`*	Is a unary operator that increases the size of `shape_list` by `value`.
`SIZEX` *`value`*	Is a unary operator that increases the size of list of shapes in the X-dbuirection by `value`
`SIZEY` *`value`*	Is a unary operator that increases the size of list of shapes in the Y-dbuirection by `value`

Note: All shapes processed by `get_computed_shapes` must be non-zero area. If a *shape_list* contains a list of rectangles and one of them is zero-area, it will be dropped.
Also, during negative sizing or other operations, if a zero-area shape is created, it will be dropped from the output (or intermediate results).

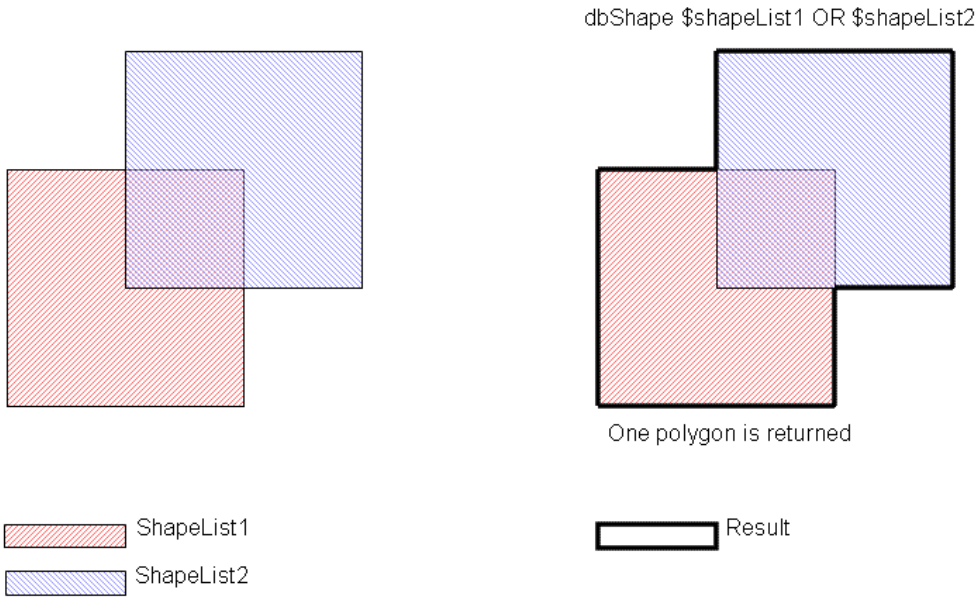
For example, a rectangle that is 4x4 or smaller will disappear when using `get_computed_shapes $rect SIZE -2 SIZE 2` while shapes larger than 4 will be returned to their original state.
The same is true for protrusions with width less than 4.

Examples

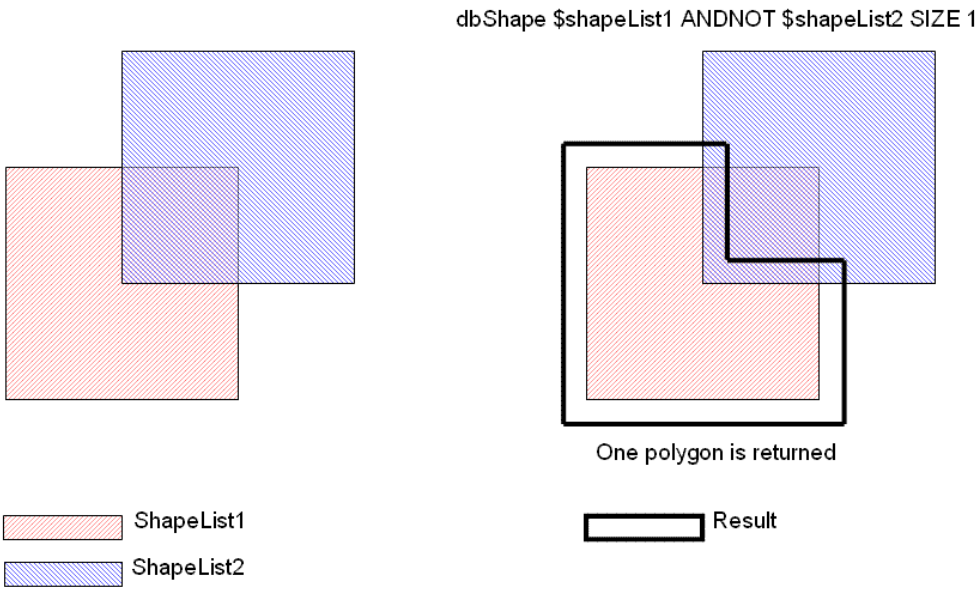
- **Unary**:



- **Binary**:



- **Combination**:



Example: Area calculation

`get_computed_shapes $shapes1 AND $shapes2`

Returns intersection (as a list of rectangles) between the shapes in the `$shapes1` and `$shapes2` lists.

`get_computed_shapes -output area $shapes1 AND $shapes2`

Returns intersection area between the shapes in the `$shapes1` and `$shapes2` lists.

get_db

```
get_db
[-help]
{[start [chain] [pattern1 [pattern2 ..] [[-if expression] | [-expr expression]] [-dbu]
[-foreach tcl_body] [-index {object | obj_type1 name1 [object | obj_type2 name2 …]}] [-
invert] [-regexp] [-match_hier] [-unique] [-computed]}
[-category category]
[-depth {[min] max}]
[-skip_fail]
```

Returns objects and attributes in the database with different types of filtering.
The `get_db` commands takes a dual-ported object, dual-ported object list or a collection as input, then uses a period (.) to traverse to other related objects or to access attributes. You can use additional periods to traverse the data base schema.

Note: A Dual-Ported Object (DPO) is a Tcl_Obj in the C++ programming interface. The pointer is kept as a C++ pointer inside Tcl unless Tcl forces it to be evaluated as a string. In normal usage it never gets converted to a string which is more efficient, but if you do a "puts $var" or return the value to the shell, it will be converted to its "dual" string form.

- The string form is normally a useful name preceded by the `obj_type`, so a `base_cell` object string form might look like `base_cell:and2`.
- A netlist name will have the current design name included, so an instance named `i1/i2` in design top, would look like `inst:top/i1/i2`.
- An object with no name like a wire, will just show the pointer hex-value like `wire:0x22222`.

The `get_db` command accepts collections as an input but it only returns objects in dual-ported object list form.

For descriptions on all objects and attributes, see Common UI Database Object Information.

The root object has the current design data under the `current_design` attribute, while technology and library data are normally directly accessible as root attributes (e.g. layers and base_cells). The `root` object is the default starting point if there is no input list or collection given.

You can query a root attribute in any of the following ways:

- `get_db / .layers`
  where, '/' is the notation for the root object
- `get_db layers`
  This implicitly starts at the root. In this usage, no "." is used before the "layers" attribute name.

For example, you can use the following command to get all the driver-pins of all nets in `current_design`:

```
>get_db current_design .nets.drivers
```

Note: The `current_design` is implicitly created by the `init_design` or the `read_db` commands.

Frequently used current_design attributes like insts or hinsts (hierarchical insts) have root attribute aliases (or shortcuts). These attributes can be accessed by just typing the root attribute alias name. For example, the following commands gets all instances of the current design with names that start with `i1/i2*`:

- `get_db current_design .insts i1/i2*`
- `get_db insts i1/i2*`
  Where, insts is a root attribute alias for current_design .insts

Some of the more commonly used aliases from the `current_design` attributes include: `nets`, `pins`, `insts`, `hnets`, `hpins`, and `hinsts`.

The `get_db` command supports auto-complete when you press the `<Tab>` key. It will give a list of all matching attributes starting with the letters entered so far, or it will complete the name if it is unique.

For example:

```
>get_db insts .par<Tab>
parent partition
```

If you press the `<Shift+Tab>` keys together, the matching attributes are displayed along with their help string, like this:

```
>get_db insts .par<Shift+Tab>

parent # The parent of the inst, which is either a hinst for an instance within
hierarchy or the design object for an instance at the toplevel.
partition # If this inst is a physical black_box, this partition will carry the pin
constraints and related data for the blackbox. Otherwise it is empty.
```

There are two database definition objects: `obj_type` and `attribute`. If you want to query all the details for an attribute definition on an `obj_type`, rather than an attribute value, use the following command:

```
>get_db attribute:<obj_type>/<attribute_name> .*
```

You can use `get_db` to query user-defined properties from LEF. Following is the mapping between the LEF `objectType` and database object:

```
LAYER -> layer
LIBRARY -> design
MACRO -> base_cell
```

```
NONDEFAULTRULE -> route_rule
PIN -> base_pin
VIA -> via_def
VIARULE -> via_def_rule
```

Note: If `PROPERTYDEFINITION` is defined for `LIBRARY`, the `PROPERTY` will be mapped as a design attribute with a `tech_lef_property_*` prefix.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_db` parameter.

`-category` *category*	Specifies a category of root attributes to return. This is intended to return all the root attributes associated with one application category. You cannot use a *start* argument with the `-category` option, so this option cannot be used for anything but root attributes. The return value is list of root attribute names.
	Example:
	The following command can be used to see all the root attributes associated with the category place:
	`get_db -category place`
	The output is as follows:
	`place_design_floorplan_mode place_design_refine_place place_detail_allow_border_pin_abut ...`
	You can view the complete list of categories with the get_db categories command as follows:
	`innovus 1> get_db categories`
	`active_logic_view add_endcaps add_fillers add_rings ...`

chain	Specifies the *chain* in the form [.*attribute_name1* [.*attribute_name2*]...]. A chain always starts with a ".". The specified attribute names are valid for each level of the traversal in the *chain*. Example: The following command returns the names of pins of $my_insts get_db $my_insts .pins.name If the chain has a * or ? in the attribute name, it will be expanded to all matching attribute names, and the display will show the attribute name and current value for every object. This is intended for interactive usage, and no values are returned to Tcl. Values that may require significant time to compute (like timing data that requires the timing-graph) will only be displayed if -computed is given. For example: get_db $my_insts .b* Object: inst:top/i1 base_cell: base_cell:BUFX2 bbox: {34.32 10.08 38.94 15.12} Object: inst:top/i2 ...

`-computed`	Forces `get_db` to output the values of attributes that may require significant time to compute (like timing data that requires the timing-graph) when browsing many attributes at the same time. This is intended for interactive usage when using a pattern match with * or ? for an attribute name. If only a single attribute name is given, then the value is always returned.
	
	Example:
	
	`>get_db $my_pins .slack_max*`
	
	`Object: pin:top/inst$odd/Y`
	
	`slack_max: NC`
	
	`slack_max_edge: NC`
	
	`...`
	
	`>get_db $my_pins .slack_max* -computed #if timing-graph not available yet, it will be computed`
	
	`Object: pin:top/inst$odd/Y`
	
	`slack_max: 550.12`
	
	`slack_max_edge: rise`
	
	`...`
	
	`>get_db $my_pins .slack_max #single attribute names always return value and force computation if needed`
	
	`550.12 ...`

`-dbu`	Indicates that attribute values (coord, pt, rect, polygon, area, ...) should be returned as an int in database units.
	
	Default: If not given, the value returned is a double in microns (or microns**2).
	
	Examples:
	• In the following command the return type is a double in microns.
	`get_db $my_inst .location`
	`{1.0 1.0}`
	• In the following command the return type is an int in database units.
	`get_db -dbu $my_inst .location`
	`{1000 1000}`

`-depth {[min] max}`	Specifies the hierarchy depth range. It expands each `design` or `hinst` object in the input list into a list of `design` or `hinst` objects by descending the hinst hierarchy from *`min`* to *`max`* depth. The default value of *`min`* is `0`, if not specified. `0` here signifies the level of the starting object. The `-depth` expansion happens first, and the resulting expanded list is then processed as the input list of objects by `get_db`. For example: `get_db hinst:top/h1 -depth {0 1}` Returns `hinst:top/h1`, and the first level of hinsts below it.

`-expr` *`expression`*	Returns the object if the *`expression`* is true. The *`expression`* argument is a strict Tcl expression that can be passed as is to the Tcl expression command. It has a pre-defined Tcl array named `$obj` that is populated by `get_db` with any attribute or chain value for each object as they are visited, and `$obj(.)` returns the object itself. For example,
	`get_db insts -expr {$obj(.base_cell.name) eq "AND2"}  #all insts that are AND2`
	Normally the `-if` *`expression`* parameter is faster for simple expressions, but it is limited to a few operators, and simple `$var` substitution. If you need a full Tcl expression semantics including the proc calls inside [], special char escaping, or complex `$var` substitution, you must use `-expr`.
	This parameter can be used with a *`chain`* and `pattern` like this:
	`get_db $my_insts .pins i1/* -expr {$obj(.base_name) eq "A"}`
	Here, the `i1/*` pattern is checked first for pin names that match `i1/*`, then the pin `.base_name` is checked if it is equals `"A"`.
	`-expr` cannot be used at the same time with `-if`.

`-foreach` *`tcl_body`*	Specifies a list of Tcl commands to process for each object. This parameter avoids the overhead of creating long lists of objects. The Tcl array `$obj()` is populated with the attributes of each object when it is visited, and `$obj(.)` is the object itself. The Tcl commands break and continue also work as expected in a `foreach` loop (break stops the traversal of any more objects and `get_db` returns). Unlike inside `-if`, the `tcl_body` is strict Tcl. The *`.attribute`* names are not substituted, you must use `$obj()` to access the attribute values instead. If you have a *`pattern`* or `-if`, or `-expr` usage, then `-foreach` only processes each `object` that matches the *`pattern`*, `-if`, or `-expr` expression. The following command prints the names of every inst that starts with "abc" and is an AND2 gate: `get_db insts abc* -if {.base_cell.name == AND2} -foreach {puts $obj(.name)}` It returns the total number of `insts` that are printed.

`-if` *`expression`*	Specifies a boolean expression, with extensions to make pattern matching simpler.
	
	• == and != do not require a quoted string, and they support pattern matching with * and ?.
	• Other operators like >, <, <=, >=, &&, ||, and the unary negate operator ! are supported.
	• Parentheses “( )” can be used to group complex expressions.
	
	Note: Unlike the <pattern> argument, inside the -if expression, you cannot use a string pattern to implicitly match the .name attribute of an object. You must use the .name field directly. For example, the following command will not match anything because .base_cell will become something like `base_cell:AND2`.
	
	`>get_db insts -if {.base_cell == buf*} #matches nothing`
	
	use either of these instead:
	
	`get_db insts -if {.base_cell.name == buf*} #match the .name attribute`

	`get_db insts -if {.base_cell == base_cell:buf*} #match the DPO name directly`
	
	Note: If you use an attribute that is an object list (like .pins), every object in the list is tested in an "implicit OR", so if any of them pass, the expression is matched.
	
	Handling of indexed attributes inside of the `-if` statement:
	• The `-index` option is kept outside of the `-if` statement
	• The `-index` option MUST be valid for ALL attributes inside the `-if` statement otherwise an error occurs
	
	Examples:
	• The following command finds all instances with `base_cell` names that matches "`buf*`".
	`get_db insts -if {.base_cell.name == buf*}`
	• The following command returns all insts that have one or more pins that match `CLK*`. Only one copy of the matching inst object is returned, even if multiple pin names match CLK*.
	`get_db insts -if {.pins.base_name == CLK*}`
	• The following command gets all pins where `delay_max_rise` is greater than 5 or `slew_max_fall` is smaller than 8. This is a valid command as both delay* and slew* are attributes indexed by `view` (`view` is a legal abbreviation of `analysis_view`).
	`get_db pins -if {.delay_max_rise > 5 || .slew_max_fall < 8 } -index {view v1}`
	
	Note: The `-if` parameter does not support the usage of embedded Tcl command as operands. Use the `-expr` for this requirement.

`-index {`*`object | obj_type1 name1 [object | obj_type2 name2 …]`*`}`	Some attributes represent an array of values. For example, a `pin .slack_max_rise` attribute has a different value for each `analysis_view` and each `clock` that can affect that pin. With no index, a default method defined for that attribute is used (for example, the maximum, or minimum of all values in the array). For `slack_max_rise` with no `-index` value, the value returned is the worst-case slack across all `analysis_views` and `clocks`.

	If you want to return a specific value, you must include the index value. The index values are object pointers, or a pair of `obj_type` name and object name (for example `clock clk1`). The word `view` is accepted as an abbreviation for `analysis_view`.
	
	For example:
	
	`get_db $my_pins .slack_max_rise -index "view func_wc_cworst"`
	
	Will return the worst case `.slack_max_rise` for the `analysis_view func_wc_cworst` across all clocks. This attribute also accepts a clock index, so you can further refine it like this:
	
	`get_db $my_pins .slack_max_rise -index "view func_wc_cworst clock clk1"`
	
	To get the value for that specific `analysis_view` and `clock`.
	
	In the special case of a clock and `analysis_view` as shown above, a clock object is sufficient for both, because a clock object is based on both the clock name and the view name. So you could also do this:
	
	`set my_clk_obj [get_db clocks -if {.view_name == func_wc_worst && .base_name == clk1}]`
	
	`get_db $my_pins .slack_max_rise -index $my_clk_obj`
	
	The `-index` parameter also accepts a pattern of attribute name to be matched. While specifying an attribute pattern string with `-index`, attributes that do not support `-index`, will return their normal value. But the attributes that support `-index` will return the values specified by the index. For example:
	
	`get_db net:dtmf_recvr_core/scan_out2 .* -index {view func-wc_cworst}`
	
	Will return all the attributes and values for the `net` object. Any attribute that supports a view index will return the value for that view, while the other attributes will return the value as if `-index` was not specified.

`-invert`	Negates the *`pattern`*, `-if` *`expression`*, and `-expr` *`expression`* filters to return the objects/attributes that do not match instead of the ones that do.

`-match_hier`	Specifies that the `*` in the *`pattern`* and *`expression`* expands to the next hierarchical divider character `/` instead of a complete string match. Example: The following command matches i1, i2, but not i1/i2. `get_db insts i* -match_hier`

pattern1 [*pattern2* ..]	Specifies string pattern(s), where:
	* matches 0 or more of any characters
	? matches exactly one character
	More complex patterns require `-regexp` or `-if` expression.
	Refer to the following examples:
	• The following command finds all instance names starting with A:
	`get_db insts .name A*`
	`A0 A1/A2 …`
	• The pattern matching can also work directly on a dual-ported object. In that case, it will match the '.name' attribute of the object, but return the object rather than the .name string. This matching works only for objects that have a `.name` attribute defined. For example, the following command finds all instance objects (not names) with a name starting with A:
	`get_db insts A*`
	`inst:top/A0 inst:top/A1/A2 …`
	• The pattern matching can also work to match multiple lists of the name patterns. Refer to the following example:
	`get_db insts */BC* *TC*`
	`CG/BC1 CG/BC2 TC1 TC2 TC3 TC4`
	Note: For the special case where a pattern string starts with '.' you need to use an extra '.' (an empty attribute name)
	so that '.text' is treated as a pattern and not as an attribute name.
	• The following command finds all nets whose names equal to `.odd_name`
	`get_db nets . .odd_name`
	• The following command finds all instance names starting with A or B:
	`get_db insts A* B*`

`-regexp`	Specifies that *`pattern`* use *regexp* syntax instead of the simple * and ? style. It does not change the `-if` pattern matching. See the example below to understand the usage of the `-regexp` parameter: `get_db insts -regexp {u_warmreset_cntr/reset_cntr_reg_(\d+)_}`
`-skip_fail`	Skips the fail error, when the object does not have such an attribute, or the object/attribute itself cannot be recognized.

`start`	The starting objects or root attributes. Can be an object list, collection, root (/), or root attribute.
	
	Example:
	
	• The following command returns the root attribute `insts` that is an object list of all `insts` in `current_design`.
	`>get_db insts`
	• The following command starts from a dual-ported object list or collection of objects in the Tcl variable `$my_list`, and returns a dual-ported object list.
	`>get_db $my_list`
	• The following command returns a root attribute that is a simple value.
	`>get_db timing_cap_unit`
	• If the name has a * or ?, it will be expanded to all matching root attribute names, and the display will show the attribute names and current value. This is intended for interactive usage, and no values are returned to Tcl. For example,
	`>get_db timing*`
	`Object: root:/`
	`timing_allow_input_delay_on_clock_source: false`
	`timing_analysis_aocv: false`
	`...`

`-unique`	Removes duplicate entries from the return list. If the specified attribute is a list, this parameter returns a unique set of lists., and does not uniquifies the elements within the list. In such a case, use `lsort`.

Built-In Sub-attributes

The types `rect`, `line`, `point`, and `polygon` have built-in sub-attributes. For a full list of all built-in sub-attributes, see Common UI Database Object Information.

Example:

The `rect` type has built-in sub-attributes like:

area	Area of rect
dx	Delta x
dy	Delta y
ll	Lower-left point
ur	Upper-right point
width	Shorter of dx and dy
length	Longer of dx and dy
perimeter	Length of the rect perimeter

A `point` has sub-attributes like:

x	X coordinate
y	Y coordinate

Like other attributes you can chain them together.

For example, an inst object has a .bbox (bounding box) that is type “rect”. Here are some operations allowed on the .bbox attribute of an inst.

```
get_db $my_inst .bbox
{1.0 2.0 3.0 4.0}
get_db $my_inst .bbox.ll
{1.0 2.0}
```

```
get_db $my_inst .bbox.area
4.0
get_db $my_inst .bbox.ll.x
1.0
set box [get_db $my_inst .bbox]
{1.0 2.0 3.0 4.0}
get_db $box .ll
{1.0 2.0}
```

Examples

- The following command finds insts with names starting with PTN* that have at least one pin capacitance > 0.1. The pattern is checked first, and those that pass will be tested with the `-if` expression. Inside the `-if` expression, there are many `.pins` for one inst, so each pin is tested. If any of the pins have a capacitance > 0.1, the single inst will be matched.
  ```
  get_db insts PTN2* -if {.pins.capacitance_max_rise  > 0.1}
  ```
- The following command returns insts that are not physical-only cells whose name matches `i1/i2*`.
  ```
  get_db insts i1/i2* -if {!.is_physical}
  ```
- The two commands below generate the same list of pins from `$my_insts` that have a base_name `clk`. The first uses a * pattern to match all pins, while the second has no pattern and defaults to all pins, so both cases check the `-if` expression for all pins of `$my_insts`.

  ```
  get_db $my_insts .pins * -if {.base_name == clk}
  get_db $my_insts .pins  -if {.base_name == clk}
  ```
- The following command finds `buf1` cell instances connected to nets named `clk` and changes their placement status to `fixed`.

  Note: The examples below is inefficient and used to illustrate different methods of traversal done in different styles. A more efficient method would be to start from the nets and looking at their pin connections directly.

  Example 1:

  ```
  set cell_name buf1
  set net_name clk
  foreach inst_obj [get_db insts] {
        set cell_obj [get_db $inst_obj .base_cell]
        if { [get_db $cell_obj .name] == $cell_name } {
                  foreach pin_obj [get_db $inst_obj .pins] {
                        if { [get_db $pin_obj .net.name] == $net_name } {
                            set_db $inst_obj .place_status fixed
  ```

```
                        }
                    }
                }
            }
```

Example 2: Same output using full traversal style with intermediate variables. The second `-if` returns true if any of the .pins.net.name match `$net_name` (implicit OR).

```
set buf1_list [get_db insts -if {.base_cell.name == $cell_name}]
set buf1_clk1_list [get_db $buf1_list -if {.pins.net.name == $net_name}]
set_db $buf1_clk1_list .place_status fixed
```

Example 3: Same output using full traversal style (fully chained)

```
set_db [get_db insts -if {.base_cell.name == $cell_name && .pins.net.name ==
$net_name}] .place_status fixed
```

Example 4: Or use the `-foreach` method so no list is returned, but the number of buffers processed is returned.

```
get_db insts -if {.base_cell.name == $cell_name && .pins.net.name == $net_name} -
foreach {set_db $obj(.) .place_status fixed}
```

- The following command finds all the insts named `i1/i2` that end with `_buf`:
  ```
  get_db insts i1/i2/*_buf
  ```
  Or your could start inside the `hinst` named `i1/i2`:
  ```
  get_db [get_db hinsts i1/i2] .insts *_buf
  ```
  Or use the hinst dual-ported object name directly. Note that netlist dual-ported object names have the design name included, so if `top` is the design name you could do this:
  ```
  get_db hinst:top/i1/i2 .insts *_buf
  ```
- The following command gets `arrival_max_rise` for the `analysis_view setup_test` for all pins. Here the abbreviation "view" is allowed for analysis_view.
  ```
  get_db pins .arrival_max_rise -index "view setup_test"
  ```
- The following command finds insts that have a cell_port named CK and that pin is connected to a net named clk*:
  ```
  get_db insts -if {.lib_cells.lib_pins.name == */CK && .pins.net.name == clk*}
  ```
- The following command uses -foreach to find the first inst named u1/* with .place_status "fixed" and then break. It uses strict Tcl inside the -foreach.
  ```
  get_db insts u1/* -foreach { if {[string match fixed $obj(.place_status)]} {
  puts $obj(.name) ; break } }
  ```
- The following `get_db` command uses `-foreach` to count the number of each base_cell used in

the netlist inside the cell_count Tcl array. You can then use `array get cell_count` to see a list of all the `base_cell` names and how many times they are used in the netlist.

```
get_db insts .base_cell -foreach {incr cell_count($obj(.name))}
```

- The following `get_db` command displays all the details of the `flow_template_type` attribute of the `root obj_type`:

```
get_db attribute:root/flow_template_type .*
Object: attribute:root/flow_template_type
base_name: flow_template_type
category: flow
check_function: {}
compute_function: {}
data_type: string
default_value: {}
help: {Type of template being run.}
indices:
is_computed: false
is_obsolete: false
is_settable: true
is_user_defined: true
name: root/flow_template_type
obj_type: obj_type:root
set_function: {}
skip_in_db: false
```

- The following example depicts the use of the `-if` parameter to find all HVT insts:

```
get_db insts -if {.base_cell.name == *_HVT}
```

- The following command returns all hinsts 1 or 2 levels inside the design:

```
get_db design:top -depth {1 2}
hinst:top/h1 hinst:top/h1/h2
```

- The following command finds all `local_insts` of each `hinst` at depth 2 that have `.dont_touch == true`:

```
get_db design:top .local_insts -depth {2 2} -if { .dont_touch == true }
inst:top/h1/h2/i1 inst:top/h1/h2/i2
```

- All the following commands return inst with `.name i1/i2`:

```
set a i1/i2
get_db insts i1/i2
get_db insts -expr {$obj(.name) eq $a}
get_db insts -expr {[get_db $obj(.) .name] eq "i1/i2"}
```

- The following command returns all nets with more than 5 loads:
  ```
  get_db nets -expr { [llength $obj(.loads)] > 5 }
  ```
- The following command returns all insts with `base_cell.name AND2`:
  ```
  get_db insts -expr {$obj(.base_cell.name) eq "AND2"}
  ```

Related Topics

- For details on LEF property definitions, see the 'Property Definitions' section on the LEF Syntax chapter in the *LEF/DEF Language Reference.*
- For information on how to set DB attributes, see the `set_db` command.
- For information on how to reset attribute values, see the `reset_db` command.
- For a list of objects attributes that can be queried for information or changed, see Common UI Database Object Information.

get_db Category Attributes

The root attributes in the `get_db` category are:

- get_db_display_limit

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `get_db` category, use the following command:

```
get_db -category get_db
```

get_db_display_limit

`get_db_display_limit {integer}`

Default: `10`

Controls the length of lists and number of objects displayed on the screen using queries that contain wildcards.

Note: This variable does not change the return value of `get_db` command.

Example

The following example sets the number of objects and length of the lists to 1 and then calls `get_db` to display instance attributes. `get_db` command outputs one instance object and every attribute list is truncated to one element.

```
set_db get_db_display_limit 1
get_db insts .*

Object: inst:TOP/I0
arcs:                   NC
area:                   0.7056
base_cell:              base_cell:BUFFD0BWP
bbox:                   {0.0 0.0 0.0 0.0}
.....
```

```
pg_base_pins:          base_pin:BUFFD0BWP/VDD ... (total length 2) #truncated as length
of the list is set to 1
pg_pin_nets:           net:TOP/VDD ... (total length 2) #truncated as length of the
list is set to 1
pins:                  pin:I0/Z ... (total length 2) #truncated as length of the list
is set to 1
...
route_halo_top_layer:  {}
test_reset_inst:       1
```

Note: Number of objects exceeds limit. Use `set_db get_db_display_limit <value>` to control the number of objects evaluated by `get_db <object>.*` query.

get_edge

```
get_edge
[-help]
[-clockwise | -counterclockwise]
[{shape_list} [-direction direction] [-db_units]]
[-index index_number]
```

Returns edges of a polygon of a shape along with the direction list (N, S, E, W and NE, NW, SE, SW). This helps you identify not only all the edges but also the direction in which the edges are pointing. At a time you can observe one polygon with 'n' number of points.

Parameters

`-help`	Prints a brief description that includes type and default information for each `dbedge` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_edge`
`-clockwise`	Sorts the returned list of edges clockwise.
`-counterclockwise`	Sorts the returned list of edges counterclockwise.

`{shape_list}`	Can be a rect like `{x1 y1 x2 y2}` or a polygon like `{{x1 y1}{x2 y2}{x3 y3}.....}`. Only one rect or polygon is accepted.
	Examples
	Using a simple L-shape polygon
	`set shape_list {{0 0} {2 0} {2 1} {1 1} {1 2} {0 2}}`
	• All edges
	`get_edge $shape_list`
	`{{S{0 0}{2 0}} {E{2 0}{2 1}} {N{2 1}{1 1}} {E{1 1}{1 2}}`
	`{N{1 2}{0 2}} {W{0 2}{0 0}}}`
	• Only E facing edges
	`get_edge -direction E $shape_list`
	`{{E{2 0} {2 1}} {E{1 1} {1 2}}}`
	The format of the output is
	`{{<direction> <edge_vertex_1> <edge_vertex_2 }....}`

`-db_units`	Returns the specified values in database units. Default is `microns`.
`-direction` *direction*	Specifies the types of edges needed. This can be n, s, e, w, ne, nw, se, sw. It can also be a list. For example: `get_edge -direction {n s nw ne} {10 20 30 40}`
`-index` *index_number*	Specifies the index number of the edge to be returned. The index number starts with `0` as the first edge specified in the shape list, and increases gradually along with the shape list. This number may range from `0` to '*number_of_edges* `- 1`'.

get_obj_in_area

```
get_obj_in_area
[-help]
[-areas {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2} ...}
[-bbox_overlap]
[-dbu]
[-layers {layer1 layer2 ...} [-strict_via_inst_layer_check]]
[-obj_type {bump bus_guide inst inst_all_shapes pin port marker net place_blockage
patch_wire pg_pin route_blockage routes resize_blockage row special special_via
special_wire via wire what_if_wire what_if_via port_shape partition}]
[-abut_only] | [-enclosed_only] | [-overlap_only]
[-polygons {x11 y11 x12 y12 x13 y13 ...}]
```

Returns a list of objects based on an area query without needing to select them. This command supports all types of routing blockages such as partial, fill, and slot.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_obj_in_area` parameter. For a detailed description of the command and all its parameters, use the man command: `man get_obj_in_area`
`-abut_only`	Returns only those objects that abut the search area. No part of the object polygon overlaps the search area, but at least one point or line of intersection occurs between the object polygon and the search area. A single point of abutment (such as a corner or crossing edges) is also considered as abutted. When multiple areas are specified, the object is returned as long as it abuts one of the search areas.
`-areas {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2} ...}`	

	Specifies the search areas, where `ll` = lower left, `ur` = upper right. By default, all the objects overlapping or abutting these areas are returned, unless `-abut_only`, `-enclosed_only`, or `-overlap_only` is specified. The polygon shape of the object is used for the overlap check. The units are in microns (unless `-d` is specified). The input can be in either a single `rect {llx1 lly1 urx1 ury1}`, or a list of `rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}}`. A `rect` using "two points" format like `{{{llx1 lly1} {urx1 ury1}} {{llx2 lly2} {urx2 ury2}}}` is also allowed. But input format like `{llx1 lly1 urx1 ury1 llx2 lly2 urx2 ury2}` or `{{llx1 lly1} {urx1 ury1} {llx2 lly2} {urx2 ury2}}` is illegal and not allowed. The resulting area to check for overlap can be a rectangle, line, or a point. When one object intersects multiple search areas, it will only be returned once.
`-bbox_overlap`	
	Uses bounding-box of the object rather than the polygon shape of the object for overlap for overlap, abut, or enclosure check. For example, an `inst` that is of L-shape, will not be returned if the `-areas` parameter overlaps only the upper-right of the bounding-box without touching any part of the L-shaped, unless the `-bbox_overlap` parameter is included.
`-dbu`	Specifies the `-areas` values that are in database units rather than microns. *Default*: `um`
`-enclosed_only`	Returns only those objects, which are completely enclosed inside the search area. The object polygon is fully enclosed by the search area. No part of the object polygon sticks outside the search area, although it may abut the search area edges. When multiple areas are specified, the object will be returned as long as it is enclosed in one of the search areas.
`-layers {`*`layer1 layer2 …`*`}`	

	Returns all the objects with the layer information for the specified layers. This parameter applies only to object types: `port_shape`, `patch_wire`, `route_blockage`, `regular`, `special`, `special_via`, `special_wire`, `via`, `wire`, `what_if_wire`, and `what_if_via`. A `via`, `special_via`, or `what_if_via` object will be returned if any of the via's bottom layer, cut layer, or top layer matches with one of the specified layers. Other object types ignore this parameter and will always be returned.
`-obj_type` *`objTypeList`*	
	Specifies a list of object types to search for overlaps in the area. Multiple object types can be searched at the same time. The objects currently supported include: `bump bus_guide inst inst_all_shapes pin port port_shape marker net place_blockage patch_wire pg_pin route_blockage routes resize_blockage row special special_via special_wire via wire what_if_wire what_if_via partition` Here: • `routes` is the union of `wire`, `patch_wire`, and `via`. • `special` is the union of `special_via` and `special_wire`. Objects with multiple shapes or non-rectangular shapes use a polygon to check for overlap, as described below. • `bump` has no placement overlap area, so it uses the polygon of all the OBS and PIN shapes. • `inst` uses the polygon of its placement overlap checks (for example the LEF SIZE or OVERLAP shapes), including any placement halo (if there is any) but not any routing halo. OBS shapes or PIN shapes (like standard-cell power-pins) that stick outside the placement overlap area are not included. • `inst_all_shapes` is the same as `inst`, except that it includes all OBS shapes and PIN shapes even if they stick outside the placement overlap area. • `pin` uses the polygon of all the pin-shapes of the `pin`. • `pg_pin` uses the polygon of all the pin-shapes of the `pg_pin`. • `port_shape` uses the polygon of all the shapes of the top-level `port_shape`.

	• `net` uses the polygon of all the regular routing shapes on the net. Pin-shapes or special-route shapes attached to the net are not included. • `port` uses the polygon of all the pin-shapes of the top-level port. • `port_shape` uses the polygon of all the pin-shapes of the top-level port shape. • `via` uses the polygon of all the shapes of the via. • `special_via` uses the polygon of all the shapes of the via. • `partition` uses the polygon of all the shapes of the partition. *Default*: `inst`
`-overlap_only`	Returns only those objects, which are partially overlapped with the search area. Some part of the object polygon overlaps the search area, whereas some of its part is outside the search area. A single point of contact is not considered as an overlap. When multiple areas are specified, the object is returned as long as it overlaps with one of the search areas.
`-polygons {`*`x11 y11 x12 y12 x13 y13 ...`*`}`	
	Specifies a polygon search area. By default, all the objects overlapping or abutting this area are returned on using the `-polygons` parameter, unless the `-abut_only`, `-enclosed_only`, or `-overlap_only` parameter is specified. The units are in microns (unless `-d` is specified, which indicates the database units). The format can be in either like `{x11 y11 x12 y12 x13 y13 ...}` or `{{x11 y11} {x12 y12} {x13 y13} ...}`. The polygon must be well-formed, not have zero area, and not have crossing edges.
`-strict_via_inst_layer_check`	
	Returns only those objects on the specified layer, which match with one of the via's bottom, cut, or top layers. This parameter does not return the objects of the via on the unmatched layers. It works only for `special_via` and `via`.

Examples

- The following command returns all the instances and routing-blockages on the `metal6` and `metal7` layers that are partially overlapped, abutted, or enclosed in the given

triangle area:

```
get_obj_in_area -obj_type {inst route_blockage} -polygons {100 100 110 110 100 110} -layers {Metal6 Metal7}
```

- The following command returns all the instances and routing-blockages on the `metal6` and `metal7` layers that are partially overlapped, abutted, or enclosed in the given triangle area using the bounding-box of the objects:

```
get_obj_in_area -obj_type {inst route_blockage} -polygons {100 100 110 110 100 110} -layers {Metal6 Metal7} -bbox_overlap
```

- The following command returns all the instances and regular wires on the `metal6` and `metal7`layers that are abutted in the give search areas:

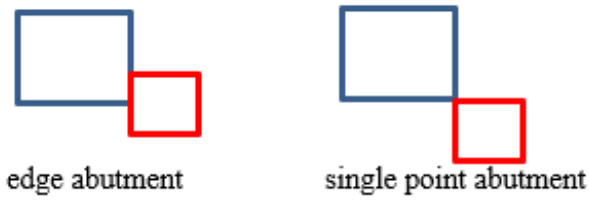
```
get_obj_in_area -obj_type {inst wire} -areas {{0 0 100 100} {150 100 200 200}} -layers {Metal6 Metal7} -abut_only
```

- The following command returns all the instances and regular wires on the `metal6` and `metal7`layers that are abutted in the given search areas using the bounding-box of the objects:

```
get_obj_in_area -obj_type {inst wire} -areas {{0 0 100 100} {150 100 200 200}} -layers {Metal6 Metal7} -abut_only -bbox_overlap
```

- Following are the figures to illustrate each position relation. The instances presented by the red rectangles are returned with the specified parameter when the search area presented in the blue rectangle are specified.

  - **`-abut_only`**



  - **`-overlap_only`**



  - **`-enclosed_only`**



enclosed only



enclosed and abut

Note:

1. If none of the above three parameters is specified, all the instances presented by the red rectangles will be returned, by default.
2. For querying object with multi-shape (such as `port_shape`) in multiple search areas, if one of its shapes is enclosed/abutted/overlapped with any one of the specified areas, the object will be returned.

get_rect

```
get_rect
[-help]
<rect> {-ll | -ur | -llx | -lly | -urx | -ury | -size | -dx | -dy | -width | -length |
-center | -area}
```

Returns the coordinates of the points of a rectangle. When you need to find the coordinates of a rectangle, in order to either move the rectangle or reshape the rectangle, the command helps you find the coordinates.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_rect` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_rect`.
`<rect> {-ll | -ur | -llx | -lly | -urx | -ury | -size | -dx | -dy | -width | -length | -center | -area}`	
	Specified rectangle to access values from. • `-area`: Returns the area of the rectangle. • `-center`: Returns the center of the rectangle. • `-length`: Returns the larger of the `-dx` and `-dy` values. • `-ll`: Returns the lower left point of the rectangle. • `-llx`: Returns the lower left x coordinate of the rectangle. • `-lly`: Returns the lower left y coordinate of the rectangle. • `-dx`: Returns the x difference between the larger and smaller x values of the rectangle. • `-dy`: Returns the y difference between the larger and smaller y values of the rectangle. • `-ur`: Returns the upper right point of the rectangle. • `-urx`: Returns the upper right x coordinate of the rectangle. • `-ury`: Returns the upper right y coordinate of the rectangle. • `-size`: Returns the size of the rectangle from lower left to upper right. • `-width`: Returns the smaller of the `-dx` and `-dy` values.

Examples

- `-area {8.1 9.2 10.3 11.4} returns 4.84`
- `-center {8.1 9.2 10.3 11.4} returns 9.2 10.3`
- `-length`: `get_rect -length {8.1 9.2 10.3 11.4}` returns: `2.2`
- `-ll`: `get_rect -ll {8.1 9.2 10.3 11.4}` returns: `8.1 9.2`
- `-llx`: `get_rect -llx {8.1 9.2 10.3 11.4}` returns: `8.1`
- `-lly`: `get_rect -lly {8.1 9.2 10.3 11.4}` returns: `9.2`
- `-dx`: `get_rect -dx {8.1 9.2 10.3 11.9}` returns: `2.2`
- `-dy`: `get_rect -dy {8.1 9.2 10.3 11.9}` returns: `2.7`
- `-ur`: `get_rect -ur {8.1 9.2 10.3 11.4}` returns: `10.3 11.4`
- `-urx`: `get_rect -urx {8.1 9.2 10.3 11.4}` returns: `10.3`
- `-ury`: `get_rect -ury {8.1 9.2 10.3 11.4}` returns: `11.4`
- `-size: get_rect -size {8.1 9.2 10.3 11.4}` returns `2.2 2.2`
- `-width`: `get_rect -width {8.1 9.2 10.3 11.4}` returns: `2.2`

get_transform_shapes

```
get_transform_shapes
[-help]
[-db_units]{-inst instPtr | -hier_inst instPtr | {-cell cellPtr -orient orientEnum -pt
{x y}}}
{-local_pt list | -global_pt list}
```

Takes coordinates local to a cell and returns them in context with the global design space. You can specify a specific instance of a cell in the design, or you can specify a library cell, its location in the design, and instruct the software to perform a "what-if" translation of coordinates within that cell.

Parameters

-help	Pints a brief description that includes type and default information for each `get_transform_shapes` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_transform_shapes`
`-cell` *`cellPtr`*	Specifies a pointer to a library cell. You must specify the `-pt` and `-orient` parameters with `-cell`.
`-db_units`	Specifies values in database units, specified in integers. This is an optional parameter. If you use the `-db_units` option, the Boolean value is 1, so the value will be displayed in database units. If you don't specify the `-db_units` option, the Boolean value is 0, the default. *Default*: Specifies values in user units, in microns
`-global_pt` *`globalPts`*	Specifies the global coordinates to convert into local coordinates.
`-hier_inst`	Specifies the hierarchical instance of a database object. This option is mandatory.
`-inst` *`instPtr`*	Specifies a pointer to a specific instance/bump of a cell in the design.The software derives the coordinates and the orientation for the cell instance from the design.
`-local_pt` *`list`*	Specifies the local coordinates to convert into global coordinates
`-orient` *`orientEnum`*	Specifies the orientation of the cell specified with `-cell`. You must specify the `-cell` and `-pt` parameters with `-orient`. Valid orientation values are: `r0`, `r90`, `r180`, `r270`, `mx`, `mx90`, `my`, and `my90`.
`-pt` *`x y`*	Specifies the coordinates for the cell specified with `-cell`. You must specify the `-cell` and `-orient` parameters with `-pt`.

Examples

- Assume that a shape exists on *M1* within a NAND2 cell, from (`1 1`) to (`2 2`). If an instance of the NAND2 cell is placed at (`10 10`), with an `r0` (N) orientation, the following command converts the local points (`1 1 2 2`) of the NAND2 cell into their global coordinates:

  ```
  get_transform_shapes -cell [get_db -p head.allCells.name NAND2] -pt {10 10} -
  orient r0 -local_pt {1 1 2 2}
  ```

  The software returns the following information:

  ```
  {11 11 12 12}
  ```

- Assume that the NAND2 cell placed at (`10 10`), with an `r0` (N) orientation is named `i1`. The following command converts the local points (`1 1 2 2`) of `i1` into their equivalent global coordinates:

  ```
  get_transform_shapes -inst [get_db -p top.insts.name i1] -local_pt {1 1 2 2}
  ```

  The software returns the following information:

  ```
  {11 11 12 12}
  ```

get_via_pillars

```
get_via_pillars
[-help]
{-base_pin {base_pin | pg_base_pin} | -pin {pin | pg_pin}}
[-required]
```

The command returns a via pillar list on a cell pin.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `get_via_pillars` parameter. For a detailed description of the command and all its parameters, use the man command: `man get_via_pillars`
`-base_pin {base_pin | pg_base_pin}`	Specifies the base pin name or pointer.
`-pin {pin | pg_pin}`	Specifies the pin name or pointer.
`-required`	Returns the required flag.

Related Information

`set_via_pillars`

init_hier_cell

```
init_hier_cell
[-help]
cell_names
[-dual_view]
```

Initializes one or more hierarchical cells based on the applicable order.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `init_hier_cell` parameter. For a detailed description of the command and all its parameters, use the man command: `man init_hier_cell`
`cell_names`	Specifies the names of the hierarchical cells to be initialized.
`-dual_view`	Binds the top and global views, and initializes the hierarchical cells inside ILMs.

is_attribute

```
is_attribute
[-help]
attribute_name
-obj_type obj_type
```

The `is_attribute` command checks whether attribute exists or not.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `is_attribute` parameter. For a detailed description of the command and all of its parameters, use the man command: `man is_attribute`.
`attribute_name`	Specifies the name of the attribute to be checked.
`-obj_type` *`obj_type`*	Specifies the object type of the attribute. All valid object types can be found by typing `get_db object_types .name` at the prompt. The attribute is assigned to the specified type.

Examples

- The following `is_attribute` command applies check to existing attribute:

```
define_attribute test_delete -obj_type inst -data_type int -category test -
help_string "test delete function" -default 1
is_attribute -obj_type inst test_delete
1
```

- The following `is_attribute` command applies check to non-existent attribute:

```
is_attribute -obj_type pins test_delete
0
```

Related Topics

- For information on how to query DB attributes, see the get_db command.
- For a list of objects attributes that can be queried for information or changed, see the Common UI Database Object Information document.

report_floating_nets

```
 report_floating_nets
[-help]
[-out_file file_name]
```

Reports the number of nets with zero terms, including the PG nets. This command can also be used to report the number of dangling nets, which could or could not be removed by the `delete_floating_nets` command. The command reports the following information, when used:

- Total number of nets
- Number of dangling nets
- Number of removable dangling nets
- Number of non-removable dangling nets

The removable and non-removable nets are reported based on the usage of the `delete_floating_nets` command. Those dangling nets, which could not be removed by the `delete_floating_nets` command, are reported in the output file due to multiple possible reasons, such as the dangling net:

- Is a physical net.
- Is a PG net.
- Is a bus bit net.
- Is an assign net.
- Has multiple module ports and at least one port is associated with some constraints.
- Has some constraints, such as in an ILM module or some timing constraint.

Note: If a dangling net cannot be deleted due to any reason, only the first reason listed above will be reported in the output file.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_floating_nets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_floating_nets`
`-out_file` *`file_name`*	Reports the list of dangling nets that cannot be deleted by the `delete_floating_nets` command, along with its reasons.

Examples

- The following command reports the information about the nets listed in the `dangling_net.rpt` file:

```
report_floating_nets -out_file dangling_net.rpt
```

- Following is the output of the above command:

```
################################################################
# Generated by: Cadence Innovus 19.10-p002_1
# OS: Linux x86_64(Host ID sjfsb473)
# Generated on: Sat Aug 18 11:14:18 2018
# Design: design_name
# Command: report_floating_nets -out_file dangling_nets.rpt
################################################################

Total number of nets: 123456
The number of dangling nets: 26
Removable dangling nets: 22
Non-removable dangling nets: 4
Is physical net: 2
Is P/G net: 0
Is bus net: 1
Is assign net: 0
Has module port:0
Has constraints: 1


Reasons for the net not being removed:
net:xxxx is physical net ## net:xxxx here is the DPO name of the net
net:xxxx is physical net
net:xxxx is bus net
net:xxxx has constraints
```

report_metal_stack

```
report_metal_stack
[-help]
[-non_preferred_direction]
```

Reports the metal stack configuration and routing track information. For each routing layer, this includes the layer name, preferred routing direction, routing offset (offset of routing track from 0), routing pitch (space between each routing track), min-width DRC rule, min-spacing DRC rule, and the routing default-width (wire width of the default routing rule). Note that for a horizontal direction track, the offset and pitch are in the Y direction (for example, the Y offset from 0, and the Y pitch of horizontal tracks). This command only supports uniform track-patterns with a single pitch value, and will not report non-uniform pitch routing track patterns that can be created with add_tracks or create_tracks.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_metal_stack` parameters. For a detailed description of the command and all its parameters, use: `man report_metal_stack`
`-non_preferred_direction`	By default, the `report_metal_stack` command displays the metal stack and routing information only related to the preferred routing direction. When the `-non_preferred_direction` parameter is used, the information related to the non-preferred routing direction is also displayed.

Examples

- The following command displays the metal stack and routing information:
  `report_metal_stack`

  The following is an example of the output of the above command:

  ```
  ================================================================================
  ```

```
Layer   Direction   Offset    Pitch      Min_Spacing  Min_Width   Default_Width
================================================================================
M1      horizontal  0.000000  0.040000   0.020000     0.020000     0.020000
M2      vertical    0.000000  0.057000   0.020000     0.037000     0.037000
M3      horizontal  0.000000  0.040000   0.020000     0.020000     0.020000
M4      vertical    0.000000  0.048000   0.020000     0.028000     0.028000
```

- The following command adds in the non-preferred routing direction information:

  ```
  report_metal_stack -non_preferred_direction
  ```

  The following is the output of the above command:

```
================================================================================
=======
Layer  Direction     Preferred Offset    Pitch      Min_Spacing  Min_Width
 Default_Width
================================================================================
=======
M1      horizontal   yes       0.000000  0.040000   0.020000     0.020000
0.020000
M1      vertical     no        0.000000  0.040000   0.000000     0.000000
0.060000
M2      vertical     yes       0.000000  0.057000   0.020000     0.037000
0.037000
M2      horizontal   no        0.000000  0.057000   0.000000     0.000000
0.060000
M3      horizontal   yes       0.000000  0.040000   0.020000     0.020000
0.020000
M3      vertical     no        0.000000  0.040000   0.000000     0.000000
0.060000
M4      vertical     yes       0.000000  0.048000   0.020000     0.028000
0.028000
M4      horizontal   no        0.000000  0.048000   0.000000     0.000000
0.060000
```

report_via_def

```
report_via_def
[-help]
[-detail]
[-on_clock_nets]
[-on_selected_nets]
[-regular]
[-special]
```

Reports single and multi-cut via summary for special and/or regular vias. Such a summary could be used for generating automotive statistics regarding the usage of single and multi-cut used vias.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `report_via_def` parameter. For a detailed description of the command and all its parameters, use the man command: `man report_via_def`
`-detail`	Prints the detailed report related to every via def.
`-on_clock_nets`	If specified, the command reports vias only on the clock nets in the design. If not specified, the command reports vias on all nets in the design by default. This parameter can be used with the `-detail`, `-special`, and `-regular` parameters.
`-on_selected_nets`	If specified, the command reports vias only on the specified selected nets. If not specified, the command reports vias on all nets in the design by default. This parameter can be used with the `-detail`, `-special`, and `-regular` parameters.
`-regular`	Prints the report related to regular routing vias.
`-special`	Prints the report related to special routing vias.

Examples

Use the following command to display a detailed summary of the single and multi-cut vias:

```
report_via_def -detail
```

Following is the sample output of the above command:

```
#-------------------------------------------------------------------------

# V1                1628089 (100.0%)              0 (  0.0%)        1628089

# V2                1685800 ( 99.1%)          15198 (  0.9%)        1700998

# MF                1076133 (100.0%)            131 (  0.0%)        1076264

# Q4                 536537 (100.0%)              0 (  0.0%)         536537

# FD                 250646 ( 71.3%)         101082 ( 28.7%)         351728

# S6                 189504 ( 63.8%)         107464 ( 36.2%)         296968

# S7                 131419 ( 54.8%)         108215 ( 45.2%)         239634

# S8                  96592 ( 57.5%)          71325 ( 42.5%)         167917

# S9                  64882 ( 50.8%)          62728 ( 49.2%)         127610

# S10                 33187 ( 38.5%)          52936 ( 61.5%)          86123

# S11                 22774 ( 42.7%)          30613 ( 57.3%)          53387

# DI                  12255 (100.0%)              0 (  0.0%)          12255

# II                      0 (  0.0%)           7268 (100.0%)           7268

#-------------------------------------------------------------------------

# Total             5727818 ( 91.1%)         556960 (  8.9%)        6284778
```

reset_db

```
reset_db
[-help]
[start[chain]] |
[-category category_name]
[-quiet]
[-verbose]
```

Resets attribute value to a pre-defined default value. It returns a single value when resetting a single attribute, a list of values when resetting same attribute on different object instances, or a human readable table for multiple attributes in the following format:

```
Attribute_name   value
```

For example:

```
Place_xxx true
Add_filler  true
```



- The `-category` parameter of the `reset_db` command can be used to reset root attributes only. Both user-defined and system root attributes can be reset using `the -category` parameter.
- The attribute value can be reset if the `default_value` attribute is specified.

When the `-category` parameter is used, the command resets the values of both the hidden and unmapped attributes. In such a case, the command does not return the count of the reset attributes. The output in such cases will be like the following:

```
Some hidden attributes were also reset.
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_db` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_db`.
`start`	Can be any object list (or collection) or *shorthand*. For example nets, insts...
`chain`	Specifies the *chain* in the form `[.attribute_name1 [.attribute_name2]...]`. The specified attribute names are valid for each level of the traversal in the *chain*.
`-category` *`category_name`*	Specified root attribute category based on predefined list of categories. For example timing. **Note:** Applies only to root level attributes and ignored if applied to lower levels.
`-quiet`	By default `set_db` is quiet. If the `set_db_verbose` root attribute is set to true, then the name of each object that is modified is echoed to the terminal (see the `-verbose` parameter's description for an example). `-quiet` forces `set_db` to be quiet even if`set_db_verbose` is true. This is intended for internal scripts that want to be certain that the verbose output is off.

`-verbose`	Turns on verbose output that echoes the name of each object that is modified. This can be useful during interactive debugging to ensure that the correct objects were affected. For example:
	
	`set_db -verbose [get_db hinsts D*] .dont_touch false`
	`Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = false`
	`Setting attribute of hinst 'DECODE_INST': 'dont_touch' = false`
	`2 false`
	
	Note: The `set_db_verbose` root attribute can also be set to true, to force verbose output for all `set_db` and `reset_db` usage without requiring `-verbose`.

Examples

- The following reset_db command resets the value of instance attribute:

```
define_attribute test_reset_inst -obj_type inst -data_type int -category reset -
help_string "test reset function" -default 1
1
set_db insts .test_reset_inst 10
10 10 10 10 10       #Four instances in the design
reset_db insts .test_reset_inst
1 1 1 1 1            #All 4 attributes are reset to default values
```

- The following command resets same set of attributes using the `-category` parameter.

```
reset_db -category reset
```

  The attributes are reset to their default values and a human readable table with new values is displayed.

```
test_reset1 1
test_reset2 1
test_reset3 1
```

Related Topics

- For information on how to query DB attributes, see the `get_db` command.
- For information on how to set DB attributes, see the `set_db` command.
- For a list of objects attributes that can be queried for information or changed, see the Common UI Database Object Information document.
- set_db Category Attribute

set_db

```
set_db
[-help]
start
[chain]
value
[-dbu]
[-index {index_type1 name1 [index_type2 name2 …]}]
[-quiet]
[-verbose]
```

Changes the specified attribute value for a database object. The `set_db` command takes a dual-ported object, dual-ported object list, or a collection as input, and then uses a period (.) to traverse to other related objects or to access attributes. You can use additional periods to traverse the data base schema.

This command returns the number of attributes set and the value. See the example below:

```
set_db hinsts .ungroup_ok false
6 false
```

In the above scenario usage, `values == value` as a single command can set only a single value (for multiple objects).

Note: A Dual-Ported Object (DPO) is a `Tcl_Obj` in the C++ programming interface. The pointer is kept as a C++ pointer inside Tcl unless Tcl forces it to be evaluated as a string. In normal usage it never gets converted to a string which is more efficient, but if you do a `puts $var` or return the value to the shell, it will be converted to its dual string form.

- The string form is normally a useful name preceded by the `obj_type`, so a `base_cell` object string form might look like `base_cell:and2`.
- A `netlist` name will have the current design name included, so an instance named `i1/i2` in design top, would look like `inst:top/i1/i2`.
- An object with no name like a wire, will just show the pointer hex-value like `wire:0x22222`.

The `set_db` command does not create or delete objects.

For information on what object attributes are settable, see the Common UI Database Object Information document. It contains a description of all database objects and attributes.

The root object has the current design data under the `current_design` attribute, while technology and library data are normally directly accessible as root attributes (For example, layers and

libraries). The root object is the default starting point if there is no input list or collection given.

You can set any root attribute in any of the following ways:

- `set_db / .flow_mail_to name`
  where, '/' is the notation for the root object , or,
- `set_db flow_mail_to name`
  This implicitly starts at the root.

For example, you can use the following command to change the design text font:

```
set_db current_design .texts.font_number  4
```

Note: The `current_design` is implicitly created by the `init_design` or the `read_db` commands.

Frequently used `current_design` attributes like `insts` or `hinsts` (hierarchical insts) have root attribute aliases (or shortcuts). These attributes can be accessed by just typing the root attribute alias name.
For example, the following two commands set all insts .place_status to fixed.

```
set_db current_design .insts.place_status fixed
set_db insts .place_status fixed
```

Some of the more commonly used aliases from the current_design attributes include: insts, hinsts, nets, hnets, ports, hpins, pins.

The `set_db` command supports auto-complete when you press the <`Tab`> key. It will give a list of all matching attributes starting with the letters entered so far, or it will complete the name if it is unique.

For example:

```
set_db plan_design_use<Tab>
```

returns `plan_design_use_guide_boundary plan_design_use_sdp_group`

For information on how to access DB objects, see the `get_db` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_db` parameter.
`start`	The starting objects or attributes. Can be an object list, collection, root (/), or root attribute.

chain	Specifies a *chain* in the form `[.attribute1[.attribute2]...]`. The attribute names must be valid for each level of the traversal in the *chain*, where the first name is valid for the start object. For example, `set_db $my_pins .inst.place_status fixed` where `.inst` is an attribute of an pin object and `.place_status` is an attribute of an inst object.
value	Specifies new value for the specified object attribute elements. The value type depends on which object attribute type is being modified.
`-dbu`	Indicates that attribute values (coord, pt, rect, polygon, area, ...) should be in database units. *Default*: The default is microns (or microns**2).
`-index {`*index_type1 name1* `[`*index_type2 name2* `…]}`	
	Some attributes represent an array of values. For example, a pins .slack_max_rise attribute has a different value for each analysis_view. With no index, all the values defined for that attribute are modified by the `set_db` command. If you want to change a specific value, you must include the index value. The index values are pairs of obj_type names (for example, analysis_view, clock, skew_group) followed by the name. The word “view” is accepted as an abbreviation for “analysis_view”. Example: Some ccopt attributes can take both a view and clock index like this: `-index "view func_wc_cworst clock clk1"`
`-quiet`	By default `set_db` is quiet. If the `set_db_verbose` root attribute is set to true, then the name of each object that is modified is echoed to the terminal (see the `-verbose` parameter's description for an example). `-quiet` forces `set_db` to be quiet even if `set_db_verbose` is true. This is intended for internal scripts that want to be certain that the verbose output is off.

`-verbose`	Turns on verbose output that echoes the name of each object that is modified. This can be useful during interactive debugging to ensure that the correct objects were affected. For example:
	`set_db -verbose [get_db hinsts D*] .dont_touch false`
	`Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = false`
	`Setting attribute of hinst 'DECODE_INST': 'dont_touch' = false`
	`2 false`
	Note: The `set_db_verbose` root attribute can also be set to true, to force verbose output for all `set_db` and `reset_db` usage without requiring `-verbose`.

Examples

- The following command sets a value of root attribute.
  ```
  set_db place_global_solver_effort high
  ```
- The following commands define an instance attribute and change its value to 0.
  ```
  define_attribute myattr -default 1 -obj_type inst -data_type int -help_string "my attribute" -category my_cat
  set_db insts .myattr 0
  ```
- The following command sets an instance attribute value to 5 only if its current value is 0.
  ```
  set_db [get_db insts -if {.myattr == 0}] .myattr 5
  ```

Related Topics

- For information on how to query DB attributes, see the `get_db` command.
- For information on how to reset attribute values, see the `reset_db` command.
- For a list of objects attributes that can be queried for information or changed, see the Common UI Database Object Information document.
- set_db Category Attribute

set_db Category Attribute

The root attribute in the `set_db` category is:

- set_db_verbose

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `get_db` category, use the following command:

```
get_db -category set_db
```

set_db_verbose

```
set_db_verbose {true | false}
```

Default: false

When true, additional messages are displayed describing every object modified by set_db or reset_db commands.

Refer to the following example:

```
set_db set_db_verbose true
set_db [get_db hinsts */D*] .dont_touch true
Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = true
Setting attribute of hinst 'DECODE_INST': 'dont_touch' = true
2 true
set_db set_db_verbose false
Setting attribute of root '/': 'set_db_verbose' = false
1 false
set_db [get_db hinsts */D*] .dont_touch true
2 true
```

set_via_pillars

```
set_via_pillars
[-help]
{-base_pin {base_pin | pg_base_pin} | -pin {pin | pg_pin}}
[-required {0 | 1 | 2 | 3 | 4}]
stack_via_rule*
```

This command sets a via pillar list for a cell `base_pin` or a single via pillar for a `pin`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `set_via_pillars` parameter. For a detailed description of the command and all its parameters, use the man command: `man set_via_pillars`
`-base_pin {base_pin	pg_base_pin}`	Specifies the base pin name or pointer. A via pillar list is allowed for this parameter.
`-pin {pin	pg_pin}`	Specifies the pin name or pointer. Only one via pillar is allowed for this parameter.

`-required {0 | 1 | 2 | 3 | 4}`	Sets the `.stack_via_rule_Required` attribute for the *`base_pin`*/*`pin`* values specified by the `-base_pin` or `-pin` parameters. Setting the `.stack_via_rule_Required` attribute determines whether the stack vias are required for the specified base_pin. • When set to `0` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `0`, which ignores this `base_pin` for `stack_via_rule` checking. If the parameter is set to `0` only for a `pin`, other `pin` values instantiated from the same `base_pin` will not be affected. • When set to `1` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `1` on the `base_pin` values. When set to `1` for an `pin`, it enforces the via pillar only on the specified `pin`. However, if the via pillar list is empty for an `pin`, no via pillar is used even when the required bit is `1`. If set to `1`, the software performs min via pillar insertion with dirty mode, in which the via pillars are added even if there is DRC. • When set to `2` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `2` on the `base_pin` values. If set to `2`, the software performs min via pillar insertion with clean mode, in which the via pillars are not added if there is DRC. • When set to `3` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `3` on the `base_pin` values. If set to `3`, the software performs multiple tries from large to min via pillar insertion with dirty mode, in which the via pillars are added even if there is DRC. • When set to `4` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `4` on the `base_pin` values. If set to `4`, the software performs multiple tries from large to min via pillar insertion with clean mode, in which the via pillars are not added if there is DRC. ⚠ The `2`, `3`, and `4` values of this parameter are applicable only for a `base_pin`.

*stack_via_rule**	Provides a list of via pillars for a cell `base_pin`, or a single via pillar for a `pin`. This list further provides the candidates of `pin` via pillar setting. On running:
	`set_via_pillars -base_pin BUFX1/Z -required 1 {vp_rule1 vp_rule2}`
	`set_via_pillars` automatically sets the minimal cost via pillar rule for the corresponding `pin`, which is `vp_rule1`. You can also set the other via pillar rule to the `pin` by running:
	`set_via_pillars -pin i1/i2/Y vp_rule2`
	An empty *`stack_via_rule`* will remove the via pillar from `pin`:
	`set_via_pillars -pin i1/i2/Y -required 1 “”`

Examples

- Where the cell of `inst i1/i2` and `i1/i3` are both `BUFX1`, the following commands indicate that `pin i1/i2/Z` requires a via pillar `M2M4_1X3`:
  ```
  set_via_pillars -base_pin BUFX1/Z -required 1 {M2M4_1X2 M2M4_1X3}
  set_via_pillars -pin i1/i2/Z -required 1 M2M4_1X3   #The via pillar for pin must be on the cell base_pin’s via pillar list. Otherwise, a proper ERROR should be returned.
  ```
- The following command removes the via pillar `M2M4_1X3` from `pin i1/i2/Z`:
  ```
  set_via_pillars -pin i1/i2/Z “” -required 1
  ```
- The following command sets a via pillar list on a cell pin:
  ```
  set_via_pillars -base_pin BUFX1/Z -required 1 {M2M4_1X2 M2M4_1X3}
  ```
- The following command sets a via pillar with via pillar list of the corresponding cell pin on a `pin`; the cell of inst `i1/i2` is `BUFX1`:
  ```
  set_via_pillars -pin i1/i2/Z M2M4_1X3
  ```
- The following command sets via pillar to cell pin `AN2_NOM_D16*/Z`:
  ```
  foreach cell [dbGet head.libCells.name -e AN2_NOM_D16*] {set_via_pillars -base_pin $cell/Z -required 0 {VPPERF_M2_M4_1_2_1 VPPERF_M2_M5_1_2_2_1 VPPERF_M2_M9_1_2_2_2_2_2_1} }
  ```

Related Information

`get_via_pillars`

sort_db

```
sort_db
[-help]
obj_list
[attribute_list]
[-descending]
[-dictionary]
```

Returns a sorted DPO (dual-ported object) list based on the contents of one or more attributes. It is based on `lsort` using get_db to access the various attributes.

This command sorts objects only based on the attributes that have a "simple" data type of string, float, or integer, or object. If it is an object, it is implicitly sorted by the `.name` field and treated as a string. By default, the `sort_db` command sorts any input list as string. But when the `-dictionary` parameter is specified, the numeric values are sorted numerically, like `lsort`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `sort_db` parameter. For a detailed description of the command and all of its parameters, use the man command: `man sort_db`

`obj_list`	A DPO list to be sorted. If the first element of the specified list is not a DPO, then it is treated as a root attribute name and passed directly to `get_db` to return a DPO list. So, the following two commands have identical results:
	`sort_db insts`
	`sort_db [get_db insts]`

`attribute_list`	Sorts the DPO list by the specified attribute values. The first attribute in the *`attribute_list`* has the highest sort precedence. Each attribute can be a chain identical to what `get_db` supports. A leading "." is optional, so all the following commands presenting the use of `attribute_list` specify legal arguments for an inst list:
	`sort_db insts name`
	`sort_db insts .name`
	`sort_db insts {location.x location.y}`
	`sort_db insts {.location.x .location.y}`

`-descending`	Sorts the list of objects by the descending order.
`-dictionary`	Sorts the objects using string comparison in all cases, except when the strings contain numbers. The numbers in strings are compared as integers/real.

Examples

- The following command sorts all the insts by name:
  `sort_db insts`
- The following command sorts the pins based on their slack. The pins with the same slack are further sorted by the x-location:
  `sort_db [get_db pins] {.slack .location.x}`

Related Topics

- get_db
- reset_db
- set_db

tech Category Attributes

The root attributes in the `tech` category are:

- tech_db_units
- tech_finfet_grid_direction
- tech_finfet_grid_offset
- tech_finfet_grid_pitch
- tech_inst_mask_shift_layers
- tech_mfg_grid

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `tech` category, use the following command:

```
get_db -category tech
```

tech_db_units

`tech_db_units` *int*

Default: `{}`

Specifies the database units per micron from LEF UNITS statement or OA techfile, unless `init_min_dbu_per_micron` is larger and overrides the LEF/OA value.

tech_finfet_grid_direction

`tech_finfet_grid_direction` *enum*

Default: `vertical`

Specifies an enum value indicating the direction of the finfet grid from the LEF FINFET statement or OA.

tech_finfet_grid_offset

`tech_finfet_grid_offset` *coord*

Default: `{}`

Specifies the finfet grid offset from `0` in microns from the LEF FINFET statement or OA.

tech_finfet_grid_pitch

`tech_finfet_grid_pitch` *coord*

Default: `{}`

Specifies the finfet grid pitch in microns from the LEF FINFET statement or OA.

tech_inst_mask_shift_layers

`tech_inst_mask_shift_layers` *layer**

Default: `{}`

Specifies the ordered list of layer pointers to the layers that allow instance mask shifting. This is the list of layers that show up in the DEF COMPONENTMASK statement for layers that can have mask values shifted in standard cells.

tech_mfg_grid

`tech_mfg_grid` *coord*

Default: `{}`

Specifies the manufacturing grid from LEF MANUFACTURINGGRID statement or OA techfile.

Note: This is the default value for each layer, but it can be overridden for specific layers. In some technologies, the `mfg_grid` for higher metal layers is coarser than for the lower layers.

uniquify

```
uniquify
[-help]
{[design | module] | -new_master hinsts} [-exclude modules]
[-verbose]
```

Supports regrouping of master-clone partitions inside the non-unique modules. This command uniquifies or regroups the master/clones (one master module used by two or more `hinst` clones) by creating unique modules for each `hinst` clone, as needed. In all the cases, the new module names are generated by appending the '`_number`' suffix (such as `_1` and `_2`) to the original module name for creating a unique module name.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `uniquify` parameter. For a detailed description of the command and all its parameters, use the man command: `man uniquify`
`design` | *`module`*	Regroups the master/clones within the *`design`* or *`module`* and recursively below it. This assures that all the `hinsts` inside those would have their own unique module, except for the modules specified using the `-exclude` parameter. In case multiple `hinsts` are referenced to the specified `module`, those will remain as clones. *Type:* Mandatory
`-new_master` *`hinsts`*	Splits the `hinst` clones of a master module into the clones of a new master. The new master is also regrouped, except for the modules specified using the `-exclude` parameter. It may also happen that one of such `hinst` clones is forced to have a new master (refer to the **Examples** section). In case a `hinst` is not a clone of the same module, an error message is displayed. *Type:* Mandatory
`-exclude` *`modules`*	Excludes the specified modules from unification or regrouping. Use this parameter if a specific sub-module is a partition, which will be implemented and shared only once. *Type:* Optional
`-verbose`	Reports the `hinst` clones (if any), which are regrouped. *Type:* Optional

Examples

- The following command uniquifies the whole design, where the `h1`, `h2`, and `h3` `hinsts` are clones of the `mod1` module, and the `h4` and `h5` `hinsts` (clones of the `mod2` module) are inside `mod1`:

```
uniquify top -verbose
#hinst name module name
h1 mod1 (unchanged)
h2 mod1_1
h3 mod1_2
h1/h4 mod2 (unchanged)
h1/h5 mod2_1
h2/h4 mod2_2
h2/h5 mod2_3
h3/h4 mod2_4
h3/h5 mod2_5
```

  Note that the first clone of each module is not modified. So, `h1` still uses `mod1`, and `h1` and `h4` still use `mod2`. However, these are still reported to see the `hinst` clone, which forces a new module for the other `hinst` clones.

- Using the same scenario as in the previous example, the following command uniquifes the design below `mod1`:

```
uniquify mod1 -verbose
#hinst name module name
h1/h4 mod2 (unchanged)
h1/h5 mod2_1
h2/h4 mod2 (unchanged)
h2/h5 mod2_1
h3/h4 mod2 (unchanged)
h3/h5 mod2_1
```

  Note that only the design inside `mod1` is uniquified. So, `h1`, `h2`, and `h3` are still the clones of `mod1`, and there are still master/clones grouping between `h1`, `h2`, and `h3`. For instance, all the `h4`s use `mod2`, and all the `h5`s use `mod2_1`.

- Using the same scenario as in the first example, the following command splits `h1` and `h2` into a new master. This is normally done, if `h1`, `h2`, and `h3` are partitions and need a different implementation than `h3`. It is recommended not to uniquify `mod2`, if it is a lower-level partition, which will be implemented once and shared by all `h1`, `h2`, and `h3`.

```
uniquify -new_master {h1 h2} -exclude mod2 -verbose
#hinst name module name
h1 mod1_1
h2 mod1_1
```

Note that `h3` is still using `mod1`, and the `mod2 hinsts` inside `mod1` and `mod1_1` (such as `h4` and `h5`) are still the clones of `mod2`. If there is also a `mod3` inside `mod1`, it will also be uniquified, but nothing inside `mod2` is uniquified.

- Using the same scenario as in the first example, the following command splits `h1` and `h4` into a new master. This implicitly forces `h1` to be uniquified from `h2` and `h3`.

```
uniquify -new_master {h1/h4} -verbose
#hinst name module name
h1 mod1_1 (forced by lower level change)
h1/h4 mod2_1
```

If there is a `mod4` inside `mod2_1`, it will also be uniquified, but it will not be shared with `mod2`.

write_preserves

```
write_preserves
[-help]
[-dont_touch]
[-dont_use]
[-obj_types {design hinst module inst net hnet pin hpin base_cell}]
[-out_file string]
```

Writes preserve constraints in a consumable TCL file so that the exported file can sourced.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_preserves` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_preserves`
`-dont_touch`	Writes the preserve constraints only for the non-default attributes related to the `dont_touch` object.
`-dont_use`	Writes the preserve constraints only for the non-default attributes related to the `dont_use` object.
`-obj_types {design hinst module inst net hnet pin hpin base_cell}`	
	Writes the preserve constraints only for the specific object types.
`-out_file string`	Displays the TCL file name.

4

Check Commands and Attributes

- check_ac_limits
- check_connectivity
- check_pg_shorts
- check_power_vias
- create_marker
- delete_markers
- gui_show_markers
- read_markers
- report_markers
- check_ac_limit Category Attributes

check_ac_limits

```
check_ac_limits
[-help]
```

Checks for the following types of AC current violations on signal nets:

- Root-mean-square (RMS) current limit violations (I_{rms} violations)
- Peak current limit violations (I_{peak} violations)

In addition, `check_ac_limits` can be used for average current limit (I_{avg}) analysis.

AC current limit violations are sometimes also referred as wire self-heat violations. DRC manuals have these current limits to avoid over-heating a signal-nets wire with too much AC current. To prevent wire self-heating or AC signal electromigration, signal interconnects should be analyzed for their AC current carrying capacity and measured against the AC current limits specified by foundry.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_ac_limits` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_ac_limits`

Related Commands

`check_ac_limit` Category Attributes

check_connectivity

```
check_connectivity
[-help]
[-add_new_open_markers]
[-append]
[-bbox_open_markers]
[-check_pg_ports]
[-check_wire_loops | -check_geometry_loops | -geometry_connect]
[-delete_old_open_markers]
[-die_abstract_file abstractdiefilenames]
[-divide_power_nets]
[-error integer]
[-ignore_dangling_wires]
[-ignore_floating_metal]
[-ignore_opens]
[-ignore_soft_pg_connects]
[-ignore_unconnected_pins]
[-ignore_unrouted_nets]
[-ignore_weak_connects]
[-marker_on_highest_layer]
[-nets netNames | -selected]
[-no_fill]
[-out_file filename]
[-preferred_top_layer integer]
[-type {all | special | regular}]
[-use_external_connects]
[-warning value]
```

Detects conditions such as opens, unconnected wires (geometric antennas), unconnected pins, loops, partial routing, and unrouted nets; generates violation markers in the design window; reports violations. Running this command does not have database impact unless you save the design, which also saves the violation markers.

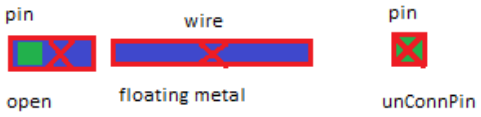
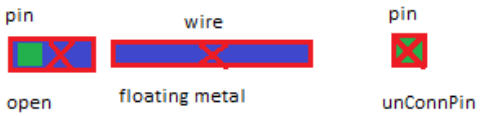
The Verify Connectivity feature now uses `set_multi_cpu_usage` and other multi-CPU commands for multi-threading.

For more information, see the Multiple-CPU Processing Commands chapter in the *Stylus Common UI Text Command Reference* document.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `check_connectivity` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man check_connectivity`
`-add_new_open_markers`	Uses an open violation marker to show unconnected objects.
`-append`	Displays incremental results in the Violation Browser. By default, violation markers are over-written with new results during each `check_connectivity` run. When you specify this parameter, violation markers are appended to the violations of the previous `check_connectivity` run.

`-bbox_open_markers`	Displays violation markers for opens as the bounding box of the island. By default, violation markers for opens are displayed as polygons that include all wires, pins, and vias that connect to the island.
`-check_geometry_loops`	Checks for loop violations of regular nets using a geometrical model. The nets do not have to overlap on the center line. When you specify this parameter, Innovus does not perform any other connectivity checks. Use this parameter if you use a third-party router that does not route using the centerline connection routing technique. In this case, the `-check_wire_loops` parameter might not detect connectivity loop violations.
`-check_pg_ports`	Verifies all PG ports connection.
`-check_wire_loops`	Checks for connectivity loops in regular wires. It also detects connectivity loops based on the end points of the center line of a regular wire segment or the center of a via.
`-delete_old_open_markers`	Removes old open violations.
`-die_abstract_file` *`abstractdiefilenames`*	Specifies the TSV abstract die files to be used to check the uBumps between the current die and the adjacent dies. Here, *`abstractdiefilenames`* specifies the names of the abstract die files created using the command `write_die_abstract`.
`-divide_power_nets`	Divides power nets into four subareas for connectivity verification. Use this parameter to decrease memory usage in 32-bit machines with limited memory. This parameter might increase or decrease the run time, depending on the design.
`-error` *`integer`*	Specifies the maximum number of errors to report. The command stops when the maximum value is reached. *Default:* `1000`
`-geometry_connect`	Checks for connectivity violations of regular wires. Uses a geometrical model instead of the center-line model. In other words, if the wires overlap at any point, they are considered to be connected--they do not have to connect at the center line. Use this parameter if you manually change the routing or use a third-party router that does not route using the centerline connection routing technique.
`-ignore_dangling_wires`	Ignores violations due to unconnected wires (also called geometrical antennas or dangling wires). A wire is called an antenna when the wire end to via center distance is less than ½ of the wire width as shown in the picture below:  In this picture, the blue rectangle is the wire and the yellow squares are the vias.
`-ignore_floating_metal`	Ignores floating metals. A wire is called floating methal, when both ends of the wire are not connected to pin shapes as shown in the picture below.  In this picture, the blue rectangle is the wire and the green squares are pin shapes.

`-ignore_opens`	Ignores open violations. A wire is said to be open when one of its ends is connected to a pin shape but the other end is not connected to any pin as shown in the picture below.  In this picture, the blue rectangle is the wire and the green squares are pin shapes. **Note:** This parameter automatically turns on the `-ignore_unconnected_pins` parameter, because open and unconnected pin verification are performed together. A pin shape that is not connected to any wire is called an unconnected pin (`unConnPin`).
`-ignore_soft_pg_connects`	Disables checking of soft Power/Ground connects. By default, `check_connectivity` checks the connectivity on masterslice layers. If `-ignore_soft_pg_connects` option is specified, connectivity on these layers is ignored.
`-ignore_unconnected_pins`	Ignores violations due to pins that are not connected to any other objects. In the picture below, the pin on the right is an unconnected pin (`unConnPin`) as it is not connected to any wire.  **Note:** This parameter is specified automatically when you specify the `-ignore_opens` parameter.
`-ignore_unrouted_nets`	Ignores nets that are not routed.
`-ignore_weak_connects`	Disables checking for routing to more than one port of the weakly connected pin ports.
`-marker_on_highest_layer`	Reports the opening on the highest layer of the pin, which has more one shape.
`-nets` *netNames*	Specifies that connectivity should be verified for specified nets only. If you specify more than one net, separate the net names with space and surround the list of net names with braces or quotes. You can use wildcards (`*` and `?`) when you specify net names. **Note:** The `-nets` and `-selected` parameters are mutually exclusive. *Default:* If you do not specify either the `-nets` or the `-selected` parameter, `check_connectivity` verifies all nets.
`-no_fill`	Specifies that metal fill also needs to be checked.
`-out_file` *filename*	Specifies the report file for connectivity violation data.
`-preferred_top_layer` *integer*	Specifies the preferred top metal layer. When this parameter is used, `check_connectivity` does not check the specified top layer and above layers.

`-selected`	Specifies that connectivity should be checked for selected nets or the nets connected to the pins of selected instances/macros. **Note:** The `-nets` and `-selected` parameters are mutually exclusive. *Default:* If you do not specify either the `-nets` or the `-selected` parameter, `check_connectivity` verifies all nets in the design.
`-type {all | regular | special}`	Specifies the type of wires to verify. `check_connectivity` is only concerned with the type of the wire and via (special or regular) and not the type of the pin. Irrespective of the `type` specified, all unconnected terminals (special and regular) are always reported unless `-ignore_unconnected_pins` is on. *Default:* `all` **Note:** To check connectivity for whole nets, you do not need to specify a type. **Note:** A pin-open violation is listed under VC DRC type "Problem(s) (ENCVFC-92): Pieces of the net are not connected together". Such a violation means the isolated open island only includes some pins' shapes that come from at least two different ports or pins. Choose one of the following: `all`: Checks all wires, including those that have been previously verified. `regular`: Reports all unconnected pins and open islands that do not include any special wire and via. Does not check a net that has no regular objects (wire, via, pin). `special`: Reports all unconnected pins and open islands that do not include any regular wire and via. Does not report isolated open islands that have any regular wire and via.
`-use_external_connects`	Implies a virtual connection for all bumps and external I/O pins of the same net. Set this parameter to override default behavior of `check_connectivity`, in which bumps and external I/O pins of the same net are not considered to be interconnected. It is useful in flip chip designs, where the power bumps are connected outside the chip.
`-warning` *value*	Specifies the maximum number of warnings to report. *Default:* `50` *Range:* `0` to `1000000`

Examples

- The following command verifies connectivity for all nets and generates a maximum of 50 error and 50 warning messages:
  ```
  check_connectivity -type all -error 50 -warning 50
  ```
- The following command reports special unconnected pins and open islands that do not include any regular wire and via
  ```
  check_connectivity -type special -ignore_dangling_wires -error 1000 -warning 50
  ```

check_pg_shorts

```
check_pg_shorts
[-help]
[-area {x1 y1 x2 y2}]
[-net net_name]
[-no_cell_blockages]
[-no_route_blockages]
[-out_file filename]
```

Checks for power and ground shorts between two geometries belonging to different nets. The command performs power and ground short check between the following:

- PG and PG nets
- PG and signal nets
- PG and other special net

This command enables you to check for power and ground shorts in the design before performing any kind of power-grid analysis. You can use this command after restoring the physical design (`read_design -physical_data *.enc` or `read_def filename`).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_pg_shorts` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_pg_shorts`.
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area to verify. For example, `check_pg_shorts -area {0 0 100 100}`. The unit of coordinates is micron.
`-net net_name`	Specifies the name of the power or ground net to be verified.
`-no_cell_blockages`	Ignores cell blockages during during power and ground short checking.
`-no_route_blockages`	Ignores routing blockages during during power and ground short checking.
`-out_file filename`	Specifies the name of the report file that contains the violation information.

Example

- The following command checks for power and ground shorts in the area specified and generates a detailed report named `shorts.rpt`.

  ```
  check_pg_shorts -area {0 0 1600 1600} -out_file shorts.rpt
  ```

check_power_vias

```
check_power_vias
[-help]
[-append]
[-area {x1 y1 x2 y2}]
[-check_fill]
[-check_wire_pin_overlap]
[-cut_area_ratio value]
[-edge_to_edge]
[-error integer]
[-exclude_region_file check_file]
[-hookup_pitch pitch_value]
[-ignore_objects {drc_fill | io_wire}]
[-ignore_partial_overlap {true | false | value}]
[-layer_range {bottomLayer topLayer}]
[-nets {netNames} | -selected]
[-non_orthogonal_check [-non_orthogonal_distance {x y}]]
[-pitch pitch_value]
[-rail_layers layer_name]
[-report file]
[-search_range {x y}]
[-shielding]
[-stacked_via]
[-stripe_layers layer_list]
[-stripe_rule value]
[-what_if_report filename]
[-width_range string]
```

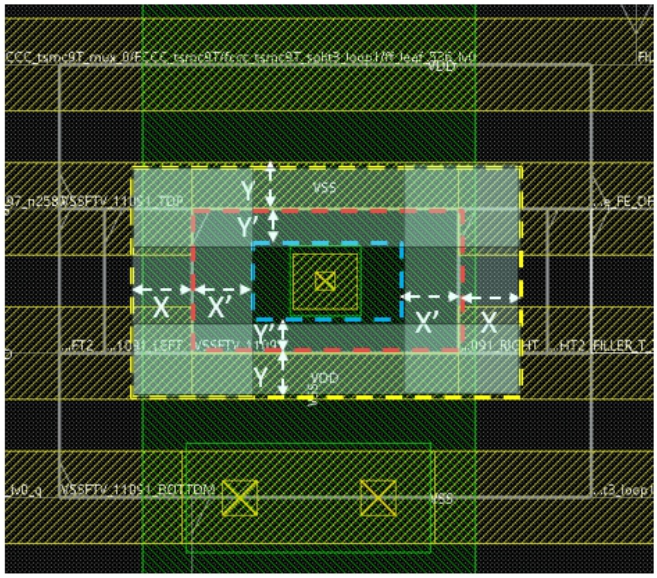
This command has a variety of power-rail overlap checks to look for missing power-grid vias. By default, it checks that orthogonal power-routes on adjacent routing layers have a via between them at every intersection. For example, there should be a via at the point where a Metal3 power stripe overlaps a Metal2 power stripe. By default, horizontal routes get checked only for overlaps with vertical routes. If the power shapes are polygon, the polygon is split into rectangles and the rectangles are used to decide the direction. Support for checking stacked vias between non-adjacent routing layers is optional. Violations are highlighted in the layout window, and a text report is generated with the location of the missing vias.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_power_vias` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_power_vias`.
`-append`	Specifies that all previous `check_power_vias` settings will be kept.
`-area x1 y1 x2 y2`	Specifies the coordinates of the area to be checked.
`-check_wire_pin_overlap`	Specifies that wire overlaps of pin shapes should also be checked. By default, only wire overlaps with wires are checked.

`-non_orthogonal_distance {x y}`	Specifies distances for finding missing via within a specified window for a non-orthogonal check, with `x` and `y` representing the distance in the horizontal and vertical directions, respectively. When the `x` or `y` distance is provided, `check_power_vias` checks if the wire is parallel with another wire. If yes, it checks: • The via center to via center spacing between parallel wires against the value *x (y)* • The via center to wire end distance against the value *x/2 (y/2)* If the actual distance is greater than the required distance in either case, it marks a violation. When the `x` or `y` distance is not provided or set to `0`, the via to via distances are not checked. **Note**: This parameter can only be specified with the `-non_orthogonal_check` parameter.
`-check_fill`	Specifies that tied-off metal fill should also be checked. Floating fill is still ignored. *Default*: Metal fill will be ignored.
`-edge_to_edge`	Checks the non-orthogonal distance from edge to edge within a layer.
`-error integer`	Specifies the maximum number of errors to report.

`-exclude_region_file check_file`	Ignores missing VIA checks within the specified `check_file` regions for a cell extension area. The `check_file` format can be a set of the following values:
	• `cell`: Specifies the cell type.
	• `ext_layer_bottom`: Specifies the extension area bottom layer.
	• `ext_layer_top`: Specifies the extension area top layer.
	• `ext_x`: Specifies the extension length in the X direction.
	• `ext_y`: Specifies the extension length in the X direction.
	• `exc_x`: Specifies the exception length in the X direction.
	• `exc_y`: Specifies the exception length in the Y direction.
	Example 1:
	The following set of `check_file` formats ignores missing VIA check within the yellow rectangle (in the output), but checks missing VIA within the blue rectangle:
	`FTV_CELL, B_M0, B_M1, X, Y, X', Y'`
	`INV_D2, M0, M1, X2, Y2, X2', Y2'`
	Following is the output:

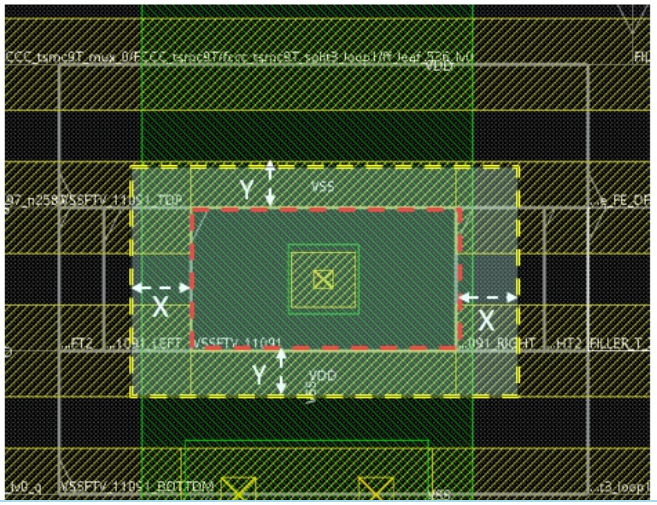


Example 2:

The following set of `check_file` formats ignores missing VIA check within the yellow rectangle (in the output):

```
FTV_CELL, B_M0, B_M1, X, Y
INV_D2, M0, M1, X2, Y2
```

Following is the output:



`-hookup_pitch` *pitch_value*	Specifies the distance between two vias, which are on the filler instance and are greater than or equal to the given value.
`[-ignore_objects {drc_fill | io_wire}]`	Skips checking the power vias of the specified wire type.

`-ignore_partial_overlap {true | false | value}`	Specifies if the partially overlapped metal layers need to be checked for missing vias. Specify one of the following parameter values: • `true`: Ignores checking the vias missing in partially overlapped metal layers. • `false`: Checks the vias missing in partially overlapped metal layers. • *`value`*: Checks the vias missing in partially overlapped metal layers on and above the specified portion of *`value`* range from 0 to 1. For example, specifying the *`value`* as 0.2 will check the vias missing in partially overlapped metal layers on or above 20% of the overlapped wires. ⚠ The instance pin and physical pin shapes are considered as wires. *Default*: `false`

`-layer_range {`*`bottomLayer topLayer`*`}`	Checks for the missing stacked vias between the bottom and top layers. It also checks for the missing vias between all the adjacent intermediate layers, but does not check for the stacked vias between the intermediate layers. While using this parameter, you can specify full layer names like `METAL1`, or abbreviations like `1`. For example, `check_power_vias -layer_range {2 5}` checks for missing stacked vias between overlapping `metal2` and `metal5`, and also missing vias for overlapping `2/3`, `3/4`, `4/5`.
	This parameter checks the missing adjacent vias if `bottomLayer` is specified, but `topLayer` is not, with considering that `topLayer` is just one layer above. To review the missing stacked via or missing adjacent via DRC marker, use the `gui_show_markers` command for filtering the DRC marker. For example:
	1. To display only the missing stacked via DRC markers, run:
	`gui_show_markers -all -filter {stacked via} -search_description -filter_mode AND`
	2. To display only the missing via DRC markers between the adjacent layers, run:
	`gui_show_markers -all -filter {adjacent via} -search_description -filter_mode AND`
	See `gui_show_markers` for more information on the command usage.

`-pitch` *`pitch_value`*	Specifies the distance between two vias, which are not on the filler instance and are greater than or equal to the given value.
`-rail_layers` *`layer_name`*	

	Specifies a single power rail layer for missing via (only the via above power rail) check. The check depends on the `-stripe_rule` option. When `-stripe_rule` is specified, the via to via distance should be less than or equal to the specified value. If a distance value is not specified with the `-stripe_rule` option, the check is ignored. See the examples below. The `-rail_layers` option is not controlled by `-layer_range`. You can specify a power rail layer other than that specified with `-layer_range` for checks against the `stripe_rule` and `search_range` rules.
`-stripe_layers` *layer_list*	Specifies the power stripe layers to be checked for a valid connection path from power rail to power stripe. The check depends on the `-search_range` option. If `-search_range` is specified, at least one valid connection path should be found from the power rail to power stripe in that search range. If a search range is not specified with the `-search_range` option, the check is ignored. You can specify the power rail layer with the `-rail_layers` option. By default, only the uppermost power rail layer is checked. See the examples below. The `-stripe_layers` option is not controlled by `-layer_range`. You can specify power stripe layers other than that specified with `-layer_range` for checks against the `search_range` rule.
`-nets {`*netNames*`}`	Specifies the names of the power nets that will be checked. A single net or a list of nets enclosed in curly braces({}) or quotes (" ") can be specified. Wildcards (* and ?) are supported. For example, when you specify `check_power_vias -nets {v*}`, all PG net names having initial letter as `v` are checked. **Note:** Currently, wildcards are supported for checking PG nets only. *Default*: All power grid nets.
`-non_orthogonal_check`	Checks the overlap area for non-orthogonal crossing wires (e.g. wires that are both in the same direction) in adjacent as well as non-adjacent layers. Checks for missing vias between two or more parallel wires. The via center-to-center distance between parallel wires should be less than or equal to the specified spacing If this parameter is not specified, horizontal wires get checked only against vertical wires. **Note**: You can use `-non_orthogonal_distance {x y}` parameter with `-non_orthogonal_check` to find missing vias within a specified window. When the `x` or `y` distance is provided, `check_power_vias` checks if the wire is parallel with another wire. If yes, it checks the via center-center spacing and via center-wire end distance between parallel wires. If the actual distance is greater than the required distance, it marks a violation.
`-report file`	Specifies the name of the output file for the report.

`-search_range {x y}`	Specifies a rectangle area with `XY`. There should be one valid connection from power rail to power stripe in the specified area. By default, the valid connection would be searched from the uppermost power rail layer to all the power stripe layers. You can use the `-rail_layers` option to specify the power rail layer, and the `-stripe_layers` option to limit the power stripe layers. With the `-rail_layers` and `-stripe_layers` options, only the specified power rail and power stripes layers are used for valid connection check. The x and y values are floats in units of microns. Refer to the **Examples** section for search range examples.
`-selected`	Checks for the missing vias between the selected PG wires and other intersecting selected/unselected PG wires. When this parameter is used with the `-check_wire_pin_overlap` parameter, it checks for the missing vias between the intersection of selected PG wires and pins of the selected instances. This parameter eliminates the requirement of running the command on the entire design, and hence improves the run time.
`-shielding`	Specifies that shielding wires should also be checked. By default, shielding wires are not checked.
`-stacked_via`	Checks for missing vias between all non-adjacent as well as adjacent layers.
`-stripe_rule` *value*	Specifies in microns the distance for power via. Only the power via above the power rail is checked. If the via to via center-to-center spacing is greater than the specified value, a violation marker is created. You can specify the power rail layer with `-rail_layers` for checking. By default, only the power via above the uppermost power rail is checked against the rule. Refer to the **Examples** section for stripe rule examples.
`-cut_area_ratio` *value*	Checks if the cut area in the metal intersection area is sufficient. `cut_area_ratio = Total cut area / Metal intersect area` If the actual ratio is less than specified ratio, `check_power_vias` marks it as violation. By default, the metal intersection area is the metal overlap area. If `setAddStripeMode -via_using_exact_crossover_size` is set to `false`, the partial overlapped metal will be extended to full width intersection area.
`-what_if_report` *filename*	
	Writes out `check_power_vias` violations data into a ECO file that can be used directly as input for whatIf rail analysis. When you specify this parameter, the missing via information generated can be used as input for what-if virtual wires.
`-width_range` *string*	

	Specifies the metal width to be considered. `check_power_vias` uses the specified width range to filter wires. Only the bottom and top wires that fall in the range are checked. The *`string`* value takes the following format: `-width_range {`*`minWidth`* `maxWidth [layerName …] [`*`minWidth maxWidth [layerName`* `…]] … }` • If *`minWidth`* is 0, all wires with width less than specified *`maxWidth`* will be checked. • If *`maxWidth`* is 0, all wires with width greater than specified *`minWidth`* will be checked. • If *`layerName`* is specified in the string, `widthRange` is enabled only for the specified layer. • If no *`layerName`* is specified in the string, `widthRange` is enabled for all non-specified layers. Refer to the **Examples** section for width range examples. **Note:** Use braces or quotation marks to enclose the string value for `widthRange`.

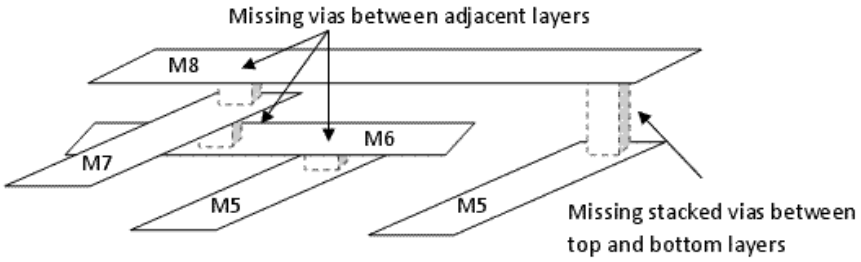
Examples

- Specifies that all vias for all powergrid nets will be checked, except for metal fill and shielding wires, and reports it to a file called `power_via.report`:

  ```
  check_power_vias -report power_via.report
  ```

- The following command checks for missing stacked vias between `METAL5` and `METAL8` layer intersections on the VSS net. In addition, it checks for missing vias between `METAL5-METAL6`, `METAL6-METAL7` and `METAL7-METAL8` layer intersections. The violations are reported in the `powerVia.rpt` file and highlighted in GUI.

  ```
  check_power_vias -layer_range {METAL5 METAL8} -nets VSS -report powerVia.rpt
  ```



- The following command checks that the overlapping power routes on adjacent layers have vias that cover at least 70% of the overlap area..

  ```
  check_power_vias -cut_area_ratio 0.7
  ```

- The following command checks that the overlapping power routes have vias that cover at least 70% of the overlap area for layers between `METAL3` and `METAL4`.

  ```
  check_power_vias -cut_area_ratio 0.7 -layer_range {METAL3 METAL4}
  ```

- The following command checks for wires overlaps for all pin shapes included, enclosed, or touched by the specified

area:

```
check_power_vias -area {100 100 1000 1000} -check_wire_pin_overlap
```

- The following command checks wires with width greater than `0.032` and less than `0.096` for all layers:

  ```
  check_power_vias -width_range {0.032 0.096}
  ```

- The following command checks wires with width in the `0.032` ~ `0.096` range for M1/M2 and width `0.038~0.114` for M3/M4. For other layers, there is no width control:

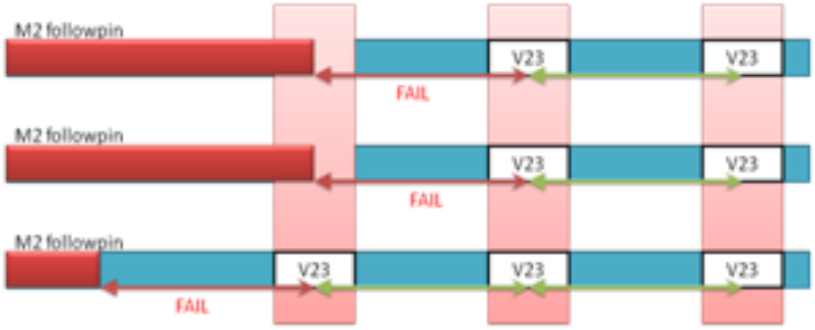
  ```
  check_power_vias -width_range {0.032 0.096 M1 M2 0.038 0.114 M3 M4}
  ```

- The following command checks wires with width in the `0.032` ~ `0.096` range for M1/M2. For other layers, it checks wires with width in the `0.038~0.114` range:

  ```
  check_power_vias -width_range {0.032 0.096 M1 M2 0.038 0.114}
  ```

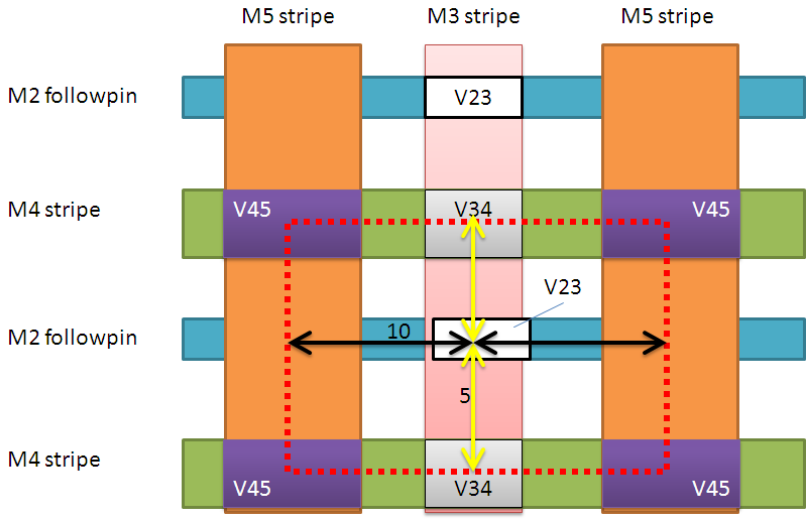
- The following command checks the via between power rail layer `M2` and `M3`. If the via's center-to-center distance is more than `17u`, the tool reports a violation and places a marker on the power rail wires.

  ```
  check_power_vias -stripe_rule 17 -rail_layers M2
  ```



- The following command checks the valid connection for `M2` power rail to stripe layer `M4` and `M5` with the search rectangle area `10*5`. In the diagram below, there is no violation as the connection is built from `M2` to `M4`. If the `V34` vias are removed, then there is a violation as the connection cannot be built from `M2` to `M4`.

  ```
  check_power_vias -search_range {10 5} -rail_layers M2 -stripe_layers {M4 M5}
  ```


M5 stripe
M3 stripe
M5 stripe
M2 followpin
V23
M4 stripe
V45
V34
V45
V23
M2 followpin
10
5
M4 stripe
V45
V34
V45

create_marker

```
create_marker
[-help]
{-bbox {x1 y1 x2 y2} | -poly {x1 y1 x2 y2...}}
[-description description]
[-layer layerName]
[-rule_map]
[-sub_type subtypeName]
[-tool toolName]
[-type typeName]
```

Creates markers for violations in the database and imports DRC markers generated by other software applications. Use this command if you want to import only a specific marker or check a specific marker that has been created by a third party tool.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_marker` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_marker`
`-bbox` *x1 y1 x2 y2*	Specifies the area of the bounding box of the marker. For example, `create_marker -bbox 100 100 100.5 100.5`. *Type:* Real
`-description` *description*	Describes the marker. For example, `create_marker -description "the user mark is non default rule spacing check"`. *Default:* `""` *Type:* String
`-layer` *layerName*	Specifies the layer. For example, `create_marker -layer METAL2`. *Type:* String
`-poly` *{x1 y1 x2 y2...}*	Specifies the area of the bounding polygon of the marker.
`-rule_map`	Specifies if the marker needs to be created through `rule_map`.
`-sub_type` *subtypeName*	Specifies the marker subtype. For example, `create_marker -sub_type user_spacing`. *Default:* `Other` *Type:* String
`-tool` *toolName*	Specifies the source software application. For example, `create_marker -tool Voltus`. *Default:* `Other` *Type:* String
`-type` *typeName*	Specifies the marker type. For example, `create_marker -type user_verify`. *Default:* `Other` *Type:* String

Example

- The following command generates markers for tool named `Calibre` with type `METAL1` and subtype `S.1`:

  ```
  create_marker -bbox { 1 2 1 2 } -tool Calibre -type METAL1 -sub_type S.1
  ```

  The violation browser displays the marker in the above example as follows:

  ```
  + Calibre
      + METAL1
          + S.1
              ...
  ```

- Suppose the cadence LPA tool has created the following marker on layer `METAL3`:

  ```
  CDNLitho;SPACING; Severity: 3; MARKER_INFO: "CD = 45.39"

  bbox = (468.248, 63.511) (468.310, 63.599)
  ```

  The following command creates a corresponding marker in the design:

  ```
  create_marker -bbox {468.248 63.511 468.310 63.599} -layer M3 /
               -type CDNLitho -sub_type Spacing
  ```

delete_markers

```
delete_markers
[-help]
[-all]
[-area {x1 y1 x2 y2}]
```

Deletes markers from a specified area or the entire design area.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_markers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man delete_markers`
`-all`	Deletes markers from the entire design area.
`-area {x1 y1 x2 y2}`	Specifies a bounding box for the area from which to delete markers. For example, `delete_markers -area {0 0 100 100}`.

gui_show_markers

```
gui_show_markers
[-help]
[-ac_limit]
[-all]
[-area {x1 y1 x2 y2}]
[-connectivity]
[-density]
[-display_by_layer]
[-geometry]
[-max_error_per_type value]
[-no_false_marker]
[-overlap]
[-process_antenna]
[-search_description]
[-short]
[-filter_query LO_formula | {-filter filterString -filter_mode {AND | OR | NOT}}]
```

Displays a list of violations in the. The software generates markers for the violations after you use one or more verification commands.

Connectivity or AC Limit Violations

Overwrites connectivity and AC limit violation markers if the `check_connectivity` or `check_ac_limits` commands are run more than once during a Voltus session. These commands are net-based, not area-based, so the browser does not make incremental updates for them. As a result of this behavior, the software does not keep the information from the first verification run.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_show_markers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_show_markers`
`-ac_limit`	Displays AC limit violations.
`-all`	Displays all the violations that are present in the design.
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area whose violations to display. For example, `gui_show_markers -area {0 0 100 100}`.
`-connectivity`	Displays connectivity violations.
`-density`	Displays metal density violations.
`-display_by_layer`	Displays the list of violations by the layer instead of route type.

`-filter` *filterString*	Specifies a text string in the violation message. Violation Browser searches the message of each violation and displays only the violations whose messages match the search conditions. To specify a list of strings, separate each string with a space. To specify a literal string with space in it, enclose the string in double quotation marks. For example, you can specify strings such as the following: • `METAL5` • `(600, 500)` • `METAL3 (700, 400)` • `"Pin of Net"`
`-filter_mode {AND | OR | NOT}`	Specifies the condition for searching multiple strings in the violation browser. You cannot filter complex expressions because the `AND`, `OR`, and `NOT` parameters are mutually exclusive. • The `AND` and `OR` conditions work on single strings or multiple strings. • The `NOT` condition works only on a single string.

`-filter_query` *LO_formula*	Specifies a complex logical expression for filtering violations. This option cannot be used along with `-filter` or `-filter_mode`. You can specify the AND, OR, and NOT conditions in the search expression. Use:
	• `&&` between strings in the expression to specify the AND condition.
	• `||` between strings in the expression to specify the OR condition.
	• ! before a string to specify the NOT condition. Note that ! is an one's complement operator. So for example, `!{object(net1) && layer(m2)}` is not allowed.
	The `-filter_query` parameter supports searches in the object, layer, and description areas. The area can be specified using the `object`, `layer`, and `desc` keywords. For example:
	`object(net1) || layer(m2) && desc(xxx)`
	You can also use the wildcards * and ? in the search expression. For example:
	`!object(clk*) && layer(m?)`
	To specify a literal string with space in it, enclose the string in double quotation marks. For example:
	`!object(clk*) && desc("prl spacing")`

`-geometry`	Displays geometry violations.
`-max_error_per_type` *value*	Specifies a limit for the number of violations parsed for each violation type. For example, if you set `gui_show_markers -max_error_per_type 500`, the Violation Browser parses and displays only 500 markers per error type. Use this parameter if you are loading a large DRC file with millions of violations to prevent the database from becoming slow while loading the file and to limit the number of violations you review in the browser. By default, the Violation Browser displays **all** markers.

`-no_false_marker`	Prevents the software from displaying false violations.
`-overlap`	Displays overlaps in the design.
`-process_antenna`	Displays process antenna violations.
`-search_description`	Searches for text in the violation message and the violation description.
`-short`	Displays short violations.

Examples

The following command displays all violations whose marker title contains `METAL3`:

```
gui_show_markers -all -filter METAL3
```

The following command displays geometry violations whose marker title contains `METAL3` or `tdsp`:

```
gui_show_markers -geometry -filter {METAL3 tdsp} -filter_mode OR
```

The following command displays geometry violations whose marker title or marker description field contains `Min` and `0.24` and `METAL3`:

```
gui_show_markers -geometry -filter {Min 0.24 METAL3} -filter_mode AND -search_description
```

The following command displays violations occuring on Metal4 or Metal5 layers but excludes short violations:

```
gui_show_markers -filter_query {!Short && (m4 || m5)}
```

read_markers

```
read_markers
[-help]
in_file fileName
[-orient {r0 r90 ...}]
[-rule_map_file mapName]
-type {Assura | Calibre | PVS | Hercules | ICV | cdn_litho | CDNCMP | inShapeLitho | CalibreLitho | DRV |
ClockTran | CLP | VerifyPower}
[-x_offset value]
[-y_offset value]
```

Loads a violation marker file or a hotspot interface format (HIF) file in one of the following formats and creates markers that the software can interpret. Use this command to load third-party markers into the software:

- Assura™
- Calibre
- PVS
- Hercules
- ICV
- Cadence or Calibre HIF
- Chemical and Mechanical Polishing (CMP) HIF
- inShapeLitho
- Calibre litho hotspot
- NanoRoute litho
- Transition violation report
- Low power violation reports

After you load the file, you can view the markers in the Violation Browser. By cross-probing between Violation Browser and the design area, you can analyze the violations and take steps to resolve them. The Auto Query feature displays the rule names for the markers.

Parameters

`-help`	Prints a brief description that includes type and default information for each `read_markers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_markers`
`in_file fileName`	Specifies the marker file to load. Assura files must have a `.err` extension. Calibre files must have a `.db` extension.
`-orient {r0 r90 ...}`	Supports advanced orientation transforms for loading rotated DRC violation markers before loading. This helps in overlaying the design to the top of the layout.

`-rule_map_file` *mapName*	Specifies the rule map file for a PVS or Calibre Litho report. The rule map file provides the layer/error type information.
	`read_markers` saves this information to the database. When the markers are created, the layer, error type, and object information can be identified on the GUI in the Violation Browser. NR could fix the converted drc markers (width, spacing, enclosure, area, minstep, etc). The rule map file should be a txt file with 3 columns:
	Rule_name error_type layer_name
	Below is an example:
	`M1.W_4.5 width metal1`
	`M1.A_0.048 minhole metal1`
	`M1.M_1 minstep metal1`

`-type {Assura | Calibre | PVS | Hercules | ICV | cdn_litho | CDNCMP | inShapeLitho | CalibreLitho | DRV | ClockTran | CLP | VerifyPower}`	
	Specifies the file type. The following file types contain violation markers: • Assura - For Assura violations • Calibre - For Calibre violations • PVS - For PVS violations • Hercules - For Hercules violations • ICV - For ICValidator report violations • CDNCMP - For Cadence CMP Predictor (CCP) violations • DRV - For transition violations (reportTranViolation) • ClockTran, CLP, and VerifyPower - For low power violations The following file types contain hotspot markers, which are used to identify areas susceptible to lithography problems: • cdn_litho (LPA as well as NanoRoute litho HIF) • inShapeLitho • CalibreLitho
`-x_offset` *value*	Specifies the X offset. When you specify the X and Y offsets, Violation Browser displays the violations at the correct coordinates after accounting for the offsets.
`-y_offset` *value*	Specifies the Y offset, which Violation Browser takes into account to displays violations at the correct coordinates.

Examples

- The following command loads a CDN Litho marker file named `LPA.hif`:

  ```
  read_markers -type cdn_litho -x_offset 0.05 -y_offset 0.05 in_file LPA.hif
  ```

- The following command loads a NanoRoute litho repair file named `NR.hif`:

  ```
  read_markers -type cdn_litho in_file NR.hif
  ```

report_markers

```
report_markers
[-help]
[-all]
[-ac_limit]
[-connectivity]
[-density]
[-geometry]
[-max_error_per_type integer]
[-no_false_marker]
[-overlap]
[-process_antenna]
[-short]
[-area {x1 y1 x2 y2}]
[-report filename]
[-filter filterString]
[-filter_mode {AND | OR | NOT}]
[-search_description]
```

Reports violations flagged by the verification commands. The report file and violation browser list the violations in the same order.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_markers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_markers`
`-ac_limit`	Reports AC limit violations.
`-all`	Reports all violations.
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area for which to report violations. For example, `report_markers -area {0 0 100 100}`.
`-connectivity`	Reports connectivity violations.
`-density`	Reports metal density violations.

`-filter` *`filterString`*	Specifies a text string in the violation message (in the bottom portion of the Violation Browser). When you specify a text string, the Violation Browser searches the message of each violation and displays only the violations whose messages match the search conditions. To specify a list of strings, separate each string with a space. To specify a literal string with space in it, enclose the string in double quotation marks. For example, you can specify strings such as the following: • `METAL5` • `(600, 500)` • `METAL3 (700, 400)` • `"Pin of Net"`
`-filter_mode {AND | OR | NOT}`	
	Specifies the condition for searching multiple strings in the violation browser. You cannot filter complex expressions because the AND, OR, and NOT parameters are mutually exclusive. • The `AND` and `OR` conditions work on single strings or multiple strings. • The `NOT` condition works only on a single string.
`-geometry`	Reports geometry violations.
`-max_error_per_type` *`integer`*	
	Specifies the error marker number limit for each type of error. Specifies a limit for the number of violations parsed for each violation type. For example, if you set `report_markers -max_error_per_type 500`, the Violation Browser parses and displays only 500 markers per error type in the report. Use this parameter if you are loading a large DRC file with millions of violations to prevent the database from becoming slow while loading the file and to limit the number of violations you review in the browser. By default, the Violation Browser displays **all** markers in the report.
`-no_false_marker`	Prohibits the software from displaying false violations.
`-overlap`	Reports overlap violations.
`-process_antenna`	Reports process antenna violations.
`-report` *`filename`*	Specifies the report file for the violation information. *Type:* String
`-search_description`	Searches the description field for each marker in addition to the table entries.
`-short`	Reports short violations.

Examples

The following command reports all violations whose marker title contains `METAL3`:

```
report_markers -all -filter METAL3
```

The following command reports geometry violations whose marker title contains `METAL3` or `tdsp`:

```
report_markers -geometry -filter {METAL3 tdsp} -filter_mode OR
```

The following command reports geometry violations whose marker title or marker description field contains `Min` and `0.24` and `METAL3`:

```
report_markers -geometry -filter {Min 0.24 METAL3} -filter_mode AND -search_description
```

check_ac_limit Category Attributes

The root attributes in the `check_ac_limit` category are:

- check_ac_limit_additional_individual_violation_report
- check_ac_limit_avg_recovery
- check_ac_limit_check_thermal_aware_em
- check_ac_limit_current_file
- check_ac_limit_current_scale_factor
- check_ac_limit_current_scale_table
- check_ac_limit_default_freq_for_unconstrained_nets
- check_ac_limit_delta_temperature
- check_ac_limit_delta_temperature_layer_list
- check_ac_limit_detailed
- check_ac_limit_effort_level
- check_ac_limit_em_cdf_percentage
- check_ac_limit_em_limit_scale_factor
- check_ac_limit_em_limit_scale_table
- check_ac_limit_em_res_width
- check_ac_limit_em_temperature
- check_ac_limit_em_temperature_layer_list
- check_ac_limit_em_threshold
- check_ac_limit_enable_seb
- check_ac_limit_env_temperature
- check_ac_limit_extraction_tech_file
- check_ac_limit_force_hold_view
- check_ac_limit_handle_pin_obs_via
- check_ac_limit_ict_em_models
- check_ac_limit_ircx_models
- check_ac_limit_lifetime
- check_ac_limit_lifetime_rating_factor_file
- check_ac_limit_max_error
- check_ac_limit_method
- check_ac_limit_min_peak_duty_ratio
- check_ac_limit_min_peak_freq

- check_ac_limit_net_file
- check_ac_limit_nets
- check_ac_limit_out_file
- check_ac_limit_peak_td_method
- check_ac_limit_read_detail_delta_temperature_file
- check_ac_limit_report_db
- check_ac_limit_report_unannotated_shapes
- check_ac_limit_seb_lifetime
- check_ac_limit_seb_table
- check_ac_limit_seb_temperature
- check_ac_limit_selected
- check_ac_limit_simulation_net
- check_ac_limit_simulation_net_file
- check_ac_limit_skip_category_mode
- check_ac_limit_skip_net
- check_ac_limit_skip_net_file
- check_ac_limit_toggle
- check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path
- check_ac_limit_use_db_freq
- check_ac_limit_use_qrc_tech
- check_ac_limit_use_rms_delta_t
- check_ac_limit_view

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `check_ac_limit` category, use the following command:

```
get_db -category check_ac_limit
```

check_ac_limit_additional_individual_violation_report

```
check_ac_limit_additional_individual_violation_report {true | false}
```

Default: `false`

Data_type: bool, read/write

Specifies to generate separate violation reports for each current waveform calculation method (`rms`, `peak`, or `avg`). If this

attribute is not specified, a single violation report file is generated using the `check_ac_limit_method {RMS AVG peak}` attribute. This attribute gives you the flexibility of generating either a single collated report or individual reports for each analysis method.

Example:

```
set_db  check_ac_limit_method {rms peak avg} \
check_ac_limit_out_file context1.rpt \
...
check_ac_limit_additional_individual_violation_report true
```

This command will generate 3 different reports, namely `context1.rpt_avg`, `context1.rpt_peak`, and `context1.rpt_rms`. If there are no EM violations for a method, the tool will not generate the corresponding separate report. In this example, if there are violations for `avg` and `rms` but none for `peak`, then it will generate only two reports, `context1.rpt_avg` and `context1.rpt_rms`.

check_ac_limit_avg_recovery

```
check_ac_limit_avg_recovery em_recover
```

Default: `1.0`

Data_type: double

Specifies the recovery factor used for calculating I_{avg} current density with recovery. By default, the recovery factor (per layer) is read from the QRC techfile. If you specify `em_recover`, it overrides the QRC tech `em_recover` factor for all layers used in I_{avg} limits. The minimum value of this attribute is `-1` and the maximum is `1`.

check_ac_limit_check_thermal_aware_em

```
check_ac_limit_check_thermal_aware_em {true | false}
```

Default: `false`

Specifies to use the thermal map for EM checks. The thermal map file is specified using the `set_rail_analysis_config -read_thermal_map` parameter. The Rail Analysis engine uses the temperature specified in the thermal map file for EM analysis. EM limit has a strong temperature dependency. For a chip that has substantial temperature variation across the chip, the Rail Analysis engine can use the temperature specified in the thermal map file to perform accurate EM analysis.

check_ac_limit_current_file

```
check_ac_limit_current_file filename
```

Default: `""`

Data_type: string, read/write

Specifies the current of a macro's output pins for EM calculation, in case the macro does not have the respective timing library. When the macro output pin current is set by the current file, the value calculated (if any) from AAE is overwritten.

check_ac_limit_current_scale_factor

```
check_ac_limit_current_scale_factor {{avg value} {rms value} {peak value}}
```

Default: `""`

Data_type: string, read/write

Scales the signal EM current by the respective factor for avg, rms, or peak current specified by `{avg value} {rms value} {peak value}`.

The value specified for this attribute must be > 0 and <= 10.

check_ac_limit_current_scale_table

```
check_ac_limit_current_scale_table current_scale_table_file
```

Default: `""`

Data_type: string, read/write

Specifies the name of the scale table file used for layer-based scaling of the signal EM current. The scale table contains the scale factor for avg, rms, or peak current for each layer.

check_ac_limit_default_freq_for_unconstrained_nets

```
check_ac_limit_default_freq_for_unconstrained_nets freq_in_Hz
```

Default: `1e+06`

Data_type: double

Specifies the frequency for EM calculation when a net has no defined frequency or has a defined frequency of `0` Hz in the design.

check_ac_limit_delta_temperature

```
check_ac_limit_delta_temperature temp
```

Default: `5.0`

Specifies the maximum temperature rise permitted in units of Celsius. This value is normally used in the QRC tech file for the RMS current limit equation. For example:

```
em_jmax_ac_rms EQU sqrt(12.66 * delta_T * (w - 0.003)^2 * (w - 0.003 + 0.264) / (w - 0.003 + 0.0443))
```

check_ac_limit_delta_temperature_layer_list

```
check_ac_limit_delta_temperature_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}
```

Default: `""`

Data_type: string, read/write

Specifies the delta temperature for specific LEF layers during signal EM RMS current limit analysis.

check_ac_limit_detailed

```
check_ac_limit_detailed
```

Default: `false`

Data_type: bool, read/write

Generates a detailed report containing information for all signal nets, including those without violations.

check_ac_limit_effort_level

```
check_ac_limit_effort_level {low | medium | high}
```

Default: `low`

Data_type: enum {low medium high}, read/write

Specifies the method used for current calculation of nets in the Signal ElectroMigration analysis flow. The two methods are:

- **non-simulation mode** - use the capacitance ratio to distribute the current down the stream from the driver to the receiver.
- **simulation mode** - simulates resistance and capacitance to get the true current on each segment. This mode is more accurate but requires a longer run time.

This attribute has the following three arguments:

- `low`: enables the non-simulation mode for all nets where the current distribution is based on the capacitance ratio.
- `medium`: enables the simulation mode for the nets with physical loops, and the non-simulation mode for the non-loop nets.
- `high`: enables the simulation mode for all nets.

The `medium` and `high` levels require a VTS-AA license.

Note: For the simulation mode with effort level `medium` or `high`:

- If you use `extract_rc`, run `set_db extract_rc_qrc_cmd_type partial extract_rc_qrc_cmd_file` *`qrc.user.cmd`*, where *`qrc.user.cmd`* must include the following CCL commands:

  ```
  extraction_setup -analysis em -cluster_vias_in_em true
  output_db -type spef -subtype "EXTENDED"
  ```

  Refer to the example below:

  ```
  output_db \
   -type spef -busbit_delimiter "[]" -escape_special_character false \
   -hierarchy_delimiter "/" -output_incomplete_nets true \
   -output_unrouted_nets true -pin_delimiter ":" -subtype "EXTENDED" \
   -disable_subnodes false -add_explicit_vias true \
   -include_parasitic_res_width true -include_parasitic_res_width_drawn true
  ```

- If you run standalone Quantus to generate the required SPEF file in the extended format separately, then instead of `extract_rc`, use the following command for signal EM analysis:

  ```
  read_spef -keep_star_node_locations -extended spef_file
  ```

check_ac_limit_em_cdf_percentage

`check_ac_limit_em_cdf_percentage` *value*

Default: `1e-9`
Data_type: double

Calculates the Cumulative Distribution Factor (CDF) derating factor based on the EM rules specified in the ICT EM file, given by the `check_ac_limit_ict_em_models` attribute. This attribute is used to calculate the fail rate and reliability of a design. The value of the CDF percentage should be between 0 and 100.

When the CDF related rule exits in the ICT EM file, the software uses the default value `1e-09` for `check_ac_limit_em_cdf_percentage` if it is not specified.

check_ac_limit_em_limit_scale_factor

`check_ac_limit_em_limit_scale_factor {{avg` *value*`} {rms` *value*`} {peak` *value*`}}`

Default: `""`

Data_type: string, read/write

Scales the calculated signal EM limits by the respective factor for avg, rms or peak current limit specified by `{avg` *value*`} {rms` *value*`} {peak` *value*`}`. The software compares the current through the net against the scaled EM limit to report violations in signal EM.

The value specified for this attribute must be > 0 and <= 10.

check_ac_limit_em_limit_scale_table

`check_ac_limit_em_limit_scale_table` *em_limit_scale_table_file*

Default: `""`

Data_type: string, read/write

Specifies the name of the scale table file used for layer-based scaling of the signal EM limit. The scale table contains the scale factor for avg, rms, or peak current for each layer.

check_ac_limit_em_res_width

`check_ac_limit_em_res_width {drawn silicon}`

Default: `drawn`

Data_type: enum {drawn silicon}, read/write

Specifies width used to get EM limit.

check_ac_limit_em_temperature

`check_ac_limit_em_temperature` *value*

Default: `-1.0`

Data_type: double

Specifies the temperature expected for the device and is used to look up the temperature scaling factor. By default, no scaling is done. The *value* is a float, in units of degrees Celsius.

check_ac_limit_em_temperature_layer_list

```
check_ac_limit_em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}
```

Default: ""

Data_type: string, read/write

Sets a different temperature for specific LEF layers during signal EM AVG current limit analysis.

check_ac_limit_em_threshold

```
check_ac_limit_em_threshold value
```

Default: 1.0

Data_type: double, read/write

Specifies to report violations in signal EM above the threshold value. The value is the ratio of signal current (avg/peak/rms) to the respective EM limit.

check_ac_limit_enable_seb

```
check_ac_limit_enable_seb {true | false}
```

Default: false

Data_type: bool, read/write

Specifies to enable the Statistical ElectroMigration Budgeting (SEB) analysis.

check_ac_limit_env_temperature

```
check_ac_limit_env_temperature value
```

Default: -1.0

Data_type: double

Specifies the ambient temperature used to calculate S_{DC} for SEB analysis.

check_ac_limit_extraction_tech_file

```
check_ac_limit_extraction_tech_file filename
```

Default: ""

Data_type: string, read/write

Specifies an extraction technology file for the Signal ElectroMigration analysis flow. If this attribute is not specified, then by default the extraction technology file will be taken from the analysis view (current RC corner).

check_ac_limit_force_hold_view

`check_ac_limit_force_hold_view`

Default: `false`

Data_type: bool, read/write

Forces `set_signal_em_analysis_mode` to use the hold view for current calculation even during setup fixing.

check_ac_limit_handle_pin_obs_via

`check_ac_limit_handle_pin_obs_via`

Default: `false`

Data_type: bool, read/write

Specifies to consider the vias defined in the `OBS` (obstructions/blockages) section of LEF in the Signal ElectroMigration flow. By default, the software does not consider via in obstruction.

This attribute does not honor the via obstructions with "`SPACING 0`".

check_ac_limit_ict_em_models

`check_ac_limit_ict_em_models` *`file`*

Default: `""`

Data_type: string, read/write

Specifies the ICT-EM file which only contains the EM rules and excludes the RC extraction data in ICT. The software will use the EM rules in the specified ICT-EM file to calculate EM limits for signal net EM analysis even if there are embedded EM rules in the Quantus QRC tech file.

check_ac_limit_ircx_models

`check_ac_limit_ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}`

Default: `""`

Data_type: string

Specifies an encrypted or unencrypted RC/EM technology (.ircx) file. The foundry-specific IRCX file, which contains the process information for extraction and EM modeling, can be used when the ICT-EM file is not available for electromigration analysis.

You can either specify two IRCX files, one file containing the RC process data (`RC.ircx`) and the other file containing EM rules (`EM.ircx`), or a single IRCX file that includes both the RC and EM sections (`RC.ircx`). A single IRCX file is used for the mature nodes, and separate RC and EM IRCX files are used for the advanced process nodes.

check_ac_limit_lifetime

`check_ac_limit_lifetime` *`value`*

Default: `87600.0`

Data_type: double

Specifies the total hours of operation expected for the device and is used to look up the lifetime scaling factor. *value* is used to find the scaling factor by interpolating from a piece-wise-linear table in the LEF or the QRC tech file (em_jmax_ac model with jmax_lifetime table). If there is no scaling factor table, no scaling is done. *value* is a float, in units of hours.

check_ac_limit_lifetime_rating_factor_file

```
check_ac_limit_lifetime_rating_factor_file filename
```

Default: ""

Data_type: string, optional

Specifies a lifetime rating factor file that contains jmax_life values for each layer. These values are defined based on temperature and lifetime pairs. When you provide this file, EM analysis will automatically select the appropriate jmax_life factor for each layer according to the specific temperature-lifetime combinations defined within the file. The lifetime and EM temperature can be specified using the check_ac_limit_lifetime and check_ac_limit_em_temperature attributes.

Following is a snippet of the lifetime rating factor file:

```
*Rating factor of M0/V0/M1/V1/M2/V3/M3...
TEMPERATURE        90        95       100     105     110 ...
LifeTime
--------------------------------------------------------------------------------
0.5                2.578    2.242    2.024   1.774   1.547
1                  2.520    2.191    1.979   1.734   1.512
2                  2.430    2.113    1.908   1.672   1.458
3                  2.382    2.072    1.871   1.639   1.430
...
--------------------------------------------------------------------------------
```

check_ac_limit_max_error

```
check_ac_limit_max_error value
```

Default: 10000

Data_type: int, read/write

Controls the number of errors to be included in the signal EM violation report file. If you do not specify this attribute, the top 10000 signal nets with violation will be included in the violation report.

The check_ac_limit_max_error attribute can be used to control the size of the violation report file.

check_ac_limit_method

```
check_ac_limit_method {rms peak avg}
```

Default: ""

Data_type: string, read/write

Specifies the current waveform calculation method. You can specify one or more of the following options:

- `avg`: Checks I_{avg} limits and reports any average current limit violations in the design.
- `peak`: Checks I_{peak} limits and reports any peak current limit violations in the design.
- `rms`: Checks I_{rms} limits and reports any RMS current limit violations in the design.

check_ac_limit_min_peak_duty_ratio

```
check_ac_limit_min_peak_duty_ratio duty_ratio
```

Default: `-1.0`

Data_type: double

Sets the minimum duty ratio value for I_{peak} calculation. If the calculated duty ratio of a net is lower than the specified value, the specified value will be used to calculate I_{peak}. The minimum value for *duty_ratio* is `0.0`and the maximum is `1.0`.

check_ac_limit_min_peak_freq

```
check_ac_limit_min_peak_freq freq_in_Hz
```

Default: `1e+06`

Data_type: double

Ignores I_{peak} current limit calculation for signal nets that have an effective frequency below the specified value (in Hertz).

check_ac_limit_net_file

```
check_ac_limit_net_file list_file
```

Default: `""`

Data_type: string, read/write

Specifies the net from file to check the signalEM analysis. The format is one net name per line, with comments using # as the first character. For example:

```
#list of nets to skip
net_a
net_b
net_c
```

check_ac_limit_nets

```
check_ac_limit_nets {netNames}
```

Default: `""`

Data_type: string, read/write

Specifies whether to check named or selected nets. If you name only one net, you do not need the braces around the net name. If you specify more than one net, you must separate the net names with a space.

check_ac_limit_out_file

`check_ac_limit_out_file` *`filename`*

Default: `""`

Data_type: string, read/write

Specifies the report file for the violation data. The report file includes name of the net, I_{rms}, interpolated `ACCURRENTDENSITY` limit (I_{limit}), layer, width, X and Y location, rise and fall time, effective frequency for the net (F_{eff}), Cnet and Vdd for each net.

check_ac_limit_peak_td_method

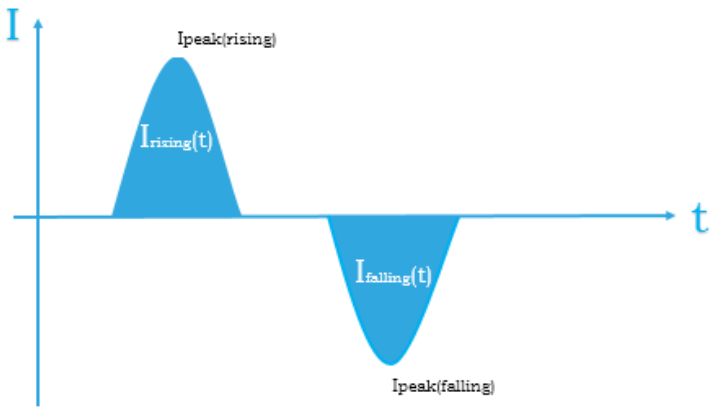
`check_ac_limit_peak_td_method {effective_width_from_integration effective_half_peak_width max_equivalent_dc_peak sum_half_peak_width}`

Default: `effective_width_from_integration`

Data_type: enum {effective_width_from_integration effective_half_peak_width max_equivalent_dc_peak sum_half_peak_width}, read/write

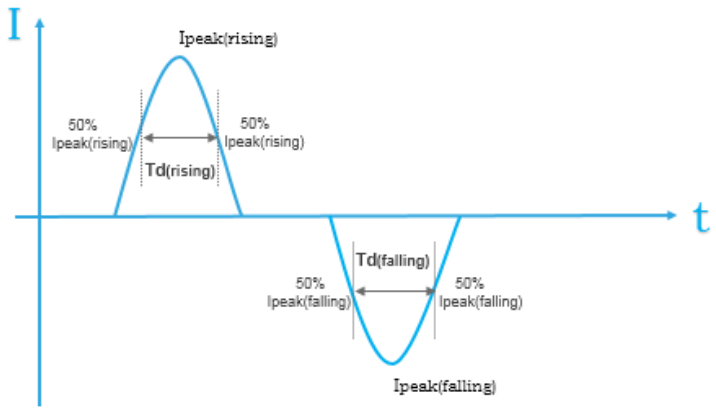
Specifies the Td calculation method for Ipeak check. The legal values are depicted below.

`effective_width_from_integration`



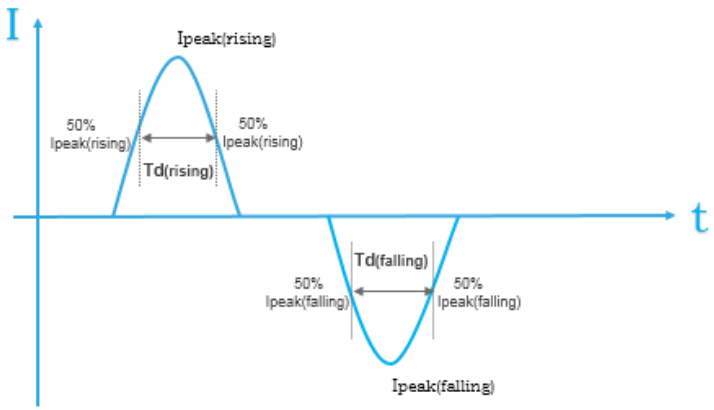
- $\mathrm{Td} = \frac{\int_0^{Tsw} |I(t)|dt}{\mathrm{Ipeak_measured}}$
  - Tsw = Period of one switching cycle (rising + falling)
  - Ipeak_measured = abs.max(Ipeak(rising), Ipeak(ralling))

`effective_half_peak_width`



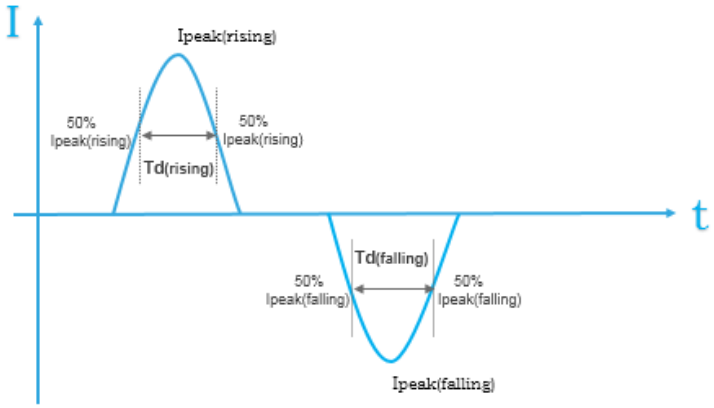
- When Ipeak(rising) >= Ipeak(falling)
  $Td = Td(rising) + (\frac{Ipeak(falling)}{Ipeak(rising)})^2 * Td(falling)$
- When Ipeak(rising) < Ipeak(falling)
  $Td = Td(falling) + (\frac{Ipeak(rising)}{Ipeak(falling)})^2 * Td(rising)$

max_equivalent_dc_peak



- $R(rising) = Td(rising)/(T/2)$
- $R(falling) = Td(falling)/(T/2)$
- Choose $max(Ipeak_{(rising)}*sqrt(R_{(rising)}), Ipeak_{(falling)}*sqrt(R_{(falling)}))$ to compare against Ipeak_dc_limit
- $Ipeak_ac_limit = Ipeak_dc_limit/sqrt(R)$
- When $Ipeak_{(rising)}*sqrt(R_{(rising)}) >= Ipeak_{(falling)}*sqrt(R_{(falling)})$
  $Td = 2*Td(rising)$
- When $Ipeak_{(rising)}*sqrt(R_{(rising)}) < Ipeak_{(falling)}*sqrt(R_{(falling)})$
  $Td = 2*Td(falling)$

sum_half_peak_width



- $Td = Td_{(rising)} + Td_{(falling)}$

check_ac_limit_read_detail_delta_temperature_file

`check_ac_limit_read_detail_delta_temperature_file` *`string`*

Default: `""`

Data_type: string, read/write

Specifies the detail delta temperature file for thermal aware checking.

check_ac_limit_report_db

`check_ac_limit_report_db {true | false}`

Default: `false`

Data_type: bool, read/write

Enables writing a database file to store data for GUI display.

check_ac_limit_report_unannotated_shapes

`check_ac_limit_report_unannotated_shapes {true | false}`

Default: `false`

Data_type: bool, read/write

Provides visibility into partial net coverage within SPEF files. When this attribute is set to `true`, the `<report_name>.rpt.unannotated_shapes` is generated. This report identifies and lists nets that contain shapes without resistance annotations, or unanalyzed/ partially analyzed segments. The feature helps to detect incomplete parasitic extraction coverage and improve accuracy during validation and signoff workflows. The reporting of partial net coverage works only when `check_ac_limit_effort_level` is set to `high`.

Use Model:

```
set_db check_ac_limit_effort_level high check_ac_limit_report_unannotated_shapes true
```

Report Format:

```
NET: <net_name>          <- Specifies the name of the analyzed simulation nets with unannotated shapes
LAYER (x1 y1 x2 y2)      <- Layer name and shape coordinates
```

Sample Report:

```
NET: Net_A
LAYER (x1          y1          x2          y2)

M2    (30.2850    12.0800    37.6150    12.2200)
M2    (30.2000    12.0650    30.2850    12.2350)
```

This attribute applies only when running in DP mode; it is not supported in XP mode.

check_ac_limit_seb_lifetime

`check_ac_limit_seb_lifetime` *value*

Default: `43800.0`

Data_type: double

Specifies the life time of product in hours.

check_ac_limit_seb_table

`check_ac_limit_seb_table` *table_filename*

Default: `""`

Data_type: string, optional

Data_type: string, read/write

Specifies the file name of the Failures in Time (FIT) table. This file is provided by the foundry. The file has the following columns:

- Layer
- MTF
- Sigma
- Activate energy
- Current density exponent

check_ac_limit_seb_temperature

`check_ac_limit_seb_temperature` *temperature*

Default: `-273.0`

Data_type: double

Specifies the temperature used to update the median time to failure (MTF) with respect to the ambient temperature.

- If this temperature is specified, it would be used globally for all the resistors.

- If this temperature is not specified, the `seb_temperature` may take the following possible values depending on the specific circumstances:
  - If detailed delta temperature file is not used for SEB, the `seb_temperature` always takes the same value as that used for EM analysis.
  - If detailed delta temperature file is used for SEB, the `seb_temperature` is calculated by the following equation:
    ```
    seb_temperature  = env_temperature + delta_T
    ```
    Where `delta_T` is the maximum temperature rise, including joule heat and coupling heat from FEOL, in the resistor's bounding box with zero extension.

check_ac_limit_selected

```
check_ac_limit_selected filename
```

Default: `false`

Data_type: bool, read/write

Analyzes the net specified by selecting a net in the main GUI window.

check_ac_limit_simulation_net

```
check_ac_limit_simulation_net list_of_nets
```

Default: `""`

Data_type: string, read/write

Forces the specified nets to use the simulation mode.

check_ac_limit_simulation_net_file

```
check_ac_limit_simulation_net_file filename
```

Default: `""`

Data_type: string, read/write

Forces the nets in the specified file to use the simulation mode.

check_ac_limit_skip_category_mode

```
check_ac_limit_skip_category_mode {0 | 1 | 2}
```

Default: `1`

Data_type: int, read/write

Specifies to skip nets that belong to categories mapping to the specified mode. This parameter allows you to specify three modes, namely 0, 1 (default), and 2.

The following table lists the common categories for all modes, and the categories specific to each mode:

Common Categories	0 Skip Category Mode	1 Skip Category Mode	2 Skip Category Mode
• Non top scope nets • Internal nets on boundary path • P/G nets • Constant nets • User-defined skipped nets • Nets below effective frequency threshold • Physically broken nets • Nets without loading capacitance annotation • Nets with SPEF back-annotation failures during simulation • Nets failed in analysis	• Slave nets by Verilog Assign • Nets (non-constant) without frequency annotation • Nets without physical shape for output pin • Nets without wire shape connection to output pin • Nets with zero pin current	• Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver	• Nets without pins • Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver

For example, if you set `set_signal_em_analysis_mode -skip_category_mode to 1`, the report file lists all categories listed in the *Common Categories* and *1 Skip Category Mode* columns.

Following is a snippet of the report file for mode **1**:

```
Total Nets in Design:               33451
Total Selected Nets in Design:          4
Total Analyzed Nets:                    0
Total Skipped Nets:                     4
  Non top scope nets:                                                0
  Internal nets on boundary path:                                    0
  Equivalent nets by Verilog Assign:                                 0
  P/G nets:                                                          0
  Constant nets:                                                     2
  User-defined skipped nets:                                         0
  Nets below effective frequency threshold:                          0
  Nets without driver:                                               1
  Nets without receiver:                                             1
  Physically broken nets:                                            0
  Nets without loading capacitance annotation:                       0
  Nets with SPEF back-annotation failures during simulation:         0
  Nets failed in analysis:                                           0
```

check_ac_limit_skip_net

`check_ac_limit_skip_net` *`net_name_list`*

Default: `""`

Data_type: string, read/write

Specifies the net to be skipped in the signalEM analysis.

Refer to the example below:
`{net_a net_b net_c}`

check_ac_limit_skip_net_file

`check_ac_limit_skip_net_file` *`list_file`*

Default: `""`

Data_type: string, read/write

Specifies the net file to be skipped in the signalEM analysis.

The format is one net name per line, with comments using # as the first char. For example:

```
#list of nets to skip
net_a
net_b
net_c
```

check_ac_limit_toggle

```
check_ac_limit_toggle value
```

Default: `0.0`

Data_type: double, read/write

Specifies the switching factor for signal nets. A value of `1.0` means that if a flip-flop is clocked by a 20 Mhz clock, it changes state on 100% of the clock cycles. Therefore, the signal has 20 million transitions, which is half the number of transitions of the input clock. As a result, the final switching frequency of the signal net is 10 Mhz.

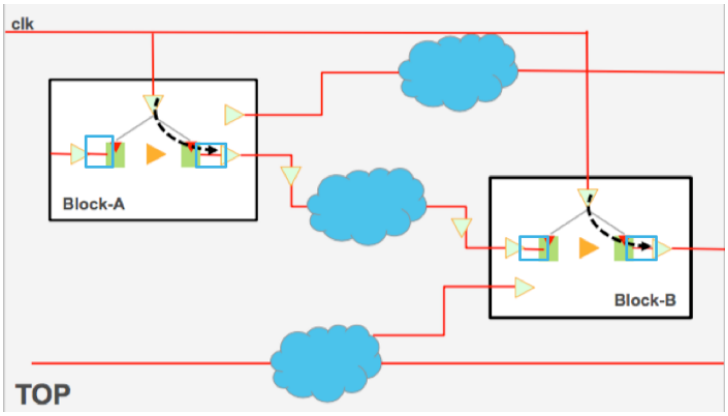
Value Range: `0.0` to `1.0`

check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path

```
check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path {true | false}
```

Default: `false`

Ignores the internal nets from the first instance to the first flop on the boundary path for the signal EM checking. The following figure represents that when this parameter is used, the internal nets (highlighted in the blue rectangles) are ignored. However, all the nets highlighted through the red lines are reported in the TOP scope (`create_top_scope`) flow:



check_ac_limit_use_db_freq

```
check_ac_limit_use_db_freq
```

Default: `false`

Data_type: bool, read/write

Uses the database frequency value as the effective frequency per net.

check_ac_limit_use_qrc_tech

```
check_ac_limit_use_qrc_tech
```

Default: `true`

Data_type: bool

Forces I_{rms} checks to use the QRC tech file instead of the technology LEF file. If either I_{peak} or I_{avg} is also checked, all checks will use the QRC tech file, including I_{rms}. This attribute is enabled by default.

⚠ The `-method avg` and `-method peak` analysis will only support AC current density rules from QRC tech file. If either `avg` or `peak` is also specified, then all checks will use the QRC tech file equations, including RMS.

check_ac_limit_use_rms_delta_t

```
check_ac_limit_use_rms_delta_t {true | false}
```

Default: `false`

Data_type: bool, read/write

When this parameter is set to `true`, the `delta_T` used to calculate `seb_temperature` would be based on the RMS delta T (joule heat only).

If this parameter is set to `false`, the `delta_T` used to calculate `seb_temperature` would be based on the sum of self-heat delta T and RMS delta T.

check_ac_limit_view

```
check_ac_limit_view viewName
```

Default: ""

Data_type: string

Specifies the name of the view.

5

Flowkit Commands and Attributes

- check_flow
- create_flow
- create_flow_step
- define_feature
- delete_flow_config
- edit_flow
- end_flow
- flow Category Attributes
- flowtool Category Attributes
- get_feature
- get_flow_config
- init_flow
- is_feature
- is_flow
- read_flow
- report_flow
- reset_flow
- run_flow
- schedule_flow
- set_feature
- set_flow_config
- set_flowkit_read_db_args

- set_flowkit_write_db_args
- write_flow
- write_flow_template

check_flow

```
check_flow
[-help]
[-fail string]
[-flow string]
[-from string]
[-proc string]
[-step string]
[-to string]
```

Creates a single flow_step object and fills-in the required, and some optional attributes. Further modifications of the step can be performed using the `set_db` command. If you try to define a step with the same name as an existing step, it results in error. To redefine a flow step, you must use the `set_db` command to modify the attributes, or delete and recreate it.

Note: Unlike Tcl procedures, flow steps do not take arguments and refer to attributes instead. The use of attributes requires that the data on which a step relies, is stored in the database and may be examined later.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man check_flow`.
`-fail` *string*	Reports a problem. this is not used for analysis. It can only be used from Tcl called as part of static checking. Calling from outside static checking results in error.
`-flow` *string*	Runs the specified flow. the `from`/`to` values are based on the flow steps. *Default:* `flow_step_order`
`-from` *string*	Specifies the name of the step to start the flow from. By default, the flow starts from the first step that has not been run, or the first step if all steps have been run previously. **Note:** This must be a step in the flow.
`-proc` *string*	Checks the specified procedure instead of analysing a flow step.
`-step` *string*	Runs just the specified step. This takes priority over `from`/`to` options.
`-to` *string*	Specifies the name of the step to run to (inclusive; in other words, the named step is executed). By default, runs to the end of the last step in the flow. Must be a step in the flow, later than the `-from` step (if any).

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

create_flow

```
create_flow
[-help]
[steps]
-name name
[-owner string]
[-skip_metric]
[-tool tool]
[-tool_options options]
```

Creates a new, named flow object. `run_flow` then allows the execution of a named flow (in addition to its existing functionality).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_flow`.
`[steps]`	Specifies a list of flow step names in the order they must be executed. **Note:** This is an optional parameter. You can either provide `[steps]` or use the `-tool` parameter.
`-name` *name*	The name of the flow object to create.
`-owner` *string*	Specifies the owner of the flow.
`-skip_metric`	Restricts the tool from capturing the flow's metric snapshot.
`-tool` *tool*	Specifies the name of the tool with which to run this flow. This option allows you to create flows for other tools than the one currently running. If not specified, default is the current tool.
`-tool_options` *options*	Specifies options for use when launching this flow.

See also:

Flowkit chapter of the *Innovus Stylus Common UI Migration Guide* document.

create_flow_step

```
create_flow_step
[-help]
body
[-categories string]
[-check checkTcl]
[-description string]
[-exclude_time_metric]
-name name
[-owner string]
[-write_db]
[-skip_metric]
```

Creates a single `flow_step` object and fills-in the required, and some optional attributes. Further modifications of the step can be performed using the `set_db` command. If you try to define a step with the same name as an existing step, it results in error. To redefine a flow step, you must use the `set_db` command to modify the attributes, or delete and recreate it.

Note: Unlike Tcl procedures, flow steps do not take arguments and refer to attributes instead. The use of attributes requires that the data on which a step relies, is stored in the database and may be examined later.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_flow_step` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_flow_step`.
`body`	Specifies a block of Tcl that is run as the main action of the step.
`-categories` *`string`*	Defines the metric categories to calculate the flow step.
`-check` *`checkTcl`*	Specifies a block of Tcl to run step-specific static checks.
`-description` *`string`*	Describes the purpose for the current flow step.
`-exclude_time_metric`	Ensures that the CPU and wall time of the corresponding step is not included in any parent steps.
`-name` *`name`*	Specifies the name of the Tcl of the step. This can consist of alphanumeric characters and underscore only, and may not start with a digit. The following names have special meanings and may not be used for steps: • next • end
`-owner` *`string`*	Specifies the owner of the flow.
`-write_db`	Automatically saves a database when the flow step is complete.
`-skip_metric`	Skips generating metrics when the step is complete. By default, every step saves the design and runs metrics when completed.

See also:

Flowkit chapter of the *Innovus Stylus Common UI Migration Guide* document.

define_feature

```
define_feature
[-help]
feature
[-default string]
[-description string]
[-group string]
[-obj string]
[-possible_values string]
[-required]
[-type {bool string enum}]
```

Defines the feature for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `define_feature` parameter. For a detailed description of the command and all of its parameters, use the man command: `man define_feature`
feature	Specifies the name of the feature.

`-default` *string*	Specifies the default value of the feature.
`-description` *string*	Specifies the detailed description of the flow feature. If it matches multiple features, then these features are listed along with a short description for each. If it matches exactly one feature, a full description of this feature is printed. Matching no features results in an error.
`-group` *string*	Outputs the exclusive group that contains features.
`-obj` *string*	Specifies the flow or flow_step.
`-possible_values` *string*	Specifies the list of values that are valid for the features.

`-required`	Specifies that the mentioned feature is mandatory.
`-type {bool string enum}`	Specifies the type of feature value.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

delete_flow_config

```
delete_flow_config
[-help]
key ...
```

Deletes the configuration value at the specified key, allowing the user to change the type of a `setup.yaml` key by first deleting it.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_flow_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_flow_config`
`key ...`	Specifies the key to delete the configuration value.

See also:

`set_flow_config`

edit_flow

```
edit_flow
[-help]
[-after string]
[-append string]
[-before string]
[-enabled_feature string]
[-owner string]
[-prepend string]
[-reset]
[-unique]
```

Allows you to insert flow steps into an existing flow without redefining the flow itself. This command is very useful for flow customization.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `edit_flow` parameter.
`-after` *string*	Adds the specified `flow` and/or `flow_step` objects after the flow supporting a full canonical path to a flow step object.
`-append` *string*	Appends the specified `flow` and/or `flow_step` objects before or after the flow.
`-before` *string*	Adds the specified `flow` and/or `flow_step` objects before the flow supporting a full canonical path to a flow step object.
`-enabled_feature` *string*	Executes the `edit_flow` command only if the specified user-defined features are enabled.
`-owner` *string*	Specifies the owner of the flow being edited.
`-prepend` *string*	Adds the specified `flow` and/or `flow_step` objects before or after the flow.
`-reset`	Resets the customizations for the specified `flow` and/or `flow_step` objects before or after the flow.
`-unique`	Prevents the addition of the flow and/or `flow_step` objects, if those are already provided. This avoids duplication of the flow defined in the YAML file and the `edit_flow` statements scripted in the `tech_flow_steps` file. **Note:** This parameter prevents the specified `flow_step` from getting added to the same flow twice, irrespective of the values provided to the `-after` or `-before` parameters. For example, if `edit_flow -append bob -unique` is called twice to insert different `flow_steps` in the floorplan flow, it will add `bob` only once.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

Examples

- Following is an example of customizing the reference flow process using the `flow_config.template` file:

```
create_flow_step -name write_sdf {
write_sdf [get_db flow_report_directory]/[get_db local_flow].sdf
}
edit_flow -after flow_step:innovus_report_late_timing -append flow_step:write_sdf
```

- Following is an example of inserting an additional `step_2` step after the `step_1` step:

```
set file_dir [file dirname [ dict get [ info frame [ info frame ] ] file ] ]
source ${file_dir}/scripts/run_flow.tcl
create_flow_step -name step_2 {
}
edit_flow -append step_2 -after step_1
```

- Following is an example of adding flow customization to the `flow_config.tcl` file:

```
create_flow_step -name output_files {
set design_name [get_db current_design .name]
write_netlist [get_db flow_report_directory]/${design_name}.v
write_def [get_db flow_report_directory]/${design_name}.def
}
edit_flow -append flow_step:output_files -after flow_step:block_finish
```

end_flow

```
end_flow
[-help]
flow_or_step
```

Ends the specified flow or step. This command accepts a single string argument, which is a flow or step name. The specified string must match a flow or step in the path for the current step. Else, a Tcl error occurs. Flowkit/flowtool then skips all the contiguous steps to match the given flow or step name in the hierarchy.

Note: Calling `end_flow` outside of a `flow_step` inside `run_flow` (for example, `init_flow plugin`, `flow_header_tcl` or as a plain command) will produce an error.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `end_flow` parameter. For a detailed description of the command and all its parameters, use the man command: `man end_flow`
`flow_or_step`	Specifies the flow or step to be ended and skipped.

See also:

Flowkit chapter of the *Innovus Stylus Common UI Migration Guide* document.

flow Category Attributes

The root attributes in the `flow` category are:

- flow_branch
- flow_caller_data
- flow_db_directory

- flow_error_errorinfo
- flow_error_message
- flow_error_write_db
- flow_exclude_time_for_init_flow
- flow_exit_when_done
- flow_features
- flow_feature_values
- flow_footer_tcl
- flow_header_tcl
- flow_hier_path
- flow_history
- flow_log_directory
- flow_log_prefix_generator
- flow_mail_on_error
- flow_mail_to
- flow_metrics_file
- flow_metrics_snapshot_parent_uuid
- flow_metrics_snapshot_uuid
- flow_overwrite_db
- flow_plugin_names
- flow_plugin_steps
- flow_post_db_overwrite
- flow_remark
- flow_report_directory
- flow_run_tag
- flow_schedule
- flow_starting_db

- flow_startup_directory
- flow_status_file
- flow_step_canonical_current
- flow_step_check_tcl
- flow_step_current
- flow_step_last
- flow_step_last_msg
- flow_step_last_status
- flow_step_next
- flow_summary_tcl
- flow_template_feature_definition
- flow_template_tools
- flow_template_type
- flow_template_version
- flow_top
- flow_user_templates
- flow_verbose
- flow_working_directory
- flow_write_db_snapshot
- flow_write_db_snapshot_exclude_time
- flow_write_db_snapshot
- flow_write_db_snapshot_exclude_time
- flow_yamllint_exec

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `flow` category, use the following command:

```
get_db -category flow
```

flow_branch

`flow_branch` *`string`*

Default: ""

Specifies the branch being run for a hierarchical flow.

flow_caller_data

`flow_caller_data` *`string`*

Default: *""*

Specifies the data used by the caller of the tool for identifying the flow.

flow_db_directory

`flow_db_directory` *`string`*

Default: `flow.database`

Specifies the flow directory where results databases are stored.

flow_error_errorinfo

`flow_error_errorinfo` *`string`*

Default: ""

Specifies the TCL error stack in the error database.

flow_error_message

`flow_error_message` *`string`*

Default: ""

Specifies the TCL error message in the error database.

flow_error_write_db

```
flow_error_write_db {true | false}
```

Default: `true`

Runs the database when an error occurs in a flow step.

flow_exclude_time_for_init_flow

```
flow_exclude_time_for_init_flow {true | false}
```

Default: `false`

Marks all those snapshots as `exclude_time_metric`, which were created after running the `init_flow` command. This marking adds user steps to the `init_flow` internal plugin locations.

flow_exit_when_done

```
flow_exit_when_done {true | false}
```

Default: `false`

Exits the flow after running the final step.

flow_features

```
flow_features string
```

Default: ""

Specifies the global feature definitions.

flow_feature_values

```
flow_feature_values string
```

Default: ""

Specifies the global feature settings.

flow_footer_tcl

`flow_footer_tcl` *`string`*

Default: ""

Specifies the TCL script to run when a flow ends.

flow_header_tcl

`flow_header_tcl` *`string`*

Default: ""

Specifies the TCL script to run when a flow starts.

flow_hier_path

`flow_hier_path` *`string`*

Default: ""

Displays the path of the current flow within the flow hierarchy.

flow_history

`flow_history` *`string`*

Default: ""

Displays the complete flow run history.

flow_log_directory

`flow_log_directory` *`string`*

Default: `flow.log`

Specifies the flow directory where log files are stored.

flow_log_prefix_generator

`flow_log_prefix_generator` *`string`*

Default: {

```
set logPrefix ""
# Work out the subset of steps we are running
set startPath [split $start_step "."]

set endPath [split $end_step "."]
for {set i 0} {$i < [llength $startPath]} {incr i} {
if {[lindex $startPath $i]
ne [lindex $endPath $i]} {
if {$subflow_start_step && $subflow_end_step} {
set logPrefix [join [lrange
$startPath 0 [expr $i - 1]] "."]
} else {
set logPrefix $start_step
if {$i < [llength
$endPath]} {
set logPrefix "${logPrefix}-[join [lrange $endPath $i end] "."]"
}
}

break
}
}
# Add the branch name
if {$branch ne {}} {
if {$logPrefix eq {}} {
set
logPrefix $branch
} else {
set logPrefix "$branch.$logPrefix"
}
}
# Add the top-level flow
to the prefix
if {$logPrefix eq {}} {
set logPrefix [string range $flow 5 end]
} else {
set logPrefix
"[string range $flow 5 end].$logPrefix"
}
# Fall back to tool name
if {$logPrefix eq {}} {
set
```

```
logPrefix $tool
}
# Add the flow log directory
set logPrefix [file join $flow_log_directory $logPrefix]

return $logPrefix
}
```

Specifies the TCL script to create log filenames in flowtool.

flow_mail_on_error

```
flow_mail_on_error {true | false}
```

Default: `false`

Sends an email to 'flow_mail_to', if an error is detected.

flow_mail_to

```
flow_mail_to string
```

Default: ""

Specifies the email address where results are sent.

flow_metrics_file

```
flow_metrics_file string
```

Default: ""

Specifies the flow metrics file for reporting results.

flow_metrics_snapshot_parent_uuid

```
flow_metrics_snapshot_parent_uuid string
```

Default: ""

Specifies the snapshot uuid to which the results from this flow will be appended.

flow_metrics_snapshot_uuid

`flow_metrics_snapshot_uuid` *`string`*

Default: ""

Specifies the snapshot uuid of the most recent flow step executed.

flow_overwrite_db

`flow_overwrite_db {true | false}`

Default: `false`

Enables overwriting databases while saving.

flow_plugin_names

`flow_plugin_names` *`string`*

Default: *`list`*

Displays the names of all the internal plugin points.

flow_plugin_steps

`flow_plugin_steps` *`string`*

Default: *`""`*

Displays the list of plugin steps that have been added through `edit_flow`.

flow_post_db_overwrite

`flow_post_db_overwrite` *`string`*

Default: *`""`*

When this attribute is set within `skip_db flow_step`, you can identify the name and type of database from the current to future flows as `flow_starting_db`.

flow_remark

`flow_remark` *`string`*

Default: *`""`*

Displays the remarks from the last loaded `flow.yaml` file.

flow_report_directory

`flow_report_directory` *`string`*

Default: `flow.report`

Specifies the flow directory where reports are written.

flow_run_tag

`flow_run_tag` *`string`*

Default: ""

Specifies the tags for the particular flow run.

flow_schedule

`flow_schedule` *`string`*

Default: ""

Specifies the set of flows to execute after the current flow completes.

flow_starting_db

`flow_starting_db` *`string`*

Default: ""

Specifies the starting database for the flow run.

flow_startup_directory

`flow_startup_directory` *`string`*

Default: `/home/user_id`

Specifies the directory from where the software was run.

flow_status_file

`flow_status_file string`

Default: ""

Specifies the file for flow status.

flow_step_canonical_current

`flow_step_canonical_current string`

Default: ""

Specifies the currently running canonical flow step.

flow_step_check_tcl

`flow_step_check_tcl string`

Default: ""

Specifies the procedure to check the steps.

flow_step_current

`flow_step_current string`

Default: ""

Specifies the currently running flow step.

flow_step_last

`flow_step_last string`

Default: ""

Specifies the last flow step to be completed.

flow_step_last_msg

`flow_step_last_msg` *`string`*

Default: ""

Specifies the message provided for the last step that was run.

flow_step_last_status

`flow_step_last_status` *`string`*

Default: `not_run`

Specifies the status for the last flow step run.

flow_step_next

`flow_step_next` *`string`*

Default: ""

Specifies the next step to run in the flow.

flow_summary_tcl

`flow_summary_tcl` *`string`*

Default: ""

Specifies the TCL script to run during run_flow summary.

flow_template_feature_definition

`flow_template_feature_definition` *`string`*

Default: ""

Features list and status for the current template.

flow_template_tools

`flow_template_tools` *`string`*

Default: ""

Specifies the list of tools included in the flow templates being run.

flow_template_type

`flow_template_type` *`string`*

Default: ""

Specifies the type of template being run.

flow_template_version

`flow_template_version` *`string`*

Default: ""

Specifies the version of the template being run.

flow_top

`flow_top` *`string`*

Default: ""

Specifies the flow to be executed by default.

flow_user_templates

`flow_user_templates` *`string`*

Default: ""

Specifies the flow user template definitions.

flow_verbose

`flow_verbose {true | false}`

Default: `true`

Enables printing the run information in the log file.

flow_working_directory

`flow_working_directory` *`string`*

Default: "."

Specifies the flow directory where the flow is being run.

flow_write_db_snapshot

`flow_write_db_snapshot {true | false}`

Default: `true`

Data_type: bool, read/write

Creates a snapshot for `write_db` on flow steps, which saves a database.

flow_write_db_snapshot_exclude_time

`flow_write_db_snapshot_exclude_time {true | false}`

Default: `true`

Data_type: bool, read/write

Marks all snapshots created for the `write_db` command on the flow steps, which save a database as `exclude_time_metric`.

flow_write_db_snapshot

`flow_write_db_snapshot {true | false}`

Default: `true`

Data_type: bool, read/write

Creates a snapshot for `write_db` on flow steps, which saves a database.

flow_write_db_snapshot_exclude_time

`flow_write_db_snapshot_exclude_time {true | false}`

Default: `true`

Data_type: bool, read/write

Marks all snapshots created for the `write_db` command on the flow steps, which save a database as `exclude_time_metric`.

flow_yamllint_exec

`flow_yamllint_exec` *`string`*

Default: `yamllint`

Specifies where `yamllint` can be found to check the `yaml` syntax.

flowtool Category Attributes

The root attributes in the `flowtool` category are:

- flowtool_exit_timeout
- flowtool_extra_arguments
- flowtool_metrics_qor_html
- flowtool_metrics_qor_excel
- flowtool_metrics_qor_text
- flowtool_metrics_qor_vivid
- flowtool_predict_full_names
- flowtool_summary_tcl

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `flowtool` category, use the following command:

```
get_db -category flowtool
```

flowtool_exit_timeout

`flowtool_exit_timeout` *int*

Default: `30`

Specifies the maximum time (in seconds) for which flowtool will wait to complete post exiting from the tool.

flowtool_extra_arguments

`flowtool_extra_arguments` *string*

Default: `""`

Specifies the extra arguments passed to flowtool when running in the tool mode..

flowtool_metrics_qor_html

`flowtool_metrics_qor_html` *string*

Default: ""

Specifies the metrics file in the HTML format to write after tool's exit.

flowtool_metrics_qor_excel

`flowtool_metrics_qor_excel` *string*

Default: ""

Specifies the metrics file in the Excel format to write after tool's exit.

flowtool_metrics_qor_text

`flowtool_metrics_qor_text` *string*

Default: ""

Specifies the metrics file in the txt format to write after tool's exit.

flowtool_metrics_qor_vivid

`flowtool_metrics_qor_vivid` *`string`*

Default: ""

Specifies the metrics file to write after the tool's exit.

flowtool_predict_full_names

`flowtool_predict_full_names {true | false}`

Default: `false`

Shows the `skip_metric` flow name in prediction.

flowtool_summary_tcl

`flowtool_summary_tcl` *`string`*

Default: ""

Specifies the TCL script to run during run_flow summary.

get_feature

```
get_feature
[-help]
[feature]
[-obj string]
```

Returns the feature for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_feature` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_feature`
`feature`	Specifies the name of the feature.
`-obj` *`string`*	Specifies the `flow` or `flow_step` to be returned.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

get_flow_config

```
get_flow_config
[-help]
key ...
[-quiet]
```

Returns the feature configuration for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_flow_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_flow_config`
`key ...`	Specifies the key to get configuration value.
`-quiet`	Suppresses errors in case the key does not exist.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

init_flow

```
init_flow
[-help]
```

The command processes flow information provided by flowtool to setup and run a provided flow sequence. It outputs the affected flow root attributes, such as:

- init_flow summary
- Flow script: /grid/tfo/vol102/tiqa/flow/demo_dtmf/scripts/run_flow.tcl
- Flow: flow:block
- From: prects.block_start
- To: prects.block_finish
- Working directory: /grid/tfo/vol102/tiqa/flow/demo_dtmf
- Starting database: /grid/tfo/vol102/tiqa/flow/demo_dtmf/dbs/block.fplan.block_finish.enc
- Run tag:
- Branch name:
- Metrics file: /grid/tfo/vol102/tiqa/flow/demo_dtmf/flow.metrics
- Status file: /grid/tfo/vol102/tiqa/flow/demo_dtmf/flow.status

Parameters

`-help`	Outputs a brief description that includes type and default information for each `init_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man init_flow`

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

is_feature

```
is_feature
[-help]
[feature_name]
[-obj string]
```

Tests if the specified flow feature is defined. The format of this command is similar to that of the `get_feature` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `is_feature` parameter. For a detailed description of the command and all of its parameters, use the man command: `man is_feature`
`feature_name`	Specifies the name of the feature to be tested.
`-obj` *`string`*	Specifies if the `flow` or `flow_step` will be returned.

See also:

- `get_feature`
- Flowkit chapter of the *Stylus Common UI Migration Guide* document.

is_flow

```
is_flow
[-help]
[-after string]
[-before string]
[-hierarchical]
[-inside string]
[-quiet]
```

Allows you to query during the execution of a flow for further action.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `is_flow` parameter.
`-after string`	Specifies the current execution after the `flow` or `flow_step` object.
`-before string`	Specifies the current execution before the `flow` or `flow_step` object.
`-hierarchical`	Displays all parent flows.
`-inside string`	Specifies the current execution within the `flow` object.
`-quiet`	Ensures that errors are not reported for missing `flow` or `flow_step` objects.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

read_flow

```
read_flow
[-help]
filename
[-no_check]
```

Loads the content for the current flow and debugs the flow in the tool. The command accepts single argument as the name of the files that describe the `flow` and `flow_step` objects, as shown below:

```
read_flow scripts/run_flow.tcl
```

```
read_flow scripts/flow.yaml
```

This command should not be used for YAML or TCL files, in general. For example, the following command should not be used, in which the `setup.yaml` file is used by the flow, but does not contain flow information:

```
read_flow scripts/setup.yaml
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_flow` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_flow`.
`filename`	Specifies the flow script to read.
`-no_check`	Skips running the `yamllint` and `check_tcl` flow checks.

See also:

Flowkit chapter of the *Innovus Stylus Common UI Migration Guide* document.

report_flow

```
report_flow
[-help]
```

Generates a report with the following information:

- The flow type, section list and generation date of the templates
- The flow step order
- One line for every item in flow_history , indicating the success of that item and the configuration of optional flow features, and any error message

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_flow`.

Example

```
Flow Features: low_power
   (optional): prefer_area ccopt
Flow Generated: 2013-05-14 14:40 GMT
Flow Step Order:
  init place prects cts postcts postcts_hold signoff

Step       Status     prefer_area    ccopt    Message
-------------------------------------------------------------------------
init       success    on             off
place      success    on             off
prects     success    on             off
cts        error      on             off      "Prefer_area" doesn't allow CTS.
cts        success    off            off
```

This report indicates:

- The flow was generated with the `low_power` feature enabled, and the `prefer_area` and `ccopt` features optional.
- The init, place and prects steps were run with `prefer_area` enabled, ccopt disabled, and

succeeded.

- The cts step was then run with `prefer_area` enabled, `ccopt` disabled, and it failed.
- The cts step was then run with both `prefer_area` and `ccopt` disabled, and it succeeded.

There is some overlap between `report_flow` and the summary printed at the end of `run_flow` commands. The difference is that `report_flow` reports information about the entire flow history (since the last `run_flow -reset`, if any), whereas the summary at the end of `run_flow` only includes data about the steps run by that command.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

reset_flow

```
reset_flow
[-help]
[-all]
[-edits]
[-features]
[-flows]
[-history]
[-steps]
```

Resets the content for the current flow in the software by clearing any existing flows, steps, or features. The functionality of the `-history` parameter of this command is equivalent to `run_flow -reset`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `reset_flow` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man reset_flow`
`-all`	Resets all the flows, steps, features, edits, and history set in the software. *Data_type*: bool, optional
`-edits`	Resets all the flow edits appended through the `edit_flow` command. *Data_type*: bool, optional
`-features`	Resets all the flow features and associated values. *Data_type*: bool, optional
`-flows`	Resets all the flow objects. *Data_type*: bool, optional
`-history`	Resets flow history and state of all the attributes (such as `flow_step_canonical_current`, `flow_history`, and `flow_top`). *Data_type*: bool, optional
`-steps`	Resets all the `flow_step` objects. *Data_type*: bool, optional

See also:

Flowkit chapter of the *Innovus Stylus Common UI Migration Guide* document.

run_flow

```
run_flow
[-help]
[-directory string]
[-flow string]
[-from string]
[-no_check]
[-no_summary]
[-predict {summary | tcl}]
[-reset]
[-step string]
[-to string]
[-yaml string]
```

The command runs one or more flow steps by default, in the order specified in a single flow.

If more than one flow is defined, the `run_flow` command must be run with `-flow`. However, if `-step` is used, that step does not have to be part of a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `run_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man run_flow`
`-directory` *`string`*	Specifies the working directory in which to run the flow. If a directory is not specified, the current directory is used. **Note:** Any relative paths in attributes such as *flow_report_directory* are relative to this directory.
`-flow` *`string`*	Runs the specified flow (created with the `create_flow` command). `from`/`to` options are based on the flow steps. *Default:* `flow_step_order`

`-from` *string*	Specifies the name of the step to start from accepting a full dotted canonical path to a flow step object. By default, flow starts from the step immediately after the last successfully run step (determined by examining the value of *flow_history*), or the first step if that attribute is not set.
`-no_check`	Skips the error checks prior to running. By default, the checks are run.
`-no_summary`	Skips the full summary from printing. By default, summary is printed and emailed if `flow_mail_to` is set.
`-predict {summary | tcl}`	Displays the projected future steps to run honoring the `-step`, `-flow`, `-from`, and `-to` parameters.
`-reset`	Before running, clears the flow history. After this, the flow starts from the first step if no explicit `-from` step is specified.
`-step` *string*	Runs just the specified step. This takes priority over `from`/`to` options.
`-to` *string*	Specifies the name of the step to run to (inclusive; in other words, the named step is executed) accepting a full dotted canonical path to a flow step object. By default, runs to the end of the last step in the flow. Must be a step in the flow, later than the `-from` step (if any).
`-yaml` *string*	Specifies the yaml file to source before running the flow.

Examples

- The following command prints the hierarchical list of steps to be run post the specified run:

  ```
  run_flow -from cts.block_start -to cts.schedule_report_cts -predict
  ```

  Following is the output of the above command:

  ```
  Flow steps which would run
  --------------------------
  cts.block_start
  cts.init_innovus
  cts.init_innovus.extra_settings
  cts.add_clock_spec
  cts.add_clock_tree
  cts.add_tieoffs
  cts.block_finish
  cts.schedule_report_cts
  ```

- The following command prints the list of steps as DPO names to be run post the specified run:

  ```
  run_flow -from cts.block_start -to cts.schedule_report_cts -predict -tcl
  ```

  Following is the output of the above command:

  ```
  flow_step:block_start flow_step:init_innovus flow_step:extra_settings
  flow_step:add_clock_spec flow_step:add_clock_tree flow_step:add_tieoffs
  flow_step:block_finish flow_step:schedule_report_cts
  ```

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

schedule_flow

```
schedule_flow
[-help]
[-branch string]
[-db string]
[-defer]
[-directory string]
[-flow string]
[-include_in_metrics]
[-no_db]
[-sync | -no_sync]
```

Schedules a flow to be run after the current flow. The details of scheduled flows are stored in the `flow_schedule` attribute.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `schedule_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man schedule_flow`.
`-branch` *string*	Specifies the branch for the new flow.
`-db` *string*	Specifies the starting database of the flow. If not supplied, the final database of the preceding flow (the one currently running) is used.
`-defer`	Prevents the scheduled flow from running, until requested.
`-directory` *string*	Specifies the working directory to use for the flow. By default, the current directory is used.
`-flow` *string*	Specifies the flow to be scheduled.
`-include_in_metrics`	Includes the flow in the current metric snapshot.
`-no_db`	When specified, flowtool does not use the database saved at the end of the step as the starting database for this flow.
`-no_sync`	When specified, flowtool does not wait for this flow to complete before continuing.
`-sync`	When specified, flowtool waits for all flows in the current branch to complete before starting.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

set_feature

```
set_feature
[-help]
[feature]
[-obj string]
-value string
```

Saves the feature for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_feature` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_feature`
feature	Specifies the name of the feature.
`-obj` *string*	Specifies the `flow` or `flow_step` to be saved.
`-value` *string*	Specifies the value to set the feature.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

set_flow_config

```
set_flow_config
[-help]
key
value ...
```

Saves the feature configuration for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_flow_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_flow_config`
key	Specifies the key to set configuration value.
value ...	Specifies the value to set configuration value.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

set_flowkit_read_db_args

```
set_flowkit_read_db_args
[-help]
[-dynamic_view analysis_view]
[-hold_views hold_views]
[-leakage_view analysis_view]
[-lef_files list_of_files]
[-setup_views setup_views]
[{-oa_lib_cell_view {libname cellname viewname}}] [{-no_timing}] [{-no_timing_graph}]
[{-mmmc_file fileName}]
```

Controls the run-time options used by `read_db` when a flow script is loading `flow_starting_db` specified by `flowtool` or used by `schedule_db`.

Note: This command supports all the parameters supported by `read_db`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `set_flowkit_read_db_args` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_flowkit_read_db_args`
`-dynamic_view analysis_view`	Specifies the analysis view for the dynamic power analysis.
`-hold_views hold_views`	Specifies the hold views for loading rather than the active hold views.
`-leakage_view analysis_view`	Specifies the analysis view for the leakage power analysis.

`-lef_files` *`list_of_files`*	Specifies the list of LEF files for loading. Use this option if you want to override the existing LEF files in the design being restored and load different LEF files instead. • This option is intended to allow upgrading to new versions of LEF files that are compatible with the old versions. The list of all LEF files used by this design must be provided. • If any pin locations, cell boundaries, or OBS shapes are changed for used cells, new DRC violations may occur. • If pins are added or removed from used cells, or used cells are removed, fatal errors may occur while loading the netlist. • If the removed pins and cells are unused in the design, warnings or errors may still occur, if any library-specific data was stored for those cells or pins during the design flow (such as `dont_touch` data).
`-mmmc_file` *`fileName`*	Specifies the path name to a Multi-Mode/Multi-Corner (MMMC) configuration file (for example, the same file format used by `read_mmmc`) to replace the MMMC file saved by `set_flowkit_write_db_args` (and all the corresponding .lib files, .sdc files, etc. referenced by the MMMC file).

`-no_timing`	Skips reading of the MMMC file, timing libraries and timing constraints. **Note:** • Most designs that use LEF rely on the .lib information to define the bus order for MACROs that have bus bit terminals. Therefore, using the `-no_timing` parameter can result in the connectivity to bus ports of leaf level instances being reversed. When `.lib` information is not available, the default is for bus ports to be treated as descending (for example, [7:0]). While using Verilog modules or OpenAccess abstract views to define the bus order, the connectivity will still be correct even when `-no_timing` is specified. • Timing libraries cannot be loaded in the same session after a design is read using the `-no_timing` option. You must use the `set_flowkit_read_db_args` command without the `-no_timing` parameter in a new session to load the design with timing libraries and constraints. • If you save a design after `set_flowkit_read_db_args -no_timing`, it will not save timing information unless you use `set_flowkit_write_db_args -add_ignored_timing`.
`-no_timing_graph`	Skips reading the timing graph even if it was saved to the disk by `set_flowkit_write_db_args`.
`-oa_lib_cell_view {`*`libname cellname viewname`*`}`	Specifies the library name, cell name, and view name for reading an OpenAccess DB design.
`-setup_views` *`setup_views`*	Specifies the setup views for loading rather than the setup views active during `set_flowkit_write_db_args`.

References

`read_db`

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

set_flowkit_write_db_args

```
set_flowkit_write_db_args
[-help]
[-add_ignored_timing]
[-def]
[-lib]
[-no_constraint]
[-no_fill]
[-no_pvs_fill]
[-no_wait string]
[-sdc]
[-user_path]
[-verilog]
[{-rc_extract}] [{-timing_graph}] [{-oa_lib_cell_view string}] [{-oa_view string}] [{-
lib_to_ldb}]
```

Allows you to save databases through flowkit in the Open Access format and specify the library/cell/view information. The `set_flowkit_write_db_args` command can be used to extend the options used to save a design on flow-steps, which are marked with `skip_db false`. This command needs to be placed inside the `flowstep body_tcl` of interest since it only affects that `flowstep` and resets back to default state when that step completes. The format of this command is similar to what would be used for `write_design`:

```
create_flow_step -name save {set_flowkit_write_db_args -rc -lib2ldb}
```

This saves a database named `save`, which uses the `-rc` and `-lib2ldb` options for `write_db`. Previous and subsequent flowsteps use default `write_design` options unless overridden using the `set_flowkit_write_db_args` command.

Note: This command supports all the parameters supported by `write_db`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_flowkit_write_db_args` parameter.

`-add_ignored_timing`	Copies the old timing data when loaded in physical-only mode. If you use `set_flowkit_read_db_args -no_timing` to read the DB, the timing data is not loaded. This option allows you to copy the ignored timing data from the original `read_db` location and add it back during `set_flowkit_write_db_args` like this:
	`set_flowkit_read_db_args -no_timing test1`
	`<physical editing commands ...>`
	`set_flowkit_write_db_args -add_ignored_timing test2`
	This usage model is intended to allow faster physical-only editing, like DRC fixing, where the netlist is not modified. If you change the netlist, unpredictable results may occur when reading the `test2` DB using timing data copied from `test1`.

`-def`	Saves a DEF file, in addition to the normal DB files in the DB directory. This parameter is normally not used for flat or block flows and is not used by `set_flowkit_read_db_args`. It can be useful in a hierarchical flow, where you want to use `assemble_design` to read only the netlist and the DEF files without opening up each design DB and writing out the Verilog and DEF files separately (see `-verilog`).
`-lib`	Resolves symbolic links of the timing lib files and side files. Normally external file references, such as to the LEF, .lib, SDC, files are saved using full path-name, symbolic links to the original files inside a libs sub-directory inside the DB directory. This parameter forces copies of the files rather than symbolic links in the libs sub-directory so there are no external file references. That makes the DB directory much larger, and makes it suitable for sending to a different network where the library and external data files are not available. An OA DB will still have OA library references using the normal cds.lib methodology, but any other external file references will be copied inside the cellview directory.

`-lib_to_ldb`	Converts the `.lib` timing libraries referenced in the MMMC file to the `.ldb` format and writes them in a libs sub-directory inside the output design directory. References to the original .lib files in the MMMC file are changed to the corresponding `.ldb` files inside the libs sub-directory. This makes the design directory much larger because it has all the timing library data inside it and is not recommended for normal flows. Normally, you should setup `.ldb` files in an external library directory and have the MMMC files use them directly to avoid this overhead. Using `.ldb` files will make `set_flowkit_read_db_args` faster when it reads the library data, so this option is useful for doing “throw-away” debugging experiments that require repeated sessions at the cost of a one-time conversion, and carrying the library data with the design.
`-no_constraint`	Saves the timing graph data without the SDC constraints data. This parameter requires the `-timing_graph` parameter and is useful in reducing the disk size, and speeding up `set_flowkit_read_db_args` and `set_flowkit_write_db_args`. However, the timing-graph data cannot be loaded by any other release version. Additionally, the timing constraints would be lost without the SDC files, if loaded by a different release version. **Note:** You can still load the timing-graph with the current release version, and then use `set_flowkit_write_db_args` without the `-no_constraint` parameter to regenerate the SDC files from the timing-graph and then use `set_flowkit_read_db_args` with a later release.
`-no_fill`	Prevents writing of any metal or via fill shapes inside the database. This is useful during the ECO routing, since the fill data is regenerated anyway. This way, it is not worth saving the current fill data. All the fill data that would go to DEF FILLS section (floating fill data), or the DEF SPECIALNETS section with + SHAPE FILLWIRE or FILLWIREOPC will be ignored and not written.

`-no_pvs_fill`	Prevents saving the PVS fill data file. In some PVS flows, a stream (GDS) file from PVS is saved inside the software DB directory. During `set_flowkit_read_db_args`, this file is not read, but can be optionally loaded or used later in the flow. By default, if the PVS fill file exists, `set_flowkit_write_db_args` will copy it from the `set_flowkit_read_db_args` location to the new DB directory. If ECO changes to the current design have made the PVS fill file invalid, use this parameter to stop it from being copied.
`-no_wait` *string*	Writes the design in a separate process. You can continue executing new commands simultaneously while the design is being saved. This is useful when you want to save intermediate results without waiting for the save to complete.
`-oa_lib_cell_view` *string*	Saves the database to an Open Access library cell view.
`-oa_view`	Specifies the Open Access view name.
`-rc_extract`	Writes RC extraction data, including data for incremental extraction. By default, the software does not automatically save RC extraction data to save disk space and run-time. The `-rc_extract` parameter does not save RC extraction data if the design is not extracted or if extract_rc_engine = pre_route.
`-sdc`	Saves the timing constraint file even if it is not modified.
`-timing_graph`	Saves the timing graph data.
`-user_path`	Saves external file references exactly same as the user entered it.
`-verilog`	Saves a Verilog file inside the DB directory, in addition to the normal DB files. This parameter is not normally used for flat or block flows. It can be used in a hierarchical flow, where you want to use `assemble_design` to read only the Verilog and the DEF files without opening up each design DB and writing out the Verilog and DEF files separately.

References

- write_db
- Flowkit chapter of the *Stylus Common UI Migration Guide* document.

write_flow

```
write_flow
[-help]
[-directory string]
-out_file string
[-flow_tcl]
[-flow_yaml]
[-include_state]
[-setup_yaml]
```

Generates a Tcl script that may be sourced to reproduce the current flow. The script includes commands to:

- Set the value of all top-level attributes with non-default values to their current values
- Recreate all flow step objects with their current attributes, aside from those relating to run history (specifically, `run_count` and `status`).

Sourcing this script reproduces (but not runs) the current flow in a fresh session, and forms a user readable and editable record of the flow (compared to the flow data saved in the DB, which should not be read or edited directly).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_flow` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_flow`.
`-directory` *string*	Specifies the directory is which all the files need to be written.
`-out_file` *string*	Specifies the name of the output file to generate.
`-flow_tcl`	Writes only the `flow.tcl` file in the directory.
`-flow_yaml`	Writes only the `flow.yaml` file in the directory.
`-include_state`	Writes out the current flow state. This can be used to restore the flow.
`-setup_yaml`	Writes only the `setup.yaml` file in the directory.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

write_flow_template

```
write_flow_template
[-help]
[-describe string]
[-directory string]
[-enable_feature string]
[-ip_steps string]
[-list]
[-optional_feature string]
[-tech_steps string]
[-template_file string]
[-tools]
[-type string]
[-user_steps string]
[-version string]
```

Creates a set of template TCL files to be used as the basis for a flow. This command is used at the beginning after which, you can manage the generated and modified files. When invoking the command, you will select the type of flow required (implying a subset of template scripts to include) and the arrangement of sections across generated files.

On completion, a message is given to indicate creation of the flow template in the specified directory.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_flow_template` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_flow_template`.
`-describe string`	Outputs a detailed description of available flow features for flow types matching the provided string. A features table includes names and description for each flow (refer to the Examples section). Matching no flow types results in an error.

`-directory` *string*	Specifies the name of the directory for saving the generated files. If no directory is specified, the command uses the name *flow*. If required, the tool appends "_" and an integer to the directory name to avoid overwriting an existing directory. For example, *flow_1* and *flow_2*). Default: *scripts*
`-enable_feature` *string*	Specifies a list of flow features that are unconditionally incorporated into the generated scripts.
`-ip_steps` *string*	Overlays the contents of the generated `flow.yaml` file and inserts the specified scripts under the `ip_flow_steps` flow.
`-list`	Lists the available flow templates.
`-optional_feature` string	Specifies a list of flow features that are conditionally incorporated into the generated scripts. Both the enabled and disabled code are included, with conditionals based on a new attribute.
`-tech_steps` *string*	Overlays the contents of the file(s) generated by running the `write_flow_template` command. The file(s) provided with this parameter are copied into the `scripts` directory inside a `tech` sub-folder and appear in `tech_flow_steps` for the stylus flows.
`-template_file` *string*	Specifies the flow template file.
`-tools`	Returns the list of tools whose scripts should be included in the flow.
`-type` *string*	Specifies the type of flow to be used.
`-user_steps` *string*	Overlays the contents of the generated `flow.yaml` file and inserts the specified scripts under the `user_flow_steps` flow.
`-version` *string*	Specifies the version of the flow to be used.

Examples

- Following is an example of fetching the detailed description of the available flow features for the `block` flow sequence using this parameter:

```
write_flow_template -describe block
Features for flow templates matching 'block'

Template block
--------------
Provider : tool
Max version : 1
Description : Standard block flow
This is the default template
+-------------------------+-----------------------------------------------------
---------------+
| Feature | Description |
+-------------------------+-----------------------------------------------------
---------------+
| clock_design | Run discrete clock expansion |
| dft_compressor | Add flow support for scan chains with compression insertion |
| dft_simple | Add flow support for scan chain insertion |
| express | Enable express synthesis and implementation flow |
| opt_early_cts | Implement early clock tree for use during prects optimization
|
| opt_eco | Run opt_design during eco flow |
| opt_postcts_hold_disable | Disable postcts hold fixing |
| opt_postcts_split | Run postcts opt_design for setup and hold as separate
steps |
| opt_postroute_split | Run postroute opt_design for setup and hold as separate
steps | | opt_signoff_area | Run signoff base area optimization as part
implementation flow|
| opt_signoff_drv | Run signoff base drv optimization as part implementation
flow | enabled |
| opt_signoff_dynamic | Run signoff base dynamic optimization as part
implementation flow | disabled |
| opt_signoff_hold | Run signoff base hold optimization as part implementation
flow | disabled |
| opt_signoff_leakage | Run signoff base leakage optimization as part
implementation flow | enabled |
| opt_signoff_setup | Run signoff base setup optimization as part implementation
flow |
| report_inline | Run report generation as part of parent flow versus
schedule_flow |
| route_track_opt | Adds track based optimization to route_design |
| synth_physical | Full physically aware synthesis flow |
| synth_spatial | Physically aware synthesis flow with spatial final
optimization |
```

```
+-------------------------+-------------------------------------------------
---------------+
```

- When one or more of these features are enabled, a new `opt_signoff` flow is added to the `block` flow sequence. Each of these features corresponds to calling `opt_signoff`. For example, on enabling all the `opt_signoff` features, the following flow sequence is displayed:

```
flow:block
flow:opt_signoff
flow_step:run_opt_signoff_area
opt_signoff -area
flow_step:run_opt_signoff_drv
opt_signoff -drv
flow_step:run_opt_signoff_setup
opt_signoff -setup
flow_step:run_opt_signoff_hold
opt_signoff -hold
flow_step:run_opt_signoff_leakage
opt_signoff -leakage
flow_step:run_opt_signoff_dynamic
opt_signoff -dynamic
```

- Following is an example of viewing available flow types using the `-list` parameter:

```
write_flow_template -list
tool provided templates:
+-----------+---------+------------------------------+
| Name | Version | Description |
+-----------+---------+------------------------------+
| block | 1 | Standard block flow (default) |
| hier | 1 | Hierarchical flow |
+-----------+---------+------------------------------+
```

- Following is an example of generating template of `block` type in the `scripts` directory using the `-enable_feature` parameter:

```
write_flow_template \
+ -enable_feature dft_simple \
+ -tools "genus"
##FLOW_KIT: generating tool provided flow type 'block'
Flowkit template writing file scripts/run_flow.tcl
Flowkit template writing file scripts/common_steps.tcl
Flowkit template writing file scripts/genus_config.template
Flowkit template writing file scripts/design_config.template
```

```
Flowkit template writing file scripts/mmmc_config.template
Flowkit template writing file scripts/flow_config.template
Flowkit template writing file scripts/genus_steps.tcl
FLOWKIT MSG: 200 Completed template generation of type 'block' in directory
'scripts'.
```

- Following is an example of generating template of `block` type in the `scripts` directory using the `-optional_feature` parameter:

```
write_flow_template \
+ -optional_feature synth_physical \
+ -tools "genus"
##FLOW_KIT: generating tool provided flow type 'block'
Flowkit template writing file scripts/run_flow.tcl
Flowkit template writing file scripts/common_steps.tcl
Flowkit template writing file scripts/genus_config.template
Flowkit template writing file scripts/voltus_config.template
Flowkit template writing file scripts/design_config.template
Flowkit template writing file scripts/mmmc_config.template
Flowkit template writing file scripts/flow_config.template
Flowkit template writing file scripts/genus_steps.tcl
FLOWKIT MSG: 200 Completed template generation of type 'block' in directory
'scripts'.
```

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

6

General Commands and Attributes

- alias
- check_ccr
- check_netlist
- convert_legacy_to_common_ui
- create_metric_page
- create_metric_table
- create_metric_table_heading
- create_snapshot
- decrypt
- define_metric
- define_proc
- define_variables
- delete_dangling_ports
- delete_metric
- delete_metric_page
- delete_metric_run
- delete_metric_table
- delete_metric_table_heading
- deselect_obj
- encrypt
- eval_legacy
- get_all_layers
- get_common_ui_map
- get_legacy
- get_message
- get_metric
- get_metric_alias
- get_metric_config

- get_metric_definition
- get_metric_header
- get_preference
- gui_bind_key
- gui_move_cursor
- help
- is_common_ui_mode
- man
- metric Category Attributes
- pop_snapshot_stack
- print_message
- program Category Attributes
- push_snapshot_stack
- puts
- read_metric
- read_metric_config
- read_metric_html_package
- redirect
- report_area
- report_dangling_ports
- report_hidden_usage
- report_messages
- report_metric
- report_netlist_statistics
- report_obj
- report_obj Category Attributes
- report_qor
- reset_metric_config
- reset_metric_run_id
- resume
- run_metric_category
- select_obj
- set_compress_level

- set_layer_preference
- set_license_check
- set_limited_access_feature
- set_log_post_cmd
- set_message
- set_metric
- set_metric_alias
- set_metric_header
- set_preference
- set_proc_verbose
- source
- suspend
- system Category Attributes
- uniquify_filename
- update_inst_name
- update_metric
- update_metric_definition
- view_log
- write_metric
- write_metric_config
- rename_snapshot
- delete_floating_constants
- rename_obj

alias

```
alias
[-help]
new_cmd
old_cmd
[-args {{new old} ...}]
```

Create command aliases for any Tcl command. It is a wrapper around the built-in Tcl “interp alias” command. All command aliases are public, so partial command matching and using <tab> to complete a typed in command work on them. It also supports argument name aliases for procedures that use `define_proc` for parsing the arguments, and for

application commands. The argument aliases do not work for built-in Tcl commands. The argument aliases are private, so they are not shown by `-help`, and do not support partial-option completion or matching.

You can use this command to migrate an existing Tcl script when some of the commands or argument names have changed, but are otherwise compatible.

Parameters

`-help`	Prints a brief description that includes type and default information for each `alias` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man alias`
`new_cmd`	Specifies the new command name that is an alias for the existing *`old_cmd`* name. It can be the same name as the *`old_cmd`* if you only want to create new `-arg` aliases, and do not need a new command alias.
`old_cmd`	Specifies the existing old command name.
`-args {{`*`new old`*`} …}`	Specifies a list of new and old argument names to create aliases. You can give multiple *`new`* names for the same *`old`* name. The new aliases are private, and do not show up with `-help`. You can create more than one alias for the same *`old`* name.

Example

The following command adds a new command name alias `new_read_sdf` and some argument name aliases for `read_sdf`:

```
>alias new_read_sdf read_sdf -args {
  {-ignore_error -continue_on_error}
  {-summary -report_summary}
  {-sum -report_summary}
}
```

Then, the following commands

```
>new_read_sdf in.sdf -ignore_error -summary
>new_read_sdf in.sdf -ignore_error -sum
>new_read_s in.sdf -ignore_error -sum
```

would behave the same as

```
>read_sdf in.sdf -continue_on_error -report_summary
```

Note: Tcl partial command matching works if the partial command is unique as shown in the above example, but the alias argument names are private, and will not support partial option matching, or appear in the `-help` output.

Related Topic

- `define_proc`

check_ccr

```
check_ccr
[-help]
ccr_number
```

Returns 1 if the specified CCR is integrated into this build and 0 if it is not.

- It includes all integrated CCRs that have the `Checked_In` or `Validated` status at the time of the build.
- It only includes master CCRs. Duplicate CCRs are not included.
- It currently lists all CCRs fixed in the current release stream or previous release. For example, checking in release version 16.22, will find all CCRs integrated into 16.10 through 16.22 that are in the current 16.22 release.

Parameters

`-help`	Prints a brief description that includes type and default information for each `check_ccr` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_ccr`
`ccr_number`	Specifies the CCR number to check.

Examples

- In the example below, the return value "1" indicates that the CCR 1234567 is integrated into this build.

```
innovus> check_ccr 1234567

1
```

check_netlist

```
check_netlist
-out_file filename
[-sub_module_ports]
```

Performs several checks on the design netlist and creates a report. The report includes the following information:

- Design Summary
- Design Statistics
- I/O Pad Summary
- Design Rule Checking
- Submodule Port Definition Checking (optional)

You can use this command after the design netlist is imported.

Parameters

`-out_file filename`	Specifies the name of the file where the report will be written. If no name is specified, the default filename is `checknetlist.rpt`.
`-sub_module_ports`	If this option is included, `check_netlist` checks the port definitions for any submodules.

Example

Error conditions are indicated by an asterisk (*) preceding the reported statistic, such as

```
Number of Ports Connect to Core Cells *: 57

Number of Ports Connect to multiple Pads *: 0

Number of Floating Ports *: 0
```

Warnings, or possible errors are indicated by a question mark in front of the statistic, such as

```
Number of undefined Floating Pins ?: 0
```

The following example shows the information contained in the generated by `check_netlist` report.

```
Design: TOPModule

------ Design Summary:

Total Standard Cell Number (cells) : 5908

Total Block Cell Number (cells) : 4

Total I/O Pad Cell Number (cells) : 0

Total Standard Cell Area ( um^2) : 133864.32

Total Block Cell Area ( um^2) : 366346.56
```

```
Total I/O Pad Cell Area ( um^2) : 0.00


------ Design Statistics:
Number of Instatnces : 5912
Number of Nets : 6217
Average number of Pins per Net : 3.69
Maximum number of Pins in Net : 541

------ I/O Pad summary
Number of Primary I/O Ports : 57
Number of Input Ports : 28
Number of Output Ports : 29
Number of Bidirectional Ports : 0
Number of Power/Ground Ports : 0
Number of Ports Connect to Core Cells *: 57
Number of Ports Connect to multiple Pads *: 0
Number of Floating Ports *: 0

------ Design Rule Checking:
Number of Output Pins connect to Power/Ground *: 0
Number of Input Floating Pins *: 1
Number of Output Floating Pins : 245
Number of InOut Floating Pins : 0
Number of UniPort Floating Pins : 0
Number of undefined Floating Pins ?: 0

Number of Multiple Driving Nets *: 1
Number of Parallel Driving Nets : 1
Number of Tristate Driving Nets : 0
Number of Input Floating Nets(No FanIn) *: 0
Number of Output Floating Nets(No FanOut) : 1
Number of Hign Fanout Nets (>50) : 3

------ Sub Module Port Definition Checking
```

```
* module arb has 3 port define error

* module accum_stat has 1 port define error

* module tdsp_core_glue has 128 port define error
```

convert_legacy_to_common_ui

```
convert_legacy_to_common_ui
[-help]
<in_dir_or_file>
<out_dir_or_file>
```

Allows usage of legacy scripts to be used in Stylus Common UI.

Parameters

`-help`	Prints out the command usage.
`<in_dir_or_file>`	Specifies the name of the legacy script or script directory name.
`<out_dir_or_file>`	Specifies the Common UI script or script directory name.

create_metric_page

```
create_metric_page
[-help]
[-after page_name]
[-before page_name]
[-format {html excel text}]
-name page_name
```

Creates a new metric page.

Parameters

`-after page_name`	Specifies to insert after the page with this name.
`-before page_name`	Specifies to insert before the page with this name.
`-format {html excel text}`	Specifies configuration for the `report_metric` format.
`-help`	Prints out the command usage. For a detailed description of the command, use the man command: `man create_metric_page`
`-name page_name`	Specifies the name of the new page. The page name must be unique.

Related Topic

- `report_metric`

create_metric_table

```
create_metric_table
[-help]
[-auto_hide]
[-collapsible_key]
[-format {html excel text}]
-heading table_heading
[-hide_footers]
[-hide_graph_footers]
[-hierarchy_separator separator]
[-id table_id]
[-key regexp_key]
[-per_snapshot]
[-show_label_every frequency]
-type {horizontal vertical cpu dict time_statement histogram imageplot}
```

Creates a metric table.

Parameters

`-auto_hide`	Specifies whether to auto hide the table.
`-collapsible_key`	Specifies to include collapse key columns in the table with the option to expand them.
`-format {html excel text}`	Specifies configuration for the `report_metric` format.
`-heading` *heading_id*	Specifies the id of the heading to append.
`-hide_footers`	Hides the minimum, average, and maximum rows from the bottom of the table. Graph buttons are still shown.
`-hide_graph_footers`	Hides the graph buttons from the bottom of the table.
`-hierarchy_separator` *separator*	Shows the metric names with hierarchy using the provided separator.
`-id` *table_id*	Specifies the id for the new table.
`-key` *regexp_key*	Specifies the key for any regexp.
`-per_snapshot`	Specifies to expand the table per snapshot.
`-show_label_every` *frequency*	Specifies the frequency of X-axis labels in the histogram.
`-type {horizontal vertical cpu dict time_statement histogram imageplot}`	Specifies the type (horizontal vertical cpu dict time_statement histogram imageplot) of table

Related Topics

- `report_metric` command

create_metric_table_heading

```
create_metric_table_heading
[-help]
[-format {html excel text}]
[-id heading_id]
-page page_name
-title heading_title
[-type {fixed collapse}]
```

Create a new metric heading.

Parameters

-help	Prints out the command usage. For a detailed description of the command, use the man command: `man` create_metric_table_heading
`-format {html excel text}`	Specifies configuration for the `report_metric` format.
`-id` *`heading_id`*	Specifies the id for the new heading.
`-page` *`page_name`*	Specifies the page name where the heading will be added.
`-title` *`heading_title`*	Specifies the title of the heading.
`-type {fixed collapse}`	Specifies the type of heading element, fixed or collapse.

Related Topics

- `report_metric` command

create_snapshot

```
create_snapshot
[pattern]
[-auto {min default}]
[-categories [* flow design setup power check clock hold route]]
[-exclude_time_metric]
-name metric_name
```

Creates snapshots of the current metric list at various points in the flow.

All super commands that save metrics call `create_snapshot` to save all metrics in a consistent manner. When `create_snapshot` is called, it creates an internal record designated by a label. The label for super commands are the command names.

Metrics that do not require extraction and/or timing/power analysis are calculated when the metric snapshot is saved. Timing/clock/power metrics are reported only if they are up-to-date(i.e. if `time_design`/`report_ccopt*`/`report_power` have been run). Metrics are saved as part of db and can be accessed using `get_db`.

Note: Metric values should be saved such that the unit is easily known/accessible.

Parameters

`-auto {min default}`	Specifies the set of auto metrics to `run`. *Unit*: `enum`
`-categories [* flow design setup power check clock hold route]`	Specifies the metric categories to compute.
`-exclude_time_metric`	Excludes the CPU and wall time from the parent snapshot calculations. *Unit*: `boolean`
`-name metric_name`	Specifies the name of the metric. *Unit*: `string`
`-help`	Outputs a brief description that includes type and default information for each `set_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_metric`
pattern	Specifies specific metrics to save. *Unit*: `string`

Example

The following code shows the manual usage of `create_snapshot`:

```
# ------------------------------------------------------------------
# Manual Usage:
# ------------------------------------------------------------------

opt_design -preCTS
report_power ...
create_snapshot -label prects
ccopt_design
report_power
create_snapshot -label cts
add_filler
route_design
create_snapshot -label route ; # Timing and power are not up-to-date
opt_design -postRoute
report_power
create_snapshot -label postroute
```

The following code shows the automatic/mixed usage of `create_snapshot`:

```
# ------------------------------------------------------------------
# Automatic / Mixed Usage:
# ------------------------------------------------------------------

setDesignMode -enableMetrics true
opt_design -preCTS ; # Saves snapshot "opt_design_<count>"
report_power
create_snapshot -name "prects"
ccopt_design ; # Saves snapshot "ccopt_design"
report_power
create_snapshot -name "postcts" ; # Includes up-to-date power metrics
add_fillers
route_design ; # Saves snapshot "route_design"
opt_design -postRoute ; # Saves snapshot "opt_design_<count>"
report_power
create_snapshot -name "postroute" ; # Includes up-to-date power metrics
```

decrypt

```
decrypt
[-help]
file_name
```

Decrypts and evaluates a Tcl file that was encrypted with the encrypt command.

Parameters

`-help`	Prints out the command usage.
file_name	Specifies the name of the Tcl file to be decrypted and evaluated. *Data_type*: string, required

Related Topics

- encrypt

define_metric

```
define_metric
[-help]
[-compare compareFunction]
[-description metricDescription]
[{-id databaseID}]
[-inheritance_tcl TclName]
-name metric_name
[-no_inheritance]
[-precision number_of_decimal_places]
[-target metric_name]
[-tcl TCL_command_list [-tools listOfTools]]
[-threshold reporting_unit]
[-type metricType]
[-units reporting_unit]
[-warning threshold]
```

Defines new metrics.

You can use this command to create new metrics that you need to track. To create new metrics, you need to provide the TCL code required to generate the metric. The TCL can be a sequence of commands that return a value or a procedure that returns a value.

The following command provides the TCL commands `dbFindInstsByCell HDR*` to define the design.switch_cells metric.

```
define_metric -name design.switch_cells -tcl { return [dbFindInstsByCell HDR*] }
```

Similarly, a proc can be used to define a metric. The following command provides the `my_get_switch_cells{}` proc to define the `design.switch_cells` metric.

```
define_metric -name design.switch_cells -tcl { return [my_get_switch_cells{}] }
```

The metric is calculated and stored with the other metrics when `create_snapshot` is executed. All `define_metric` commands are saved and restored in the design database.

Parameters

`-compare` *compareFunction*	Specifies the compare function, `[moreBetter|lessBetter|moreBetterBelowTarget|lessBetterAboveTarget]`. The comparison options show the good or bad color when the value is below or above the target value as appropriate. *Unit*: `string`
`-description` *metricDescription*	Specifies the description of the metric. *Unit*: `string`
`-id` *databaseID*	Specifies the database ID. *Unit*: `string`

`-inheritance_tcl` *`TclName`*	Specifies Tcl for combining children values. The metric is not executed in the global namespace. The equivalent proc args is {children}. *Unit*: `string`
`-name` *`metric_name`*	Specifies the name of the metric. *Unit*: `string`
`-no_inheritance`	Disables the inheritance of the metric. *Unit*: `boolean`
`-help`	Outputs a brief description that includes type and default information for each `define_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man define_metric`
`-precision` *`number_of_decimal_places`*	Specifies the number of decimal places. *Unit*: `integer`
`-target` *`metric_name`*	Specifies the target or the name of metric containing the target value.
`-tcl { TCL_command_list }`	Specifies the list of TCL commands or proc to generate (return) the metric which is of the format `{<value> <unit>}`. *Unit*: `string`
`-threshold` *`reporting_unit`*	Specifies the threshold for comparing numbers in HTML. If the delta between the two numbers being compared is less than the threshold then the metric will be colored “yellow”. If it is outside the threshold, then it is colored red or green depending on the compare function i.e. whether more is better or less is better.
`-tools` *`listOfTools`*	Specifies the list of tools which the tcl code will run. *Unit*: `string`

`-type` *`metricType`*	Specifies the type of metric.
	Unit: `string`
	The various metrics types are:
	• imageplot: A metric type that the renders to an image in the HTML report.
	• Format:
	`<name1> { \`
	` fill_color "<color>" \`
	`  stroke_color "<color>"\`
	`  data <list of data sets>`
	`} \`

```
<name2> { \
<sub-name1> { \
    fill_color <color> \
    data <list of data sets> \
  } \
  <sub-name2> {...} \
}
```

Where a data set can be one or more of the following:

The data on each layer is a string of coordinates (separated by sequences of whitespace and comma) with types. These types are:

- R - rectangle (x, y, width, height)
- B - bounding box (x_min, y_min, x_max, y_max)
- G - closed polygon (x, y, x, y, ...)
- L - open polyline (x, y, x, y, ...)
- C - circle (x, y, r)
- P - point (x, y)

For rectangles, boxes, circles and points, multiple shapes can be defined without having to repeat the type letter. e.g. R1,2,3,4,5,6,7,8 is two rectangles.

A value can be defined for shapes by adding a V suffix to the type; this value can then be used to color the shapes according to the value for heat maps. The value is always the first number after the type. This means that two points with values 20 and 40 may look like: PV20,1,2 40,6,5 or: PV20,1,2PV40,6,5.

Example imageplot TCL dict data:

```
macros { \
fill_color "#FF0000" \
  stroke_color "#0000FF88" \
  data "R5,8,1,2 6,9,2,1 G1,2,3,4,5,6,7,8" \
} \
drcs { \
placement { \
    fill_color "#FF0000" \
    data "P3,4" \
  } \
  routing {...} \
```

```
}
```

Layers in the image are named (macros, congestion, normal, other in the above example) and support hierarchy. Layers should not be nested beyond one level deep. The hierarchy is used in the renderer (see below) to toggle the visibility of the layers. It is an error for any layer to be called data (reserved to keep hierarchy clear).

Colors can be defined either as hex or RGB values with optional opacity. Hex values are prefixed with #. RGB values are separated with whitespace.

- #FF0000: Red
- #00FF0040: Green (25% opaque)
- 0 0 255: Blue
- 255 0 255 192: Magenta (75% opaque)

- histogram: The data will be structured as a set of series with shared X-axis labels that are rendered as a stacked histogram.

  Format (TCL dict):

```
labels <list of labels> \
axis_x <name> \
axis_y <name> \
series { \
<series1> { \
   values <list of buckets \
   color "#FF0000" \
} \
<series2> { \
   values {2 1 0 0} \
   color "#0000FF" \
} \
}
```

  Example:

```
get_metric timing.setup.histogram
timing.setup.histogram {labels {-0.320 -0.240 -0.160 -0.080} \
                        series { \
                          reg2reg {color #FF0000 values {43 3 2
3 0}} \
                          reg2cgate {color #0000FF values {22 4
5 0 0}} \
```

	`} \`
	`axis_x WNS(ns) \`
	`axis_y {Failing End Points} \`
	`}`
	The labels (a list of strings, shown in the order provided) define the X-axis labels for the histogram, the values in each series define the Y-axis values.
	Colors can be defined either as hex or RGB values with optional opacity. Hex values are prefixed with #. RGB values are separated with whitespace.
	`#FF0000 - Red`
	`0 0 255 - Blue`
	• html: Supports the inclusion of raw HTML in a metric. You can place simple html into tables allowing simple links to reports. For example, the following command defines `test_html` as an html type of metric:
	`define_metric -name test_html -type html`
	You can the html metric, test_html, using the `set_metric` command:
	`set_metric -name test_html -value {<a href="url">test</a>}`

`-units` *`reporting_unit`*	Specifies the unit for reporting. *Unit*: `string`
`-warning` *`threshold`*	Specifies the warning threshold or name of metric containing the threshold value.

Examples

The following command defines the `design.vtmix` metric :

```
define_metric -name design.vtmix -visibility default -tcl {return [TCL commands]}
```

Related Topics

- `create_snapshot` command

define_proc

```
define_proc
[-help]
proc_name
arg_list
proc_body
[-description proc_help]
[-hidden]
[-extended_help help_description]
```

This command defines a Tcl proc that uses an extended argument syntax to check for legal argument names and legal values.

The arguments are parsed, and if any errors occur, an error message is written, and a TCL_ERROR is returned, so the *proc_body* is not executed. If the arguments are parsed successfully the Tcl vars are set accordingly, and the *proc_body* is executed.

It supports simple types like int, string, etc., built-in types like point, rect, and objects like inst, net, pin.

Commands defined with define_proc also have the same support as other Voltus commands, including:

- help support (e.g. "cmd -help" to see the options, and "help c*" would find the command named cmd, etc.)
- tab-completion of partial command or option names (e.g. "cmd<tab>" or "cmd -opt<tab>")
- partial argument name matching is allowed (e.g. "cmd -opt" will work if -opt only matches one option name prefix like -opt_test)
- redirect output to files or Tcl vars using > or >> (e.g. "cmd > my_file", or "cmd > &my_tcl_var")

Parameters

arg_list	Defines the argument list of the Tcl procedure in the following format for each argument:
	`{var default arg_name data_type flags help}`
	For example:
	`{report_file report.out {-out_file file_name} string optional {The output file for the report} }`
	In this format:
	• *var* is the local Tcl variable to assign, such as report_file in the example above. The value for the argument is assigned to this variable before the body of `define_proc` is invoked.
	• *default* is the default value, such as report.out in the example above. If the argument is not given, then *var* will be assigned the *default* value. Note, unlike an argument value passed to the command, the type of the default value is not checked.
	For example, the following command would set int_var to "no_value" if -my_int is not given. Normally the default value should be a legal int value, but

you can use this to check if the argument was given for an optional argument when there is no good default value.

```
{int_var no_value {-my_int int} int optional {int value help description}}
```

The default value is ignored if the argument is a required argument (see *flags* below).

- *arg_name* is the argument name with optional value help. Both named {-opt value} and unnamed {value} arguments are supported.

  If the first word starts with a dash, it is a named argument with optional help string. For example, `{-out_file file_name}` will appear in the help output as: -out_file <file_name>.

  If no help string is given like this:
  `{-out_file}`

  The help output will include the data_type like: -out_file <string>.

  If the first word does not start with a dash, it is an unnamed argument and the first word is used as the help string. For example, `{out_file}` will appear in the help output as: `<out_file>`.

- *data_type* is the argument data type. Many types are supported including simple types, bool type, enum type, built-in types, and object types.
  - These simple types are supported: **int, float, double, string.** They all require an argument value that matches the type (e.g. -my_int 23, or -my_string "my test string"). **float** and **double** are both allowed, and mean the same thing. For example

    ```
    {int_var 0 {-my_int int} int optional {int value help description}}
    ```

    Note that a integer passed to a float or double type will get a .0 added to the end so it can be passed to expr and forced to use floating point arithmetic.
  - The **bool** type supports an argument with or without a value. If the *arg_name* definition has a value help string, then a bool value is required, otherwise a value is not allowed. So, a switch that takes no value would look like:

    ```
    {switch_var false -my_switch bool optional "-my_switch help description"}
    ```

    and would be used like this with no value
    `-my_switch`
    The switch_var will be set to true if -my_switch is given, and set to the default value of false if not given.
    A bool with a value required, would look like this

```
{bool_var false {-my_bool bool} bool optional "bool with value
help description"}
```

and would be used like this

```
-my_bool true
```

Any Tcl allowed bool value can be used for the input value (e.g. true, false, 1, 0, on, off, etc.) but the bool_var will always have either true, false or the *default* value if the argument is not given.

- These built-in types are supported: **point, rect.**
  - **point** is exactly two doubles. The input can be either a single list or list of lists style, so both {1.0 2.0} or {{1.0 2.0}} are allowed as input. The point_var will always be assigned as a simple list of two values for either type of input. An integer value gets a .0 added, so it can be used in a Tcl expr and forced to be treated as a double. So an input like {1 2} will be assigned to the point_var as {1.0 2.0}.
  - **rect** is exactly four doubles. It can be in a single list or list of lists style, so {0 1 2 3}, {{0 1} {2 3}} or {{0 1 2 3}} are allowed. The rect_var will always be assigned as a simple list of four values.
- An **enum** type must include the list of legal enum values like this {**enum** val1 val2 ...}. It supports an optional list_type of ***** (a list of 0 or more) or **+** (a list of 1 or more) as the last enum value. It allows an optional {**hidden** val1 val2 ...} list for hidden enum values. The enum value inputs are case-insensitive, but the return value assigned to *var* is always forced to be all lower-case characters.

  For example,

  ```
  {sort_var name {-sort} {enum name age {hidden city state} +}
  optional {help description}}
  ```

  would allow `-sort age`, or `-sort "name CITY"`, but would give errors for `-sort ""`, or `-sort abc`.

  The help string will show the enum values but not the hidden enum values like this:

  ```
  -sort {name age} # help description (enum+, optional)
  ```

- An **object** type allows object names and pointers to be passed in as values like this {**object** *obj_type1 [obj_type2 ...]* [*list_type*]}. It supports an optional *list_type* of ***** (a list of 0 or more) or **+** (a list of 1 or more) as the last value. If *list_type* is not given, exactly one value is required for input. For example,

  ```
  {driver_var {} {-drivers} {object pin port *} optional {help
  description}}
  ```

  would allow a list of 0 or more pin or port objects for input. The input could be a list of pin or port pointers, or a name-pattern like "i1/p*" that allows * and ? pattern matching. *driver_var* will be assigned a list of pointers of the correct type (or types). So both of these uses would have

the same result:

```
-drivers [concat [get_db pins i1/p*] [get_db ports i1/p*]]
```

```
-drivers i1/p*
```

In both cases, all the pins and ports that match i1/p* would be returned as a list of objects and assigned to *driver_var*.

If any objects are passed in that are not pin or port objects, an error will occur during parsing.

If at least one pointer is required (e.g. list_type **+** or no list_type), and none are matched by the name pattern, an error will occur during parsing.

Design objects like **hinst, inst, hnet, net, hpin, pin**, **port**, etc and library objects like **base_cell, base_pin, lib_cell, lib_pin**, etc, and technology objects like **layer**, **route_rule**, etc. are supported, but only objects that have a .name attribute can use a name-pattern in addition to pointers (e.g. a wire object has no .name so it cannot be looked up with a name-pattern). See the Stylus Common UI Database Reference for a list of all the obj_types.

- *flags* is list of flags that may be an empty list. The supported flags are
  - **optional** means the argument is optional. If the argument is not given, *var* will be set to the *default* value. Arguments are **optional** by default, so this is mostly to make it easy to see it in the definition. It cannot be combined with **required**.
  - **required** means the argument is required. It is an error to not have the argument.
  - **multiple** means the argument can appear multiple times and *var* will be set to a list of all the values.
  - **hidden** means the argument is not shown in the help output, partial argument name matching will not match it, and <tab> complete will not show it. **hidden** must be optional, so it cannot be combined with **required**, **required_group,** or **exclusive_required_group** flags.
  - **obsolete** means the argument still works but is considered obsolete. It will give a warning message if it is used that the user should stop using this argument to be compatible with future releases. An **obsolete** argument will automatically be **hidden** and cannot be a required argument. See **alias** if you just want to change the *arg_name* to a new *arg_name*.
  - {**exclusive_group** *group_name*} means only one of the arguments can be given from the same *group_name*. For example, only one of -my_int or -my_int2 are allowed:

    ```
    {int_var 0 {-my_int int} int {{exclusive_group group1}} {help for -my_int}}
    {int_var2 0 {-my_int2 int} int {{exclusive_group group1}} {help
    ```

	`for -my_int2}}`
	◦ {**exclusive_required_group** *group_name*} means exactly one of the arguments must be given from the same *group_name.*
	◦ {**required_group** *group_name*} means one or more of the arguments must be given from the same *group_name.*
	◦ {**requires** *arg_name1* [*arg_name2* ...]} means this argument requires all the other arguments in the list. For example both -arg1 and -arg2 are required if -my_int is given.
	`{int_var 0 {-my_int int} int {optional {`**`requires`**` -arg1 -arg2}} {help for -my_int}}`
	◦ {**alias** *obsolete_name* [**warn**]} means an alias of *obsolete_name* to the *arg_name* will be added. If **warn** is given, a warning message will be given if the *obsolete_name* is used telling the user to change to *arg_name,* otherwise *obsolete_name* will behave just like *arg_name*. The *obsolete_name* is not shown in help output. This can be used to migrate users to new argument names, but keep the old name for backward compatibility. For example, to migrate from `-old_my_int` to `-new_my_int` you would use this:
	`{int_var 0 {-new_my_int int} int {{`**`alias`**` -old_my_int}} {help for -new_my_int}}`
	• *`help`* is a help string for the argument, such as *"a file name to read"*.

`-description` *`proc_help`*	Specifies the help description for the defined procedure, shown by "help cmd" or "cmd -help".
`-extended_help` *`help_description`*	Provides the extended help description for a procedure. It is printed after `-description` and `arg_list`; shown by `help cmd` or `cmd -help`.
`-help`	Outputs the command usage and a brief description about the command parameters.
`-hidden`	Makes the procedure hidden and not public. That means <tab> complete will not return this proc name, and "info commands *" will not return this proc name. But if you type the full name like "info commands full_cmd_name" it will return the command, or "full_cmd_name -help" will return help for it.
`proc_body`	Defines the Tcl procedure body that uses var names from *`arg_list`*.
`proc_name`	Specifies the name of the Tcl procedure.

Example

Here is an example of some common usage:

```
define_proc test -description "An example of the extended argument syntax" {
```

```
{out_file "default.out" out_file string optional "The output file name"}
{my_count 0 {-count num_inputs} int required "The number of inputs"}
{verbose false -verbose bool optional "Verbose output"}
{check_err not_set {-check_err bool} bool optional "Override global check setting"}
{effort low -effort {enum low medium high} optional "How much effort to use"}
{my_insts {} {-insts} {object inst +} optional "One or more insts"}
} {
# Proc body goes here
...
}
```

The above procedure generates the following help output:

```
> test -help
```

An example of the extended argument syntax

```
Usage: test [-help] [<out_file>] [-check_err <bool>] -count <num_inputs> [-effort {low medium high}] [-
verbose]
-help                      # Prints out the command usage
<out_file>                 # The output file name (string, optional)
-check_err <bool>          # Override global check setting (bool, optional)
-count <num_inputs>        # The number of inputs (int, required)
-effort {low medium high}  # How much effort to use (enum, optional)
-insts <inst>+             # One or more insts (Iinst)+, optional)
-verbose                   # Verbose output (bool, optional)
```

define_variables

```
define_variables
[-help]
name_list
[-transient  | -delete ]
```

Creates Tcl variables in the global namespace if they do not already exist, and adds them to a save list to be saved by `write_db` and restored by `read_db`. The variables will be saved even if the root attribute `write_db_auto_save_user_globals` is false. It can also remove Tcl globals that would be automatically saved if `write_db_auto_save_user_globals` is true. It is intended for Tcl global variables, but it also supports namespace variables.

During read_db, the Tcl variables are created if they do not already exist and the value is set. If a namespace variable is given, it will also create the namespace if it does not already exist.

Parameters

`-delete`	Deletes the Tcl variables and removes them from the save and transient lists. It returns an error if the Tcl variables do not exist.
`-help`	Outputs a brief description that includes type and default information for each `define_variables` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man define_variables`
`name_list`	Specifies a list of Tcl variable names. If the specified Tcl variable does not already exist, it is created and added to a save list of variables to be saved by `write_db`.
`-transient`	Adds the Tcl variables to a transient list used by `write_db` (and removes them from the save list if they are there). The transient list will override the automatic save list created if the root attribute `write_db_auto_save_user_globals` is `true`, so the variables will not be saved.

Examples

The following command saves the variables `global1` and `namespace1::var1` with `write_db`:

```
>define_variables {global1 namespace1::var1}
```

Related Topics

- `read_db`
- `write_db`
- `write_db_auto_save_user_globals attribute`

delete_dangling_ports

```
delete_dangling_ports
-help
-partition
```

Deletes ports that are disconnected from both the inside and outside of a module, and ports that are connected within the module but disconnected outside the module.

Parameters

`-help`	Prints a brief description that includes type and default information for each `delete_dangling_ports` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_dangling_ports`
`-partition`	Specifies the partition only flag of the type boolean. This is an optional.

delete_metric

```
delete_metric
-help
[{-id databaseID}]
[-name metric_name]
```

Deletes a previously defined (user) metric from all labels.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_metric`
`{-id databaseID}`	Specifies the database ID.
`-name metric_name`	Name of the metric to delete.

Examples

- The following command deletes the `design.vtmix` metric:

```
delete_metric -name design.vtmix
```

delete_metric_page

```
delete_metric_page
[-help]
[-format {html excel text}]
-name page_name
```

Deletes a metric page.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the man command: `man delete_metric_page`
`-format {html excel text}`	Specifies configuration for the `report_metric` format.
`-name page_name`	Specifies the name of the metric page to be deleted.

Related Topics

- `report_metric` command

delete_metric_run

```
delete_metric_run
-help
[-name metric_name]
```

Deletes the specified metric run.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_metric_run` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_metric_run`
`-name` *`metric_name`*	Name of the metric run to delete.

delete_metric_table

```
delete_metric_table
[-help]
[-format {html excel text}]
[-id table_id]
```

Deletes a metric table.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the man command: `man delete_metric_table`
`-format {html excel text}`	Specifies configuration for the `report_metric` format.
`-id table_id`	Specifies the id for the table to be deleted.

Related Topics

- `report_metric` command

delete_metric_table_heading

```
delete_metric_table_heading
[-help]
[-format {html excel text}]
```
[-id *heading_id*]

Deletes a metric heading.

Parameters

-help	Prints out the command usage. For a detailed description of the command, use the man command: `man delete_metric_table_heading`
`-format {html excel text}`	Specifies configuration for the specified `report_metric` format.
`-id` *`heading_id`*	Specifies the id of the heading to be deleted.

Related Topics

- `report_metric` command

deselect_obj

```
deselect_obj
[-help]
{[-all ] | [obj_list]}
```

Deselects the object or the list of objects from a set. The object type can be any object type drawn on GUI such as instance wire, via instance.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deselect_obj` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselect_obj`
`-all`	Deselect all selected objects.
`obj_list`	Specifies the list of objects to be deselected.

Example

```
deselect_obj [dbGet -p top.insts.name i1]
```

encrypt

```
encrypt
[-help]
input_file_name
[-pragma]
[> file]
[-tcl  | -verilog ]
```

Encrypts a file in an nc-protect compatible format.

Parameters

`-help`	Prints out the command usage.
input_file_name	Specifies the file to be encrypted. *Data_type*: string, required
> file	Specifies the name of the encrypted file. By default, the encrypted file is printed to standard out (stdout). *Data_type*: redirect_operator, optional
`-pragma`	Specifies to only encrypt the text between the protect begin and protect end NC Protect pragmas. Encryption block must start with pragma protect comment. *Data_type*: bool, optional
`-tcl`	Specifies that the file to be encrypted is a Tcl file. This is the default option. *Data_type*: bool, optional
`-verilog`	Uses Verilog style comments for NC Protect pragmas. *Data_type*: bool, optional

Related Topics

- decrypt

eval_legacy

```
eval_legacy
[-help]
script
```

Runs a legacy command in the legacy Tcl interpreter. This command is not required in normal usage. However, it may be needed in cases where some legacy usage is not yet available in the Stylus Common UI. This command is intended as a temporary workaround as users transition to the Common UI and will be removed in a future release.

You should first check if there is a Common UI equivalent command before using this command by using the `get_common_ui_map` command:

```
>get_common_ui_map <legacyCommand>
```

Most legacy commands are available, but legacy commands that have behavior changes in the Common UI will not work properly (`saveDesign`, `restoreDesign`, `init_design`, `report_timing` etc.).

The legacy commands run in a separate Tcl interpreter, so Tcl variables in the current interpreter are not available. That means the *script* should normally be inside " " rather than { } so Tcl variable substitution occurs in the current interpreter before passing the script to the legacy Tcl interpreter.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `eval_legacy` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man eval_legacy`
`script`	Specifies the name of the legacy script.

Example

The following command runs `setFillerMode` and `addFiller` commands in the legacy Tcl interpreter.

```
set my_prefix fill_

eval_legacy "setFillerMode -corePrefix $my_prefix"

eval_legacy "addFiller ..."
```

Related Topics

- `get_common_ui_map`
- `convert_legacy_to_common_ui`

get_all_layers

```
get_all_layers
[-help]
[type]
```

Returns a complete list of all layers and floorplan object settings. If you specify type, the software returns all the layers of the specified type. This command can be used at any stage in the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_all_layers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_all_layers`
type	Specifies the type of the layer. The software supports six types of layers, which can be specified as follows: • `object`: If you specify type as `object`, the software returns all object layers, which represent db objects, such as instances, modules, pins, and so on. • `display`: If you specify type as `display`, the software returns display-only or view-only layers, including flightlines, rulers, and congestion. • `multi`: If you specify type as `multi`, the software returns multiple color layers, such as congestion, maps, and yield map. • `metal`: If you specify type as `metal`, the software returns wire/via layers, including metal/via, pin, and blockage. • `custom`: If you specify type as `custom`, the software returns custom layers, which are used to represent custom objects and GDSII data. • `internal`: If you specify type as `internal`, the software returns all internal layers.

Example

Returns all metal layer names:

```
get_all_layers metal
```

get_common_ui_map

```
get_common_ui_map
-help
legacy_name
```

Provides the mapping of a legacy command and its options in Stylus Common UI.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_common_ui_map` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_common_ui_map`
`legacy_name`	Specifies the legacy command name or the UDM name.

Examples

- The following command gets the mapping of the `placeDesign` command in Common UI:

  ```
  get_common_ui_map placeDesign
  ```

  Output:

  ```
  placeDesign             place_design
  -clkGateRecloning       -clock_gate_recloning
  -ilm                    -ilm
  -incremental            -incremental
  -noPrePlaceOpt          -no_pre_place_opt
  -outDir                 -out_dir
  ```

get_legacy

```
get_legacy
-help
common_ui_cmd_name
```

Provides the legacy command of a Stylus Common UI command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_legacy` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_legacy`
`common_ui_cmd_name`	Specifies the Common UI command name.

Examples

- The following command provides the legacy command of the Common UI command `add_route_via_defs`:

```
@voltus 1> get_legacy add_route_via_defs
Command: generateVias
```

get_message

```
get_message
-help
-id msgID
[-severity | -limit | -suppress | -count | -short | -long ]
```

Gets the limit, severity or suppression for a message.

Returns the following information about `set_message` parameters in the Voltus log file and in the Voltus console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays values for all of the `set_message` mode parameters.

Related Topic

- `set_message`

get_metric

```
get_metric
-help
[-branch list_of_patterns]
[-exclude_inherited]
[-id id_name]
[-names label_name]
pattern
[-pending]
[-raw]
```

Gets the current state of metric(s).

You can use this command to query the current state of any metric or metrics from the current specified list of metrics as well as a previous state.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_metric`
`-branch list_of_patterns`	Specifies list of patterns to match snapshot branches. This will limit all output to metrics stored in snapshots whose branch matches the specified branch. This requires any inheritance to be re-run to exclude inherited values from non-matching snapshots.
`-exclude_inherited`	Specifies not to return any inherited value.
`-id id_name`	Specifies the database ID to output. Default is current database.
`-names label_name`	Name of label to get metrics from (wildcards supported); no label reports current metrics
`pattern`	List of patterns to match metrics. This option supports wildcards.
`-pending`	Returns metrics from the pending (not yet created) snapshot.
`-raw`	Returns metrics as raw value (without normalization to defined units).

Examples

- The following command gets all metrics (current behavior):

```
get_metric *
```

- The following command gets the worst negative setup slack for all path groups for the preCTS snapshot from the gold database:

```
get_metric -id gold -names prects timing.setup.wns

prects {timing.setup.wns -0.042}
```

get_metric_alias

```
get_metric_alias
[-help]
-from old_name
[-no_overwrite]
[-order order_of_priority]
-to new_name
```

Returns a tcl dictionary of aliases as dictionaries along with the from, to, and order properties.

Parameters

None

Example

- The following command queries the set of aliases:

  ```
  > get_metric_alias
  ```

  Output

  ```
  >path_group:reg2reg {to reg2reg order 1} path_group:reg2cgate {to reg2cgate order 2}
  ```

Related Topic

- set_metric_alias

get_metric_config

```
get_metric_config
-help
-format {html excel text}
```

Gets the current metric configuration setting.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_metric_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_metric_config`
`-format {html excel text}`	Specifies configuration for the specified `report_metric` format.

Related Topics

- `report_metric` command

get_metric_definition

```
get_metric_definition
-help
-name metric_name
```

Returns the metric definition of the specified metric.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_metric_definition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_metric_definition`
`-id metricid`	Specifies the database ID to output.
`-name label_name`	Specifies the metric name.

get_metric_header

```
get_metric_header
-help
[-id db_id]
-name header_name
```

Returns the metric header information set by the `set_metric_header` command.

Parameters

`-id` *db_id*	Specifies the database ID.
`-name` *header_name*	Specifies the name of the header.
`-help`	Outputs a brief description that includes type and default information for each `get_metric_header` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_metric_header`

Example

The following command sets the value `bar` of the metric header `foo`:

```
> set_metric_header -name timing -value high
```

The following command returns the header value of the metric header `foo`

```
> get_metric_header -name timing
```

```
high
```

Related Topics

- set_metric_header

get_preference

```
get_preference
[-help]
preference_name
```

Returns the current preference settings.

Parameters

`-help`	Prints out the command usage
preference_name	Specifies the name of a preference. *Data_type*: string, required

Related Information

- set_preference

gui_bind_key

```
gui_bind_key
[-help]
key
-cmd cmd
[-is_public]
```

Enables you to create keyboard shortcuts.

You can use this command at any time after starting the software.

Parameters

`-help`	Prints a brief description that includes type and default information for each gui_bind_key parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man` gui_bind_key
key	Specifies the key to be used as the shortcut. You can specify a: • Single shortcut key-Specify any one of the following keys: ◦ `A-Z` ◦ Function keys (`F1-F12`) ◦ Arrow keys ◦ Number keys ◦ Special character keys, such as `@` and `#` ◦ `Space`, `Delete`, and `Esc` keys • Key combination-If you want to use a key combination, specify `Shift`, `Ctrl`, or `Alt` as the first key and any one of the keys allowed as a single shortcut key as the second key in the combination. Examples: ◦ `Alt+G` ◦ `Ctrl+B`
`-cmd` *cmd*	Specifies the command to be executed when key is pressed.
-is_public	Specifies the bindkey as user defined.

Example

Runs the `gui_fit` command when you press `f` on your keyboard:

```
gui_bind_key f "gui_fit"
```

```
gui_bind_key
```

gui_move_cursor

```
gui_move_cursor
[-help]
x y
```

Places the cursor at the specified location in the main window.

You can specify this command at any time after starting the software.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_move_cursor` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_move_cursor`
x	Specifies the x coordinate of the cursor location.
y	Specifies the y coordinate of the cursor location.

Example

The following command places the cursor at the coordinates (`133`, `277`).

```
gui_move_cursor 133 277
```

help

```
help
[-help]
[pattern1]
[pattern2]
[-category category]
[-detail  | -summary ]
[-command  | -attribute  | -obj  | -message ]
```

Displays help information related to a command, object, attribute or message based on the specified arguments.

By default, only commands, objects and attributes are searched for help.

Parameters

`-attribute`	Searches attributes only.
`-category` *category*	Returns only those attributes that are in the given category.
`-command`	Searches commands only.
`-detail`	Provides detailed help, even when multiple help strings are returned. By default, a short help description is returned if more than one items are matched.
`-message`	Searches messages only. Note, By default messages are not searched. This option is required to see the message help.
`-obj`	Searches objects only.
pattern1	Searches for any command, object, or attribute that matches pattern1. Pattern matching with * and ? is allowed. If the pattern ends in : (e.g. inst:) only objects will be matched. If the pattern starts with . (e.g. .place*) then only attributes will be matched.
pattern2	Returns help only for those attributes that match pattern2 for the objects matching pattern1. For example, the following command will return all attributes that match `place*` on the `inst` object. `help inst place*`
`-summary`	Provides a short description even for a single match. By default, if only a single item is matched, the full help description is returned.

Examples

- The following command prints the help for any command, object, or attribute that matches "port*". For attributes, the owning object is shown in (), followed by a summary of the data_type, read/write status, default value, allowed `get_db -index` values, and a description:

```
help port*
Objects:
port: External logical ports of the design. See pg_ports for power/ground ports

Attributes:
port(bump) # port, read-only, default={}, indices={}
# The port the bump is assigned to.
ports(design) # port*, read-only, default={}, indices={}
# ports of design. Does not include pg_ports, unless PG port is explicitly in the
# Verilog netlist.
ports(root) # port*, read-only, default={}, indices={}
# Short-cut for [get_db current_design .ports]
```

- The following command shows the help output for the attribute `.base_cell` on the object `inst:`. The : and . chars in `inst:` and `.base_cell` are optional as they restrict the search to objects and attributes, respectively.

```
help inst: .base_cell
Attribute: base_cell (on obj_type: inst)
Description: The base_cell of the inst.
category:
default:
is_computed: false
is_settable: true
is_user_defined: false
skip_in_db: true
type: base_cell
```

- The following output shows the help for a single command:

```
help write_metric
Command: write_metric:

Usage: write_metric [-help] [<pattern>] [-format <string>] [-id <string>] [-merge] [-names <string>]
{-file <string> | -inline }

-help      # Prints out the command usage
<pattern>   # List of patterns to match metrics (string, optional)
-file <string> # Filename to write (string, optional)
-format <string> # Data format for the return value (string, optional)
-id <string> # Database ID to output (string, optional)
-inline # Return the result instead of writing it to a file (bool, optional)
-merge # Merge the results into an existing file (bool, optional)
-names <string> # List of patterns to match snapshot names (string, optional)
```

- The following command shows the help output for a message.

```
help -message IMPTCM-48
Error/Warning message: IMPTCM-48: "%s" is not a legal option for command "%s". Either the current
option or an option prior to it is not specified correctly.
```

is_common_ui_mode

```
is_common_ui_mode
[-help]
```

Checks if the Stylus mode is enabled.

Parameters

None

Examples

The following example checks it on the Voltus prompt:

```
voltus> is_common_ui_mode

1
```

Return value `1` indicates that the Stylus mode is enabled.

man

```
man
command_name | msg_id
```

Displays information for a command or variable from the *Voltus Text Command Reference*, or prints detailed information for a specified message ID. If no manual page is located, man prints an error message.

The man pages for Voltus commands and variables are located in the following directory:

<install_dir>/<platform>/share/fe/man

Parameter

command_name	Indicates the name of the text command (or variable) for which the manual information is required.
msg_id	Specifies the unique message ID for which you want to display detailed information. The output is categorized in three sections: • NAME: Displays the message ID and its severity level. • SYNOPSIS: Displays the summary of the message text. • DESCRIPTION: Displays detailed information about the message. The detailed information can include the background of the message, likely causes, and alternative steps to troubleshoot the problem.

Example

- The following command returns the manual description of the command report_power:

  ```
  man report_power
  ```

- The following command returns the manual description of the SOCSYC-139 message ID:

  ```
  man SOCSYC-139
  ```

metric Category Attributes

The root attributes in the metric category are:

- metric_advanced_url_endpoint
- metric_capture_3d_hotspots
- metric_capture_depth
- metric_capture_design_image
- metric_capture_design_image_blockages
- metric_capture_design_image_blockages_threshold
- metric_capture_design_image_power_intent
- metric_capture_design_image_route_drc
- metric_capture_max_drc_markers
- metric_capture_min_count
- metric_capture_overwrite
- metric_capture_pba_tns_histogram
- metric_capture_reg2reg_metrics
- metric_capture_timing_analysis_mode
- metric_capture_tns_histogram
- metric_capture_tns_histogram_buckets
- metric_capture_tns_histogram_max_slack
- metric_capture_tns_histogram_paths
- metric_capture_timing_paths
- metric_capture_timing_path_groups
- metric_capture_per_view
- metric_capture_vth_metrics
- metric_capture_vth_per_power_domain
- metric_category_default
- metric_current_run_id
- metric_enable
- metric_summary_metrics

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `metric` category, use the following command:

```
get_db -category metric
```

metric_advanced_url_endpoint

```
metric_advanced_url_endpoint base_url
```

Default: `""`

Specifies the base URL for the advanced metric server.

metric_capture_3d_hotspots

```
metric_capture_3d_hotspots {true | false}
```

Default: `true`

Data_type: boolean, read/write

Specifies the depth for capturing hinst design and power metrics.

metric_capture_depth

```
metric_capture_depth integer
```

Default: `0`

Specifies the depth for capturing hinst design and power metrics.

metric_capture_design_image

```
metric_capture_design_image {true | false}
```

Default: `true`

Captures design image.

metric_capture_design_image_blockages

```
metric_capture_design_image_blockages {true | false}
```

Default: `true`

Data_type: string, read/write

Captures blockages in the design image.

metric_capture_design_image_blockages_threshold

```
metric_capture_design_image_blockages_threshold threshholdValue
```

Default: `100`

Data_type: integer, read/write

Threshold for capturing blockages in metric image.

metric_capture_design_image_power_intent

```
metric_capture_design_image_power_intent {true | false}
```

Default: `true`

Data_type: string, read/write

Captures power intent objects in the design image.

metric_capture_design_image_route_drc

```
metric_capture_design_image_route_drc {true | false}
```

Default: `true`

Data_type: string, read/write

Captures route DRC markers in the design image.

metric_capture_max_drc_markers

```
metric_capture_max_drc_markers max_number
```

Default: `100000`

Specifies the maximum number of DRC markers to include in metric image.

metric_capture_min_count

```
metric_capture_min_count integer
```

Default: `1000`

Specifies the minimum instance count for capturing hinst design and power metrics.

metric_capture_overwrite

```
metric_capture_overwrite {true | false}
```

Default: `false`

Overwrite pending metrics during `create_snapshot` category capture.

metric_capture_pba_tns_histogram

```
metric_capture_pba_tns_histogram {true | false}
```

Default: `false`

Captures the PBA histogram data for TNS.

metric_capture_reg2reg_metrics

```
metric_capture_reg2reg_metrics [true | false}
```

Default: `false`

Data_type: bool, read/write

Forces the capture of reg2reg metrics regardless of path groups.

metric_capture_timing_analysis_mode

```
metric_capture_timing_analysis_mode {gba | ipba}
```

Default: `gba`

Data_type: enum, read only

Controls the timing analysis mode for metric capture during create_snapshot categories setup and hold; gba, ipba.

- `gba`: graph based analysis
- `ipba`: path based analysis, retime_method == path_slew_propagation, retime_mode == exhaustive, timing_path_based_exhaustive_pba_bounded_mode == true

metric_capture_tns_histogram

```
metric_capture_tns_histogram {true | false}
```

Default: `true`

Captures the histogram data for TNS.

metric_capture_tns_histogram_buckets

```
metric_capture_tns_histogram_buckets integer
```

Default: `50`

Specifies the number of buckets for the TNS histogram.

metric_capture_tns_histogram_max_slack

```
metric_capture_tns_histogram_max_slack maximum_slack_value
```

Default: `0`

Specifies the maximum slack value for the TNS histogram. You can change its value if you need to shift the y axis.

metric_capture_tns_histogram_paths

```
metric_capture_tns_histogram_paths integer
```

Default: `10000`

Specifies the number of paths to capture for the TNS histogram.

metric_capture_timing_paths

```
metric_capture_timing_paths integer
```

Default: `10`

Specifies the number of paths to capture for detailed display.

metric_capture_timing_path_groups

```
metric_capture_timing_path_groups path_group
```

Default: `10`

Data_type: string

Controls the path groups for which detailed timing path information (full timing path report and display) is to be captured . By default, it is empty and the tool captures all and reg2reg. If the global is set to a list of groups, the tool will use the listed groups. The related metric name is `timing.setup.paths.path_group:<group>`.

metric_capture_per_view

```
metric_capture_per_view {true | false}
```

Default: `true`

Captures timing metrics per analysis_view.

metric_capture_vth_metrics

```
metric_capture_vth_metrics {true | false}
```

Default: `true`

Data_type: bool, read/write

Captures vth metrics.

metric_capture_vth_per_power_domain

```
metric_capture_vth_per_power_domain {true | false}
```

Default: `false`

Captures vth metrics per power domain.

metric_category_default

```
metric_category_default category
```

Default: `design`

Specifies to capture the default metric categories if none are provided.

metric_current_run_id

```
metric_current_run_id run_id
```

Default: ""

Specifies the current unique run ID returned by the advanced metric server.

metric_enable

```
metric_enable {true | false}
```

Default: `false`

Specifies the current unique run ID returned by the advanced metric server.

metric_summary_metrics

```
metric_summary_metrics {flow.cputime flow.realtime timing.setup.tns timing.setup.wns}
```

Specifies summary metrics to be inherited when viewing snapshots and runs.

pop_snapshot_stack

```
pop_snapshot_stack
```

Pops the metrics saved since the last push.

The `pop_snapshot_stack` command works in conjunction with `push_snapshot_stack` to create metric hierarchies. Any metrics captured inside a push/pop must be saved via `create_snapshot` (implicit or explicit) or they will be lost.

Parameters

None

Example

The following code shows the usage of `pop_snapshot_stack`:

```
read_design ...
push_snapshot_stack
place_design
pop_snapshot_stack
```

```
create_snapshot -label place
push_snapshot_stack
opt_design -pre_cts
pop_snapshot_stack
create_snapshot -label prects
push_snapshot_stack
ccopt_design
time_design -post_cts
time_design -post_cts -hold
pop_snapshot_stack
create_snapshot -label cts
# ------------------------------------------------------------------------
```

This will result in the following snapshot hierarchy:

```
place {
place_design {
<metrics>
}
}
prects {
opt_design -pre_cts {
<metrics>
}
}
cts {
ccopt_design {
<metrics>
}
time_design -post_cts {
<metrics>
}
time_design -post_cts -hold {
<metrics>
}
}
```

Related Topic

- create_snapshot

print_message

```
print_message
-help
formatString
[args]
[-error]
[-warning]
```

Generates a message that is included in the `report_messages` output if the message is designated as an error or a warning.

Parameters

args	Specifies the format argument value to print.
-error	Specifies the message is an error. A prefix will be added.
formatString	Specify the format specifier like sprintf or Tcl's format command.
-help	Prints a brief description that includes the type and default information for each `print_message` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man print_message`.
-warning	Specifies that the message is a warning. A prefix will be added.

Examples

The following commands generate an error and a message, which are included in the report_message output:

```
> print_message -error "A test %s." "from the user"
ERROR: (user): A test from the user.


> print_message -warning "A test %s." "from the user"
WARN: (user): A test from the user.


> print_message "A test %s." "from the user"
A test from the user.


> report_messages
*** Summary of all messages that are not suppressed in this session:
Severity      ID       Count         Summary
ERROR       no_id       1           A test %s.
```

```
ERROR       user        1            A test %s.
WARN        user        1            A test %s.
*** Message Summary: 1 warning(s), 2 error(s)
```

Related Topic

- report_messages

program Category Attributes

The root attributes in the program category are:

- cmd_file
- log_file
- logv_file
- program_major_version
- program_name
- program_short_name
- program_version
- selected

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `program` category, use the following command:

```
get_db -category program
```

cmd_file

```
cmd_file fileName
```

Default: ""

Specifies the cmd log file name of the program.

log_file

```
log_file fileName
```

Default: ""

Specifies the log file name of the program.

logv_file

```
logv_file fileName
```

Default: ""

Specifies the verbose log file name of the program.

program_major_version

`program_major_version`

Default:

Returns the major version and one digit after the decimal point.

For example, the following command gives the major version of the software:

```
@voltus 1> get_db program_major_version
20.1
```

program_name

`program_name`

Default:

Specifies the name of the program.

program_short_name

`program_short_name`

Default:

Specifies the short name of the program.

program_version

`program_version`

Default:

Specifies the version of the program.

selected

`selected` *`list`*

Default: ""

Specifies the list of currently selected objects. It can be set to a list of objects as long as they are selectable.

push_snapshot_stack

`push_snapshot_stack`

Creates a hierarchy for metric capture. All metrics captured inside the hierarchy is saved at this level during snapshot creation (implicit or explicit). Snapshot hierarchies can be nested so any number of hierarchies are supported.

Note: Each `push_snapshot_stack` must have a corresponding `pop_snapshot_stack`.

Parameters

None

Example

The following code shows the usage of `push_snapshot_stack`:

```
read_design ...
push_snapshot_stack
place_design
pop_snapshot_stack
create_snapshot -label place
push_snapshot_stack
opt_design -pre_cts
pop_snapshot_stack
create_snapshot -label prects
push_snapshot_stack
ccopt_design
time_design -post_cts
time_design -post_cts -hold
pop_snapshot_stack
create_snapshot -label cts
# ----------------------------------------------------------------------
```

This will result in the following snapshot hierarchy:

```
place {
place_design {
<metrics>
}
}
prects {
opt_design -pre_cts {
<metrics>
}
}
cts {
ccopt_design {
<metrics>
}
time_design -post_cts {
<metrics>
}
time_design -post_cts -hold {
<metrics>
}
}
```

Related Topic

- pop_snapshot_stack

puts

```
puts
[-help]
string
```

Outputs information to the screen (stdout) and Voltus log files (.log/.logv).

read_metric

```
read_metric
-id id of labels
metrics file | database
[-merge]
[-previous UniqueID]
```

Loads a previously saved metric file for querying, reporting, and comparing.

The `read_metric` command allows you to manually load previous saved metrics files into a Tempus session. You can load any number of these files. However, these files should have a unique id.

Parameters

-id id of labels	Specifies a list of labels to report.
metrics file	Specifies the name of the metrics file.
-merge	Merges the metrics into the named snapshot, instead of replacing.
-previous *UniqueID*	Specifies the uuid of previous snapshots for the new pending snapshot.

Example

The following code shows the usage of the `read_metric` command along with the `write_metric` command:

```
# ------------------------------------------------------------------

# Generate a XML file for all timing, power, and flow metrics from

# from prects to postcts for a previous run (id v101)

# ------------------------------------------------------------------

read_metric -id v101 run101/dbs/postcts.enc.dat/design.metrics


write_metric -id v101 -format xml -labels prects-postcts v101.xml
```

Related Topic

- write_metric

read_metric_config

```
read_metric_config
[-help]
 fileName
[-format {html excel text}]
```

Reads the YAML file written by the `write_metric_config` command.

Parameters

fileName	Specifies the name to read metric configuration from.
`-format {html excel text}`	Specifies the configuration for specified `report_metric` format.
`-help`	Outputs a brief description that includes type and default information for each `read_metric_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_metric_config`

Example

The following example writes the YAML structure in the `text` format in `small.text`, reads the YAML file, and generates the report in the text format.

```
> write_metric -format text small.text ;
> read_metric_config -format text small.yaml
> report_metric  -format text
```

Related Topics

- `report_metric` command

read_metric_html_package

```
read_metric_html_package
[-help]
filename
```

Overwrites the HTML for `report_metric -format vivid` and the vivid page configuration (accessible via `read_metric_config`/`write_metric_config`) with the contents from the specified *`filename`*.

Parameters

`filename`	Specifies the name of the file from where the tool will read the metric package. If the specified *`filename`* is not a supported format, the tool will give an error message that the file format is invalid.
`-help`	Outputs a brief description that includes the type and default information for each `read_metric_html_package` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_metric_html_package`

Related Topics

- read_metric_config
- `write_metric_config`

redirect

```
redirect
[-help]
[file_or_var_name]
[command]
[-tee]
[-stdin | -append]
[-stdin | -variable]
[-variable]
[-stderr | -stdin]
```

Redirects the output to a file or variable. You can also directly use redirect characters without explicit use of the redirect command for Voltus commands.

Any Tcl return value from *command* is returned by redirect. If redirect is used at the prompt, any echoing of the Tcl return at the prompt is not redirected.

You can also use the `redirect` command to write the contents directly to a gzip compressed file by ending the file name with .gz.

Parameters

`-help`	Prints a brief description that includes type and default information for each `redirect` parameter.
`file_or_var_name`	File or variable name for output to go.
`command`	Command to be executed.
`-append`	Appends stdout/stderr to file or variable.
`-stderr`	Redirects stderr.
`-stdin`	Redirects stdin.
`-tee`	Redirects output to both screen and file or variable.
`-variable`	Redirects to a Tcl variable.

It is also possible to use redirect characters similar to Unix usage, for Voltus commands, but not for built-in Tcl commands. This usage works for any Voltus command that does not explicitly register > or >> as options (if > or >> are visible in the man page or -help output for the command, it is explicitly registered). Note that the "source" command is overlaid by Voltus to add various verbose options, so it also supports these characters.

The table below shows the usage of redirect characters, and how to use & to indicate a Tcl var name.

Redirect Characters	Meaning
> <name>	redirect stdout to the file <name>
>> <name>	redirect stdout and append to the file <name>
> &<name>	redirect stdout to the Tcl var <name>

>> &<name>	redirect stdout and append to the Tcl var <name>

Examples

- `redirect timing.txt "time_design -post_route"`
  Redirects the output to a new file `timing.txt`.
- `redirect test.out.gz "source test.tcl" -append`
  Appends the output to the file `test.out.gz in compressed gzip format.`
- `redirect timing.txt "time_design -post_route" -append -tee`
  Redirects the output to both screen and a file.
- `source test.tcl > a.log.gz` #redirect to the file a.log.gz in compressed gzip format
- `report_area >> a.log` #redirect and append to the file a.log
- `report_area > &a` `#redirect to the Tcl var a`
- `report_area >> &a` `#redirect and append to the Tcl var a`

report_area

```
  report_area
[-help]
[-detail]
[-hierarchical_instance hinst_name]
[-include_physical]
[-min_area area_per_module]
[-min_count instance_count]
[-out_file filename]
[-show_leaf_cells]
[-show_msv_cells]
[-sort_by {name count area}]
[-table_style {horizontal vertical}]
[-summary | -depth depth_of_hierarchy]
```

Reports the combined standard cell area in each of the hierarchy modules and the top-level design. Also reports the cell area based on different cell types if the `-detail` option is specified.

Note: The units used in the report depend on the units used in the library. The area for each module is calculated from all the standard cells within each module using the LEF information.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_area` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_area`
`-depth depth_of_hierarchy`	Specifies the depth of hierarchy, all levels if not specified; the top of the hierarchy (design) is 1.
`-detail`	Report the area in different cell type (buffer, inverter, combinational, flop, latch, clock_gate, macro, physical).
`-hierarchical_instance hinst_name`	Reports the area of the leaf instances in the specified hierarchical instance. You can use the -depth option to control the analysis depth of the specified hierarchical instance.
`-include_physical`	Specifies to include physical cells in the area calculation and report. By default, it reports only logical cells.
`-min_count instance_count`	Specifies the min instance count per module. The generated output will report the area of hinsts with number of instances greater than the specified `min_count value`. *Default*: `0`

`-min_area` *`area_per_module`*	Specifies the min area per module. The generated output will report the area of hinsts with area greater than the specified min_area lib_units. *Default*: `0`
`-out_file` *`filename`*	Specifies the name of the output file.
`-show_leaf_cells`	Specifies to include the leaf cells at the design level and each hinst level. These are the instances whose parent is the hinst display. *Default*: `0` (Shows the hinst children in the report)
`-show_msv_cells`	Includes multiple supply voltage cells in the report. This option works with "-table_style vertical"
`-sort_by {name count area}`	Specifies the order of hinst names. *Default*: `name`
`-summary`	Prints the area summary for the design. It is equivalent "`-depth 0`".
`-table_style {horizontal vertical}`	Specifies whether the base report should be expanded in the vertical or horizontal direction. *Default*: `horizontal`

Examples

The following command shows the standard cell area in different base classes.

```
report_area -detail

        Hinst Name                          Module Name                Inst Count Total Area  Combinational   Buffer   Inverter     Flop    Latch   Clock-Gate     Macro
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------
dtmf_recvr_core                                                          4283  37949.153       5223.910     9.526    262.836  2535.221   0.000        0.000  19464.267
  RESULTS_CONV_INST                 results_conv                         1331   2201.472       1126.490     0.000     79.380   995.602   0.000        0.000      0.000
  TDSP_CORE_INST_MPY_32_INST_csa_tree_mul_61_28   csa_tree                623   1158.419       1115.024     0.000     43.394     0.000   0.000        0.000      0.000
  TDSP_CORE_INST_EXECUTE_INST       execute_i                            570   1251.382        504.151     0.000     28.048   719.183   0.000        0.000      0.000
  TDSP_CORE_INST_ALU_32_INST        alu_32                               466    308.593        742.644     0.000     36.515     0.000   0.000        0.000      0.000
    sub_84_22                       sub_unsigned_801                      93     39.421        170.050     0.000     15.876     0.000   0.000        0.000      0.000
    add_81_22                       add_unsigned_799                      77     28.816        170.755     0.000      0.529     0.000   0.000        0.000
```

```
 0.000
  inc_add_54_27_1                          increment_unsigned                 31     14.262        107.604     0.000       0.000       0.000    0.000
0.000       0.000
 TDSP_CORE_INST_MPY_32_INST_final_adder_mul_61_28 add_unsigned                        74     27.634        165.287     0.000       0.000       0.000
 0.000        0.000       0.000
 TDSP_CORE_INST_MPY_32_INST_addinc_add_62_54_8    add_signed_carry                    76     28.573        163.523     0.000       0.529
0.000   0.000        0.000       0.000
 TDSP_CORE_INST_TDSP_CORE_GLUE_INST_sll_120_49    shift_left_vlog_unsigned_393        92     51.014        128.772     0.000       3.175
0.000   0.000        0.000       0.000
 TDSP_CORE_INST_TDSP_CORE_GLUE_INST_sll_121_36    shift_left_vlog_unsigned            56     30.828        94.550      0.000       1.058
0.000   0.000        0.000       0.000
```

The following command shows the leaf cells at the hinst level.

```
report_area -show_leaf_cells
```

```
        Hinst Name                        Module Name              Cell Count  Cell Area  Total Area
-------------------------------------------------------------------------------------------------------------
dtmf_recvr_core                                                     4283  37949.153  41732.426
 (Leaf Cells)                                                      995  32001.000  33406.000
 RESULTS_CONV_INST                        results_conv              1331   2201.472   3311.412
 TDSP_CORE_INST_MPY_32_INST_csa_tree_mul_61_28    csa_tree                  623   1158.419   1618.517
 TDSP_CORE_INST_EXECUTE_INST              execute_i                 570   1251.382   1613.524
 TDSP_CORE_INST_ALU_32_INST               alu_32                    466    779.159   1087.752
  sub_84_22                           sub_unsigned_801            93    185.926    225.347
  add_81_22                           add_unsigned_799            77    171.284    200.101
  inc_add_54_27_1                       increment_unsigned          31    107.604    121.866
 TDSP_CORE_INST_MPY_32_INST_final_adder_mul_61_28 add_unsigned                74    165.287    192.921
 TDSP_CORE_INST_MPY_32_INST_addinc_add_62_54_8    add_signed_carry            76    164.052    192.625
 TDSP_CORE_INST_TDSP_CORE_GLUE_INST_sll_120_49    shift_left_vlog_unsigned_393    92    131.947    182.961
 TDSP_CORE_INST_TDSP_CORE_GLUE_INST_sll_121_36    shift_left_vlog_unsigned        56     95.609    126.436
```

The following command shows the base report in the vertical direction.

```
report_area -detail -table_style vertical
```

```
 Hinst Name          Module Name          Inst Count          Total Area
```

---

dtmf_recvr_core		4283	37949.15
(Buffer)		9	9.53
(Inverter)		491	262.84
(Combinational)		3279	5223.91
(Flop)		501	2535.22
(Latch)		0	0.00
(Clock-Gate)		0	0.00
(Macro)		3	29917.66
RESULTS_CONV_INST	results_conv	1331	2201.47
(Buffer)		0	0.00
(Inverter)		150	79.38
(Combinational)		981	1126.49
(Flop)		200	995.60
(Latch)		0	0.00
(Clock-Gate)		0	0.00
(Macro)		0	0.00

report_dangling_ports

```
report_dangling_ports
-help
-partition
-out_file fileName
```

Reports ports that are disconnected from both the inside and outside of a module, and ports that are connected within the module but disconnected outside the module.

Note: If the design has some dangling or unconnected ports, tool will add `FE_UNCONNECTED_*` nets for these dangling or unconnected ports in the design during optimization. You can report these dangling ports using the `report_dangling_ports` command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_dangling_ports` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_dangling_ports`
`-out_file`	Specifies the name of the report generated by this command.
`-partition`	The option a partition only flag of the type boolean. This is an optional.

Examples

The following command writes information on dangling reports to file `myreport`:

```
report_dangling_ports -out_file myreport
```

report_hidden_usage

`report_hidden_usage`
`in_files`

Reports a summary of all the hidden application Tcl globals and mode options in the specified files. Hidden globals and mode options are normally used for temporary fixes or early access, and often disappear or change in a new release. This command will help to find out the hidden settings used in existing scripts that might not be needed in the new release.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_hidden_usage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_hidden_usage`
`in_files`	Specifies input files to search for hidden usage.

Examples

- The following command reports the hidden settings in the floorplan.tcl and place.tcl files.

```
>report_hidden_usage {floorplan.tcl cts.tcl} > hidden.rpt

################################################################################
#  Generated by:    Cadence Voltus 18.10-a072_1
#  OS:              Linux x86_64(Host ID sjfib146)
#  Generated on:    Tue Apr 17 22:47:31 2018
#  Design:          top
#  Command:         report_hidden_usage floorplan.tcl cts.tcl > hidden.rpt
################################################################################


### floorplan.tcl ###
--------------------------------------------------------------------------------------------
Line                      Hidden name                    Command
--------------------------------------------------------------------------------------------
```

```
12                          floorplan_xxx                   set_db floorplan_xxx true

32                          init_xxx                        set_db init_xxx 100


### cts.tcl ###

-------------------------------------------------------------------------------------

Line                         Hidden name                    Command

-------------------------------------------------------------------------------------

3                            cts_xxx                        set_db cts_xxx true
```

report_messages

```
report_messages
[-help]
[[{-errors | -warnings | -all } [-count ]] | -suppressed ]
```

Reports the messages that have been issued since the tool was launched. When you run `report_messages` without any argument, it prints a summary table and returns no Tcl value. However, when you provide an argument, it returns a list a of message IDs and does not print a table.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_messages` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_messages`
`-all`	Returns a list of all issued message IDs as a Tcl result. The result does not include suppressed message IDs.
`-count`	Returns count of every message when `-errors`, `-warnings`, or `-all` is specified.
`-errors`	Returns a list of issued error message IDs as a Tcl result.
`-suppressed`	Returns a list of suppressed message IDs as a Tcl result.
`-warnings`	Returns a list of issued warning message IDs as a Tcl result.

Examples

- The following command prints a summary table and returns no Tcl value.

```
report_messages
```

Output:

```
*** Summary of all messages that are not suppressed in this session:
Severity           ID               Count              Summary
WARNING          ENCFP-2101         6                Shifter for %s to %s is not defined in s...
WARNING          ENCFP-3961         1                techSite '%s' has no related Cells in LE...
WARNING          ENCMSMV-1610       4                View %s domain %s operating voltage %g d...
WARNING          ENCTS-3            692              Can't create min and max cell relation f...
WARNING          ENCTS-419          119              Timing arc(s) mismatch found in cell '%s...
ERROR            ENCTS-423          5                No library found for instance '%s', cell...
ERROR            ENCTS-424          1                Missing library for some instance found...
WARNING          ENCEXT-6202        1                In addition to the technology file, capa...
WARNING          ENCSYT-3046        2                Unknown preference name "%s".
WARNING          ENCSYC-1592        2                No default tech site found for power dom...
WARNING          ENCPP-4022         2                Option "-%s" is obsolete and has been re...
WARNING          ENCSC-1001         2                Unable to trace scan chain "%s". Check t...
WARNING          ENCSC-1020         1                Instance's output pin "%s/%s" (Cell "%s"...
WARNING          ENCSC-1114         2                The scan chain "%s" is not traced.
WARNING          ENCSC-1143         1                Unable to apply DEF ordered sections for...
WARNING          ENCSC-1144         2                Scan chain "%s" was not traced through. ...
WARNING          ENCOPT-3058        1003             Cell %s/%s has already a dont_use attrib...
WARNING          ENCOAX-571         1                Property '%s' from OA is a hierarchical ...
WARNING          ENCOAX-1257        1                Allowing saveDesign to create OpenAccess...
WARNING          ENCTCM-77          1                Option "%s" for command %s is obsolete a...
*** Message Summary: 1843 warning(s), 6 error(s)
```

- The following command reports all issue message IDs.

```
report_messages -all
```

- The following command reports all issue message IDs.

```
report_messages -suppressed
```

report_metric

```
report_metric
[-help]
pattern
-out_file fileName
-format {html json excel text vivid}
[-id metricID]
[-names listOfNames]
```

Allows the user to generate reports in various file formats.

Parameters

`-out_file fileName`	Specifies the name of the target file.
`-format {html json excel text vivid}`	Specifies the output type.
`-help`	Outputs a brief description that includes type and default information for each `report_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_metric`
`-id metricID`	Specifies the ID of metrics to report.
`-names listOfNames`	Specifies the list of names. **Note**: Wildcards are supported.
`pattern`	Specifies the pattern to match.

report_netlist_statistics

```
report_netlist_statistics
```

Reports statistical data of the design netlist such as number of nets, number of IOs, number of pins in the design and number of terms per net(s).

Example

```
*** Statistics for net list DTMF_CHIP ***
Number of cells      = 4263
Number of nets       = 4551
Number of tri-nets   = 0
Number of degen nets = 21
Number of pins       = 16490
Number of i/os       = 57

Number of nets with    1 terms = 21 (0.5%)
Number of nets with    2 terms = 2510 (55.2%)
Number of nets with    3 terms = 771 (16.9%)
Number of nets with    4 terms = 510 (11.2%)
Number of nets with    5 terms = 223 (4.9%)
Number of nets with    6 terms = 108 (2.4%)
Number of nets with    7 terms = 76 (1.7%)
Number of nets with    8 terms = 46 (1.0%)
Number of nets with    9 terms = 30 (0.7%)
Number of nets with >=10 terms = 256 (5.6%)

*** 72 Primitives used:
Primitive ADDFHX1 (195 insts)
Primitive ADDHX1 (15 insts)
Primitive AND2X1 (279 insts)
Primitive AND3X1 (1 insts)
Primitive AND4X1 (8 insts)
Primitive AOI211X1 (65 insts)
Primitive AOI21X1 (282 insts)
```

report_obj

```
report_obj
[<start> [<pattern>]]
[-all | [-computed] | [-selected]]
[-categories list_of_categories]
[-file fileName]
[-index index_name index_value ...]
[-help]
[-set_by_user | -read_only]
[-tcl | -verbose]
```

Creates a report about the attributes for a given pattern/category/object. This command is mainly intended for root attributes but can be used for any object.

The default output reports those attributes that are set to non-default values and are not computed. This makes it easy to see any changes that were made in the run scripts.

You can also add more attributes or filter out some attributes using various parameters below.

Parameters

`-help`	Prints a brief description that includes the type and default information for each system root attribute.
`-all`	Reports all non-computed attributes and their current value, even if the current value matches the default value.
`-categories list_of_categories`	Reports only those attributes that are in the given categories. Wildcards * and ? are allowed. Attributes that do not have a category are assigned the reserved category name `NULL`. The report puts all the attributes in a category together.
`-computed`	Includes attributes that are computed (for example `is_computed == true`). By default, the computed attributes are not included. If the data is not computed, this parameter triggers building the timing graph, and so on, that may take significant run time. This parameter cannot be used with `-all`.
`-file fileName`	Specifies the output file name.
`-index index_name index_value ...`	Specifies the list of indices in a form of object pointers or pairs of *obj_type object_name*, for example. '-index {view view1}' or '-index $obj'.
`<pattern>`	Returns all the attribute names that match the specified `<pattern>` using the * and ? wildcards. The `<start>` value is required before the `<pattern>` value. *Default:* *
`-read_only`	Reports attributes with `.is_settable = false`.

`-set_by_user`	Reports the root attributes set by the user. In the legacy system, the `setXXXMode` options are tracked if the user sets them or not independently from the value, while the legacy Tcl globals (e.g. timing_xxx) do not track this. For the root attributes derived from `setXXXMode` options, the `set_by_user` flag can be checked and the root attribute value returned even if the value equals the default value (use `reset_db` to clear the `set_by_user` flag). For root attributes derived from legacy Tcl globals, there is no `set_by_user` flag, so only the root attributes that have a value different from the default value are returned. **Note**: If `-categories` is used along with `-set_by_user`, the attributes in the given categories, as well as all unmapped attributes will be printed. *Data_type*: bool, optional
start	Returns attributes for the objects in the <start> list. *Default:* `root`
`-selected`	Reports attributes for only selected objects.
`-tcl`	Returns the objects and their values in a TCL style list, which is a list of keyword-value pairs.
`-verbose`	Returns the object's data type and default value. This parameter cannot be used with the `-tcl` parameter.

Examples

- The following command displays the root attributes for the `timing` category that are not computed, and do not match the default value with their current values:

```
report_obj -categories timing -all

******************************************************************************** Object:
root:*********************************************************************************

========================================================= Category:
timing=========================================================

Attribute Name                                          Current Value

-------------------------------------------------------------------------------

budget_timing_compare_with_new_opt_data                0budget_timing_push_down_virtual_clock
0budget_timing_write_clock_to_virtual_clock_map         0timing_all_registers_include_icg_cells
truetiming_allow_input_delay_on_clock_source              falsetiming_analysis_aocv
false

...
```

- The following command displays all the root attributes that start with place*.

```
report_obj / place* -
all*******************************************************************************************
```

```
Object: /*************************************************************************************

Attribute Name                                                  Current Value ----------------------------
-------------------------------------------------------------------place_blockages
{}place_cell_edge_spacing                                         {}place_design_floorplan_mode
falseplace_design_refine_place                                      true...
```

- The following command displays the attributes that do not match the default value for the net objects in the list:

```
report_obj [get_db nets *clk]
************************************************************************************** Object:
net:top/tclk*************************************************************************************
*

Attribute Name                                                  Current Value ----------------------------
-------------------------------------------------------------------arcs
arc:0x2b1c79348088 arc:0x2b1c793485b0base_name
tclkbbox                                                           {1073741.8235 1073741.8235 -
1073741.824 -1073741.824}...

************************************************************************************** Object:
net:top/tclk2************************************************************************************
**

Attribute Name                                                  Current Value ----------------------------
-------------------------------------------------------------------arcs
arc:0x2b1c79348088base_name                                                        tclk2bbox
{1073741.8235 1073741.8235 -1073741.824 -1073741.824}...
```

report_obj Category Attributes

The root attributes in the report_obj category are:

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `program` category, use the following command:

```
get_db -category program
```

report_obj_display_limit

```
report_obj_display_limit displayLimit
```

Default: `int`

Data_type: `1000`

Controls the number of objects that will be displayed when using `report_obj`. If the value is `10`, 'report_obj insts' will only display 10 instance objects with all their non-default attributes.

report_qor

```
report_qor
[-help]
[pattern]
-out_file fileName
[-format {html json}]
[-id metricID]
[-names listOfNames]
```

Allows the user to generate reports in various file formats.

Parameters

`-help`	Prints a brief description that includes the type and default information for each system root attribute.
`pattern`	Specifies the pattern to be matched.
`-out_file` *`fileName`*	Specifies the name of the target file.
`-format {html json}`	Specifies the output format.
`-id` *`metricID`*	Specifies the ID of metrics to report.
`-names` *`listOfNames`*	Specifies the list of names. You can use wildcards.

reset_metric_config

`reset_metric_config`

Removes all metric configuration.

Parameters

None

reset_metric_run_id

```
reset_metric_run_id
```

Clears the run ID and the related data from the current state so that other commands do not add snapshots to the original reference run in AUM.

Parameters

None

resume

```
resume [-help]
```

Resumes a suspended script from the point where it had stopped.

Use the `suspend` and `resume` commands to debug your scripts interactively. The `suspend` command stops a script and restores access to the Voltus prompt. You can then type any command required for debugging. Whenever you want to resume the suspended script, type `resume` at the Voltus prompt.

Parameters

`-help`	Prints the usage of the `resume` command. For a detailed description of the command, use the `man` command: `man resume`.

Related Topic

- `suspend`

run_metric_category

```
run_metric_category
[-help]
-category category_name
```

Runs the specified category.

Parameters

`-help`	Describes the command usage.
`-category category_name`	Specifies the name of the category, such as design, setup, hold, clock, power, route, check, flow, to be run. Each category updates the metric associated with that category.

select_obj

```
select_obj
[-help]
<any_object>+
```

Selects the object or the list of objects for putting into a set. The object type can be any object type selectable on the GUI such as inst and net.

Parameters

`-help`	Prints a brief description that includes type and default information for each `select_obj` parameter. For a detailed description of the command and all of its parameters, use the man command: `man select_obj`
`<any_object>+`	Specifies the list of objects.

Example

```
select_obj [get_db insts i1/*]
```

set_compress_level

`set_compress_level` *`compression_level`*

Specifies the compression speed to use when compressing large data files. A faster compression speed creates a less compressed file. A slower compression speed creates a more compressed file. By default, the software uses the `gzip` default speed compression_level.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_compress_level` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_compress_level`
`compression_level`	Specifies the speed level to use when compressing files. You can specify a compression_level between 1 and 9, where: `1` Performs the quickest compression, and least compressed files. `9` Performs the slowest compression, and most compressed files. **Note:** To return to the `gzip` default compression_level (`Z_DEFAULT_COMPRESSION`), specify `-1`.

set_layer_preference

```
set_layer_preference
[-help]
layer_name
[-is_visible {1 | 0}]
[-is_selectable {1 | 0}]
[-color {color_name | color_value}]
[-line_width {lw}]
[-stipple stipple_name | -stipple_data width height "stipple_data"]
```

Sets a new preference for the specified object or layer.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_layer_preference` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_layer_preference`
`-color`	Specifies a color for the object. Specify either a color name or a numeric RGB value.
`-is_selectable`	Specifies whether the object is selectable. Specify `1` to make the object selectable. Specify `0` to make the object non-selectable.

`-is_visible`	Specifies whether the object is visible. Specify `1` to make the object visible. Specify `0` to make the object invisible.
	Examples
	• You can turn off the visibility of all custom layers by using the following command:
	`set_layer_preference customLayers -is_visible 0`
	• You can turn off display of flightlines by using the following command:
	`set_layer_preference flightLine -is_visible 0`

`layer_name`	Specifies the name of the layer or the object type. The valid layer names are `M#`, where `#` is `0` through `1`, and `M#AVio`, `M#Cont`, `M#ContPin`, `M#ContRB`, `M#Pin`, `M#RB`, `M#RTrk`, and `M#Vio`. The valid object types are `blackBox`, `block`, `blockHalo`, `bump`, `bump1`, `bump2`, `bump3`, `cellBlkgObj`, `channel`, `clkTree`, `congest`, `congestH`, `congestObj`, `congestV`, `datapath`, `fence`, `gcellOvflow`, `guide`, `hinst`, `inst`, `ioSlot`, `ioRow`, `macroSitePattern`, `metalFill`, `mGrid`, `net`, `netRect`, `nonPrefTrackObj`, `obstruct`, `pinObj`, `pinText`, `power`, `powerNet`, `ptnFeed`, `ptnPinBlk`, `pwrdm`, `pwrdm1`, `pwrdm2`, `pwrdm3`, `layerBlk`, `region`, `routeGuide`, `ruler`, `screen`, `select`, `sitePattern`, `stdRow`, `term`, `text`, `trackObj`, `userGrid`, and `violation`.
`-line_width`	Specifies the number of pixels in the border of the specified object. Specify this value as integer.
`-stipple`	Specifies information about the stipple pattern of the object. The valid stipple names are `None`, `Horizontal`, `Vertical`, `Grid`, `Slash`, `Backslash`, `Cross`, and `Brick`.
`-stipple_data`	Specifies the stipple information for the object. Specify this information in x-window bitmap format.

Example

The following command sets a layer preference for the module object in green with dotted stipple pattern (8 by 4 bitmap with dot at every 4 pixels):

```
set_layer_preference hinst -color green -stipple_data 8 4 "0x00 0x11 0x00 0x00"
```

set_license_check

```
set_license_check
[-help]
[-option_list {license1 license2 …}]
[-wait_time time_in_minutes]
[-default]
[-checkout license]
[-status_report]
[-server_license_report]
```

Manages licenses for product options. Specifies licenses to check out, license wait time, and the status of all product option licenses.

Note: This command does not manage multi-CPU licenses. For information on the commands that manage multi-CPU licenses, see the "*Multiple-CPU Processing Commands*" chapter.

Parameters

`-checkout` *`license`*	Checks out the licenses for the specified product options when this command runs.
`-default`	Restores the default. values for this command.
`-server_license_report`	Outputs a table that provides the following information for each of the licenses that exist on the license server. • License name • Product number • Product name • License string • Version
`-help`	Outputs a brief description that includes type and default information for each `set_license_check` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_license_check`.
`-option_list {`*`license1 license2 …`*`}`	Specifies licenses to check out dynamically. Dynamic licenses are not checked out until they are needed. If you specify a license that is not allowed with your base license, or a license that is not available, the software does not check out that license and instead issues a warning message. If you specify more than one license, begin and end the option list with double quotation marks or braces.
`-status_report`	Outputs the following information: • The name of the base license that is checked out • The current wait time • A list of the allowed license options • A list of the available license options • A list of the currently checked-out license options.
`-wait_time` *`time_in_minutes`*	Specifies the amount of time the system waits for a license to become available. If the license is available in less than the specified wait time, the system checks out the next needed license without waiting. *Default:* `0` (no wait time) *Value range:* `0` to `10,000`

Examples

- The following command outputs a table that provides information about the licenses that exist on the license server:
  ```
  set_license_check -server_license_report
  ```

  The system outputs the following information:

  ```
  Name       Prod #    Product Name                        License string            Version
  vtsaa      VTS201    Voltus Advanced Analysis GXL Option Voltus_Power_Integrity_AA 16.1
  vtsng      VTSENG100 Voltus Next Generation Product 100  VTS_NG100                 1.0000
  tpsxl      TPS200    Tempus Timing Signoff Solution XL   Tempus_Timing_Signoff_XL  16.1
  tpsl       TPS100    Tempus Timing Signoff Solution L    Tempus_Timing_Signoff_L   16.1
  invs_ms    INVS30    Innovus Mixed Signal Option         Innovus_MS_Opt            16.1
  ```

- The following command specifies that licenses for VTS-AA may be checked out. .
  ```
  set_license_check -option_list {vtsaa}
  ```
- The following command sets the wait time to 5 minutes:
  ```
  set_license_check -wait_time 5
  ```

Related Topics

- Product and Licensing Information chapter in the *Voltus User Guide*
- Getting Started chapter in the *Voltus User Guide*

set_limited_access_feature

```
set_limited_access_feature
[-help]
featureName {1 | 0}
```

Enables or disables a limited access feature.

Innovus includes certain limited access features that showcase advanced technology integrated in the software. These features have been internally qualified at Cadence but have had only limited customer testing.

> Limited access features are enabled by a variable specified through the `set_limited_access_feature` command. To use these limited access features, contact your Cadence representative to qualify your usage and to make sure it meets your needs before deploying it widely.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_limited_access_feature` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_limited_access_feature`
featureName {*1	0*}	Specifies the name of the variable that enables the feature. Specifying a value of `1` enables the feature. Specifying a value of `0` disables the feature.

set_log_post_cmd

```
set_log_post_cmd
[-help]
[-disable command_list]
[-enable command_list]
[-report]
```

Turns on/off specific commands post-expansion. It prevents showing lots of `get_db` or set commands and cluttering of the log file.

Note: This command does not affect pre-expansion logging.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_log_post_cmd` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_log_post_cmd`
`-disable command_list`	Specifies a a list of command names to not log
`-enable command_list`	Specifies a a list of command names to be logged.
`-report`	Writes out each disabled commands to stdout. One command is written per line.

Related Topic

- `get_db`

set_message

```
  set_message
-help
-id <list_of_msgIDs>
[-severity {warn error info} | -limit <number> | -no_limit | -suppress | -unsuppress | -on_error {msg_only
exit stop_script} | -stop_script_limit limitNumber]
```

Specifies to change the message severity levels, `INFO, WARNING` or `ERROR`, at the beginning of the message. This command allows you to control the messages that are displayed for a particular design. For example, if there are some warnings that are critical for the design, you can change the message severity to `ERROR`. Similarly, if there are messages that you want to ignore, you can change the message severity to `INFO`.

Parameters

-help	Outputs a brief description that includes type and default information for each `set_message` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_message`.
-id *list_of_msgIDs*	Specifies the list of message IDs that require a change in the message severity level. If you do not specify a message ID, the change in severity level is applied to all message IDs.
-limit *number*	Sets the message limit to *number* per every public command. The default limit is 20, so any specific message will be written out 20 times for one command and then further output of that message will be disabled (along with an additional message saying the message limit has been exceeded). If the number is set to 0, the message will be completely suppressed for the next public command that triggers the message. You can unsuppress the message by setting the number greater than 0.
-no_limit	Removes any existing limit on the selected messages. Suppressed messages are not unsuppressed.
-on_error {msg_only exit stop_script}	Specifies an extra action for error messages (not for warn or info messages). If no message is specified, the action applies to all error messages. • `msg_only`: Prints error message only, and the command proceeds as it normally would. This is the default. • `exit`: Exits immediately and returns to Linux. • `stop_script`: Waits for the current Tcl command to finish, then stops executing any Tcl script, and returns to the interactive Tcl prompt. Note, if a message severity is changed from error to either warn or info, it will no longer trigger this extra action.
-severity {warn error info reset}	Specifies the new severity level for the message ID being modified. The `reset` argument sets the message severity back to its original state.
-stop_script_limit *limitNumber*	Sets the message limit for a public command. If you set `-stop_script_limit` for a command as `10` and message for the command is triggered for the 10th time, the tool will wait for the current Tcl command to finish, then stop executing any Tcl script, and return to the interactive Tcl prompt. The default value of `-stop_script_limit` is `-1`, which is unlimited.
-suppress	Suppresses the message.
-unsuppress	Unsuppresses the message.

Examples

- The following command changes the message ENCLF-81 from `ERROR` to `WARN`:

  ```
  set_message -id ENCLF-81 -severity warn
  ```

- The following command returns all messages to their original severity:

  ```
  set_message -severity reset
  ```

- The following command resets the specified message to its original severity:

  ```
  set_message -id ENCLF-18  -severity reset
  ```

set_metric

```
set_metric
-name metric_name
-help
-value metric_value
```

Sets any defined metric to a value.

Parameters

`-name metric_name`	Specifies the name of the metric.
`-help`	Outputs a brief description that includes type and default information for each `set_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_metric`
`-value metric_value`	Specifies the value of the metric.

Examples

The following command sets the value `dbFindInstsByCell HDR*` to the metric `design.switch_cells`.

```
set_metric -name design.switch_cells [dbFindInstsByCell HDR*]
```

Related Topic

- `get_metric`

set_metric_alias

```
set_metric_alias
[-help]
-from old_name
[-no_overwrite]
[-order order_of_priority]
-to new_name
```

Renames a metric name, prefixed with the name of a layer, analysis view, path group etc, to another name. For example, you can rename `path_group:reg2reg` to `reg2reg` using the `set_metric_alias` command.

Parameters

`-from old_name`	Specifies the metric object name to be changed.
`-help`	Outputs a brief description that includes type and default information for each `set_metric_alias` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_metric_alias`.
`-no_overwrite`	Specifies to not overwrite an already set alias.
`-order order_of_priority`	Specifies the order of priority of metric object in tables.
`-to new_name`	Specifies the new name for the metric object.

Example

- The following command renames `path_group:reg2reg` to `reg2reg` with priority order `1`.

  ```
  > set_metric_alias -from path_group:reg2reg -to reg2reg -order 1
  ```

- The following command renames `path_group:reg2cgate` to `reg2cgate` with priority order `2`.

  ```
  > set_metric_alias -from path_group:reg2cgate -to reg2cgate -order 2
  ```

- The following command queries the set of aliases:

  ```
  > get_metric_alias
  ```

  Output

  ```
  >path_group:reg2reg {to reg2reg order 1} path_group:reg2cgate {to reg2cgate order 2}
  ```

Related Topic

- `get_metric`
- `get_metric_alias`

set_metric_header

```
set_metric_header
-name header_name
[-id db_id]
-help
-value header_value
```

Sets the name of a metric header.

Parameters

`-id db_id`	Specifies the database ID.
`-name header_name`	Specifies the name of the header.
`-help`	Outputs a brief description that includes type and default information for each `set_metric_header` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_metric_header`
`-value header_value`	Specifies the value of the header.

Related Topics

- `get_metric_header`

set_preference

```
set_preference
[-help]
preference_name
value
```

Defines preferences, such as, commands issued during a session can be logged to a log or the screen.

When writing to a log and screen, the commands are preceded by <CMD>.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_preference` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_preference`.
`preference_name`	The name of the preference variable you want to set. To see a list of preferences, select ***View - Set Preference*** from the menu, then click *Save*. In the Save Preferences form, select the required save option and click *OK*. If you select the default *Save to Home Directory* option, the list of preferences and their values are saved in `.enc` file in your home directory.
`value`	The value for the preference. For example, the `CmdLogMode` preference has the following values. `0`--write to command only `1`--write to command and log (default) `2`--write to command, log, and screen

Preference Variables

The following table lists the preference variables you can set using the `set_preference` command and their GUI equivalent in the Preferences form:

Note: The following table might not be readable when viewed with the Unix man command. Check the `set_preference` command description in the *Tempus Text Command Reference* to view the table.

Variable Name	GUI Equivalent in Preferences Form	
	Tab Name	Option Name
`DesignName`	***Design***	*Design Name*
`DesignHierChar`		*Design Hierarchical Character*
`DEFHierChar`		*DEF Hierarchical Character*
`PDEFBusDelim`		*PDEF Bus Delimiter*

LecDofile		*Write Conformal/LEC dofile When Design is Saved*
CmdLogMode		*Command Log Mode* ◦ *Command Only* ◦ *Command and Log* ◦ *Command, Log, and Screen*
LogTypeInCmd		*Log Type-In Command*
logviewer		*Invoke viewLog at Start Up*
EnableRectilinearDesign		*Enable Rectilinear Design*
ShowFPObjInPlace	***Display***	*Show Floorplan Objects in Physical View*
ShowAllFence		*Show All Constraint*
DisplayPtnPin		*Display Partition Pin*
ShowAttrPopup		*Show Attributes Popup*
ShowUnplacInnovusnst		*Display Unplaced Instances*
ShowRectilinearPad		*Show Rectilinear Pad Cell*
DisplayCellPin		*Display Instance Pin*
ShowLefLayerName		*Show Lef Layer Name*
AutoRedraw		*Auto-redraw after Layer Change*
EnlargeLogicalPin		*Enlarge Logical Pin In Fit View*
InstDisplayThreshold		*Instance Display Threshold*
WirInnovussplayThreshold		*Wire Display Threshold*
WirInnovussplayCheck2D		*Apply Threshold in 2D*
percentageOfPan		*Percentage of Window to Pan*
scaleOfZoom		*Scale of Zoom In/Out*
zoomPrevCount		*Number of Previous Zoom Saved*
MinFPModuleSize		*Min. Floorplan Module Size*
ShowParentModule		*Show Parent Module With Level*
ShowChildPartition		*Show Child Partition With Level*

`AutoDetailDisplay`		*Automatic Detail Display for Large Design* ◦ Special Wire ◦ Regular Wire
`DetailDisplayFactor`		*Factor*
`DisplayRelFPlan`	***Innovust***	*Honor Relative Floorplan Constraints*
`SingleLayerMode`		*Single Layer Mode For Routing Blockage, Ptn Pin Blockage and Ptn Feed*
`ShowDelBox`		*Show Delete Confirmation Box*
`StrecthRestriction`		*Box Stretch Restrictions* ◦ *No Restriction* ◦ *Maintain Area* ◦ *Maintain Aspect Ratio*
`StretchRectilinear`		*Rectilinear Stretch Restrictions* ◦ *No Restriction* ◦ *Maintain Area*
`MoveRestriction`		*Move Restrictions* ◦ *No Restrictions* ◦ *Orthogonal*
`GuideSnapRule`	***Floorplan***	*Snap Guides/Regions/Fences to* ◦ *Std Row and Metal 2 Pitch* ◦ *Manufacture Grid* ◦ *User-defined Grid* ◦ *Block Placement Grid*
`BlockSnapRule`		*Snap Macros/Blackboxes to* ◦ *Manufacture Grid* ◦ *Std Row and Metal 2 Pitch* ◦ *User-defined Grid* ◦ *Block Placement Grid*
`SnapAllCorners`		*Snap All Corners to Grid*
`ConstraintUserXGrid`		*User-defined Grid:* ◦ *X*

ConstraintUserXOffset		◦ *Offset*
ConstraintUserYGrid		◦ *Y*
ConstraintUserYOffset		◦ *Offset*
UserGridUnit		*Unit* ◦ *Micron* ◦ *Track*
MoveMacrosWithGuide		*Descendant Macros Move with Their Ancestor Modules for Constraints* ◦ *Guide*
MoveMacrosWithRegion		◦ *Region*
MoveMacrosWithFence		◦ *Fence*
MoveStdCellWithGuide		*Descendant Std Cell Move with Their Ancestor Modules for Constraints* ◦ *Guide*
MoveStdCellWithRegion		◦ *Region*
MoveStdCellWithFence		◦ *Fence*
MovePreplacedStdCellOnly		*Move Preplaced Cell Only*
clone_row_orientation_snapping		*Clones snapping to same master row orientation*
SelectByArea	***Selection***	*Select by Area* ◦ *Enclosed by Box* ◦ *Intersecting Box*
SelectByLine		*Select by Line*
SelectStickyMode		*Sticky Mode*
SelectNetWhenSelectPin		*Hilite Net When Selecting a Pin*
HiliteNetWhenQueryObj		*Hilite Nets When Querying an Instance*
HiliteShapeWhenSelectNet		*Hilite Pin Shapes When Selecting a Net*
QueryWireNet		*Query Nets When Wires are Selected*

`QuerySkipInst`	*Skip Objects by Auto Query* ◦ *Cell*
`QuerySkipInstObs`	◦ *Obs in Cell*
`QuerySkipInstPin`	◦ *Pin in Cell*
`QuerySkipRegular`	◦ *Regular Wire*
`QuerySkipSpecial`	◦ *Special Wire*
`WinSelectMargin`	*Window Select Margin*
`HighlightColorNumber`	*Number of Highlight Color*

MinFlightLineWidth	***Flightline***	*Minimum Flightline Width*
MaxFlightLineNetTerms		*Maximum Flightline Net Terms*
DisplayPlaceFlightLine		*Show Flightline In Physical View*
ShowNumberBlockConnection		*Show Number of Connections For Block*
FlightLineInMove		*Draw Flightline When Moving Instance(s)*
ShowConnectionInOutNumber		*Show Input/Output Connection Number*
ShowFlightLineTermMark		*Show Pin's Input/Output Mark*
ShowBothInputConnection		*Show Both Input Pins Connection*
ShowNetWeightConnection		*Show Connection With Netweight 0*
ShowFlightLineThruPtnPin		*Show Flightlines Through Partition Pins*
noInsideMacro		*Don't Show Flightline to Inside Macro*
SkipBufferFlightline		*Skip Buffer/Invert Cell*
NoClockFlightline		*Don't Show Clock Net*
OnlyClockFlightline		*Show Clock Net Only*
DrawFlightlineLast		*Draw Flightline on Top of Selection*
OnlyBundleNetFlight		*Only Show Net Bundle Flight Line*
ExclusiveFlight		*Show Connection Only Between Selected Objects*
ConnectionColorType		*Single Color* *Different Color by Type of Connections* *Different Color by Number of Connections*
InputConnectionColor		• *Different Color by Type of Connections: Input* • *Different Color by Number of Connections: Small*
OutputConnectionColor		• *Different Color by Type of Connections: Output* • *Different Color by Number of Connections: Medium*
InoutConnectionColor		• *Different Color by Type of Connections: Inout* • *Different Color by Number of Connections: Large*
MixtureConnectionColor		• *Single Color: <color>* • *Different Color by Type of Connections: Inout*

ShowConnectionWithWidth		*Different Width by Number of Connections*
FLWidthThresholdLow		*Number of Connections* ◦ *Small* - Fewer than specified low threshold
FLWidthThresholdHigh		◦ *MInnovusum* - Equal to or between than specified low and high thresholds ◦ *Large* - Greater than specified high threshold
NoInstFlightLine		*Don't Show Connection to Unselected Object* ◦ *Instance*
NoBlockFlightLine		◦ *Block*
NoModuleFlightLine		◦ *Module*
NoBlackBlobFlightLine		◦ *Black Blob*
NoBlackBoxFlightLine		◦ *Black Box*
InstanceText	***Text***	*Instance Text* ◦ *Instance Name* ◦ *Master Name*
ShowNetFullName		*Show Full Net Name*
ShowNetNameWithLayerColor		*Show Net Name With Layer Color*
ShowModuleText		*Object Text Display* ◦ *Module*
ShowAmoebaModuleText		◦ *Module in Amoeba*
ShowRowSiteText		◦ *Row Site*
ShowIoPadText		◦ *IO Pad*
ShowInstanceText		◦ *Instance*
ShowInstancePinText		◦ *Instance Pin*

ShowBlackBlobText		◦ *Black Blob*
ShowIoPinText		◦ *IO Pin*
ShowGroupText		◦ *Group*
ShowBumpText		◦ *Bump*
ShowClockTreeText		◦ *Clock Tree*
ShowChannelText		◦ *Channel*
ShowLefPortNumText		◦ *Lef Port Num*
ShowMacroSitePtnText		◦ *Macro Site Ptn*
ShowSIPFingerText		◦ *SIP Finger*
ShowNetText		◦ *Net*
TextDisplaySize		*Text Display Size* ◦ Small ◦ MInnovusum ◦ Large ◦ Auto
TermCrossPix		*Term Cross Symbol Size*

The following table lists the preference variables you can set using the `set_preference` command and their type, range/default/sample value.

Note: The following table might not be readable when viewed with the Unix man command. Check the `set_preference` command description in the *Tempus Text Command Reference* to view the table.

Variable Name	Type	Range/Default/Sample Value (Default values in bold)
DesignName	String	/spc/spc4/FE_tests_7.1/designs.. ../dtmf_pso/dtmf_fpd.enc.dat
DesignHierChar	String	/
DEFHierChar	String	/
PDEFBusDelim		[]

LecDofile	Boolean	**0 – False** 1 – True
CmdLogMode	Enum	*0 – Command Only* ***1 – Command and Log*** *2 – Command, Log, and Screen*
LogTypeInCmd	Boolean	**0 – False** 1 – True
logviewer	Boolean	**0 – False** 1 – True
EnableRectilinearDesign	Boolean	**0 – False** 1 – True
ShowFPObjInPlace	Boolean	**0 – False** 1 – True
ShowAllFence	Boolean	**0 – False** 1 – True
DisplayPtnPin	Boolean	0 – False **1 – True**
ShowAttrPopup	Boolean	0 – False **1 – True**
ShowUnplacInnovusnst	Boolean	**0 – False** 1 – True
ShowRectilinearPad	Boolean	**0 – False** 1 – True
DisplayCellPin	Boolean	**0 – False** 1 – True
ShowLefLayerName	Boolean	0 – False **1 – True**
AutoRedraw	Boolean	0 – False **1 – True**
EnlargeLogicalPin	Boolean	**0 – False** 1 – True

InstDisplayThreshold	Real	**0.0**
WirInnovussplayThreshold	Integer	**0**
WirInnovussplayCheck2D	Boolean	0 – False **1 – True**
percentageOfPan	Enum	10, 20, 30, 40, 50, 60, 70, 80, 90, **95**, 100
scaleOfZoom	Real	**2.0**
zoomPrevCount		**1** to 6
MinFPModuleSize	Integer	**100**
ShowParentModule	Integer	**0**
ShowChildPartition	Integer	**0**
AutoDetailDisplay	Boolean	**0 – False** 1 – True
DetailDisplayFactor	Integer	**16**
DisplayRelFPlan	Boolean	0 – False **1 – True**
SingleLayerMode	Boolean	**0 – False** 1 – True
ShowDelBox	Boolean	**0 – False** 1 – True
StrecthRestriction	Enum	**0 – No Restriction** 1 – Maintain Area 2 – Maintain Aspect Ratio
StretchRectilinear		**0 – No Restriction** 1 – Maintain Area
MoveRestriction		**0 – No Restrictions** 1 – Orthogonal
GuideSnapRule	Enum	**stdRow – *Std Row and Metal 2 Pitch*** *user-define – User-defined Grid*

BlockSnapRule	Enum	**0 – Manufacture Grid** 1 – Std Row and Metal 2 Pitch 2 – User-defined Grid 3 – Placement Grid
SnapAllCorners	Boolean	**0 – False** 1 – True
ConstraintUserXGrid	Real	**0.0**
ConstraintUserXOffset	Real	**0.0**
ConstraintUserYGrid	Real	**0.0**
ConstraintUserYOffset	Real	**0.0**
UserGridUnit	Boolean	**0 – Micron** 1 – Track
MoveMacrosWithGuide	Boolean	**0 – False** 1 – True
MoveMacrosWithRegion	Boolean	**0 – False** 1 – True
MoveMacrosWithFence	Boolean	0 – False **1 – True**
MoveStdCellWithGuide	Boolean	**0 – False** 1 – True
MoveStdCellWithRegion	Boolean	**0 – False** 1 – True
MoveStdCellWithFence	Boolean	**0 – False** 1 – True
MovePreplacedStdCellOnly	Boolean	**0 – False** 1 – True
clone_row_orientation_snapping	Boolean	**0 – False** 1 – True
SelectByArea	Boolean	**0 – Enclosed by Box** 1 – Intersecting Box

SelectByLine	Boolean	**0 – False** 1 – True
SelectStickyMode	Boolean	**0 – False** 1 – True
SelectNetWhenSelectPin	Boolean	**0 – False** 1 – True
HiliteNetWhenQueryObj	Boolean	0 – False **1 – True**
HiliteShapeWhenSelectNet	Boolean	**0 – False** 1 – True
QueryWireNet	Boolean	**0 – False** 1 – True
QuerySkipInst	Boolean	**0 – False** 1 – True
QuerySkipInstObs	Boolean	**0 – False** 1 – True
QuerySkipInstPin	Boolean	**0 – False** 1 – True
QuerySkipRegular	Boolean	**0 – False** 1 – True
QuerySkipSpecial	Boolean	**0 – False** 1 – True
WinSelectMargin	Integer	**8**
HighlightColorNumber	Enum	**8**, 12, 16
MinFlightLineWidth	Integer	**1**
MaxFlightLineNetTerms	Integer	**500**
DisplayPlaceFlightLine	Boolean	0 – False **1 – True**
ShowNumberBlockConnection	Boolean	**0 – False** 1 – True

FlightLineInMove	Boolean	0 – False **1 – True**
ShowConnectionInOutNumber	Boolean	**0 – False** 1 – True
ShowFlightLineTermMark	Boolean	**0 – False** 1 – True
ShowBothInputConnection	Boolean	**0 – False** 1 – True
ShowNetWeightConnection	Boolean	0 – False **1 – True**
ShowFlightLineThruPtnPin	Boolean	**0 – False** 1 – True
noInsideMacro	Boolean	**0 – False** 1 – True
SkipBufferFlightline	Boolean	**0 – False** 1 – True
NoClockFlightline	Boolean	**0 – False** 1 – True
OnlyClockFlightline	Boolean	**0 – False** 1 – True
DrawFlightlineLast	Boolean	**0 – False** 1 – True
OnlyBundleNetFlight	Boolean	**0 – False** 1 – True
ExclusiveFlight	Boolean	0 – False **1 – True**
ConnectionColorType	Enum	**0 – Single Color** 1 – Different Color by Type of Connections 2 – Different Color by Number of Connections

InputConnectionColor		**green**
OutputConnectionColor		**yellow**
InoutConnectionColor		**pink**
MixtureConnectionColor		**blue**
ShowConnectionWithWidth	Boolean	**0 – False** 1 – True
FLWidthThresholdLow	Integer	**20**
FLWidthThresholdHigh	Integer	**80**
NoInstFlightLine	Boolean	**0 – False** 1 – True
NoBlockFlightLine	Boolean	**0 – False** 1 – True
NoModuleFlightLine	Boolean	**0 – False** 1 – True
NoBlackBlobFlightLine	Boolean	**0 – False** 1 – True
NoBlackBoxFlightLine	Boolean	**0 – False** 1 – True
InstanceText		**Instance** Master
ShowNetFullName	Boolean	**0 – False** 1 – True
ShowNetNameWithLayerColor	Boolean	**0 – False** 1 – True
ShowModuleText	Boolean	0 – False **1 – True**
ShowAmoebaModuleText	Boolean	0 – False **1 – True**

ShowRowSiteText	Boolean	0 – False **1 – True**
ShowIoPadText	Boolean	0 – False **1 – True**
ShowInstanceText	Boolean	0 – False **1 – True**
ShowInstancePinText	Boolean	0 – False **1 – True**
ShowBlackBlobText	Boolean	0 – False **1 – True**
ShowIoPinText	Boolean	0 – False **1 – True**
ShowGroupText	Boolean	0 – False **1 – True**
ShowBumpText	Boolean	0 – False **1 – True**
ShowClockTreeText	Boolean	0 – False **1 – True**
ShowChannelText	Boolean	0 – False **1 – True**
ShowLefPortNumText	Boolean	0 – False **1 – True**
ShowMacroSitePtnText	Boolean	0 – False **1 – True**
ShowSIPFingerText	Boolean	0 – False **1 – True**
ShowNetText	Boolean	0 – False **1 – True**
TextDisplaySize	Enum	**s**, m, l, a
TermCrossPix	Enum	**4**, 6, 8

Example

Writes log to command, log, and screen:

```
set_preference CmdLogMode 2
```

set_proc_verbose

```
set_proc_verbose
[-help]
{[procedure_name [-quiet ]] | [-report ]}
```

Makes a procedure verbose.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_proc_verbose` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_proc_verbose`.
`procedure_name`	Specifies the name of the procedure to be made verbose.
`-quiet`	Changes the procedure back to not being verbose.
`-report`	Reports procedures that are verbose.

Examples

- The following commands sets `myprocedure` as verbose:

  ```
  > set_proc_verbose myprocedure
  myprocedure
  ```

- The following commands reports `myprocedure` as the verbose procedure:

  ```
  > set_proc_verbose -report
  myprocedure
  ```

source

```
source
[-help]
filename
[-quiet | verbose]
[-quiet  | -echo ]
```

Reads a file and executes the commands in the file (scripts).

Parameters

`-help`	Describes the command usage.
`-echo`	Enables verbose source.
`filename`	Specifies the name of the file to source.
`-quiet`	Forces source to be normal Tcl source command. **Note:** This is optional.
`-verbose`	Forces source to be in verbose mode. **Note:** This is optional.

suspend

```
suspend [-help]
```

Suspends your script and returns to the Voltus prompt.

The `suspend` command gives you more flexibility in managing and debugging your scripts. You can insert multiple `suspend` commands in a script. The `suspend` command can also be used in nested scripts. When the script reaches a point where you have inserted the `suspend` command, it stops and restores access to the Voltus prompt. You can then type any command required for debugging.

Whenever you want to resume your script, just type `resume` at the Voltus prompt.

Parameters

`-help`	Prints the usage of the `suspend` command. For a detailed description of the command, use the `man` command: `man suspend`.

Related Topics

- `resume`

system Category Attributes

The root attributes in the system category are:

- auto_file_dir
- auto_file_prefix
- distributed_mmmc_disable_reports_auto_redirection
- gui_verbose
- log_verbose_type
- read_db_directory
- read_db_file_check
- read_db_stop_at_design_in_memory
- read_db_tool_name
- read_db_version
- script_search_path
- source_echo_filename
- source_continue_on_error
- set_db_verbose
- soft_stack_size_limit
- source_verbose
- source_verbose_line_length_limit
- ui_precision
- tcl_error_info
- ui_precision_capacitance
- ui_precision_derating
- ui_precision_power
- ui_precision_sensitivities
- ui_precision_timing
- write_db_auto_save_user_globals

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `system` category, use the following command:

```
get_db -category system
```

auto_file_dir

```
auto_file_dir directoryName
```

Default: ./

Specifies the directory to store files/sub-directories 'auto-generated' by the tool. This includes report files or other output files with default names that are automatically created by some commands in the current run directory. Default names that have been overridden with a name chosen by the user, are not affected. This allows multiple simultaneous runs from the same run directory to direct output files to different result directories to avoid collisions.

auto_file_prefix

```
auto_file_dir fileName
```

Default: ""

Specifies a prefix to be applied to all files 'auto-generated' by the tool. This includes report files or other output files with default names that are automatically created by some commands in the current run directory. So a default name like "cmd.report" will become "<prefix>cmd.report". Default names that have been overridden with a name chosen by the user, are not affected. This allows multiple simultaneous runs from the same run directory to direct default output files to different locations to avoid collisions.

distributed_mmmc_disable_reports_auto_redirection

```
distributed_mmmc_disable_reports_auto_redirection {true| false}
```

Default: `false`

Specifies the unique path for each view to avoid all clients writing to same report file. With default setting, the distributed mmmc run automatically updates report paths to a view unique directory structure. When set to true, it will preserve path specified in the report command.

gui_verbose

```
gui_verbose {all db none}
```

Default: `all`

Controls which GUI actions are logged. You can select:

- `all`: Logs all GUI actions.
- `db`: Logs the GUI actions that affect the DB, such as `gui_select`, but does not capture display-only actions such as `gui_dim_foreground`.
- `none`: Does not log any GUI actions. So, the .log and .cmd files are incomplete.

log_verbose_type

```
log_verbose_type {pre post both}
```

Default: `pre`

When verbose logging is enabled, the default logging of each commands is pre-Tcl expansion. In some cases it is helpful to see post-Tcl expansion, or even both pre-Tcl and post-Tcl expansion when doing detailed debugging. You can switch between settings in the middle of a script.

read_db_directory

```
read_db_directory directory_name
```

Default: ""

Specifies the directory of the current loaded database.

read_db_file_check

```
read_db_file_check {true | false}
```

Default: `true`

By default, `read_db` will check all the files inside the saved DB directory. If any file is edited, renamed, or deleted, the tool will give an error. However, read_db will not perform this check if `read_db_file_check` is set to `false`.

read_db_stop_at_design_in_memory

```
read_db_stop_at_design_in_memory {true | false}
```

Default: `true`

By default, tool gives error and stops if you call `read_db` more than once in the same Innovus session unless `read_db_stop_at_design_in_memory` is set to `false`. Using `read_db` twice in one Tool session is not recommended as settings from the first database may not be cleaned up properly and can cause problems in the second database.

read_db_tool_name

read_db_tool_name

Default: "."

Indicates the tool name of the tool that created the restored db on the disk database.

read_db_version

read_db_version

Default: "."

Indicates the version of the tool that generates an on disk database.

script_search_path

`script_search_path` *`search_directory_path`*

Default: "."

Provides a set of directories for the software to search for files that are sourced using the Tcl `source` command. The software will search in each search path directory for the specified file.

source_echo_filename

`source_echo_filename` *`value`*

Default: `0`

Displays the file name being sourced before the actual sourcing.

source_continue_on_error

`source_continue_on_error {true| false}`

Default: `false`

Enable or disables `source` to continue the script on TCL_ERROR. If `true`, the `source` command will ignore Tcl errors, and continue processing the script files. If `false`, a Tcl error will halt the script.

set_db_verbose

`set_db_verbose {true| false}`

Default: `false`

Specifies additional messages are issued describing what attribute is being set on what object.

soft_stack_size_limit

`soft_stack_size_limit` *`size_limit`*

Default: `10`

Specifies the soft stack size limit in mbytes. When the tool is launched, the the soft stack size limit is auto-enlarged to 0.2% RAM. If 0.2%RAM is larger than the hard limit, the tool sets it to the hard limit.

source_verbose

`source_verbose {true| false}`

Default: `true`

Controls whether scripts are sourced in quiet or verbose mode:

- By default (`true`), files are sourced in verbose mode. This is equivalent to `source` *`filename`* `-verbose`.

- If you set this variable to `false`, files are sourced in quiet mode. This is equivalent to `source` filename `-quiet`.

source_verbose_line_length_limit

`source_verbose_line_length_limit` *`value`*

Default: `-1`

Data_type: int

Limits the length of the verbose source Tcl commands that are logged. -1 means there is no limit. If you use the default pre-Tcl expansion logging, then -1 is generally the best. If you turn on post-Tcl expansion logging (see), a command such as “set a [get_db insts]” will by expanded by Tcl into “set a inst:top/i1 inst:top/i2 …”, and log a list of every inst that will often make the log file too big to be useful. You can limit the length of the longest command line to avoid that. This is different than the Tcl output length limit which only affects the length of the output echoed by Tcl when typing at the prompt.

ui_precision

`ui_precision` *number*

Default: 3

Specifies the number of significant digits to be displayed in the absence of an explicit precision for that type.

tcl_error_info

`tcl_error_info {true| false}`

Default: `false`

Data_type: bool, read/write

When set to `true`, the `tcl_error_info` prints the error information if there is an issue in the Tcl script.

ui_precision_capacitance

`ui_precision_capacitance` *`number`*

Default: 3

Specifies the number of significant digits to be displayed for quantities of type capacitance.

ui_precision_derating

`ui_precision_derating` *`number`*

Default: 3

Specifies the number of significant digits to be displayed for derating factors such as those specified via set_timing_derate constraints or AOCV derating libraries.

ui_precision_power

`ui_precision_power` *number*

Default: 3

Specifies the number of significant digits to be displayed for quantities of type power.

ui_precision_sensitivities

`ui_precision_sensitivities` *number*

Default: 3

Specifies the number of significant digits to be displayed for statistical sensitivity values and SOCV sigma values.

ui_precision_timing

`ui_precision_timing` *number*

Default: 3

Specifies the number of significant digits to be displayed for delay type values including cell and net delays, transitions, arrival and required times, and slacks.ui_precision ui_precision_capacitance

write_db_auto_save_user_globals

`write_db_auto_save_user_globals {true| false}`

Default: `true`

If set to true, `write_db` automatically saves user created Tcl global variables. User globals are defined as any "info globals" names that do not exist at startup (e.g. created by user executed code).

uniquify_filename

```
uniquify_filename
[-help]
filename
```

Uniquifies a file.

Parameters

filename	Specifies the name of the file to uniquify.
-help	Prints a brief description that includes type and default information for each `uniquify_filename` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man uniquify_filename`

update_inst_name

```
update_inst_name
-inst instName
-new_base_name baseName
```

Changes the base name of the specified instance to the given base name.

Note: The `update_inst_name` command does not change the path of hierarchy.

Parameters

`-inst instName`	Specifies the name of the instance.
`-new_base_name baseName`	Specifies the new name for the instance.

Example

The following example changes the instance name from `h3/inv3` to `h3/h2/inv1`:

```
update_inst_name -inst h3/inv3 -new_base_name h2/inv1
```

update_metric

```
update_metric
[-help]
[-format {html excel text}]
[-graph_type {treemap line none}]
[-group group_title]
[-header header_name]
[-link_file_metric metric_name]
[-link_table table]
[-metric metric_name]
[-no_shared_max]
[-renderer html_format]
[-show_graph]
-table table_id
[-title html_table_title]
```

Updates a metric.

Parameters

`-format {html excel text}`	Specifies configuration for the specified `report_metric` format.
`-graph_type {treemap line none}`	Specifies the type of graph to use in the table (treemap |line |none). *Default*: `line`
`-group` *`group_title`*	Specifies the group title to be used in the html table.
`-header` *`header_name`*	Specifies the header name to be added.
`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man update_metric`
`-metric` *`metric_name`*	Specifies the metric name to be added.
`-no_shared_max`	When specified for a `-renderer` histogram metric, the Y axis of all the histograms in a single column stop sharing the same max Y value.
`-link_file_metric` *`metric_name`*	Specifies the name of the metric containing filename to open when the metric value is clicked.
`-link_table` *`table`*	Specifies the table to scroll to when the metric value is clicked.
`-renderer` *`html_format`*	Forces a specific html format renderer for the histogram metric.
`-show_graph`	Specifies whether to show this metric in a comparison graph.
`-table` *`table_id`*	Specifies the id of table to which the metric will be added.
`-title` *`html_table_title`*	Specifies the title to be used in the html table.

Related Topics

- `report_metric` command

update_metric_definition

```
update_metric_definition
[-help]
[-compare compareFunction]
[-description metricDescription]
[-inheritance_tcl TclName]
-name metric_name
[-no_inheritance]
[-precision number_of_decimal_places]
[-target metric_name]
[-tcl {TCL_command_list}]
[-threshold reporting_unit [-tools listOfTools]]
[{-id databaseID}
[-type metricType]
[-units reporting_unit]
[-warning threshold]
```

Updates the specified properties of an already-existing metric definition with the same name.

Note: If a metric does not exist with the name, the command returns an error.

Parameters

`-compare compareFunction`	Specifies the compare function, `[moreBetter|lessBetter|moreBetterBelowTarget|lessBetterAboveTarget]`. The comparison options show the good or bad color when the value is below or above the target value as appropriate. *Unit*: `string`
`-description metricDescription`	Specifies the description of the metric. *Unit*: `string`
`-inheritance_tcl TclName`	Specifies Tcl for combining children values. The metric is not executed in the global namespace. The equivalent proc args is {children}. *Unit*: `string`
`-name metric_name`	Specifies the name of the metric. *Unit*: `string`
`-no_inheritance`	Disables the inheritance of the metric. *Unit*: `boolean`

`-help`	Outputs a brief description that includes type and default information for each `update_metric_definition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man update_metric_definition`
`-id` *databaseID*	Specifies the database ID.
`-precision` *number_of_decimal_places*	Specifies the number of decimal places. *Unit*: `integer`
`-target` *metric_name*	Specifies the target or the name of metric containing the target value.
`-tcl { TCL_command_list }`	Specifies the list of TCL commands or proc to generate (return) the metric which is of the format `{<value> <unit>}`. *Unit*: `string`
`-threshold` *reporting_unit*	Specifies the threshold for comparing numbers in HTML. If the delta between the two numbers being compared is less than the threshold then the metric will be colored “yellow”. If it is outside the threshold, then it is colored red or green depending on the compare function i.e. whether more is better or less is better.
`-tools` *listOfTools*	Specifies the list of tools in which the tcl code will run.

`-type` *metricType*	Specifies the type of metric.
	Unit: `string`
	The various metrics types are:
	• imageplot: A metric type that the renders to an image in the HTML report.
	• Format:
	`<name1> { \`
	` fill_color "<color>" \`
	`  stroke_color "<color>"\`
	`  data <list of data sets>`
	`} \`
	`<name2> { \`
	`<sub-name1> { \`
	`  fill_color <color> \`
	`  data <list of data sets> \`
	`  } \`
	`  <sub-name2> {...} \`
	`}`

Where a data set can be one or more of the following:

The data on each layer is a string of coordinates (separated by sequences of whitespace and comma) with types. These types are:

- R - rectangle (x, y, width, height)
- B - bounding box (x_min, y_min, x_max, y_max)
- G - closed polygon (x, y, x, y, ...)
- L - open polyline (x, y, x, y, ...)
- C - circle (x, y, r)
- P - point (x, y)

For rectangles, boxes, circles and points, multiple shapes can be defined without having to repeat the type letter. e.g. R1,2,3,4,5,6,7,8 is two rectangles.

A value can be defined for shapes by adding a V suffix to the type; this value can then be used to color the shapes according to the value for heat maps. The value is always the first number after the type. This means that two points with values 20 and 40 may look like: PV20,1,2 40,6,5 or: PV20,1,2PV40,6,5.

Example imageplot TCL dict data:

```
macros { \
fill_color "#FF0000" \
  stroke_color "#0000FF88" \
  data "R5,8,1,2 6,9,2,1 G1,2,3,4,5,6,7,8" \
} \
drcs { \
placement { \
    fill_color "#FF0000" \
    data "P3,4" \
  } \
  routing {...} \
}
```

Layers in the image are named (macros, congestion, normal, other in the above example) and support hierarchy. Layers should not be nested beyond one level deep. The hierarchy is used in the renderer (see below) to toggle the visibility of the layers. It is an error for any layer to be called data (reserved to keep hierarchy clear).

Colors can be defined either as hex or RGB values with optional opacity. Hex values are prefixed with #. RGB values are separated with whitespace.

- #FF0000: Red
- #00FF0040: Green (25% opaque)
- 0 0 255: Blue
- 255 0 255 192: Magenta (75% opaque)

- histogram: The data will be structured as a set of series with shared X-axis labels that are rendered as a stacked histogram.

  Format (TCL dict):

```
labels <list of labels> \
axis_x <name> \
axis_y <name> \
series { \
<series1> { \
   values <list of buckets \
   color "#FF0000" \
} \
<series2> { \
   values {2 1 0 0} \
   color "#0000FF" \
} \
}
```

  Example:

```
get_metric timing.setup.histogram
timing.setup.histogram {labels {-0.320 -0.240 -0.160 -0.080} \
                        series { \
                          reg2reg {color #FF0000 values {43 3 2
3 0}} \
                          reg2cgate {color #0000FF values {22 4
5 0 0}} \
                        } \
                        axis_x WNS(ns) \
                        axis_y {Failing End Points} \
                        }
```

  The labels (a list of strings, shown in the order provided) define the X-axis labels for the histogram, the values in each series define the Y-axis values.

	Colors can be defined either as hex or RGB values with optional opacity. Hex values are prefixed with #. RGB values are separated with whitespace.
	`#FF0000 - Red`
	`0 0 255 - Blue`
	• html: Supports the inclusion of raw HTML in a metric. You can place simple html into tables allowing simple links to reports. For example, the following command defines `test_html` as an html type of metric:
	`define_metric -name test_html -type html`
	You can the html metric, test_html, using the `set_metric` command:

`-units` *`reporting_unit`*	Specifies the unit for reporting. *Unit*: `string`
`-warning` *`threshold`*	Specifies the warning threshold or name of metric containing the threshold value.

Examples

The following command defines the `design.vtmix` metric :

```
update_metric_definition -name design.vtmix -tcl {return [TCL commands]}
```

view_log

```
view_log
[-help]
[file_name logFileName]
```

Opens up a log file in a separate console window for viewing within Voltus.

The log file viewer within the software opens the current log file by default, and updates it in real time. If you specify a log file name, the software opens up the specified log file.

The log file displayed contains `[+]` and `[-]` markers that expand and collapse command information, so that you can control how much detail you want to read. Log messages are color coded for easier identification: blue messages are warnings, red messages are errors.

You can access documentation for commands in the log file that are underlined. Click on the command in the file, and the software opens an HTML window displaying the *Voltus Text Command Reference* documentation for that command.

You also can access the log file viewer from the *Tools Menu - Log Viewer*.

Note: You can specify `view_log` more than once to view multiple log files in separate console windows simultaneously. However, you cannot open multiple versions of the current log file.

Parameters

`file_name` *logFileName*	Specifies the name of the log file to open.
`-help`	Outputs a brief description that includes type and default information for each `view_log` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man view_log`.

write_metric

```
write_metric
[pattern]
[-format output_format]
[-id metric_id]
[-merge]
[-names list_of_patterns]
{-out_file fileName }
```

Writes the current set of metrics to a file or stdout.

The write_metric command is used by `write_db` to write the metrics in the JSON format in the database directory. Super commands use `write_metric` to write metrics into the log file (stdout). This can also be used to write metrics in various formats by the user.

Parameters

`-out_file fileName`	Specifies the name of the file in which metrics is written.
`-format output_format`	Specifies the output format, json, xml, csv, tcl. *Default*: JSON
`-help`	Outputs a brief description that includes type and default information for each `write_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_metric`
`-id metric_id`	Specifies a list of labels from read_metrics. *Default*: Current metric
`-merge`	Merges the results into an existing file.
`-names list_of_patterns`	Specifies the list of patterns to match snapshot names.
`pattern`	List of regular expressions to chose metrics to write. Default is all metrics (*)

Example

The following code shows the usage of the `write_metric` command:

```
# ------------------------------------------------------------------
# Generate a XML file for all timing, power, and flow metrics from
# from prects to postcts for a previous run (id v101)
# ------------------------------------------------------------------
```

```
read_metric -id v101 run101/dbs/postcts.enc.dat/design.metrics

write_metric -id v101 -format xml
```

Related Topics

- write_db

write_metric_config

```
write_metric_config
[-help]
 fileName
[-format {html excel text}]
```

Writes the YAML structure for the requested format (text, html, or excel) to a file. As the configuration is written in YAML format, you can customize the page layout and table content you need in the report such as adding pages and tables, or adding new metrics to existing tables, removing metrics from existing tables, and rearranging the metric ordering.

Parameters

fileName	Specifies the name of the file in which metric config is written.
`-format {html excel text}`	Specifies the config for specified `report_metric` format.
`-help`	Outputs a brief description that includes type and default information for each `write_metric_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_metric_config`

Example

generates the report in the text format.

```
> write_metric_config -format text small.text ; # Edit the summary table
> read_metric_config -format text small.yaml
> report_metric  -format text
```

Related Topics

- `read_metric_config`
- `report_metric`

rename_snapshot

```
rename_snapshot
[-help]
[-branch newBranch]
[-id metricID]
[-name newName]
-uuid snapshotuuid
```

Allows you to rename existing snapshots.

`rename_snapshot` does not support canonical paths and cannot change the location of a snapshot within the hierarchy.

Parameters

`-branch` *newBranch*	Specifies the new branch for the snapshot. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `rename_snapshot` parameter. For a detailed description of the command and all of its parameters, use the man command: `man rename_snapshot`
`-id` *metricID*	Specifies the metric database ID to change. If you do not specify any id, the tool will take current database as default. *Data_type*: string, optional **Note**: You can find the uuid of a snapshot by using `get_metric` for the uuid metric.
`-name` *newName*	Specifies the new name for the snapshot. The tool will error out if you set the name to empty string. *Data_type*: string, optional
`-uuid` *snapshotuuid*	Specifies the snapshot UUID. The tool will error out if there is no snapshot with the provided uuid. *Data_type*: string, required

Related Information

- `get_metric`

delete_floating_constants

```
delete_floating_constants
-help
```

Deletes dangling 1'b1s or 1'b0s from the Verilog. They are either dangling nets assigned 1'b1 or 1'b0 or 1'b1/1'b0 connections that are not hierarchically connected to a leaf instance.

Parameters

None

Example

In the code below only BUFX2 is a leaf.

```
//
// Original Verilog
module hier4 (A,B);
input A;
input B;

BUFX2 u0 (.A(A));
BUFX2 u1 (.A(1'b1));
endmodule

module hier3 ();

hier4 u0 (.A(1'b1),.B(1'b0));
hier4 u1 (.A(1'b0),.B(1'b1));
endmodule

module hier2 ();

hier3 u0 ();
endmodule

module top ();

// Internal wires
wire a;
wire b;
wire c;
wire d;

assign a = 1'b1 ;
assign b = 1'b1 ;
assign c = 1'b0 ;
assign d = 1'b0 ;

hier2 u0 ();
BUFX2 u1 (.A(a));
BUFX2 u2 (.A(c));
endmodule
```

```
//
// Verilog after delete_floating_constants is issued

module hier4 ( A, B);
input A;
input B;

BUFX2 u0 (.A(A));
BUFX2 u1 (.A(1'b1));
endmodule

module hier3 ();

hier4 u0 (.A(1'b1));
hier4 u1 (.A(1'b0));
endmodule

module hier2 ();

hier3 u0 ();
endmodule

module top ();

// Internal wires
wire a;
wire c;

assign a = 1'b1 ;
assign c = 1'b0 ;

hier2 u0 ();
BUFX2 u1 (.A(a));
BUFX2 u2 (.A(c));
endmodule
```

rename_obj

```
  rename_obj
[-help]
[-escape_ok]
[-flexible]
object
new_name
```

Renames the `base_name` of a single design object, which could only be a `inst`, `hinst`, `design`, `module`, `port`, `hport`, or `hnet`. This command changes only the `base_name` of the specified objects, and cannot move those across the netlist hierarchy. This command also returns the new name of the renamed object.

The names used for SDC timing constraints can only be modified using this command, if they have been loaded into the memory. So, ensure that all the timing views are loaded using the `set_analysis_view` command before running the `rename_obj` command so that the SDC constraints are updated properly. The modified SDC files can be retrieved after renaming the objects.

The `rename_obj` command cannot update the names used inside a CPF `power_intent` file. This is because the CPF usage assumes that the CPF file is golden and should not be modified. To change the names used inside a CPF file, first update the CPF file manually, and then use the updated CPF file to run the `rename_obj` command.

An IEEE1801 `power_intent` file is allowed to be changed during the design flow. So, changing names using the `rename_obj` command will automatically update the IEEE1801 object names. You can use the `rename_obj` command to write out the new IEEE1801 file with the updated names.

Parameters

`-help`	Provides a brief description of the command that includes the type and default information for its parameters. For a detailed description of the command and all its parameters, use the man command: `man rename_obj`
`-escape_ok`	Allows `new_name` to have either the hierarchy chars '`/`' or the bus-bit chars '`[]`' in it, which will automatically be escaped. If this parameter is not specified, any set of '`/`' or '`[]`' chars in `new_name` will cause an error and the object will not be renamed. *Data_type*: bool, optional
`-flexible`	If `new_name` coincides with an existing name, this parameter appends a suffix (such as `_1` and `_2`) to differentiate it. If this parameter is not specified, a name collision in such a scenario causes an error, and the specified object is not renamed. *Data_type*: bool, optional

object	Specifies the object to be renamed. The object here must be an `inst`, `hinst`, design, `module`, `port`, `hport`, or `hnet`. If *object* is a `port` or `hport`, any `hnet` connected to it with the same `base_name` is also renamed to keep it matched (for example, renaming `hport:top/i1/clkin` also renames `hnet:top/i1/clkin`). If *object* is an `hnet` that is connected to a `port` or `hport` with the same `base_name`, an error occurs. In such a case, you must change the `port` or `hport` name to proceed. If an `hnet` name matches with its associated `net` name (for example, `get_db $my_hnet .net`), then the net name is also renamed on using the `rename_obj` command. For example, renaming `hnet:top/net1` also renames `net:top/net1`. Therefore, to rename a `net`, you need to rename the `hnet` with the same name. This command does not allow renaming a bus-bit object. So any `port`, `hport`, or `hnet` that has a `.base_name` like `busA[0]` (a bus-bit object), cannot be renamed, While a `.base_name` with a scalar name (like `busA\[0\]`) can be changed. *Data_type*: (`inst | hInst | vCell | topCell | term | hTerm | hNet`), required
new_name	Specifies the new `base_name` for the specified object. *Data_type*: string, optional

Examples

- The following command renames an `inst base_name` to `i2_new`:
  ```
  rename_obj inst:top/i1/i2 i2_new
  inst:top/i1/i2_new                #returned object with new name
  ```
- The following command renames an `inst base_name i2\/i3` with escaped hierarchy:
  ```
  rename_obj {inst:top/i1/i2\/i3} i2/i3_new -escape_ok
  inst:top/i1/i2\/i3_new            #returned inst object, no hierarchy change
  ```
- The following command renames a scalar `hport` name to remove escaped bus-bit chars:
  ```
  rename_obj {hport:top/i1/busA\[0\]} busA_0
  hport:top/i1/busA_0               #returned hport object
  ```
  Note: `hnet:top/i1/busA\[0\]` is also renamed to `hnet:top/i1/busA_0`.
- The following command renames a `port` name:
  ```
  rename_obj port:top/clkin clkin_new
  port:top/i1/clkin_new             #returned port object
  ```
  Note: Both `hnet:top/i1/clkin` and `net:top/i1/clkin` are renamed.

7

GUI Commands

- clear_all_rulers
- create_gui_shape
- create_ruler
- gui_add_marker
- gui_add_ui
- gui_attach_to_cursor
- gui_clear_highlight
- gui_create_user_bundle_net
- gui_delete_objs
- gui_delete_ui
- gui_deselect
- gui_dim_foreground
- gui_write_picture
- gui_find_ui
- gui_fit
- gui_get_ui
- gui_get_visible_nets
- gui_hide
- gui_highlight
- gui_highlight_pin
- gui_highlight_pin_connection
- gui_list_marker

- gui_open_schematic
- gui_pan
- gui_pan_center
- gui_pan_page
- gui_redraw
- gui_remove_marker
- gui_select
- gui_select_highlighted
- gui_set_obj_color
- gui_set_power_rail_display
- gui_set_power_rail_layers_nets
- gui_set_tool
- gui_set_ui
- gui_set_visible_nets
- gui_show
- gui_show_edge_number
- gui_show_user_bundle_nets
- gui_view_box
- gui_zoom
- read_power_rail_results
- read_temperature_map_file
- report_power_rail_results
- view_last
- view_next
- write_to_gif

clear_all_rulers

```
clear_all_rulers -help
```

Removes all ruler lines from the main window. You can also clear the ruler lines with the *Clear All Rulers* icon on the toolbar in the main window.

Related Commands

- `create_ruler`

create_gui_shape

```
create_gui_shape
[-help]
-layer layerName
[-width width]
{-rect {x1 y1 x2 y2} | -line {x1 y1 x2 y2 ...} | -polygon {x1 y1 x2 y2 ...}}
[ -line {x1 y1 x2 y2 ...} [-arrow ] ]
```

Adds a shape to the specified custom layer. You can use the `create_gui_shape` command at any point in the design flow. The shapes added by the command annotate comment shapes and do not show up in a DEF output.

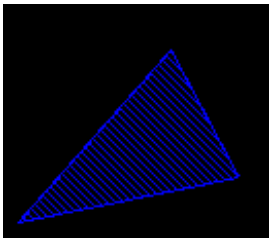
Parameters

`-help`	Prints a brief description that includes type and default information for each `create_gui_shape` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_gui_shape`
`-arrow`	Draws an arrow in the middle of the line segment to indicate the start to end direction. This parameter can only be used with `-line`.
`-layer layerName`	Specifies the GUI layer name to use or create for the rectangle/line. If the specified layer does not exist, it will be created automatically.
`-line {x1 y1 x2 y2 ...}`	Creates a line, with *x1 y1 x2 y2 ...* specifying the coordinates for the line segments. The values are specified in microns.
`-polygon {x1 y1 x2 y2 ...}`	Creates a polygon shape, with *x1 y1 x2 y2 ...* specifying the coordinates for the polygon segments. The values are specified in microns.
`-rect {x1 y1 x2 y2}`	Specifies the coordinates of the lower-left and upper-right points of the rectangle. The values are specified in microns.
`-width width`	Specifies the width of the shape border in pixels. The minimum (default) value is `1` and the maximum is `7`.

Examples

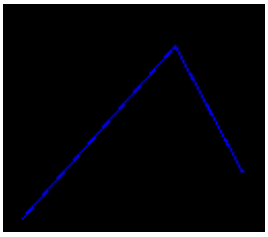
- The following command adds a triangular shape in layer `1`:

```
create_gui_shape -layer 1 -polygon {95.841 -645 96.592 -644.147 96.926 -644.777}
```



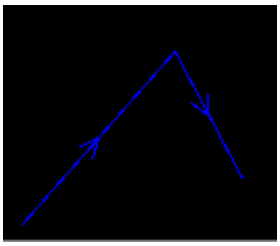
- The following command adds a line shape with two segments:

```
create_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777}
```



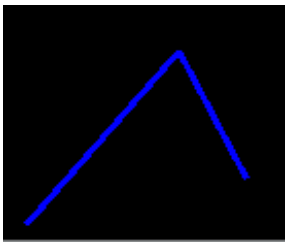
- The following command adds a line shape with two segments and direction arrows:

```
create_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777} -
arrow
```



- The following command adds a line shape of 3-pixel width:

```
create_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777} -
width 3
```



create_ruler

```
create_ruler
[-help] {{-center {x y} -radius value} | -coords {x1 y1 x2 y2 ...}}
```

Adds a ruler. For an orthogonal ruler, use `-coordinates` to specify ruler segments. For a circular ruler, use `-center` and `-radius`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `create_ruler` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_ruler`
`-center {x y}`	Specifies the coordinates for the center of a circular ruler. *Data_type*: point, optional
`-coords {x1 y1 x2 y2 ...}`	Specifies the beginning and ending coordinates for the ruler line segments. The values are in microns. *Data_type*: (point)+, optional
`-radius value`	Specifies the radius of a circular ruler. *Data_type*: double, optional

Example

- The following command creates a circular ruler of radius 500 with the center at `{0 0}`:

  ```
  create_ruler -center {0 0} -radius 500
  ```

Related Commands

- `clear_all_rulers`

gui_add_marker

```
gui_add_marker
[-help]
-color <string>
-name <string>
-point {x y}
-type {X TICK STAR}
```

Places a marker to indicate location on the design layout.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-color <string>`	Specifies the color of the marker. You can specify 16 colors for the marker. These are: `red blue green yellow magenta cyan pink orange brown purple violet teal olive gold maroon wheat`
`-name <string>`	Specifies the name of the marker.
`-point {x y}`	Specifies the location `{x y}` at which the marker will be placed on the design layout.
`-type {X TICK STAR}`	Specifies the shape of the marker. There are three shapes for the markers: X, TICK, and STAR.

Example

- The following command shows that a X type marker of color RED has been placed at the location {100 100}.
  ```
  gui_add_marker -name aa -color red -point {100 100} -type X
  ```

gui_add_ui

```
gui_add_ui
[-help]
name
[-before name]
[-checked {true | false}]
[-command string]
[-disabled {true | false}]
[-enable_variable string]
[-foreground string]
[-icon filename]
[-in name]
[-label string]
[-movable {true | false}]
[-newline {true | false}]
[-shortcut string]
[-tooltip string]
-type {menu | toolbar | submenu | command | check | radio | separator | toolbutton}
[-underline integer]
[-value string ]
[-variable string]
[-visible_variable string]
```

Adds a new interface element with the specified name of the specified type. Use this command to customize the graphical interface by adding menus, toolbars, status bar, and other interface elements, as required.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_add_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_add_ui`

name	Specifies as a string the internal name of the interface element you want to add. **Note:** Choose the internal name of the element carefully as this is what is used internally by the software to identify the element for any further manipulation. *Data_type*: string, required
-before *name*	When specified, adds the new element before the element identified by *name*. *Data_type*: string, optional
-checked {true | false}	Specifies whether or not the check box or radio button you are adding should be checked. **Note:** You can use the -checked parameter only if you are adding elements of type check or radio. *Default*: false
-command *string*	Specifies the Tcl command to be executed when the element you are adding is invoked. **Note:** You can use the -command parameter only if you are adding elements of type command, check, radio or toolbutton.
-disabled {true | false}	Specifies the state of the interface element you are adding. **Note:** You can use the -disabled parameter only if you are adding elements of type menu, submenu, command, check, radio or toolbutton. The parameter is NOT applicable for elements of type toolbar and separator. *Default*: false
-enable_variable *string*	Represents the Tcl global variable for Enable status.
-foreground *string*	Specifies foreground color of the icon. **Note:** You can use the -foreground parameter only if you are adding an element of type toolbutton.

`-icon` *`filename`*	When adding a toolbutton, specifies the file name of the toolbutton icon. *`filename`* can be the full path of the icon file or an icon name supported by the software. The icon names used in the software can be queried using the `gui_get_ui` command. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`.
`-in` *`name`*	Specifies the name of the parent element. Use this parameter along with `-before` to specify the location of the element you are adding. By default, `-in` adds the element to the end of the parent element. Use `-before` to add the element at a specific location within the parent element. **Note:** If `-in` is not specified, the new element is created but is not added to any location.
`-label` *`string`*	Specifies the label to be displayed on the new element. **Note:** This parameter is NOT applicable for elements of type `separator`.
`-movable {true | false}`	When adding a toolbar, specifies whether or not it will be movable. *Data_type*: bool, optional
`-newline {true | false}`	When adding a toolbar, specifies whether or not the toolbar should be added as a new row. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`. *Default*: `false`
`-shortcut` *`string`*	Specifies the shortcut key for a command, check box, radio button, or toolbutton. **Note:** You can use the `-shortcut` parameter only if you are adding elements of type `command`, `check`, `radio`, or `toolbutton`.
`-tooltip` *`string`*	Specifies the tooltip text for the toolbar or toolbutton you are adding. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`.

`-type {menu | toolbar | submenu | command | check | radio | separator | toolbutton}`	
	Specifies the type of the interface element you want to add. Currently, elements of type `main` and `statusbar` are not supported.
`-underline` *`integer`*	Specifies the integer index of the character to be underlined in the element label. The index begins with `0` and should have a value less than or equal to the length of the element label. For instance, if you want the second letter of the element label to appear underlined, specify *`integer`* as `1`. **Note:** The `-underline` parameter is NOT applicable for elements of type `toolbar, separator,` and `toolbutton`. **Note:** This parameter can be used only if the `-label` parameter is also specified.
`-value` *`stringValue`*	When adding a radio button, specifies the value to be stored in the associated variable when the radio button is selected. **Note:** The `-value` parameter is applicable only for elements of type `radio`.
`-variable` *`string`*	When adding a check box or radio button, specifies the Tcl global variable for the item.
`-visible_variable` *`string`*	Represents the Tcl global variable for Visible status

Examples

- The following command will add a new menu labeled *ExampleMenu* in the main window:
  ```
  gui_add_ui exmpMenu -type menu -label ExampleMenu -in main
  ```
- The following command will add a new menu item labeled *ExampleCommand* in *ExampleMenu*:
  ```
  gui_add_ui exmpCommand -type command -label ExampleCommand -in exmpMenu -command
  [list puts Example Command]
  ```

gui_attach_to_cursor

```
gui_attach_to_cursor
[-help]
[ -selected  | objects ]
```

Attaches selected or specified objects to the cursor. The use model is as follows:

1. Select the object(s) to be moved by using the `gui_attach_to_cursor` command.
2. Zoom into the location where you want to place the object(s).
3. Click at the required location to place the selected object(s) there.

This command is useful during floorplanning when you may need to move multiple objects into the same zoomed in area.

Parameters

`-help`	Prints a brief description of the `gui_attach_to_cursor` command. For a detailed description of the command, use the `man` command: `man gui_attach_to_cursor`.
`objects`	Specifies the object pointer list.
`-selected`	Attaches the selected objects to the cursor.

Example

The following command attaches objects `A` and `B` to the cursor:

```
gui_attach_to_cursor {A B}
```

Now, move the cursor to the layout and click at the desired location to place objects `A` and `B` there.

Related Commands

- `gui_select`
- `gui_zoom`

gui_clear_highlight

```
gui_clear_highlight
[-help]
[obj_list]
[-all | -index indexValue | -original | -select]
```

Removes highlights from all the highlighted objects at any stage of the design flow. You can choose to clear highlight from all objects, selected objects, objects highlighted with original color, or objects with specified index values.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_clear_highlight` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_clear_highlight`
`-all`	Specifies that highlight will be cleared from all objects.
`-index` *`indexValue`*	Specifies that highlight will be cleared from the objects with the specified index value. The index value is the value of the Highlight Set as specified through the *View - Highlight Selected* menu or through the `gui_highlight` command. The *View - Highlight Selected* menu provides eight highlight patterns to choose from. The value of index should, therefore, be a number be from 1 to 8. As an example, if you specify a value of 3 for index, all highlight will be cleared from from all objects that are highlighted as Highlighted Set 3.
`obj_list`	Specifies the object pointer list for clearing highlight.
`-original`	Specifies that highlight should be cleared from the objects highlighted with original color.
`-select`	Specifies that highlight should be cleared from all selected objects.

Examples

- The following command clears highlight from all highlighted objects.

  ```
  gui_clear_highlight
  ```

- The following command clears highlight from all objects that have been highlighted as Highlight Set 4 through the *View - Highlight Selected* menu or through the `gui_highlight` command.

  ```
  gui_clear_highlight -index 4
  ```

- The following command clears highlight from all highlighted objects that are currently selected in the display window.

  ```
  gui_clear_highlight-select
  ```

Related Topics

- `gui_highlight`
- `gui_highlight_pin`

gui_create_user_bundle_net

```
gui_create_user_bundle_net
[-help]
name
-nets net_list
-start object_name
-end object_name
```

Creates a bundle of specified nets that can be displayed as flightlines. Bundled nets help you with data flow analysis before and after layout. Net bundles can be created and viewed in floorplan and amoeba views. Each custom net bundle displays the number of nets in it and works for macros, blobs, and giudes.

Using `gui_create_user_bundle_net`, you can create custom bundled nets that allow you to define your own net bundles as per your requirements. This is useful while debugging physical properties of interconnects within a complicated hierarchical design because you can define start and end points of a net bundle. As a result, you can create and display only relevant flightlines, instead of looking at all the flightlines of specified nets.

Note: One net can be part of multiple net bundles simultaneously.

You can display flightline net bundles using the `gui_show_user_bundle_nets` command and delete them using the `gui_delete_user_bundle_net` command.

Note: You can choose to display flightlines for only bundled nets by using the *Only Show Net Bundle Flight Line* option in the Preferences form. If this option is not selected, the net bundle flightlines are displayed along with other selection-based flightlines by default.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_create_user_bundle_net` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_create_user_bundle_net`
`name`	Specifies the name of the custom net bundle.
`-nets` *`net_list`*	Specifies the list of nets to be included in the net bundle. You can use patterns with wildcards to specify net names. All the nets that match the specified pattern are put in the same bundle.
`-start` *`object_name`*	Specifies the name of object which is the start point of the nets. The object type includes inst, hinst, port, base_pin, and group.
`-end` *`object_name`*	Specifies the name of object which is the end point of the nets. The object type includes inst, hinst, port, base_pin, and group.

Examples

- The following command creates a bundle called `bundle_1`, which contains flightlines of nets `net_1` and `net_2`:

  ```
  gui_create_user_bundle_net bundle_1 -nets {net_1 net_2} -start Module1 -end Module2
  ```

  You can then use the gui_show_user_bundle_nets command to display `bundle_1` flightlines as follows:

  ```
  gui_show_user_bundle_nets -net_bundle bundle_1
  ```

gui_delete_objs

```
gui_delete_objs
[-help]
[-shape | -text | -marker | -all | -selected ]
```

Deletes specified or selected GUI objects.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man gui_delete_objs`
`-all`	Deletes all GUI objects.
`-marker`	Deletes GUI markers.
`-selected`	Deletes selected GUI objects.
`-shape`	Deletes GUI shapes.
`-text`	Deletes GUI texts.

Related Commands

- `gui_add_marker`

gui_delete_ui

`gui_delete_ui [-help] name`

Deletes an existing interface element with the specified name. If multiple interface elements are found with the same name, the first matched element is deleted. Use this command to customize the graphical interface by removing menus, menu items, and other interface elements that you do not require any longer.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_delete_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_delete_ui`
`name`	Specifies the internal name of the interface element as a string. An error is returned if an interface element with the specified name does not exist. **Note:** Here, *`name`* represents the internal name assigned to the element when it was added to the interface and not the label that is displayed on the interface. If you specify the label instead of the name, the software will return an error.

Examples

- The following command will delete an interface element with the internal name *`Menu1`*:

  ```
  gui_delete_ui Menu1
  ```

gui_deselect

```
gui_deselect
[-help]
{-all  |
-rect {x1 y1 x2 y2} |
-point {x y} |
-line {x1 y1 x2 y2}}
```

Deselects objects as per the parameters you specify. If `-all` is specified, deselects all selected objects.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_deselect` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_deselect`
`-all`	Deselects all objects.
`-line {x1 y1 x2 y2}`	Deselects objects through which the specified line passes.
`-point {x y}`	Deselects objects at the specified point.
`-rect {x1 y1 x2 y2}`	Deselects objects within the specified box coordinates.

gui_delete_user_bundle_net

```
gui_delete_user_bundle_net
[-help]
name
```

Deletes the specified flightline net bundle. If a net is part of multiple net bundles and you delete one of the net bundles, other net bundles containing the net are not affected.

You can create flightline net bundles using the `gui_create_user_bundle_net` command and display them using the `gui_show_user_bundle_nets` command.

Note: `gui_delete_user_bundle_net` removes the specified net bundle. If you want to retain the net bundle but just want to clear its flightlines from the display area, use the `-remove` option of `gui_show_user_bundle_nets` instead.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_delete_user_bundle_net` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_delete_user_bundle_net`
`name`	Specifies the name of the flightline net bundle to be removed.

Examples

- The following command deletes the flightline net bundle `bundle_1`:

  ```
  gui_delete_user_bundle_net bundle_1
  ```

gui_dim_foreground

```
gui_dim_foreground
[-help]
-light_level {default medium dark}
```

Dims the display area to the specified level. Use this command to view selected and highlighted objects clearly against the dimmed foreground. This command is especially useful when you are debugging.

The equivalent bindkey is F12.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_dim_foreground` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_dim_foreground`.
`-light_level {default medium dark}`	
	Specifies the brightness level of the foreground in the display area.

gui_write_picture

```
gui_write_picture
[-help]
filename
[-format {bmp gif jpg jpeg png pbm pgm ppm xbm xpm}]
[-width integer_value -height integer_value]
[-window string]
```

Saves a snapshot of the layout or specified window with the specified name in the current directory. You can specify the size as well as the format in which you want to save the snapshot. The command also prints out overlay maps with legends.

You can use this command at any stage of the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_write_picture` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_write_picture`

`filename`	Specifies the name of the picture file as a string. You can specify the file extension as well.
	
	Note: The file extension you specify does not control the format of the picture. The picture file is dumped in the PNG format by default or the format you explicitly specify with `-format`. For example, the following command saves the `testShot` picture file in the PNG format:
	`> gui_write_picture testShot.gif`
	
	`current_directory``@file testShot.gif`
	`testShot.gif: PNG image data, 540 x 688, 8-bit/color RGB, non-interlaced`

`-format {bmp gif jpg jpeg png pbm pgm ppm xbm xpm}`	
	Specifies the format of the picture file. *Default*: `png`

`-height` *integer_value*	
	Specifies the height of the picture as an integral value. **Note**: The `-width` parameter is required when `-height` is specified. *Min*: 100 *Max*: 20000
`-width` *integer_value*	
	Specifies the width of the picture as an integral value. **Note**: The `-height` parameter is required when `-width` is specified. *Min*: 100 *Max*: 20000
`-window` *string*	Specifies the ID of the window that you want to dump. You can use the following IDs: • For Layout Viewer: `designview.win` • For Cell Viewer: `.uiCellView.win` If `-window` is not specified, the main layout window is dumped out by default.

Examples

- The following command captures the layout window and saves it in the default PNG format with the name *test2* and the dimensions 1000 x 1000 in the current directory:

  ```
  gui_write_picture test2 -width 1000 height 1000
  ```

- The following command captures the layout window and saves it in the GIF format with the name *test1.gif* in the current directory:

  ```
  gui_write_picture -format gif test1.gif
  ```

- The following command captures the layout viewer window and saves it in PNG format with the name testShot.png in the current directory:

  ```
  gui_write_picture testShot.png -window designview.win
  ```

- The following command captures the view in the cell viewer window and saves it in PNG format with the name testCell.png in the current directory:

```
gui_write_picture testCell.png -window .uiCellView.win
```

gui_find_ui

```
gui_find_ui
[-help]
[name]
[-before name]
[-checked true | false]
[-command string]
[-disabled true | false]
[-enable_variable string]
[-foreground string]
[-icon filename]
[-in parent_name]
[-label string]
[-movable true | false]
[-newline true | false]
[-shortcut string]
[-tooltip string]
[-type {menu toolbar submenu command check radio separator toolbutton}]
[-underline index]
[-value string]
[-variable string]
[-visible_variable string]
```

Enables you to retrieve an element's name by specifying a property-value combination.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_find_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_find_ui`
`name`	Specifies the name of the parent element to define the search scope. If multiple elements are found with the same *`name`*, the first matched element is searched. If *`name`* is not specified, a global search is performed. *Data_type*: string, optional

`-before` *name*	Returns the element that is positioned just before the element identified by *name*. *Data_type*: string, optional
`-checked {true | false}`	Returns the names of check boxes and radio buttons with the specified checked state. *Data_type*: bool, optional
`-command` *string*	Returns the name of the interface element that executes the specified command string. **Note:** You can use the `-command` parameter only for elements of type `command, check, radio` or `toolbutton`. *Data_type*: string, optional
`-disabled {true | false}`	Returns the name of the interface elements with the specified state. **Note:** You can use the `-disabled` parameter only for elements of type `menu, submenu, command, check, radio` or `toolbutton`. *Data_type*: bool, optional
`-enable_variable` *string*	Returns the name of the interface elements with the specified Enable status. *Data_type*: string, optional
`-foreground` *string*	Returns the name of the tool button with the specified foreground color of the icon. **Note:** You can use the `-foreground` parameter only for elements of type `toolbutton`. *Data_type*: string, optional

`-icon` *`filename`*	Returns the name of the tool button with the specified image file in its icon. *`filename`* can be the full path of the icon file or an icon name supported by the tool. The icon names used in the tool can be queried using the `gui_get_ui` command. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`. *Data_type*: string, optional
`-in` *`parent_name`*	Returns the name of the interface elements with the specified parent name. *Data_type*: string, optional
`-label` *`string`*	Returns the name of the interface element with the specified label. **Note:** This parameter is NOT applicable for elements of type `separator`. *Data_type*: string, optional
`-movable true | false`	Returns whether the object is movable or not. This parameter is applicable only for elements of type `toolbar`. *Data_type*: bool, optional
`-newline {true | false}`	Returns the name of the toolbars with the specified newline status. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`. *Data_type*: bool, optional
`-shortcut` *`string`*	Returns the name of the interface elements with the specified shortcut key. **Note:** You can use the `-shortcut` parameter only for elements of type `command`, `check`, `radio`, or `toolbutton`. *Data_type*: string, optional

`-tooltip` *`string`*	Returns the name of the toolbar or toolbutton with the specified tooltip text. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`. *Data_type*: string, optional
`-type {menu | toolbar | submenu | command | check | radio | separator | toolbutton}`	
	Returns the name of the interface elements of the specified type. *Data_type*: enum, optional
`-underline` *`index`*	Returns the name of the interface elements that has character with the specified integer index underlined in the element label. The index begins with `0`, so for example, to find elements that have the third character in their label underlined, specify an index value of 2. *Data_type*: int, optional
`-value` *`stringValue`*	Returns the name of the radio button with the specified value. **Note:** The `-value` parameter is applicable only for elements of type `radio`. *Data_type*: string, optional
`-variable` *`string`*	Returns the name of the check boxes or radio buttons with the specified the Tcl global variable. *Data_type*: string, optional
`-visible_variable` *`string`*	Returns the name of the interface elements with the specified Visible status. *Data_type*: string, optional

Examples

- The following command returns the names of all the menus in the main window:

```
gui_find_ui main -type menu
```

- The following command returns the name of the element before the Help menu in the main window:

```
gui_find_ui -before help
```

gui_fit

```
gui_fit [-help]
```

Fits the entire design in the viewable window. You can use this command at any stage of the design flow to view the design in entirety.

Parameters

`-help`	Prints a brief description of the `gui_fit` command. For a detailed description of the command, use the `man` command: `man gui_fit`.

gui_get_ui

```
gui_get_ui
[-help]
[name]
[-quiet]
[ -before
 | -checked
 | -children
 | -command
 | -disabled
 | -dock_widget
 | -enable_variable
 | -foreground
 | -geometry
 | -icon
 | -in
 | -label
 | -menu
 | -message
 | -movable
 | -newline
 | -shortcut
 | -statusbar
 | -title
 | -toolbar
 | -tooltip
 | -type
 | -underline
 | -value
 | -variable
 | -visible
 | -visible_variable ]
```

Retrieves the current value of the specified property of the element that is being queried. Using the `gui_get_ui` command, you can query all the properties in the `gui_add_ui` command for each type. In addition, depending on the `type`, some special properties are supported too as described below:

- For elements of type `menu/submenu/toolbar`:
    - `gui_get_ui` *name* `-children`
      Returns a list of all the child elements of the specified interface element.

- For `main` (main window):
  - `gui_get_ui main -menu`
    Returns a list of all the menus in the main window.
  - `gui_get_ui main -toolbar`
    Returns the list of all the toolbars in the main window.
  - `gui_get_ui main -statusbar`
    Returns the status bar in the main window.
  - `gui_get_ui main -geometry`
    Returns the geometry of the main window in the format *`width`*x*`height`*+*`x`*+*`y`*.
  - `gui_get_ui main -title`
    Returns the title of the main window.
  - `gui_get_ui main -closecmd`
    Returns the close confirmation command of the main window.
  - `gui_get_ui main -visible`
    Returns the visibility status of the main window.
- For `statusbar`:
  - `gui_get_ui name -message`
    Returns the current message of the status bar.
  - `gui_get_ui name -visible`
    Returns the visibility status of the status bar.
  - `gui_get_ui name -in`
    Returns the parent element of the status bar.

`statusbar` also supports the `-enable_variable` and `-visible_variable` parameters.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_get_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_get_ui`
`name`	Specifies as a string the name of the interface element you want to query. If multiple widgets are found with the same *`name`*, the first matched widget is queried. Returns an error if there is no element with the specified name.
`property`	Specifies the property whose value is to be retrieved. You can specify the following properties: `-before | -checked | -children | -command | -disabled | -dockwin | -dock_widget | -enable_variable | -foreground | -geometry | -icon | -in | -label | -menu | -message | -newline | -shortcut | -statusbar | -title | -toolbar | -tooltip | -type | -underline | -value | -variable | -visible | -visible_variable`
`-before` *`name`*	When specified, returns the element that occurs before the element identified by *`name`*.
`-checked`	Returns the on/off state of the specified check box or radio button. If the check box or radio button is selected, returns `true`. If it is deselected, returns `false`.
`-children`	Returns child elements of specified interface element. **Note:** You can use this parameter with elements of type `menu`, `submenu`, and `toolbar`.
`-command`	Returns the Tcl command to be executed when the specified element you are adding is invoked. **Note:** You can use the `-command` parameter only with elements of type `command`, `check`, `radio` or `toolbutton`.

`-disabled`	When specified, returns whether the specified interface element is enabled (`false`) or disabled (`true`). **Note:** You can use the `-disabled` parameter only with elements of type `menu, submenu, command, check, radio` or `toolbutton`. The parameter is NOT applicable for elements of type `toolbar` and `separator`.
`-dock_widget`	Returns the dock widget in the window.
`-enable_variable`	Represents the Tcl global variable for Enable status.
`-foreground`	Returns the foreground color of the icon for the specified toolbutton. **Note:** You can use the `-foreground` parameter only with elements of type `toolbutton`.
`-geometry`	Returns the width, height, and coordinates of the main window. **Note:** The `-geometry` parameter is applicable only for the main window, which is type `main`.
`-icon`	Returns the file name of the specified toolbutton. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`.
`-in`	Returns the name of the parent element.
`-label`	Returns the label of the specified element. **Note:** This parameter is NOT applicable for elements of type `separator`.
`-menu`	Returns the names of the menus in the main window. **Note:** This parameter is applicable only for the main window, which is type `main`.
`-message`	Returns the message on the status bar. **Note:** This parameter is applicable only for the status bar.
`-movable`	Returns whether object is movable or not. *Data_type*: bool, optional

`-newline`	Returns whether or not the specified toolbar starts in a new row. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`.
`-shortcut`	Returns the shortcut key for a command, check box, radio button, or toolbutton. **Note:** You can use the `-shortuct` parameter only with elements of type `command`, `check`, `radio`, or `toolbutton`.
`-statusbar`	Returns the name of the main window's status bar. **Note:** This parameter is applicable only for the main window (`main`).
`-title`	Returns the text on the main window's title bar. **Note:** This parameter is applicable only for the main window (`main`).
`-toolbar`	Returns the toolbars in the window.
`-tooltip`	Returns the tooltip text for the specified toolbar or toolbutton. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`.
`-type`	Returns the type of the specified element.
`-underline`	Returns the integer index of the underlined character in the element label. **Note:** The `-underline` parameter is NOT applicable for elements of type `toolbar, separator,` and `toolbutton`.
`-value`	Returns the value stored in the associated variable of the specified radio button. **Note:** The `-value` parameter is applicable only for elements of type `radio`.
`-variable`	Returns the Tcl global variable for the specified check box or radio button.
`-visible`	Returns the visibility status of the element.
`-visible_variable`	Represents the Tcl global variable for Visible status.

`-quiet`	Displays the current settings for the specified parameters in the Tcl list format.

Examples

- The following command returns the names of all the menus in the main window:

  ```
  gui_get_ui main -menu
  ```

  The software returns the following:

  ```
  file view edit partition fplan power place eco clock route timing verify pvs tools windows flow help
  ```

- The following command returns the label on element named `verify`:

  ```
  gui_get_ui verify -label
  ```

  The software returns the following:

  ```
  Check
  ```

gui_get_visible_nets

```
gui_get_visible_nets
[-help]
```

Returns the names of the visible nets if some nets have been hidden in the design. If all nets are visible, this command has no effect.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_get_visible_nets` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_get_visible_nets`

Examples

- The following command hides all nets except the specified net:

  ```
  > gui_get_visible_nets
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/tpo_a_9_
  ```

Related Commands

- `gui_set_visible_nets`

gui_hide

```
gui_hide
[-help]
```

Hides the Voltus main window.

You can run the `gui_show` command to re-display the main window.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man gui_hide`

gui_highlight

```
gui_highlight
[-help]
[<any_object>+]
[-index indexValue | -auto_color  | -original]
[-color colorValue | -auto_color  | -original]
[-pattern patternValue | -original]
```

Highlights selected or specified objects with the Highlight Set available through the *View - Highlight Selected* menu. The value for the index corresponds to the Highlight Set that will be applied. For example, a value of 4 for the index specifies that all selected objects will be highlighted with the same color and pattern as available for the *Highlight Set #4* in the *View - Highlight Selected* menu. You can use this command at any stage in the design flow.

You can also provide a specific color and/or pattern to be used. All selected objects will be highlighted with the color and/or pattern that you specify.

If you specify a color and or pattern, the Highlight Set corresponding to the index value will be updated to use the color and pattern specified. For example, if you specify an index value of 5, color `green`, and pattern `backslash`, the *Highlight Set #5* will get updated to have the color `green` and the pattern `backslash`.

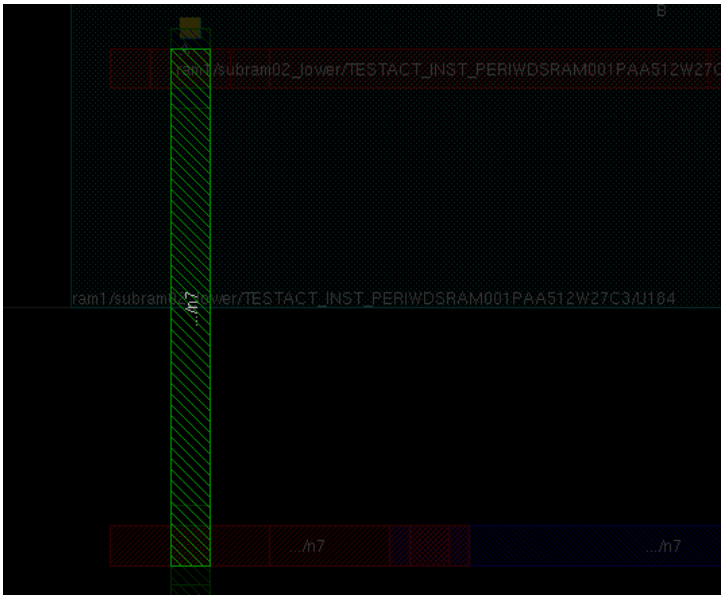
You can remove highlight from the highlighted objects through the `gui_clear_highlight` command or through the *View - Clear Highlight* menu.

Note: If an object is selectable, you can highlight it using the `gui_highlight` command.

Use the `gui_highlight_pin` command to highlight shapes for specified SIGNAL/POWER pins.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_highlight` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_highlight`
any_object+	Specifies the object pointer list for highlighting.
`-auto_color`	Specifies the object color automatically. **Note**: This parameter cannot be used with `-index` or `-color`.

`-index` *`indexValue`*	Specifies the index value for the specified object. All objects that are selected will be highlighted with the same color and pattern as defined in corresponding Highlight Set in the *View - Highlight Selected* menu As an example, if you specify a value of 3 for index, all objects that are selected will be highlighted with the same color and pattern as available for the *Highlight Set #3* in the *View - Highlight Selected* menu. The value of index should be between 1 to 64.
`-color` *`colorValue`*	Specifies the color value for the specified object. The color value is the same as that available through the Edit Highlight Color form (*View - Edit Highlight Color*). You can also use the Red Green Blue (RGB) value in the following format: #*`V1V2V3`* where V1, V2, and V3 are the hexadecimal values for the red, green, and blue color components respectively. For example, you can specify the value `#CC66FF`. *Default:* The color defined for the Highlight Set corresponding to the index value is used.
`-original`	Highlights the object in original color. This option is useful if you want to check a specific object more closely with the background dimmed. An object highlighted in original color will stand out in the dimmed background:  **Note**: This parameter cannot be used with `-index`, `-color`, or `-pattern`.

`-pattern` *`patternValue`*	Specifies the pattern value for the specified object. The pattern value is the same as the Stipple Value available through the Edit Highlight Color form. *Default:* The pattern (Stipple Value) for the Highlight Set corresponding to the index value is used.

Examples

- The following command highlights all selected objects with the same color and pattern as defined for the *Highlight Set #4* in the *View - Highlighted Selected* menu.

  ```
  gui_highlight -index 4
  ```

- The following command highlights all selected objects with the color `green` and the pattern `dot4`. The index values is set to 5 and, therefore, the command will also change the definition of the *Highlight Set #5* (*View - Highlight Selected* menu) to have color `green` and pattern `dot4`.

  ```
  gui_highlight -index 5 -color green -pattern dot4
  ```

- The following command highlights all selected objects with the color corresponding to the RGB hexadecimal value `#33FF66` and the pattern `horizontal`. The index values is set to 3 and, therefore, the command will also change the definition of the *Highlight Set #5* (*View - Highlight Selected* menu) to have color `#33FF66` and pattern `horizontal`.

  ```
  gui_highlight -index 3 -color #33FF66 -pattern horizontal
  ```

gui_highlight_pin

```
gui_highlight_pin
[-help]
-inst string
-pin string
[-original | -index indexValue]
[-original | -color colorValue]
[-original | -pattern patternValue]
```

Highlights the shapes of all specified SIGNAL/POWER instance pins with the Highlight Set available through the *View - Highlight Selected* menu. The value for the index corresponds to the Highlight Set that will be applied. For example, a value of 4 for the index specifies that all selected objects will be highlighted with the same color and pattern as available for the *Highlight Set #4* in the *View - Highlight Selected* menu.

You can also provide a specific color and/or pattern to be used. All specified pins will be highlighted with the color and/or pattern that you specify.

Note: If you specify a color and or pattern, the Highlight Set corresponding to the index value will be updated to use the color and pattern specified. For example, if you specify an index value of `5`, color `green`, and pattern `backslash`, the *Highlight Set #5* will get updated to have the color `green` and the pattern `backslash`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `gui_highlight_pin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_highlight_pin`

`-color` *colorValue*	Specifies the color value for the specified object. The color value is the same as that available through the Edit Highlight Color form, which can be accessed by selecting *View - Edit Highlight Color*. You can also use the Red Green Blue (RGB) value in the following format: `#`*V1V2V3* where *V1*, *V2*, and *V3* are the hexadecimal values for the Red, Green, and Blue color components, respectively. For example, you can specify the value `#CC66FF`. *Default:* The color defined for the Highlight Set corresponding to the specified index value is used.
`-index` *indexValue*	Specifies the index value for the specified object. By default, all specified pins are highlighted with the same color and pattern as defined in the *Highlight Set* corresponding to the specified index value in the *View - Highlight Selected* submenu. For example, if you specify a value of 3 for index, all specified pins are highlighted with the same color and pattern as available for *Highlight Set #3* in the *View - Highlight Selected* submenu. **Note**: The *View - Highlight Selected* submenu provides eight highlight patterns. Therefore, the value of the index should be a number from 1 to 8.
`-inst` *string*	Specifies the name of instance.
`-original`	Highlights the instance pin in original color. This option is useful if you want to check a specific instance pin more closely with the background dimmed. An object highlighted in original color will stand out in the dimmed background. **Note**: This parameter cannot be used with `-index`, `-color`, or `-pattern`.
`-pattern` *patternValue*	Specifies the pattern value for the specified object. The pattern value is the same as the *Stipple* value available through the Edit Highlight Color form. *Default:* The pattern (*Stipple* value) defined for the Highlight Set corresponding to the specified index value is used.
`-pin` *string*	Specifies the name of SIGNAL/POWER pin to be highlighted. You can specify more than one pin name.

Examples

- The following command highlights the SIGNAL pins `E` and `D` of instance `U1` using the color yellow and the solid pattern:

  ```
  gui_highlight_pin -inst U1 -pin {E D} -color yellow -pattern solid -index 1
  ```

- The following command highlights the SIGNAL pin `E` and the POWER pin `gnd` of instance `U1` using the color yellow and the solid pattern:

  ```
  gui_highlight_pin -inst U1 -pin {E gnd} -color yellow -pattern solid -index 1
  ```

Related Topics

- `gui_clear_highlight`
- `gui_highlight`

gui_highlight_pin_connection

```
gui_highlight_pin_connection
[-help]
-from_pin pin
[-mode {wire_segment | whole_net | flight_line}]
-to_pin pin
[-with_arrow | -select_only]
[-net_color_index index_value | -select_only]
[-inst_color_index index_value | -select_only]
```

Highlights the full timing path or its segment in the text format while generating the sign-off timing report. Use this command to quickly debug the timing path (timing point information) used in the tool.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `gui_highlight_pin_connection` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man gui_highlight_pin_connection`
`-from_pin` *pin*	Specifies the name of the starting pin to be highlighted.
`-inst_color_index` *index_value*	Specifies the instance color index value (an integer value from 1 to 64) for the specified timing path.
`-mode {wire_segment | whole_net | flight_line}`	Specifies the draw mode between the starting and ending pins.
`-net_color_index` *index_value*	Specifies the net color index value (an integer value from 1 to 64) for the specified timing path.
`-select_only`	Selects only the specified path for highlighting.
`-to_pin` *pin*	Specifies the name of the ending pin to be highlighted.
`-with_arrow`	Specifies whether or not the connection is defined with an arrow.

Example

The following command highlights a line drawn from the specified input pin to output pin while generating the sign-off timing report:

```
gui_highlight_pin_connection -from_pin ffa/Q -U7/A -mode flight_line
```

The timing report generated is shown below:

:::::::Timing Report::::::::::::

```
  Startpoint: ffa (rising edge-triggered flip-flop clocked by CLK)
```

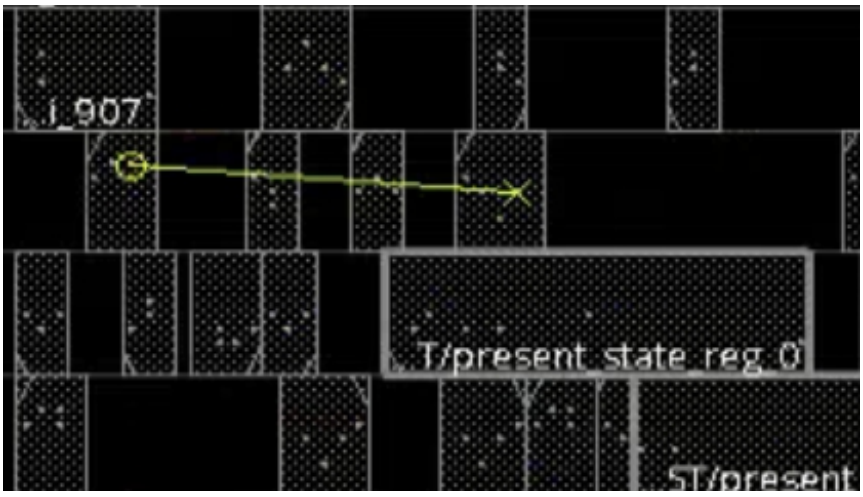
```
Endpoint: ffd (rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max
Point                          Fanout    Cap     Trans       Incr       Path
-----------------------------------------------------------------------------
clock CLK (rise edge)                                         0.00       0.00
clock network delay (propagated)                              4.90       4.90
ffa/CLK (DTC10)                                 0.30          0.10       5.00 r
ffa/Q (DTC10)                                   0.57          1.70       6.70 f
b (net)                          2     3.85
U7/A (IV110)                                    0.57          0.00       6.70 f
U7/Y (IV110)                                    1.32          0.84       7.55 r
QA (net)                        50    32.59
U12/A (NA310)                                   3.02          4.00      11.55 r
U12/Y (NA310)                                   2.47          4.04      15.58 f
n9 (net)                         3     8.87
U17/A (NA211)                                   2.47          0.00      15.58 f
U17/Y (NA211)                                   1.01          1.35      16.94 f
n14 (net)                        2     4.87
U23/A (IV120)                                   1.01          0.00      16.94 f
U23/Y (IV120)                                   0.51          0.37      17.30 r
n13 (net)                        1     2.59
U15/A2 (BF003)                                  0.51          0.00      17.31 r
U15/Y (BF003)                                   0.88          0.81      18.12 f
n12 (net)                        1    15.61
U16/B2 (BF003)                                  3.88          3.00      21.12 f
U16/Y (BF003)                                   2.46          0.99      22.11 r
n7 (net)                         1     2.61
```

```
 U10/A (AN220)                                  2.46       0.00       22.11 r
 U10/Y (AN220)                                  0.46       1.04       23.15 r
 n15 (net)                      1      2.63
 ffd/D (DTN10)                                  0.46       0.00       23.15 r
 data arrival time                                                    23.15

clock CLK (rise edge)                                      10.00       10.00
 clock network delay (propagated)                           3.50       13.50
 ffd/CLK (DTN10)                                                       13.50 r
 library setup time                                        -1.33       12.17
 data required time                                                    12.17
 ---------------------------------------------------------------------------
 data required time                                                    12.17
 data arrival time                                                    -23.15
 ---------------------------------------------------------------------------
 slack (VIOLATED)                                                     -10.98

:::::::END:::::::::::
```

The drawn layout along with the highlighted line is shown in the below picture:



gui_list_marker

```
gui_list_marker
```

Gives a list of GUI markers in the design layout.

Example

- The following command shows a list of GUI markers:
  ```
  gui_list_marker
  ```
  The following is displayed:
  ```
  {name:aa  coordinate:(100.000 100.000)}
  ```

gui_open_schematic

```
gui_open_schematic
[-help]
[-name string]
[-obj inst | net | pin | port | module]
[-type {module cone}]
```

Opens the schematic view of an object.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man gui_open_schematic`
`-name` *`string`*	Specifies a unique window name for the schematic view to be opened.
`-obj` *`inst`* `|` *`net`* `|` *`pin`* `|` *`port`* `|` *`module`*	
	Specifies the object to be added in the schematic view. You can also use `get_db selected` to pass the selected object to `-obj`.
`-type {module cone}`	Specifies the type of schematic to be opened. If you specify `cone`, the tool opens the Schematic Viewer to display schematic in flattened mode. If you specify `module`, the tool opens the Schematic Viewer to display schematic in hierarchical mode. *Default*: `module`

Examples

- The following command opens the schematic for the specified instance in the hierarchical mode:

  ```
  gui_open_schematic -obj DTMF_INST/PLLCLK_INST
  ```

- The following command opens the schematic for the specified instance in the flattened mode:

  ```
  gui_open_schematic -obj DTMF_INST/PLLCLK_INST -type cone
  ```

- The following opens the schematic for the selected object in the flattened mode:

  ```
  gui_open_schematic -obj [get_db selected] -type cone
  ```

gui_pan

```
gui_pan
[-help]
dx
dy
```

Specifies that the new center of the viewing window should shift from the current center of the viewing window by (*x*, *y*) microns. Use this command at any stage in the design flow to pan or move the design view in a specific direction.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_pan` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_pan`.
dx	Specifies that the design center should shift *dx* microns on the x-axis from the center of the viewing window. You can specify positive or negative values for *dx*. A positive value specifies shifting to the right on the x axis.
dy	Specifies that the design center should shift `dy` microns on the y axis values from the center of the viewing window. You can specify positive or negative values for *dy*. A positive value specifies shifting up on the y axis.

Example

The following example shifts the design center to the right by 200 microns and down by 300 microns from the center of the viewing window.

```
gui_pan 200 -300
```

gui_pan_center

`gui_pan_center [-help] x y`

Pans in the viewable window to the center point defined by the specified coordinates. Use this command at any stage of the design flow to view the design area around a specific point.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_pan_center` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_pan_center`.
x y	Specifies the x and y coordinates of the center point.

Example

The following command pans the viewable window to the center point (x=100, y=200)

```
gui_pan_center 100 200
```

gui_pan_page

`gui_pan_page [-help] x y`

Pans the viewable window in pages defined by the offsets. The size of each page is equal to the size of the current viewable window. Use this command at any stage of the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_pan_page` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_pan_page`.
x y	Specifies `x` and `y` offsets to indicate number of pages to pan. A positive value of *x* specifies a pan of *x* pages to the right. A positive number of *y* specifies a pan of *y* pages toward the top. You can specify integer values for *x* and *y*.

Examples

- The following command pans the display to the right by one page
  ```
  gui_pan_page 1 0
  ```
- The following command pans the display to the left by one page
  ```
  gui_pan_page -1 0
  ```

gui_redraw

```
gui_redraw [-help]
```

Refreshes the display in the viewable window. This command performs the same function as the *Redraw* icon (CTRL+R keyboard shortcut). You can use this command at any stage in the design flow.

Parameters

`-help`	Prints a brief description of the `gui_redraw` command. For a detailed description of the command, use the `man` command: `man gui_redraw`.

gui_remove_marker

```
gui_remove_marker
[-help]
{-name <string> | -all }
```

Removes a marker from the design layout.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-name <string>`	Specifies the name of the marker to be removed from the design layout.
`-all`	Removes all markers from the design layout.

gui_select

```
gui_select
[-help]
{-point {x y} |
-line {x1 y1 x2 y2} |
-rect {x1 y1 x2 y2}}
[-toggle  | -append  | -next]
[-point {x y} [-next ]]
```

Selects objects as per the parameters you specify. Use `-point` to select the object at the specified coordinate, `-line` to select the objects through which the specified line passes, or `-rect` to select the objects falling within the specified box coordinates.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_select` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_select`
`-append`	Appends the selection. This is equivalent to pressing the `Shift` key while selecting an object using the GUI.
`-line {x1 y1 x2 y2}`	Selects by line. Use this parameter to select the objects through which the specified line passes.
`-next`	Steps to next object under point. This is same as using space bar while selecting an object using the GUI.
`-point {x y}`	Selects the object at the specified point.
`-rect {x1 y1 x2 y2}`	Selects by box. Use this parameter to select the objects falling within the specified rectangular area.
`-toggle`	Toggles the selection. This is equivalent to pressing the `Ctrl` key while selecting an object using the GUI.

Examples

- The following command selects the objects that are totally enclosed within the box defined by the coordinates (`x = 10, y = 200`) and (`x = 100, y = 50`):
  ```
  gui_select -rect {10 200 100 50}
  ```
- The following command selects all the objects falling across the specified line coordinates:
  ```
  gui_select -line {168.654 994.049 688.129 436.224}
  ```
- The following command selects the objects crossed by the specified line and appends them to the current selection:
  ```
  gui_select -line {169.622 996.049 698.290 446.240} -append
  ```
- The following command toggles the selection on the objects in the specified bounding box. If the objects are previously selected, the command deselects them and vice versa:
  ```
  gui_select -rect {10 200 100 50} -toggle
  ```

gui_select_highlighted

```
gui_select_highlighted
[-help]
[-type string]
```

Selects highlighted objects. If `-type` is specified, selects highlighted objects of the specified type.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_select_highlighted` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_select_highlighted`
`-type` *`string`*	Specifies the type of the highlighted objects to be selected. Options are `instance`, `wire`, and `via`.

Examples

- The following command selects all highlighted instances.
  ```
  gui_select_highlighted -type instance
  ```

Related Commands

- `gui_highlight`

gui_set_obj_color

```
gui_set_obj_color
[-help]
{[-obj_name InstOrHInstName | -obj_type typeList]
[-multicolor | -color_id colorID]
[-include_children {off | color | multicolor}]
[-reset]}
```

Sets the color for objects of the specified name or type. You can choose to include children when specifying object color for a hierarchical instance (HInst).

The `gui_set_obj_color` command is typically used during floorplanning to color objects that are currently displayed in the design display area. You can use this command to distinguish between objects of different type or between objects belonging to different instances, hierarchical instances, instance groups, or power domains.

Before using the `gui_set_obj_color` command, you should have:

- Loaded the floorplan
- Ungrouped objects so that only required objects are displayed in the design display area in the Floorplan or Physical view.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_set_obj_color` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_set_obj_color`.
`-color_id` *`colorID`*	Specifies the color ID from 0 to 29. The color value for the color ID is the same as specified in the Module Color Preference form (accessed by clicking *Floorplan - Edit Floorplan - Color Module*). *Default*: Color ID `0`
`-include_children {off | color | multicolor}`	Specifies whether the children of the hierarchical instance will be colored simultaneously when you specify a color for an HInst. You can set one of the following options: • `off`: The child instances will not be colored. • `color`: The child instances are colored the same color as their parent instance. • `multicolor`: The child instances are colored in multiple colors.
`-multicolor`	Uses multiple colors to color objects of the specified type, instance, or hierarchical instance.
`-obj_name` *`InstOrHInstName`*	Specifies the name of the instance, hierarchical instance, power domain, or instance group whose objects are to be colored. You can specify an object list, such as `objectName {A,B}`.
`-obj_type` *`typeList`*	Specifies the list of type of objects that are to be colored, such as `BusGuide`, `Module`, `PowerDomain`, and `InstGroup`.
`-reset`	Resets the object color.

Example

- The following command colors the `DTMF_INST/TDSP_CORE_INST` instance with color `3`, as specified in the Module Color Preference form:

```
gui_set_obj_color -obj_name DTMF_INST/TDSP_CORE_INST -color_id 3
```

- The following command colors the `DTMF_INST/TDSP_CORE_INST` instance and its children with color `3`, as specified in the Module Color Preference form:

  ```
  gui_set_obj_color -obj_name DTMF_INST/TDSP_CORE_INST -color_id 3 -include_children
  color
  ```

- The following command colors the children of the `DTMF_INST/TDSP_CORE_INST` instance in multiple colors:

  ```
  gui_set_obj_color -obj_name DTMF_INST/TDSP_CORE_INST -include_children multicolor
  ```

gui_set_power_rail_display

```
gui_set_power_rail_display
[-help]
[-enable_result_browser {true | false}]
[-enable_rlrp {true | false}]
[-enable_voltage_sources {true | false}]
[-filter_max max_value]
[-filter_min min_value]
[-legend {off ne nw se sw}]
[-plot plot_type]
[-range_max max_value]
[-range_min min_value]
[-reset_color_scale {true | false}]
[-filter_color {on | off | color_name}]
[-enable_percentage_range {true | false}]
[-thermal_layer_name thermal_layer]
[-tile_count_x value]
[-tile_count_y value]
[-tile_size_x value]
[-tile_size_y value]
[-customized_range {value1  value2 ...  value9}]
[-grid {on | off}]
[-enable_auto_apply {true | false}]
```

Controls the display of the main window widgets, such as legend, result browser, voltage source, filter range, and RLRP path.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_set_power_rail_display` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_set_power_rail_display`.
`-customized_range {value1  value2 ...  value9}`	

	Specifies the color scale range values for the 8 filters between the minimum and maximum values of the color gradient based (*Basic*) filter. The value list must contain 9 values. The filter ranges for the 8 buckets are evenly divided by default. This parameter can be used to customize the filter ranges. You can either modify the color range using the GUI (Basic button) or the `-customized_range` parameter. **Example**: `gui_set_power_rail_display -plot ir -customized_range { 1 10 12 13 14 15 16 17 49 }`
`-enable_auto_apply {true | false}`	
	Specifies to automatically redraw the layout after modifying the color scale range values. This parameter allows you to enable/disable the *Auto Apply for Color Scale* checkbox of the Power Rail Plots form using the Tcl script instead of the GUI option. The default value of this parameter is `false`.
`-enable_percentage_range {true | false}`	
	Specifies to display the color gradients for the following plots by percentage in the GUI: • IR Drop • IVDD • IVDN The percentage value is based on the nominal voltage value of each rail/instance. It displays high percentage voltages in red and low percentage voltages in blue. *Default* : `false`
`-enable_result_browser {true | false}`	
	When set to `true` it enables the result browser. *Default* : `false`
`-enable_rlrp {true | false}`	

	When set to `true` it enables the display of the RLRP path. The rail analysis must include RLRP analysis (`set_rail_analysis_config -enable_rlrp_analysis true`) in order to have the required data for this functionality. *Default* : `false`
`-enable_voltage_sources {true | false}`	
	Displays voltage sources (power pads) using white pixels in the layout canvas. *Default* : `false`
`-filter_color {on | off | color_name}`	
	Allows to set the color of the filtered out resistors specified using the scissor filter range. By default, the filtered resistors are not displayed. If you want to view these resistors, you can use this parameter to enable the visibility of resistors and set desired colors for them. The valid values of this parameter are: • `on` – enables the visibility of the filtered out resistors. • `off` - disables the visibility of the filtered out resistors. • *`color_name`* - enables the visibility of filtered out resistors and set the color of resistors per the specified color name.
`-filter_max` *`max_value`*	
	Specifies the maximum value of the scissor filter range that will be plotted. You can use the scissor filter range to filter out the resistors and nodes that you do not want to view, and obtain statistics only for the instances in the specified filter range. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-filter_min` *`max_value`*	
	Specifies the minimum value of the scissor filter range that will be plotted. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-grid {on | off}`	

	Allows you to control the grid display when viewing the Voltage Source Current (VC) and Voltage across Package Model (VU) plots. The default value of this parameter is `on`. You can set this parameter to `off` to hide the grid view for better display and easier analysis of the plot data.
`-legend {off ne nw se sw}`	
	Displays the filter color and range details. When you set `set_power_rail_display -legend` to one of the location (`ne nw se sw`) , this opens the legend at the specified location on the design layout. Use `set_power_rail_display -legend off` to close the legend. *Default*: `off` .
`-plot` *plot_type*	
	Plots the specified plot type. See Plot Types for a description of each of the plot types.
`-range_max` *max_value*	
	Specifies the maximum value of the color scale. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-range_min` *min_value*	
	Specifies the minimum value of the color scale. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-reset_color_scale {true | false}`	
	Specifies to reset the maximum (`-range_max`) and minimum (`-range_min`) values of the color scale to the default plot data values. For example, when you specify `-plot ir`, the default value for `-range_min` is `0` and `-range_max` is `100`. You can modify the color scale, such as set `-range_min` to `0` and `-range_max` to `20`. To reset the color scale of the IR drop plot to default, you can specify the `-reset_color_scale` parameter.
`-thermal_layer_name` *thermal_layer*	

	Displays the specified layer of the temperature map. **Example:** `set_power_rail_display -thermal_layer_name VIA_7`
`-tile_count_x` *value*	
	Specifies the number of tiles in the x direction. This parameter is used to resize the tile power density plot. You can either specify the tiles by count (`tile_count_x` and `tile_count_y`) or specify the tiles by size (`tile_size_x` and `tile_size_y`). When you resize the tile power density (`tpd`) plot in the batch or GUI mode, the software automatically invokes the `report_power` command and resizes the plot. This feature allows you to resize the `tpd` plot without having to restore the power database (`power.db`) again.
`-tile_count_y` *value*	
	Specifies the number of tiles in the y direction. This parameter must be specified with the `tile_count_x` parameter.
`-tile_size_x` *value*	
	Specifies the horizontal dimension of each tile polygon in µm. This parameter is used to resize the tile power density plot. You can either specify the tiles by count (`tile_count_x` and `tile_count_y`) or specify the tiles by size (`tile_size_x` and `tile_size_y`). When you resize the tile power density (`tpd`) plot in the batch or GUI mode, the software automatically invokes the `report_power` command and resizes the plot. This feature allows you to resize the `tpd` plot without having to restore the power database (`power.db`) again. The following is an example of specifying tiles by size: `set_power_rail_display -plot tpd -tile_size_x 100 -tile_size_y 100`
`-tile_size_y` *value*	
	Specifies the vertical dimension of each tile polygon in µm. This parameter must be specified with the `tile_size_x` parameter.

Plot Types

Plot Type	Description
`ac`	Available/feasible capacitance (Farad) Displays the feasible decoupling capacitance that can be added in the area. The feasible capacitance in the area is calculated based on the filler cells placement. The program matches the placed filler cells with the decap cells available in the power-grid library and computes the feasible capacitance. The capacitance of the decap cell is stored in the power-grid library.
`cap`	Ggrid capacitance (Farad) Plots the capacitance of the extracted power-grid network.
`cvi1`	Customized File 1 Plots the user-specified instance voltage file.
`dd`	Decoupling capacitance (decap) density (Farad/um ^{2}) Plots decap density of the design. The decap density is the cell internal decoupling capacitance in the unit area (µm ^{2}).
`effr`	Effective resistance (ohm) - instance based Display the effective resistance results for all instances.
`node_reff`	Effective resistance (ohm) - node based Display the effective resistance results for all nodes.
`ipc`	Instance Peak Current Display the peak current of each instance.
`ivdn`	Instance Voltage Drop (Volts) – net-based Plots the net-based instance voltage drop.
`ivdd`	Instance Voltage Drop (Volts) – domain-based Plots the domain-based instance voltage drop. For domain based, at least one power and one ground net must be selected.

`fd`	Filler cell density (1/µm ^{2}) Displays the filler cell density per unit area (µm ^{2}). The plot can be used to analyze the feasibility of adding decap cells by replacing the filler cells and not changing the placement of the logic cells.
`freq`	Frequency domain (Hz) Displays the associated clock frequency of all instances in the design. The instances associated with multiple clock domains are displayed using the fastest clock frequency. This plot can be used to analyze and debug the power distribution in the design.
`ip`	Instance total power (mW) The total power consumed by each instance is displayed. The total power is the sum of the internal, switching and leakage power of the instance. The power data is read from the power database and can be specified using the `-power_db` option or it is queried directly from memory, if power calculation is run during the session.
`ip_i`	Instance internal power (mW) The power consumed by charging and discharging of the internal interconnect and device capacitances of the instance.
`ip_l`	Instance leakage power (mW) The power consumed by devices when not switching.
`ip_s`	Instance switching power (mW) The power consumed by charging and discharging of the interconnect or output load.
`ipd_i`	Specifies internal power density in mW/um ^{2} .
`ipd_l`	Specifies leakage power density in mW/um ^{2} .
`ipd_s`	Specifies switching power density in mW/um ^{2} .
`ipd_t`	Specifies total power density in in mW/um ^{2} .
`ir`	IRdrop or voltage drop (Volts) Displays the IRdrop across each extracted power-grid segment in the design.

`jrms`	Specifies current density as j/jmax based on the Irms current. This plot is only supported for dynamic analysis. A jrms gif file will be saved in the state directory that shows where the Irms based EM violation occurred. This plot type allows you to verify the analysis results of "jrms" using the GUI.
`load`	Loading capacitance (Farad) Displays the output loading capacitance driven by the instance. This plot can be used to analyze and debug the regions with high switching power.
`pi`	Power-switch (gate) current violation: I/Idsat Displays the ratio of current through the power-switch and the saturation current (Idsat) of the power-switch. This plot can be used to analyze and debug whether the current through power-switches exceeds the saturation current characterized for the power-switch devices. The saturation current and on-resistance of the power-switches are characterized and stored inside the power-grid library of the power-switch cell. A ratio of greater than 1 means that the current requirement of the power-gated block can not be met with placed power-switch instance. Either the power-switch needs to be made larger or power-switch instances need to be added.
`ptpd`	Profiling Tile Power Density Displays the tile-based power density plot for the vector profiling flow. You can enable this plot using the `read_power_rail_results -profiling_power_file` *`file`* command. You can also view the power density for the different tile co-ordinates in the *Details* tab of the Result Browser.
`pv`	Voltage drop across power-switch (Volt) Displays the IRdrop across power-switch instances. This plot can be used to analyze and debug the IRdrop inside the power-gated block, e.g. if the IRdrop across power-switches is already high, the IRdrop inside the power-gated block will be much higher. In this case, the power-switch placement will have to be refined in order to resolve the IRdrop problem.

`rc`	Resistor current (Amp) Displays the current across power-grid resistor segments. This plot can be used to analyze and debug how the current flows from the voltage sources into the rest of the design, e.g. high current should flow from pads to the top-level routing level layers and then to the lower level metal layers. This plot can also be used to debug current density violations (`rj`).
`rlrp`	The total resistance of the instance along the least resistive path. Displays the total resistance of the instance along the least resistive path, meaning the resistance of the instance along the path from where it gets the most amount of current. This plot can be used to analyze and debug the power-grid integrity early in the design stage. An instance with high resistance is likely to have high static or dynamic IRdrop depending on the vector that is simulated later in the design cycle. This plot can be also used during decap optimization, since regions with high dynamic IRdrop and high resistance may not be improved upon by adding decaps, because the effectiveness of decoupling capacitance depends on the resistance of the power-grid network. The plot displays the RLRP paths for power (purple color) and ground (white color) nets in different colors for the identification of paths and ease of analysis. You can also highlight the least resistive path of the instance using the GUI and selecting the instance with left mouse button. The *RLRP Path* checkbox should be enabled.
`res`	Grid resistance (ohm) Displays the total resistance of the least resistive path from each node to voltage source. This plot can be used to view high resistance regions of the power-grid. It is useful during IRdrop and decap analysis to identify weak power-grid connectivity. The GUI also allows you to view the resistor-based RLRP path. To view the resistor-based RLRP path: 1. Select the *`res`* Rail plot, and the *RLRP Path* checkbox. 2. Select a resistor. The GUI highlights the least effective resistance path to the nearest voltage source, and opens the *Resistance Path* form.

`rj`	Resistor current density, J/Jmax. Displays the current density (current per unit area) violation on each power-grid segment using the ratio of calculated current density and maximum allowed current density. A ratio of greater than 1 means that the current density limit of the segment is violated. The current density limits are supplied by the foundry and can be read during analysis using the `set_rail_analysis_mode -em_models` command.
rs	Specifies the plot type for resistor sensitivity.
`sem_avg`	Average Current Density Displays average current limit violations in the design. This plot is enabled when you specify `check_ac_limits -method avg`.
`sem_peak`	Peak Current Density Displays peak current limit violations in the design. This plot is enabled when you specify `check_ac_limits -method peak`.
`sem_rms`	RMS Current Density Displays RMS current limit violations in the design. This plot is enabled when you specify `check_ac_limits -method rms`.
`slack`	Instance slack (Seconds) Displays worst instance slack. The slack data is queried from either the timing database or if the power database is specified using `-power_db` option, then the worst slack data is read from the power database. When using an external TWF, the slack data is read from the TWF.
`tc`	Tap current (Amp) Displays the current consumed by the instance devices or current taps. During rail analysis, it is based on the power consumption of the instance and is distributed inside the cell. The current distribution inside the cell is based on the extracted device netlist stored inside the power-grid library of the cell. The current distribution is generally based on the device width, length, and spice models. This plot can be used to analyze and debug IRdrop inside the macro.

`td`	Transition density (Hz) Displays the worst transition density (activity * frequency) of the instance. This plot can be used to analyze and debug activity distribution and high power density regions.
`thermal`	Instance thermal Displays the instance temperature distribution for the chip. The plot shows tile-based temperature for instances. When you click on an instance, the *Attribute Viewer* window opens and displays the instance temperature in the *Thermal* field.
`unc`	Unconnected segments Displays power-grid segments that are disconnected from the voltage source (power pads). This plot can be used to debug unusually high IRdrop in the region.
`vc`	Voltage source current (Amp) Displays current through the voltage sources (power pads). This plot can be used to analyze the current carrying capacity of the power-pad.
`vu`	Voltage drop across package (Volt) Displays voltage drop across package. If analysis is run with package model attached to the power pads, this plot displays the IRdrop due to package RLCK elements.

Examples

- The following command specifies that `div` data will be plotted without loading design DEF:

```
gui_set_power_rail_display\
-plot div
```

- The following command enables the visibility of the filtered out resistors without changing the color:

```
gui_set_power_rail_display -filter_color on
```

- The following command enables the visibility of the filtered out resistors and also changes the color to red:

```
gui_set_power_rail_display -filter_color red
```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

gui_set_power_rail_layers_nets

```
set_power_rail_layers_nets
[-help]
[-design_opacity value]
[-rail_analysis_opacity value]
[-nets {{all_power 0/1}{all_ground 0/1}} |
 -enable_nets {names} |
 -disable_nets {names} ]
[-enable_switch_nets {all|net_names} |
 -disable_switch_nets {all|net_names} ]
[-enable_report_layers {all|layer_names} |
 -disable_report_layers {all|layer_names} |
 -layers {{name report0/1 visible0/1}{...}} ]
[-enable_visible_layers {all|layer_names} |
 -disable_visible_layers {all|layer_names} |
 -layers {{name report0/1 visible0/1}{...}} ]
[-enable_visible_signal_em_layers {all|layer_names} |
 -disable_visible_signal_em_layers {all|layer_names} ]
[-show_layer_net_setting_list {true | false}]
[-instance_voltage_method {Worst | Best | Avg | WorstAvg}]
[-instance_voltage_window {1| 0}]
[-via_size {1 | 2 | 3 | 4 | 5 | 6}]
```

Controls the visibility of the layers, the opacity of the rail analysis or design database, and the visibility of power/ground nets.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_set_power_rail_layers_nets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_set_power_rail_layers_nets`.

`-design_opacity` *value*	
	Specifies the design opacity value for the design database. The opacity range is from 0-100%, the default value is `60`.
`-disable_nets {`*names*`}`	
	Hides the specified nets. This parameter is mutually exclusive to the `-enable_nets` and `-nets` parameters.
`-disable_report_layers {all|`*layer_names*`}`	
	Hides the report (user-defined layers to be used for processing) layers. You can use `all` to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the `-enable_report_layers` and `-layers` parameters.
`-disable_switch_nets {all|`*net_names*`}`	
	Hides the specified switch nets. You can use `all` to hide all the switch nets, or list the specific switch nets that are not to be displayed. This parameter is mutually exclusive to the `-enable_switch_nets` parameter.
`-disable_visible_layers {all|`*layer_names*`}`	
	Specifies which layers should not be made visible when plotting. You can use `all` to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the `-enable_visible_layers` and `-layers` parameters.
`-disable_visible_signal_em_layers {all|`*layer_names*`}`	
	Specifies which signal EM layers should not be made visible when plotting. You can use `all` to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the `-enable_visible_signal_em_layers` parameter.
`-enable_nets {`*names*`}`	

	Displays only the specified nets. This parameter is mutually exclusive to the `-disable_nets` and `-nets` parameters.
`-enable_report_layers {all|`*`layer_names`*`}`	
	Specifies to select the user-defined layers to be used for processing. You can use `all` to select all layers, or list the specific layers that are to be selected. This parameter is mutually exclusive to the `-disable_report_layers` and `-layers` parameters.
`-enable_switch_nets {all|`*`net_names`*`}`	
	Displays the specified switch nets. You can use `all` to display all the switch nets, or list the specific switch nets that are to be displayed. This parameter is mutually exclusive to the `-disable_switch_nets` parameter.
`-enable_visible_layers {all|`*`layer_names`*`}`	
	Specifies which layers should be made visible when plotting. You can use `all` to display all layers, or list the specific layers that are to be made visible. This parameter is mutually exclusive to the `-disable_visible_layers` and `-layers` parameters.
`-enable_visible_signal_em_layers {all|`*`layer_names`*`}`	
	Specifies which signal EM layers should be made visible when plotting. You can use `all` to select all layers, or list the specific layers that are to be selected. This parameter is mutually exclusive to the `-disable_visible_sem_layers` parameter.
`-instance_voltage_method {Worst | Best | Avg | WorstAvg}`	

	Allows different methods of computing effective instance voltage during rail analysis. Using this option, you can write out the Best, Worst, or Average effective instance voltage between power and ground nets. The WorstAvg method computes the average instance voltage during each evaluation window defined by the IV Window field, and reports the worst average instance voltage out of all the windows. If IV Method is WorstAvg and IV Window is whole, then this method will report average voltage across all time steps in simulation. Default : `Worst`
`-instance_voltage_window {1| 0}`	
	Controls the EIV plot. EIV is obtained by processing the voltage waveforms and finding the worse-case effective voltage between the power and ground pins during a specific window. There are two options to view EIV plots: • `1` – both switching and timing windows (window EIV) • `0` – worst of the entire rail simulation (worst EIV)
`-layers {{name report0/1 visible0/1}{...}}`	
	Controls the visibility of layers. You can enable or disable the reporting and visibility of specific layer. For example, `-layer {{M1 1 1} {M2 0 0}` means enable reporting and visibility of layer M1, and disable reporting and visibility of layer M2 This parameter is mutually exclusive to the `-disable_report_layers, -disable_visible_layers, -enable_visible_layers,` and `-enable_report_layers` parameters.
`-nets {{all_power 0/1}{all_ground 0/1}}`	

	Controls the display of all the power and ground nets. When `all_power` is set to `0`, hides all power nets, and when set to `1`, displays all power nets. When `all_ground` is set to `0`, hides all ground nets, and when set to `1`, displays all ground nets. This parameter is mutually exclusive to the `-enable_nets` and `-disable_nets` parameters.
`-rail_analysis_opacity` *value*	
	Specifies the opacity value for the rail analysis output. The opacity range is from 0-100%, the default value is `100`.
`-show_layer_net_setting_list {true | false}`	

	Displays the current status (OFF/ON) of all layers and nets in the console. The following is an example of the output:
	`#-------------------------------#`
	`# Layers  Report  Visible #`
	`# RD  ON  ON #`
	`# M0  ON  ON #`
	`.....`
	`#-------------------------------#`
	`# PowerNets  Status #`
	`# VDD  ON #`
	`# VDDm  ON #`
	`#-------------------------------#`
	`# GroundNets Status #`
	`# VSS  ON #`
	`#-------------------------------#`

`-via_size {1 | 2 | 3 | 4 | 5 | 6}`	
	Specifies to modify and control the size of vias in the GUI. The default via size is `1`. The possible via size values are `1`, `2`, `3`, `4`, `5`, and `6`.

Examples

- The following command specifies the layers that will be visible:

```
gui_set_power_rail_layers_nets \
 -enable_visible_layers{RD M1 V12 M2 V23 M3 V34 M4 V45 M5 V56 M6} \
 -enable_nets {vdd vdd_1}
```

- The following commands describe the use model for enabling and disabling switch nets. Here, consider a net `VDD` with three switch nets `VDD_sw0`, `VDD_sw1`, and `VDD_sw2`:
  - The following command hides the switch net `VDD_sw0`:

    ```
    set_power_rail_layers_net -disable_switch_nets {VDD {VDD_sw0}}
    ```

    Displays `VDD_sw1` and `VDD_sw2`.
  - The following command hides all the switch nets:

    ```
    set_power_rail_layers_net -disable_switch_nets {VDD {all}}
    ```
  - The following command displays the switch net `VDD_sw0`:

    ```
    set_power_rail_layers_net -enable_switch_nets {VDD {VDD_sw0}}
    ```
  - The following command displays all the switch nets:

    ```
    set_power_rail_layers_net -enable_switch_nets {VDD {all}}
    ```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

gui_set_tool

```
gui_set_tool [-help] mode
```

Selects specified tool widget in the main window and enters interactive mode.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_set_tool` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_set_tool`
mode {select | move | ruler | util | cut | obstruct | layerBlk | ptnFeed | ptnPinBlk | ptnPinGuide | addWire | addVia | moveWire | cutWire | stretchWire | addPoly | addBusGuide | p2pRoute}	

	Specifies the interactive mode to enter. The following modes are supported: • `select` - Chooses the Select tool widget in the main window. • `move` - Selects the Move/Resize/Reshape tool widget in the main window. • `ruler` - Selects the Create Ruler tool widget in the main window. • `util` - Selects the Query Area Density tool widget in the main window. • `cut` - Selects the Cut Rectilinear tool widget in the main window. • `obstruct` - Selects the Create Placement Blockage widget in the main window. • `layerBlk` - Selects the Create Routing Blockage tool widget in the main window. • `ptnFeed` - Selects the Create Physical Feedthrough tool widget in the main window. • `ptnPinBlk` - Selects the Create Pin Blockage tool widget in the main window. • `ptnPinGuide` - Selects the Create Pin Guide tool widget in the main window. • `addWire` - Selects the Edit Wire tool widget in the main window. • `addVia` - Selects the Add Via tool widget in the main window. • `moveWire` - Selects the Move Wire tool widget in the main window. • `cutWire` - Selects the Cut Wire tool widget in the main window. • `stretchWire` - Selects the Stretch Wire tool widget in the main window. • `addPoly` - Selects the Add Polygon tool widget in the main window. • `addBusGuide` - Selects the Edit Bus Guide tool widget in the main window. • `p2pRoute` - Selects the Point to Point Route tool widget in the main window.

Example

The following command is used to enter the Create Ruler mode in the design window.

```
gui_set_tool ruler
```

gui_set_ui

```
gui_set_ui
[-help]
name
[-before name]
[-checked {true | false}]
[-command string]
[-disabled {true | false}]
[-enable_variable string]
[-foreground string]
[-geometry string | -full_screen {true | false}]
[-icon filename]
[-in name]
[-label string]
[-message string]
[-movable {true | false}]
[-newline {true | false}]
[-shortcut string]
[-statusbar string]
[-title string]
[-tooltip string]
[-underline integer]
[-value string ]
[-variable string]
[-visible {true | false}]
[-visible_variable string]
```

Sets the specified property of an element. Use this command to customize the graphical interface by configuring various interface elements, such as submenus and commands.

For elements of type `menu`, `submenu`, `command`, `check`, `radio`, `separator`, `toolbar`, and `toolbutton`, the `gui_set_ui` command works like the tk command *`widget`*`configure`. For the main window (`main`), it works like the tk command `wm`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_set_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_set_ui`
`name`	Specifies the name of the element whose property is to be set. An error is returned if specified *`name`* does not exist. *Data_type*: string, required
`-before` *`name`*	Sets the specified element before the element identified by *`name`*. *Data_type*: string, optional
`-checked {true | false}`	Checks or unchecks the specified element. **Note:** You can use the `-checked` parameter only for elements of type `check` or `radio`. *Data_type*: bool, optional
`-command` *`string`*	Specifies the Tcl command to be executed when the specified element is invoked. **Note:** You can use the `-command` parameter only if you are adding elements of type `command, check, radio` or `toolbutton`. *Data_type*: string, optional
`-disabled {true | false}`	Disables or enables the specified interface element. **Note:** You can use the `-disabled` parameter only for elements of type `menu, submenu, command, check, radio` or `toolbutton`. The parameter is NOT applicable for elements of type `toolbar` and `separator`. *Data_type*: bool, optional
`-enable_variable` *`string`*	Sets the Tcl global variable for Enable status. *Data_type*: string, optional

`-foreground` *`string`*	Specifies the foreground color of the icon of the specified interface element. **Note:** You can use the `-foreground` parameter only if you are adding an element of type `toolbutton`. *Data_type*: string, optional
`-full_screen {true | false}`	
	Maximizes the main window. This property can be set only for the element `main`. **Note:** This option cannot be used with `-geometry`. *Data_type*: bool, optional
`-geometry` *`string`*	Modifies the geometry of the specified window element. *Data_type*: string, optional
`-icon` *`filename`*	Specifies the file name of the toolbutton icon. *`filename`* can be the full path of the icon file or an icon name supported by Innovus. The icon names used in Innovus can be queried using the `gui_get_ui` command. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`. *Data_type*: string, optional
`-in` *`name`*	Specifies the name of the parent element. *Data_type*: string, optional
`-label` *`string`*	Specifies the label to be displayed on the element being modified. **Note:** This parameter is NOT applicable for elements of type `separator`. *Data_type*: string, optional
`-message` *`string`*	Specifies the message to be displayed on the element being modified. **Note:** This parameter is applicable for elements of type `statusbar`. *Data_type*: string, optional

`-movable {true | false}`	Specifies whether or not the element being modified should be movable. **Note:** The `-movable` parameter is applicable only for elements of type `toolbar`. *Data_type*: bool, optional
`-newline {true | false}`	Specifies whether or not the element being modified should be on a new row. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`. *Data_type*: bool, optional
`-shortcut` *`string`*	Specifies the shortcut key for a command, check box, radio button, or toolbutton. **Note:** You can use the `-shortcut` parameter only if you are modifying elements of type `command`, `check`, `radio`, or `toolbutton`. *Data_type*: string, optional
`-statusbar` *`string`*	Sets the statusbar of the specified window element. *Data_type*: string, optional
`-tooltip` *`string`*	Specifies the tooltip text for the toolbar or toolbutton you are adding. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`. *Data_type*: string, optional
`-underline` *`integer`*	Specifies the integer index of the character to be underlined in the element label. The index begins with `0` and should have a value less than or equal to the length of the element label. For instance, if you want the second letter of the element label to appear underlined, specify *`integer`* as `1`. **Note:** The `-underline` parameter is NOT applicable for elements of type `toolbar, separator,` and `toolbutton`. *Data_type*: int, optional

`-value` *stringValue*	Specifies the value to be stored in the associated variable when the specified radio button is selected. **Note:** The `-value` parameter is applicable only for elements of type `radio`. *Data_type*: string, optional
`-variable` *string*	When modifying a check box or radio button, specifies the Tcl global variable for the item. *Data_type*: string, optional
`-visible {true | false}`	Specifies the visibility status of the element being modified. *Data_type*: bool, optional
`-visible_variable` *string*	Sets the Tcl global variable for Visible status. *Data_type*: string, optional

Accepted Properties for Each Type

The accepted properties for each type are listed below.

- For the main window (`main`), the following properties can be set:

  `-geometry {`*widthxheight* `|` *widthxheight+x+y*`}`

  `-title` *title*

  `-close_command` *string*

  `-visible true | false`

- For a status bar (`statusbar`), the following property can be set:

  `-enable_variable` *string*

  `-in` *parentname*

  `-message` *msg*

  `-visible true | false`

  `-visible_variable` *string*

- For elements of type `menu` or `submenu`, the following properties can be set:

  `-enable_variable` *string*

  `-before` *nextSiblingName*`]`

  `-disabled true | false`

```
-in parentName
-label label
-underline index
-visible true | false
-visible_variable string
```

- For elements of type `command`, `radio`, and `check`, the following properties can be set:

```
-disabled true | false
-label label
-underline index
-command commandName
-in parentName
-before nextSiblingName
-enable_variable string
-visible true | false
-visible_variable string
-shortcut string
```

  For `radio` and `check`, the `-checked` and `-variable` properties can also be set.

- For elements of type `toolbutton`, the following properties can be set:

```
-state normal | disabled
-label label
-tooltip toolTipText
-underline index
-foreground colorName
-icon iconName
-command commandName
-in parentName
-before nextSiblingName
-enable_variable string
-visible true | false
-visible_variable string
-shortcut string
```

Examples

- The following command changes the label of the *Verify* menu in the main window to *NewCheck*:

```
gui_set_ui verify -label NewCheck
```

gui_set_visible_nets

```
gui_set_visible_nets
[-help]
{nets | -all}
```

Displays only the specified nets and hides all other nets. To display all nets, specify the `-all` parameter.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_set_visible_nets` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_set_visible_nets`
`-all`	Displays all nets.
nets	Displays only the specified nets and hides all other nets. You can specify multiple net names.

Examples

- The following command hides all nets except the specified net:

  ```
  gui_set_visible_nets
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/tpo_a_9_
  ```

Related Commands

- `gui_get_visible_nets`

gui_show

```
gui_show
[-help]
```

Opens the Voltus main window.

If you run the `gui_show` command when the Voltus main window is already displayed, the tool retains the current zoom level and displays the main window without refreshing it. For example:

```
voltus 23> zoom_box 94 -640 95 -645
voltus 24> gui_show
```

This displays the main window with the design zoomed to the coordinates " `94 -640 95 -645` ".

If you started the Voltus software with the `voltus -no_gui` command, you can use `gui_show` to pop up the main window.

You can run the `gui_hide` command to hide the Voltus main window.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man gui_show`

gui_show_edge_number

```
gui_show_edge_number [-help]
[-objects string]
[-reset]
```

Displays edge numbers on the sides of the selected or specified objects in the GUI. Currently, you can display edge numbers on partitions and blocks. This feature is useful if you need to indicate the partition or block edge number in your script.

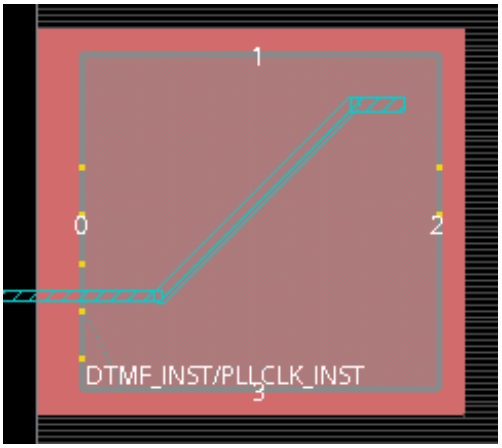
Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_show_edge_number` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_show_edge_number`
`-objects` *`string`*	Specifies the name of the object for which the edge numbers should be displayed. If you want to specify multiple object names, enclose them in braces or double quotation marks. Currently, you can display the edge numbers for partitions and blocks.
`-reset`	Clears the edge numbers from the specified or selected objects on the GUI. If you want to clear the edge numbers for all objects, use the `-reset` parameter with `-objects *`.

Examples

- The following command displays the edge numbers on the `DTMF_INST/PLLCLK_INST` block in the GUI:

  ```
  gui_show_edge_number -objects DTMF_INST/PLLCLK_INST
  ```

- The following command clears the edge numbers from all blocks:

  ```
  gui_show_edge_number -objects * -reset
  ```

gui_show_user_bundle_nets

```
gui_show_user_bundle_nets
[-help]
[-keep_existing]
-net_bundle bundleName
[-remove]
```

Displays flightlines for the specified net bundle. Bundled nets help you with data flow analysis before and after layout. Net bundles can be created and viewed in floorplan and amoeba views. When you run the `gui_show_user_bundle_nets` command, the software switches automatically to the net bundle mode and clears flightlines based on selection.

You can create flight line net bundles using the `gui_create_user_bundle_net` command and delete them using the `gui_delete_user_bundle_net` command.

Note: When selecting objects, you can choose to display flightlines for only bundled nets by using the *Only Show Net Bundle Flight Line* option in the Preferences form. If this option is not selected, the net bundle flightlines are displayed along with other flightlines.

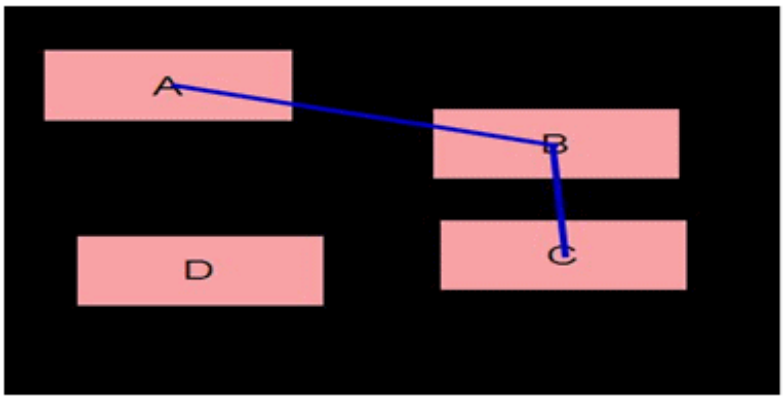
Parameters

`-help`	Prints a brief description that includes type and default information for each gui_show_user_bundle_nets parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_show_user_bundle_nets`
`-keep_existing`	Retains existing net bundle flightlines while displaying the flightline of specified net bundles. If this option is not specified, the command clears existing flightlines before displaying specified net bundle flightlines.
`-net_bundle` *bundleName*	Specifies the name of the net bundle whose flightlines you want to display. You can specify more than one net bundle.
`-remove`	Clears flightlines of specified net bundle from the display area. If no net bundle name is specified, all flightlines based on net bundles are removed. **Note:** This option only clears the flightlines of the net bundle from the display area. If you want to remove the net bundle itself, use the `gui_delete_user_bundle_net` command.

Examples

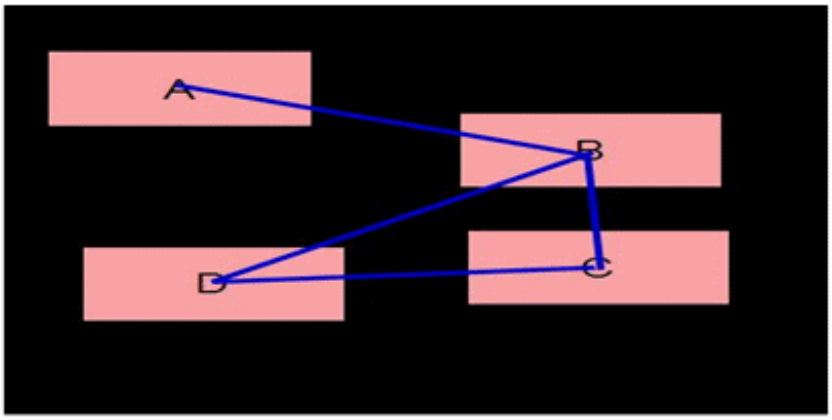
- The following command displays flightlines of the net bundle `bundle_1`. Here, `bundle_1` contains `net_1`, which is just between A and B, and `net_2`, which is just between B and C:

  ```
  gui_show_user_bundle_nets -net_bundle bundle_1
  ```

- The following command displays flightlines of the net bundle `bundle_2` while retaining flightlines of `bundle_1`. Here, `bundle_2` contains `net_3`, which is among B, C, and D:

  ```
  gui_show_user_bundle_nets -net_bundle bundle_2 -keep_existing
  ```

  Note: The same flightlines could also be displayed in a single step by using the following command:

  ```
  gui_show_user_bundle_nets -net_bundle {bundle_1 bundle_2}
  ```

gui_view_box

```
gui_view_box [-help]
```

Reports the coordinates of the current area being viewed in the GUI.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man gui_view_box`

Example

The following command reports the coordinates of the current area being viewed in the GUI:

```
gui_view_box
93.367 -646.465 103.294 -638.724
```

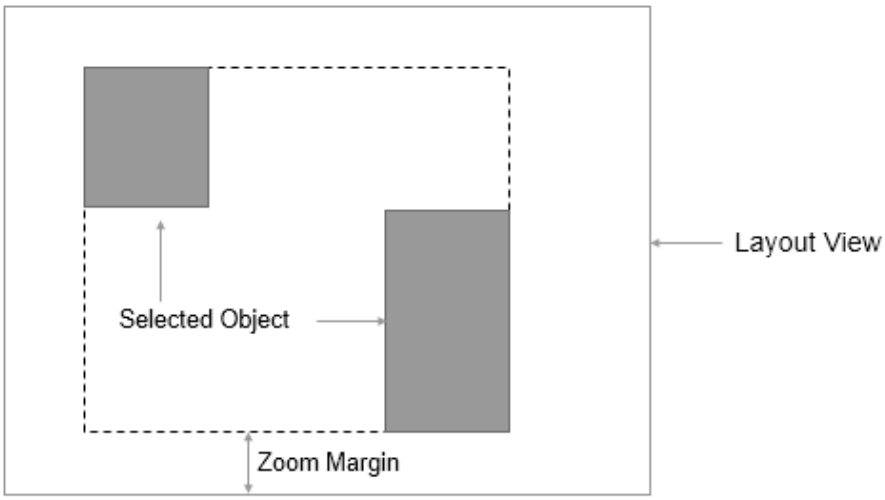
gui_zoom

```
gui_zoom
[-help]
[-to {x y}[-to_radius value]]
[-selected [-margin integer_value]]
{-in  | -out  | -selected  | -to {x y} | -rect {x1 y1 x2 y2}}
```

Zooms into or out of the viewable window as per the specified parameters.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_zoom` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_zoom`.
`-in`	Zooms into the viewable window by two times.

`-margin integer_value`	Specifies the zoom margin ratio for the viewable window. The zoom margin is the minimum spacing between the bounding box of the selected object(s) and the layout view.  The minimum value is `0` and the maximum value is `99`. The `gui_zoom -selected` parameter must be specified if you are specifying `-margin`. *Default*: 50
`-out`	Zooms out from the viewable window by two times.
`-rect {x1 y1 x2 y2}`	Zooms in the viewable window to show all of the rectangular area that is defined by the specified coordinates.
`-selected`	Zooms in the viewable window to the area that encloses the selected objects.
`-to {x y}`	Zooms in the viewable window to the point defined by the coordinates, `x` and `y`.
`-to_radius value`	Zooms in the viewable window to a radius of the specified value in microns, with center at the point specified by `-to {x y}`. *value* is of type float. *Default*: 150 microns **Note**: The `-to {x y}` parameter must be specified with this parameter.

Examples

- The following command zooms in the viewable window to the area enclosed by the lower-left coordinates (`450 531`) and the upper-right coordinates (`550631`):

  ```
  gui_zoom -rect {450 531 550 631}
  ```

- When the following command is run, the display zooms in to point with x=100 and y=200. The display area is centered at this point and has the default radius of 150 microns:

  ```
  gui_zoom -to {100 200}
  ```

- When the following command is run, the display zooms in to point with x=100 and y=200. The display area is centered at this point and has a radius of 50 microns:

  ```
  gui_zoom -to {100 200} -to_radius 50
  ```

read_power_rail_results

```
read_power_rail_results
[-help]
[-detail_delta_temperature_file file]
[-effective_resistance_file file]
[-instance_delta_temperature_file file]
[-power_db file ]
[-rail_directory dir]
[-tile_delta_temperature_file file]
[-cell_library { library1.cl}]
[-reset]
[-force]
[-custom_value_inst_file_1 file]
[-nets net_names]
[-instance_voltage_method { best | worst | avg | worstavg }]
[-instance_voltage_window { timing | whole }]
[-instances instancenames]
[-enable_plots plot_names]
[-signal_em_db file]
[-short_file file]
[-esd_directory dir]
[-thermal_file filename.txt]
[-esd_em  dir]
[-die_instance_name   dieinstname]
[-data_type {rail | all}]
[-physical_db dir_name]
[-profiling_power_file file]
[-top_cell_name topcell_name]
[-tile_thermal_config_file filename]
[-power_directory power_output_directory]
```

Loading power/rail analysis results into GUI.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_power_rail_results` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_power_rail_results`.
`-cell_library { library1.cl}`	
	Specifies the single cell library that is required to load the state directory. The specified cell library must contain the technology layer information that is required to load the state directory. Therefore, you do not need to load all cell libraries used during analysis.
`-custom_value_inst_file_1 file`	
	Specifies the name of the custom value instance file (CVIF). This file can be used to display a user-specified instance voltage file in GUI. The format of the CVIF file is: *value1* *instancename1* *value2* *instancename2* .... where, *value1* and *value2* are voltage values mapping to the specified instance. You can use the `cvil` argument of the `gui_set_power_rail_display -plot` command parameter to view the EIV map in GUI.
`-data_type {rail | all}`	
	Controls the type of data to be loaded in the GUI. The possible arguments of this parameter are: • `rail` - Loads only the rail analysis results. • `all` - Loads both the power and rail analysis results. This is the default value.
`-detail_delta_temperature_file file`	
	Specifies the name of the detailed wire-based delta temperature file from self-heating analysis.

`-die_instance_name` *`dieinstname`*	
	Specifies the name of the die whose results you need to view. This parameter is only applicable to the multi-die analysis flow (`set_rail_analysis_mode -die_mode multi-die`). The `-die_instance_name` parameter must be specified with the `-rail_directory` parameter (multi-die rail directory).
`-effective_resistance_file` *`file`*	
	Specifies the result file from effective resistance analysis (only instance-based result is supported). When the rail analysis plot type `effr` is selected and a result file is specified, the layout shows effective resistance analysis result for each instance. The `-effective_resistance_file` parameter is not supported in the Voltus-XP flow. **Note**: Currently, this feature only supports loading one net at a time.
`-enable_plots` *`plot_names`*	
	Specifies the list of power or rail plots names that are to be loaded.
`-esd_directory` *`dir`*	
	Specifies the ESD analysis directory.
`-esd_em` *`dir`*	
	Specifies the name of the directory where the current density/EM results generated by a previous ESD analysis run reside. This parameter allows you to selectively load the required EM result directory if there are multiple ESD EM result directories under the `ESD` directory. You must either load the rail directory (`-rail_directory`) before running this command, or specify the rail directory with the `-esd_em` parameter.
`-force`	

	Specifies to block the pop-up window that appears when the `read_power_rail_results -reset` parameter is specified. The pop-up window displays a confirmation message for resetting the power and rail directories. Example: `read_power_rail_results -reset -force`
`-instances` *instancenames*	
	Specifies a list of instances to load in the GUI. This parameter loads only those partitions that include the specified instances. The `-instances` parameter allows you to analyze parts of the designs, thereby reducing the loading time and memory footprint. The `-instances` parameter is applicable to the Voltus-XP flow only.
`-instance_delta_temperature_file` *file*	
	Specifies the name of the instance-based delta temperature file from self-heating analysis. This is the device instance temperature.
`-instance_voltage_method { best | worst | avg | worstavg }`	
	Loads the instance voltage data for the specified EIV computing method. If this parameter is not specified (default), the instance voltage results for all the four methods are loaded.
`-instance_voltage_window { timing | whole }`	
	Loads the instance voltage data for the selected EIV window. If this parameter is not specified (default), the instance voltage results for both the timing and switching windows, and the entire rail simulation are loaded.
`-nets` *net_names*	
	Loads only the specified nets in the domain-based state directory to the GUI. If this parameter is not specified with the `-rail_directory` parameter, all the nets will be loaded.
`-physical_db` *dir_name*	
	Specifies the name of the hierarchical design directory to be loaded to the GUI along with the rail analysis results.

`-power_db` *file*	
	Specifies the power database file. This file was created by using the `power_write_db` attribute.
`-power_directory` *power_output_directory*	
	Specifies to load the power analysis results and display the power plots directly without loading the rail analysis results.
`-profiling_power_file` *file*	
	Specifies to display the tile-based power density results generated from the vector profiling flow.
`-reset`	
	Frees the loaded analysis state directory or power database from the memory.
`-rail_directory` *dir*	
	Specifies the rail analysis directory. You can load all the domain results by selecting the automatically generated analysis directory under the output directory (`<domain>_<temp>_<dynamic/avg>_#`), or load individual nets by going one level further and selecting the net directory.
`-short_file` *file*	
	Specifies the path to the short violations report (`voltus_short.report`) . This file is generated using the `set_rail_analysis_config -report_shorts true` command parameter, and is located in the work directory.
`-signal_em_db` *file*	
	Specifies to read the signal electromigration data from the specified database file generated by the `check_ac_limits -report_db` command parameter.
`-thermal_file` *filename*`.txt`	

	Specifies to load the tile-based temperature map file generated by Sigrity Celsius to be used for both power calculation and IR drop analysis.
	The following is an example of reading the thermal map file:
	`read_power_rail_results -rail_directory output/static_rail/core_25C_avg_1/ -thermal_file output/power_map_by_layer.txt`
	`gui_set_power_rail_display -plot thermal`

`-tile_delta_temperature_file` *file*	
	Specifies the name of the tile-based delta temperature file of self-heating analysis. This is the worst instance temperature among all wire segments in a tile. This is also layer-based. The FEOL or cell layer is used to show the device instance temperature.
`-tile_thermal_config_file` *filename*	

	Specifies to load the configuration file that contains the mapping between the hierarchical levels with the corresponding layer-based temperature map files generated by Celsius (system-level thermal analysis tool). When specified, you can view tile-based temperature distribution for each layer of the chip.
	The following is the format and example of the hierarchical configuration file:
	<hierarchy> <block name> <X_tile_no,Y_tile_no> <LEF/DEF Flip/Rotation> <temperature map>
	`TOP DIE 0,0 N /thermal_data/DIE_TemperatureMap`
	`TOP/IP1 IP1 25,50 N /thermal_data/IP1_TemperatureMap`
	`TOP/IP1/B1 B1 50,125 N /thermal_data/B1_TemperatureMap`
	`TOP/IP1/B2 B2 200,125 S /thermal_data/B2_TemperatureMap`
	This is a mandatory parameter for the `tile_thermal` plot when loading hierarchical temperature maps.

`-top_cell_name` *topcell_name*	
	Specifies the name of the top module or top cell name to be loaded. This parameter must be specified with the `-physical_db` parameter.

Examples

- The following command specifies the appropriate files needed for viewing the instance voltage and power data:

```
read_power_rail_results\
    -rail_directory analysis/domain_25C_avg_1 \
    -power_db powermeter.db \
```

- The following command specifies to load the VDD and VSS nets for the worst and best EIV methods during the entire simulation:

```
read_power_rail_results -rail_directory dynamic_rail/core_25C_dynamic_1 \
    -nets { VDD VSS } -instance_voltage_window { whole } -instance_voltage_method
{ worst best }
```

- The following command specifies to load only the rail directory:

```
read_power_rail_results -rail_directory dynamic_rail/core_25C_dynamic_1 -data_type
rail
```

- The following command specifies to load the hierarchical database and the rail analysis results to the GUI:

```
read_power_rail_results -rail_directory ./dynamic_rail/latest/ -physical_db
/dma/Dynamic/saveDB -top_cell_name dma_mac
```

- The following command specifies to load the thermal configuration file and the rail directory in the XP mode:

```
read_power_rail_results -rail_directory xp_rail_directory -
tile_thermal_config_file ./thermal_data/DIE_Temperature_config
gui_set_power_rail_display -plot tile_thermal
```

- The following command specifies to display the tile-based power density results in the GUI:

```
read_power_rail_results -profiling_power_file
./vectorProfileResults/profile.rpt.density
```

- The following command specifies to load the power analysis results and display the power plots directly by running the power analysis without loading the rail analysis results:

```
read_power_rail_results -power_directory ./dynamic_power
```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

read_temperature_map_file

```
  read_temperature_map_file
[-help]
-file <string>
```

Loads the tile-based temperature map file generated by Sigrity Celsius to be used for displaying temperature.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-file <string>`	Specifies the name of the tile-based temperature map file generated by Sigrity Celsius.

report_power_rail_results

```
report_power_rail_results
[-help]
[-append]
[-enable_filter_summary]
[-file_name file ]
[-filter_max max_value]
[-filter_min min_value]
[-ignore_limit_bound]
[-layers {All, layer_names } ]
[-limit N ]
[-nets {all | all_pwr | all_gnd | net_names}]
[-plot plot_type]
[-range_max max_value]
[-range_min min_value]
[-region x1 y1 x2 y2]
[-rlrp_inst {instance_name}]
```

Generates RLRP, region-based, and rail/power/capacitance reports.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_power_rail_results` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_power_rail_results`.
`-append`	Appends instance reports to the current report file.
`-enable_filter_summary`	
	Displays a summary of the filtered data in the histogram given in the *Summary* tab of the Result Browser form.
`-file_name` *file*	
	Specifies the name of the text based violation report file to be created. The default file name is: • for region-based report: `region_x1_y1_x2_y2.rpt` • for RLRP report: `rlrp_instname.rpt` • for plot-based report: `power_rail_`*plot*`.rpt`
`-filter_max` *max_value*	
	Specifies the maximum value to include in the report. Use this option to further filter out the data within the range specified. This is used to obtain statistic of filter range. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-filter_min` *max_value*	

	Specifies the minimum value to include in the report. Use this option to further filter out the data within the range specified. This is used to obtain statistic of filter range. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-ignore_limit_bound`	
	Specifies to ignore the maximum upper limit of `10000`, and instead use the higher value given with the `-limit` parameter for specifying the number of data points to be printed in the power/rail report.
`-layers {all, layer_names }`	
	Specifies the layers to be reported. You can use `all` for reporting all layers, or list the specific layers that are to be reported. If this parameter is specified, you must specify the `-plot` parameter.
`-limit N`	
	Specifies the number of data points printed in the report. If this parameter is specified, you must specify the `-plot` parameter. *Default* : first `1000` data points
`-nets {all | all_pwr | all_gnd | net_names}`	

	Specifies the nets to be used for analysis reporting. You can use `all` for reporting all nets, `all_pwr` for reporting all power nets, `all_gnd` for reporting all ground nets, or list the specific nets that are to be reported. If this parameter is specified, you must specify the `-plot` parameter. This parameter also allows you to specify the switched and always-on nets to be used for analysis reporting. The following is the use model to generate a report for the specified switched net: `report_power_rail_results -plot ir -nets {`*`switchedNetName`*`}` The following is the use model to generate a report for the specified always-on net, including its switched nets: `report_power_rail_results -plot ir -nets {`*`alwaysOnNetName`*`}` **`-all`** You must specify `-all` to include all switched nets associated with the specified always-on net.
`-plot` *`plot_type`*	
	Plots the specified plot type. See Plot Types for a description of each of the plot types.
`-range_max` *`max_value`*	
	Specifies the maximum value to report. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-range_min` *`min_value`*	
	Specifies the minimum value to report. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-region` *`x1 y1 x2 y2`*	
	Measures tap current and capacitance information in the specified region defined by x1, y1, and x2 y2 coordinates, and prints results in the log file.
`-rlrp_inst` *`instance_name`*	

	Reports the RLRP path for the specified instance. The default file name of the RLRP path report is `<rlrp_instname.rpt>`. You can modify the file name using the `-file_name` parameter.

Plot Types

Plot Type	Description
`ac`	Available/feasible capacitance (Farad) Displays the feasible decoupling capacitance that can be added in the area. The feasible capacitance in the area is calculated based on the filler cells placement. The program matches the placed filler cells with the decap cells available in the power-grid library and computes the feasible capacitance. The capacitance of the decap cell is stored in the power-grid library.
`activity`	Displays the worst activity for the output and inout pins of every instance.
`cap`	Grid capacitance (Farad) Plots the capacitance of the extracted power-grid network.
`dd`	Decoupling capacitance (decap) density (Farad/um ^{2}) Plots decap density of the design. The decap density is the cell internal decoupling capacitance in the unit area (µm ^{2}).
`ivdn`	Instance Voltage Drop (Volts) – net-based Plots the net-based instance voltage drop.
`ivdd`	Instance Voltage Drop (Volts) – domain-based Plots the domain-based instance voltage drop. For domain based, at least one power and one ground net must be selected.

`fd`	Filler cell density (1/µm ^{2}) Displays the filler cell density per unit area (µm ^{2}). The plot can be used to analyze the feasibility of adding decap cells by replacing the filler cells and not changing the placement of the logic cells.
`freq`	Frequency domain (Hz) Displays the associated clock frequency of all instances in the design. The instances associated with multiple clock domains are displayed using the fastest clock frequency. This plot can be used to analyze and debug the power distribution in the design.
`ip`	Instance total power (mW) The total power consumed by each instance is displayed. The total power is the sum of the internal, switching and leakage power of the instance. The power data is read from the power database and can be specified using the `-power_db` option or it is queried directly from memory, if power calculation is run during the session.
`ip_i`	Instance internal power (mW) The power consumed by charging and discharging of the internal interconnect and device capacitances of the instance.
`ip_l`	Instance leakage power (mW) The power consumed by devices when not switching.
`ip_s`	Instance switching power (mW) The power consumed by charging and discharging of the interconnect or output load.
`ipd_i`	Specifies internal power density in mW/um ^{2} .
`ipd_l`	Specifies leakage power density in mW/um ^{2} .
`ipd_s`	Specifies switching power density in mW/um ^{2} .
`ipd_t`	Specifies total power density in in mW/um ^{2} .

`ip_tr`	Displays toggle rates for all instances. The toggle rate for an instance is calculated as the average of toggle rates of all outputs of the instance. This plot allows you to determine the design area that toggles more than the others.
`ir`	IRdrop or voltage drop (Volts) Displays the IRdrop across each extracted power-grid segment in the design.
`jrms`	Specifies current density as j/jmax based on the Irms current. This plot is only supported for dynamic analysis. A jrms gif file will be saved in the state directory that shows where the Irms based EM violation occurred. This plot type allows you to verify the analysis results of "jrms" using the GUI.
`load`	Loading capacitance (Farad) Displays the output loading capacitance driven by the instance. This plot can be used to analyze and debug the regions with high switching power.
`pi`	Power-switch (gate) current violation: I/Idsat Displays the ratio of current through the power-switch and the saturation current (Idsat) of the power-switch. This plot can be used to analyze and debug whether the current through power-switches exceeds the saturation current characterized for the power-switch devices. The saturation current and on-resistance of the power-switches are characterized and stored inside the power-grid library of the power-switch cell. A ratio of greater than 1 means that the current requirement of the power-gated block can not be met with placed power-switch instance. Either the power-switch needs to be made larger or power-switch instances need to be added.
`pv`	Voltage drop across power-switch (Volt) Displays the IRdrop across power-switch instances. This plot can be used to analyze and debug the IRdrop inside the power-gated block, e.g. if the IRdrop across power-switches is already high, the IRdrop inside the power-gated block will be much higher. In this case, the power-switch placement will have to be refined in order to resolve the IRdrop problem.

`rc`	Resistor current (Amp) Displays the current across power-grid resistor segments. This plot can be used to analyze and debug how the current flows from the voltage sources into the rest of the design, e.g. high current should flow from pads to the top-level routing level layers and then to the lower level metal layers. This plot can also be used to debug current density violations (`rj`).
`rlrp`	The total resistance of the instance along the least resistive path. Displays the total resistance of the instance along the least resistive path, meaning the resistance of the instance along the path from where it gets the most amount of current. This plot can be used to analyze and debug the power-grid integrity early in the design stage. An instance with high resistance is likely to have high static or dynamic IRdrop depending on the vector that is simulated later in the design cycle. This plot can be also used during decap optimization, since regions with high dynamic IRdrop and high resistance may not be improved upon by adding decaps, because the effectiveness of decoupling capacitance depends on the resistance of the power-grid network. You can also highlight the least resistive path of the instance using the GUI and selecting the instance with left mouse button. The *RLRP Path* checkbox should be enabled.
`res`	Displays the total resistance of the least resistive path from each node to voltage source. This plot can be used to view high resistance regions of the power-grid. It is useful during IRdrop and decap analysis to identify weak power-grid connectivity.
`rj`	Resistor current density, J/Jmax. Displays the current density (current per unit area) violation on each power-grid segment using the ratio of calculated current density and maximum allowed current density. A ratio of greater than 1 means that the current density limit of the segment is violated. The current density limits are supplied by the foundry and can be read during analysis using the `set_rail_analysis_mode -em_models` command.
rs	Specifies the plot type for resistor sensitivity.

`slack`	Instance slack (Seconds) Displays worst instance slack. The slack data is queried from either the timing database or if the power database is specified using `-power_db` option, then the worst slack data is read from the power database. When using an external TWF, the slack data is read from the TWF.
`tc`	Tap current (Amp) Displays the current consumed by the instance devices or current taps. During rail analysis, it is based on the power consumption of the instance and is distributed inside the cell. The current distribution inside the cell is based on the extracted device netlist stored inside the power-grid library of the cell. The current distribution is generally based on the device width, length, and spice models. This plot can be used to analyze and debug IRdrop inside the macro.
`td`	Transition density (Hz) Displays the worst transition density (activity * frequency) of the instance. This plot can be used to analyze and debug activity distribution and high power density regions.
`tpd`	Displays power density map with user-specified grid size.
`unc`	Unconnected segments Displays power-grid segments that are disconnected from the voltage source (power pads). This plot can be used to debug unusually high IRdrop in the region.
`vc`	Voltage source current (Amp) Displays current through the voltage sources (power pads). This plot can be used to analyze the current carrying capacity of the power-pad.
`vu`	Voltage drop across package (Volt) Displays voltage drop across package. If analysis is run with package model attached to the power pads, this plot displays the IRdrop due to package RLCK elements.

Examples

- The following command will report the worst 100 IR segments on Metal 1 of the VDD net:
  ```
  report_power_rail_results -plot ir -layers M1 -limit 100 -nets VDD
  ```
- The following command will report the IR drop of the VDD_ring switched net:
  ```
  report_power_rail_results -plot ir -net {VDD_ring}
  ```
- The following command will report the IR drop of the VDD_AO always-on net, including its switched nets:
  ```
  report_power_rail_results -plot ir -net {VDD_AO} -all
  ```
- The following command ignores the max limit of 10000 and prints 10008 values in the report file:
  ```
  report_power_rail_results -plot cap -limit 10008 -ignore_limit_bound
  ```
- The following command filters the data such that it does not display the data points below 200 in the Result Browser histogram:
  ```
  report_power_rail_results -plot ivdd -filter_min 200 -enable_filter_summary
  ```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

view_last

```
view_last [-help]
```

Displays the last view window or the previous view. This command provides the same function as the Zoom Previous icon (bindkey *w*).

Parameters

`-help`	Prints a brief description of the `view_last` command. For a detailed description of the command, use the `man` command: `man view_last`.

view_next

`view_next [-help]`

Displays the next view window. This command provides the same function as the *Next* toolbar widget. You can use the `view_next` command or the *Next* toolbar widget to switch from the current display to the next location in the *Go To Saved Area* list or the next zoom level in the zoom sequence saved in the design memory. The equivalent bindkey is `Y`. When you press the `Y` bindkey or execute the `view_next` command repeatedly, the tool cycles through all saved views (saved areas and zoom levels).

Parameters

`-help`	Prints a brief description of the `view_next` command. For a detailed description of the command, use the `man` command: `man view_next`.

Related Topics

- `gui_zoom`
- `view_last`

write_to_gif

`write_to_gif [-help] out_file`

Saves a snapshot of the current screen to a GIF file with the specified name in the current directory. You can use this command at any stage of the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `write_to_gif` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_to_gif`.
`out_file`	Specifies the name of the output file as a string.

Example

The following example captures the current screen and saves it to *`test1.gif`* in the current directory.

```
write_to_gif test1.gif
```

8

Import and Export Commands and Attributes

- add_route_via_defs
- add_route_vias Category Attributes
- check_design
- create_hier_design
- design Category Attributes
- init_design
- init_hier_design
- init Category Attributes
- read_db
- read_def
- read_hier_db
- read_libs
- read_mmmc
- read_netlist
- read_physical
- read_physical Category Attributes
- read_sdc
- read_twf
- relink_db_files
- reset_design
- set_cell_power_domain
- set_power_def_files
- set_power_lib_files

- set_power_spef_files
- specify_oa
- write_db
- write_db Category Attributes
- write_def
- write_def Category Attributes
- write_hier_db
- write_mmmc
- write_netlist Category Attributes
- write_via_defs
- write_twf

add_route_via_defs

```
add_route_via_defs
-help
[ -ndr_only ]
```

This command is used to auto-generate vias required by nanoroute for the default and non-default rules (NDRs). The vias are based on the wire-widths of the routing-rule and are created to meet all the DRC via enclosure rules with variations for good pin-access, variations of double-cut vias for good double-cut via utilization, and min-area vias required for stacked-vias that have no route connected to meet the min-area metal rule.

The command only creates vias in the current software session; they are not saved and restored for usage in a later software session. So this command is normally only used for experimentation, or possibly if you just want temporary extra DFM vias to be used for this one session. If you want to save the vias for usage by nanoroute in later software sessions, use `add_route_vias_auto true`. The command is not set to "on" by default as add_route_vias Category attributes are set to `false` by default.

The command requires correct DRC rules (e.g. enclosure, spacing, minimum-cut, min-area rules, etc), and at least one LEF VIARULE GENERATE statement (or OpenAccess standardViaDef) for every via layer used for routing (as required by OpenAccess). The cut enclosure values in the VIARULE GENERATE statement are not used except for old LEF technology files (LEF version 5.5 and older) that have no enclosure rules (this is not recommended usage, but is kept for backward

compatability). The VIARULE cut-spacing values are ignored–only the DRC cut-spacing values are used.

No predefined vias are required (e.g LEF VIA DEFAULT, or NONDEFAULTRULE USEVIA or an OpenAccess validVias list for any routing ConstraintGroup are not needed).

Any predefined vias that do exist, are kept unless `add_route_vias_delete_via_before_generation` is used (see below). Generated vias are compared to predefined vias and if they are geometrically identical, that specific generated via is not kept so no duplicate vias are created.

Note: NDRs that have the same widths as the default rule will inherit the same vias, including any pre-defined vias.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `add_route_via_defs` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man add_route_via_defs`.
`-ndr_only`	Specifies vias generation for non-default rules only. When specified, it generates signal routing vias for all NDR (non-default rules) and skips generating via for default rule. It is always used for designs having full set of default rule vias, but not sufficient non-default rule vias.

Example

- `add_route_via_defs`
  Will add auto-generated vias to any existing vias for all routing rules.
- The following command generates vias for all NDRs, including the NDRs created by create_route_rule command.
  `add_route_via_defs -ndr_only`

  You must use `add_route_vias_auto true` and `add_route_vias_ndr_only true` in the flow.

See also:

- `write_via_defs` to write out the vias and look at them.

add_route_vias Category Attributes

The root attributes in the `add_route_vias` category are:

- add_route_vias_auto
- add_route_vias_ndr_only

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_route_vias` category, use the following command:

```
get_db -category add_route_vias
```

add_route_vias_auto

```
add_route_vias_auto {true | false}
```

Default: `false`

Enables auto via generation when design is loaded. The option is of type bool.

add_route_vias_ndr_only

```
add_route_vias_ndr_only {true | false}
```

Default: `false`

Generates vias for non default rules only.

check_design

```
check_design
[-help]
[-out_file fileName]
[-type {{power_intent feedthru pin_assign budget place opt cts route signoff all} | -
no_check ]
```

Checks that all prerequisites to run all or part of the hierarchical or block implementation flow are met.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `check_design` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_design`.

`-type`	Specifies the category of the flow to be checked: • `assign_statements`: Performs checks for the netlist for assign statements. • `power_intent`: Performs checks related to MSV setup including power domain/fence checks. • `timing`: Performs checks related to timing setup integrity. • `feedthru`: Performs checks related to feedthru insertion for partitioning flows. • `pin_assign`: Performs checks related to pin assignment for partitioning flows. • `budget`: Performs checks related to time budgeting for partitioning flows. • `place`: Performs checks related to placement such as PG rail alignment and pin access issues. • `opt`: Performs checks related to optimization such as checks for available cells. • `cts`: Performs checks related to clock tree definition such as constraint targets and route types. • `route`: Performs checks related to routing such as congestion, placement, and pin access issues. • `signoff`: Checks related to the signoff activities including extraction and ECOs. • `all`: Runs all categories.
`-no_check`	Just echo out all the messages that would be checked, don't do any checks Do not run the checks but echo the checks that would run per the categories provided by the user
`-out_file` *`fileName`*	Specifies the output report file. Contents dumped to this file in the Tcl format.

create_hier_design

```
create_hier_design
[-help]
[-add_on_shape_def {{def1 x_loc y_loc orient} {def2 x_loc y_loc orient}...} ]
-def list_of_DEF_files
-hdb_dir dir_name
[-keep_blockages]
-lef list_of_LEF_files
[-rdl_def def_file]
[-rdl_orient { N|S|W|E|FN|FS|FE|FW }]
[-rdl_placement {X Y}]
[-skip_restore]
[-skip_signal]
[-top_cell cell_name]
```

Specifies to create and load a hierarchical database (HDB), that is, it constitutes the functions of the `init_hier_design`, `write_hier_db`, and `read_hier_db` commands. The command allows you to work with HDB-based GUI in the multi-CPU mode.

It can read multiple DEF files for the design with multiple levels of hierarchy and display a flattened database. When multiple DEFs are passed to `create_hier_design`, the software flattens all the levels of hierarchical DEFs and displays the design layout.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-add_on_shape_def {{def1 x_loc y_loc orient} {def2 x_loc y_loc orient}...}`	

	Specifies to merge physical shapes (such as pseudo what-if shapes) from additional side DEF files to the current top database. This parameter allows you to add multiple side DEF files together. For each DEF file, you need to specify the DEF file name (def1, def2), x and y or the physical shape coordinates, and the shapes's orientation. A shape can have any of the following orientations: N (North), S (South), W (West), E (East), FN (Flipped North), FS (Flipped South), FW (Flipped West), or FE (Flipped East).
`-def` *`list_of_DEF_files`*	
	Specifies the name of the DEF files, starting with the top-level DEF.
`-hdb_dir` *`dir_name`*	
	Specifies the name of the directory that stores the design data.
`-keep_blockages`	
	Specifies to load all routing and placement blockages.
`-lef` *`list_of_LEF_files`*	
	Specifies the name of the LEF files for loading the hierarchy design.
`-rdl_def` *`def_file`*	
	Specifies to instantiate the RDL instance/DEF at the top level and merge it with the DEF of the top level block being analyzed.
`-rdl_orient { N|S|W|E|FN|FS|FE|FW }`	
	Specifies the orientation of the RDL DEF file.
`-rdl_placement {X Y}`	
	Specifies to place the RDL at a given X,Y location (in micron).
`-skip_restore`	Specifies that the saved HDB design should not be restored.
`-skip_signal`	Reads PG nets with `USE POWER/GROUND` attributes in both `SPECIALNETS/NETS` sections in the input list of DEF files. Skips signal nets when reading [*list_of_DEFs*].

-top_cell *cell_name*	
	Specifies the top cell name.

Examples

- The following command specifies to load an HDB in the distributed mode:

```
set_distribute_host \
-lsf \
      -queue ssvpe \
      -timeout 500000 \
      -lsf_args {-P VOLTUS} \
      -resource  {rusage[mem=2800] select[ OSNAME==Linux &&
(OSREL==EE60||OSREL==EE50)]}

set_multi_cpu_usage -remoteHost 8 -cpuPerRemoteHost 16 -localCpu 16
create_hier_design -skip_signal -lef ../hdb.lef -def ../hier1.def ../hier2.def
../hier3.def  ../hier4.def -top_cell MURKKU2 -hdb_dir saveDB
```

- The following command specifies to load an HDB with physical shape DEF files:

```
create_hier_design -lef $lefFiles \
-def $defFiles \
-skip_signal \
-add_on_shape_def {{/../rdl.def -1000 -1000 FS} {/../rdl.def -1000 -1000 N}
{/../rdl.def -1000 -1000 FW} } \
-rdl_def /../rdl.def \
-rdl_placement {-1000 -1000} \
-rdl_orient {W} \
-top_cell DMA_WRAP \
-hdb_dir saveDB
```

design Category Attributes

The root attributes in the design category are:

- design_backside_bottom_routing_layer
- design_backside_top_routing_layer
- design_bottom_routing_layer
- design_compressed_pg_db
- design_cong_effort
- design_early_clock_flow
- design_high_freq_interposer_flow
- design_ideal_hold_fixing
- design_merge_trim_shapes
- design_optimization_density_screen_margin
- design_pessimistic_mode
- design_process_node
- design_tech_node
- design_top_routing_layer
- design_trim_grid_group

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `design` category, use the following command:

```
get_db -category design
```

design_backside_bottom_routing_layer

`design_backside_bottom_routing_layer` *layerName*

Default: `""`

Data_type: string, read/write

Specifies the highest backside LEF layer name for global and detail routing.

design_backside_top_routing_layer

```
design_backside_top_routing_layer layerName
```

Default: `""`

Data_type: string, read/write

Specifies the lowest backside LEF layer name for global and detail routing.

design_bottom_routing_layer

```
design_bottom_routing_layer layerName
```

Default: `""`

Data_type: bool

Specifies the lowest LEF layer name for global and detail routing.

design_compressed_pg_db

```
design_compressed_pg_db {true | false}
```

Default: `false`

Data_type: bool

Specifies whether the compressed PG feature is enabled. When it is set to `true`, `read_db` will automatically compress the PG data while loading the design. You can use `get_db current_design .is_pg_compressed` to check if PG in the design is compressed.

design_cong_effort

```
design_cong_effort {low medium high auto}
```

Specifies the effort level for relieving congestion.

Default: `auto`

Specify one of the following values:

- `auto`: Automatically determines whether the design is congested and performs extra congestion driven effort for highly congested designs.
- `high`: Runs more iterations of placement in an effort to achieve better congestion results. This parameter increases the run time.
- `low`: Runs fewer iterations of placement to arrive quickly at a legal placement. This parameter might decrease placement quality.
- `medium`: Runs placement on designs at a normal effort level.

design_early_clock_flow

```
design_early_clock_flow {true | false}
```

Default: `false`

Enables the early clock flow.

design_high_freq_interposer_flow

```
design_high_freq_interposer_flow {true | false}
```

Default: `false`

Data_type: `bool`

Specifies the flow as a high frequency interposer flow. When set to `true`, `set_integration_route_constraint` will auto turn off the options which are not suitable for interposer routing.

design_ideal_hold_fixing

```
design_ideal_hold_fixing {true | false}
```

Default: `false`

Data_type: bool, read/write

Enables the ideal hold fixing flow.

design_merge_trim_shapes

`design_merge_trim_shapes {{layers` *`layer_name_list`* `gap` *`gap_value`* `max_merge_num` *`number_of_max_merge`* `masks {`*`mask_num ...`*`}} ...}`

Default: `""`

Data_type: string, read/write

Specifies the setting for trim shapes merge.

- `layers`: You use this t specify the trim layer list which needs to merge trim shapes.
- `Gap`: You use this setting when the end to end gap is less the value.
- `max_merge_num`: You use this to specify the max trim shapes number that are merged.
- `masks`: You use this to specify the trim shapes color that needs to be merged.

design_optimization_density_screen_margin

design_optimization_density_screen_margin `{{layers` *`layer_name_list`* `gap` *`gap_value`* `max_merge_num` *`number_of_max_merge`* `masks {`*`mask_num ...`*`}} ...}`

Default: `0`

Min: `0`
Max: `1`

Data_type: float, read/write

Increases margin on the top of original density screen for optimization.

design_pessimistic_mode

`design_pessimistic_mode {true false}`

Default: `false`

Specifies that the delay calculation compatibility mode should be used for correlating to external signoff analysis tools. This mode adjusts delay calculation settings to ensure the best match to the third party engine while maintaining a degree of pessimism to help limit optimistic outliers in the external signoff analysis tool.

design_process_node

```
design_process_node ProcessTechnologyValue
```

Default: `90`

Specifies the process technology value to set for all the applications. Units in nanometers (nm).

design_tech_node

```
design_tech_node {N12 N10 N7 N7Plus N6 N5 N3 S11 S10 S8 S7 S5 S4 S3 G7 G5 ICF I7
unspecified}
```

Default: `unspecified`

Sets the design technology node.

The node value is normally automatically set by `read_physical` while loading the LEF technology and the tool prints a message mentioning the auto-detected node. However, if the auto-detection does not work, you will see a message like this:

```
## Check design process and node:
## Both design process and tech node are not set.
```

In such cases, you will need to set the value directly using `design_process_node` and `design_tech_node`.

design_top_routing_layer

```
design_top_routing_layer layerName
```

Default: `""`

Data_type: string

Specifies the highest LEF layer name for global and detail routing.

design_trim_grid_group

```
design_trim_grid_group groupName
```

Default: `""`

Specifies the GROUP name of the set from which the trim metal grid will be used for placement and routing.

If the specified group name does not exist in LEF, the tool will return an ERROR.

See the LEF documentation on the TRIMMETALTRACK keyword for more details.

init_design

```
init_design [-help]
```

Initializes the database and ensures that the tool is ready for full execution.

You must use the following commands in the order shown below before calling `init_design`.

1. set_db to set various init root attributes as desired. Normally init_power_nets and init_ground_nets are set here. See init Category Attributes chapter for all the init root attributes. You can see the current values with: get_db init*
2. read_mmmc to read timing data from an MMMC file. This step does error checking and creates MMMC objects (analysis_views, delay_corners, etc.). It reads in any timing libraries required by the set_analysis_view command inside the MMMC file, but it defers reading SDC and other files until they are needed later. This is optional for Innovus but required for Tempus/Voltus.
3. read_physical to read in physical technology and library data from LEF or OpenAccess. This is required for Innovus, and optional for Tempus/Voltus.
4. read_netlist to read in the netlist from Verilog or OpenAccess. This is required, and the design and netlist objects are created at this time.
5. read_power_intent to read in CPF or IEEE 1801 files. The files are read in and various error checks are done, but no power_domain objects are created until init_design is called. This step is optional.
6. `init_design` to step through the defined MMMC objects, the netlist objects, and the power intent data to build up the full representation of the design. After initialization is complete, the design is in a consistent state and is ready for timing analysis, floorplanning, placement, etc. Any data required by active views from set_analysis_view inside the MMMC file (e.g. SDC files) is read in at this time.

The `init_design` command can be broken into the following sub-steps:

1. Power Domain Creation. The power_domain objects are created and then bound to the design data as defined in the CPF or IEEE1801 files. At the end of this sub-step, every instance in the system is associated with its power domain.
2. Bind Power Domains to Timing Conditions. The power_domains are bound to the correct timing_conditions (library_sets and opconds) based on the `create_delay_corner` commands in the MMMC file. In this sub-step, the active delay_corners are processed with each instance getting assigned to the appropriate library_set and opcond.
3. Constraint Loading. The SDC files (and any other timing related files) required by the active

analysis_views are loaded and error checking is done.

4. Active View Setting. The final step is to set the active views based on the `set_analysis_view` command in the MMMC file, and to perform final checks to ensure any active analysis_view is fully and completely defined. Now the design is ready for execution.

Timing Incremental Updates

After `init_design`, you can enter new MMMC commands (e.g. create_analysis_view, create_delay_corner, etc.) at the command line. These commands populate the mmmc database objects, and perform error checking to report missing data. No data load takes place during command entry. Instead, the loading of new libraries, constraints, and so on, is deferred until a set_analysis_view command that requires the new data is given.

Note: Reading new design data (netlist) and new physical data (LEF/OA) incremental updates is not allowed. After the first `init_design` is completed, these steps are effectively disabled. If you issue a `read_netlist` or `read_physical` command after the first `init_design`, the tool returns an error.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man init_design`

Example

Here is an example of a typical init script.

```
set_db init_power_nets {VDD VDD2}

set_db init_ground_nets VSS

read_mmmc mmmc.tcl                      #Only .lib files for the active views are read at
this time

read_physical -lef {tech.lef stdcell.lef} #The technology LEF must be first

read_netlist {design.v block.v}

read_power_intent -cpf design.cpf

init_design
```

Related Topics

- init Category Attributes
- read_netlist
- read_physical

init_hier_design

```
init_hier_design
-defs list_of_DEF_files
[-help]
[-design_orient { N|S|W|E|FN|FS|FE|FW }]
[-design_offset {X Y}]
[-keep_blockages]
[-rdl_def def_file]
[-rdl_orient { N|S|W|E|FN|FS|FE|FW }]
[-rdl_offset {X Y}]
[-skip_signal_nets]
[-top_cell cell_name]
```

The `init_hier_design` command can read multiple DEF files for the design with multiple levels of hierarchy and display a flattened database. When multiple DEFs are passed to `init_hier_design`, the software flattens all the levels of hierarchical DEFs and displays the design layout.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-defs list_of_DEF_files`	
	Specifies the name of the DEF files, starting with the top-level DEF.
`-design_orient { N|S|W|E|FN|FS|FE|FW }`	
	Specifies the orientation of the top DEF block.
`-design_offset {X Y}`	
	Specifies to place the top DEF block at a given X,Y location (in micron) with respect to virtual top (0,0).
`-keep_blockages`	
	Specifies to load all routing and placement blockages.
`-rdl_def def_file`	

	Specifies to instantiate the RDL instance/DEF at the top level and merge it with the DEF of the top level block being analyzed.
`-rdl_orient { N|S|W|E|FN|FS|FE|FW }`	
	Specifies the orientation of the RDL DEF file.
`-rdl_offset {X Y}`	
	Specifies to place the RDL at a given X,Y location (in micron).
`-skip_signal_nets`	Reads PG nets with `USE POWER/GROUND` attributes in both `SPECIALNETS/NETS` sections in the input list of DEF files. Skips signal nets when reading [*list_of_DEFs*].
`-top_cell` *cell_name*	
	Specifies the top cell name.

Examples

- The following command reads the hierarchical DEF files：

```
init_hier_design -defs $designDir/Data/def/design1.def $designDir/Data/def/Cpu.def
$designDir/Data/def/design2.def -skip_signal_nets
```

init Category Attributes

The root attributes in the `init` category are:

- init_check_netlist
- init_delete_floating_hnets
- init_keep_empty_modules
- init_oa_tech_lib
- init_read_netlist_allow_port_mismatch
- init_read_netlist_allow_undefined_cells
- init_sync_relative_path
- init_power_intent_files
- init_timing_enabled

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `init` category, use the following command:

```
get_db -category init
```

init_check_netlist

```
init_check_netlist {true | false}
```

Default: `false`

After reading in the netlist, it performs the same checks as `check_design -netlist`.

Related Commands

- `init_design`

init_delete_floating_hnets

```
init_delete_floating_hnets {true | false}
```

Default: `true`

If `true`, an hnet (Verilog wire statement) that has nothing connected to it, is deleted from the netlist. The default value is true in Voltus.

init_keep_empty_modules

```
init_keep_empty_modules {true | false}
```

Default: `true`

Keeps empty modules. If set to `0`, the software converts the empty modules into special leaf cells that do not have a physical library associated with them.

If the module is a hard macro, using a value of `1` or `0` produces the identical behavior: the software keeps the module.

Related Commands

- `read_db`

init_oa_tech_lib

```
init_oa_tech_lib lib_name
```

Default: `{}`

Data_type: string, read/write

Specifies the OpenAccess library to use as a source of technology information. If not specified, the first library in the `init_oa_ref_lib` list will be used. Applies only when `read_netlist` is used to process a Verilog netlist. It is set by the `read_physical -oa_tech_lib` command.

init_read_netlist_allow_port_mismatch

```
init_read_netlist_allow_port_mismatch {true | false}
```

Default: `0`

Creates a dummy .lib file a

init_read_netlist_allow_undefined_cells

```
init_read_netlist_allow_undefined_cells value
```

Default: `false`

Specifies whether the new Instance ports can be created during the Verilog netlist parsing.

Related Commands

- `init_design`

init_sync_relative_path

```
init_sync_relative_path {true | false}
```

Default: `false` (You must change your working directory to the directory saved in the previous design session before you restore the design.)

Synchronizes all relative paths in the top.globals file to the current working directory. Use this parameter if you want to restore the design without first changing your current working directory to the directory saved in the previous design session.

Example

- The following command synchronizes all relative paths to the current working directory:
  ```
  setImportMode -syncRelativePath 1
  ```

Related Commands

- `read_db`

init_power_intent_files

```
init_power_intent_files string
```

Default: ""

Specifies the power intent files read by the previous run of the `read_power_intent` command.

init_timing_enabled

```
init_timing_enabled {true | false}
```

Default: `true`

If `read_physical` is issued before `read_mmmc` or `read_libs` in Genus and Tempus, an error will be issued. In Innovus, doing this will result in `init_timing_enabled` being set to `false` by the tool. Innovus is now in physical only mode, and any attempt to set timing data will result in an error.

read_db

```
read_db
[-help]
[-debug]
[-no_timing_graph]
[-physical_data]
[{{in_dir} [-mmmc_file fileName]}
 |{-oa_lib_cell_view {libname cellname viewname} [-mmmc_file fileName]}]
[-setup_views setup_views -hold_views hold_views]
[-leakage_view analysis_view -dynamic_view analysis_view]
```

Loads the saved design files of a previous design session. The files included depend on the work completed in the design session when it was saved. The directory can include floorplan files, special route files, placement files, trial route files, and power switch state files.

To restore designs saved in Voltus format, usage is as follows:

```
read_db sessionName.dat
```

Note: You must rebuild the timing graph after you restore the design. Run extraction, then timing analysis.

You must specify all pin types in the LEF file. Without these definitions, the software attempts to determine the pin type from the pin name when restoring a design, and does not always do this correctly. If this is the case, the software generates the following error message:

```
**ERROR: Current net list does not match wires file.
```

Modifying the contents of the `sessionName.dat` directory created by `write_db` can cause major unexpected results. Never modify the `design.dat` directory contents.

To restore OpenAccess designs, usage is as follows:

```
read_db
{-oa_lib_cell_view string}
```

You can use the `read_db` command after starting the software.

Note: The software supports only the following begin/end extension values for DEF `NETS` style wiring segments (0, width/2 or LEF `LAYER WIREEXTENSION`). If any of the wire extensions has unsupported extension value, then software converts it to a special wire (DEF `SPECIALNETS`). While writing back OpenAccess (`oaOut` or `write_db -oa_lib_cell_view`), these special wires will again be converted back to DEF `NETS` style wiring.

From the 14.1 release, the `read_db` command accepts a tar file, when a tar ball is given. For instance, if you specify the following command:

```
read_db myDesign.dat.tar.gz
```

`read_db` will untar the tar ball under the current working directory, restore the design, and then remove the temporary data directory. This enables a single DB file operation all the time.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `read_db` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_db`
`-debug`	Enables data debug.
`-dynamic_view` *analysis_view*	Specifies the analysis view for the dynamic power analysis.
`-leakage_view` *analysis_view*	Specifies the analysis view for leakage power analysis.
`-oa_lib_cell_view` {*libname cellname viewname*}	Specifies the library name, cell name, and view name for restoring design information from the previous design session in the OpenAccess database format.

`-mmmc_file`	Specifies the path name to a Multi-Mode/Multi-Corner (MMMC) configuration file (`viewDefinition.tcl`). Use this option if you want to override the settings in the existing MMMC configuration file in the design being restored and instead load a different MMMC file.

`-no_timing_graph`	Disables reading of timing libraries and constraints during the restore design process. **Notes** • Most designs that use LEF rely on the `.lib` information to define the bus order for MACROs that have bus bit terminals. Therefore, using the `-no_timing_graph` option can result in the connectivity to bus ports of leaf level instances being reversed. When `.lib` information is not available, the default is for bus ports to be treated as descending (for example, [7:0]). When using Verilog modules or OpenAccess abstract views to define the bus order, the connectivity will still be correct even when `-no_timing_graph` is specified. • Timing libraries cannot be loaded after a design is restored using the `-no_timing_graph` option. Use `read_db` again without the `-no_timing_graph` option to reload the design with timing libraries and constraints. • If you save a design after `read_db -no_timing_graph`, it will not save timing information (`viewDefinition.tcl`). That is in the `no_timing` session, `write_db` cannot save `viewDefination.tcl` search dir in `*.globals`: 1. `read_db -no_timing_graph` 2. `write_db new.dat` (`viewDefination.tcl` will not be saved in `new.dat`)
`-hold_views` *`hold_views`*	Specifies the hold views for loading, rather than the hold views active during `write_db`.
`-physical_data`	Enables physical data to be read.

in_dir	Specifies the directory from which to load the files. If the session name of the saved design includes an extension `.inn`, make sure that you include this extension in the *in_dir* name: for example, `session1.inn`.
`-setup_views` *setup_views*	Specifies the setup views for loading. Rather than the setup views active during `write_db`.

Examples

- The following command loads the saved design files from the `test.dat` directory:
  ```
  read_db test.dat
  ```
- The following command restores the `my.dat` design without loading the timing libraries and MMMC configuration information:
  ```
  read_db -no_timing_graph my.dat
  ```
- The following command restores the `placed.dat` design but overrides the existing MMMC settings in the design with the settings in the `new_mmmc.tcl` file:
  ```
  read_db placed.dat -mmmc_file new_mmmc.tcl
  ```
- The following command restores the `"designLib top placed"` OpenAccess cellview but overrides the existing MMMC settings in the design with the settings in the `new_mmmc.tcl` file:
  ```
  read_db -oa_lib_cell_view "designLib top placed" -mmmc_file new_mmmc.tcl
  ```

read_def

```
read_def
[-help]
[def_files]
[-design]
[-design_offset (x y)]
[-design_orient orientation]
[-force_create_phys_inst]
[-keep_pin_geometry]
[-preserve_shape]
[-rdl_def def_file]
[-rdl_offset (x y)]
[-rdl_orient orientation]
[-read_flat_dir directoryName]
[-skip_signal_nets]
[-write_flat_dir directoryName]
[-skip_pg]
[-top_scope]
[-top_scope_ignore_block_internal_nets_on_boundary_path]
```

Loads the specified DEF file. `read_def` can be used at any step after importing a design. The typical usage is:

- Load a DEF file containing the floorplan saved by Voltus or other tool.
- Load a DEF file containing the scan chain information so that placement can do scan chain reorder.
- Load a DEF file containing the cell's placement and routing saved by Voltus or other tool.

> When you load a DEF file, you must be at the same level in the hierarchy where the DEF file was saved.

The software loads the DEF file information into the database in several ways, depending on the object:

- Tracks, gcells, rows, die area
  The software deletes all of the existing objects in the database and reads in the new information.

  Note: The `read_def` command can read a rectilinear die area, but converts it to a rectangular die area with blockages at the cut off area.

- Blockages, fills, special nets
  The software compares the physical data (such as shape and layer) to the existing data in the database. If it finds the same data in the database, it ignores the data in the DEF file. If the data does not already exist, the software adds it to the database. For special nets, the software merges wire segments in the DEF file with any existing ones in the database that overlap and have the same attributes, such as layer, width, shape, and direction. The software does not delete any existing data.
- Nondefault rules, vias
  The software adds the objects to the database if they do not already exist there. If a nondefault rules definition with the same name already exists, the software ignores the one in the DEF file. If a via definition with the same name already exists, and the via is a fixed via, the software ignores the one in the DEF file. If a generated via definition with the same name already exists, the geomoetries are read, but the name might be changed. The software does not delete any existing objects of these types.
- Nets
  For each net in the DEF file, the software removes the existing regular routing (not special routing), then adds the routing from the DEF file for this net.
- Pins
  For each pin in the DEF file, the software removes the existing physical information for the pin, then adds the physical information from the DEF file for this pin. If a pin does not already exist, and it is a power or ground pin, the software adds it to the database. If the pin is not a power or ground pin, the software generates an error message.
- Groups, regions
  If a group name in the DEF file matches an existing hierarchical instance name in the database, the region's boundary constraint is attached to the hierarchical instance, overriding any existing boundary constraint. The software checks whether the group members belong to the hierarchical instance, and generates a warning message if any do not belong. If a group name does not match any hierarchical instance name, the software creates the instance group with an attached region boundary constraint, if one exists. Regions that are not used by groups in the same DEF file are ignored. The software does not remove existing groups.

  Note: You can import DEF files containing wildcards (`*`) in the `GROUPS` statement. The `read_def` command does not support multiple groups per region.
- Components
  You cannot add new logical cells. You can add new physical cells, such as:
  - Cells with `CLASS PAD`

- Cells with `CLASS COVER` and no signal pin
- Instance with `+ SOURCE DIST`

You can use the `read_def` command after importing a design.

Note: Do not attempt to use the `read_def -verilog_from_def_netlist_flow` command in any other Tcl scripts.

After loading the DEF netlist using the `read_def -verilog_from_def_netlist_flow` command, you can perform floorplanning, non-timing driven placement and routing, wire Innovusting, and verification. You cannot use the DEF netlist flow for parasitic extraction, delay calculation, and timing-driven placement and routing effectively because the DEF names do not properly match the Verilog names used in timing constraints and timing analysis.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `read_def` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_def`
`def_files`	Specifies the names of the DEF files to load.
`-design`	Specifies to read def files with the same top cell name
`-design_offset (x y)`	Specifies the placement for the topcell.
`-design_orient orientation`	Specifies the orientation for the topcell.
`-force_create_phys_inst`	Specifies to force create physical cell instances.
`-keep_pin_geometry`	Keeps pin shapes, equivalent to the block pin shapes, in the form of special route.
`-preserve_shape`	

	Specifies to prevent creation of symbolic center-line connected routing data and preserve the routing data as-is.
`-rdl_def` *`def_file`*	Specifies the RDL def file.
`-rdl_offset (`*`x y`*`)`	Specifies the offset of RDL def.
`-rdl_orient` *`orientation`*	Specifies the orientation for RDL def.
`-read_flat_dir` *`directoryName`*	Reads flat data from the specified directory.
`-skip_pg`	Skips the power and ground nets with `USE POWER/GROUND` attributes when reading the input list of DEF files. This parameter can be used to reduce the memory footprint in the signal electromigration flow.
`-skip_signal_nets`	Specifies to skip reading signal nets.
`-top_scope`	Reads the top-level nets only, and skips creation of the physical nets. By default (when `-top_scope` is not specified), the `read_def` command will create physical nets in the physical database if nets in the `SPECIALNETS` section do not exist in the database. This parameter must be used in the top scope signal electromigration flow to skip reading of these physical nets. The `read_def -top_scope` parameter can be used only after the running the `create_top_scope` command.
`-top_scope_ignore_block_internal_nets_on_boundary_path`	

	Specifies to ignore the physical layout of the internal nets from the first instance to the first flop on the boundary path for the signal EM checking. If this parameter is used with `read_def`, you must specify the same parameter with `verify_AC_limit`. The `read_def -top_scope_ignore_block_internal_nets_on_boundary_path` parameter can be used only after the running the `create_top_scope` command. The `read_def -top_scope_ignore_block_internal_nets_on_boundary_path` parameter is mutually exclusive with the `-top_scope` parameter.
`-write_flat_dir` *directoryName*	Writes flat data to the specified directory.

Example

- The following command loads the `TOPCHIP` DEF placement file:

  ```
  read_def TOPCHIP.def
  ```

- The following command loads the power grid from a DEF file:

  ```
  read_def -special_nets TOPCHIP.def
  ```

read_hier_db

`read_hier_db`
dir_name topcell_name

The `read_hier_db` command restores the hierarchical design information (hierarchical database) from the previous design session saved in the software. In order to restore a design in the hierarchical database format, you must have saved the design using the `save_hier_design` command in the software session.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`dir_name topcell_name`	
	Specifies the name of the design data directory and top cell.

Examples

- The following command reads the design data from the specified directory and top cell:

  ```
  read_hier_db saveDB Chip
  ```

read_libs

```
read_libs
```
[*library_files*]
[-aocv_libs *aocvLibs*]
```
[-max_aocv <string>]
```
[-max_libs *maxLibs*]
[-max_noise_libs *maxNoiseLibs*]
```
[-min_aocv <string>]
```
[-min_libs *minLibs*]
```
[-min_noise_libs minNoiseLibs]
[-noise_libs noiseLibs]
[-oa_ref_libs oaLibs]
[-power_ground_files powerGridLibFiles]
[-socv_libs soceLibs]
```

Schedules reading of one or more technology library files. These files either contain timing-related or noise-related information for the primitive cells used in the design.

Timing libraries can be in text format or Cadence binary library format (LDB). The LDB (library database) files generated from other tools, like Innovus, Voltus, or Genus are read in to Tempus with an error message (TECHLIB-1249). The LDB files are not forward compatible, that is, a LDB generated from 15.2 version cannot be used with 15.1 or prior versions of the software.

Note: You can use wildcards with the `read_libs` command as follows:

```
read_libs [glob *.lib]
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each read_libs parameter.
library_files	Specifies to read the library files.
-aocv_libs *aocvLibs*	Specifies to read the default AOCV libraries.
`-max_aocv` *`string`*	Specifies to read the max AOCV libraries.
-max_libs *maxLibs*	Specifies to read the max LIBs
-max_noise_libs *maxNoiseLibs*	Specifies to read the CDBs for max noise analysis
`-min_aocv` *`string`*	Specifies to read the minimum AOCV libraries.
-min_libs *minLibs*	Specifies to read the min LIBs
-min_noise_libs *minNoiseLibs*	Specifies to read the CDBs for min noise analysis
-noise_libs *noiseLibs*	Specifies to read the default CDB files for noise analysis
-oa_ref_libs *oaLibs*	Specifies to read the OA reference library
-power_ground_files *powerGridLibFiles*	Specifies to read the power grid library files
-socv_libs *soceLibs*	Specifies to read the default SOCV libraries

read_mmmc

```
read_mmmc
[-help]
file_name
```

Reads in the Multi-Mode, Multi-Corner (MMMC) file. MMMC refers to the full set of timing libraries, constraints, and the associated modes.

When the `read_mmmc` command is run, the MMMC file is read and processed. The majority of elements in the MMMC control file is saved in the MMMC caching attributes. Except for the library sets defined by the first `set_analysis_view` command, no data is loaded into the database. This command does basic syntax checking on the MMMC commands, rejecting commands that have incorrect options and commands that are not MMMC or tcl commands. However, no consistency checking is performed. For example, if a constraint mode is used in an analysis view, but is not defined, it will not be flagged.

The MMMC file contents are cached in attributes in the database. These attributes allow you to write out a full and complete MMMC file later in the flow. In addition, the names of the MMMC files are cached for querying.

The exceptions to data set load are the libraries pointed to by the first `set_analysis_view` command in the MMMC file. These library sets are loaded into the database. This load uses the standard library parser, and includes standard library syntax checking.

The `read_mmmc` command corresponds to the *Timing* step of the Initialization flow:



In the *Timing* step, the MMMC objects are set up by populating the basic MMMC information attributes. As Synthesis needs library information before the *Design* step, the *Timing* step also loads some library information. The library data to be loaded is pointed to by the first `set_analysis_view` command in the MMMC file.

See **Example** section for more details.

At a minimum, the MMMC file must contain one each of:

```
create_library_set
create_rc_corner
create_timing_condition
create_delay_corner
create_constraint_mode
create_analysis_view
set_analysis_view
```

If the MMMC file does not contain this minimum set, an error will be issued.

Note: The library sets can be specified with absolute paths in the MMMC, or relative to the local directory or to library search path attributes. The tool first attempts to load based on the name in the MMMC. If the library is not found, the tool sweeps through the search directory list in the specified order. Only the first hit is loaded.

Parameters

`-help`	Prints a brief description that includes type and default information for each `read_mmmc` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_mmmc`
`file_name`	Specifies the name of the MMMC file.

Example

When reading an MMMC file with `read_mmmc`, the tool sets the MMMC attributes with the data specified in the file. When the first `set_analysis_view` command is processed, the tool loads in the libraries specified by this analysis view. No other data is loaded.

For example, consider an MMMC file with the following commands:

```
create_library_set -name LS1 -timing [list lib1.lib lib2.lib lib3.lib]
create_library_set -name LS2 -timing [list lib4.lib lib5.lib lib6.lib]
create_library_set -name LS3 -timing [list lib7.lib lib8.lib lib9.lib]
create_rc_corner -name QX -qrc_tech file.qrc
create_timing_condition -name TC1 -library_set LS1
create_timing_condition -name TC2 -library_set LS2
```

```
create_timing_condition -name TC3 -library_set LS3
create_delay_corner -name DC1 -timing_condition {TC1 PD2@TC2} -rc_corner QX
create_delay_corner -name DC2 -timing_condition {TC1 PD2@TC3} -rc_corner QX
create_constraint_mode -name CM1 -sdc_files cond1.sdc
create_constraint_mode -name CM2 -sdc_files cond2.sdc
create_analysis_view -name AV1 -constraint_mode CM1 -delay_corner DC1
create_analysis_view -name AV2 -constraint_mode CM2 -delay_corner DC2
set_analysis_view -setup AV1 -hold AV2
```

When the `set_analysis_view` command is processed, the tool traces backwards from that command to determine the library set. In this case, the analysis view specified by `set_analysis_view` is AV1. The AV1 analysis view points to domain corner DC1, which in turn points to two library sets, LS1 and LS2. As a result, lib1.lib and lib2.lib are loaded into the database.

In this case, lib3.lib is not loaded into the database during `read_mmmc` because it is not in the library sets required by the analysis view.

Related Topics

- `init_design`
- `read_physical`
- `read_netlist`

read_netlist

```
read_netlist
[-help]
{{in_file | -oa_cell_view {lib cell view}} [-top top_module_name]}
```

Reads in a Verilog structural netlist. A structural Verilog file contains only structural Verilog-1995 constructs, such as module and gate instances, concurrent assignment statements, references to nets, bit-selects, part-selects, concatenations, and the unary ~ operator.

`read_netlist` also supports OpenAccess through the `-oa_cell_view` option.

The `read_netlist` command corresponds to the *Design* step of the Initialization flow:



During the *Design* step, the design is loaded into the database with `read_netlist`. After issuing this command, the database objects are populated and can be queried and manipulated (ungrouped, renamed, and so on). Full error checking is performed on the design details. This includes, but is not limited to, unresolved references, port mismatches, and empty modules.

Parameters

`-help`	Prints a brief description that includes the type and default information for each read_netlist parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_netlist`.
`in_file`	Specifies the list of netlist files.
`-oa_cell_view {`*`lib cell view`*`}`	Specifies the name of the design object to be loaded in {*`lib cell view`*} format. No other options can be used if this option is specified.
`-top` *`top_module_name`*	Specified the top module name. By default, `read_netlist` identifies the highest module and links to that. If there are multiple top modules, the command returns an error.

Related Topics

- `init_design`
- `read_mmmc`
- `read_physical`

read_physical

```
read_physical
[-help]
{[-oa_ref_libs lib1 [lib2] ... [-oa_search_libs lib1 [lib2] ...][-oa_tech_lib lib1 [lib2]
...]] | -lefs lib1 [lib2] ... | [-add_oa_ref_libs lib1 [lib2]... [-oa_lib_path libPath]| -
add_lefs lib1 [lib2] ...}
```

Reads the physical library information. When this command is run, the physical library information is loaded into the database. Both LEF and OA are supported, but not simultaneously.

The `read_physical` command corresponds to the *Physical* step of the Initialization flow:



During the *Physical* step, the physical libraries (OA or LEF) are loaded into the tool's database using `read_phsyical`. When the command is issued, the database is loaded with the specified objects, and is available for querying of attributes associated with the physical data, such as size, layer details, and so on.

The tool uses the standard OA parser, and performs full error checking on the physical data. When the command is complete, a full timing library set (via `read_mmmc`) and a physical library set would have been loaded into the database.

If `-lef` is used, the technology LEF must be first specified.

This command is optional for Synthesis and STA, but is required for Implementation.

The tool can support a physical only mode when the attribute `init_physical_only` is set to `true`. This attribute must be set prior to `read_physical`, otherwise an error will be issued.

Parameter

`-help`	Prints a brief description that includes type and default information for each `read_physical` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_physical`
`-add_lefs` *lib1* [*lib2*]	Specifies the list of new LEF files to read and add to the existing LEF files.
`-add_oa_ref_libs` *lib1* [*lib2*]	Specifies the list of OpenAccess library names to be loaded incrementally. These library names are added in the `cds.lib` file, if not present. Use this parameter to specify standard cell libraries or I/O driver libraries that should all be loaded after the design is read.
`-lefs` *lib1* [*lib2*] ...	Specifies the list of LEF files to be loaded. The first LEF file must have the technology section present. The file names are saved to the `init_lef_files` root attribute. This option cannot be combined with `-oa_ref_libs`.

`-oa_ref_libs` *lib1* [*lib2*] ...	Specifies the list of OpenAccess library names to be loaded. These names must also be listed in the `cds.lib` file. All the abstract views in the specified libraries are loaded. Use this parameter to specify standard cell libraries or I/O driver libraries that should all be loaded, even if some of the cells are not currently used in the netlist. The first OpenAccess library that is specified is also used to load the OpenAccess technology data. The specified library names are saved to the `init_oa_ref_libs` root attribute.
`-oa_lib_path` *libPath*	Specifies the path of oa reference libraries. This option can only be used with the `-add_oa_ref_libs` option.
`-oa_search_libs` *lib1* [*lib2*] ...	Specifies the list of OpenAccess libraries to be searched. This parameter can be specified only with `-oa_ref_libs`. As the netlist is read, these libraries are searched for cell abstracts to be load on demand for any Verilog module that does not have a lib_cell already defined. This parameter is typically used for libraries of block IP, such as RAMs, where only the abstracts that are actually used in the netlist need to be loaded. The specified library names are saved to the `init_oa_search_libs` root attribute.
`-oa_tech_lib` *lib1* [*lib2*] ...	Specifies the OA tech file to use if the netlist is coming from Verilog rather than an OA cellview.

Related Topics

- `read_mmmc`
- `init_oa_ref_libs` attribute
- `init_oa_search_libs` attribute
- `init_physical_only` attribute

read_physical Category Attributes

The root attribute in the `read_physical` category is:

- read_physical_check_colored_shape
- read_physical_check_colored_shape
- read_physical_extend_cell_obs_shapes_under_trim
- read_physical_extend_polygon_cell_shapes
- read_physical_merge_multi_port_to_single_port
- read_physical_must_join_all_ports

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `read_physical` category, use the following command:

```
get_db -category read_physical
```

read_physical_check_colored_shape

```
read_physical_check_colored_shape {true | false}
```

Default: `false`

Data_type: boolean, read/write

Specifies to check and ignore colored shapes on colorless layers while reading the LEF or DEF data. When set to `true`, the tool errors out if a colored shape is on a colorless layer. When set to `false`, the tool gives error message and ignores the color.

read_physical_check_colored_shape

```
read_physical_check_colored_shape {true | false}
```

Default: `false`

Data_type: boolean, read/write

Specifies to check and ignore colored shapes on colorless layers while reading the LEF or DEF data. When set to `true`, the tool errors out if a colored shape is on a colorless layer. When set to `false`, the tool gives error message and ignores the color.

read_physical_extend_cell_obs_shapes_under_trim

```
read_physical_extend_cell_obs_shapes_under_trim listOfRoutingLayers
```

Default: `""`

Data_type: string

Specifies a list of routing layers. On each specified *layer* the LEF OBS shapes that are overlapped partially by a shape on the TRIMMETAL layer (that trims the routing layer), will be extended in the preferred routing direction to be fully covered by the trim shape. You can use this attribute for some advanced node libraries to avoid trim/OBS short violations.

read_physical_extend_polygon_cell_shapes

```
read_physical_extend_polygon_cell_shapes { {layer extension_amount}… }
```

Default: `""`

Specifies a list of layer and a value for creating obs shape for 2D pin/obs shapes. Cells could have 2D objects, which require larger line-end spacing. You can use `read_physical_extend_polygon_cell_shapes` to automatically add SPACING 0 OBS for routing during LEF in.

For any 2D shapes, you can extend edges that are perpendicular to the preferred routing direction of the rectilinear shape by specifying the extension_amount in `read_physical_extend_polygon_cell_shapes`.

read_physical_merge_multi_port_to_single_port

```
read_physical_merge_multi_port_to_single_port {true | false}
```

Default: `false`

Data_type: bool

When set to `true`, multiple ports in a LEF PIN are merged into a single port. However, the pin should not have the `MUSTJOINALLPORTS` attribute as it specifies that all of the ports must be connected individually.

read_physical_must_join_all_ports

```
read_physical_must_join_all_ports {true | false}
```

Default: `true`

If set to `false`, the LEF_MUSTJOINALLPORTS property on a LEF PIN auto-creates MUSTJOIN pins for each PORT. If set to `true`, the MUSTJOIN pins are not created, but the router must connect to every port with a signal wire or signal via that physically overlaps all the ports.

read_sdc

```
read_sdc
[-reset]
fileName
```

Loads a timing constraints file. By default, the constraints files are loaded incrementally.

The constraints can be in the design constraints (SDC) format, or they can be more complicated Tcl constraints embedded in a Tcl script using commands such as `get_property` and `filter_collection`.

A list of constraints files also can be specified in the configuration file.

Parameter

fileName	Specifies the name of the constraint file.
-reset	Replaces all previously loaded constraints information with the information in the specified *fileName*.

Examples

- The following command reads a SDC file called dma_mac_m.sdc:
  ```
  voltus > read_sdc dma_mac_m.sdc
  ```

Command Order

Use this command after importing the design and loading the timing library.

read_twf

```
read_twf
[-help]
[filenames]
[-cell master_cell_name]
[-prefix string]
[-quiet]
[-reset]
[-scope string]
[-skip_constant]
[-skip_timing_window]
[-strip_prefix string]
[-verbose]
[-view string]
```

Reads the external Timing Window File (TWF) for noise and power calculation. This file is used to get clocks, constants, external load, slews, timing windows, and slack information. The external TWF specification is optional. In absence of the TWF, this information is automatically derived using Common Timing Engine if timing constraints are read.

Note: If the `VOLTAGE_THRESHOLD` construct is missing from the TWF file header, the values 10 and 90 are automatically assigned as the minimum and maximum threshold values, respectively. For example:

```
VOLTAGE_THRESHOLD (10 90)
```

Parameters

`-help`	Prints out the command usage.
`filenames`	Specifies the names of the timing window files.
`-cell master_cell_name`	

	Specifies the master cell name for the timing window file. This parameter allows you to support master cell based TWF annotation. That is, you need to specify only the master cell name for the timing window file, instead of specifying all the instances that were instantiated from a master cell.
	Example
	Let us consider a design that has a cell named "`cpu_core`" with two instances "`cpu_core_0`" and "`cpu_core_1`":
	Use Model for Instance-based TWF annotation
	`read_twf -scope cpu_core_0 cpu_core.twf.gz`
	`read_twf -scope cpu_core_1 cpu_core.twf.gz`
	Use Model for Master Cell based TWF annotation
	`read_twf -cell cpu_core cpu_core.twf.gz`

`-prefix` *`string`*	Adds a prefix to the net or pin name.
`-quiet`	Skips information and warning messages.
`-reset`	Clears the previously read TWF files.
`-scope` *`string`*	Specifies the scope for the given hierarchical timing window file.
`-skip_constant`	Skips constant processing.
`-skip_timing_window`	Skips timing windows processing.
`-strip_prefix` *`string`*	Removes hierarchical path name.
`-verbose`	Prints the command progress details.
`-view` *`string`*	Specifies the view for the timing window file.

Examples

- The following command reads the timing window file TW.in for the view 'view01', while skipping the constant processing from the timing window file:

  ```
  voltus> read_twf TW.in -skipconst -view view01
  ```

- The following command clears the previously read TWF file:

  ```
  voltus> read_twf -reset
  ```

relink_db_files

```
relink_db_files
[-help]
-db_dir db_directory
-new_lib_dirs {new_lib_directory1 new_lib_directory2 ...}
```

Relinks symbolic links to library files inside a DB directory.

By default, `write_db` saves the location of library files as symbolic links under `<db_dir>/libs/…`. When a DB directory is copied to a new network where all the libraries are in a different location, or a new version of the library files are needed, you can use `relink_db_files` to change the symbolic links to new locations .

The `relink_db_files` command will search each `-new_lib_dirs` directory and the sub-directory tree below it for each library file name inside `<db_dir>`, and create a symbolic link to the new location. Only the base file name is used for the search (except for `qrcTechFile`). Therefore, two path names inside the new directory, such as `/timing/fast/std.lib` and `/timing/slow/std.lib`, will have duplicate names of "`std.lib`". The duplicate names are not supported and will cause an error. You can use the `read_db` command to update all the LEF full-path names or use a new MMMC files with all the new .lib full-path names if duplicate file names need to be updated. If there are missing library file names, the tool will show an error.

The `relink_db_files` command supports `.lib`, `.lef`, and `qrcTechFile` library file types.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `relink_db_files` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man relink_db_files`
`-db_dir` *db_directory*	The current db directory which contains incorrect library symbolic links.
`-new_lib_dirs` *{new_lib_directory1 new_lib_directory2 ...}*	The new directories to search for library files.

Example

If the current LEF files are all under`/libs/lefs_v1.0/` (or sub-directories) and the current `.lib` and `qrcTechFiles` are all under`/libs/timing_v1.0/` (or sub-directories), you use the following command to update the LEF files to use `/libs/lefs_v2.0/`:

```
>relink_db_files -db_dir my_db -new_lib_dirs {/libs/lefs_v2.0 /libs/timing_v1.0}
```

Note: The command must find every library file. Therefore, you need to list the top of the old timing directory tree along with the top of the new LEF files directory tree.

Related Information

- `write_db`
- `read_db`

reset_design

```
reset_design [-help]
```

Removes libraries and design-specific data from the software session. It can be used as a shortcut in place of exiting and re-starting the software.

When you specify the `reset_design` command, the software does not free collections but only invalidates them. If you try to access the collections using TCL variables, you will not be able to see those collections and an appropriate error message will be displayed.

Parameters

None

Example

This example shows the location of `reset_design` in a command sequence:

```
read_db pre_route
reset_design
read_db post_route
```

set_cell_power_domain

```
set_cell_power_domain
[-help]
[-reset]
[-file file_name]
```

Defines domains for multi-VDD and multi-VSS cells. When reading an instance ASCII power file to perform power-rail analysis, if there are missing power values associated with ground pins, the software uses the information specified with the `set_cell_power_domain` command to derive domain association and distributes power to each ground net accordingly.

This domain information is stored in a side file that is used by power analysis to calculate power using the correct voltage for each domain. If you are specifying an external instance ASCII power file with pin information, this domain information is used to correctly distribute power to each pin.

Use this command before performing power and rail analysis.

Parameters

`-file` *file_name*	

	Specifies the name of the file containing the cell names and corresponding power-ground pairs.
	The format of the file is:
	`CELL: <cell name1>`
	`<power1> <gnd1>`
	`<power2> <gnd2>`
	`CELL: <cell name2>`
	`<power1> <gnd1>`
	`...`

`-help`	Outputs a brief description for the `set_cell_power_domain` parameter. For a detailed description of the command, use the `man` command: `man set_cell_power_domain`.

`-reset`	Resets all specified options back to default values.

Examples

- The following command define two domains, `VDD:VSS` at `1v` and `VDDm:VSSm` at `0.84v`, of the cell `mycell` in the file `domain_file`:

  ```
  set_cell_power_domain -file domain_file
  ```

  The following is a snippet from `domain_file`:

  ```
  CELL: mycell
  VDD_1v  VSS_1v
  VDDm_0.84v  VSSm_0.84v
  ```

set_power_def_files

```
set_power_def_files fileNameList
[-die_instance dieinstname]
```

Specify the name of the DEF file(s). This command can be used in lieu of `read_def` command for analysis purpose. When you use the `set_power_def_files` command, the software passes specified DEF files to the power and rail analysis programs for internal processing without loading their physical data in memory. As a result, you will not be able to display and query layout of the design. These programs will only process the necessary data from DEF file(s). The usage of this command is recommended in dynamic power analysis, and static and dynamic rail analysis for large (>5M components) designs with hierarchical DEFs or in general to obtain better performance during analysis. You will be able to save memory and runtime that takes to load design during analysis. Once the analysis is complete, you can subsequently load DEFs in the same or new session using the `read_def` command to view design layout and analysis results.

Note that for MSMV design, in order to use `set_power_def_files`, static power analysis mode will require CPF (Common Power Fomat) that defines logical connectivity of the instances to power nets. If CPF is not available, you can optionally run dynamic power analysis which calculates dynamic as well as static power.

`-die_instance` *dieinstname*	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode multi-die`).
fileNameList	List of DEF files.

set_power_lib_files

```
set_power_lib_files
[-help] [<in_file <file1[file2] ...>>] [-reset]
```

Specify the name of the timing library file(s) for dynamic power analysis. This command allows you to run the dynamic power analysis flow without reading a dotlib into design database to reduce background memory consumption. You must have an external TWF file to run this flow as the timing engine will not be invoked from within the software. This flow is recommended for large designs where front-end processing of timing libraries is high. Some of the database query commands will not work if `set_power_lib_files` is used.

This command works in conjunction with the `set_power_def_files` and `set_power_spef_files` commands.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_power_lib_files` parameter.
in_file <file1[file2] ...	
	List of timing library files.
`-reset`	Resets all specified options back to default values.

Examples

- The following Tcl shows the use model of the **set_power_lib_files** command that works in conjunction with the set_power_def_files/set_power_spef_files flow to perform dynamic power analysis:

```
read_libs -lef tech1.lef tech2.lef
set_power_lib_files /../library1.lib /../library2.lib
set_power_def_files adder_routed.def
set_power_spef_files {test.spef }
read_twf test.twf2
read_netlist ../design/postRouteOpt.enc.dat/super_filter.v.gz -top super_filter
init_design
set_default_switching_activity -combinational_clockgate_ratio 1 -period 2.01ns -
input_activity 0.2 -integrated_clock_gate_ratio 1
set_db power_default_supply_voltage 1.1 power_method dynamic_vectorless
report_power
```

set_power_spef_files

`set_power_spef_files` *`spef_file_list`*

Specify the name of the SPEF file(s) for dynamic power analysis. When you use the `set_power_spef_files` command, you can run the dynamic power and rail analysis flow without loading design data, thereby, improving the runtime and memory footprint for the whole flow. You must have an external TWF file to run this flow as the timing engine will not be invoked from within Voltus.

`-help`	Outputs a brief description that includes type and default information for each `specify_spef` parameter.
`spef_file_list`	List of SPEF files.

For more information, refer to *Running Analysis without Design Loading* in the *"Importing Design"* chapter of the *Voltus User Guide*.

Examples

- The following command specifies the SPEF file `adder_routed.spef` for dynamic power analysis:

  ```
  set_power_spef_files ./adder_routed.spef
  ```

specify_oa

```
specify_oa
[-help]
-input {{ top_library top_cell top_view} {block1_library block1_cell block1_view} ...
{blockn_library blockn_cell blockn_view}}
```

Specify the names of the top-level and block-level OpenAccess (OA) databases for use in power and rail analysis. When you use the `specify_oa` command, the software passes the specified OA database files to the power and rail analysis programs for internal processing without loading their physical data in memory. As a result, you will not be able to display and query the layout of the design. These programs will only process the necessary data from the OA files. The usage of this command is recommended in dynamic power analysis, and static and dynamic rail analysis for large (>5M components) designs to obtain better performance during analysis. You will be able to save the memory and runtime that takes to load the design during analysis.

The OA database files must be specified in the top-down order of the design hierarchy, that is, top-level OA, followed by block-level OA database files.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `specify_lib` parameter.
`-input` *{{top_librarytop_celltop_view} {block1_libraryblock1_cellblock1_view} ... {blockn_libraryblockn_cellblockn_view}}*	
	List of OA databases.

Examples

- The following Tcl shows the use model of the `specify_oa` command to perform dynamic power analysis:

```
specify_oa -input {{TopOaLib UL20 layout} {BlockOaLib super_filter1 layout}
{BlockOaLib super_filter2 layout} {BlockOaLib super_filter3 layout} {BlockOaLib
super_filter4 layout} {BlockOaLib super_filter5 layout} }
```

write_db

```
write_db
[-help]
[-no_timing_graph]
[-overwrite]
{{dir_name} | {-oa_lib_cell_view {libname cellname viewname} | -oa_view {view_name}}}
[[-sdc]
[-no_rc]
[-no_compress]
[-gzip]
[-lib]
[-lib_to_ldb]]
[-user_path]
```

Saves the files for design import, floorplan, placement, routing, and power switch state information in the specified directory. You can save a design to the same location from which you restored the design. The new data, including the netlist and constraints, overwrite the previous data.

You can use the `write_db` command at any time after importing a design.

> Modifying the contents of the *`filename.dat`* directory created by `write_db` can cause major unexpected results. Never modify the directory contents.

To save a design in OpenAccess database format, use `write_db -oa_lib_cell_view {libname cellname viewname}`.

`write_db` saves a tool design in the portable mode by default. In this mode, the design and library data as well as the control files do not have any path dependency. As a result, the saved database can be restored from any directory within the same network. The saved database can also be moved to any directory within the same network without any change.

The portable `write_db` feature is currently not supported for OpenAccess databases.

Note: The `write_db` command does not save the state of the `timing_enable_simultaneous_setup_hold_mode` timing attribute. When the attribute is set to `true`, various physical design functions in the system are disabled. When the design is restored (using the `read_db` command), the physical design commands are required to be active so that the system can be properly initialized.

To save a complete testcase including all the libraries please see the command `write_testcase`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_db` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_db`.
`-oa_lib_cell_view {`*`libname cellname viewname`*`}`	Saves the design in OpenAccess database format using the specified library, cell and view names. The design is saved in the *`libname/cellname/viewname`* directory. **Note**: Usually, the cell name is the same as the design top cell name. If the cell name specified is different from the top cell name, it means that the top module created in OpenAccess would have the new cell name instead of the original name.
`dir_name`	Represents the user-defined name of the current Voltus session. The recommended file extension is `.enc`. The associated data directory is *dir_name*`.enc.dat`.
`-lib_to_ldb`	Converts the `.lib` timing libraries used in the design being saved to the `.ldb` format for better performance with `restoreDesign`.
`-lib`	Resolves symlinks of timing library files and side files. When you specify `-lib` with write_db while saving a design in OpenAccess format, the links under `<`*`libname`*`>/libs` directory are replaced with real files. When you specify `-lib` with write_db while saving a design in Voltus format, the links under `<`*`filename.enc.dat`*`>/libs` directory are replaced with real files.
`-no_compress`	Does not compress output files when you save a design. Use only with small designs.

`-no_rc`	Does not write RC extraction data.
`-no_timing_graph`	Does not save timing graph information.
`-sdc`	Creates a timing constraint file even if the timing constraints file from the current Voltus session contains no changes.

`-gzip`	Saves design data as a tar ball. If you specify `-gzip`, write_db generates a single file that represents a portable Voltus design tar ball.
	• When saving the design in Voltus format as follows, specifying `-gzip` results in the `myDesign.enc.dat.tar.gz` tar ball:
	`write_db myDesign.enc -gzip`
	All libraries in the tar ball are real files, not links. The `myDesign.enc` file is updated to contain the following command:
	`restoreDesign [file dirname [info script]]/myDesign.enc.dat.tar.gz <top>`

`-oa_view view_name`	Specifies the name of the view. This view will be saved in `libname/cellname/` that you specified with `restoreDesign` or `init_design`. This option provides a shortcut to save a view quickly in the same `libName/cellName` structure that you restored. **Example**: `restoreDesign -write_db -oa_view {designLib top layout} write_db -oa_view {designLib top place} write_db -oa_view place` Both the above `write_db` commands are equivalent.
`-overwrite`	Overwrites the information.
`-user_path`	By default all external file references (such as. library files) are stored as absolute paths without any symbolic-links. So, the DB directory is portable anywhere on the same network. This option stores

external file references exactly as you entered it For example, full paths like /x/y or relative paths like ../x/y, ./x/y), and keeps any symlinks used inside the path names.

Example:

Consider that there are two library files a.lib and b.lib under:

```
/global/project1/libs/mmmc
```

And two lef files c.lef and d.lef under:

```
/global/project1/libs/lef
```

There are two symlinks to the libs:

```
/usa/db1/libs1.0/mmmc/a.lib
→ /global/project1/libs/mmmc/a.lib

/usa/libs/mmmc/b.lib           → /global/proj
ect1/libs/mmmc/b.lib

And two symlinks to the lefs:

/usa/db1/libs1.0/lef/c.lef
    → /global/project1/libs/lef/c.lef

/usa/libs/lef/d.lef            → /global/proj
ect1/libs/lef/d.lef
```

Your current work dir is /usa/db1.1/. You have an init script like:

```
read_mmmc ./mmmc.tcl

read_physical -lefs
{/usa/db1/libs1.0/lef/c.lef
../libs/lef/d.lef}

read_netlist design.v

init_design
```

Where mmmc.tcl has:

```
create_library_set -name test -timing
```

```
{/usa/db1/libs1.0/mmmc/a.lib
../libs/mmmc/b.lib }
```

Then after `write_db`, **without `-user_path`**, will save the libs and lefs to link to the real files.

```
> write_db db1.2
```

The ./mmmc.tcl file is always stored inside the DB dir, so no external reference is used.

By default, external file references are resolved to their fullpath name without any symlinks.

The .lib files inside the mmmc.tcl file will be referenced as: `{/global/project1/libs/mmmc/a.lib /global/project1/libs/mmmc/b.lib}`

The .lef files will be referenced as: `{/global/project1/libs/lef/c.lef /global/project1/libs/lef/d.lef}`

This is portable and can be loaded by read_db anywhere on the same network.

```
/usa/db1.2/libs1.2/mmmc/a.lib    →
/global/project1/libs/mmmc/a.lib

/usa/db1.2/libs1.2/mmmc/b.lib    →
/global/project1/libs/mmmc/b.lib

/usa/db1.2/libs1.2/lef/c.lef    →
/global/project1/libs/lef/c.lef

/usa/db1.2/libs1.2/lef/d.lef    → /global/pro
ject1/libs/lef/d.lef
```

With `-user_path` will save the libs and lefs to keep the symlinks of the current db.

```
> write_db db1.2 -user_path
```

The .lib files will be referenced as entered by the user: `{/usa/db1/libs1.0/mmmc/a.lib ../libs/mmmc/b.lib}`

The .lef files will referenced as entered by the

	user: `{/usa/db1/libs1.0/lef/c.lef ../libs/lef/d.lef}`
	
	So `read_db` must be run in the same directory where `write_db` was run, or in a directory with the same ./y and /x directory structure.
	
	`/usa/db1.2/libs1.2/mmmc/a.lib → /usa/db1/libs1.0/mmmc/a.lib`
	
	`/usa/db1.2/libs1.2/mmmc/b.lib → ../../../libs/mmmc/b.lib`
	
	`/usa/db1.2/libs1.2/lef/c.lef → /usa/db1/libs1.0/lef/c.lef`
	
	`/usa/db1.2/libs1.2/lef/d.lef → ../../../libs/lef/d.lef`

Example

- The following command copies the current design import configuration, floorplan, placement, and route files into directory `test.dat`:
  ```
  write_db test.dat
  ```

write_db Category Attributes

The root attributes in the `write_db` category are:

- write_db_create_read_file
- write_db_include_metal_fill_rules

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `write_db` category, use the following command:

```
get_db -category write_db
```

write_db_create_read_file

```
write_db_create_read_file {true | false}
```

Default: `false`

When set to `true`, `write_db my_db` will also create a '`my_db.read`' file with a '`read_db my_db`' command inside it. This is a Tcl file that can be passed to the Linux executable to make it easy to load the DB at startup like this: '`voltus -files my_db.read`'.

For OA usage, it would have the appropriate '`read_db -oa_lib_cell_view {<lib> <cell> <view>}`' command inside it.

When set to `false`, it only saves the design data ('`my_db`'). User can read design as follows:

```
read_db my_db (voltus data)
read_db -oa_lib_cell_view {<lib> <cell> <view>} (OA data)
```

write_db_include_metal_fill_rules

```
write_db_include_metal_fill_rules {true | false}
```

Default: `false`

Specifies whether metal fill settings are to be saved in the .init file when saving the design. By default, these metal fill settings are lost when you exit the software. To enable the software to record metal fill settings in the .init file, you must set the `write_db_include_metal_fill_rules` to `1` before saving the

design. You can easily reload your settings by restoring the saved design:

```
set write_db_include_metal_fill_rules 1

set_metal_fill -iterationName iter1 ...

write_db

read_db

#The previous metal fill settings are reloaded.
```

Related Commands

- write_db
- read_db

write_def

```
write_def
[-help]
def_file
[-add_half_wire_extension_on_pin]
[-all_layers]
[-bump_as_pin]
[-cut_row]
[-flatten_via_pillar]
[-floorplan]
[-io_row]
[-netlist]
[-no_core_cells]
[-no_special_net]
[-no_std_cells]
[-no_tracks]
[-output_mask_layers layerNames]
[-scan_chain]
[-selected]
[-skip_trimmetal_layers]
[-trial_route]
[-trimmetal_gap_fill]
[-unit unit_per_micron]
[-unplaced]
[-used_via]
[-verilog_from_def_netlist_flow]
[-with_shield]
[-routing  [-routing_to_specialnet  | -wrong_way_routing_to_specialnet ]]
```

Writes the specified information to a DEF file. By default, the `write_def` command writes floorplan, and all placed and unplaced standard cell information to the DEF file.

The `write_def` command does not support multiple groups per region. If you run Trial Route before you run the `write_def` command, the software does not write information about `FIXED` status wires to the DEF file. The `write_def` command supports rectilinear die area.

By default, the `write_def` command does not write out any `NONDEFAULTRULE` or `VIA` defined in the LEF file. To write out a LEF via, set the `write_def_include_lef_vias` attribute to `true`. To write out a nondefault rule, set the `write_def_include_lef_vias` attribute to `true`.

You can use the `write_def` command after importing a design.

Note: You can select the DEF version for your output file by setting the `write_def_lef_out_version`

attribute to the version number. For example, to set the version number to 5.6, type the following command:

```
set_db write_def_lef_out_version 5.6
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_def` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_def`.
`-add_half_wire_extension_on_pin`	Typically, signal routing in the `NETS` section uses the default half-width wire extensions everywhere except for zero extensions on pins. This parameter forces the zero-extension on pins to the default half-width extension, and makes the DEF compatible for older tools that do not support DEF zero wire extensions. **Note:** Using this parameter can cause DRC violations.
`-all_layers`	Writes out any wires and shapes on non-routing layers to SPECIALNETS section.
`-bump_as_pin`	Writes bump cells as top-level pins in the `PINS` section. If a PORT in a block has a PASSIVATION layer, the tool treats it as an embedded bump. `write_def -bump_as_pin` outputs all the layer information for embedded bumps in the PIN section as for normal bumps. Use the `-bump_as_pin` parameter to generate a DEF file for use with Quantus gate-level extraction. A DEF file generated with this parameter cannot be read back into the software.

`-cut_row`	Cuts rows against placement obstructions, block halos, or power domain fence minimum gaps. *Default:* Placement blockages are written to the DEF file.
`def_file`	Specifies the DEF output file.
`-flatten_via_pillar`	Some advanced nodes that use SADP (self-aligned double patterning) use via-pillars. A via-pillar is a single via definition with two or more disjoint metal-shapes on the bottom (or top) routing layer of the via. This option will flatten the via pillar into several separate "normal" vias (e.g. a via with only a single metal shape on the top and bottom via layers). The multiple normal vias will be geometrically equivalent to the single via pillar. This can be useful for external tools that do not support a via pillar. **Note:** DEF created with this option should not be read back into the software. It should only be used for external tools that do not support via-pillars properly.
`-floorplan`	Writes the floorplan data to the DEF file. This information includes rows, chip size, `CLASS PAD` instances, `CLASS BLOCK` instances, and all placed standard cells.
`-io_row`	Generates I/O rows based on I/O pad placement, and writes the I/O row information to the DEF file.
`-netlist`	Writes the netlist (that is, the routing connectivity information) to the DEF file. This option is typically used after the design has been placed. **Note:** Use `-netlist` with `-unplaced` if the design has not been placed.
`-no_core_cells`	Excludes core cell information from the DEF file to reduce the data size for import into APD.
`-no_special_net`	Excludes special vias from the DEF file.

`-no_std_cells`	Excludes placed and unplaced standard cell information from the DEF file. **Note:** If you specify this parameter with the `-floorplan` or `-unplaced` parameters, it overrides their setting.
`-no_tracks`	Excludes output track and gcell statements to the DEF file.
`-output_mask_layers` *layerNames*	Outputs MASK data on specified layers only.
`-routing`	Writes the routing information to the `NETS` section of the DEF file. If the `-routing` option is not specified, both power and signal vias are not written to the DEF file. **Note:** The `-routing` parameter implies the `-netlist` parameter; therefore, if you specify this parameter, you do not need to specify `-netlist`.
`-routing_to_specialnet`	Writes all regular NETS routing to the SPECIALNETS section.
`-scan_chain`	Writes scan chain information to the DEF file.

`-selected`	Writes out information for selected as follows: • If an instance or net is selected, writes only the DEF header and COMPONENT section for instances you have selected in the main window of the software. If you specify the `-routing` parameter along with this parameter, the `write_def` command writes selected nets with their special routing and regular routing to the NETS section of the DEF file. • If a net is not selected but individual segments or vias are selected, writes only the selected vias to the DEF file. All LEF vias are excluded by default. If the `write_def_include_lef_vias` attribute is set to `true`, `write_def -selected` includes selected LEF vias when writing out selected vias. If the `-no_special_net` option is specified, special vias on the selected net will not be written out. • If a bump is selected, writes bump pin connection to the `SPECIALNETS` section of the DEF file. If a net is selected but bump pins are not individually selected, bump pin connection is not written. • If a blockage is selected in the main window, writes blockage information to the DEF file.
`-skip_trimmetal_layers`	Specifies to not write trimmetal shapes in the FILLS section.
`-trial_route`	Writes information about the wires generated by Trial Route to the `NETS` section of the DEF file. This option implies the `-routing` parameter; therefore, if you specify this parameter, you do not need to specify `-routing`.

`-trimmetal_gap_fill`	Writes trimmetal gap fills to a dummy net _TRIMMETAL_FILLS_RESERVERD in SPECIALNETS section.
`-unit` *`unit_per_micron`*	Defines the units for the DEF file.
`-unplaced`	Writes information about unplaced standard cells to the DEF file.

`-used_via`	Specifies that only used vias should be written to the DEF file. Unused vias are ignored. All LEF vias are also excluded by default. To include used LEF vias in the DEF file, set the `write_def_include_lef_vias` attribute is set to `true` before specifying `write_def -used_via`. • If the `-routing` option is specified along with `-used_via`, used signal vias are also written out. • If the `-no_special_net` option is specified along with `-used_via`, used special vias are not written out. • If you specify both `-used_via` and `-selected`, the `-used_via` option is ignored and only selected vias are written to the DEF file. The following warning message is issued: `**WARN: (ENCDF-1039): Option -used_via will be ignored because option -selected is specified. Only selected vias will be output.` • If neither `-selected` nor `-used_via` is specified and the `write_def_include_lef_vias` attribute is set to `false` (default), all DEF vias are written out. • If neither `-selected` nor `-used_via` is specified and the `write_def_include_lef_vias` attribute is set to `true`, all DEF and LEF vias are written out.
`-verilog_from_def_netlist_flow`	

	Reconciles the names in the DEF and Verilog files generated by the `loadDefFile` command. The `loadDefFile` command generates a flat Verilog file in which all of the special characters are escaped. Use this parameter to convert the escaped names back to the original names in the DEF file.
`-with_shield`	By default, `write_def` does not write out the shielding routing on the power/ground net when outputting regular routing of selected nets in NETS section. When `-with_shield` is specified, `write_def` writes out those shielding routing (if any) on the power/ground net which is in the SPECIALNETS section. The`-with_shield` option must be used with the `-routing` and `-selected` options. **Note:** Multiple nets can be selected. If none of the selected nets have shielding routing, the `-with_shield` option has no effect.
`-wrong_way_routing_to_specialnet`	Writes wrong way routing in the SPECIALNETS section.

Example

- The following command writes floorplanning data, and information on placed and unplaced standard cells to `TOPCHIP_SP.def`:

  ```
  write_def -floorplan -unplaced TOPCHIP_SP.def
  ```

- The following command also writes floorplanning data, and information on placed and unplaced standard cells:

  ```
  write_def test.def
  ```

- The following command writes floorplan and standard cell information to the DEF file. It does not write out unplaced standard cell information.

  ```
  write_def -floorplan
  ```

- The following command excludes all information on placed and unplaced standard cells from the DEF file:

  ```
  write_def -floorplan -no_std_cells
  ```

The resulting DEF file contains floorplan data only.

- The following command writes the shield wires that are shielding the selected nets to the DEF file:

```
write_def -selected -routing -with_shield test.def
```

- The following command writes bump cells as top-level pins in the PINS section of the `test.def` file:

```
write_def -bump_as_pin test.def
```

The PINS section in the test.def file is as follows:

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
PINS 102 ;
- iot_schm_2m16m_3v + NET iot_schm_2m16m_3v + DIRECTION OUTPUT + USE SIGNAL
+ SPECIAL + POLYGON ap ( 93300 65970 ) ( 65970 65970 )
( 65970 93300 ) ( 27330 93300 ) ( 27330 65970 ) ( 0 65970 )
( 0 27330 ) ( 27330 27330 ) ( 27330 0 ) ( 65970 0 )
( 65970 27330 ) ( 93300 27330 ) + FIXED ( 386690 2658905 ) N ;
- tck + NET tck + DIRECTION INPUT + USE SIGNAL
+ SPECIAL + POLYGON ap ( 93300 65970 ) ( 65970 65970 )
( 65970 93300 ) ( 27330 93300 ) ( 27330 65970 ) ( 0 65970 )
( 0 27330 ) ( 27330 27330 ) ( 27330 0 ) ( 65970 0 )
( 65970 27330 ) ( 93300 27330 ) + FIXED ( 53350 2658905 ) N ;
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
```

Related Commands

- write_def Category Attributes

write_def Category Attributes

The root attributes in the write_def category are:

- write_def_lef_out_version

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `write_def` category, use the following command:

```
get_db -category write_def
```

write_def_lef_out_version

`write_def_lef_out_version` *version_num*

Default: `5.8`

Specifies LEF/DEF output version. Possible string values are `5.6, 5.7, and 5.8`. The default is `5.8`.

write_hier_db

```
write_hier_db
[-help]
directory_name
```

Saves a snapshot of the physical design data that includes data from reading the DEF files, and the placement, routing, and floorplan information. In addition, the `write_hier_db` command saves the design in the portable mode by default. In this mode, the design data does not have any path dependency. As a result, the saved database can be restored from any directory within the same network. The saved database can also be moved to any directory within the same network without any change.

To use this command, you must launch Voltus with the extra Innovus license, as shown below:

```
voltus -lic_startup_extra invs
```

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`directory_name`	Specifies the name of the directory that stores the design data.

Examples

- The following command saves a snapshot of the hierarchical design data to a directory called `saveDB`:

  ```
  write_hier_db saveDB
  ```

Related Topics

- `read_hier_db`

write_mmmc

```
write_mmmc
[-help]
[-library]
```

Writes out the MMMC file.

Parameters

`-help`	Prints out the command usage.
`-library`	Writes out only the library part of the MMMC file.

Related Topics

- `read_mmmc`

write_netlist Category Attributes

The root attributes in the write_netlist category are:

- write_netlist_full_pin_out
- write_netlist_port_association_style

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `write_netlist` category, use the following command:

```
get_db -category write_netlist
```

write_netlist_full_pin_out

```
write_netlist_full_pin_out {true | false}
```

Default: `false`

Specifies whether to write out every instance port connection, including unconnected ports unused in the input netlist.

write_netlist_port_association_style

```
write_netlist_port_association_style {true | false}
```

Default: `false`

Writes out the netlist with implicit port mapping (that is, ordered port connections or position-based port connections) for hierarchical instances.

write_via_defs

```
write_via_defs
[-help]
[-generate]
[out_file filename]
[-all_vias | -signal | -all_rules | -default | -ndr ]
```

Helps in debugging and testing. All the boolean options are default to false. If none is set, the vias are dumped out in LEF format.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_via_defs` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_via_defs`.
`-all_rules`	Dumps out vias from all routing rules. this option is of type boolean.
`-all_vias`	Dumps out all vias. This option is of type boolean.
`-generate`	Dumps out auto-generated vias. This option is of type boolean.
`-default`	Dumps out vias of default routing rule. This option is of type boolean.
`out_file` *filename*	Specifies the output file name. This option is of type string. *Default*: `dump_via.out`
`-ndr`	Dumps out vias of nondefault routing rules. This option is of type boolean.
`-signal`	Dumps out signal vias. This option is of type boolean.

Example

The following command dumps out all signal vias to the default output file, `dump_via.out`:

```
write_via_defs -signal
```

Here's the sample output:

```
#### Begin signal vias ####
VIA V01  # LEF
  LAYER POLY ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C01L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V01
VIA V01N  # LEF
  LAYER NDIFF ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C01L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V01N
VIA V01P  # LEF
  LAYER PDIFF ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C01L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V01P
VIA V1L2L  # LEF
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M2L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C1L2L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V1L2L
VIA V2L3L  # LEF
  LAYER M2L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M3L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C2L3L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V2L3L
VIA V3L4L  # LEF
  LAYER M3L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M4L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C3L4L ;
    RECT -0.033 -0.033 0.033 0.033 ;
.......................
.......................
.......................
END SP4V4L5LHVR1S5
VIA SP4V4L5LHVR3S5  # LEF
  LAYER M4L ; RECT -0.066 -0.033 0.264 0.033 ;
  LAYER M5L ; RECT -0.033 -0.066 0.033 0.066 ;
  LAYER C4L5L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END SP4V4L5LHVR3S5
#### End signal vias ####
```

write_twf

```
write_twf

-help
out_file
[-exclude_instances string]
[-include_instances string]
[-include_lib_cells string]
[-pin]
[-power_compatible]
[-ssta_sigma_multiplier <float>]
[-view <string>]
[-voltage_threshold <string>]
```

Generates a Timing Window File (TWF). The TWF file mainly contains the earliest and the latest possible arrival times that a signal may arrive on a net or a pin.

Parameters

`-help`	Prints out the command usage.
`-exclude_instances` *string*	Specifies a list of hierarchical instances to be excluded from the TWF file.
`-include_instances` *string*	Specifies a list of hierarchical instances to be included in the TWF file.
`-include_lib_cells` *string*	Specifies the library cells that require the input timing window.
out_file	Writes the TWF file to the specified file name. The *outfile* parameter must be the last argument in the list. **Note:** To write a compressed file, add the `.gz` extension to the file name.
`-pin`	Generates a TWF for each pin instead of each net. *Default*: Generates a TWF for the crosstalk analysis tool, which contains timing windows on each net.

`-power_compatible`	Generates a timing windows file that is suitable for power analysis.
`-ssta_sigma_multiplier` *value*	Specifies a sigma multiplier between `0` and `3`. The value that you specify increases the arrival window by that amount and worst slack value is generated. The transition and impedance values are not impacted.
`-view` <*string*>	Specifies name of the analysis view.
`-voltage_threshold {`*lower_thresholdupper_threshold*`}`	Specifies voltage threshold values for generating a Timing Window File (TWF). All the slews in the TWF file will be scaled to these values. The lower threshold value must be less than the higher threshold value. *Value range:* `0` to `100`

Examples

- The `write_twf -voltage_threshold {20 80}` command writes the following in the TWF file:
  `(VOLTAGE_THRESHOLD 20 80)`

9

Interactive ECO Commands and Attributes

- connect
- connect_hpin
- connect_pin
- create_hinst
- create_hnet
- create_hport
- create_inst
- create_module
- create_port
- delete_inst
- delete_notch_fill
- disconnect
- disconnect_hpin
- disconnect_pin
- gui_show_buffer_tree
- update_inst

connect

```
connect
[-help]
port|pin|hpin|hport|pg_pin+
[-net net_object | -net_name net_name | -constant -hnet hnet_object ]
```

Connects objects together. If these object exist in different levels of hierarchies, hpins/hport will be created to make these connections and the 'prefix' options provide a mechanism for controlling how

they are named.

If more than one object has a driver (i.e. has a net connected), it creates a multi driver situation. If only one object has a net, all other objects will be connected to that net. If no object has a net, a net will be created and they all will be connected to that net and the `-net_name` option provides a mechanism to optionally name the new net. Otherwise, a new net will be created using n_<i> convention. If the `-net` option is used (along with a net object), then all objects will be disconnected from their existing connection and connected to the new net. For local connections, it returns then hnet object pointer used to make the connection. For hierarchical connection, it returns the parent net object pointer.

⚠ The `connect` command is a limited-access feature in this release. It is enabled by a variable specified using the `set_limited_access_feature` command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `connect` parameter. For a detailed description of the command and all of its parameters, use the man command: `man connect`
`port|pin|hpin|hport|pg_pin+`	List of objects to connect. Multiple pin/port objects are supported. Note that only object pointers like hport:h1/hport1 are allowed, and simple names like h1/hport1 are not allowed. All objects in the list will be connected together so if objects have existing drivers, then a multidriver situation can be created.

`-constant`	Connect the object to a constant (0 or 1). This option will only work for hpin/pin/hport/port objects. When specified, any existing net connections to the *objects* specified will be disconnected prior to connecting to the constant. It does not connect the objects together, but instead adds a constant to each separate object at the level of hierarchy where the object exists. If an hpin or a pin is connected to an hnet (e.g. is not dangling), the constant is added to the hnet, otherwise it is added directly to the hpin/pin. If a constant is added to a port/hport, it is always added to the hnet of the port/hport. If there are existing drivers or constants attached to any net of the hnets mentioned above, an error message is given, but the connection is done anyway.
`-hnet` *`hnet_object`*	Specifies the hnet object to be used for the connections. It is mutually exclusive with `-net` (and `-constant`, `-net`). Additionally, the `-hnet` option can only be used to make a local connection (i.e. at the same level of hierarchy)
`-net` *`net_object`*	Net object to be used for the connections. The object will be connected to the hnet of the specified net at the same level of hierarchy as the object. If there is no hnet at the same level of hierarchy, hports/hpins will be created using the base_name of the net object. Also, when `-net` is specified, any existing net connections to the *objects* specified will be disconnected prior to connecting to this net (for example, equivalent to running disconnect on each object first).

`-net_name` *net_name*	Net name to create when only pins specified. Net name to create when only pins specified (i.e. when -net is not specified). This is the base_name of the net and will replace the name of all nets connected to the pins in the object list. If a hierachical connection is being made the base_name will also be used to name any hports/hpins that are created. If *net_name* has any illegal chars, it will be automatically escaped. Thus -name "i1/n2" will result in "i1\/n2" for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if *net_name* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name.

Examples

The following command connects two pins with the given net name:

```
connect {pin:H1/I1/Y pin:H1/I3/A} -net_name H1/N1
```

Related Topics

- disconnect

connect_hpin

```
connect_hpin
[-help]
-hinst moduleName
-net netName
[-no_new_hpins]
-pin_name hpinName
```

Attaches an hpin in the specified hinst (or top level design) to a net.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `connect_hpin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man connect_hpin`
`-hinst` *hinstName*	Specifies the hinst to that has the hpin. To specify the top design, enter `` `- ``.
`-net` *netName*	Specifies the name of the net.
`-no_new_hpins`	Prevents the creation of new hpins. If the hpin cannot connect to the net in a different hierarchy through existing hpins, the software displays an error message and the command stops running. Default: If you do not specify this parameter, the software creates new hpins as required to reach the net.
`-pin_name` *pinName*	Specifies the base_name of the hpin on the hinst.

Examples

- The `connect_hpin` command connects the hpin with base_name `p1` on hinst named h1/h2 to the net `h1/h2/net3`:

  ```
  connect_hpin -hinst h1/h2 -pin_name p1 -net h1/h2/net3
  ```

Related Topics

- disconnect_hpin

connect_pin

```
connect_pin
[-help]
[-hport portName]
-inst instName
-net netName
[-no_new_port]
-pin termName
[-reference_pin refInstName refTermName]
```

Attaches a terminal to a net. If the terminal already connects to a different net, the software first detaches the terminal from the current net, then attaches it to the new net.

Note: Detaching a terminal that drives an output terminal of a module produces a Verilog violation at the output terminal if you use `detachTerm`. Instead, use `connect_pin` to attach the terminal to a new net. The `connect_pin` command automatically detaches the terminal from the net connecting to the output terminal, then attaches the terminal to the net you specify. This command also enables you to specify a hierarchical net name, in addition to a flat net name.

Parameters

`-inst` *instName*	Specifies the instance containing the terminal. If the instance name does not exist, the software displays an error message and the command stops.
`-net` *netName*	Specifies the name of the net to attach to the terminal. If the net name does not exist, the software displays an error message and the command stops.
`-no_new_port`	Prohibits the software from creating hierarchical ports when it attaches a terminal. If the terminal cannot connect to the net through existing ports, the software displays an error message and the command stops. *Default:* If you do not specify this parameter, the software creates hierarchical ports as needed.
`-pin` *termName*	Specifies the name of the term.
`-reference_pin` *refInstName refPinName*	Specifies the pin *refPinName* on instance *refInstName* connected to the net that the software connects to the terminal. You cannot specify the `-port` parameter if you use this parameter.
`-hport` *portName*	Specifies the hierarchical port used to connect the terminal with the net. If you specify this parameter, the hierarchical port must exist in the module that contains the instance. The hierarchical port must connect to the net. This parameter lets you use a specific port to maintain the same netlist topology, which simplifies equivalence checking later in the design flow. You cannot specify the `-reference_pin` parameter if you use this parameter. *Default:* If you do not specify this parameter, the software uses existing ports or creates new hierarchical ports as necessary to connect the terminal to the net.
termName	Specifies the name of the terminal that the software connects to the specified net. If the terminal name does not exist, the software displays an error message and the command stops.

Examples

- The following command attaches terminal `in1` of instance `i1/i2/i3` to net `i1/i2/net26`:

  ```
  connect_pin i1/i2/i3 in1 i1/i2/net26
  ```

  If `i1/i2/i3` does not exist, or if `in1` is not a terminal of `i1/i2/i3`, or if `i1/i2/net26` does not exist, the software displays an error message and the command stops.

- The following command attaches terminal `in1` of instance `i1/i2/i3` to net `net27`, using hierarchical port `myPort`:

  ```
  connect_pin i1/i2/i3 in2 net27 -hport myPort
  ```

  The `myPort` port must exist in the module definition for `i1/i2`, and `myPort` must already connect to `net27`. If this is not the case, the software displays an error message and the command stops.
- The following command attaches terminal `in3` on instance `i1/i2/i3` through existing ports to `net28`:

  ```
  connect_pin -no_new_port i1/i2/i3 in3 net28
  ```

  If ports do not exist, the software displays an error message and the command stops.

Related Topics

- `disconnect_pin`

create_hinst

```
create_hinst
[-help]
[-parent module|hinst|design]
{-name hinst_name }
{-module module|design}
```

Adds a hierarchical module to the design.

You can use this command after importing a design.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_hinst` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_hinst`
`-module` *`module|design`*	Specifies the module to use for the new hinst. The new hinst will have hports/hpins created based on the module ports as well as hnets of the same name.
`-name` *`hinst_name`*	Specifies the name of the new hinst. If this is the full hierarchical path, then the parent will be derived from the name (i.e. the parent name is the string before the last "/". In this case, the base_name of the new hinst (i.e. the string after the last "/" should not have any special characters that require escaping. When the `-parent` option is used, then the name is the base_name of the new hinst. if the name has any illegal chars, it will be automatically escaped. Thus -name "i1/i2" will result in "i1\/i2" for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if <hinst_name> is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name.
`-parent` *`module|hinst|design`*	Specifies the parent for the new hinst. It supports both the object or the name for the specified object types. Default is the current design.

Examples

The following example creates the module `module:carve/mod1` and instantiates it:

```
@voltus 1> create_module -name mod1
module:carve/mod1
@voltus 2> create_hinst -name H1 -module mod1
hinst:carve/H1
```

create_hnet

```
create_hnet
[-help]
[-bus <msb:lsb>]
{-name <hnet_name> }
[-parent <module|hinst|design>]
```

Creates a new hnet (and parent net) object and returns the newly created hnet object pointer.

Parameters

`-bus msb:lsb`	Specifies the bit range for bus nets. By default, the tool creates a scalar hnet.
`-ground`	Creates a ground net.
`-help`	Outputs a brief description that includes the type and default information for each `create_hnet` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_hnet`
`-name` *`hnet_name`*	Specifies the name of the new hnet. If this is the full hierarchical path, then the parent will be derived from the name (i.e. the parent name is the string before the last "/". In this case, the base_name of the new hnet (i.e. the string after the last "/" should not have any special characters that require escaping. When the `-parent` option is used, the name is the base_name of the new hnet. If the name has any illegal chars, it will be automatically escaped. Therefore, `-name "i1/n2"` will result in "i1\n2" for the .escaped_name, as it uses DEF escaping semantics. This is equivalent to checking if *`hnet_name`* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name.
`-parent` *`module|hinst|design`*	Specifies the parent for the new hinst. It supports both the object or the name for the specified object types, *`module`*, *`hinst`*, *`design`*. Default is the current design.
`-power`	Creates a power net.

Example

```
    CPU
     |
     +-----------------+
     |                 |
     H1 (alu)          H2 (fsm)
```

```
 |
 +-----------------+
 |                 |
+ H1 (add)         H2 (mult)
```

- The following command creates a scalar top level net/hnet named N1

  ```
  create_hnet -name N1
  ```

- The following command creates a 8-bit net/hnet named H1/H1/B1

  ```
  create_hnet -name B1 -bus 7:0 -parent module:add
  ```

- The following command creates an 8-bit net/hnet named H1/H1/B1

  ```
  create_hnet -name B1 -bus 7:0 -parent H1/H1
  ```

create_hport

```
create_hport
[-help]
[-bus msb:lsb]
-direction {in out inout}
-name hport_name
[-parent module|hinst|design]
```

Create new hports and corresponding hpins for the parent object. It also implicitly creates hnet and net objects inside the parent object with the same name as the new hport. If the new hport is a bus, then the bus_bits objects are also created. The hnet object has no corresponding hnet or net object but is left dangling. It returns the newly created hport object pointer.

Parameters

`-bus msb:lsb`	Specifies the bit range for bus hports. By default, the tool creates a scalar hport.
`-direction {in out inout}`	Specifies the direction of the new port, in, out, inout.
`-help`	Outputs a brief description that includes the type and default information for each `create_hport` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_hport`
`-name hport_name`	Specifies the name of the new hport. If this is the full hierarchical path, then the parent will be derived from the name (i.e. the parent name is the string before the last "/". In this case, the base_name of the new hport (i.e. the string after the last "/") should not have any special characters that require escaping. When the `-parent` option is used, then the name is the base_name of the new hport. If the name has any illegal chars, it will be automatically escaped. Therefore, `-name "hp1/2"` will result in `"hp1\/2"` for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if *hport_name* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name.
`-parent module|hinst|design`	Specifies the parent for the new hport.

Example

```
CPU
 |
 +-----------------+
 |                 |
 H1 (alu)          H2 (fsm)
 |
 +-----------------+
 |                 |
+ H1 (add)         H2 (mult)
```

- The following command creates new scalar objects `hpin:CPU/H1/H1/P1`, `hport:CPU/H1/H1/P1`, and `hnet/net H1/H1/P1`.

  ```
  create_hport -name P1 -parent add -direction in
  ```

- The following command creates a new 8-bit objects `hpin:H1/H2/BP1`, `hport:H1/H2/BP1`, and `hnet/net H1/H2/BP1`.

  ```
  create_hport -name BP1 -parent H1/H2 -bus 7:0
  ```

- The following command creates new scalar objects `hpin:CPU/H1/H2/P2`, `hport:CPU/H1/H2/P2`, and `hnet/net H1/H1/P2`.

  ```
  create_hport -name H1/H2/P2 -direction out
  ```

create_inst

```
create_inst
[-help]
[-dont_snap_to_placement_grid]
[-location {x y}]
[-orient {r0 r90 r180 r270 mx mx90 my my90}]
[-physical]
{-base_cell base_cell} {-name inst_name}
[-parent module|hinst|design]
[-place_status place_status]
```

Creates a new instance object, optionally places it in the design, and returns the newly created object pointer.

For MSV designs, a cell specified using the `-base_cell` parameter should exist in libraries bound to the power domain of the newly added inst. When `-location` is specified, the location is checked for a power domain, which is checked against the power domain of the parent. If the power domains are different, a warning is issued.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_inst` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_inst`
`-physical`	Places a physical instance without updating the netlist.
`-base_cell` *base_cell*	Specifies the base_cell to use for the new instance.
`-dont_snap_to_placement_grid`	If `-dont_snap_to_placement_grid` is specified, the tool places the instance at the specified location even if the location is off the placement grid. This parameter is used with `-location`. *Default*: `false`

`-name` *`inst_name`*	Specifies the name of the new inst. If this is the full hierarchical path, then the parent will be derived from the name (i.e. the parent name is the string before the last "/". In this case, the base_name of the new inst (i.e. the string after the last "/") should not have any special characters that require escaping. When the `-parent` option is used, then the name is the base name of the new inst. If the name has any illegal characters, it will be automatically escaped. Therefore, `-name "i1/i2"` will result in `"i1\/i2"` for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if *`inst_name`* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name.
`-location {`*`x y`*`}`	Specifies the location of the placed instance. *Default:* If you do not specify a location, Voltus places the instance at the design origin.
`-orient {r0 | r90 | r180 | r270 | mx | mx90 | my | my90}`	Specifies orientation of the placed instance. *Default:* If you do not specify an orientation, Voltus uses `r0`.
`-parent` *`module|hinst|design`*	Specifies the parent for the new instance. It supports both the object or the name for the specified object types. Only honored for logical instances and is ignored when used with `-physical`. Default is the current design.

`-place_status` *place_status*	Specifies the placement status of the added instance. You can specify the following as the placement status: • `fixed`: The status of the added instance is fixed. It has a location and cannot be moved by automatic tools, but can be moved using interactive commands. • `placed`: The status of the added instance is placed. It has a location, and can be moved using automatic tools and interactive commands. • soft_fixed: The status of the added instance is soft_fixed. It cannot be moved by global placement and can only be moved by the legalization step of detail placement • `unplaced`: The status of the added instance is unplaced. It does not have a location. Default: `unplaced` **Note:** This option is used along with the `-location` option. It should not be used along with the `-parent` option.

create_module

```
create_module
[-help]
-name module_name
```

Create new hports and corresponding hpins for the parent object. It also implicitly creates hnet and net objects inside the parent object with the same name as the new hport. If the new hport is a bus, then the bus_bits objects are also created. The hnet object has no corresponding hnet or net object but is left dangling. It returns the newly created hport object pointer.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_module` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_module`
`-name` *`module_name`*	Specifies the name of the new module. If the name has any illegal chars, it will be automatically escaped. Therefore, `-name "m1/m2"` will result in `"m1\/m2"` for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if *`hinst_name`* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name.

Example

- The following command creates `module:<design>/mod1`.

  ```
  create_module -name mod1
  ```

- The following command creates `hinst:<design>/H1`.

  ```
  create_hinst -name H1 -module mod1
  ```

- The following command creates `hpin:<design>/H1/in`.

  ```
  create_hport -name in -module mod1 -input
  ```

- The following command creates `hpin:<design>/H1/out`.

  ```
  create_hport -name out -module mod1 -output
  ```

- The following command creates `inst:<design>/H1/I1`.

  ```
  create_inst -name I1 -parent H1 -base_cell BUF1
  ```

create_port

```
create_port
[-help]
[-bus msb:lsb]
-direction {in out inout}
-name port_name
```

Create new ports for a design object and returns the newly created port object pointer. It also implicitly creates hnet and net objects with the same name connected to the port.

Parameters

`-bus msb:lsb`	Specifies the bit range for bus ports. Default is to create a scalar port.
`-direction {in out inout}`	Specifies the direction of the new port, in, out, inout.
`-help`	Outputs a brief description that includes the type and default information for each `create_module` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_module`
`-name module_name`	Specifies the name of the new port. If the name has any illegal chars, it will be automatically escaped. Therefore, `-name "p1/2"` will result in `"p1\/2"` for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if *port_name* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name.

Example

- The following command creates a new input scalar port called P1:

  ```
  create_port -name P1 -direction in
  ```

- The following command creates a new 8-bit output port called BP1

  ```
  create_port -name BP1 -bus 7:0
  ```

delete_inst

```
delete_inst
-inst
```

Deletes an instance from a design. The software checks whether the instance is physical or logical, and deletes the instance accordingly.

You can use wildcards (`*?`) to specify the instances you want the tool to delete.

Parameters

-inst	Specifies the name of the instance to delete.

Example

- The following command deletes instance `i1/i2`:

  ```
  delete_inst i1/i2
  ```

- The following command deletes all instances whose names begin with `i`:

  ```
  delete_inst i1/i*
  ```

delete_notch_fill

`delete_notch_fill`

Deletes all gap, notch, and hole geometries.

- A gap is a minimum spacing violation with two edges without a common adjacent side.
- A notch is a minimum spacing violation with two edges with a common adjacent side.
- A hole is a minimum enclosure violation.

Parameters

None

disconnect

```
disconnect
[-help]
pin|hpin|port|hport|pg_pin+
[-keep_dangling_net]
```

Disconnects pins, ports, hports, or hpins from any hnet or net. For example, if A, B, and C are connected to net N , and you disconnect A, then B and C remain connected to N, but A is now left unconnected. For hpin/hport objects, they are disconnected only from the local hnet they are connected to. So, for an hpin, it will be disconnected from the hnet above the hpin (outside the hinst of the hpin). For hports, it will be disconnected from the hnet below the hport (the hnet inside the hinst of the hport) but the associated hpin/hport to hnet connection is not affected. You need to disconnect them both if you want both connections removed. Also, if an hpin/pin is directly connected to a constant (e.g. has .constant 0/1 and no hnet), the .constant attribute on the hpin/pin will be cleared, and return “no_constant”.

By default, dangling hnets that have no port, hport, hpin, or pin attached to it are deleted. Any constants like 1’b0/1’b1 are ignored, so a constant hnet is considered dangling if there are no port, hport, hpin or pin objects attached to it.

⚠ The `disconnect` command is a limited-access feature in this release. It is enabled by a variable specified using the `set_limited_access_feature` command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `connect` parameter. For a detailed description of the command and all of its parameters, use the man command: `man connect`
`-keep_dangling_net`	Keeps the net object even if it has no loads or drivers. Default is to delete dangling nets.
pin|*hpin*|*port*|*hport*|*pg_pin*+	List of objects to disconnect (port/pin/hpin/hport/pg_pin). Only objects are supported, no simple names.

Example

Consider the following example:

```
@voltus1>get_db hport:H1/B1 .hnet

hnet:H1/B1

@voltus2>get_db hpin:H1/B1.hnet

hnet:P1
```

If the following command is run, the tool will disconnect hport:H1/B1 from hnet:H1/B1, but leave hpin:H1/B1 connected to hnet:P1

```
@voltus2>disconnect hport:H1/B1
```

disconnect_hpin

```
disconnect_hpin
[-help]
-hinst hinstName
-pin_name portName
```

Detaches the net connected to the specified port on the specified instance.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each disconnect_hpin parameter. For a detailed description of the command and all of its parameters, use the man command: `man disconnect_hpin`
`-hinst hinstName`	Specifies the name of the hierarchical instance.
`-pin_name portName`	Specifies the port on the module.

Example

- The following command detaches port `p1` from `moduleA`:

  ```
  disconnect_hpin moduleA p1
  ```

disconnect_pin

```
disconnect_pin
-inst instName
-pin
[-net]
```

Disconnects a terminal from a net.

Note: Detaching a terminal that drives an output terminal of a module produces a Verilog violation at the output terminal if you use `disconnect_pin`. Instead, use `connect_pin` to attach the terminal to a new net. The `connect_pin` command automatically detaches the terminal from the net connecting to the output terminal, then attaches the terminal to the net you specify.

Parameters

`-inst instName`	Specifies the instance that contains the terminal you want to detach.
`-net`	Specifies the net that is already connected to the terminal. If the terminal does not connect to the specified net, or if the net does not exist, the software displays an error message and the command stops. The software uses this parameter for error checking only.
`-pin`	Specifies the terminal to disconnect. If the terminal does not exist on the specified instance, the software displays an error message and the command stops.

Example

- The following command disconnects terminal `in1` of instance `i1/i2/i3`:

  ```
  disconnect_pin i1/i2/i3 in1 i1/i2/net26
  ```

  If instance `i1/i2/i3` does not exist, or terminal `in1` is not a terminal of `i1/i2/i3`, the software displays an error message and the command stops. The net `i1/i2/net26` is specified, so if net `i1/i2/net26` does not exist, or if the terminal is not already connected to net `i1/i2/net26`, the software displays an error message and the command stops.

gui_show_buffer_tree

```
gui_show_buffer_tree
-net netName
[-out_file fileName]
```

Displays a buffer tree associated with the specified net.

Parameters

`-net` *`netName`*	Specifies the name of the net for which you want to display the buffer tree.
`-out_file` *`fileName`*	Specifies the text file containing buffer locations.

Example

- The following command highlights instances and wires in the buffer tree associated with the net `reset` in the design display area:

  ```
  gui_show_buffer_tree -net reset
  ```

update_inst

```
update_inst
[-help]
-base_cell base_cell
[-dont_snap_to_placement_grid]
[-location {x y}] -name inst_name
[-orient {r0 r90 r180 r270 mx mx90 my my90}]
[-place_status {fixed soft_fixed placed unplaced cover}]
```

Updates the base cell of an existing instance object and returns the updated inst object pointer. The new base cell and existing base cell must have the identical pins.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `update_inst` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_inst`
`-base_cell` *base_cell*	Specifies the base cell to use for the new instance.
`-dont_snap_to_placement_grid`	If `-dont_snap_to_placement_grid` is specified, the tool places the instance at the specified location even if the location is off the placement grid. This parameter is used with `-location`. *Default*: `false`
`-name` *instName*	Specifies the name of the instance to update.
`-location {`*x y*`}`	Specifies the location of the changed instance. *Default:* Default is the same location.
`-orient {r0 r90 r180 r270 mx mx90 my my90}`	Specifies orientation of the changed instance. *Default:* Default is the same orientation.
`-place_status {fixed soft_fixed placed unplaced cover}`	Specifies the placement status of the changed instance. *Default*: Default is the same place_status.

Related Topics

- `create_inst`

10

Low Power Commands and Attributes

- check_power_domains
- clone_msv_gate
- commit_power_intent
- cut_power_domain_by_overlaps
- delete_power_intent
- get_cpf_user_attributes
- power_intent Category Attributes
- read_power_intent
- report_isolation
- report_partition_boundary_ports
- report_power_domains
- report_shifters
- report_voltage
- update_power_domain
- write_power_intent

check_power_domains

```
check_power_domains
[-help]
[-all_inst_in_power_domain]
[-always_on_bias]
[-always_on_buffer_type]
[-cell_bind {cell ...}]
[-drc_file string]
[-lib_binding_voltage]
[-lib_cell_binding]
[-global_connection [module_name_list]]
[-nets_missing_iso [fileName]
[-ignore_net_to_port_iso]
[-ignore_net_to_port_shifter]
[-place [-place_report report_name]]
[-power_switch]
[-power_within_min_gap]
[-retention {fileName ...}]
[-nets_missing_shifter [fileName] [-ignore_net_to_port_shifter]]
```

Verifies whether the power domains are set up correctly in terms of timing library binding and PG connection. This command also checks whether any power domains overlap. The `check_power_domains` command checks that libraries specified for each power domain of each `dc_corner` are complete for each power domain member list.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_power_domains` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_power_domains`
`-all_inst_in_power_domain`	Verifies all instances are in the power domain.
`-always_on_bias`	Checks the `always_on_bias` net connection.
`-always_on_buffer_type`	Checks the AOB type in domain.

`-cell_bind {cell ...}`	Checks against all views and power domains if binding exists in the list of specified cells.
`-drc_file fileName`	Creates the Design Rule Checking (DRC) violation markers and save those in the specified file.
`-lib_binding_voltage`	Checks voltage against the binding library.
`-lib_cell_binding`	Verifies that all cells are bound to correct timing library cells.
`-global_connection module_name_list`	
	Verifies that power and ground pins of instances within the power domain are connected to the power and ground nets of that power domain. In addition, this parameter verifies that the tie-high and tie-low connections of an instance are connected to the same power and ground nets of the power domain to which the cell belongs. If you do not specify a list of module names, all modules are verified.
`-nets_missing_iso`	Verifies if the nets are missing isolation instances according to the power intent file. It also checks if the isolation uses right type of cell specified in the IEEE1801 `map_isolation_cell` command (if any).
`-ignore_net_to_port_iso`	Ignores the nets connecting to I/O when verifying isolation using `-nets_missing_iso`.
`-ignore_net_to_port_shifter`	Ignores the nets connecting to I/O when verifying shifter using `-nets_missing_shifter`.
`-place`	Verifies if the instances are placed in the boundary of their power domain.
`-place_report report_name`	
	Specifies a name and location for the report generated when you use this command with the `-place` parameter.

`-power_switch`	Verifies whether each row in the power domains contain at least one power switch. The command issues a warning message detailing which rows do not contain at least one switch. It also checks if the power switch uses right type of cell specified in the IEEE1801 `map_power_switch_cell` command (if any).
`-power_within_min_gap`	
	Verifies that the power domain ring is within the distance specified by the `modifyPowerDomainAttr -minGaps` or `createPowerDomain -minGaps` command.
`-retention {`*`fileName ...`*`}`	
	Specifies a name for storing the verify retention information. It checks if the retention uses the right type of cell specified in the IEEE1801 map_retention_cell command (if any).
`-nets_missing_shifter`	Verifies if the nets are missing level shifter according to the power intent file. It also checks if the level shifter uses right type of cell specified in the IEEE1801 map_level_shifter_cell command (if any).

Example

The following command issues warning messages if instances that are not members of a power domain are placed within that power domain boundary, or if any power domains overlap:

```
check_power_domains -place
```

clone_msv_gate

```
clone_msv_gate
[-iso]
[-shifter]
[-instances inst1 inst2 .... ]
[-max_fanout integer]
[-max_transition num]
[-max_cap num]
[-honor_pre_placed]
[-honor_dont_touch]
```

Ensures that the cloned isolation or level shifter can drive a group of the receivers and be placed close to this group of receivers. By doing this, the timing QoR can be improved.

Note: The command will not automatically update your CPF file to match the new netlist. It is advised that you run Conformal Low Power to check for any issues. When the enable signal for isolation cells is cloned (creating new ports on the boundary of the power domain) the users may need to update the `create_isolation_rule` in CPF by defining the isolation control differently.

Parameters

`-honor_dont_touch`	Does not clone don't touch instances or cells. The default is to clone don't touch instances.
`-honor_pre_placed`	Does not clone pre-placed (fixed) instances. The default is to clone fixed instances
`-instances` *inst1 inst2 ...*	Clones specified instances.
`-iso`	Clones isolation cells.
`-max_fanout` *integer*	Specifies the maximum fanout count.
`-max_cap` *num*	An optional check to clone instances with maximum capacitance violation. The default is not to use `-max_cap` violation.
`-max_transition` *num*	Cloning will be only on instances with maximum transition time violation. The default is `2ns`.
`-shifter`	Clones level shifters.

commit_power_intent

```
commit_power_intent
[-help]
[-verbose]
```

Commits IEEE1801 power intent specifications for the design for use in verification and implementation of the structure and behavior of the design in context of a given power management architecture. The method supports incremental refinement of power intent specifications required for IP-based design flows.

Parameters

`-help`	Prints a brief description that includes type and default information for each `commit_power_intent` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man commit_power_intent`
`-verbose`	Prints the detailed information related to the `commit_power_intent` command.

cut_power_domain_by_overlaps

```
cut_power_domain_by_overlaps
power_domain_name
[-help]
```

Stylus Common UI does not support the overlapped power domains; to floorplan one power domain (inner domain) completely inside the other power domain (outer domain), use this command to cut the outer domain as a hollow or donut shape.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `cut_power_domain_by_overlaps` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man cut_power_domain_by_overlaps`
`power_domain_name`	Specifies the outer power domain where the nested power domain will reside.

delete_power_intent

```
 delete_power_intent
[-help]
[-keep_fence]
```

The command deletes all the power intent data stored by the `commit_power_intent` command in current session and cleans up the design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `delete_power_intent` parameter. For a detailed description of the command and all its parameters, use the man command: `man delete_power_intent`
`-keep_fence`	Retains domain groups and their fences at the time of removing the LP database for completing non-LP status of applications. After completing the non-LP status of the required applications, use `read_power_intent` and `commit_power_intent -keep_rows` to restore the retained domain groups and domain fences of the removed LP database.

get_cpf_user_attributes

```
get_cpf_user_attributes
[-help]
-domain domainName | -net netName
```

Reports the `user_attributes` of the power domain or net specified in the CPF file. If the domain or net has no `user_attributes` in the CPF file, then this command reports an empty string.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_cpf_user_attributes` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_cpf_user_attributes`
`-domain`	Specifies the name of the power domain containing the user_attributes.
`-net`	Specifies the name of the net having the `user_attributes`.

power_intent Category Attributes

The root attributes in the `power_intent` category are:

- power_intent_do_not_use_top_domain_for_port_voltage

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `power_intent` category, use the following command:

```
get_db -category power_intent
```

power_intent_do_not_use_top_domain_for_port_voltage

```
power_intent_do_not_use_top_domain_for_port_voltage {true | false}
```

Default: `false`

Controls voltage using top fterm domain for IO pin voltage.

read_power_intent

```
read_power_intent
[-help]
power_intent_file
{-cpf | -1801 | -msv_db }
```

Reads IEEE1801 file for specifying power intent for the design for use in verification and implementation of the structure and behavior of the design in context of a given power management architecture. The method supports incremental refinement of power intent specifications required for IP-based design flows.

Parameters

`-help`	Prints a brief description that includes type and default information for each `read_power_intent` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_power_intent`
`power_intent_file`	Specifies name of the IEEE1801 input file.
`-cpf`	Specifies name of the input file in CPF format.
`-1801`	Specifies the input file in IEEE1801 format.
`-msv_db`	Specifies the IEEE1801 data file (`.cpfdb`). **Note:** `-msv_db` is for partition flow and must be read for power intent in partition implementation.

report_isolation

```
report_isolation
[-help]
[-highlight]
[-out_file fileName]
[-from powerDomain]
[-to powerDomain]
```

Reports or shows the added isolation cells and instances.

Note: Use this command after you add isolation cells.

Parameters

-help	Outputs a brief description that includes type and default information for each `report_isolation` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_isolation`
`-from`	Reports all the isolation instances connected to the output pins of the power domain specified in `-from` option. *Default:* Reports all isolation cells.
`-highlight`	Highlights isolation instances in the main window.
`-out_file` *fileName*	Writes isolation cell and instance names to a report. *Default:* The software writes the report to a default file: *designName*`.isolation.rpt`
`-to`	Reports all the isolation instances connected to the input pins of the power domain specified in `-to` option. *Default:* Reports all isolation cells.

Example

The following command highlights isolation cells and writes the cell names to `isolationCellReport`:

```
report_isolation -highlight -out_file isolationCellReport
```

report_partition_boundary_ports

```
report_partition_boundary_ports
-pins
```

The `report_partition_boundary_ports` command prints out the boundary pins' debug information in a log which shows how to assign the power domain for the partition pins.

This is useful when debugging CPF partitions as you get the information about how the software assigns the power domain for the partition boundary pins during debugging.

Parameters

-pins	Specifies a list of the partition pins. If wildcard * is specified, it means all partition pins.

Example

Specify the partition pins(Hterm) you want to debug:

```
report_partition_boundary_ports "I2/d I2/clk"
```

After partition, the command looks for the following DBG info in the log file
DBG: deriving hterm I2/d power domain
DBG: ….

report_power_domains

```
report_power_domains
[-help]
[-bind_lib]
[-insts inst_name]
[-iso_inst]
[-modules {list_of_modules}
[-nets net_name]
[-out_file outFile]
[-pg_nets]
[-pins]
[-power_domain powerDomainName [-check_pg_scope]]
[-power_domain_boundary_port string]
[-power_domain_coverage pd1 pd2]
[-shifter]
[-supply_crossing supp1 supp2]
[-verbose]
[-voltage]
```

Reports information about power domains. The report includes the following attributes:

- Power domain name
- Power domain bounding box
- Timing library
- Mingap
- RS extension
- Row type
- Row spacing

The command also reports whether the power domain is always on.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_power_domains` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_power_domains`
`-bind_lib`	Reports the timing library bound to each instance of a power domain.
`-check_pg_scope`	Checks if there are any PG connections that cross UPF-defined scope. All PG connections for UPF scope should be at the boundary PG port. This parameter is used with the `-power_domain` parameter.
`-insts` *inst_name*	Reports the power domain-related information for the specified instance: the input, output, and power/ground connections.
`-iso_inst`	Reports all isolation cells in the power domain.
`-modules {list of modules}`	Checks whether the specified instances or hierarchical instances are in the power domain. *Default:* The software does not perform this check.
`-nets` *net_name*	Reports power domain-related information for the specified net: • The driver instance of the net and its power domain. • The receiver instances of the net and their power domains.
`-out_file` *outFile*	Specifies the name of the output file. *Default:* *designName*`-powerdomain.rpt`
`-pg_nets`	Reports all the power/ground nets in the power domain.
`-pins` *pin1, pin2 ...*	Lists the pins and the option is of type string. This option is optional.
`-power_domain` *powerDomainName*	Specifies the power domain whose attributes you want to report. *Default:* If you do not specify the power domain, this command report data about all power domains.

`-power_domain_boundary_port` *string*	Reports if the specified port name of the power domain boundary has an IEEE1801 constraint.
`-power_domain_coverage` *pd1 pd2*	Reports the coverage between two specified domains.
`-shifter`	Reports all shifters in the power domain.
`-supply_crossing` *supp1 supp2*	Reports if the two specified supplies have the on/off and low/high crossing.
`-verbose`	Prints out the detail information and is optional.
`-voltage`	Reports domain voltage.

Examples

- The following command reports all shifter cells, all isolation cells, and all power/ground nets in power domain `PD1`. The command also checks whether HInst `instA` is contained in `PD1`, and reports all results to `myDesign-powerdomain.rpt`.

  ```
  report_power_domains pdName PD1 -out_file myDesign-powerdomain.rpt -modules {instA} -shifter -iso_inst -pg_nets
  ```

- The following example shows how output file `outfile` contains the binding between the timing libraries and instances in power domain VCORE:

  ```
  -report_power_domains -bind_lib -power_domain VCORE -out_file outfile

  Timing library for instance(s):
  ==========================
  INST = uLSCVSocDownVCore/uACLKEND/uLVHL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownVCore/uACLKENI/uLVLHL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownvCore/uaCLKENP/uLVLHL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownvCore/uaCLKENRW/uLVLHL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownvCore/uaBIGENDINIT/uLVHlL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownvCore/uaCLKIN/uLVHL (lvl_slow_1v08_1v08)
  INST = uLSCVSocDownvCore/uCP15SDISABLE/uLVHL (lvl_slow_1v08_1v08)
  ```

- The following command reports the power domain-related information about instance `PM_INST/state_inst/g45`:

```
report_power_domains -insts PM_INST/state_inst/g45
```

Result:

```
Analyzing PM_INST/state_inst/g45 (Cell: AN2D1) power domain:AO
------from side ------
Pin A2 => Net PM_INST/state_inst/n_28
PM_INST/state_inst/g71/ZN in power domain A0
Pin A1 => Net PM_INST/state_inst/n_25
PM_INST/state_inst/g47/Z in power domain A0
------to side ------
Pin Z => Net PM_INST/state_inst/n_36
PM_INST/state_inst/fsm_reg[1]/qi_reg/D in power domain A0
==========PG pins =============
VDD conneted to Net: VDD
VSS connected to Net:VSS
```

report_shifters

```
report_shifters
[-help]
[-cell cellName]
[-from powerDomainName]
[-to powerDomainName]
[-out_file fileName]
[-highlight]
```

Reports the level shifters that have been read into or defined for the software. In CPF flow, the level shifters are inserted.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_shifters` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_shifters`.
`-cell`	Reports shifters with the specified cell. *Default:* Reports all level shifters.
`-from` *powerDomainName*	Reports all the shifter instances connected to the output pins of the power domain specified in the `-from` option. *Default:* Reports all level shifters.
`-highlight`	Highlights placed level shifters.
`-out_file` *fileName*	Reports all instances in an output file. *Default:* The instances will be reported to a default output file *designName*`.shifter.rpt`. A warning message is printed to indicate a default output file is created.
`-to` *powerDomainName*	Reports shifter instances connected to the input pins of the power domain specified in the `-to` option. *Default:* Reports all level shifters.

Examples

- The following command reports all level shifters in design `chip`:

```
loadShifter -infile chip.vsf
addShifter
report_shifters
```

- The following command reports all level shifters in the `shifter` output file in highlighted mode:

```
report_shifters -highlight -out_file shifter.rpt
#DomainName     CellName        InstanceName
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LIN
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LCLK
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LOUT
```

- The following command reports all level shifters in the `shifter_power` output file from the `AO` pin to the `PD1` pin:

```
report_shifters -from AO -to PD1 -out_file shifter_power.rpt
#DomainName     CellName        InstanceName
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LIN
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LCLK
```

- The following command reports all level shifters in the `shifter_cell` output file for the `A2LVLUO_X1M_A12TH` cell:

```
report_shifters -cell A2LVLUO_X1M_A12TH -out_file shifter_cell.rpt
#DomainName     CellName        InstanceName
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LIN
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LCLK
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LOUT
```

report_voltage

```
report_voltage
[-help]
[-early]
[-inst string]
[-late]
[-net string]
[-pin string]
[-quiet]
[-view string]
```

This command reports the voltage for the net/pin (term)/inst in the specific view under a specific corner.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_voltage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_voltage`
`-early`	Specifies that the voltage is reported in the early mode.
`-inst string`	Specifies the instance power domain voltage (PVT).
`-late`	Specifies that the voltage is reported in the late mode.
`-net string`	Specifies the net voltage (driver's pin voltage).
`-pin string`	Specifies the term voltage.
`-quiet`	Displays only the pin voltage-related information without other additional unnecessary information to avoid huge log size.
`-view string`	Specifies the analysis view.

update_power_domain

```
update_power_domain
[-help]
power_domain_name
[-add_block_box string]
[-core_to_bottom valueInMicron]
[-core_to_left valueInMicron]
[-core_to_right valueInMicron]
[-core_to_top valueInMicron]
[-default_tech_site string]
[-disjoint_hinst_box_list string]
[-extra_row_pattern {row_pattern_site_name ...}]
[-first_row_site_index integer]
[-gap_sides {values}|-gap_edges {values}]
[-last_row_site_index integer]
[-power_extend_sides {values}|-power_extend_edges {values}]
[-row_flip {first second noflip auto}]
[-row_pattern_site site_name]
[-row_space_type {0 1 2}]
[-row_spacing float]
```

Renames a power domain or changes its structure.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `update_power_domain` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_power_domain`
`-add_block_box`	Adds a hard block to the disjoint power domain. `-add_block_box` supports both rectangular and rectilinear macros.
`-core_to_left valueInMicron`	Specifies the row margin from left boundary.
`-core_to_right valueInMicron`	Specifies the row margin from right boundary.

`-core_to_top` *valueInMicron*	Specifies the row margin from top boundary.
`-core_to_bottom` *valueInMicron*	Specifies the row margin from bottom boundary.
`-default_tech_site` *string*	Specifies the default tech site for power domain.
`-disjoint_hinst_box_list`	Specifies a region for an Hinst. It is defined as an Hinst and Region pair.
`-extra_row_pattern {`*row_pattern_site_name* `...}`	Specifies the ROWPATTERN site name for the specified power domain. You can also specify a list of ROWPATTERN sites with this parameter. ⚠ • The rows are created for the *specified* ROWPATTERN sites within the domain fence, if specified, from the domain fence bottom edge, as per the defined order and orientation of all sites. • The ROWPATTERN site needs to be defined in the LEF file. *Default*: ""
`-first_row_site_index` *integer*	Specifies the first row site index value.
`-last_row_site_index` *integer*	Specifies the last row site index value.
`-power_extend_edges`	Specifies the maximum search distance for power extension connections by domain edge in microns. You can specify the value per edge in clockwise order, starting with the vertical edge at the lower-left corner (smallest Y, and then smallest X).
`-gap_edges`	Specifies the minimum space value to other domain by power domain edge in microns. You can specify the value per edge in clockwise order, starting with the vertical edge at the lower-left corner (smallest Y, and then smallest X).

`-gap_sides` *`T B L R`*	Defines the distance, in microns, that must be reserved from the power domain boundary edges for power routing. You can specify four values, one for each edge.
`power_domain_name`	Specifies the current name of the power domain to be modified. This is a positional parameter, and you must specify this value immediately after the command name.

`-row_flip {first | second | noFlip | auto}`	Flips the orientation of either the first or second row, then follows the flipped and abutted row pattern.
	`first`
	(from bottom to top). MX -> R0 -> MX -> R0 - > …
	`second`
	(from bottom to top). R0 -> MX -> R0 - > MX -> …
	`noFlip`
	(from bottom to top). R0 -> R0 -> R0 - > R0 -> …
	The noFlip option preserves the R0 orientation for all rows (above).
	`auto`
	Detects orientation of row outside of power domain and aligns with power domain bottom. Orientation is applied to the first row at the bottom of power domain.
	So if row on left is R0, the pattern will be (from bottom to top). R0 -> MX -> R0 - > MX -> …
	If row on left is MX, the pattern will be (from bottom to top). MX -> R0 -> MX -> R0 - > …
	The `auto` option resets the software for this automatic behavior if you have previously specified `first`, `second`, or `noFlip`.
	Default: `auto`
	Note: *saveRowImage* resets the option `-row_flip` to `auto`. In this case, try moving the power domain and check the row changes.

`-row_pattern_site` *site_name*	Specifies the site name for the row pattern.
`-row_spacing` *value*	Specifies the row spacing between each row (1) or each pair of rows (2) as specified in `-row_space_type`.
`-row_space_type {0|1|2}`	Determines whether the row spacing value applies between no row (0), each row (1), or each pair of rows (2).

`-power_extend_sides` *T B L R*	Specifies the boundary for legal targets to be used by the power planning and routing commands, in conjunction with the power domain boundary. You can specify four values, one for each edge.

Examples

The following command changes the route search extension values of `PowerDomain1` to 15 µm on each side:

```
update_power_domain PowerDomain1 -power_extend_sides 15 15 15 15
```

write_power_intent

```
write_power_intent
[-help]
file_name
-incremental_1801
{-cpf [[-hinsts -top_hier] | [-hinst_only]] | -1801}
[-no_extra_connects]
```

Writes the CPF/IEEE1801 file. The command options help to write the file for CPF as well as for the IEEE1801 standard.

Parameters

`-help`	Prints a brief description that includes type and default information for each `write_power_intent` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_power_intent`
`file_name`	Specifies name of the IEEE1801 input file.
For CPF	
`-cpf`	When specified, the software writes out the CPF file.
`-hinst_only`	Specifies the hierarchical instance name. Writes (partitions) the CPF for the hierarchical instance specified in this option. **Note:** Must be used along with `-cpf` option.
`-hinsts`	Writes top-level CPF where the hierarchical instances specified in `-hinsts` option are hard blocks when specified together with `-top_hier -cpf`.
`-top_hier`	Writes top-level CPF when specified together with `-hinsts` and `-cpf`.

For IEEE1801	
`-1801`	Writes the IEEE1801 file from the low-power database. It filters out the non-existing instances from the original IEEE1801 file and appends to it the following: • PG connection for the newly-inserted always-on buffers • Nwell/Pwell connections for the low-power cells • `-no_isolation`/`shifter` for the isolation/level shifter strategies for the newly-created ports along the power domain boundary
`-incremental_1801`	Specifies the incremental IEEE1801 power intent.
`-no_extra_connects`	If this parameter is specified, `write_power_intent -1801` does not write out any extra `connect_supply_net` (PG connection). For example: `write_power_intent -1801 -no_extra_connects out.upf` **Note:** Extra connects means that the extra `connect_supply_net` commands appended by the tool, which are not included in the original UPF.

11

Multiple-CPU Processing Commands

- check_multi_cpu_usage
- get_distributed_hosts
- get_multi_cpu_usage
- monitor_distributed_hosts
- report_resource
- set_distributed_hosts
- set_multi_cpu_usage

check_multi_cpu_usage

`check_multi_cpu_usage`

Checks if distributed processing can work in the software environment. This command also ensures that the working directory is accessible from the master machine as well as the client machines. The `check_multi_cpu_usage` command is specified after configuring the multi-CPU settings (`set_distributed_hosts` and `set_multi_cpu_usage`) to check if all the specified CPUs can be accessed. This command should be invoked before running any commands that use multiple CPUs.

Parameters

None

Examples

- The following example command script checks the distributed computing environment:

```
> check_multi_cpu_usage

Connected to sjfnl046 34224 2
Connected to sjfnl046 34225 1
Connected to sjfnl046 34226 0
Connected to sjfnl046 34227 3
Task 1 successfully accessed the current working directory.
Task 2 successfully accessed the current working directory.
Task 3 successfully accessed the current working directory.
Task 0 successfully accessed the current working directory.
Task 0: cpu(workload/cpu): 0.02/8 mem(free/total): 79634896k/103112480k.
Task 3: cpu(workload/cpu): 0.02/8 mem(free/total): 79634904k/103112480k.
Task 1: cpu(workload/cpu): 0.02/8 mem(free/total): 79635224k/103112480k.
Task 2: cpu(workload/cpu): 0.02/8 mem(free/total): 79635232k/103112480k.
Test heartbeat for task -1...
```

  The distributed processing environment has been set up correctly.

get_distributed_hosts

```
get_distributed_hosts
[-help]
[-args]
[-custom_script]
[-hosts]
[-mode]
[-queue]
[-report_lsf_info <file>]
[-resource]
[-shell_timeout]
[-single_cpu_lsf_args]
[-timeout]
[-wait_for_lsf_info <integer>]
```

Displays settings for the `set_distributed_hosts` command.

Parameters

`-args`	Displays the additional arguments when the software is running in the Sun Grid Engine (SGE) or Load Sharing Facility (LSF) mode.
`-custom_script`	Displays the name of the custom script when the software is running in `custom` mode.
`-hosts`	Displays the list of host names when the software is running in `rsh` mode.
`-help`	Outputs the command usage and a brief description about the command parameters.
`-mode`	Displays the distributed processing mode. The possible values are `local`, `sge`, `rsh` , `ssh` , `lsf` and `custom`. *Default*: `local`
`-queue`	Displays the queue name when the software is running in `sge` or `lsf` mode.
`-report_lsf_info` *file*	Specifies to restore the LSF information to the specified file. This parameter can only be used when `set_distribute_host -report_lsf_info` is specified in the current session. You must specify the `get_distributed_hosts -report_lsf_info` only after the LSF tasks exit to obtain the task CPU time. If you specify this command before the task exits, the command gives a warning message.
`-resource`	Displays the resource string when the software is running in `sge` or `lsf` mode.
`-shell_timeout`	Displays the number of seconds the host machine waits for `rsh/ssh` machines to become available for multiple-CPU processing.
`-single_cpu_lsf_args`	Displays the lsf arguments when the software is running in `lsf` mode for a single CPU application.
`-timeout`	Displays the default timeout for a submitted job that will never start.
`-wait_for_lsf_info` *value*	Specifies the timeout period required to wait for task exit and generate the LSF information.

Examples

- The following command displays the current setting for the `-mode` parameter:

  ```
  get_distributed_hosts -mode
  local
  ```

- The following command saves the LSF information to the `lsf.log` file:

  ```
  get_distributed_hosts -report_lsf_info lsf.log
  ```

get_multi_cpu_usage

```
get_multi_cpu_usage
[-local_cpu | -remote_host | -cpu_per_remote_host | -keep_license | -thread_info | -
auto_page_fault_monitor]
[-verbose]
[-help]
```

Displays the requested number of threads or hosts.

To request threads or hosts, use `set_multi_cpu_usage`.

Parameters

`-auto_page_fault_monitor`	Displays the warning rank set for performance issues caused by major page faults
`-cpu_per_remote_host`	Displays the number of CPUs available on every remote host for Superthreading.
`-help`	Outputs the command usage and a brief description about the command parameters.
`-keep_license`	Displays the current setting for the `set_multi_cpu_usage -keep_license` parameter.
`-local_cpu`	Displays the number of available local CPUs for multi-threading.
`-remote_host`	Displays the number of available remote hosts for distributed processing or Superthreading.
`-thread_info`	Displays the level of informational messages for each thread.
`-verbose`	Displays a message with the current multiple-CPU setting.

Examples

- The following command displays the requested number of threads for multi-threading:
  `get_multi_cpu_usage -local_cpu`
- The following command displays the requested number of remote hosts for Superthreading or

distributed processing:

```
get_multi_cpu_usage -remote_host
```

- The following command displays the requested number of CPUs per remote hosts for Superthreading:

```
get_multi_cpu_usage -cpu_per_remote_host
```

Related Commands

- set_multi_cpu_usage

monitor_distributed_hosts

```
monitor_distributed_hosts
[-help]
[-kill_on_insufficient_resource]
[-log string]
[-min_required_tmp_space integer]
[-overwrite]
[-period integer]
```

Monitors hosts and periodically dumps memory or CPU usage information for each host to a file. If there are multiple hosts for a distributed flow (e.g., DMMMC in STA), then details of each host will be logged. This command automatically determines the hosts to be monitored. By default, after every 60 seconds hosts are monitored and the disk information is written to a log file.

The `monitor_distributed_hosts` command is also supported in DSTA mode.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man monitor_distributed_hosts`.
`-kill_on_insufficient_resource`	Terminates the session if a host resource is below the specified threshold values.
`-log string`	Specifies the log file name. *Default*: host-monitor.log
`-min_required_tmp_space integer`	Specifies the condition for insufficient resources. This option should be used with the `-kill_on_insufficient_resource` parameter.
`-overwrite`	Overwrites the log file, if it already exists.
`-period integer`	Specifies the sampling period after which hosts will be monitored. *Default*: 60

Examples

- A sample monitor log file is shown below:

```
Status keys: E excellent, G good, B bad, T terrible.
Started at: 14/09/04 04:45:06
sjfsb432(0) TMPDIR is : /tmp/inn_tmpdir_24708_IsKujp

============ ======== =====================
Host CPU Memory (GB) TMPDIR (GB)
name(id) util % total progs cache %progs used avail
============ ======== ===== ===== ===== ======
04:45:06
sjfsb432(0) 22 E 755 432 343 57 E 0.00 426.90 E
04:46:06
sjfsb432(0) 24 E 755 431 343 57 E 0.00 426.90 E
04:47:07
sjfsb432(0) 23 E 755 430 343 56 E 0.00 426.90 E
04:48:07
sjfsb432(0) 23 E 755 431 344 57 E 0.00 426.90 E
04:49:07
sjfsb432(0) 22 E 755 430 345 56 E 0.00 426.96 E
04:50:07
sjfsb432(0) 23 E 755 431 345 57 E 0.00 426.96 E
```

Related Commands

- set_multi_cpu_usage

report_resource

```
[-help]
[-check_memory_cpu {true | false} [-period <integer>]]
[-peak]
[-start string | [-end string [-session_cpu] [-diff_memory]]]
[-verbose]
```

Displays the peak memory used during a software session. This command is used to measure the peak memory at any given point in the flow and and make the script better for automation flows.

⚠ The `report_resource` command is only supported in the 64-bit version of the software and Linux port.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
`-check_memory_cpu {true|false}`	Checks the machine's physical memory and CPU resources. When specified, the software enables memory/CPU checking and reports a warning if the free memory is below 5 percent of the total memory or if the average CPU load is larger than 1.35.
`-diff_memory`	Outputs different peak memory and current memory between the step start and step end.
`-peak`	Reports current peak memory till a given point in a flow. This parameter also works with the `-start` and `-end` parameters of the `report_resource` command so that peak memory observed by an individual command can be reported.
`-start` *startpoint*	Specifies the start point to measure the peak memory attained during a specific command. **Note:** The `-start` and `-end` parameters support any user-specified string, such as fullFlow, time_design, and TD_PR. You must specify the same string for both the `-start` and `-end` parameters to measure the peak memory attained during a specific command.

`-session_cpu`	Reports the current CPU and real time after the reports last step status.
`-end` *`endpoint`*	Specifies the end point to measure the peak memory attained during a specific command.
`-period` *`integer`*	Specifies the period for checking the physical memory and CPU usage. *Default*: 1800s *Min*: 1 *Max*: INT_MAX
`-verbose`	Gives detailed memory usage information. It displays how much memory is being used at any time and of what form (physical vs. virtual). When specified, the following information is displayed: • `Cpu(s)` is the number of available processors in the machine. • `Load` average is the system load averages for the past 1 minute. • `Mem` and `Swap` are the current memory information of the machine. • `Data Resident Set (DRS)` is the amount of physical memory devoted to other than executable code. "`current mem`" shows this value (Total current DRS) . • `Private Dirty (DRT)` is the memory which must be written to disk before the corresponding physical memory location can be used for some other virtual page. "`peak res`" shows this value (Total peak DRT). This is the minimum number that you must reserve to run the program. • `Virtual Size (VIRT)` is the total amount of virtual memory used by the task. • `Resident Size (RES)` is the non-swapped physical memory a task has used. The number of "Total Peak RES" is the recommended physical memory to reserve.

Examples

- The following command script specifies to display the peak memory attained during the `opt_design` command:

  ```
  report_resource -start "optD"
  opt_design
  report_resource -end "optD"
  ```

  The following message is displayed:

  ```
  Ending "optD" (total cpu=0:15:00, real=0:15:09, peak res=468.8M, current
  mem=396.3M)
  ```

- The following command script specifies to display the peak memory attained during the `time_design` command:

  ```
  report_resource -start fullFlow
  report_resource -start TD_PR
  time_design -postRoute
  report_resource -end TD_PR
  report_resource -start OD_PR
  opt_design -postRoute
  report_resource -end OD_PR
  report_resource -end fullFlow
  ```

  The following message is displayed:

  ```
  Ending TD_PR (total cpu=0:15:00, real=0:15:09, peak res=468.8M, current
  mem=396.3M)
  Ending OD_PR (total cpu=0:15:00, real=0:15:09, peak res=968.8M, current
  mem=796.3M)
  Ending fullFlow (total cpu=0:35:03, real=0:40:09, peak res=968.8M, current
  mem=796.3M)
  ```

- The following command script specifies to display the peak memory attained during the `opt_design` command:

  ```
  report_resource -peak -start optD1
  report_resource -start optD
  opt_design -postRoute
  ```

```
report_resource -end optD
`Ending "optD" (total cpu=0:00:12.9, real=0:00:48.0, peak res=358.8M, current
mem=300.2M)'
report_resource -peak -end optD1
`Ending "optD1" peak res=358.8M'
```

- The following command specifies to display detailed memory information:
  `report_resource -verbose`

  The following is displayed:

Current (total cpu=0:00:12.9, real=0:05:48, peak res=275.8M, current mem=383.9M)				
Cpu(s) 2, load average: 4.63				
Mem: 16443800k total, 16378412k used, 65388k free, 105704k buffers				
Swap:16777208k total, 17460k used, 16759748k free, 12528212k cached				
Memory Detailed Usage:	Data Resident Set(DRS)	Private Dirty(DRT)	Virtual Size(VIRT)	Resident Size(RES)
Total current:	383.9M	275.8M	854.1M	358.9M
peak:	383.9M	275.8M	854.1M	358.9M

set_distributed_hosts

```
set_distributed_hosts
[-help]
[out_file]
[-add string]
[-args string]
[-custom]
[-custom_script string]
[-custom_script_list string]
[-local]
[-lsf]
[-nc]
[-no_wait_task_come_up]
[-queue string]
[-remove string]
[-report_lsf_info]
[-resource string]
[-rsh]
[-shell_timeout integer]
[-single_cpu_lsf_args string]
[-ssh]
[-time_out integer]
```

Specifies the multiple-CPU processing configuration for distributed processing or Superthreading. The software finds hosts from the last `set_distributed_hosts` command specified before a distributed processing or Superthreading command starts.

Use `get_distributed_hosts` to display the current settings for the `set_distributed_hosts` command.

> This command is required for distributed processing and Superthreading.

Parameters

`-help`	Prints out the command usage

out_file	Specifies to report LSF information in a single report. This parameter allows you to collect the system statistics on the status of the distributed jobs. It has the following information: • Host name for master and LSF tasks • Number of CPUs on each machine that are being used by a process • Job ID/PID for each task process • Total memory used on each machine • Total virtual memory used on each machine • Total CPU time used on each machine You must specify this parameter with the `-lsf` parameter. To get the LSF information, specify the `get_distribute_host out_file` command parameter. You must specify the `get_distribute_host out_file` only after the LSF tasks exit to obtain the task CPU time. If you specify this command before the task exits, the command gives a warning message.

`-add` *string*	Adds the specified hosts to the `rsh/ssh` configuration. If you specify more than one host, separate each host name with a space.
	Each time you specify a host, the software uses it to run one client process. To run more than one client process on a host, specify it more than once. For example, if you have two machines that each has two CPUs and you want to use both CPUs in both machines, type a command like the following:
	`set_distributed_hosts -rsh -add {machine1 machine2 \`
	`machine1 machine2}`

`-args` *string*	Specifies any additional arguments you need to provide to the Sun Grid Engine (SGE) or Load Sharing Facility (LSF) job submission command.
`-custom`	Use custom job submission script for distributed processing.

`-custom_script` *`string`*	Specifies a custom script to run distributed processing. This includes custom job submission script and arguments for "custom" distributed processing mode.
`-custom_script_list` *`string`*	Specifies custom job submission script list and arguments for "custom" distributed processing mode. for example, `{{{stringCommand1} count1} {{stringCommand2} count2} ... {{stringCommandN} countN}}`. Specifies to customize the launch of multiple jobs, based on resources required for these jobs. This parameter allows you to specify multiple launch strings in order of priority with a maximum allowed job count for each string. If you request for N number of jobs, the command starts submitting jobs starting from *`string1`*, until the corresponding count is reached, at which point it starts submitting the next string in the list. Therefore, the command submits the first "*`count1`*" jobs with *`string1`*, the next "*`count2`*" jobs with *`string2`* and so on. It stops when the total submitted jobs has reached the requested number (N). **Note**: This parameter can only be specified with the `-custom` parameter.
`-local`	Run distributed processing job on the local machine.
`-lsf`	Specifies a Load Sharing Facility configuration. If you do not specify any parameters for `-lsf`, the software uses default settings. **Note:** To use this parameter, LSF must already be set up (typically by specifying the LSF_ENVDIR and LSF_SERVERDIR environment variables) and bsub must be in your search path. Contact your LSF administrator for details.
`-nc`	Allows the software to use NC commands (Network Computer from RTDA) for distributed processing.
`-no_wait_task_come_up`	Does not allow the software to wait for a long time if a task does not come up.
`-queue` *`string`*	Specifies the queue for the LSF or SGE configuration.

`-remove` *`string`*	Removes the specified hosts or all hosts from the `rsh/ssh` configuration. If you specify more than one host, separate each host name with a space.
`-report_lsf_info`	Collects the LSF slave information after EDP finishes.
`-resource` *`string`*	Specifies a resource string for the SGE or LSF queue. **Note:** The correct resource string is specific to your installation. Contact your LSF administrator for the appropriate parameters and values.
`-rsh`	Specifies a remote shell configuration. For information, see the `rsh` documentation.
`-shell_timeout` *`integer`*	Specifies the number of seconds the host machine waits for `rsh/ssh` machines to become available for multiple-CPU processing. *Default:* `5`
`-single_cpu_lsf_args` *`string`*	Specifies the `bsub` options. For information, see the LSF documentation. **Note:** You must use this parameter to adjust the LSF argument for a single CPU application. Alternatively, use the `-args` parameter to adjust the LSF argument for an application that runs in the Superthreading mode.
`-ssh`	Specifies a secure shell configuration.

`-time_out` *integer*	Specifies the default timeout for a submitted job that will never start. This functionality prevents the master machine from hanging when the clients do not come up for some reason. After timeout, a warning message appears informing you that the distributed CPUs did not come online within the expected time. The master will continue waiting for the clients and it is up to you to continue waiting or kill the run if there are problems with the Distributed Resource Management (DRM) system. A sample warning message is given below: Task id 2 did not come up within the specified timeout period (300 seconds). Use `set_distributed_hosts -timeout` to change the value. This may be a real error due to incorrect or incomplete platform/machine specification provided via set_distributed_hosts to your distributed resource management system (LSF/Sun Grid Engine etc.). If running locally (`set_distributed_hosts -local`), the current machine may be overloaded with too many jobs and swapping (check free memory and CPUs with top). For local runs, consider reducing the number of CPUs you are using via the `set_multi_cpu_usage` command or free up some memory by killing other memory intensive processes. *Default:* `30(local/rsh/custom)and 3600(lsf/sge)`

Examples

rsh

The following command creates an `rsh` configuration and adds hosts `host10`, `host11`, and `host12`:

```
set_distributed_hosts -rsh -add {host10 host11 host12}
```

The following commands create an `rsh` configuration with `host1` and `host2` as remote hosts and then adds `host3` and `host4` to the configuration:

```
set_distributed_hosts -rsh -add {host1 host2}
set_distributed_hosts -rsh -add {host3 host4}
```

LSF

The following command uses an existing LSF environment, and relies on the general LSF set up by the system administrator. In this example, the name of the queue implies that all the machines have 4 Gb of memory.

```
set_distributed_hosts -lsf -queue mem4G
```

The following command schedules a LSF job named `bigQueue` to run on a Linux machine for a maximum of 30 minutes. No other jobs can run on the machine while `bigQueue` is running.

```
set_distributed_hosts -lsf -queue bigQueue -args {-x -W 30} \
   -resource {OSNAME==Linux}
```

The following command schedules a LSF job named `smallQueue` to run on Linux machines with minimum speed of 2000 MHz.

```
set_distributed_hosts -lsf -queue smallQueue -args {-W 20} \
    -resource {OSNAME==Linux && SPEED>2000}
```

Note: Check with your LSF administrator for the correct values for the resource string, as the parameters and their values differ in each installation. For example, the parameter name for the speed of the machine might be `MHZ` instead of `SPEED` (as in the preceding example), and the units might be specified in gigahertz instead of megahertz.

The following command specifies to report LSF information:

```
set_distributed_hosts -lsf -args "-q lnx64" out_file
get_distribute_host out_file lsf.log
```

Note: Use the above two commands together to save the LSF information in the specified file.

SGE

The following command schedules an SGE job named `all.q` to run on Solaris machines with minimum speed of 2000 MHz.

```
set_distributed_hosts -sge -queue all.q \
-resource {OSNAME==Solaris && SPEED>2000} -args {-A [account name] -N [name]}
```

Local

The following command runs all distributed processing jobs on the machine on which you are running the master version of the software:

```
set_distributed_hosts -local
```

Custom

The following command runs distributed processing with a custom script:

```
set_distributed_hosts -custom -custom_script {bsub -q bigQueue -x -R "OS==Linux &&
memory>20000"}
```

The following command runs distributed processing with a custom script list:

```
set_distributed_hosts -custom -custom_script_list {{{bsub -P ICD -W 2 -q admin -R
"OSNAME==Linux && OSREL==EE50"}1} {{rsh hpc1 -e} 1}}
```

set_multi_cpu_usage

```
set_multi_cpu_usage
[-help]
[-acquire_license integer]
[-auto_page_fault_monitor {0 1 2 3}]
[-cpu_per_remote_host integer]
[-keep_license {true | false}]
[-license_list string]
[-local_cpu string]
[-release_license]
[-remote_host integer]
[-reset]
[-thread_info {0 | 1 | 2}]
[-verbose]
```

Specifies the number of threads to use for multi-threading, or the maximum number of computers to use for distributed processing, or the maximum number of computers and the number of threads to use for Superthreading. Optionally, reports usage information.

> This command is required for multi-threading, distributed processing, and Superthreading.

The `set_multi_cpu_usage` command enables multi-threaded timing reporting under the following conditions:

- `the timing_report_group_based_mode` attribute is set to `true`. If there are multiple paths with the same slack then the selection can change on multiple runs. This may result in some changes in the `report_timing` output.
- `the report_analysis_coverage` in non-verbose mode.
- `the check_timing -include clock_crossing` option is specified.
- `the -set_table_style -no_frame_fix_width [-nosplit]` option is specified.

To disable multi-threaded reporting, you can set the `timing_enable_multi_threaded_reporting` attribute to `false`. This attribute will only disable multi-threaded reporting feature, while other components (RC extraction, delay calculation, timing analysis, and so on) of the software will continue to run in multi-threaded mode.

Parameters

`-help`	Prints out the command usage.

`-acquire_license` *integer*	Acquires licenses to enable the specified number of CPUs. For example, if you specify -acquire_license 5, the software checks out enough licenses to enable 5 CPUs. When specified, the tool displays the number of available CPUs.
	`[DEV]set_multi_cpu_usage -acquirelicense 5`
	`Total CPU(s) requested: 5`
	`Total CPU(s) enabled with current License(s): 2`
	`Current free CPU(s): 2`
	`Additional license(s) checked out: 1`
	`Encounter_CPU_accelerator_GXL license(s) for 4 CPU(s)`
	`Total CPU(s) now enabled: 6`
	After the licenses are checked out, they can be used for multi-threading, distributed processing, or superthreading.

`-auto_page_fault_monitor {0 | 1 | 2 | 3}`	Specifies to set a warning rank for performance issues caused by major page faults. The parameter supports the following values: • `0` - specifies to disable the reporting of major page faults. • `1` - specifies to set the warning rank as 1. This means `PF/total PF` > 0.8, `b` > 10, and `wa` > 85% • `2` - specifies to set the warning rank as 2. This means `PF/total PF` > 0.5, `b` > 5, and `wa` > 65% • `3` - specifies to set the warning rank as 3. This means `PF/total PF` > 0.2, `b` > 1, and `wa` > 40% where `PF` is the number of major page faults from the software, `total PF` is the total number of major page faults in the machine, `b` is the number of blocked processors, and `wa` is CPU waiting time.

`-cpu_per_remote_host` *integer*	Specifies the number of CPUs on each of the remote machines. This parameter is required for Superthreading. For Superthreading, you must use this parameter in conjunction with the -remote_host parameter. *Default:* `1`
`-keep_license {true | false}`	Specifies whether to keep the acquired multiple CPU-licenses until the current session ends. Specify this parameter before running any commands that require multiple-CPU applications. To release all multiple-CPU licenses immediately, use the releaseLicense option. *Default:* `true`
`-license_list` *string*	Specifies the list or order of licenses the software should use for checking out licenses during multiple-cpu processing. The parameter does not support an empty license list {}. You must specify at least one license from the following list: enccpu, nru, edsl, and edsxl.
`-local_cpu` *string*	Specifies the number of CPUs on the local machine . This parameter is required for multi-threading. *Default:* `1` When `set_multi_cpu_usage - localcpu 1` command is specified, multi-threaded timing propagation is enabled. Also, both CPU multi-threading and distributed process are enabled.
`-release_license`	Releases all multiple-CPU license(s) immediately. By default, the software holds multiple-CPU licenses until the end of the current session. To specify that the software release multiple-CPU licenses after every multiple-CPU command runs, use -keep_license.
`-remote_host` *integer*	Specifies the number of remote machines. This parameter is required for distributed processing and Superthreading. *Default:* `0`
`-reset`	Resets the specified parameters back to default values.

`-thread_info {0 | 1 | 2}`	Reports usage information. This parameter works for multi-threading only. *Default:* `0` Specify one of the following values:
	0: Does not write messages to the log file.
	1: Writes the final message to the log file. For example, All threaded jobs finished (10 elapsed sec: 0 processor sec (parent), 0 system sec (parent), 0 processor sec (threads), 0 system sec (threads)).

	2: Writes additional starting/ending information for each thread.
	For example,
	`Starting threaded job 1...`
	`Starting threaded job 2...`
	`Starting threaded job 3...`
	`Ending threaded job 2 (1 elapsed sec, 0 processor sec, 0 system sec, 1.160M).`
	`Ending threaded job 3 (2 elapsed sec, 0 processor sec, 0 system sec, 0.895M).`
	`Ending threaded job 1 (10 elapsed sec, 0 processor sec, 0 system sec, 1.172M).`
	`All threaded jobs finished (10 elapsed sec: 0 processor sec (parent), 0 system sec (parent), 0 processor sec (threads), 0 system sec (threads)).`
	Note: This parameter works in multi-threading mode only.

`-verbose`	Displays messages when changing the multiple-CPU settings.

Examples

To run attribute placement with four threads, specify the following commands:

```
set_multi_cpu_usage -local_cpu 4
place_design
```

To run attribute placement with the maximum number of available threads, specify the following commands:

```
set_multi_cpu_usage -local_cpu max
place_design
```

To run detailed routing with the NanoRoute router in Superthreading mode with two machines and six threads, using an LSF queue, specify the following commands:

```
set_distribute_host -lsf -queue myLSFqueue
set_multi_cpu_usage -remote_host 2 -cpu_per_remote_host 6
route_detail
```

12

OpenAccess Commands and Attributes

- compare_oa_cellview
- create_oa_lib
- create_oa_lib_property
- get_oa_default_rule_lib
- get_oa_design_data
- oa_read_write Category Attributes
- read_oa
- register_trigger
- unlock_oa_design
- update_oa_lib
- write_lef_library
- write_oa_techfile

compare_oa_cellview

```
compare_oa_cellview
[-help]
[-ignore {pin_purpose}]
[-out_file report_file_name]
{ {{-source {from_lib from_cell from_view} -compare {to_lib to_cell to_view}} | {-
lib lib -cell cell -view {sourceView compareView}}} }
```

Compares specified cell views. Use this command to compare two OpenAccess databases. For instance, you can use this command to compare the lib/cell/view specified using `-source` to the lib/cell/view specified with `-compare`. In an abstract to layout comparison, boundary and terminals (name, direction, pin location and size) are compared.

Parameters

`-help`	Prints a brief description that includes type and default information for each `compare_oa_cellview` parameter. For a detailed description of the command and all of its parameters, use the man command: `man compare_oa_cellview`.
`-cell cell`	Specifies the cell for comparing two views. *Data_type*: string, optional
`-compare {`*`to_lib to_cell to_view`*`}`	Specifies the library/cell/view that is to be compared. *Data_type*: string, optional
`-ignore {pin_purpose}`	Specifies the list of attributes to be excluded from the cell comparison. As of now, the only attribute that can be ignored is the purpose, which is specified using the value `pin_purpose` with `-ignore`. *Data_type*: (enum)+, optional
`-lib` *`lib`*	Specifies the library for comparing two views. *Data_type*: string, optional
`-out_file` *`report_file_name`*	Specifies the report file in which all the comparison detail is to be written. *Data_type*: string, optional
`-source {`*`from_lib from_cell from_view`*`}`	Specifies the source library, cell, and view. *Data_type*: string, optional
`-view {`*`sourceView compareView`*`}`	Specifies different views of the same library/cell. *Data_type*: string, optional

Example

- The following command compares two views of the same cell that are present in the same library `myLib`:

```
compare_oa_cellview -source {myLib block1 layout} -compare {myLib block1 abstract}
```

create_oa_lib

```
create_oa_lib
[-help]
oa_lib
[-out_dir path ]
[-no_compress]
{-oa_attach_tech_lib techlib |
-oa_copy_tech_lib techlib |
-oa_reference_tech_libs techliblist}
```

Copies or attaches a technology database to a library. If the library exists, this command does not update the library.

Note: The parameters must appear in the order specified by the syntax.

The `create_oa_lib` command is part of the OpenAccess design flow. Use this command to create an OpenAccess library at the specified path. `create_oa_lib` needs to be called only once before the design is saved. Subsequent `write_db` calls to the same library will automatically find out the library path that gets stored in `cds.lib`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_oa_lib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_oa_lib`.
`oa_lib`	Specifies the name of a library to be created. The software creates the library and adds its name to the `cds.lib` file.
`-out_dir` *`path`*	Specifies the name of a directory in which to save the library. The path you specify must include the directory that will become the library itself. *Default:* `./` *`oa_lib`* (Saves the library to the current working directory)
`-no_compress`	Creates a new library without any compression irrespective of the compression settings provided by the `oa_new_lib_compress_level` attribute.
`-oa_attach_tech_lib` *`techlib`*	Attaches the technology database specified by the technology library to the library that is to be created.
`-oa_copy_tech_lib` *`techlib`*	Copies the technology database specified by *`techLib`* to the library specified by *`oa_lib`* , and saves the library to the current working directory.
`-oa_reference_tech_libs` *`techliblist`*	Creates a local technology database in the design library that can be modified incrementally.

Examples

- The following command creates the library `myLib` in the current directory, and attaches the technology database in `myRefLib` to `myLib`:
  ```
  create_oa_lib myLib -oa_attach_tech_lib myRefLib
  ```
  Note: Note: If `myRefLib` is read-only, the `write_db -oa_lib_cell_view` command generates an error if it needs to update the technology database.
- The following command creates the library `TEST_LIB` at the specified path, and attaches the technology database in `TEST_REF_LIB` to TEST_LIB:
  ```
  create_oa_lib TEST_LIB -out_dir ./outputs/TEST_LIB -oa_attach_tech_lib TEST_REF_LIB
  ```

- The following command generates an error because only one reference library can be specified:
  ```
  create_oa_lib myLib -oa_attach_tech_lib {myRefLib1 myRefLib2}
  ```
- The following command creates the library `myLib`, and copies the technology database from `myRefLib` to `myLib`. If `myRefLib` is modified, the two technology databases will become out of sync. Additionally, any future write operations from the `write_db -oa_lib_cell_view` command will modify the technology database copy in `myLib` and also cause the databases to be out of sync.
  ```
  create_oa_lib myLib -oa_copy_tech_lib myRefLib
  ```
- The following command creates a new library which references existing technology databases (`refLib1` and `refLib2`), but allows local technology updates if required (typically to add new custom vias to the local techology database).
  ```
  create_oa_lib myLib -oa_reference_tech_libs {refLib1 refLib2}
  ```

create_oa_lib_property

```
  create_oa_lib_property
[-help]
-oa_lib libName
-name prop_name
-type {boolean integer float string}
-value propValue
```

Adds a new property to the OpenAccess library, `oalib`. This command cannot be used to change the value of an existing property.

Note: You cannot use the name `techLibName` for the property you are adding because this name is used by the OpenAccess infrastructure to store the property attached to the library.

Parameters

`-help`	Prints a brief description that includes type and default information for each `create_oa_lib_property` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_oa_lib_property`.
`-oa_lib libName`	Specifies the name of the OpenAccess library to which the property has to be added.
`-name prop_name`	Specifies the name of the property being created.
`-type {boolean integer float string}`	
	Specifies the type of the property being created.
`-value propValue`	Specifies the value to be set for property being created. The value is interpreted from the input string and converted to the specified type.

Example

The he following set of commands creates a library named *myLib*, adds the *prop1* property to it, and saves the design before exiting:

```
create_oa_lib myLib -oa_attach_tech_lib adiTech

create_oa_lib_property -oa_lib myLib -name prop1 -type string -value value1
```

```
write_db -oa_lib_cell_view {myLib block1 layout}

exit
```

get_oa_default_rule_lib

```
get_oa_default_rule_lib
[-help]
[-rule rule_name]
[-libs lib_list]
[-verbose]
```

Searches through a specified library (or libraries) to check whether a specified LDRS exists or not. You can run this check before reading a library.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_oa_default_rule_lib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_oa_default_rule_lib`
`-libs` *`lib_list`*	Specifies the list of library names to search through. If not specified, all libraries are searched.

`-rule` *`rule_name`*	Specifies the name of the rule to find.
	
	If not specified, the `init_oa_default_rule` attribute is used. If the attribute has an empty string value, the standard lookup (`LEFDefaultRouteSpec` by type, then by name) is used.
	
	If the specified *`rule_name`* does not exist, the command displays an error such as:
	`Rule '<rule_name>' was not found in specified libraries.`
	
	If the specified *`rule_name`* is found, the command returns the name of the library where the specified *`rule_name`* was found. This could be from the `-libs` *`lib_list`* or any ITDB that one of those libraries refers to.
	
	If the specified *`rule_name`* is found but is not a legal LDRS type as determined by the OAX reader code, the command displays an error such as:
	`Constraint group '<rule_name>' was found in library '<lib_name>', but is not a legal LEFDefaultRouteSpec`

`-verbose`	Echoes details of library name where the rule was found (including descending through technology references from libraries specified in the *`lib_list`*.

Examples

- The following example shows how `get_oa_default_rule_lib` can be used to search for a specific library and the graph of techs that it references:

```
set_db init_oa_ref_libs "tech_lib macro_lib"
set_db init_oa_default_rule LDRS_65nm_6lm
set tech_lib [lindex $init_oa_ref_libs 0]
set lib_found [get_oa_default_rule_lib -libs $tech_lib] # use the
init_oa_default_rule to look up the library
if {$lib_found != ""} {
Puts "$init_oa_default_rule was found in $lib_found or in the graph of libraries that
$tech_lib references"
} else {
Puts "$init_oa_default_rule was not found in $tech_lib nor in the graph of libraries
that $tech_lib references"
}
```

- The following command shows how the `-verbose` option can be used:

```
get_oa_default_rule_lib -verbose
INFO: Default rule 'LEFDefaultRouteSpec' was found in library 'FEOAreflib8'.
FEOAreflib8
```

get_oa_design_data

```
get_oa_design_data
[-help]
{{-all_oa_libs} | {{-design_data dd_objHandle | -oa_lib libName | -oa_cell {lib cell} | -
oa_cellview {lib cell view}}
[-children | -file | -path | -type |-name]}}
```

Returns OpenAccess library information.

Parameters

`-all_oa_libs`	Returns a list of design data (`dd`) object handles for the list of libraries present in the library definition file (`cds.lib`).
`-children`	Returns the `dd` object handles of the children of the specified `dd` object handle. Children of libraries are cells. Children of cells are views.
`-design_data dd_objHandle`	Specifies the `dd` object to be processed.
`-file`	Returns the `dd` object handles of all the files in the specified `dd` object handle.
`-name`	Returns the name of the specified `dd` object handle.
`-oa_cell {lib cell}`	Specifies the names of the library and cell to be processed.
`-oa_cellview {lib cell view}`	Specifies the names of the library, cell, and view to be processed.
`-oa_lib libName`	Specifies the name of the library to be processed.
`-path`	Returns the absolute path name (`dd` path) of the specified `dd` object handle.
`-type`	Returns the type of the specified `dd` object handle.

Examples

- The following command returns the path of the specified library/cell/view as shown below:

  ```
  get_oa_design_data -cellview {pdkLib cell1 abstract} -path
  ```

  returns:

  ```
  /libs/pkdLib/cell1/abstract
  ```

  This command is equivalent to the following SKILL function:

  ```
  ddGetObjReadPath(ddGetObj("pdkLib" "cell1" "abstract"))
  ```

- The following proc traverses the cds.lib structure (expanding #INCLUDE as required) to get the complete list of OpenAccess libraries and their full paths.

  ```
  # expand #INCLUDE as needed and print all libraries and their full paths
  proc print_all_lib_paths {} {
     Puts [format "%16s -> %s" "Library Name" "Library Path"]
     Puts "-------------------------------------------------------------------"
     foreach oa_lib [get_oa_design_data -all_oa_libs] {
         Puts [format "%16s -> %s" [get_oa_design_data -design_data $oa_lib -name]
  [get_oa_design_data -design_data $oa_lib -path]]
     }
  }
  ```

oa_read_write Category Attributes

The root attributes in the `oa_read_write` category are:

- oa_allow_analysis_only
- oa_allow_bit_connection
- oa_allow_tech_update
- oa_bindkey_file
- oa_cell_view_dir
- oa_convert_diagonal_path_to
- oa_cut_rows
- oa_display_resource_file
- oa_display_resource_file_in_library
- oa_drc_fill_purpose
- oa_enclose_quick_abstract_pins
- oa_full_layer_list
- oa_full_path
- oa_inst_placed_if_none
- oa_lib_create_mode
- oa_lock
- oa_logic_only_import
- oa_new_lib_compress_level
- oa_pin_purpose
- oa_push_pin_constraint
- oa_quick_abstract_for_custom_pcells
- oa_silently_ignore_unsupported_vias
- oa_text_purpose
- oa_tie_net
- oa_update_mode

- oa_use_virtuoso_bindkey
- oa_use_virtuoso_color
- oa_view_sub_type
- oa_write_mask_data_locked
- oa_write_net_voltage
- oa_write_relative_path

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `oa_read_write` category, use the following command:

```
get_db -category oa_read_write
```

oa_allow_analysis_only

```
oa_allow_analysis_only {true | false}
```

Default : `false`

Allows analysis only flow.

`assemble_design` is not allowed when `oa_allow_analysis_only` is set to `false` and `oa_update_mode` is set to `true`, thereby locking the OpenAccess databse. To allow `assemble_design` to proceed, you must change `oa_allow_analysis_only` to `true`. If you set `oa_allow_analysis_only` to `true` and the the software session is in `oa_update_mode` = `true` mode (whether set explicitly or internally on the fly), the `oa_update_mode` setting is ignored.

Note that there may be loss of some semantic information, such as MPPs, while saving the design when `oa_allow_analysis_only` is set to `true`. However, the saved design would be geometrically correct.

oa_allow_bit_connection

```
oa_allow_bit_connection {true | false}
```

Default: `false`

Creates databases for instances that only have bit-based terminals, without requiring the

corresponding bus terminals.

When set to `true`, it saves the connectivity information by bits if the bus connections are not available.

When set to `false`, an error will be issued if the abstract view is missing expected bus terminal information.

oa_allow_tech_update

```
oa_allow_tech_update {true | false}
```

Default: `true`

Allows the technology database to be updated. The common reason that a technology database would need to be updated is that new vias have been added to the design that are not in a parameterized form. Via masters that are a simple list of rectangles require that a "custom via" be created in the technology database, If the design library had been created with an attached technology, the value should be set to `false` allowing a reference library technology that is shared by multiple designs to have design specific vias to be added to it. For design libraries that have been created in reference mode, allowing the technology database to be updated is typically fine but normally it is best to always use paramaterized vias for OpenAccess-based flows.

Using `set_db generate_special_via_symmetrical_via_only true` before any power planning or routing will typically prevent the creation of non-parameterized vias in the flow.

When `oa_allow_tech_update` is set to `false`, the creation of the OpenAccess database will fail if any custom vias would need to be created within the design library.

Related Commands and Attributes

`generate_special_via_symmetrical_via_only`

oa_bindkey_file

```
oa_bindkey_file fileName
```

Default: `""`

Maps into the software equivalent bindkeys. If the bindkey file is not specified, the Virtuoso default bindkey information is used. This attribute must be used with the `oa_use_virtuoso_bindkey` attribute.

Related Commands and Attributes

`oa_use_virtuoso_bindkey`

oa_cell_view_dir

`oa_cell_view_dir` *string*

Default: ""

Specifies the path name to the physical location of the lib/cell/view that was restored using `read_db -oa_lib_cell_view`. `oa_cell_view_dir` is a system-defined, transient (non-persistent) attribute used in the OpenAccess flow. This should be treated as a *read-only* attribute.

Related Commands

`read_db`

oa_convert_diagonal_path_to

`oa_convert_diagonal_path_to {polygon | default_style}`

Default: `polygon`

Enables interoperability of the end style of geometries of type path segment between Virtuoso and the software.

This attribute applies to the diagonal path segments only.

If the end-style of a diagonal path segment is `custom`, then the end style is retained as is.

If the end style of a diagonal path segment is not `custom`, then if you set the value to `polygon`, The software converts the path segment into a geometrically equivalent polygon.

If the value specified is `default_style`, then the end style of the path segment is set to `default_style`. In this case, the software retains the semantics of the object to be a path segment.

> To ensure that there is no loss of semantic or geometric information, the path segments in Virtuoso -XL and Virtuoso-GXL should use the `custom` end-style on diagonal path segments which can be created using *Create-Shape-Geometric Wire*.

oa_cut_rows

`oa_cut_rows {true | false}`

Default: `false`

Cuts rows against placement obstructions, block halos, or power domain fence minimum gaps.
The following command cuts rows against placement obstructions, block halos, or power domain fence minimum gaps:

```
set_db oa_cut_rows true
```

oa_display_resource_file

`oa_display_resource_file` *pathName*

Default: `""`
Allows you to specify a specific `display.drf` file that is to be used. It overrides all the searching options and the specified file is used.
The following command searches for the `mydisplay.drf` file irrespective of the value of the `oa_display_resource_file_in_library` attribute:

```
set_db oa_display_resource_file /servers/oastore/mydisplay.drf
```

oa_display_resource_file_in_library

`oa_display_resource_file_in_library {true | false}`

Default: `false`

Allows the tool to search for the `display.drf` file in the technology library (or complete technology graph in case of ITDB) when set to `true`. If not found, it looks for the file in the current working directory followed by the home directory.

When set to `false`, the tool searches for the `display.drf` file first in the current working directory followed by the home directory.

oa_drc_fill_purpose

`oa_drc_fill_purpose {gap_fill | drawing}`

Default: `gap_fill`

Specifies whether to save DRCFILL shapes to the `gap_fill` or `drawing` purpose. By default, DEF routes with + SHAPE DRCFILL are mapped to the `gapFill` purpose in OpenAccess. However, some OpenAccess tech files do not have this purpose defined so any shapes mapped to the `gapFill` purpose would not be visible in Virtuoso and would not be written out in GDS or OASIS. In such cases, set the `oa_drc_fill_purpose` attribute to `drawing` to save these shapes to the `drawing` purpose. When you do so, these shapes get an additional internal attribute so they can round-trip back to DRCFILL shapes properly.

oa_enclose_quick_abstract_pins

`oa_enclose_quick_abstract_pins {true | false}`

Default: `true`

Data_type: bool

Specifies that shapes enclosing the layout pin shapes should be included as part of the pins during Quick Abstract Inference (QAI) pin modeling. Check the "Quick Abstract Inference" chapter in the *Mixed Signal (MS) Interoperability Guide* for details.

oa_full_layer_list

```
oa_full_layer_list {true | false}
```

Default: `true`

Imports the complete list of OpenAccess layers, when importing an OpenAccess design.

If you specify `false`, only OpenAccess layers defined with `oacMaterial` and type are imported.

oa_full_path

```
oa_full_path {true | false}
```

Default: `false`

Creates the `DEFINE` statements in the `lib.def` files using full paths. If you specify `-oa_full_path false`, the software creates the statements using relative paths.

The following command instructs the software to create the `DEFINE` statements using full paths:
`set_db oa_full_path true`

oa_inst_placed_if_none

```
oa_inst_placed_if_none {true | false}
```

Default: `true`

Maps the placement status of instances in the OpenAccess database to the placement status in the software. When set to `true`, the instances in the OpenAccess database that have an unknown placement status are treated as "placed" in the software. When set to `false`, instances with an unknown placement status are treated as "unplaced."

oa_lib_create_mode

```
oa_lib_create_mode {none | attach | copy | reference}
```

Default: `reference`

Specifies the mode in which `write_db -oa_lib_cell_view` creates a library.

- `none`: Turns off the ability to have `write_db` create a library if it does not already exist.
- `attach`: Attaches the technology database specified by the technology library to the library that is to be created.
- `copy`: Copies the technology database specified by `techLib` to the library specified by `libName`, and saves the library to the current working directory.
- `reference`: Creates a local technology database in the design library that can be modified incrementally. This is the recommended setting to handle the case where the design may have "custom vias" added at some point by power planning/routing.

Related Commands

- `write_db`

oa_lock

```
oa_lock {true | false}
```

Default: `false`

Locks the OpenAccess database when restoring the design. When set to `true`, other tools can still access the OpenAccess database in read-only mode, but cannot edit the database as long as it is locked for the software session. To remove the lock, specify the `reset_design` command, or exit the software.

Notes

- If you set the `oa_update_mode` attribute to `true`, the software automatically locks the database, even if the `oa_lock` attribute is set to `false`.
- The value of `oa_lock` cannot be changed while a design is in memory.

Related Commands

- `unlock_oa_design`

oa_logic_only_import

`oa_logic_only_import {true | false}`

Default: `false`

Controls what information the software reads when importing an OpenAccess maskLayout database.

When set to `true`, the software reads only the logical (Verilog netlist equivalent) connectivity during the `init_design` process. When set to `false` (the default value), the software automatically reads physical information, such as floorplan, placement, and routing, from the database, and updates physical net connectivity for power and ground connections.

Note: The `oa_logic_only_import` attribute applies only to OpenAccess maskLayout databases. For other types of OpenAccess databases, the software always reads only the logical connectivity.

Related Commands

- `init_design`

oa_new_lib_compress_level

`oa_new_lib_compress_level` *`value`*

Default: `1`

Sets the compression level of a library during the library creation process. Cellviews written to a library will follow the compression level of the library and not the current `oa_new_lib_compress_level` setting.

Valid values range between 0 to 9. A value of 0 indicates that compression is turned off.

Related Commands

- `create_oa_lib`

oa_pin_purpose

`oa_pin_purpose {true | false}`

Default: `false`

Writes out pin shapes with the purpose `pin`. When set to `false`, writes out pin shapes with the purpose `drawing`.

oa_push_pin_constraint

```
oa_push_pin_constraint {true | false}
```

Default: `true`

Specifies whether or not `write_db` or `write_oa_black_boxes` should push an interoperable pin constraint into its corresponding blackbox abstract cellview.

Note: Interoperable pin constraints refer to pin constraints that are understood by the software, Virtuoso, and PVS.

Related Commands

`write_db`

`write_oa_black_boxes`

oa_quick_abstract_for_custom_pcells

```
oa_quick_abstract_for_custom_pcells {true | false}
```

Default: `true`

Data_type: bool, read/write

Trims design shapes of custom parameterized cells (PCells). This capability is on by default. You can turn off this capability, if required, by setting this attribute to `false`.

Note: This feature needs the INVS30 (Innovus Mixed Signal Option) license. If this license is not available, the abstract that gets generated for custom PCells will not work properly.

oa_silently_ignore_unsupported_vias

```
oa_silently_ignore_unsupported_vias {true | false}
```

Default: `false`

Specifies whether `read_db` should issue messages about vias that are not supported in the software when a Virtuoso design is read in.

oa_text_purpose

```
oa_text_purpose purpose_name
```

Default: `drawing`

Specifies the purpose name that will be used when text labels are written.

By default, any text label created in the software is automatically mapped to the `drawing` purpose. Use the `oa_text_purpose` attribute to map the text labels to a user-defined purpose. **Note**: `oa_text_purpose` *`purpose_name`* overrides the default `drawing` purpose on output.

oa_tie_net

`oa_tie_net {`*`tieHighNetName tieLowNetName`*`}`

Default: `tie1` and `tie0`. If these names exist in the design locally in any module, then the global names should be changed.

Specifies the global tie high and tie low net names to be created in the OpenAccess database. The nets are used to represent the Verilog `1'b1/1'b0` connections to the instance ports.

oa_update_mode

`oa_update_mode {true | false}`

Default: false

Ensures that `read_db -oa_lib_cell_view` processes parameterized cells (PCells) and other custom objects correctly in the database. For information about generating/regenerating the PCell cache, refer to the "Preparing the IP Library" section in the Design Data and Technology Data Preparation chapter of the *Mixed Signal (MS) Interoperability Guide*.

The `oa_update_mode` attribute also ensures that `write_db -oa_lib_cell_view` updates the software-modified portion of an OpenAccess database without making changes to PCells or other custom objects.

This attribute provides the ability to import OpenAccess database created from Virtuoso that contains objects which are not understood by the software. Complete the digital part in the software and update only the objects modified by the software in the original OpenAccess database. If your design has:

- Pcells
- Modgen cells
- Fig-groups
- Pcell vias
- RODs (Example, Multi-Part Paths, and so on)

- User defined properties on design objects such as nets, shapes (`lxStickyNet`), instances, abstract views, intermediate hierarchical instances, and so on.

All such objects would only be visible in the software. You won't be allowed to edit or modify these objects.

On using the `oa_update_mode` attribute, the property on shapes (for example, `lxStickyNet`) and nets created in VLS-XL / GXL would be kept as-is.

In case of designs where parameterized vias are used, the software understands these attributes and evaluates them on-the-fly to create an exact shape and orientation of the cuts.

Notes

- The value of the `oa_update_mode` attribute cannot be changed while a design is in memory. After this attribute has been set to `true` before importing an OpenAccess design into the software, you cannot convert the `oa_update_mode` attribute to `false` in any new software session.
- The on-disk OpenAccess cellview is locked between the restore and save steps when the `oa_update_mode` attribute is set to `true`.

oa_use_virtuoso_bindkey

```
oa_use_virtuoso_bindkey {true | false}
```

Default: `false`

Allows you to use Virtuoso bindkeys.

oa_use_virtuoso_color

```
oa_use_virtuoso_color  {true | false}
```

Default: `false`

Allows you to display colors and stipples for the design layers (including full custom layers). Using this attribute, the software utilizes the same colors and stipples that you would see if you open the design in Virtuoso.

oa_view_sub_type

```
oa_view_sub_type {vce | vxl | none}
```

Default: `vxl`

Indicates which Virtuoso application should be used for viewing the layout cellviews in the saved OpenAccess database.

When set to `vce`, cellviews are opened in Virtuoso Chip Editor.

When set to `vxl`, cellviews are opened in Virtuoso XL.

When set to `none`, it indicates that the OpenAccess database can be edited by any editor within Virtuoso platform based on its license availability.

oa_write_mask_data_locked

```
oa_write_mask_data_locked {true | false}
```

Default: `false`

Sets the shapes with mask coloring (mask values other than 0) to locked in the saved cellview,when set to `true`.

By default, `write_oa` and `write_db` write out shapes with their mask coloring information in an unlocked state. Some tools treat the unlocked color information as the recommended color. However, for some foundry processes, shapes with unlocked mask coloring information are written to GDSII/Stream as gray (uncolored). In such cases, set the `oa_write_mask_data_locked` parameter to `true` to lock the mask coloring of shapes explicitly.

Note: This option affects only the saving of shapes with non-zero mask coloring values. Review the map file that will be used to create GDSII/Stream output from the OpenAccess cellview to determine if the map file requires shapes to be locked for them to be written out as colored.

oa_write_net_voltage

```
oa_write_net_voltage {true | false}
```

Default: `false`

Saves the minimum and maximum net voltage across all loaded timing views to every net. This is useful for Virtuoso, and other tools that may read the cellview, to understand the minimum and maximum net voltage without needing to interpret the CPF or .lib file to get that information. Roundtrip of the information into the software is not dependent on this setting.

oa_write_relative_path

```
oa_write_relative_path {true | false}
```

Default: `true`

Defines the current working directory (`$cwd`) in the configuration file as ".". When set to `true`, a design can be restored, even if the directory path of the design file is changed.

read_oa

```
read_oa
[-help]
oa_lib
oa_cell
oa_view
[-filter {block_insts blockages boundary fixed_core_insts floorplan pad_insts pin_shapes
regions regular_routing special_routing}]
[-nets { netnames }]
```

Restores floorplan, placement, and routing data using the OpenAccess format. This command only reads physical information from the OpenAccess database. For information on restoring an OpenAccess database, see read_db.

You can use the `read_oa` command after importing the design.

Note: The software supports only the following begin/end extension values for DEF `NETS` style wiring segments: 0, width/2 or LEF `LAYER WIREEXTENSION`. If any of the wire extensions has unsupported extension value, then the software converts it to a special wire (DEF `SPECIALNETS`). While writing back OpenAccess (`write_oa` or `write_db -oa_lib_cell_view`), these special wires will again be converted back to DEF `NETS` style wiring.

Parameters

`-help`	Outputs a brief description that includes type and default information for each parameter of the `read_oa` command. For a detailed description of the command and all of its parameters, use the `man` command: `man read_oa`.
`oa_lib`	Specifies the OpenAccess library name.
`oa_cell`	Specifies the name of the top cell to import.
`oa_view`	Specifies cellview name to import.
`-filter {block_insts blockages boundary fixed_core_insts floorplan pad_insts pin_shapes regions regular_routing special_routing}`	

Lists the OpenAccess cellview information data to be read.

When not specified, all information is read from the OpenAccess cellview. When `-filter` is specified, only the object types that are chosen will be read. Additionally, when reading pin_shapes or instances, information in the cellview that refers to terminals or instances that do not exist in memory will be ignored.

The legal `-filter` options are:

- `block_insts` - Indicates that any CLASS BLOCK instances should be updated. Physical-only block instances may be added to the database if the placement status is `fixed` or `cover`.
- `blockages` - Indicates that blockages should be processed. If a blockage is attached to an instance that does not exist in the in-memory database, then it is ignored.
- `boundary` - Indicates that the design boundary information, including rows and tracks, should be updated.
- `fixed_core_insts` - Indicates that any CLASS CORE instances that have placement status `fixed` or `cover` should be updated. Physical-only core instances may be added to the database if the placement status is `fixed` or `cover`.
- `floorplan` - Is equivalent to specifying "`block_insts blockages boundary fixed_core_insts pad_insts pin_shapes regions special_routing`"
- `pad_insts` - Indicates that any CLASS PAD instances should be updated. Physical-only pad instances may be added to the database if the placement status is `fixed` or `cover`.
- `pin_shapes` - Indicates that pin shapes should be read.
- `regions` - Indicates regions should be updated.
- `regular_routing` - Indicates that nets should be processed and regular routing (and associated shield net wiring) and net constraints updated.
- `special_routing` - Indicates that nets should be processed and special routing (except shield nets wiring) should be read.

Use `read_oa -filter` to read floorplanning information from a cellview whose connectivity does not match the current in-memory connectivity. The `-filter` option allows you to load some data selectively, while not loading others.

	When `-filter` is used, error checking for database consistency (nets, instances, terminals, etc.) is disabled. While similar to Virtuoso Load Physical View (LPV), the `-filter` option is only a subset and not intended to be a complete replacement.
`-nets {`*`netnames`*`}`	
	Specifies the list of nets whose routing (routing and special) and constraints have to be read with the `-filter` option. Shielding wiring/vias are also read in if they are used to shield any of the specified nets. If this option is used, typically `regular_routing` is not specified as part of the `-filter` objects.

Example

- The following command restores data using OpenAccess:

```
read_oa design amba_usb layout
```

register_trigger

```
register_trigger
[-help]
[{-pre | -post}
command_name
proc_name]
```

Registers the `-pre` or `-post` callback procedures with the specified commands.

The procedure registered with `-pre` parameter runs before the *`command_name`* parameter is used while the procedure registered with `-post` parameter runs after the *`command_name`* parameter is used.

Note: The arguments passed to *`command_name`* are passed to *`proc_name`*. If the procedure registered with the `-pre` parameter returns a `TCL_ERROR`, then *`command_name`* is not run.

You can use the `register_trigger` command after starting the software.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `register_trigger` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man register_trigger`.
`-pre`	Specifies the procedure to be called before the command.
`-post`	Specifies the procedure to be called after the command.
`command_name`	Specifies the command name. The supported command names are `read_db`, `write_db`, `read_oa`, and `write_oa`.
`proc_name`	Specifies the procedure name that is to be registered as callback.

unlock_oa_design

```
unlock_oa_design
[-help]
oa_lib
oa_cell
oa_view
```

Unlocks the specified OpenAccess design.

A design cannot be opened if `set_db oa_lock` is set to `1`, and the design is currently locked. You can use the `unlock_oa_design` command to free the lock so that other tools can access and edit the database. You also can use the command to override the lock, if the design is incorrectly locked.

You can use the `unlock_oa_design` command at any time.

Parameters

oa_cell	Specifies the name of the top oa_cell to unlock.
-help	Outputs a brief description that includes type and default information for each `unlock_oa_design` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unlock_oa_design`.
oa_lib	Specifies the name of the OpenAccess oa_library in which the design was saved.
oa_view	Specifies the name of the oa_cell oa_view to unlock.

Example

- The following command unlocks the oa_view `layout` in the oa_cell `dtmf_chip` in the oa_library `designLib`:

  ```
  unlock_oa_design designLib dtmf_chip layout
  ```

update_oa_lib

```
update_oa_lib
[-help]
lib_name
[-compress_level value]
[-create_cdsinfo_tag]
[-tech_attach_to_reference]
```

Updates the specified OpenAccess library from within Innovus. You can use this command to:

- Converts the specified library to use a tech reference from the attached tech so that the non-parameterized or custom vias can be written to it. If an OpenAccess design has non-parameterized vias, you should run `update_oa_lib` before invoking `write_db`. The tool is then able to save the design and does not throw an error because it can now write non-parameterized vias to the converted library.
- Add a cdsinfo.tag to legacy libraries created by earlier versions of Innovus.
- Modify the compression level for the library.

If you run the `update_oa_lib` command on a library that is not in attached mode, a warning is displayed. However, the script continues to run. This is useful if you want to make sure that libraries are writable for new vias and the `write_db` command does not fail.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `update_oa_lib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_oa_lib`.
`-compress_level` *value*	Sets the compression level of the specified library. Cellviews written to the library follow the specified compression level. However, existing cellviews in the library are not modified. Use the oazip utility to modify the existing cellviews in the library. Valid compression level values range from 0 to 9. A value of 0 indicates that compression is turned off.
`-create_cdsinfo_tag`	Adds a `cdsinfo.tag` file to the specified library.
lib_name	Specifies the name of the library to be updated.
`-tech_attach_to_reference`	Converts the specified library to use a tech reference from the attached tech so that new custom via definitions can be added to the library.

Examples

- The following command sets the compression level of the `lib1` library to 9. However, it does not change the compression of the existing cellviews in the library:
  ```
  update_oa_lib lib1 -compress_level 9
  ```
  The tool displays warning such as the following:
  ```
  Changed library compressLevel from '0 (uncompressed)' to '9'. Cellviews within the library are not affected by changing the library's compressLevel. To update the cellviews, use the oazip utility.
  ```
- The following command decompresses the `my_lib` library, which previously is at compression level `1`:
  ```
  update_oa_lib my_lib -compress_level 0
  ```

write_lef_library

```
write_lef_library
[-help]
[lef_file]
[-tech_only | -macro_only]
[-tech_only | -macro_list string]
```

Writes the LEF library data (composed of technology data and macros) that is present in the database.

The `write_lef_library` command facilitates comparisons between LEF-based input and OpenAccess-based input. LEF allows cases where `VIARULE GENERATE` has `WIDTH` specified for either the top or bottom layer. However, such cases are not supported by the OpenAccess tech. To make the comparison between LEF-based and OpenAccess-based input easier, `write_lef_library` dumps `"WIDTH 0 TO 1000 ;"` for any `VIARULE GENERATE` that does not have WIDTH specified in the original LEF.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_lef_library` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_lef_library`.
`lef_file`	Specifies the output LEF file name. *Data_type*: string, optional
`-macro_list` *`string`*	
	Writes the macros specified in the macro list. *Data_type*: string, optional
`-macro_only`	Writes only the macro data to the LEF file. *Data_type*: bool, optional
`-tech_only`	Writes only the technology data to the LEF file. *Data_type*: bool, optional

Examples

- The following command writes the technology data into the `tech.lef` file:

  ```
  write_lef_library tech.lef -tech_only
  ```

- You can set the LEF version to 5.7 or 5.8 using `write_def_lef_out_version`:

  ```
  set write_def_lef_out_version 5.8
  ```

write_oa_techfile

```
write_oa_techfile
[-help]
[out_file]
[-include_rules]
[-process_node coord]
```

Writes out a DFII style ASCII technology file that can be given to the `techLoadDump` executable to create an equivalent OpenAccess technology database.

This command is used to create and verify OpenAccess technology files that match LEF technology files. By default, `write_oa_techfile` writes out only DRC information and no LDRS or VIA information.

The expected flow for new processes is as follows:

1. Read a LEF file into the software with `init_design`.
2. Use `write_oa_techfile` to write out the tech (`.tf`) file.
3. Run `techLoadDump` using the `.tf` file.
4. Use `lef2oa -pnrLibDataOnly` to create the LDRS and vias.

For more information, refer to the chapter "Technology File Layer Attributes" of the *Virtuoso Technology Data ASCII Files Reference*.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `write_oa_techfile` parameter. For a detailed description of the command and all of its parameters, use the man command: man `write_oa_techfile`.
`out_file`	Specifies the output ASCII file name.
`-include_rules`	Includes LDRS and VIA information in the output tech file. This option should be used only if the LEF default and NDRs do not utilize fixed vias.
`-process_node` *`coord`*	Specifies a process node value in nm. When specified, `write_oa_techfile` creates a processNode entry in the techfile for processes using the MASK construct. For example, if you specify `-process_node 20` to indicate a 20nm process, the following entry is included in the control() section of the techfile: `processNode(0.020)` If `-process_node` is not specified and the `write_oa_techfile` command detects MASK constructs on any layer, it returns an error.

Examples

- The following set of commands updates an OpenAccess technology file with any missing information from a LEF technology file:

```
#Initialize a design using LEF files
write_lef_library -tech_only lef.lef  #A LEF file in canonical
                                      #sorted order for easy
                                      #comparison with diff
write_oa_techfile lef.tf              #A DFII ASCII tech file
                                      #that can be given to
                                      #techLoadDump executable
#Edit existing OA tech file to add any missing information in
#the lef.tf file
#Initialize a design using the new OA tech. Then:
write_lef_library -tech_only oa.lef #A LEF file in canonical
                                    #order derived from OA tech
#Compare the two LEF files. Any differences need to be fixed
#or explained.
diff lef.lef oa.lef
```

13

Power Calculation Commands and Attributes

- encrypt_thermal_file
- extract_metal_density
- get_default_switching_activity
- get_dynamic_power_simulation
- merge_vtm
- power Category Attributes
- propagate_activity
- query_power_data
- read_activity_file
- read_activity_map_file
- report_annotated_parasitics
- read_power_db
- report_inst_power
- report_power
- report_thermal
- report_unannotated_nets
- reset_power_activity
- set_default_switching_activity
- set_dynamic_power_simulation
- set_instance_temperature_file
- set_metal_density_options
- set_power
- set_power_analysis_temperature
- set_power_include_file
- set_power_output_dir
- set_switching_activity
- set_thermal_analysis_mode
- set_twf_attribute
- set_virtual_clock_network_parameters
- write_power_constraints

- write_tcf
- write_thermal_model

encrypt_thermal_file

```
encrypt_thermal_file
[-help]
[-input ASCII.txt]
[-output $encrypted_conductivity.txt]
```

Specifies to encrypt the file containing the thermal conductivity value for layers, and to protect foundry's thermal conductivity parameters used for power map generation. This encrypted file is then specified with the `report_power -thermal_conductivity_inputs` parameter for generating a tile-based power map file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `encrypt_thermal_file` parameter. For a detailed description of the command and all of its parameters, use the man command: man `encrypt_thermal_file`
`-input ASCII.txt`	

	Specifies the name of the ASCII thermal file that needs to be encrypted. This file contains the thermal conductivity parameters used for power map generation.
	The `-input` parameter supports both layer type and layer name based thermal conductivity file.
	The following is a snippet of the input thermal conductivity file content (layer type based file):
	`Metal 430`
	`Silicon 250`
	`Dielectric 0.6`

`-output $encrypted_conductivity.txt`	

	Specifies the name of the encrypted output thermal file.
	The following is a snippet of the encrypted thermal conductivity file content:
	`Metal $encryption_value`
	`Silicon $encryption_value`
	`Dielectric $ecncryption_value`

Examples

- The following command encrypts the specified input file (`format_1.txt`) to generate the encrypted output file (`format_1_encr.txt`):

  ```
  encrypt_thermal_file -input encrypt/format_1.txt -output encrypt/format_1_encr.txt
  ```

  The encrypted file (`format_1_encr.txt`) is then specified for generating a tile-based power map file, as shown below:

  ```
  report_power \
      -out_file                    static.rpt \
      -thermal_power_map_tiles     {5 5}\
      -thermal_power_map_format    stack \
      -thermal_power_map_file      powermap_format1_encr_in.txt \
      -thermal_conductivity_inputs ./encrypt/format_1_encr.txt
  ```

extract_metal_density

```
extract_metal_density
-help
```

Specifies to extract the metal density information. The command generates the Voltus Thermal Model (VTM) for Celsius only if you do not want to include power information. Before specifying this command, use the `set_metal_density_options` command to set the parameters for extracting the metal density information.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.

Examples

- The following command specifies to run extract metal density:
  ```
  extract_metal_density
  ```

Related Topics

- `set_metal_density_options`

get_default_switching_activity

```
get_default_switching_activity
[-help]
[-clip_activity_to_domain_freq]
[-clock_gates_enable]
[-clock_gates_output]
[-clock_gates_output_ratio]
[-combinational_clock_gate_ratio]
[-duty]
[-global_activity]
[-integrated_clock_gate_ratio]
[-input_activity]
[-macro_activity]
[-period]
[-sequential_activity]
```

Displays the following information about a `set_default_switching_activity` parameter in the system log file and in the system console:

- Parameter name
- Current value

If you do not specify a parameter, the software displays information for all of the `set_default_switching_activity` parameters. The `get_default_switching_activity` command prints only the user-specified value for the parameters. If a parameter has not been set using the `set_default_switching_activity` command, the `get_default_switching_activity` command does not output any information in the console.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_default_switching_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_default_switching_activity`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_default_switching_activity` for description of the parameters you can specify.

Examples

The following command displays the current setting of the `-period` parameter:

```
get_default_switching_activity -period
```

The software displays the following information:

```
-period {20}
```

get_dynamic_power_simulation

```
get_dynamic_power_simulation
[-help]
[-activity_pattern]
[-period]
[-resolution]
```

Displays the following information about a `set_dynamic_power_simulation` parameter in the system log file and in the system console:

- Parameter name
- Current value

If you do not specify a parameter, the software displays information for all of the `set_dynamic_power_simulation` parameters. The `get_dynamic_power_simulation` command prints only the user-specified value for the parameters. If a parameter has not been set using the `set_dynamic_power_simulation` command, the `get_dynamic_power_simulation` command does not output any information in the console.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_dynamic_power_simulation` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_dynamic_power_simulation.`
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_dynamic_power_simulation` for description of the parameters you can specify.

Examples

The following command displays the current setting of the `-period` parameter:

```
get_dynamic_power_simulation -period
```

The software displays the following information:

```
-period {20}
```

merge_vtm

```
merge_vtm
[-help]
-config filename
[-rundir directory]
[-merged_file filename]
[-format {simple | metal | stack}]
```

Specifies to import the top-level and other block-level Voltus thermal models (VTMs) having the specified design configuration to generate a merged uniform single VTM for Celsius usage.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-config filename`	

	Specifies the name of the design configuration file.
	The format of the file is:
	`$instance_name $Design_name $X(um),$Y(um) $Cell_orientation $VTM_file format`
	The file includes:
	• instance name
	• design name
	• X and Y locations of the instance
	• LEF/DEF style orientation for the instance
	• path to the thermal map file
	• format of the thermal map file
	Example:
	`octa octa 0,0 N vtm_output_top/top.vtm.txt both`
	`inst_8 super_filter 62.97,47.035 N vtm_output_block/block.vtm.txt both`
	`inst_1 super_filter 1157.09,765.605 FE vtm_output_block/block.vtm.txt both`
	`inst_2 super_filter 777.085,765.605 E vtm_output_block/block.vtm.txt both`
	`inst_3 super_filter 397.085,765.605 W vtm_output_block/block.vtm.txt both`
	`inst_4 super_filter 17.085,765.605 FW vtm_output_block/block.vtm.txt both`
	`inst_5 super_filter 1127.97,47.035 S vtm_output_block/block.vtm.txt both`
	`inst_6 super_filter 772.97,47.035 FS vtm_output_block/block.vtm.txt both`
	`inst_7 super_filter 417.97,47.035 FN vtm_output_block/block.vtm.txt both`

`-format {simple | metal | stack}`	

	Specifies the format of the thermal map file. The possible arguments are: • `simple`: contains only power tables. • `metal`: contains only metal density tables • `stack`: contains both power and metal density tables.
`-merged_file` *`filename`*	
	Specifies the name of the merged VTM.
`-rundir` *`directory`*	
	Specifies the output folder for the merged thermal model.

Examples

- The following command specifies to generate a merged VTM:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_both -merged_file merge_vtm.txt
  ```
- The following command specifies to generate a merged VTM containing only power tables:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_simple -merged_file merge_vtm.txt -format simple
  ```
- The following command specifies to generate a merged VTM containing only metal density tables:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_metal -merged_file merge_vtm.txt -format metal
  ```
- The following command specifies to generate a merged VTM containing both power and metal density tables:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_both_both -merged_file merge_vtm.txt -format stack
  ```

Related Topics

- `set_metal_density_options`

power Category Attributes

The root attributes in the `power` category are:

- power_add_simulation
- power_adjust_input_activity_in_iterations
- power_adjust_macro_activity_in_iterations
- power_analysis_temperature
- power_annotation_detail_report
- power_auto_twf_delay_annotation
- power_average_rise_fall_cap
- power_block_independent_peakpower
- power_bulk_pins
- power_boundary_gate_leakage_file
- power_boundary_gate_leakage_report
- power_boundary_leakage_cell_property_file
- power_boundary_leakage_edge_annotation_file
- power_boundary_leakage_multi_pgpin_support
- power_boundary_leakage_pessimism_removal
- power_boundary_leakage_pxe_support
- power_capacity
- power_case_insensitive_mapping
- power_clock_source_as_clock
- power_comprehensive_automapping
- power_compress_reports
- power_constant_override
- power_corner
- power_create_driver_db
- power_current_generation_method
- power_db_name
- power_decap_cell_list
- power_default_frequency
- power_default_slew
- power_default_supply_voltage
- power_detailed_view_current_only
- power_disable_clock_gate_clipping

- power_disable_leakage_scaling
- power_disable_static
- power_distribute_switching_power
- power_distributed_combine_report_format
- power_distributed_setup
- power_domain_based_clipping
- power_dynamic_glitch_filter
- power_enable_auto_queue
- power_enable_auto_mapping
- power_enable_disk_mapping
- power_enable_duty_propagation_with_global_activity
- power_enable_dynamic_current_slew_load_interpolation
- power_dynamic_scale_clock_by_frequency
- power_dynamic_scale_clock_by_name
- power_dynamic_vectorless_ranking_methods
- power_effective_load_cap_slew_threshold_limit
- power_enable_dynamic_scaling
- power_enable_flop_state_propagation
- power_enable_force_propagation
- power_enable_generated_clock
- power_enable_input_net_power
- power_enable_interactive_reports
- power_enable_mt_state_propagation
- power_enable_multithread_reports
- power_enable_power_target_flow
- power_enable_pba_for_tempus_pi
- power_enable_rtl_dynamic_vector_based
- power_enable_scan_report
- power_enable_single_cycle_scheduling
- power_enable_slew_based_ccs_pin_cap
- power_enable_state_propagation
- power_enable_stable_clock_gating
- power_enable_stable_flop_scheduling
- power_enable_tempus_pi
- power_enable_xp

- power_enhanced_blackbox_avg
- power_enhanced_blackbox_max
- power_equivalent_annotation
- power_event_based_leakage_power
- power_external_load_config_file
- power_extraction_tech_file
- power_extractor_include
- power_fanout_limit
- power_force_library_merging
- power_from_x_transition_factor
- power_from_z_transition_factor
- power_generate_activity_mapping_report
- power_generate_current_for_rail
- power_generate_flop_ranking_data
- power_generate_static_report_from_state_propagation
- power_generate_leakage_power_map_based_on_calculated_leakage
- power_grid_libraries
- power_handle_glitch
- power_handle_transport_glitch
- power_handle_tristate
- power_hier_delimiter
- power_honor_combinational_logic_on_clock_net
- power_honor_negative_energy
- power_honor_net_activity
- power_honor_non_mission_leakage
- power_hybrid_analysis
- power_ignore_control_signals
- power_ignore_data_phase_for_clock
- power_ignore_end_toggles_in_profile
- power_ignore_glitches_at_same_time_stamp
- power_ignore_inout_pin_cap
- power_ignore_macro_leakage_scale_for_temperature
- power_ignore_unconstraint_net_with_timing_data
- power_include_file
- power_include_initial_x_transitions

- power_include_sequential_clock_pin_power
- power_include_timing_in_current_file
- power_ir_derated_timing_view
- power_keep_clock_gate_ratio_in_iterations
- power_leakage_scale_factor_for_temperature
- power_lib_files
- power_library_preference
- power_macro_pgv_leakage
- power_macro_toggle_pin_percentage
- power_match_state_for_logic_x
- power_memory_current_scaling_method
- power_merge_switched_net_currents
- power_method
- power_min_leaf_count
- power_multibit_flop_toggle_behavior
- power_multibit_flop_toggle_pin_percentage
- power_multi_scenario_simulation
- power_off_pg_bulk_leakage
- power_off_pg_nets
- power_output_current_data_prefix
- power_output_dir
- power_partition_count
- power_partition_twf
- power_pin_based_twf
- power_precision
- power_pre_simulation_empty_period
- power_pre_simulation_period
- power_pre_simulation_power_exclude_period
- power_quit_on_activity_coverage_threshold
- power_read_rcdb
- power_relax_arc_match
- power_report_black_boxes
- power_report_clock_instance_switching_info
- power_report_idle_instances
- power_report_instance_switching_info

- power_report_instance_switching_list
- power_report_library_usage
- power_report_missing_bulk_connectivity
- power_report_missing_input
- power_report_missing_nets
- power_report_scan_chain_statistics
- power_report_statistics
- power_report_time_display_fraction_digits
- power_report_twf_attributes
- power_reuse_flop_ranking_data
- power_reuse_flop_ranking_data_hier
- power_scale_to_sdc_clock_frequency
- power_scan_chain_activity
- power_scan_chain_name_pattern
- power_scan_control_file
- power_scan_multi_bit_flop_chain_type
- power_settling_buffer
- power_show_cell_usage_stats
- power_smart_window
- power_split_bus_power
- power_start_time_alignment
- power_state_dependent_leakage
- power_stateprop_ignore_unannot_pins
- power_static_multi_mode_scenario_file
- power_static_netlist
- power_switching_power_on_rise_only
- power_thermal_input_file
- power_thermal_leakage_temperature_scale_table_file
- power_to_x_transition_factor
- power_to_z_transition_factor
- power_transition_factor_based_duty
- power_transition_time_method
- power_twf_delay_annotation
- power_twf_load_cap
- power_unified_power_switch_flow

- power_use_cell_leakage_power_density
- power_use_effective_load_cap_method
- power_use_fastest_clock_for_dynamic_scheduling
- power_use_lef_for_missing_cells
- power_use_only_ddv_current
- power_use_physical_partition
- power_use_twf_stable_randomized_arrival_delay
- power_use_zero_delay_vector_file
- power_vector_based_multithread
- power_vector_profile_mode
- power_worst_case_vector_activity
- power_worst_step_size
- power_worst_window_count
- power_worst_window_coverage
- power_worst_window_reports
- power_worst_window_size
- power_worst_window_type
- power_write_bbox
- power_write_db
- power_write_default_pti_files
- power_write_dynamic_currents
- power_write_gui_db
- power_write_profiling_db
- power_write_simulation_db
- power_write_static_currents
- power_x_count_transition_using_3_states
- power_x_transition_factor
- power_z_transition_factor
- power_zero_delay_vector_toggle_shift

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `power` category, use the following command:

```
get_db -category power
```

power_add_simulation

power_add_simulation `{true | false}`

Default: `false`

Data_type: bool

Specifies to add simulation if there are no simulation vectors or if all primary sources are not annotated by vectors. This attribute can be used to ensure complete design coverage. Event simulation is done for primary sources like primary input, flops, and macros.

This attribute is only applicable to the event-based power analysis flow (`set_power_analysis_mode -method event_based`).

Related Commands

- `set_power`

power_adjust_input_activity_in_iterations

`power_adjust_input_activity_in_iterations {true | false}`

Default: `true`

Data_type: bool

Specifies whether to keep the input activity fixed or automatically adjust the activity value for each iteration of state propagation. This attribute is only applicable to the power target flow (`power_enable_power_target_flow true`).

Related Commands

- `set_power`

power_adjust_macro_activity_in_iterations

`power_adjust_macro_activity_in_iterations {true | false}`

Default: `true`

Data_type: bool

Specifies whether to keep the macro activity fixed or automatically adjust the activity value for each iteration of state propagation. This attribute is only applicable to the power target flow (`power_enable_power_target_flow true`).

Related Commands

- `set_power`

power_analysis_temperature

`power_analysis_temperature` *`temperature`*

Default: `-274.000000`

Data_type: double

Specifies the temperature for static power calculation.

Note: Static power calculation gets the temperature value of each instance from the following sources (in order of precedence):

- Instance temperature file
- `power_analysis_temperature` attribute
- The current operating condition temperature

Related Commands

- `report_power`

power_annotation_detail_report

```
power_annotation_detail_report {true | false}
```

Default: `false`

Data_type: bool

Specifies to generate a detailed annotation report per instance in the Vector Profiling flow (`power_method vector_profile`). When specified, it creates the `AnnotationDetail.rpt` file in the user-specified output directory.

Related Commands

- `report_power`

power_auto_twf_delay_annotation

```
power_auto_twf_delay_annotation {true | false}
```

Default: `false`

Data_type: bool

Specifies to automatically detect the zero-delay vectors and perform delay annotation. As a result, you do not need to specify additional parameters (`set_db power_use_zero_delay_vector_file` and `read_activity_file -zero_delay`) to indicate the vectors that have delay annotation. This attribute is applicable only to the event-based power analysis flow.

Related Commands

- `report_power`

power_average_rise_fall_cap

```
power_average_rise_fall_cap {true | false}
```

Default: `false`

Data_type: bool

Specifies to use the average of rise and fall capacitance from the liberty file.

Related Commands

- report_power

power_block_independent_peakpower

```
power_block_independent_peakpower {true | false}
```

Default: `false`

Data_type: bool

Specifies to identify the worst power interval of each block and run power analysis of the full design using the worst interval of each block. This attribute allows you to perform power and current analysis for design blocks that have independent peak power intervals.

When set to `false`, power analysis is performed using the common worst interval found using simulation of all vectors together.

Related Commands

- report_power

power_bulk_pins

```
power_bulk_pins {bulk_pin_list}
```

Default: `""`

Data_type: string

Defines power and ground bulk LEF pins for the design.

Use this attribute when library cells have bulk pins defined in LEF but the liberty file does not contain power associated with these pins. If a liberty file does not have body bias definition, power analysis does not distribute power to the body bias domain.

Related Commands

- report_power

power_boundary_gate_leakage_file

```
power_boundary_gate_leakage_file filename
```

Default: `""`

Specifies a list of Liberty side files required for boundary gate leakage (context-aware leakage) calculation. This file defines the boundary current for different threshold voltage (Vt) layers at different operating conditions. The file includes:

- multiple leakage tables corresponding to different operating conditions.
- leakage probability factor for different abutments (when abutting instances are on the same Vt layer or when they are on different Vt layers).

When `power_boundary_gate_leakage_file` is specified, the `power.boundarygateleakage.rpt` file is dumped by default. This file contains the design-level summary for the three combinations of Vt select fail ratio (0, 1 and the one specified with HighVT_select_fail_ratio).

Related Commands

- report_power

power_boundary_gate_leakage_report

```
power_boundary_gate_leakage_report {true | false}
```

Default: `false`

Generates a verbose report for every abutted instance-edge for boundary leakage calculation. This detailed report is useful in debugging and understanding the leakage power calculation for an instance. The generated report name is `boundarygateleakage.detailed.rpt`.

Related Commands

- report_power

power_boundary_leakage_cell_property_file

```
power_boundary_leakage_cell_property_file filename
```

Default: `""`

Specifies the list of edge information files containing cell properties.

Related Commands

- report_power

power_boundary_leakage_edge_annotation_file

```
power_boundary_leakage_edge_annotation_file filename
```

Default: `""`

Specifies the edge annotation file with updated Vt layers at a given edge location to be used during boundary leakage calculation. This attribute is used to provide custom Vt layers for an abutted edge at a given location to be used during power computation.

Note: When running the edge annotation feature, the user-defined VT will have precedence over the one from the cell properties file.

Related Commands

- report_power

power_boundary_leakage_multi_pgpin_support

```
power_boundary_leakage_multi_pgpin_support {true | false}
```

Default: `false`

Enables support for cells with multiple power/ground pins.

Related Commands

- report_power

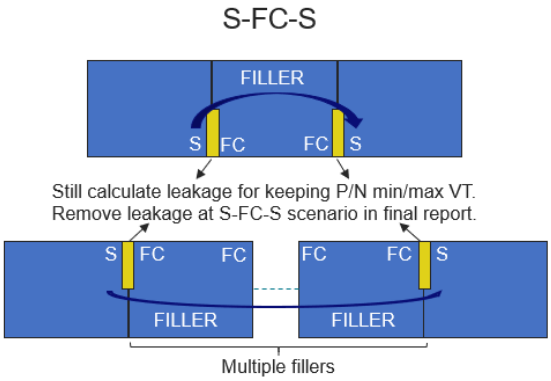
power_boundary_leakage_pessimism_removal

```
power_boundary_leakage_pessimism_removal {true | false}
```

Default: `true`

Enables pessimism removal for the Source to FCs to Source abutment type. This abutment type normally has 0 CNOD leakage. Therefore, the non-zero leakage computed at the starting S-FC and terminating FC-S boundaries needs to be removed for accurate estimation. This attribute is specified to remove the non-zero leakage or pessimism.

Revised CNOD leakage = CNOD Leakage – Leakage (S-FC-S).



Related Commands

- report_power

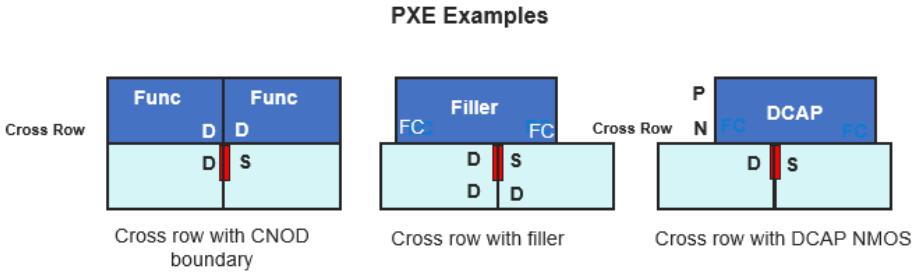
power_boundary_leakage_pxe_support

```
power_boundary_leakage_pxe_support {true | false}
```

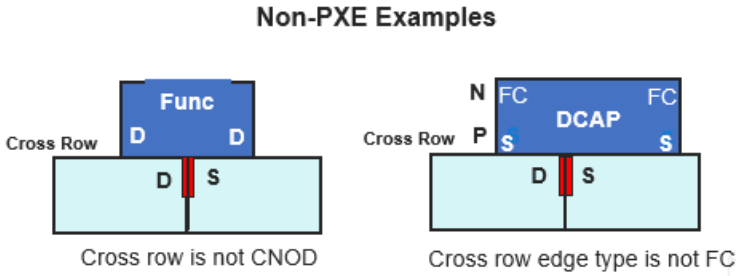
Default: `false`

Enables the PXE feature. PXE is used for calculation of CNOD leakage by cross row environment. The T structure or inverted T structure with both edges of the type FC is PXE.

The following diagram illustrates examples of PXE:



Every scenario other than the ones mentioned in PXE examples is non-PXE. The following diagram illustrates examples of non-PXE:



- If this attribute is set to `false`, Voltus does not consider the relationship by cross row environment.
- If this attribute is set to `true`, Voltus will find out the PXE or non-PXE structure and look up different leakage tables for obtaining the leakage value.

Related Commands

- `report_power`

power_capacity

```
power_capacity {low|medium|high}
```

Default: `low`

Data_type: enum {low medium high}

Enables segmented power processing. When this attribute is specified, the software performs concurrent processing of vector simulation annotation, power calculation, and report construction.

This attribute enables performance optimization with some trade-off on memory consumption and runtime. The arguments of `power_capacity` are:

- `low` - reduces memory for a flat run.
- `medium` - further reduces memory with moderate increase in runtime (1.2X).
- `high` - maximum memory reduction at about 2X runtime.

The `power_capacity` attribute is used for reducing memory when processing long simulation vectors. In case of the FSDB and SHM format activity file, the runtime can increase up to 2X.

Related Commands

- report_power

power_case_insensitive_mapping

```
power_case_insensitive_mapping {true | false}
```

Default: `false`

Data_type: bool

Specifies whether instance name mapping between the RTL netlist and GATE level netlist in the mapping file is case-sensitive or case-insensitive. When set to `true`, name mappings are case-insensitive.

Related Commands

- report_power

power_clock_source_as_clock

```
power_clock_source_as_clock {true | false}
```

Default: `false`

Data_type: bool

Specifies to use the correct clock frequency for activity calculation. If you stop the arrival of the clock signal by setting `set_case_analysis` to `false`, the transition density at the output net of the flop is calculated using the correct clock frequency instead of the default frequency.

Related Commands

- report_power

power_comprehensive_automapping

```
power_comprehensive_automapping {true | false}
```

Default: `false`

Data_type: bool

Specifies to perform all-inclusive instance name mapping between the RTL netlist and GATE level netlist. When set to `true`, the software honors all possible naming rules (includes handling of characters like '[' , ']', '.', '/', '\\', '_', ':' and handling of strings like "MBFF", "_reg", "_MB_" etc.) for better annotation coverage.

This feature is applicable to both file-based mapping (`read_activity_map_file -rtl_to_gate | -two_column_mapping_file`) and automatic mapping (`power_enable_auto_mapping`). When `power_comprehensive_automapping` is specified with either `-rtl_to_gate` or `-two_column_mapping_file`, the RTL mapping entries are given priority over the automatic rule-based mapping entries and are used to annotate the design objects.

Related Commands/Topics

- report_power
- RTL-to-Gate Rule Mapping

power_compress_reports

```
power_compress_reports {{true|false|1-9}
```

Default: `false`

Data_type: string

Specifies to compress the power output files in the GNU Zip format. The compressed files will have the suffix .gz.

The use model of this parameter is:
`set_power_analysis_mode -compress_reports {true|false|1-9}`

- `true`: Sets the compression option with compression level 6.
- `false`: default, no compression
- 1-9: the compression levels vary from 1 to 9. 1 provides the fastest compression speed but at a lower ratio, so the file size will be larger. 9 provides the highest compression ratio but at a lower speed.

Related Commands

- report_power

power_constant_override

```
power_constant_override {true | false}
```

Default: `false`

Data_type: bool

Specifies to give higher precedence to propagated constants defined using the `set_case_analysis` command. It allows propagated constants to override global activity.

Related Commands

- report_power

power_corner

```
power_corner {min|max}
```

Default: `""`

Data_type: string

Defines the library corner (for non-MMMC setup).

- `min` corner means power calculator will use min timing libraries for the power calculation.

- `max` corner means power calculator will use max timing libraries for the power calculation.

Related Commands

- `report_power`

power_create_driver_db

```
power_create_driver_db {true | false}
```

Default: `false`

Data_type: bool

Specifies to determine all the drivers/receivers on the ground net.

Related Commands

- `report_power`

power_current_generation_method

```
power_current_generation_method {avg | peak}
```

Default: `avg`

Data_type: string

Specifies the current waveform generation method during dynamic power analysis. The two possible arguments of this attribute are:

- `avg` - Specifies to use the average current value of each time step. The output current filename is *`filename`*`.ptiavg`. The default value is `avg`.
- `peak` - Specifies to use the peak current value of each time step. The output current filename is *`filename`*`.ptipeak`. It is recommended to use the `peak` method when using the Composite Current Source Power (CCSP) Liberty files.

Related Commands

- `report_power`

power_db_name

```
power_db_name filename
```

Default: `""`

Data_type: string

Specifies the name of the binary power database file (`filename.db`).

You can specify this attribute in scenarios where more than one db is generated and you need to track which one was generated in which mode when the restoring the db, or if your design has several blocks and analysis is done on one block all the time and all the databases are named `power.db`.

Related Commands

- report_power

power_decap_cell_list

```
power_decap_cell_list cell_list
```

Default: ""

Data_type: string

Specifies the physical only cells, such as decap cells, to include in the power report.

Note: This is only supported for Static Power Analysis.

Related Commands

- report_power

power_default_frequency

```
power_default_frequency value
```

Default: -1.0

Data_type: double

Specifies to set the default frequency in the MHz unit for net not annotated by TWF. You can specify a numerical value without unit.

Related Commands

- report_power

power_default_slew

```
power_default_slew value
```

Default:

Data_type: string

Specifies to set the default slew in the ps unit for net not annotated by TWF. You can specify a numerical value without unit.

Related Commands

- report_power

power_default_supply_voltage

```
power_default_supply_voltage value
```

Default:

Data_type: string

Specifies to set a default voltage for power nets in a scenario where the power engine cannot determine the voltage of these nets. The default value of this attribute is `1.0`.

Related Commands

- `report_power`

power_detailed_view_current_only

```
power_detailed_view_current_only {true | false}
```

Default: `true`

Data_type: bool

Controls instance power reporting for cells that have power information available in PGV. When set to `true`, specifies to skip writing of the switching and leakage current and use only the internal current from DDV PGV. When set to `false`, specifies to report the switching and leakage current values in addition to the internal current from DDV PGV. If a cell is defined in both PGV and dotlib, leakage is computed from dotlib. It is recommended to set this attribute to `true` because that would prevent over-estimation of the instance current.

Related Commands

- `report_power`

power_disable_clock_gate_clipping

```
power_disable_clock_gate_clipping {true | false}
```

Default: `true`

Specifies to enable clock gate output clipping so that the output transition density of an ICG cell does not exceed the input clock pin transition density.

Related Commands

- `report_power`

power_disable_leakage_scaling

```
power_disable_leakage_scaling {true | false}
```

Default: `false`

Data_type: bool

Specifies to exclude leakage power during scaling factor computation when target power is specified using the `set_power` command. When this attribute is set to `true`, the leakage power does not get scaled, and only the internal and switching components are scaled to meet the user-specified total power.

This attribute is relevant only for the `set_power` commands for which total power is specified, and has no effect for the `set_power` command specified with the `-internal`, `-switching`, or `-leakage` parameters.

Related Commands

- report_power

power_disable_static

```
power_disable_static {true | false}
```

Default: `false`

Data_type: bool

When set to `true` in the dynamic vector-based or dynamic vectorless flows, the command performs only dynamic analysis and turns-off static power calculation.

Related Commands

- report_power

power_distribute_switching_power

```
power_distribute_switching_power {true | false}
```

Default: `false`

Data_type: bool

Specifies to equally distribute the switching power among all drivers if a net has more than one driver. When set to `false`, stores all switching power on one of the net drivers.

Related Commands

- report_power

power_distributed_combine_report_format

```
power_distributed_combine_report_format {none | detailed | reduced}
```

Default: `reduced`

Data_type: string

Specifies to generate consolidated XP power reports. This attribute allows you to combine power reports that are stored across multiple partition directories to give a single unified view of power data. The possible arguments of this attribute are:

- `none` - generates distributed reports.
- `reduced` - generates consolidated reports for the following:
  - power.missingdata.* (static and dynamic flows)
  - power.stateprop.* (dynamic flow)
  - power.stats.setpower (dynamic flow)
- `detailed` - generates consolidated reports for the following:

- power.rpt (static and dynamic flows)
- power.instpwr.rpt (static flow)
- power.lib.rpt (static flow)
- power.pgpinpwr.rpt (static flow)
- power.pgnet.detailed.rpt
- power.missingdata.* (static and dynamic flows)
- power.pgnet.detailed.rpt (static and dynamic flows)
- power.stateprop.* (dynamic flow)
- power.stats.setpower (dynamic flow)
- pg_trigger.txt (dynamic flow)
- trigger.txt (dynamic flow)
- *.totalcurrent (dynamic flow)
- voltus_power.cellstat (static and dynamic flows)
- voltus_power.ptifiles (static and dynamic flows)

Related Commands

- `report_power`

power_distributed_setup

```
power_distributed_setup file
```

Default: `""`

Data_type: string

Specifies a file containing the customized setup details for running power analysis in the distributed mode.

The specified file has 4 columns:

`INST/CELL <name> block <power.inc>`

where,

- `INST/CELL`: Keyword indicating cell or instance to be run in the distributed mode. Cell can be useful for repeated DEF blocks.
- *`name`*: Name of the hierarchical block or instance.
- `block`: Keyword `block` implies that only the lower level DEF will be included
- *`power.inc`*: name of the power include file. This column is optional.

The unspecified instance/cell will be run with the top level as a single group. Each distributed block/instance should have corresponding TWF and SPEF to improve runtime.

⚠ Each block will be run independently from other blocks, thus the external load or cell seen at the pin of the block can be different from flat analysis. If the partition boundary analysis is important, distributed processing is not recommended.

Related Commands

- report_power

power_domain_based_clipping

```
power_domain_based_clipping {true | false}
```

Default: `false`

Data_type: bool

Determines the frequency to be used to clip propagated activity that is too high. When set to `false`, the frequency of the fastest clock in the design is used. If you set this attribute to `true`, the fastest clock domain on the net is used to clip propagated activity.

Related Commands

- report_power

power_dynamic_glitch_filter

```
power_dynamic_glitch_filter value
```

Default: `-1.0`

Data_type: double

Sets the absolute threshold time (in ns) for filtering glitches in net toggles. When you specify a time value for this attribute, glitches in net toggles that are equal to or smaller in width than the specified time value are removed.

This attribute is applicable only to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

Related Commands

- report_power

power_enable_auto_queue

```
power_enable_auto_queue {true | false}
```

Default: `false`

Data_type: bool

Specifies to automatically determine the queue for power jobs. This attribute is used in the Voltus-XP distributed power flow with hierarchical DEFs. In the XP flow, partitions are divided into multiple queues depending on the number of CPUs (For example, Default queue: 1 CPU, Small queue: 2 CPUs, Medium queue: 4 CPUs, Large queue: 8 CPUs, and so on). By default, all power partitions run on the same queue. When the attribute is set to `true`, the blocks can run on different queues depending on the size (number of instances) of the blocks. The `power_enable_auto_queue` attribute will be beneficial when there are uneven power block partitions. For example, if there are some blocks with 20-30M instances and there are some with 3-5M instances. Using this attribute, bigger blocks will be automatically assigned to a bigger queue, resulting in better runtime.

Related Commands

- report_power

power_enable_auto_mapping

```
power_enable_auto_mapping {true | false}
```

Default: false

Specifies to automatically perform instance name mapping between the RTL netlist and GATE level netlist. This attribute allows you to map the RTL vector to the Gate level netlist without a mapping file.

You can use the read_activity_file -name_mapping_rule <*filename*> command parameter to specify a file containing customized or design-specific rules.

Related Commands

- report_power

power_enable_disk_mapping

```
power_enable_disk_mapping {true | false}
```

Default: false

Data_type: bool

Specifies to enable disk caching of the toggles. Therefore, the toggles are stored in the disk instead of keeping them in memory. This has made large vectors or long simulation duration runs possible even with lesser available memory. This attribute is only applicable to the event-based power analysis flow (power_method event_based) in the XP mode.

Example:

The following command specifies to enable disk caching of toggles:

```
set_db power_method event_based power_enable_disk_mapping true power_enable_xp true
```

Related Commands

- report_power

power_enable_duty_propagation_with_global_activity

```
power_enable_duty_propagation_with_global_activity {true | false}
```

Default: false

Data_type: bool

Specifies to propagate duty cycle with the global switching activity setting for all data nets in the dynamic vectorless analysis flow.

Related Commands

- report_power

power_enable_dynamic_current_slew_load_interpolation

```
power_enable_dynamic_current_slew_load_interpolation {true | false}
```

Default: `false`

Data_type: bool

Specifies to use the charge-based slew load interpolation. When this attribute is set to `true`, Power Analysis takes dynamic currents from the 4 nearest slew/load values and performs either a 4-way (lower slew, lower load, upper slew, upper load) or a 2-way (lower slew, upper slew OR lower load, upper load) interpolation to calculate the dynamic current waveform. The attribute allows you to derive a more realistic dynamic current waveform based on the exact slew load values from the .lib dynamic current table.

Related Commands

- report_power

power_dynamic_scale_clock_by_frequency

```
power_dynamic_scale_clock_by_frequency { freq_Afactor_Afreq_Bfactor_Bfreq_Cfactor_C... }
```

Default: `""`

Data_type: string

Specifies the scale factors for the clock frequency values. This attribute allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The attribute is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.

Related Commands

- report_power

power_dynamic_scale_clock_by_name

```
power_dynamic_scale_clock_by_name { clk_A factor_A clk_B factor_B clk_C factor_C ... }
```

Default: `""`

Data_type: string

Specifies the scale factors for the low frequency clocks. This attribute allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The attribute is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.

Related Commands

- report_power

power_dynamic_vectorless_ranking_methods

```
power_dynamic_vectorless_ranking_methods {load | clock | vector_activity}
```

Default: `""`

Data_type: string

Specifies to change the ranking method from the default to the following:

- `load` - Prioritizes scheduling cells with higher fanout
- `clock` – Prioritizes scheduling cells associated with faster clocks
- `vector_activity` - Prioritizes scheduling cells with higher activity

Related Commands

- `report_power`

power_effective_load_cap_slew_threshold_limit

```
power_effective_load_cap_slew_threshold_limit <float between 0 and 1>
```

Default: 0

Data_type: float

Specifies the minimum threshold value for slew ratio to prevent effective capacitance from being derated beyond realistic limits.

Related Commands

- `report_power`

power_enable_dynamic_scaling

```
power_enable_dynamic_scaling {true | false}
```

Default: `false`

Data_type: bool

Enables `set_power` scaling for dynamic power analysis. The power and scaling specifications are used to scale dynamic current on both the power and ground currents of the specified instance/cell.

This attribute honors the user-defined average (static) power for leaf-level instances and hierarchies (instances/cells/modules) along with power net specification in the dynamic current scaling flows such that the average of the dynamic current is equal to the assigned average power. When you specify this attribute, the average of the dynamic current of these leaf-level instances will match the scaled static power numbers.

⚠ This attribute is not supported in the state-propagation-based vectorless flow (`power_enable_state_propagation true`). It is applicable only to the probability-based vectorless flow.

Related Commands

- `set_power`
- `report_power`

power_enable_flop_state_propagation

```
power_enable_flop_state_propagation {true | false}
```

Default: `false`

Data_type: bool

Enables accurate clock state propagation for sequential cells. When this attribute is set to `true`, flops propagate in a manner that toggles on the output pins are in sync with the clock. This attribute helps to align the output pins' toggles with the clock toggles.

Related Commands

- set_power
- report_power

power_enable_force_propagation

```
power_enable_force_propagation {true | false}
```

Default: `false`

Data_type: bool

Enables force propagation to ensure that the output toggles are according to the clock signal rather than the input pin states. This attribute assigns clock activity to outputs when input propagation is not feasible, thereby enhancing overall activity.

Related Commands

- set_power

power_enable_generated_clock

```
power_enable_generated_clock {true | false}
```

Default: `true`

Data_type: bool

Specifies to get generated clock frequency during activity propagation for static power analysis.

Related Commands

- set_power

power_enable_input_net_power

```
power_enable_input_net_power {true | false}
```

Default: `false`

Data_type: bool

Specifies to calculate the switching power of input nets.

Related Commands

- set_power

power_enable_interactive_reports

```
power_enable_interactive_reports {true | false}
```

Default: `true`

Data_type: bool

Specifies to generate multiple dynamic reports based on the power analysis run done for a netlist. When this attribute is set to `true`, you can generate various types of power reports interactively using the `report_power` command without the need to perform power calculation on the same netlist again. The `power_enable_interactive_reports` attribute is useful when you need to generate reports for the unaltered netlist within the same power analysis session.

This attribute is applicable only to the event-based power analysis flow with area-based partitioning (`power_use_physical_partition`) in the XP mode.

Example:

The following command specifies to enable interactive generation of power reports:

```
set_db power_method event_based power_use_physical_partition true power_partition_count 2
power_enable_interactive_reports true power_enable_xp true
```

Related Commands

- set_power

power_enable_mt_state_propagation

```
power_enable_mt_state_propagation {true | false}
```

Default: `true`

Data_type: bool

Specifies to enable multi-threading in state propagation to help improve the runtime for large simulations. For example, when the attribute was specified for a design with 300 million instances and simulation duration of 120 microseconds, the runtime has improved from 21 hours to 1.5 hours.

This attribute is only applicable to the event-based power analysis flow (`power_method event_based`).

Related Commands

- set_power

power_enable_multithread_reports

```
power_enable_multithread_reports {true | false}
```

Default: `false`

Data_type: bool

Specifies to enable multi-threading for generation of power calculation reports. When set to `true`, it improves the memory and runtime for power reporting in large designs with more than 100 million instances.

Related Commands

- `set_power`

power_enable_power_target_flow

```
power_enable_power_target_flow {true | false}
```

Default: `false`

Data_type: bool

Specifies to enable the power target flow. When set to `true`, the tool allows you to specify power targets for full-chip power analysis. You can use the `set_power` command to specify the power targets for full-chip, physical block, logical block, and power/ground net. When these targets are specified, Voltus will automatically scale the toggle rates of clock gate ratio, input, macro, and sequential activities to meet the specified power targets.

Related Commands

- `set_power`

power_enable_pba_for_tempus_pi

```
power_enable_pba_for_tempus_pi {true | false}
```

Default: `true`

Data_type: bool

Specifies to enable the Path-Based Analysis (PBA) for slack calculation in the Tempus Power Integrity (Tempus PI) flow. By default, the Graph-Based Analysis (GBA) is used. This attribute is only applicable to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

Related Commands

- `set_power`

power_enable_rtl_dynamic_vector_based

```
power_enable_rtl_dynamic_vector_based {true | false}
```

Default: `false`

Data_type: bool

Specifies to take an RTL or partial VCD/FSDB file as input and use that for dynamic vector-based flow. This attribute allows you to estimate current even for instances which are missing from the activity file.

You must use this attribute in conjunction with the `power_method dynamic_vectorbased` attribute.

Related Commands

- `set_power`

power_enable_scan_report

```
power_enable_scan_report {true | false}
```

Default: `false`

Data_type: bool

Specifies to generate all the scan mode analysis reports. The following reports will be generated:

- `voltus_power.scanff.duplicated.rpt` - flip-flops that are specified in multiple chains
- `voltus_power.scanff.missing.rpt` - flip-flops that are missing in the scan control file but exist in the design
- `voltus_power.scanff.nonassigned.rpt` - flip-flops that do not exist in the design and are dropped from the scan control file
- `voltus_power.scanff.nonscan`.rpt - assigned flip-flops that have no scan-out pin
- `voltus_power.scanff.nonswitched.rpt` - non-switching flip-flops in the scan control file
- `voltus_power.scanff.rpt` - detailed scan chain report containing global statistics for all scan chains and statistics for each scan chain in the design
- `voltus_power.scanff.stateconflict.rpt` - flip-flops that have conflicting state assignment

Related Commands

- `report_power`

power_enable_single_cycle_scheduling

```
power_enable_single_cycle_scheduling {true | false}
```

Default: `false`

Data_type: bool

Schedules macro instances in every dominant clock cycle in the dynamic vectorless flow. This attribute can be used achieve faster toggle coverage. After the scheduling of every instance and attaining the maximum toggle coverage, the scheduler cycles back to the same switch source element and then schedules it again with the opposite polarity. This attribute helps in achieving the same toggle coverage within half the simulation period compared to dual cycle scheduling.

When the attribute is set to `false`, the macros are scheduled every two cycles wherein the adjacent cycle is used for reversing the transition polarity.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

  `set_db` *`attribute_name value`*

- To get the current value of any root attribute, use the `get_db` command:

  `get_db` *`attribute_name`*

Related Commands

- set_power

power_enable_slew_based_ccs_pin_cap

```
power_enable_slew_based_ccs_pin_cap {true | false}
```

Default: `false`

Data_type: bool

Specifies to use the Composite Current Source (CCS) capacitance value based on the slew given in the input pin section of the library cell in the Liberty file. If it is set to `true`, the tool honors the CCS pin capacitance; if it is set to `false`, it takes `max_capacitance` from .lib. This attribute can be used when timing windows and/or a Tempus timing session is not available.

Related Commands

- set_power

power_enable_state_propagation

```
power_enable_state_propagation {true | false}
```

Default: `false`

Data_type: bool

Specifies to enable the vectorless state-propagation-based dynamic analysis flow.

For more information, refer to the **State-Propagation-Based Vectorless Methodology** section in the " *Dynamic Power and IRDrop Analysis* " chapter of the *Voltus User Guide*.

Related Commands

- set_power

power_enable_stable_clock_gating

```
power_enable_stable_clock_gating {true | false}
```

Default: `false`

Data_type: bool

Perform balanced scheduling of ICGs independent of the predecessor ICG state.

This attribute is used for the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`) only.

Related Commands

- set_power

power_enable_stable_flop_scheduling

```
power_enable_stable_flop_scheduling {true | false}
```

Default: `false`

Data_type: bool

Perform balanced scheduling of flops independent of the ECOs done on the design.

This attribute is used for the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`) only.

Related Commands

- `set_power`

power_enable_tempus_pi

```
power_enable_tempus_pi {true | false}
```

Default: `false`

Data_type: bool

Specifies to enable the Tempus PI analysis flow. When this attribute is set to `true`, data path scheduling is timing-aware, that is, scheduling is based on timing slacks. This attribute is only applicable to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

For more information, refer to the "*Tempus Power Integrity Analysis*" chapter in *Voltus Stylus Common UI User Guide*.

Related Commands

- `set_power`

power_enable_xp

```
power_enable_xp {true | false}
```

Default: `false`

Data_type: bool

Specifies to run standalone report generation in the XP mode.

Related Commands

- `set_power`

power_enhanced_blackbox_avg

```
power_enhanced_blackbox_avg {true | false}
```

Default: `false`

Data_type: bool

Specifies to use the average toggle rate of related inputs for enhanced blackbox propagation.

Related Commands

- set_power

power_enhanced_blackbox_max

```
power_enhanced_blackbox_max {true | false}
```

Default: `false`

Data_type: bool

Specifies to use the maximum toggle rate of related inputs for enhanced blackbox propagation.

Related Commands

- set_power

power_equivalent_annotation

```
power_equivalent_annotation {true | false}
```

Default: `false`

Data_type: bool

Specifies to enable equivalent annotation for Q or Qbar pins when only one of them is annotated.

Related Commands

- set_power

power_event_based_leakage_power

```
power_event_based_leakage_power {true | false}
```

Default: `false`

Data_type: bool

Specifies to calculate state-dependent leakage power. When this attribute is set to `true`, it calculates leakage power and current based on the state of an instance at each time point.

For example, at time 0ns, instance instA is in state !I ZN (0 on input, 1 on output), and at time 71ns, the instance is in state I !ZN (1 on input, 0 on output). The leakage power is calculated based on the different states of instA at different time points.

Related Commands

- set_power

power_external_load_config_file

```
power_external_load_config_file filename
```

Default: ""

Data_type: string

Specifies to honor the user-specified load capacitance values for power calculation. This attribute allows you to specify an external load capacitance file when you have a timing window file without the EXTERNAL_LOAD constructs.

File Format:

The format of the user-specified input file is:

```
INST|CELL    blockName    external_load_flle
```

Here, the format of *external_load_flle* is:

```
NET|PORT       name       load_value
```

Examples:

Configuration file:

```
INST          TOP           top_wcard.load
```

External load file (top_wcard.load):

```
UNIT pF
NET   result     0.02
NET   overflow   0.02
```

Related Commands

- set_power

power_extraction_tech_file

```
power_extraction_tech_file filename
```

Default: ""

Data_type: string

Specifies the extraction technology file to be used for top-level power-grid extraction.

Related Commands

- set_power

power_extractor_include

```
power_extractor_include filename
```

Default: ""

Data_type: string

Specifies the extractor (ZX) commands and variables in the form of an include file. This attribute is applicable only for the standalone report generation flow in the XP mode (power_enable_xp true).

Related Commands

- set_power

power_fanout_limit

```
power_fanout_limit value
```

Default: -1

Data_type: int

Specifies the fanout limit for nets. If an instance has nets more than or equal to the fanout limit, its switching/internal/leakage power is taken to be zero.

Related Commands

- set_power

power_force_library_merging

```
power_force_library_merging {true | false}
```

Default: false

Data_type: bool

Specifies to force merge PGVs even if there are some minor conflicts in the merging libraries. This attribute is applicable only for the standalone report generation flow in the XP mode (power_enable_xp true).

Related Commands

- set_power

power_from_x_transition_factor

```
power_from_x_transition_factor value
```

Default: 0.5

Data_type: double

Specifies how transitions from X to 0/1 are counted.

Related Commands

- set_power

power_from_z_transition_factor

```
power_from_z_transition_factor value
```

Default: 0.25

Data_type: double

Specifies how transitions from Z to 0/1 are counted.

Related Commands

- `set_power`

power_generate_activity_mapping_report

```
power_generate_activity_mapping_report  {true | false}
```

Default: `false`

Data_type: bool

Specifies to generate the `activity_mapping.rpt` report in the vector-based dynamic power flow. This report gives the list of signal name mappings between the Golden (RTL netlist - input name in the vector file) and Revised (Gate netlist - annotated design object name) netlists that were used in the flow. The report displays the list of annotated and unannotated objects and helps to debug the mapping file for unannotated design objects.

`activity_mapping.rpt` supports activity mapping done through the following methods:

- User-specified mapping file (`read_activity_map_file -two_column_mapping_file`)
- Conformal-generated RTL mapping file (`read_activity_map_file -rtl_to_gate`)
- Voltus auto-generated mapping file (`power_enable_auto_mapping`).

Related Commands

- `set_power`

power_generate_current_for_rail

```
power_generate_current_for_rail railnames
```

Default: `""`

Data_type: string

Generates current files for the specified rail.

By default, the command generates current files for all power rails.

Related Commands

- `set_power`

power_generate_flop_ranking_data

```
power_generate_flop_ranking_data directoryname
```

Default: `""`

Data_type: string

Specifies the name of the directory where the flop ranking data is saved.

Related Commands

- set_power

power_generate_static_report_from_state_propagation

```
power_generate_static_report_from_state_propagation  {true | false}
```

Default: `false`

Data_type: bool

Specifies to directly generate the static power analysis result based on the RTL-VCD input and state-propagation. When this attribute is specified, the RTL vector power analysis flow does not require the generation of a TCF/FSDB file.

This attribute is applicable to the dynamic propagation-based flow (vector-based/vectorless/RTL/Mixed-mode) and is not applicable to the `power_static_netlist verilog` flow.

Related Commands

- set_power

power_generate_leakage_power_map_based_on_calculated_leakage

```
power_generate_leakage_power_map_based_on_calculated_leakage  {true | false}
```

Default: `false`

Data_type: bool

Specifies to scale the instance leakage power by directly multiplying the user-specified scaling factor with the final calculated leakage power value, instead of scaling the leakage power value from the input dotlib.

If this attribute is not specified, the user-specified scaling factor will be applied on the leakage power values from the input dotlib.

Related Commands

- set_power

power_grid_libraries

```
power_grid_libraries library_list
```

Default: `""`

Data_type: string

Specifies the name of the Cadence power cell libraries (`.cl`). You can also specify a list of defined power-grid libraries having decap cells tagged in them.

⚠ When specifying a list of PGVs, you must specify the technology library as the first PGV library in the list, followed by the standard and macro PGV libraries. The selection between the standard and macro PGV library is order based. Therefore, if multiple power-grid libraries are specified for a cell, the power engine maps the cell with the first library after the technology library.

Related Commands

- `set_power`

power_handle_glitch

```
power_handle_glitch {true | false}
```

Default: `false`

Data_type: bool

A transition is defined as a glitch when the time difference between two sequential toggles are less than half of the rise transition time + fall transition time in the vector-based static power calculation. Both static and dynamic current are derated for glitch transitions. The same derate factor is used for both power and current.

If set to `true`, glitch power will be identified and calculated according to this definition and it will be reported in a separate column. Switching or internal power calculation does not include power due to glitches.

If set to `false`, glitch power will not be identified, and the respective power will be calculated as normal signal switches. Hence, the total power will be a little higher.

Related Commands

- `set_power`

power_handle_transport_glitch

```
power_handle_transport_glitch {true | false}
```

Default: `false`

Data_type: bool

Transport glitch is referred to as functional toggles for a net in a single clock period before it reaches a stable value. You can use this attribute to report the internal and switching power values for transport glitch.

The transport glitch power is reported in the following reports:

- Average Power Summary Report (`power.rpt.avgpwr`) - reported in the Total, Group, Clock, and Rail power sections.
- Instance Power Report (`power.rpt.instpwr`) - reported for all instances.
- Hierarchy Power Report (`power.rpt.hierpwr`) - reported for all hierarchies.

This attribute is automatically set to `true` with `power_handle_glitch true`. That is, reports both inertial and transport glitch power. When set to `false`, only the inertial glitch power is reported.

Related Commands

- set_power

power_handle_tristate

```
power_handle_tristate {true | false}
```

Default: `false`

Data_type: bool

When set to `true`, all tristate device enable pin values will be taken into account when determining the propagation of the activity through tristate gates.

Related Commands

- set_power

power_hier_delimiter

```
power_hier_delimiter character
```

Default: `""`

Data_type: string

Specifies the hierarchical delimiter in the DEF file. You can use this attribute to override the existing hierarchical delimiter.

Related Commands

- set_power

power_honor_combinational_logic_on_clock_net

```
power_honor_combinational_logic_on_clock_net {true | false}
```

Default: `true`

Data_type: bool

Specifies the behavior of combination logic on clock net.

- `true` (default) - specifies to honor activity propagation through combinational logic.
- `false` - specifies that the combination logic will be treated like a combinational clockgate.

Related Commands

- set_power

power_honor_negative_energy

```
power_honor_negative_energy {true | false}
```

Default: `true`

Data_type: bool

A value of `true` specifies that the software will keep negative internal energy numbers from the `.lib` internal power table. When set to `false`, negative power is treated as 0.

Related Commands

- `set_power`

power_honor_net_activity

```
power_honor_net_activity {true | false}
```

Default: `true`

Data_type: bool

Specifies to use activity from the `Net` section of the Switching Activity Interchange Format (SAIF) file for power calculation and optimization, and to ignore the pin-based state-dependent path delay (SDPD) activity. The default value of this attribute is `true`.

When this attribute is set to `false`, the power calculation engine can use both the net-based and pin-based activity. In this case, precedence will be given to activity from net or pin that appears later in the SAIF file.

Related Commands

- `set_power`

power_honor_non_mission_leakage

```
power_honor_non_mission_leakage {true | false}
```

Default: `false`

Data_type: bool

Supports modeling of well-to sub-leakage information when the circuit power supply is off for the level shifter cells. This is referred to as the non-mission mode leakage power calculation. The leakage power for a level shifter cell depends on the state of each PG pin. The Liberty file contains sections for PG pin conditions of each mode/state and non-mission mode leakage power for each PG pin of that mode.

When this attribute is set to `true`, Voltus identifies the pg_pin mode and assigns non-mission mode leakage power associated with the mode.

The use model of the non-mission mode leakage power calculation feature is:

```
power_honor_non_mission_leakage true
power_off_pg_nets {VDD VDDO}
```

Example:
Let us consider a level shifter cell with the PG mode "`retained_VDDO_off`" and the following `pg_pin_condition`:

```
pg_setting_value (retained_VDDO_off) {
pg_pin_condition : "!VBB*VDD*VDDI ";
pg_pin_active_state (VBB, V1);
pg_pin_active_state (VDD, V2);
pg_pin_active_state (VDDI, V2);
```

When you specify a list of PG nets to be turned off along with the `power_honor_non_mission_leakage` attribute, all cells/instances that have the PG state "`retained_VDDO_off`" associated with the specified PG pin condition, will use the corresponding non-mission leakage power.

Related Commands

- `set_power`

power_hybrid_analysis

```
power_hybrid_analysis {true | false}
```

Default: `false`

Data_type: bool

Specifies to perform both vector-based and vectorless power analysis in a single session.

Related Commands

- `set_power`

power_ignore_control_signals

```
power_ignore_control_signals {true | false}
```

Default: `true`

Data_type: bool

Specifies that control signals will be ignored when propagating activity. It is recommended to set this attribute to `true`. When set to `true`, the control pins (reset, preset) will not impact the sequential instance's output pin activity during activity propagation.

Related Commands

- `set_power`

power_ignore_data_phase_for_clock

```
power_ignore_data_phase_for_clock {true | false}
```

Default: `false`

Data_type: bool

Specifies to ignore data frequency in the clock path for the clock instances in the design. When this attribute is set to `true`, the software propagates only the fastest clock pin frequency instead of the data pin frequency for activity propagation through clock network. In the default mode, the software picks the fastest frequency between the data phase and clock phase on that pin.

Related Commands

- `set_power`

power_ignore_end_toggles_in_profile

```
power_ignore_end_toggles_in_profile {true | false}
```

Default: `false`

Data_type: bool

By default, the software profiles the toggles in the user-selected vector time window by steps including the toggles at the start/end boundary of the window. In each step, the toggles on the start boundary edge of the step is counted in the current step; the toggles on the end boundary edge is counted in the next step.

When this attribute is set to `false`, the toggles on the end boundary edge is also counted in the last step, as there is no next step to it. This may lead to double count of the power profile for the last step, especially for zero-delay vector profiling.

When this attribute is set to `true`, the software ignores the toggle on the end boundary of the last step.

Related Commands

- set_power

power_ignore_glitches_at_same_time_stamp

```
power_ignore_glitches_at_same_time_stamp {true | false}
```

Default: `true`

Data_type: bool

Specifies to ignore glitches at the same transient time in the vector-based static and dynamic power analysis flow. When this attribute is set to `true`, only the last signal value change is honored, and all the preceding value changes at the same time stamp are ignored. The last signal value will overwrite the previous value for the same signal at the same time stamp, so the last value will represent the state of the signal at that time stamp.

Related Commands

- set_power

power_ignore_inout_pin_cap

```
power_ignore_inout_pin_cap {true | false}
```

Default: `false`

Data_type: bool

When set to `true`, ignores the bidirectional pin capacitances (`direction : inout`) defined for I/O cells in the .lib file, when calculating switching power and internal power; this also includes the default capacitance value for `inout` pins.

Related Commands

- set_power

power_ignore_macro_leakage_scale_for_temperature

```
power_ignore_macro_leakage_scale_for_temperature {true | false}
```

Default: `false`

Data_type: bool

The `power_leakage_scale_factor_for_temperature` attribute is used to perform linear scaling of the leakage power for temperature in both the standard cell and macro libraries. The `power_ignore_macro_leakage_scale_for_temperature` attribute allows you to selectively apply the scale factor only for standard cells, and not use the scale factor for macro cells.

Related Commands

- `set_power`

power_ignore_unconstraint_net_with_timing_data

```
power_ignore_unconstraint_net_with_timing_data {true | false}
```

Default: `false`

Data_type: bool

Specifies to ignore unconstrained nets/pins from toggle generation. These nets/pins do not have an associated clock but contain valid timing data with arrival windows in the TWF file. If set to `false`, the tool applies default frequency toggles to these unconstrained nets, which could cause false IR violation warnings due to high voltage drop.

Related Commands

- `set_power`

power_include_file

```
power_include_file file
```

Default: `""`

Data_type: string

Specifies a power analysis include file. You can use this file to specify certain commands of the dynamic engine which do not have a Voltus equivalent. This attribute is used to pass the legacy power analysis commands and `report_power` command options only in the dynamic mode (vectorbased and vectorless). If the `power_method` attribute is set to `static`, the `power_include_file` attribute is ignored. The `power_include_file` attribute is required only if the `power_static_netlist` attribute is set to `def`.

Related Commands

- `set_power`

power_include_initial_x_transitions

```
power_include_initial_x_transitions {true | false}
```

Default: `true`

Data_type: bool

Controls how the initial X state toggle should be counted. The initial X state means the state at time `0`.

When set to `true`, this means that the software counts the power caused by the `X->0` or `X->1` toggle. When set to `false`, the software does not count the power caused by the initial X state toggle.

Related Commands

- `set_power`

power_include_sequential_clock_pin_power

```
power_include_sequential_clock_pin_power {true | false}
```

Default: `false`

Data_type: bool

Reports the clock pin power of flip-flops as part of the clock network power.

Related Commands

- `set_power`

power_include_timing_in_current_file

```
power_include_timing_in_current_file {true | false}
```

Default: `false`

Data_type: bool

Specifies to write timing bits for non-switching instances in the current files during the state-propagation-based flow. When specified, rail analysis will report effective instance voltage (EIV) for both switching and non-switching instances. If the attribute is set to `false`, rail analysis will report EIV for the switching instances only.

Related Commands

- `set_power`

power_ir_derated_timing_view

```
power_ir_derated_timing_view view_name
```

Default: `""`

Data_type: string

Specifies the name of the timing view used to calculate timing slack and to rank the flops in the Tempus PI flow. This attribute must be specified if there are multiple views.

The `power_ir_derated_timing_view` attribute defines the timing view used to detect the IR-sensitive paths using a lower voltage than the actual analysis. This lower voltage simulates the IR-drop for each instance in the design. The software uses this view to

prioritize the end-points that need to be prioritized for switching.

This attribute is only applicable to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

Related Commands

- `set_power`

power_keep_clock_gate_ratio_in_iterations

```
power_keep_clock_gate_ratio_in_iterations {true | false}
```

Default: `false`

Data_type: bool

Specifies whether to keep the clock gate output (CGO) fixed or automatically adjust the value for each iteration of state propagation. This attribute is only applicable to the power target flow (`power_enable_power_target_flow true`).

Related Commands

- `set_power`

power_leakage_scale_factor_for_temperature

```
power_leakage_scale_factor_for_temperature view_name
```

Default: `1.0`

Data_type: double

Performs a linear scaling of the leakage power for temperature in all libraries. This replaces the k-factor for temperature in the .lib file. For process and voltage, the scaling will continue to be based on the corresponding k-factors in the .lib file. A value of 0.8 scales the leakage power in the library to 80 percent. A value of 1.2 scales the leakage power in the library to 120 percent.

Related Commands

- `set_power`

power_lib_files

```
power_lib_files in_file <file1[file2] ...
```

Default: `""`

Data_type: string

Specifies the name of the timing library file(s) for dynamic power analysis. This attribute allows you to run the dynamic power analysis flow without reading a dotlib into design database to reduce background memory consumption. You must have an external TWF file to run this flow as the timing engine will not be invoked from within Voltus. This flow is recommended for large designs where front-end processing of timing libraries is high. Some of the database query commands will not work if this attribute is used.

Related Commands

- set_power

power_library_preference

```
power_library_preference {voltage | ecsm_ccsp}
```

Default: `voltage`

Data_type: string

Instances of the same cell can be attached to different power domains in a design. If you have specified different PVT libraries, then the software will by default bind each instance to the closest library definition.

You can use this attribute to select either the closest Effective Current Source Model (ECSM)/Composite Current Source-power (CCSP), or the closest voltage library for the instance:

- `voltage`: when specified, enables library binding to the closest voltage library.
- `ecsm_ccsp`: when specified, enables library binding to the closest ECSM/CCSP library.

Related Commands

- set_power

power_macro_pgv_leakage

```
power_macro_pgv_leakage {true | false}
```

Default: `true`

Data_type: bool

Controls leakage power reporting for cells that have power information available in both dotlib and PGV. When the attribute is set to `false`, the leakage current from Liberty is used and when the attribute is set to `true`, the leakage current from PGV is used.

Related Commands

- set_power

power_macro_toggle_pin_percentage

```
power_macro_toggle_pin_percentage percentage
```

Default: `0.0`

Data_type: double

Specifies the percentage of output data pins (ignoring the control pins) to be scheduled for a macro cell. This attribute is applicable to the state-propagation based vectorless flow. The default macro pin toggle percentage is 50%.

Related Commands

- set_power

power_match_state_for_logic_x

```
power_match_state_for_logic_x value
```

Default: x

Data_type: string

Controls how the logic X will be evaluated in the boolean function of the 'when' state of a power table.

The attribute uses the following arguments:

- 0 - regards X as 0
- 1 - regards X as 1
- x/X - regards X as neither 0 nor 1, that is, whenever there is any X logic in it, the boolean function will be evaluated 0 (false).

Related Commands

- set_power

power_memory_current_scaling_method

```
power_memory_current_scaling_method {1 | 2 | 3}
```

Default: 0

Data_type: int

Specifies to enable scaling of the NLPM-based currents by a user-specified scale factor for macro cells that are switching. The applicable arguments are 1, 2, and 3, which means enable a scaling factor of 0.5, 0.25, and 0.1 respectively.

Related Commands

- set_power

power_merge_switched_net_currents

```
power_merge_switched_net_currents {true | false}
```

Default: false

Data_type: bool

Specifies to merge switched net currents into always-on currents. For designs with a large number of switched nets, this attribute helps improve performance by merging switched net instance currents with always-on instance currents. However, when this attribute is set to true, you will not be able to use the generated current files to run power-up analysis.

Related Commands

- `set_power`

power_method

```
power_method { static | dynamic_vectorless | dynamic_vectorbased | dynamic_mixed_mode | event_based | vector_profile }
```

Default: `static`

Data_type: enum {static dynamic dynamic_vectorless dynamic_vectorbased dynamic_mixed_mode vector_profile event_based}

Specifies the type of analysis to be performed. You can perform two types of power analysis, namely static and dynamic.

- `static` - specifies to perform static power analysis to calculate average power.

Dynamic power analysis can be a vector-based or vectorless approach.

- `dynamic_vectorbased` - uses the full VCD output of a logic simulator to identify the instances that are switching and when they switch.
- `dynamic_vectorless` - uses the timing arrival window information from a static timing analysis tool to determine when instances switch. A timing arrival window describes when a signal can change within a clock cycle. An additional algorithm then determines the instances that switch.
- `dynamic_mixed_mode` - specifies to combine the vector-based and vectorless (mixed) dynamic analysis. This flow can be used in the following cases:
  - **case 1**: vectors are not available for every block of a full-chip design.
    In this case, the software uses the vectorless methodology for blocks with incomplete vectors
  - **case 2**: blocks with vectors having unannotated combinational/ sequential logic.
    In this case, the software performs state propagation for the missing combinational/ sequential logic. It uses the vectorless methodology for a logic cone that does not have the primary inputs or flops annotated from the vector, and then performs state propagation

    To enable the dynamic mixed mode analysis flow, you must specify the following commands:

    ```
    set_default_switching_activity -clock_gates_output 2.0 -global_activity <activity>
    set_db power_method dynamic_mixed_mode
    read_activity_file -scope <> -block <> -> (for available blocks)
    ```
- `event_based -` specifies to enable event-based power analysis. This method supports multiple power analysis flows simultaneously. When specified, the tool performs accurate state-based power estimation for all events of a scenario in a design. Event-Based Power Analysis is applicable to all event-based vectors, such as VCD, FSDB, SHM, and PHY. You can specify this method to generate and analyze multiple types of power analysis flow reports in a single run.

  The following table shows the additional options required for the different flows:

Power Analysis Flows	Additional Options for `-method event_based`
Static	`no additional option required`
Average Power & Current	`-write_static_currents`
Dynamic Vectorbased	`-write_dynamic_currents`
Mixed Mode	`-add_simulation`
Dynamic Vectorless	
Profiling	`-worst_step_size`

- `vector_profile` - specifies to enable vector profiling. This is used to identify windows with maximum activity and power which can then be used to drive dynamic vector-based power analysis.

Related Commands

- `set_power`

power_min_leaf_count

```
power_min_leaf_count  value
```

Default: `0`

Data_type: int

Specifies the minimum number of leaf instances required for writing the hierarchy data in the profiling database. It allows you to restrict the number of hierarchies for which waveforms are reported in the profiling database when using both the `power_write_profiling_db` attribute and `report_power -hierarchy` parameters.

Any hierarchy whose leaf instance count in its complete sub-tree is less than the `power_min_leaf_count` value will be skipped from getting reported in the profiling database. This attribute can be used to skip very small hierarchies and as a result save on the database writing time and disk space.

Related Commands

- `set_power`

power_multibit_flop_toggle_behavior

```
power_multibit_flop_toggle_behavior { simultaneous | independent | sbff }
```

Default: `independent`

Specifies the toggle behavior of multi-bit flip-flops (MBFF). The possible arguments are:

- `simultaneous`: The multiple bits of the MBFF will toggle or not toggle at the same time. The MBFF will be considered as one flip-flop.
- `independent`: The multiple bits of the MBFF will toggle independently. The MBFF will be considered as one flip-flop.
- `sbff`: The MBFF will be considered as multiple Single-Bit Flip-Flops (SBFFs). Each bit will be considered as one flip-flop,

and will toggle independently.

Related Commands

- `set_power`

power_multibit_flop_toggle_pin_percentage

```
power_multibit_flop_toggle_pin_percentage percentage
```

Default: `0.0`

Data_type: double

Specifies the percentage of multi-bit flip-flops (MBFF) pins to be scheduled. This attribute is applicable to the state-propagation based vectorless flow. The attribute will only work when `power_multibit_flop_toggle_behavior` is set to `independent`.

Related Commands

- `set_power`

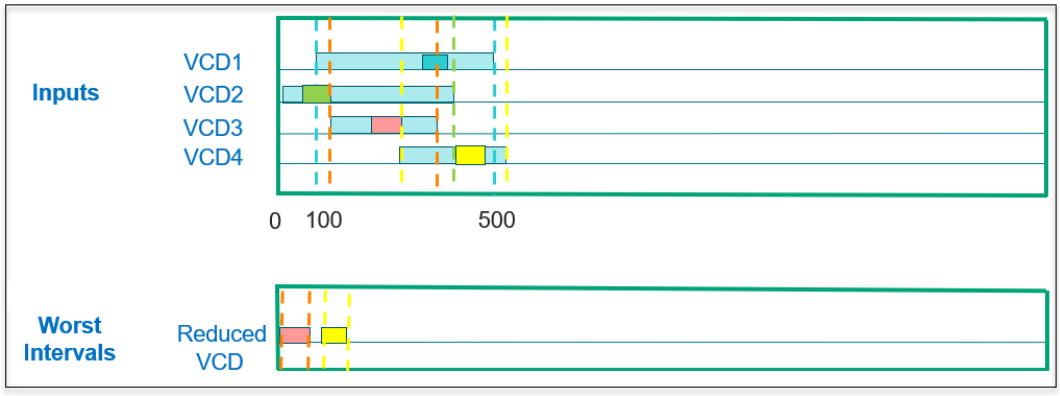
power_multi_scenario_simulation

```
power_multi_scenario_simulation {true | false}
```

Default: `false`

Data_type: bool

Specifies to create a long vector by combining multiple vectors and to identify the worst power intervals across the combined vector. The long vector includes all scenarios that are used sequentially and are non-overlapping.



This attribute allows you to perform power and current analysis using worst peak power interval across multiple simulation vectors.

Related Commands

- `set_power`

power_off_pg_bulk_leakage

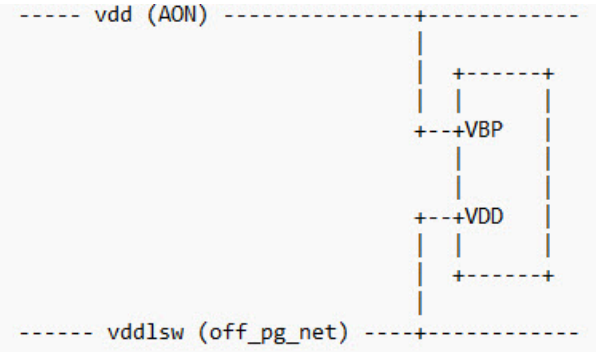
```
power_off_pg_bulk_leakage { true | false }
```

Default: `false`

Data_type: bool

Specifies to account for the leakage power through an instance's bulk pin when it is connected to a power-ground (PG) rail in the ON state.

The following diagram illustrated that the instance contributes bulk pin leakage to the ON state rail (**vdd**) through the *VBP* pin and zero leakage to the OFF state rail (`vddlsw`) through the VDD pin.



Related Commands

- `set_power`

power_off_pg_nets

```
power_off_pg_nets net_list
```

Default: `""`

Data_type: string

The name of the power rails that will be turned off. The net name can contain instance hierarchy if appropriate. The software will take this data into account when figuring out the list of power gates which are switched off.

⚠ A net is turned off if all connected power gates are turned off by control signals. When a net is off, all instances connected to that net will contribute zero power.

Related Commands

- `set_power`

power_output_current_data_prefix

```
power_output_current_data_prefix prefix
```

Default: `""`

Data_type: string

Specifies a prefix to the static or dynamic current files generated by the software. When power analysis is done in the multi-CPU mode, the output current data directory name and the underlying current files will also use the same prefix by default.

Related Commands

- `set_power`

power_output_dir

```
power_output_dir directory
```

Default: `""`

Data_type: string

Specifies the name of the directory that power information will be output to.

Related Commands

- `set_power`

power_partition_count

```
power_partition_count value
```

Default: `0`

Data_type: int

Specifies the number of physical partitions or jobs for a distributed run in the area-based partitioning flow. This attribute is only applicable to the event-based power analysis flow (`power_method event_based`) in the XP mode. The `power_partition_count` attribute must be specified with the `power_use_physical_partition` attribute.

Related Commands

- `set_power`

power_partition_twf

```
power_partition_twf  {true | false}
```

Default: `false`

Data_type: bool

Specifies to read the input TWF data (flat or hierarchical TWF files), and generate block-level TWFs.This attribute must be specified when the block/partition level TWF files are not available.

Related Commands

- `set_power`

power_pin_based_twf

```
power_pin_based_twf  {true | false}
```

Default: `false`

Data_type: bool

Specifies whether to generate pin-based TWF or net-based TWF.

- When set to `false`, a net-based TWF file is generated. Net-based TWF file is smaller in size and therefore results in faster processing.
- When set to `true`, a pin-based TWF file is generated. Pin-based TWF file is bigger in size and requires longer processing time, however, is more accurate because the slews numbers are pin-based.

⚠ This attribute must be used with the `read_sdc` command.

Related Commands

- `set_power`

power_precision

```
power_precision value
```

Default: `8`

Data_type: int

Specifies precision of decimal range [1-8] to ensure consistent decimal places are displayed for each power component. This attribute is used for precision handling in `report_instance_power` and `report_power`.

Related Commands

- `report_power`

power_pre_simulation_empty_period

```
power_pre_simulation_empty_period value
```

Default: `""`

Data_type: string

Specifies to add an empty pre-simulation period before simulation during IR drop analysis. Power analysis is not done for the pre-simulation period and flat dynamic currents are created.

Related Commands

- `report_power`

power_pre_simulation_period

```
power_pre_simulation_period value
```

Default: `""`

Data_type: string

Specifies to set a pre-simulation period for IR drop analysis. Pre-simulation is used for generating power analysis reports and dynamic currents.

Related Commands

- `report_power`

power_pre_simulation_power_exclude_period

```
power_pre_simulation_power_exclude_period value
```

Default: `""`

Data_type: string

Specifies to exclude power analysis during the pre-simulation period used for IR drop analysis. Pre-simulation is used only to create dynamic currents and not used for power reports.

Related Commands

- `report_power`

power_quit_on_activity_coverage_threshold

```
power_quit_on_activity_coverage_threshold threshold_value
```

Default: `0.0`

Data_type: double

Specifies the activity (VCD/FSDB/TCF) coverage threshold value. This attribute allows you to control the acceptable activity coverage that you are expecting and enforce the software to quit when it does not meet the criteria. When you specify the threshold value, the software exits if the real activity coverage is below the threshold.

Related Commands

- `report_power`

power_read_rcdb

```
power_read_rcdb  {true | false}
```

Default: `false`

Data_type: bool

Specifies to write a SPEF file containing only the total C for signal nets and pass it to the Dynamic Power engine. Dynamic power analysis requires only the total C for signal nets. Therefore, instead of writing a large SPEF file (with distributed RC) from

the RCDB, this attribute specifies to create and pass a SPEF file with only total C for all the signal nets. This significantly reduces the parasitic annotation time in the Dynamic Power engine.

Related Commands

- report_power

power_relax_arc_match

```
power_relax_arc_match  {true | false}
```

Default: `false`

Data_type: bool

Specifies to relax arc match requirements. This attribute is used only when the exact arc is not found in a .lib. When set to `true`, the internal energy is calculated as an average of multiple arcs with relaxed arc matching. If this attribute is set to `false`, relaxed arc search is not done and 0 internal energy is returned.

Related Commands

- report_power

power_report_black_boxes

```
power_report_black_boxes {true | false}
```

Default: `false`

Data_type: bool

Specifies to report cells that are used as black boxes.

A black box is a cell which is either missing from the liberty file or has no functionality defined in the liberty file.

Related Commands

- report_power

power_report_clock_instance_switching_info

```
power_report_clock_instance_switching_info  {true | false}
```

Default: `None`

Data_type: string

Specifies to generate a report for clock instances that switched during the state-propagation based vectorless flow. The generated report contains the toggle time, instance name, input pin event, and output pin event for each switching event of the clock instance.

Related Commands

- report_power

power_report_idle_instances

```
power_report_idle_instances {true | false}
```

Default: `false`

Data_type: bool

Specifies to generate a report (`idleinstance.rpt`) that lists all the idle or non-switching instances.

The following is a snippet of the `idleinstance.rpt` report:

```
DFF22/DFF    F
DFF42/DFF    F
...
```

Here, the first column is the instance name and the second column is the type of instance (`F` indicates flop and `C` indicates combinational).

Related Commands

- `report_power`

power_report_instance_switching_info

```
power_report_instance_switching_info {all | output_logic | none}
```

Default: `none`

Data_type: enum {all output_logic none}

Specifies to generate a report for combinational instance switched during state-propagation. In the output file, it contains the toggle time, instance name, input pin event, and output pin event for each switching event of the combination gate. The possible arguments of this attribute are:

- `all` - Specifies to report instance toggle information for both input and output pins. When this attribute is set to `all`, the switching of both the input and output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are reported in the `voltus_power.stateprop.switchsrc` file.
- `output_logic` - Specifies to report instance toggle information for the output pins only. When this attribute is set to `output_logic`, the switching of the output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are reported in the `voltus_power.stateprop.switchsrc` file.
- `none` - Disables generation of instance toggle report. This is the default option. When this attribute is set to `none`, only switching of flops are reported in the `voltus_power.stateprop.switchsrc` file. The file does not report switching of instances.

This attribute is used for generating switching details of state-propagation and should be specified only for debugging purpose.

The following is a snippet of the `voltus_power.stateprop.switchinst` report:

```
*-----------------------------------------------
*Switch Time   Instance Name   Pin Name   Logic
*-----------------------------------------------
*
   0.097        B11/BUFF         I        rise
   0.097        DFF11/DFF        Q        rise
```

...

Here,

- `Switch Time`: switching time of instances that can be either combinational or sequential
- `Instance Name`: instance name
- `Pin Name`: input/output pin of the instance
- `Logic`: present logic of the pin that can be either "rise" or "fall"

Related Commands

- `report_power`

power_report_instance_switching_list

```
power_report_instance_switching_list filename
```

Default:

Data_type: string

Specifies to generate a report (`voltus_power.stateprop.switchlist`) containing all the switching times for the specified instances when the `power_enable_state_propagation` attribute is set to `true`.

The format of the specified file is:

```
instname1
instname2
...
```

The following command is an example of the state-propagation-based dynamic vectorless flow, and also specifies to generate a report (`voltus_power.stateprop.switchlist`) containing all the switching times for the instances specified in the `inst_list` file:

```
set_db power_method dynamic_vectorless power_enable_state_propagation true power_report_instance_switching_list
inst_list
```

The following is the format of the `voltus_power.stateprop.switchlist` report:

```
*-------------------------------------------------------------------------------------
*Instance Name   Pin Name   Pin Direction   Frequency   Switch Time
*-------------------------------------------------------------------------------------
*
```

Here,

- `Instance Name`: instance name that can be either combinational or sequential
- `Pin Name`: name of the input/output pin of the instance
- `Pin Direction`: indicates whether the pin is an INPUT (I) or OUTPUT (O) pin
- `Frequency`: clock frequency
- `Switch Time`: gives the switching pattern for the instance, that is, <*time1*> <*logic*> <*time2*> <*logic*>. The logic of the pin can be "`r`" or "`f`". `r` indicates rise and `f` indicates fall.

The following is a snippet of the `voltus_power.stateprop.switchlist` report:

```
*-----------------------------------------------------------------------------------------
*Instance Name   Pin Name   Pin Direction   Frequency   Switch Time
*-----------------------------------------------------------------------------------------
*
   DFF11/DFF        QN        O              100       0.106 f 10.107 r
   DFF11/DFF        Q         O              100       0.097 r 10.099 f
   DFF11/DFF        CP        I              100       0.059 r 5.056 f 10.059 r 15.056 ...
...
```

Related Commands

- report_power

power_report_library_usage

```
power_report_library_usage {true | false}
```

Default: `false`

Data_type: bool

Specifies to report library usage statistics in a text report. The report contains information such as, cell name, library format used (CCSP/ECSM/NLPM) for each cell, library name, and so on. This attribute allows you to perform library integrity checking for debugging purpose.

The format of the text report with sample content is given below:

Example:

```
Instance Cell  Net       Pin     Operational  Library   Library     Missing        Library
Name     Name  Name      Name     Voltage     Voltage   Format      Tables           Name
INV_178  INV   VDD,VDDA  VDDG,VDD  0.9        1.0       NLDM      leakage,timing     A
INV_179  INV   VDDC      VDDG      0.75       0.75      CCSP,NLDM   leakage          B
```

Related Commands

- report_power

power_report_missing_bulk_connectivity

```
power_report_missing_bulk_connectivity {true | false}
```

Default: `false`

Data_type: bool

Specifies to report missing bulk pins in the missing input report (`power_report_missing_input`).

Usage Guidelines:

This attribute is not applicable to the `power_method static power_static_netlist verilog` flow.

Related Commands

- report_power

power_report_missing_input

```
power_report_missing_input {true | false}
```

Default: `false`

Data_type: bool

Specifies to report missing netlist information in input files, such as library, LEF/DEF, PGV, SPEF, TWF, and missing logical connectivity for a cell instance (PGNET).

The name of the report file is: `<prefix>_missingdata.<spef|lefdef|lib|pgv|twf|pgnet>`

Usage Guidelines:

This attribute is not applicable to the `power_method static power_static_netlist verilog` flow.

Related Commands

- `report_power`

power_report_missing_nets

```
power_report_missing_nets {true | false}
```

Default: `false`

Data_type: bool

Controls reporting of missing nets in both static and dynamic power analysis.

This attribute controls reporting for TWF and activity files in static analysis, and controls reporting for TWF/SDC/SPEF and activity files in dynamic analysis.

Related Commands

- `report_power`

power_report_scan_chain_statistics

```
power_report_scan_chain_statistics {true | false}
```

Default: `false`

Data_type: bool

Specifies to generate a report file called `voltus_power.scanchain.stat` during scan mode analysis that contains scan chain statistics.

The report has 2 parts:

- **Global Statistics**: this section has the cumulative statistics of all scan chains in the design
- **Chain Statistics**: this section has the statistics for each scan chain in the design

The Global and Chain Statistics sections have the following information:

- `Total`: total instances specified in the control file or in a scan chain
- `AssignedFF`: flip-flops defined in the scan control file which exist in the design

- `SwitchedFF`: flip-flops that have switching event at the output
- `Non-ScanFF`: flip-flops that have no scan-out pin
- `DuplicatedFF`: flip-flops that are specified in multiple chains
- `StateConflictFF`: flip-flops that have conflicting state assignment

Related Commands

- `report_power`

power_report_statistics

```
power_report_statistics {true | false}
```

Default: `false`

Data_type: bool

Reports a set of statistics on the instance power, instance power density, clock power, or transition density. This attribute is useful in viewing and debugging user defined activity factors.

Related Commands

- `report_power`

power_report_time_display_fraction_digits

```
power_report_time_display_fraction_digits value
```

Default: `-1`

Data_type: int

Sets the number of fractional digits or decimal places displayed for the instance switching time values in the `voltus_power.stateprop.switchinst` file. This attribute displays fractional digits that show the exact time difference if there are two events (rise/fall) of an instance switching at the same time.

Related Commands

- `report_power`

power_report_twf_attributes

```
power_report_twf_attributes {summary| detailed}
```

Default:`""`

Data_type: string

Controls the generation of the TWF attribute report based on the TWF attributes that are specified using the `set_twf_attribute` command.

The possible arguments are:

- `summary` - generates a summarized static timing analysis report "`voltus_power.twf_attribute.summary`" that displays the number of user-defined TWF attributes (delay, slew, and slack values for both the rise and fall transitions) and the number of annotated TWF attributes for dynamic power analysis. The following is a snippet of the report:

```
STATISTICS:
                    User Defined            Annotated               Annotated/Total(%)
shift_arrival_time   5                       4                       80
rise_slew            2                       2                       100
fall_slew            2                       2                       100
rise_delay           5                       4                       80
fall_delay           5                       4                       80
rise_slack           2                       2                       100
fall_slack           2                       2                       100
```

- `detailed` - generates a detailed static timing analysis report "`voltus_power.twf_attribute.detailed`" that includes the summary report, and a detailed report of the rise and fall transitions for the TWF attributes of each annotated instance/pin. It also includes the shift arrival time report.

  This argument also allows you to generate a report `voltus_power.twf.rpt` containing the TWF statistics to highlight clock-frequency and delay used for the static and dynamic power calculation.

  The TWF statistics report is generated only if:

  - the `read_twf` command is specified before specifying the `power` category attributes.
  - the `power_method static power_static_netlist def` or `power_method dynamic_vectorless | dynamic_vectorbased` command is specified.

  The format of the `voltus_power.twf.rpt` report is:

```
# Voltus TWF Statistics
# Units: Freq (Hz), Delay (second), Slew (second)
# TWF File List:
#      TWF_file1
#      TWF_fileN

USER DEFINED TWF ATTRIBUTE STATISTICS
<Clock Freq> <Min Rise_Delay> <Rise Delay> <Max Rise Delay> <Min Fall Delay> <Fall Delay> <Max Fall Delay>
<Net>
```

  The `USER DEFINED TWF ATTRIBUTE STATISTICS` section is included in the report only if the TWF attributes are defined using the `set_twf_attribute` command.

Example :

#check all instances of inst_C1 get shifted by 3ns

```
set_twf_attribute -pin inst_C1 -shift_arrival_time 3
```

#check all instances of inst_D1 get shifted by 3ns

```
set_twf_attribute -net inst_D1 -shift_arrival_time 3
```

```
set_db power_method dynamic_vectorless ; set_db power_write_db false ; set_db power_disable_static false ;
set_db power_enable_state_propagation true; set_db power_current_generation_method peak ; set_db
power_grid_libraries /../library_pv.cl ; set_db power_report_twf_attributes summary
```

Related Commands

- `report_power`

power_reuse_flop_ranking_data

```
power_reuse_flop_ranking_data directoryname
```

Default: ""

Data_type: string

Specifies the name of the directory where the flop ranking data to be reused is placed.

Related Commands

- report_power

power_reuse_flop_ranking_data_hier

```
power_reuse_flop_ranking_data_hier blockname
```

Default: ""

Data_type: string

Allows you to choose the block/hierarchy whose ranking data is to be used for the block-level analysis. This attribute is used when there are multiple instantiations of the same DEF block.

Related Commands

- report_power

power_scale_to_sdc_clock_frequency

```
power_scale_to_sdc_clock_frequency {true | false}
```

Default: false

Data_type: bool

Specifies to enable the VCD clock frequency to be scaled up to the SDC clock frequency. This allows you to match the VCD clock to the SDC clock to run analysis. By default, the clock frequency is scaled based on the activity file. When you set the power_scale_to_sdc_clock_frequency attribute to true, the power report and current waveforms apply the after-scaled clock frequency.

The default behavior for clock frequency scaling in the RTL and Gate level VCD flows is different, as described below:

- **RTL VCD** - By default, the clock frequency is scaled to the SDC clock frequency (power_scale_to_sdc_clock_frequency true) when the RTL VCD flow is enabled (power_enable_rtl_dynamic_vector_based true).
  You can disable clock frequency scaling by setting the power_scale_to_sdc_clock_frequency attribute to false.
- **Gate VCD** - By default, scaling of the clock frequency is disabled (power_scale_to_sdc_clock_frequency false) .
  You can enable the VCD clock frequency to be scaled up to the SDC clock frequency by setting the power_scale_to_sdc_clock_frequency attribute to true. This works only when power_use_zero_delay_vector_file is set to true, which means that the gate VCD is a zero-delay gate VCD.

Related Commands

- report_power

power_scan_chain_activity

```
power_scan_chain_activity {design_name  chain_name  activity_number} | {vectorless  activity_number}
```

Default: ""

Data_type: string

Specifies to either provide vectorless activity for all scan chains or provide a specific activity for a specific chain. This attribute allows you to specify vectorless or user-defined activity to scan chains.

The following are the two use models:

- Generate the scan control file for a specific chain:
  ```
  set_db power_scan_chain_activity {design_name  chain_name  activity_number}
  ```
- Generate the scan control file for all scan chains:
  ```
  set_db power_scan_chain_activity {vectorless  activity_number}
  ```

The `power_scan_chain_activity` attribute is supported only in the XP mode.

Related Commands

- report_power

power_scan_chain_name_pattern

```
power_scan_chain_name_pattern {design_name  chain_name  pattern ...}
```

Default: ""

Data_type: string

Specifies to automatically generate a scan-chain control file from a scan-chain based DEF file for the shift mode scan-chain analysis. This attribute allows you to create a control file for a large-sized DEF containing long chains or for multiple-DEF files. The generated scan-chain control file is saved in the power output directory.

The use model of the attribute is:

- `power_scan_chain_name_pattern {all pattern}` - generate scan control file for all scan chains.
  Example: `power_scan_chain_name_pattern {all 11010}`
- `power_scan_chain_name_pattern {design_name  chain_name  pattern ...}` - generate scan control file for a specific chain.
  Here, "*design_name*" is the design name of the block DEF, or the "top" keyword that means the top-level DEF.
  Example: `power_scan_chain_name_pattern {top scan_chain_1 11010 BlockA scan_chain_1 10010 ... }`

Using this attribute, you can generate the control file for both flat and hierarchical DEFs. For more information, refer to Scan Control File Format.

Related Commands

- `report_power`

power_scan_control_file

```
power_scan_control_file  filename
```

Default: `""`

Data_type: string

Specifies the scan control file name required to run the Scan Mode Analysis flow.

The scan control file contains details about the scan analysis mode used for testing. For more information, refer to Scan Control File Format.

Related Commands

- `report_power`

power_scan_multi_bit_flop_chain_type

```
power_scan_multi_bit_flop_chain_type type
```

Default: `liberty`

Data_type: string

Specifies the type of multi-bit scan flip-flop.

- `liberty` - specifies to use the syntax in Liberty to decide the multi-bit flip-flop type.
- `serial` - specifies to pick the first flop as the starting flop, and treat the scan chain as serial, regardless of .lib.
- `parallel` - specifies to treat the scan chain as parallel, regardless of .lib.

Related Commands

- `report_power`

power_settling_buffer

```
power_settling_buffer value
```

Default: `""`

Data_type: string

Specifies the settling buffer time (in ps unit) between multiple windows of a VCD file. This attribute is used when you specify multiple pairs of the start time and end time for the non-overlapping multiple windows specified in an activity file (using the `read_activity_file` command).

Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps. You must specify the buffer value without unit.

Related Commands

- report_power

power_show_cell_usage_stats

```
power_show_cell_usage_stats {true | false}
```

Default: false

Data_type: bool

Provides cell usage statistics alongside power values, indicating the number of cells used from a specific library and the corresponding power percentage. This helps identify libraries with the greatest and least impact on power consumption.

You must ensure that this attribute is specified before using the report_power command, as demonstrated below:

```
set_db power_show_cell_usage_stats true
report_power -out_file p1.rpt
```

Following is a snippet from the report:

```
Cell usage statistics:
Library j1os_h , 88 cells ( 7.351713%) , 6.56434e-06 mW ( 0.554089%  )
Library j1os_l , 123 cells ( 10.275689%) , 0.000799809 mW ( 67.511081%  )
Library j1os , 986 cells ( 82.372597%) , 0.000378334 mW ( 31.934830%  )
```

The power_show_cell_usage_stats attribute is applicable only to the static power calculation, Verilog-based (power_static_netlist verilog) flow.

Related Commands

- report_power

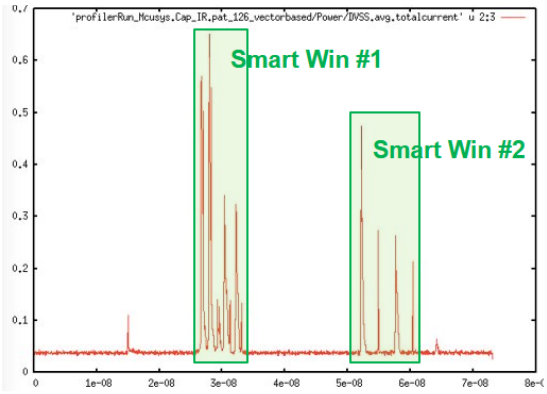
power_smart_window

```
power_smart_window {true | false}
```

Default: false

Data_type: bool

Specifies to enable the smart window feature to extract peak power scenarios from a vector-based activity file. A smart window is a variable size window to capture maximum regions of high power dissipation. An activity file can have multiple variable-sized regions that are ranked in the descending order of power metric.



The use model of the `power_smart_window` attribute is:

```
set_db power_smart_window true
set_db power_worst_step_size value
set_db power_worst_window_type [ power | activity | delta_power | delta_activity | ir | critical_inst_power ]
```

Related Commands

- `report_power`

power_split_bus_power

```
power_split_bus_power {true | false}
```

Default: `false`

Data_type: bool

A value of `false` applies the internal power number to each individual bit.

A value of `true` divides the internal power number by bus width, before applying it to the bus bit.

Related Commands

- `report_power`

power_start_time_alignment

```
power_start_time_alignment {true | false}
```

Default: `true`

Data_type: bool

Specifies to align all the signals of the activity file at time zero. When set to `true`, the simulation period is `max(endtime - starttime)`. When it is `false`, the simulation period is `max(endtime)- min(startime)`.

⚠ This attribute is not supported in the vector profiling flow. In the vector profiling flow, the start time will always be the `min(starttime)` and the end time will always be `max(endtime)`.

Related Commands

- report_power

power_state_dependent_leakage

```
power_state_dependent_leakage {true | false}
```

Default: `true`

Data_type: bool

When set to false, power analysis performs state independent leakage power calculation.

Related Commands

- report_power

power_stateprop_ignore_unannot_pins

```
power_stateprop_ignore_unannot_pins {true | false}
```

Default: `false`

Data_type: bool

Specifies to ignore unannotated pins in the functional logic of output pins for the RTL propagation flow. This attribute is set to `true` if there are unannotated input pins for instances and such pins are used in the functional logic of output pins that are propagated. For example output pin Q is propagated with functional logic A & B where A is unannotated and might cause wrong propagation for the Q pin.

Related Commands

- report_power

power_static_multi_mode_scenario_file

```
power_static_multi_mode_scenario_file  filename
```

Default: `""`

Data_type: string

Specifies a file with information on instance scaling scenario for the static multi-mode IR analysis flow. When this attribute is specified, Power Analysis creates the multi-mode static current file that is used during rail analysis. The format of the input file is:

```
INST_NAME       PIN_NAME        SCALE_FACTOR_1       …..   SCALE_FACTOR_N
```

Here, the first column indicates the macro instance name, the second column indicates the pin name, followed by multiple current scaling factors. Each column of the scaling factor is referred to as a scenario. You must specify positive values only for scale factors.

The following is a snippet from the specified input file:

```
top/macroA     VDD     1     0.5     0.5     1
top/macroA     VDDM    0.5   0.5     0.5     1
top/macroB     VDD     0.5   1       0.5     0.5
```

If scale factors are not specified for certain scenarios, the tool will automatically use the instance's last stored scale factor value to complete current file generation.

In the following example, scale factor value `0.45` will be used in the 3rd and 4th scenario for instanceC, and scale factor value `0.35` will be used in the 4th scenario for instanceD:

```
instanceA VDD 0.45 0.55 0.65 0.31
instanceB VDD 0.31 0.5 0.67 0.21
instanceC VDD 0.25 0.45
instanceD VDD 0.31 0.25 0.35
```

Related Commands

- `report_power`

power_static_netlist

```
power_static_netlist  {verilog | def}
```

Default: `verilog`

Data_type: string

Specifies to perform static power analysis using a Verilog or DEF only neltist.

By default, the static power analysis is performed using the Verilog netlist as the primary netlist. Using this attribute, you can specify DEF when a DEF netlist with logical connectivity is available, and use it as primary and not consider the Verilog netlist.

This attribute is also enabled when `set_rail_analysis_config -report_power_in_parallel` is `true` for static power computation.

Related Commands

- `report_power`

power_switching_power_on_rise_only

```
power_switching_power_on_rise_only {true | false}
```

Default: `false`

Data_type: bool

Specifies to perform switching power analysis only for rise toggles.

The attribute is only applicable to the event-based power calculation flow (`power_method event_based`). This attribute impacts only the static power report, and does not affect the dynamic power results.

Related Commands

- `report_power`

power_thermal_input_file

```
power_thermal_input_file  file
```

Default: ""

Data_type: string

Specifies to read a power map file to perform thermal analysis. Thermal input file contains die area divided into tiles, and temperature value for each tile. This temperature of each tile is used to calculate power of all the instances that fall in that tile's bounding box.

The format of the thermal input file is:

```
DIE_TEMPERATURE_MAP
DIE_AREA   xmin(um) ymin(um) xmax(um) ymax(um) nx(int) ny(int)

NUMBER_OF_LAYERS    nLayers(int)
LAYER    layer_name(string)
TEMPERATURE

XCOOR    x0(0)    x0(1)    x0(2)... ...x0(nx-1)
YCOOR    y0(0)    y0(1)    y0(2)... ...y0(ny-1)

T0(0,0)    T0(1,0)    ...    T0(nx-1,0)
T0(0,1)    T0(1,1)    ...    T0(nx-1,1)

:
T0(0,ny-1)    T0(1,ny-1)         T0(nx-1,ny-1)
END_TEMPERATURE
END_LAYER
END_DIE_TEMPERATURE_MAP
```

where,

- `DIE_AREA` specifies the coordinates of the four points of die outline as well as the tile number in X and Y direction.
- `NUMBER_OF_LAYERS` is the number of layers in each die instance
- `LAYER` specifies the LEF layer name
- XCOOR and YCOOR are the coordinates of each tile
- `T0(0,0)` …. is the temperature value assigned to a tile at a specific coordinate

Related Commands

- `report_power`

power_thermal_leakage_temperature_scale_table_file

`power_thermal_leakage_temperature_scale_table_file` *file*

Default: ""

Data_type: string

Specifies a file containing the user-defined temperature values and temperature scale factors for each instance. This attribute allows you to use the current file (.ptiavg) with re-adjusted leakage power values to perform thermal-aware IR drop analysis. The `power_thermal_leakage_temperature_scale_table_file` attribute must be specified with the `power_thermal_input_file` attribute to import the temperature map file generated by Celsius.

When the new attribute is specified, the generated power will include the scaled leakage power values.

The following is the format of the temperature-aware leakage scaling table:

```
#Type    Instance_Name        temp1             temp2              temp3              temp4
(Type: Lib/Cell/inst)

inst     inst_name1     scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
inst     inst_name2     scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
```

File Format Example:

```
#Type     Name        25    50    125    150

inst     ring/b213    1     2     5      10
inst     ring/b212    1     2     5      10
inst     ring/b211    1     2     5      10
inst     ring/b210    1     2     5      10
```

Related Commands

- report_power

power_to_x_transition_factor

```
power_to_x_transition_factor value
```

Default: 0.5

Data_type: double

Specifies how 0/1 to X transitions are counted.

Related Commands

- set_power

power_to_z_transition_factor

```
power_to_z_transition_factor value
```

Default: 0.25

Data_type: double

Specifies how 0/1 to Z transitions are counted.

Related Commands

- set_power

power_transition_factor_based_duty

```
power_transition_factor_based_duty {true | false}
```

Default: false

Data_type: bool

Specifies that duty for X and Z toggles is calculated using the X and Z transition factor of toggle specified using the following

attributes:

- `power_from_x_transition_factor`
- `power_from_z_transition_factor`
- `power_to_x_transition_factor`
- `power_to_z_transition_factor`

The default value of `false` specifies to use the duty value given with the `set_default_switching_activity`/`set_switching_activity -duty` parameter.

Related Commands

- `report_power`

power_transition_time_method

```
power_transition_time_method  {min | avg | max}
```

Default: `max`

Data_type: string

Specifies the minimum, maximum, or average transition time method that will be used with the integrated timer or the external TWF.

The default value is:

- min for static power analysis using the Verilog netlist (-static_netlist verilog).
- avg for static power analysis using the DEF netlist (-static_netlist def) and dynamic power analysis.

Related Commands

- `report_power`

power_twf_delay_annotation

```
power_twf_delay_annotation {min | avg | max}
```

Default: `avg`

Data_type: string

Allows you to choose between min, max, or avg arrival times from the timing window file (TWF). The arrival time is annotated to the vectors processed from the RTL VCD/FSDB Zero-Delay flows.

Related Commands

- `report_power`

power_twf_load_cap

```
power_twf_load_cap {min | avg | max | ignore}
```

Default: `max`

Data_type: string

Allows you to ignore or select minimum, maximum, or average value of TWF external load or capacitance for power calculation.

The `ignore` argument ignores the `EXTERNAL LOAD` statements during TWF parsing in the power calculation flow. This argument allows you to control the inclusion of external load on the block peripheral or output ports. It may be useful to specify this argument when user-specified load capacitance values (`power_external_load_config_file`) are being used.

Related Commands

- `report_power`

power_unified_power_switch_flow

```
power_unified_power_switch_flow {true | false}
```

Default: `true`

Data_type: bool

Specifies to enable the unified analysis of the always-on and switch nets. When set to `true`, the power-up and always-on analyses are performed in a single flow, wherein the switch nets and always-on nets are simulated together resulting in improved performance and accuracy.

Related Commands

- `report_power`

power_use_cell_leakage_power_density

```
power_use_cell_leakage_power_density {true | false}
```

Default: `true`

Data_type: bool

Specifies to use library leakage density times area (`default_leakage_power_density`), if defined, instead of the library's default cell leakage power (`default_cell_leakage_power`). This is used when leakage power for cells are missing or not defined.

Related Commands

- `report_power`

power_use_effective_load_cap_method

```
power_use_effective_load_cap_method {lumped | max | avg | min}
```

Default: `lumped`

Data_type: string

Specifies the method to determine the slew ratio. This parameter has the following arguments:

- `lumped` - the slew ratio is 1, which indicates that a lumped capacitance is used.

- `max` - the slew ratio is calculated as max(slew of all drivers)/ max(slew of all receivers).
- `min` - the slew ratio is calculated as min(slew of all drivers) / min(slew of all receivers).
- `avg` - the slew ratio is calculated as avg(slew of all drivers) / avg(slew of all receivers).

Related Commands

- `report_power`

power_use_fastest_clock_for_dynamic_scheduling

```
power_use_fastest_clock_for_dynamic_scheduling {true | false}
```

Default: `false`

Data_type: bool

Specifies to use only the respective fastest clock associated with each net or pin to schedule events for a given simulation period.

Related Commands

- `report_power`

power_use_lef_for_missing_cells

```
power_use_lef_for_missing_cells {true | false}
```

Default: `false`

Data_type: bool

Allows a combination of LEF and PGV in the dynamic analysis flow. If there are cells that are common to LEF and PGV, the PGV cell will get the precedence.

Related Commands

- `report_power`

power_use_only_ddv_current

```
power_use_only_ddv_current {true | false}
```

Default: `true`

Data_type: bool

Controls the use of static current of a detailed dynamic power-grid view (DDV PGV) cell for static power calculation. The attribute is set to `true` by default, that is, the DDV PGV static current is not used in the static power calculation flow. If you want to use the DDV PGV static current for static power calculation, set this attribute to `false`.

Related Commands

- report_power

power_use_physical_partition

```
power_use_physical_partition {true | false}
```

Default: `false`

Data_type: bool

Enables tile or area-based partitioning of a design for distributed processing. When this attribute is specified, the top-level design is divided into physical tiles, which are then grouped to ensure that each group or partition has the same instance count. Each partition is considered and processed as a separate job.

This attribute is only applicable to the event-based power analysis flow (`power_method event_based`) in the XP mode. The `power_use_physical_partition` attribute must be specified with the `power_partition_count` attribute.

Example:

The following command specifies to enable area-based partitioning, and distribute the event-based analysis into four jobs:

```
set_db power_method event_based power_enable_xp true power_use_physical_partition true power_partition_count 4
report_power -distribute
```

Related Commands

- report_power

power_use_twf_stable_randomized_arrival_delay

```
power_use_twf_stable_randomized_arrival_delay  {none | data_only | all}
```

Default: `none`

Data_type: string

Specifies to annotate randomized arrival time based on net names. The attribute ensures that when there is a change in design, the net delay is not impacted. This attribute is supported in both the static and dynamic flows.

The possible arguments are:

- `none` - disables randomized timing window annotation.
- `all` - performs randomized annotation for both the clock and data nets.
- `data_only` - performs randomized annotation only for the data nets.

Related Commands

- report_power

power_use_zero_delay_vector_file

```
power_use_zero_delay_vector_file {true | false}
```

Default: `false`

Data_type: bool

Enables zero delay mode in vector-based dynamic analysis to avoid pessimism in current estimation. When set to `true`, the software reads the cycle accurate (zero delay) VCD/FSDB file and adds the timing/delay information from the Timing Analyzer or external TWF file to the VCD/FSDB file.

Related Commands

- `report_power`

power_vector_based_multithread

```
power_vector_based_multithread {true | false}
```

Default: `true`

Data_type: bool

Specifies to enable multi-threading for the VCD/FSDB based dynamic vector-based flows.

To enable multi-threading in the vector-based flows, set this attribute to `true`.

Related Commands

- `report_power`

power_vector_profile_mode

```
power_vector_profile_mode {activity | event_based | power_density | transient}
```

Default: `event_based`

Data_type: enum {activity event_based power_density transient}

Specifies the method used to perform vector profiling. The possible arguments are:

- `activity` - performs activity profiling. This attribute generates only the worst activity report, and does not generate the average power report. The name of the file is as specified using the `report_power -out_file` parameter.
- `event_based` - generates the average power report. This is the average power of all events in the user-specified window. This report includes a summary based on the power components, cell type, power rail, and clock network. The worst window report is captured in a file specified with the `report_power -out_file` parameter, and the report is suffixed by '`.avgpower`'. When `-out_file` is not specified, the `vectorprofile.report.avgpower` report file is created.
- `power_density` - performs area or power density profiling. For this method, power is reported for each tile of the design, where design can be considered to be physically divided into equal area tiles. By default, a design is divided in 10x10 tiles.
- `transient` - generates the event-based average power report and current files for the worst power window.

Related Commands

- `report_power`

power_worst_case_vector_activity

```
power_worst_case_vector_activity  {true | false}
```

Default: `false`

Data_type: bool

Specifies to use the worst activity value when multiple vectors are specified in the static power calculation flow.

Related Commands

- `report_power`

power_worst_step_size

```
power_worst_step_size value
```

Default: `""`

Data_type: string

Specifies the step size or resolution for vector profiling. Activity/ Power profiling is carried out for every step size that you specify.

When this attribute is used with `power_worst_window_size`, the specified step size is the resolution at which window sliding is performed. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.

Related Commands

- `report_power`

power_worst_window_count

```
power_worst_window_count value
```

Default: `1`
Data_type: int

Specifies the number of worst windows from the last vector profile run to be used for dynamic power analysis. It provides n top windows with maximum activity/power/IR. This attribute is used when you need to combine dynamic power analysis and vector profiling for multiple worst-case windows in a single run.

Related Commands

- `report_power`

power_worst_window_coverage

```
power_worst_window_coverage { top | tile }
```

Default: ""
Data_type: string

Specifies how the worst window type is applied for multi-window analysis in the vector profiling flow. The following are the possible options of this attribute:

- `top`: worst window selection for the full design
- `tile`: interval selection as per the worst tile power

Use Model:

```
set_db power_method event_based power_worst_window_coverage tile
report_power -out_file power.rpt -power_density_tiles 100
```

This flow should be run with `power_method event_based`.

When the `power_worst_window_coverage tile` attribute is specified, you should include any one of the following parameters to specify the total number of tiles:

- `report_power -power_density_tiles_size " 15 15 "`
- `report_power -power_density_tiles_row_column " 2 2 "`
- `report_power -power_density_tiles 100`

The software generates the `power.rpt.density` report that has the power density tiles profile report with an additional section "Intervals covering unique tiles having worst power density".

Related Commands

- `report_power`

power_worst_window_reports

```
power_worst_window_reports {full | worst | both}
```

Default: `full`

Data_type: string

Specifies to generate static power analysis reports for the specified worst window type. The possible arguments are:

- `full`: creates the static power report for the full simulation window specified with the `-start`/`-end` parameters of the `read_activity_file` command.
- `worst`: creates the static power report for the worst window only.
- `both`: creates the static power report for both the full simulation and worst windows.

The worst window report is created with '**.1**' added in the file name, as shown below:

- Static reports for full simulation: `power.rpt.avgpwr`, `power.rpt.instpwr`
- Static reports for worst window: `power.rpt.1.avgpwr`, `power.rpt.1.instpwr`

Related Commands

- `report_power`

power_worst_window_size

```
power_worst_window_size value
```

Default: `""`

Data_type: string

Specifies the duration of dynamic analysis. This attribute is used for sliding window vector profiling that involves sliding the profiling windows by using a very small resolution value to get the window with the worst current profile. `power_worst_window_size` must be specified with `power_worst_step_size`.

The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.

⚠ This attribute is not supported with net-based activity profiling (`set_db power_method vector_profile power_vector_profile_mode activity`).

Related Commands

- `report_power`

power_worst_window_type

```
power_worst_window_type  {power | activity | delta_power | delta_activity | ir | critical_inst_power}
```

Default: `""`

Data_type: string

Specifies the window type to be used for multi-window analysis. The possible arguments are:

- `power` - worst instance power
- `activity` - worst instance activity
- `delta_power` - worst dp/dt (rate of change of power)
- `delta_activity` - worst da/dt (rate of change of activity)
- `ir` - worst IR
- `critical_inst_power` - worst power for critical/IR-sensitive instance (worst timing and IR)

Related Commands

- `report_power`

power_write_bbox

```
power_write_bbox  {true | false}
```

Default: `false`

Data_type: bool

Specifies to save the boundary box of each instance, its placement, and rotation at the top level. When set to `true`, generates the power database with each instance boundary box, placement, and rotation information at the top level.

Related Commands

- `report_power`

power_write_db

```
power_write_db {true | false}
```

Default: `false`

Data_type: bool

A value of `true` creates a binary database of power results.

Related Commands

- `report_power`

power_write_default_pti_files

```
power_write_default_pti_files {true | false}
```

Default: `true`

Data_type: bool

For instances that are not hooked to any power/ground rail, the current for these instances are reflected in the default static current files:

- `static_default_ground_rail.ptiavg`
- `static_default_power_rail.ptiavg`

Related Commands

- `report_power`

power_write_dynamic_currents

```
power_write_dynamic_currents {true | false}
```

Default: `false`

Data_type: bool

Specifies to create dynamic current files either for full simulation duration or for user-specified duration. This attribute is only applicable to the event-based power analysis flow (`power_method event_based`).

Example:

```
set_db power_method event_based power_write_dynamic_currents true power_enable_state_propagation true
read_activity_file sim.vcd -format vcd
report_power -out_file power.rpt
```

Related Commands

- `report_power`

power_write_gui_db

```
power_write_gui_db {true | false}
```

Default: `true`

Data_type: bool

Specifies to create a binary database of power results in the XP mode. When set to `true`, the tool creates sliced power database file for each rail partition and saves it to the `gui.db` sub-directory under the power output directory (example: `power_dir/part_*/gui.db/rail_part_*/power.x.db` (x = 1, 2, ..., 14). The generated power database allows you to perform GUI interactive debugging using the *Power & Rail Plots* and *Power Debug* forms.

Related Commands

- `report_power`

power_write_profiling_db

```
power_write_profiling_db {true | false}
```

Default: `false`

Data_type: bool

Writes out profiling databases which can later be viewed as histograms using the SimVision interface. You can specify this attribute during both the activity and power vector profiling. The profiling databases support instance level and power/ground net histograms. It creates an output file name with the `*.trn` extension.

Related Commands

- `report_power`

power_write_simulation_db

```
power_write_simulation_db  {true | false}
```

Default: `false`

Data_type: bool

Specifies to save functional waveforms for each instance when parsing an activity file. When set to `true`, creates SimVision waveform database (SHM database) for toggle simulation used during power analysis, which can later be viewed as histograms using the SimVision interface.

The simulation database (`shmdb.trn`) is generated in the power analysis output directory.

Related Commands

- `report_power`

power_write_static_currents

```
power_write_static_currents {true | false}
```

Default: `false`

Data_type: bool

A value of `true` generates the current data files per net.

⚠ When you perform static power calculation in the dynamic power analysis flow, you can use either the `power_disable_static` or `power_write_static_currents` attribute to enable static power calculation and write static current file. If only `power_disable_static` is set to `false` and `power_write_static_currents` is not set, the tool automatically sets `power_write_static_currents` to true. Similarly, if `power_write_static_currents` is set to true, and `power_disable_static` is not set, the tool automatically sets `power_disable_static` to `false`.

Related Commands

- `report_power`

power_x_count_transition_using_3_states

```
power_x_count_transition_using_3_states {true | false}
```

Default: `false`

Data_type: bool

Specifies to calculate the toggle rate using 3 states, pre-previous, previous and current states.

Related Commands

- `report_power`

power_x_transition_factor

```
power_x_transition_factor value
```

Default: `0.5`

Data_type: double

When using VCD/FSDB, one can specify how transitions to and from X are counted. These transitions are defined as shown in Table 1 below. Transitions to and from the X state will be treated as full transitions multiplied by the specified factor.

⚠ 0 -> X -> 1 or 1 -> X -> 0 count as full transitions and they are not controlled by the attribute.

Related Commands

- `report_power`

power_z_transition_factor

```
power_z_transition_factor value
```

Default: `0.25`

Data_type: double

When using VCD/FSDB, one can specify how transitions to and from Z are counted. These transitions are defined as shown in Table 1 below. Transitions to and from the Z state will be treated as full transitions multiplied by the specified factor.

Note: 0 -> XZ-> 1 or 1 -> Z-> 0 count as full transitions and they are not controlled by the `attribute`.

Related Commands

- `report_power`

Table **1: Transitions from/to X and Z values and**

Transition	Definition	Default
`to/from X`	`[ 0 or 1 or Z] <-> X`	`0.5`
`to/from Z`	`[ 0 or 1 ] <-> Z`	`0.25`

power_zero_delay_vector_toggle_shift

```
power_zero_delay_vector_toggle_shift value
```

Default: `""`

Data_type: string

Specifies to shift the current waveform in the zero-delay VCD flow. This attribute is useful in removing the idle period for the current waveform result with a long idle period (small flat current) before the first high current peak because of delay annotation. The attribute allows to offset the idle period from the delay values.

The default value is 0ns. The supported time units are s, ms, us, ns, and ps. The default unit is ns. You can specify a negative or a positive value. A positive value means right shift in the timeline or add more delay to the original arrival time, and a negative value means left shift in the timeline or trim the unwanted idle period.

The `power_zero_delay_vector_toggle_shift` attribute enables you to make a more realistic current waveform that can be easily aligned with the other current waveforms.

Note: This attribute is similar to the `set_twf_attribute -shift_arrival_time` parameter, with the only difference that `power_zero_delay_vector_toggle_shift` supports both positive and negative values while `-shift_arrival_time` supports only a positive value.

Related Commands

- `report_power`

Scan Control File Format

The use model for the Scan Mode Analysis flow is:

```
set_db power_method dynamic_vectorbased power_scan_control_file filename
```

The format of the scan control file is:

```
MODE = [ SHIFT | VECTORLESS]
VECTORLESS_GLOBAL_ACTIVITY = <factor> [Between 0 - 1]
<chain_name> <flop_inst> <initial_state> <master_cell_name> <scale> <mode> <scan_in_pin> <scan_out_pin>
<scan_en_pin>
```

Note that <`chain_name`> is optional, <`initial_state`> is optional in vector-less mode, and <`master_cell_name`>, <`scale`>, <`mode`>, <`scan_in_pin`>, <`scan_out_pin`>, <`scan_en_pin`> are optional and only needed for the first internal scan flop inside one macro.

Where,

- `Mode` specifies the mode used for scan design analysis. The two modes are: `SHIFT` and `VECTORLESS`.
- `VECTORLESS_GLOBAL_ACTIVITY` specifies the activity specification used in the `VECTORLESS` mode.
- `chain_name` is the name of the scan chain.
- `initial_state` is the initial state of all scan flops.
- `master_cell_name` is the name of the macro.
- `scale` is the scaling factor applied to the current.
- `mode` is mode name of the scan mode in the PGV view.
- `scan_in_pin` is the macro's scan_in pin name.
- `scan_out_pin` is the macro's scan_out pin name.
- `scan_en_pin` is the macro's scan_enable pin name.

propagate_activity

```
propagate_activity
[-help]
[-set_net_frequency {true | false}]
```

Propagates the activity file in the database after it is read in using the `read_activity_file` command. This command propagates the activity for all primary inputs, nets, and other devices in the design whose activity has not been previously defined through user attributes.

`propagate_activity` can be used to propagate activities in the design and check for activity annotation before running power calculation. This is not a mandatory step as execution of `report_power` will also do the same and then do power calculation based on the propagated activities.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `propagate_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: man `propagate_activity`
-set_net_frequency {true | false}	
	Specifies that activity from the generated TCF file needs to be used to capture net frequency for signal EM analysis. When this parameter is set, you do not have to read the TCF file to set net frequency.

Example

- The following commands read and propagate the activity file in the database:

```
read_activity_file
propagate_activity
```

Related Topics

- "Signal ElectroMigration" in the *Voltus User Guide*

query_power_data

```
query_power_data
[-help]
[<pti_files <file1[file2] ...>>]
[-average_current]
[-clock_average]
[-write_pwl]
[-frame <string>]
[-insts <inst_list>]
[-inst_list_file filename]
[-list]
[-out_dir <string>]
[-peak_current]
[-peak_time]
[-pin]
[-time_steps]
[-verbose]
[-total_current <input_current_file> ]
[-output_tap_name <tap_name>]
```

The `query_power_data` command allows you to query the binary current files (.ptiavg/.ptipeak) that Power Calculation/Rail Analysis creates. It reads the tap current data in the current files and reports various information for debugging.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`<pti_files <file1[file2] ...>>`	
	Specifies the names of the VDD and VSS instance voltage pti files.
`-average_current`	
	Specifies to write the average current of all instances.
`-clock_average`	
	Specifies to write the clock based current report.
`-write_pwl`	
	Specifies to write the tap current in the PWL format. You can use this parameter to view the contents of the binary data file.
`-frame <string>`	

	Specifies intervals for the clock_average report.
`-insts <inst_list>`	
	Specify the list of instances to be dumped in the ASCII format, one instance per line. If not specified, it prints all instances. The `-insts` parameter is specified with the `-write_pwl`, `-peak_current`, `-peak_time`, and`-average_current` parameters.
`-inst_list_file filename`	
	Specifies a file with the list of instances to be dumped in the ASCII format.
`-list`	
	Specifies to write the tap names from the input file.
`-out_dir <string>`	
	Specifies the name of the output file.
`-output_tap_name <tap_name>`	
	Specifies the name of the output tap in the new current file, which is created using the `-total_current` parameter.
`-peak_current`	
	Specifies to write the peak current of all instances.
`-peak_time`	
	Specifies to write the peak current time of all instances.
`-pin`	
	Specifies to print the pin names with the instance names. The `-pin` parameter is specified with the `-write_pwl` and `-list` parameters.
`-verbose`	
	Specifies to print the report to standard output.
`-time_steps`	
	Specifies to summarize the minimum current, maximum current, average current, and total current for each interval in the data file. You can use this data to estimate which interval in a transient response might contain the worst-case data.
`-total_current <input_current_file> <output_total_current_file>`	

	Specifies to create a new current (.ptiavg) file containing the total current for all taps of a net. This parameter can be used if you have a very large current file generated by the Power Calculation engine and want to see only a total current waveform in the SimVision waveform viewer. *input_current_file* is the current file that will be read in (output from the Power Calculation engine). Use the following syntax to create a total current file: `query_power_data -total_current <input_current_file> -out_dir <output_total_current_file>` `-out_dir` is used to specify a user-defined output current filename, and is not a mandatory parameter. If not specified, the output file will be generated automatically with the name of the input file suffixed with `.ptiavg`.

Example

- The following command creates a total current waveform file `VDD` from the input file `dynamic_VDD.ptiavg`:

```
query_power_data -output_tap_name vdd_tap -total_current dynamic_VDD.ptiavg -out_dir VDD
```

read_activity_file

```
read_activity_file activity_file
[-help]
[-end { time1 time2 ...timen } ]
[-format {vcd | tcf | saif | fsdb | phy | shm}]
[-hierarchy_separator separator ]
[-reset]
[-start { time1 time2 ...timen } ]
[-write_net_freq {true | false}]
[-block block_name ]
[-scope scope_name ]
[-scale_time scalefactor ]
[-start_time_shift value ]
[-rtl {true|false}]
[-name_mapping_rule file]
[-weight value]
[-zero_delay {true|false}]
[-cell cell_name ]
```

Specifies the name and type of activity file to be used as input. This command must be used when specifying activity files for static or dynamic power calculation.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_activity_file` parameter. For a detailed description of the command and all of its parameters, use the man command: man `read_activity_file`
`-block block_name`	
	Specifies the name of the FSDB/SAIF/TCF/VCD block to map a sub-block activity file with a top level Verilog file. Provides the ability to support multiple activity files based upon block instance names. The block argument is added to the FSDB/SAIF/TCF/VCD name before comparing it to the Verilog file.
`-cell cell_name`	

Specifies the master cell name for the activity file. This parameter allows you to support master-cell-based activity specification. That is, you need to specify only the master cell name for the activity file, instead of specifying all the instances that were instantiated from a master cell.

Example

Let us consider a design that has a cell named "super_filter" with four block instances "super_filter1", "super_filter2", "super_filter3",and "super_filter4":

Instance-Based Use Model:

```
read_activity_file -format VCD -block super_filter1
read_activity_file -format VCD -block super_filter2
read_activity_file -format VCD -block super_filter3
read_activity_file -format VCD -block super_filter4
```

Master-Cell-Based Use Model:

```
read_activity_file -format VCD -cell super_filter
```

`-end { time1 time2 ...timen }`

When you use the vector-driven method of performing dynamic instance-based power-consumption calculation, you can specify the part of the activity file from which you want power analysis to use for the power calculation. This option specifies the end time of this range and is used in conjunction with the `-start` option.

Specifies the end time as time-value pairs to report the average power across non-overlapping multiple windows specified in the activity file for a block, level of hierarchy, or other part of the design that you want to analyze. Units are in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`).

Default: The time unit is defined in the first `.lib` file read during design import.

You can specify the `$worst_power_window_end` variable with the `-end` parameter to take the worst case end time from the last vector profiler run.

Not required for TCF and SAIF.

Note: You can read multiple windows in a single VCD file by specifying multiple pairs of the start time and end time for the non-overlapping multiple windows specified in the activity file. The use model for reading multiple windows is:

```
read_activity_file -format VCD design.vcd -start {start1 start2 start3} -end {end1 end2 end3}
```

The *start1/2/3* and *end1/2/3* values should be in the ascending order.

Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched using the `set_power_analysis_mode -settling_buffer` parameter. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.

`-format {vcd | tcf | saif | fsdb | phy | shm}`

Specifies the net activity of the design using an activity file in the VCD, TCF, FSDB, SAIF, PHY, or SHM format. The arguments specified with the `-format` parameter are case-insensitive.

- The VCD file format uses the toggle count information as the basis for the power consumption calculation. These files can be in compressed (zipped) format using the .gz extension.

- The TCF file format contains data on the switching activity of the nets in the design. The switching activity includes the toggle count information and the probability of the net or pin being in the logic 1 state.
  These files can be in compressed (zipped) format using the .gz extension.

  Supports extended TCF which contains transition density for X and Z as shown in the following extended TCF example:

  "out" :"0.0 0" "0.005000 2" "0.0 0"

  The fields are defined as follows:

  Field 1: net or pin name
  Field 2: duty and transition density for transition of 0-1
  Field 3: duty and transition density for transition of X(0-X-1)
  Field 4: duty and transition density for transition of Z(0-Z-1)
- The SAIF file format contains aggregate switching activity information on the nets and/or ports in the design. It does not contain time-based information. This file format is used for quick average power analysis of the design.
- The FSDB file format specifies the net activity of the design using an activity file in FSDB format. Voltus supports the FSDB version up to 6.0.
  Use Model

  ```
  read_activity_file -format fsdb fsdbfile
  report_power
  ```

  (or)

  ```
  report_vector_profile
  ```

  Example

  ```
  read_activity_file -format fsdb test.fsdb -scope top
  ```
- The PHY (Palladium database format generated in the Palladium environment) format contains the simulation activity data generated during emulator-based simulation. The PHY format can be used in all vector-based power analysis flows (vector profiler, static power analysis and dynamic vector-based power analysis).
  Use Model

  ```
  read_activity_file-format phy <path to PHY database directory>
  ```

  Example

  ```
  read_activity_file -format phy ./phy2/trace.phy -scope test
  ```

  Notes:
  - The xeDebug software must be present in the `PATH` environment. Add xeDebug in the path using the following command:
    ```
    set path = (<install_dir>/bin $path)
    ```
    The xeDebug software version should be the same as the one using which the PHY database is generated.
  - The PHY database generated from the following UXE/VXE version is supported:

    ```
    UXE18.6.0 onwards
    ```

	`VXE18.6.0 onwards`
	◦ When reading a PHY format activity file, you require a DPA (DPA_ToggleCountFormat) license.
	◦ For information on error messages displayed while reading the PHY database, refer to the `xe.msg` file in the PHY database directory.
	◦ Make sure that the PHY database is not locked before invoking the Voltus software.
	◦ Make sure that the xeDebug software is not already running on the machine.
	◦ Ensure that the following directories are present in the top-level PHY database directory: `*.phy`, `PDB`, `cellList`, `QTDB`, `dbFiles`, and `.design`.
	◦ It is recommended that the write permission is assigned on the entire PHY database. In case the write permission cannot be granted, you must set the `SWFV_FREEZE_PHY` environment variable to `1`.
	◦ If you want to run parallel Voltus executables on the same top-level PHY directory with multiple `*.phy` databases, it is suggested to create local copies of the multiple top-level directories, with links to the original PHY database. This will allow local copies of the PHY database to acquire separate locks.
	• The Simulation History Manager (SHM) database format is a record of the data signal changes that occur during design simulation. The SHM database consists of a directory, typically with the "`.shm`" suffix. The directory contains two disk files, namely `*.trn` and `*.dsn`. For more information about the SHM format, refer to the *SimVision Analysis Environment* manual available on the Cadence Online Support (COS) website.
	Example
	`read_activity_file -format shm  /../shm/ADDER.trn -scope adder_e/u1 -start 0ns -end 60ns`

`-hierarchy_separator` *separator*	
	Specifies the separator character in the hierarchical net names, bus names, and pin names for the block or other part of the design that you want to analyze. It must be the same as the separator character in the design netlist. *Default*: slash (/)
`-name_mapping_rule` *file*	

	Specifies to enable rule-based mapping to map RTL vectors to gate-level netlist. This parameter helps in resolving name mismatch between RTL and gate level netlists.
	The name mapping rule file format is:
	`Name_Mapping_Rule_file`
	`# RTL  # Gate`
	`<rtlstring> <gatestring>`
	where, each line is a rule. The string in the first column(RTL) is what we want to replace with the string in the second column(Gate). The rules in every line are added onto the previous rules.
	For example, to replace all the square `[]` brackets with under score _ for RTL-to-Gate matching, the rule file will be:
	`#RTL  #GATE`
	`[  _`
	`]  _`
	Note: `-name_mapping_rule` is not supported when the `power_comprehensive_automapping` attribute is set to `true`.

`-reset`	If `-reset` is specified, all previous activity files specified using the `read_activity_file` command will be reset and ignored. *Default*: All the previous activity files specified using the `read_activity_file` command are loaded and used (They are additive).
`-rtl {true|false}`	
	Specifies to process the vector as an RTL vector and perform state propagation. The propagation behavior is global and the full design is propagated. RTL vectors are zero-delay vectors so delay annotation is always done for signals from this vector file. *Default*: `false`
`-scale_time` *scalefactor*	
	Scales the FSDB/SAIF/TCF/VCD duration value by the specified scale factor. For example, an FSDB scale of 2 will double the FSDB duration value. **Note:** The `-start` and `-end` parameters of the `read_activity_file` command are post scaled.
`-scope` *scope_name*	

	Specifies the FSDB/SAIF/TCF/VCD scope, that is, the name of the module within the activity file associated with the block or other part of the design that you want to analyze.
	
	The scope argument is removed from the FSDB/SAIF/TCF/VCD name before comparing it to the Verilog file.
	
	⚠ If the scope value has backslash '\' or square brackets '[]', you must specify the scope within curly brackets {{…..}} to ensure that '\' or '[]' are correctly processed by the Tcl Interpreter. In the following example, the scope value is specified within curly brackets as it contains '\' and '[]':
	`read_activity_file -scope {{reg_based_buffer.reg_inst\/sv_xpol_trans_q_reg[0][14][7][sv_re][4]}}`
	If '\' and '[]' are not there, curly brackets are not required.
	
	Example
	
	The following command specifies the u1 module within the VCD file as the module associated with the part of the design being analyzed, and specifies the `dut_5buf_full_chip` block instance as the block to map:
	`read_activity_file \`
	`-format VCD \`
	`-scope adder/u1 \`
	`-start {0ps 2000ps} \`
	`-end {1100ps 3100ps} \`
	`-block ../vcd/dut_5buf_full_chip`

`-write_net_freq {true | false}`	

	This option should be set when an activity from a TCF or VCD file needs to be used to capture net frequency for signal EM analysis (`check_ac_limits`). In addition, you must use the `check_ac_limits -use_db_freq` option to read the toggle rates from the activity file.
	
	Default: `false`
	
	Example
	The following example reads a TCF file which is used to capture net frequency for signal EM analysis:
	`read_activity_file -format tcf -write_net_freq true design.tcf`
	`check_ac_limits -use_db_freq -report signal_em.rpt`

`-start { time1 time2 ... timen }`	

	When you use the vector-driven method of performing dynamic instance-based power-consumption calculation, you can specify the part of the activity file from which you want power analysis to use for the power calculation. This option specifies the start time of this range and is used in conjunction with the `-end` option. Specifies the start time as time-value pairs to report the average power across non-overlapping multiple windows specified in the VCD file for a block, level of hierarchy, or other part of the design that you want to analyze. Units are in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). *Default*: The time unit is defined in the first `.lib` file read during design import. You can specify the `$worst_power_window_start` variable with the `-start` parameter to take the worst case start time from the last vector profiler run. Not required for TCF and SAIF. **Note**: You can read multiple windows in a single VCD file by specifying multiple pairs of the start time and end time for the non-overlapping multiple windows specified in the activity file. The use model for reading multiple windows is: `read_activity_file -format VCD design.vcd -start {start1 start2 start3} -end {end1 end2 end3}` The *start1/2/3* and *end1/2/3* values should be in the ascending order. Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched using the `set_power_analysis_mode -settling_buffer` parameter. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.
`-start_time_shift` *value*	
	Specifies to shift the start time of the specified activity files. When you specify the `-scale_time` parameter with the `-start_time_shift` parameter, it scales the start time shift along with the start/end time.
activity_file	The name of the activity file.
`-weight` *value*	

	Specifies the weight number of the TCF file. `1.0` is the default value. The value should be a floating number that is smaller than `1.0`. This parameter allows you to merge multiple TCF files at different function modes for static power estimation of a design.
	
	In the following example, TCF *file1*, *file2*, and *file3* has the weight numbers 0.3, 0.2, and 0.4, respectively.
	
	`read_activity_file -format TCF  file1 -weight 0.3`
	`read_activity_file -format TCF  file2 -weight 0.2`
	`read_activity_file -format TCF  file3 -weight 0.4`
	Here, *file1*, *file2*, and *file3* will be merged together to represent the toggle and duty for the nets/pins in the design. The relative weight of 0.3/0.9, 0.2/0.9, and 0.4/0.9 would be used for static power calculation. In this example, 0.9 is the sum of weights.

`-zero_delay {true|false}`	

	Specifies to process the vector as a zero-delay vector and perform delay annotation. *Default* : `false`

Activity Precedence

The following commands can define activity:

```
read_activity_file file
     [-format {vcd | tcf | saif} ]
     [-hierarchy_separator separator ]
     [-start time ]
     [-end time ]
     [-reset]
     [-scope scope_name ]
     [-block block_name ]

set_default_switching_activity
    [-input_activity factor ]
    [-sequential_activity factor ]
    [-period value ]
    [-duty value ]
    [-global_activity factor ]
    [-hierarchy hierarchy _name]


set_switching_activity
    [-reset]
    [-clock clock_name]
    [-period value ]
    [-activity factor | -transition_density ]
    [-net net_name | -port port_name | -pin pin_name ]
    [-duty value ]
    [-instance]
```

The precedence of activity is as follows:

1. User-Defined: Activities applied to specific nets/pins/ports
2. Activity File: Activity specification through a VCD/FSDB/SAIF/TCF format
3. Clock Gates Output Activity: Activity specification to the output of all clock gates
4. SDC/TWF constants: Set constants to a list of pins or ports for use by the timing engine (timing analysis)
5. Hierarchical Global Activity: Activity specification for a specific hierarchy
6. Global Activity: Activity specification for all instances that are part of the data network.
7. Sequential Element Activity: Activity specification applied to the output of all sequential elements.
8. Primary Input Activity: Activity specification for all primary inputs

If you specify multiple activity data for the same items, the power engine will use the one with the higher precedence.

Examples

- The following command reads the compressed ap_wait_test_pll1_mod.vcd.gz VCD file that contains the toggle count information for the power consumption calculations, specifies the crm_ap module within the VCD file as the module associated with the part of the design being analyzed, specifies the slash character (/) as the hierarchical separator for hierarchical net names, bus names, and pin names for the part of the design being analyzed, and specifies the start (180002ns) and end (189802ns) times from the VCD file for the part of the design being analyzed:

```
read_activity_file -format vcd \
-scope testbench/top/ap/mcu_platform/crm_ap \
-hierarchy_separator \
-start 180002ns \
-end 189802ns \
ap_wait_test_pll1_mod.vcd.gz
```

- The following command reads the compressed A.vcd.gz VCD file that contains the toggle count information for the power consumption calculations, specifies A1 as the block to map a sub-block VCD file with a block instance at the top-level, and specifies the start (180002ns) and end (189802ns) times from the VCD file for the part of the design being analyzed:

```
read_activity_file -format vcd \
-block ap/A1 \
-start 180002ns \
-end 189802ns \
 A.vcd.gz
```

- The following example specifies a VCD activity file called dmac_mac.vcd for the scope top/dma_dut.

```
read_activity_file -format vcd -start 10ns -end 20ns \
-scope top/dma_dut dmac_mac.vcd
```

- The following command specifies multiple pairs of the start time and end time for the multiple windows in the VCD file：

```
read_activity_file \
-format  VCD \
-scope  adder/u1 \
-start  {0ps 2000ps} \
-end    {1100ps 3100ps} \
-block  ../vcd/dut_5buf_full_chip.vcd
```

- The following command specifies to read the RTL-level VCD activity file:

```
read_activity_file -format VCD -scope scope_top -start 300ns -end 700ns /DesignData/VCD/rtl.vcd -block
inst_C1 -rtl true
set_db power_method vector_profile power_vector_profile_mode transient power_worst_window_type full
power_grid_libraries /DesignData/PGV/library_pv.cl
```

Related Topics

- "Dynamic Power and IRDrop Analysis" in the *Voltus User Guide*

read_activity_map_file

```
read_activity_map_file
[-help]
[-golden {rtl | gate}]
[-gate_block block_name]
[-reset]
[-rtl_block block_name]
-rtl_to_gate mapping_file
[-two_column_mapping_file mapping_file]
```

Specifies instance name mapping between RTL netlist and GATE level netlist. Power analysis uses this instance name mapping to annotate the output nets of this instance from the pin-based RTL VCD or TCF. The mapping file used for instance name mapping is generated by Conformal.

You must specify this command before `report_power`.

Parameters

`-gate_block block_name`	
	Specifies the GATE activity block name to map the block-level RTL mapping files to the top-level RTL activity files. The GATE block name is appended before the GATE level entries in the mapping file to match the entries in the top-level RTL activity file.
`-golden {rtl | gate}`	
	Specifies whether RTL or Gate is Golden. The one not specified as golden will be considered as Revised. Currently the Conformal MatchPoint file does not specify this. *Default*: `rtl`
`-help`	Outputs a brief description that includes the type and default information for each `read_activity_map_file` parameter. For a detailed description of the command and all of its parameters, use the man command `man read_activity_map_file`.
`-reset`	Specifies to reset and ignore all the previous mapping files specified using the `read_activity_map_file` command. *Default* : All the previous mapping files specified using the `read_activity_map_file` command are loaded and used.
`-rtl_to_gate mapping_file`	
	Specifies instance name mapping between RTL netlist and GATE level netlist. RTL level VCD or TCF are obtained from simulation on RTL Verilog netlist. Power analysis uses this instance name mapping to annotate the output nets of this instance from the pin based RTL VCD or TCF. The mapping file used for instance name mapping is generated by Conformal.
`-rtl_block block_name`	

	Specifies the RTL activity block name to map the block-level RTL mapping files to the top-level RTL activity files. The RTL block name is appended before the RTL level entries in the mapping file to match the entries in the top-level RTL activity file.
`-two_column_mapping_file` *`mapping_file`*	

	Specifies a file that contains the instance name mapping between the RTL netlist and GATE level netlist.
	This parameter can be used to specify a mapping file instead of the LEC generated mapping file (`-rtl_to_gate` *`mapping_file`*) to map points from RTL to Gate netlist. The format of the two column mapping file is:
	`#Gate name =>  # RTL name`
	`gate_inst_name1  => rtl_inst_name1`
	`...`
	Here, the first column corresponds to the gate name and the second column corresponds to the RTL name.

Example

- The following command specifies a mapping file rtl_map:
  ```
  read_activity_map_file -rtl_to_gate rtl_map
  ```
- The following command performs instance name mapping between RTL netlist and GATE level netlist, wherein the GATE level netlist is Golden and the RTL netlist is Revised; it specifies the mapping file MapFile.alt
  ```
  read_activity_map_file -rtl_to_gate MapFile.alt -golden gate
  ```
- The following commands appends the RTL block name and GATE block name before the RTL and GATE level entries in the mapping file to match the entries in the top-level RTL activity file:
  ```
  read_activity_map_file -rtl_to_gate LEC.top \
  -rtl_block  "core/block_top" \
  -gate_block "core/block_top"
  ```
- The following commands specify to ignore the previous mapping file (`old.map`), and use the modified mapping file (`updated.map`):
  ```
  map_activity_file -rtl_to_gate old.map
  map_activity_file -reset
  map_activity_file -rtl_to_gate updated.map
  ```

An example of the mapping file output from Conformal is described as follows:

```
Mapped points: SYSTEM class
1-th mapped points:
 (G) + 1 PI /x_ick
 (R) + 595 PI /x_ick
2-th mapped points:
 (G) + 2 PI /x_jreset_cp_p
 (R) + 594 PI /x_jreset_cp_p
3-th mapped points:
 (G) + 3 PI /x_mreset_cp_p
 (R) + 593 PI /x_mreset_cp_p
........
60-th mapped points:
 (G) + 4582 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg[35]
```

```
 (R) + 3856 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg_35_

61-th mapped points:
 (G) + 4583 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg[34]
 (R) + 3855 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg_34_

62-th mapped points:
 (G) + 4633 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[21]
 (R) + 3805 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_21_

63-th mapped points:
 (G) + 4634 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[20]
 (R) + 3804 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_20_

64-th mapped points:
 (G) + 4635 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[19]
 (R) + 3803 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_19_

65-th mapped points:
 (G) + 4636 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[18]
 (R) + 3802 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_18_
```

where, `G` refers to the Golden netlist name and `R` refers to the Revised netlist name.

By default, the tool assumes that the Golden netlist name refers to the RTL net name and Revised netlist refers to the GATE level net name.

Related Topics

- "RTL Activity File Flow in Static Power Calculation" in the *Voltus User Guide*

report_annotated_parasitics

```
report_annotated_parasitics
[-help]
[-include_nets <string>]
[-include_nets_file <string>]
[-view {viewName}]
[> {write_fileName}]
[>> {append_fileName}]
[{[-list_annotated] [-broken_nets] [-floating_nets] [-list_extra_pin_net][-no_driver_nets] [-unloaded_nets]
     [-list_zero_cap_net][-list_not_annotated] [-real_nets] [-supply_nets] [-max_missing {max_missing_int}]}]
```

Reports the back-annotated parasitics of the design. By default only summary report is generated. The software groups all the nets by net-types and annotation-status (annotated or not-annotated).

Parameters

`-help`	Outputs a brief description that includes type and default information for the `report_annotated_parasitics` parameters. For a detailed description of the command and all of its parameters, use the `man` command: `man report_annotated_parasitics`.
`>> {append_fileName}`	Writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note:** The file name must be the last argument in the list. *Default:* Report is displayed on standard output without being saved.
`-broken_nets`	Reports broken nets in the design.
`-floating_nets`	Generates a list of floating nets.
`-include_nets_file` *string*	Accepts a file name, which contains a list of net names, as input. When this parameter is specified, the parasitic annotation summary is provided only for the listed nets. The net names in the file can be specified as one net per line.
`-include_nets` *string*	Specifies a list of net names for which annotation summary is required.
`-list_annotated`	Generates a list of annotated nets.
`-list_zero_cap_net`	Generates a list of zero capacitance nets.
`-list_extra_pin_net`	Generates a list of nets that have extra pins in the SPEF but not in the Verilog.
`-list_not_annotated`	Generates a list of not annotated nets.
`-no_driver_nets`	Lists one terminal nets that have no driver.
`-real_nets`	Lists all real nets.
`-supply_nets`	Lists all supply nets.
`-max_missing {max_missing_int}`	Limits the number of missing annotations to the number specified. *Default*: 1000
`-unloaded_nets`	Lists one terminal nets that have no load.
`-view` *viewname*	Reports the back-annotated parasitics for the specified analysis view. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. *Default*: Reports against the default analysis view
`> {`*write_fileName*`}`	Writes the report to the specified file name. If the file already exists, the software overwrites it.

Examples

- The following command generates a list of annotated and not-annotated nets:

```
report_annotated_parasitics -list_annotated -list_not_annotated -list_real_net -list_float_net -
list_nodriver_net \ + -list_noload_net -list_supply_net -max_missing 1000
```

```
# list syntax: type: net_name [nrCap Cap nrXCap XCap nrRes Res]
# unit: pF, Ohm
# List the first 82 of total 82 real nets:
Annotated Real Net: [seg3/w7] [ 10 0.0690723 25 0.0512442 9 219.251 ]
Annotated Real Net: [seg3/w5] [ 14 0.0373353 24 0.0265601 13 161.843 ]
:
:
Annotated Real Net: [CLK3] [ 2 0.00046867 0 0 1 18.872 ]
Annotated Real Net (broken): [CLK4] [ 1 0.0002379 0 0 2 0.0002 ]
# List the first 20 of total 20 not annotated real nets:
Not Annotated Real Net: [seg3/wout]
Not Annotated Real Net: [CLK44]
:
:
Not Annotated Real Net: [out1]
Not Annotated Real Net: [in1]
# No annotated supply net.
# List the first 4 of total 4 not annotated supply nets:
Not Annotated Supply Net: [gnd]
Not Annotated Supply Net: [GND]
Not Annotated Supply Net: [VDD]
Not Annotated Supply Net: [vdd]
# No annotated floating net.
# No not annotated floating net.
# No annotated no driver net.
# ist the first 9 of total 9 not annotated no driver dangling nets:
Not Annotated Dangling Net (no driver): [seg3/n8]
Not Annotated Dangling Net (no driver): [seg3/n111111]
:
:
Not Annotated Dangling Net (no driver): [in]
Not Annotated Dangling Net (no driver): [out5]
# List the first 8 of total 8 annotated no load dangling nets:
Annotated Dangling Net (no load): [seg3/n14] [ 1 0.00520893 0 0 0 0 ]
Annotated Dangling Net (no load): [seg2/n14] [ 1 0.00636874 0 0 0 0 ]
:
:
Annotated Dangling Net (no load): [n6] [ 2 0.1086 0 0 1 0.0001 ]
Annotated Dangling Net (no load): [n5] [ 2 0.1065 0 0 1 0.0001 ]
# List the first 16 of total 16 not annotated no load dangling nets:
Not Annotated Dangling Net (no load): [seg3/u14:QN]
Not Annotated Dangling Net (no load): [seg3/u9:QN]
:
:
Not Annotated Dangling Net (no load): [u2:QN]
Not Annotated Dangling Net (no load): [in4]
# Summary of Annotated Parasitics:
+-----------------------------------------------------------------+
| Net Type            | Count  | Annotated (%) | Not Annotated (%) |
```

```
+--------------------+------------+------------------+------------+
| total              | 135    | 90 66.7%      | 45 33.3%          |
+--------------------+------------+------------------+------------+
| no drive (*)       | 9      | 0 0.0%        | 9 100.0%          |
| 1-term:no load     | 24     | 8 33.3%       | 16 66.7%          |
| real net (complete) | 81     | 61 75.3%      | 20 24.7%          |
| real net (broken)  | 21     | 21 100.0%     | 0 0.0%            |
+-------------------------------------------------------------------+
Note for Net Types marked "(*)": Such nets are never timed, but reported here for informational purpose.

+-----------------------------------------------+
| Annotated  | Res (KOhm) | Cap (pF) | XCap (pF) |
+-----------+----------------+----------------+--+
| Count      | 982        | 1011     | 1152      |
| Value      | 20.0869    | 7.7289   | 5.4598    |
+-----------------------------------------------+
```

Related Commands

- read_spef

read_power_db

```
read_power_db
[-help]
-in_file {file1 file2 file3 ...}
[-hierarchy {prefix1 prefix2 prefix3 ...}]
[-hierarchy_separator "/"]
```

Restores any `power.db` file in the Voltus/Innovus session. This command can be used after restoration of a design to view the static power results or incremental report generation without having to do power analysis again.

Note: You will be able to restore the power database that you created previously, only if you created the power database using the `power_write_db` attribute. Also, the command loads power databases created in the 9.1 release only, and not in the older releases.

- The command can map a hierarchical instance to a flat netlist.
- It can restore multiple power dbs for blocks and top level, that are generated separately.
- When the information for a net or an instance is present in multiple power dbs, the last one will overwrite.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `read_power_db` parameter. For a detailed description of the command and all of its parameters, use the man command `man read_power_db`.
`-in_file {file1 file2 file3 ...}`	
	Specifies the name of the power database file.
`-hierarchy {prefix1 prefix2 prefix3 ...}`	
	Specifies to map each instance in the file with a prefixN/instance (for example, prefixA/inst) in the flat netlist, where '/' is the hierarchy separator. This is an optional parameter. *Default*: `all`
`-hierarchy_separator "/"`	
	Specifies the separator character in the hierarchical net names, bus names, and pin names for the block or other part of the design that you want to analyze. It must be the same as the separator character in the design netlist. This is an optional parameter. *Default*: slash (`/`)

Use Model

```
read_db / read_design
read_power_db -in_file power.db
report_power # for text based power reports
```

Example

For each instance in file 'file2' a prefix "A" is added to instance name such as instC, A/instC. No prefix is added to `file1' since prefix1 is none for file1.

```
read_power_db -in_file {file1 file2} -hierarchy {none A}
```

report_inst_power

```
report_inst_power
[-help] instance [-out_file filename] [-inst_file filename]
```

Generates a text based report for the specified instance. This command enables you to determine how the different components of static power are calculated. You can use the report to debug static power numbers after performing power analysis (`report_power`).

This command is available through both the static and dynamic power engines.

Note: For dynamic power engine, the `report_inst_power` command does not generate text reports incrementally. If the dynamic run has been performed and you need the debug information for a static power number, you should re-run power analysis.

> The `report_inst_power` command is not supported in the event-based power analysis flow.

Parameters

`-out_file` *`filename`*	Writes out the static power analysis report into the file that you specify.
`-help`	Outputs the command usage and a brief description about the command parameters.
`instance`	Specifies the instances to include in the instance power report.
`-inst_file` *`filename`*	

	Specifies a file that includes a list of instances. The parameter allows you to report and debug power numbers for multiple instances.
	A sample file is given below:
	`INST1`
	`INST2`
	`INST3`

Example

- The following command specifies an instance power report file `debug.rep` for the instance `top/i_1`:

  ```
  report_inst_power -out_file debug.rep top/i_1
  ```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*
  - "Debugging Instance Power"

report_power

```
report_power
[-help]
[-block block_name]
[-cap]
[-cap_unit unit]
[-cell {cell_list}]
[-cell_fit_file filename]
[-cell_list_file filename]
[-cell_type {all | {macro io combinational sequential clock_combinational clock_sequential}}]
[-clock_domain names]
[-clock_gating_efficiency]
[-clock_network {all | {clock_list}}]
[-cluster_gating_efficiency]
[-combinational_sequential_power_distribution]
[-compress compression_ratio]
[-distribute]
[-exclude_cells {cell_list}]
[-exclude_cells_file filename ]
[-exclude_instances_file filename ]
[-exclude_insts {instance_list}]
[-float_precision value]
[-format { simple | detailed }]
[-group_type user_defined_group_name]
[-hierarchical_instances {inst1 inst2 inst3 …}]
[-hierarchy {all | hierarchy_level}]
[-include_group_instance_count]
[-include_sequential_in_clock]
[-inst_list_file   filename]
[-insts {instance_list}]
[-leakage]
[-net [-nworst  number_of_nets]]
[-no_wrap]
[-out_dir directory]
[-out_file filename]
[-output_cell_fit_file filename]
[-pg_net {all | pg_net_name_list}]
[-pg_pin]
[-power_db_directory directory]
[-power_density_tiles value]
[-power_density_tiles_row_column {value1 value2}]
[-power_density_tiles_size {value1 value2}]
[-power_domain {all | {power_domain_list}}]
[-power_unit unit ]
[-print_memory_power]
[-register_gating_efficiency]
[-report_prefix prefix]
[-stat]
[-sort {internal | switching | leakage | total}]
[-thermal_conductivity_inputs  file_name]
[-thermal_leakage_temperature {temp1 temp2 temp3 …}]
[-thermal_leakage_temperature_scale_table_file filename]
```

```
[-thermal_leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3
.....}]
[-thermal_material_file filename]
[-thermal_power_map_bottom_layer layer_name]
[-thermal_power_map_density_table_tiles {Xint Yint}]
[-thermal_power_map_file file_name]
[-thermal_power_map_format {simple | stack }]
[-thermal_power_map_format_version {new|default}]
[-thermal_power_map_header_include_file  filename]
[-thermal_power_map_power_table_tiles {Xint Yint}]
[-thermal_power_map_si_unit {true | false}]
[-thermal_power_map_tiles {xint yint}]
[-thermal_power_map_top_layer layer_name]
[-threshold value]
[-threshold_voltage_group { all | group_name}]
[-time_based_report]
[-time_unit unit]
[-toggle_rate]
[-use_geometry_cell_size {true | false}]
[-view view_name]
```

Reports global as well as specific instance power, clock network power, clock domain power, and net switching power. It also reports the power for power domains and specific power nets. Units are reported in milliwatts.

This command starts the execution of power analysis (both static and dynamic). The `report_power` command has a character limit of 1015.

> The parameters of the `report_power` command, except `-cap`, are not supported in the dynamic only mode (specified using the `power_disable_static` attribute). These parameters are only supported in the static mode. However, `-clock_gating_efficiency`, `-register_gating_efficiency`, and `-power_domain` parameters are not supported even if the static power mode is enabled. You can use the `set_power_include_file` command to pass the `report_power` command options in the dynamic analysis mode.

The `report_power` command allows generation of multiple power reports with a single `report_power` command. This allows you to generate all the static reports during dynamic analysis, and removes the need to run incremental `report_power` in static power analysis using a Verilog netlist. By default, a detailed power summary report, and a short summary will be printed at the beginning of all reports. To enable this flow, you need to specify the following parameters:

- `-out_dir`
- `-report_prefix`

The following `report_power` command parameters are supported by the multi-report flow, both in static and dynamic power analysis:

- `-cell`
- `-clock_gating_efficiency`
- `-register_gating_efficiency`
- `-cluster_gating_efficiency`
- `-net`
- `-insts`

- `-pg_net`
- `-clock_network`
- `-hierarchy`

The following parameters are not supported in the multi-report flow:

- `-cell_type`
- `-power_domain`
- `-clock_domain`
- `-clock_gating_efficiency` and `-clock_network` in the same run

⚠ In the static power analysis flow, you can run the `report_power` command after the `read_power_db` command without loading the design. The following command is an example of the use model:

```
read_power_db -in_file power.db
report_power -out_dir same -report_prefix report -clock_network {all} -insts {*}
```

The following parameters of the `report_power` command are supported without design loading:

- `-cell`
- `-cell_type`
- `-insts`
- `-clock_network`
- `-clock_domain`
- `-pg_net`
- `-pg_net -format detailed`
- `-hierarchy`
- `-hierarchical_instances`

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_power` parameter. For a detailed description of the command and all of its parameters, use the man command: man `report_power`
`-block` *`block_name`*	

	Specifies to generate the different power analysis reports for a block. This parameter is useful in viewing and debugging power consumption issues for a specific block after the power run is complete.
	Example:
	The following command will create the summary and pg_net reports for the design block {A/B}:
	`report_power -block {A/B} -out_file blockB.rpt -pg_net -pg_pin {all} -insts {*}`
	Following are the sample block-specific reports:
	`blockB.rpt.avppower`
	`blockB.rpt.pgnetpwr`
	`blockB.rpt.pgpinpwr`
	`blockB.rpt.instpwr`

`-cap`	Dumps out the list of net, pin, and total capacitances in three different columns.
`-cap_unit` *`unit`*	
	Sets the capacitance unit to be used for power report generation. The default unit is pF (picoFarads). By default, the capacitance unit in `report_power` is Farad (F). To get the capacitance in pF, use the command `report_power -cap_unit pf`.
`-cell` **{***`cell_list`***}**	
	Specifies the cells to include in the power report. Accepts wildcards (*)
`-cell_fit_file` *`filename`*	

	Specifies a file that accepts a cell-based FIT input table with two columns: cell name and FIT value. This parameter is used to generate an instance-level FIT report based on the cell-level data.
	Instance-based FIT report is supported only in the XP mode.
	File Format Example for the FIT Input File:
	`##Cell Name FIT`
	`----------`
	`CellA .25`
	`CellB .5`

`-cell_list_file` *`filename`*	
	Specifies a file containing a list of cells that are to be included in the power reports.
`-cell_type {all | {macro io combinational sequential clock_combinational clock_sequential}}`	
	Specifies the cell type to include in the power report. *Default*: `all` When you specify the cell types, `-cell_type combinational` or `sequential`, the power report now does not show the clock combinational or clock sequential cells.
`-clock_domain` *`names`*	

	Generates detailed instance power reports for the specified clock domains. Wildcards are not accepted. When this parameter is not specified, the `report_power` command provides a summary of clock power for instances with the clock pins driven by a clock domain. In contrast, `report_power -clock_domain` reports a clock power summary for instances with any pin driven by a clock domain, provided the instance is part of the clock network. **Usage** `report_power -clock_domain` *`name`* `-out_file` *`filename`*
`-clock_gating_efficiency`	

	Generates a report that includes the Clock Gating Efficiency (CGE) for all clock gating instances as well as different hierarchies in the design. Average CGE for different hierarchies is also reported.
	Clock Gating Efficiency is calculated as:
	`CGE = (toggles at clock gate output/toggles at clock gate input)`
	`Average CGE = (avg of all CGEs above)`
	When you generate a CGE report using the command `report_power -clock_gating_efficiency -out_file` *`cge.rpt`*, the following information will be displayed:
	• **Clock-gating Efficiency Report** - for each clock domain, it includes the toggle rate, number of registers, number of clock gates, average clock toggle at registers, average toggle savings at registers, and average toggle savings histogram.
	• **Hierarchical View of Average Toggle Savings** - number of clock gates and average toggle savings for each hierarchical module in the design

`-clock_network` {`all` | {*`clock_list`*}}	
	Reports the power consumption of the clock network, including the power for generated clocks. Use `clock_list` to report the power consumed by specific clocks. *Default*: `all`
`-cluster_gating_efficiency`	
	Specifies to generate the cluster efficiency report in the RGE report. This report gives the CGE/RGE metrics for the registers at the fanout of each clock gate instance in the design.
`-combinational_sequential_power_distribution`	
	Specifies to generate a report that gives the ratio of combinational versus sequential power for all modules of a given hierarchy. You must specify this parameter with the `-insts` parameter.
`-compress` *`compression_ratio`*	
	Specifies to compress the power analysis report files in the GNU Zip format. The compressed files will have the suffix `.gz`. *`compression_ratio`* is from 0-9. `0` indicates that the report files are not to be compressed. The compression levels vary from `1` to `9`. `1` provides the fastest compression speed but at a lower ratio, so the file size will be larger. `9` provides the highest compression ratio but at a lower speed. *Default*: 0

`-distribute`	Enables standalone distributed power analysis. This parameter allows you to perform dynamic power analysis in the multi-CPU mode without running rail analysis.
`-exclude_cells {`*`cell_list`*`}`	
	Specifies a list of cells that should be excluded from power analysis. This parameter is supported in both Verilog and DEF-based flows with the `power_method` attribute set to either `static` or `event_based`.
`-exclude_cells_file` *`filename`*	
	Specifies a file containing a list of cells (one per line) which should be excluded from power analysis. This parameter is supported only in the DEF-based flow with the `power_method` attribute set to either `static` or `event_based`.
`-exclude_insts {`*`instance_list`*`}`	
	Specifies a list of instances that should be excluded from power analysis. This parameter is supported only with the following power attribute: `power_method event_based`.
`-exclude_instances_file` *`filename`*	
	Specifies a file containing a list of instances (one per line) which should be excluded from power analysis. This parameter is supported only with the following power attribute: `power_method event_based`.
`-float_precision` *`value`*	
	Sets the precision value to be used for power report generation. The default value is 4.
`-format { simple | detailed }`	
	Specifies the type of report summary required for static power reporting. The possible arguments of this parameter are: • `simple` - reports the amount of power consumed by all power nets in a condensed format, breaking down power based on the switching/internal/leakage components. • `detailed` - reports the amount of power consumed by all power nets in a detailed format. The detailed format includes additional information, such as output pin capacitance (`Cpin Sum`), output net capacitance (`Cnet Sum`), Total Load Capacitance (`Cpin Sum` + `Cnet Sum = Total Load_Cap`), maximum frequency at the input and output pins (`Max_Clk_Freq`), maximum activity factor at the input and output pins (`Max_Activity`), cell name of the instance (`Cell Name`), clock or data cell (`C/D`), and library used in power analysis for the specific cell (`Lib`). The libraries are numbered in the header section. The name of the generated report file is `report.*.detailed.rpt`. Currently, the `-format` detailed report is only available for the `-pg_net` report.
`-include_sequential_in_clock`	
	Includes the leaf flip-flop power in the clock network power report. *Default*: The leaf flip-flop power will not be included in the clock network power report.
`-hierarchical_instances {`*`inst1 inst2 inst3 …`*`}`	

	Specifies to report all the power content for the specified hierarchical instances/ blocks. It reports the `Total Power` (a breakup of power components) and `Group` (a breakup of power by cell type) of the specified block at each hierarchy. To generate a complete hierarchical instance power report, it is recommended to specify `-hierarchical_instances` with `-report_prefix`. The report is written in the specified output directory. If you specify `-outfile` instead of `-report_prefix`, an instance file is output without hierarchy-level details. `-report_prefix` generates the following 3 reports: • `<prefix>.rpt` - reports power details for all instances in the design. • `<prefix>.hierpwr.rpt` - reports hierarchical power, that is, power summary, internal, switching, leakage, and total power for all hierarchical instances. • `<prefix>.hierinstpwr.rpt` - reports power details for the requested block instances. **Note**: `-hierarchical_instances` must be specified with `-hierarchy` to include hierarchy-level power report. When `-hierarchical_instances` is used without `-hierarchy`, only top-level summary of the power consumption is written.
`-hierarchy` {`all` | *`hierarchy_level`*}	
	Specifies the hierarchy level that is included in the power report. You can specify the *`hierarchy_level`* as a number that corresponds to a specific hierarchy level in the design. A *`hierarchy_level`* of 0 specifies the top level. *Default*: `all`
`-include_group_instance_count`	
	Specifies to report the instance count for each cell type. When specified, you can view the number of instances for the sequential, macro, IO, combinational, clock sequential, and clock combinational cell types in the power report (power.rpt). The parameter is supported in both the static and dynamic power analysis flows. This parameter allows you to verify if there are any missing instances.
`-insts {`*`instance_list`*`}`	
	Specifies the instances to be included in the power report. The power report provides the amount of power consumed by a specified list of hierarchical or leaf-level instances. *`instance_list`* accepts wildcards (*) for leaf-level instances only. An instance with 0 power will not be listed in the power report.
`-inst_list_file` *`filename`*	
	Specifies a file containing a list of instances that are to be included in the power reports.
`-leakage`	Reports only leakage power of the design.
`-net`	Reports the net switching power, as well as the load capacitance, toggle rate, and switching values for each net in the design.

`-no_wrap`	Displays the `report_power -cell_type all` output on a single line in the report output file.
	Example
	`Instance Toggle Internal Switching Leakage Total Percentage Cell`
	`Power Power Power Power (%) Name`
	`----------`
	`dma/ram0 7.102e+08 5.58 0.002307 0.02424 5.607 18.19 /data/cellA`

`-nworst` *number_of_nets*	
	Provides n top windows with maximum activity or power in the vector profile report. The default value is `10`. **Note:** You must use the `-nworst` parameter in conjunction with the `-net` parameter.
`-out_file` *filename*	
	Specifies the name of the report file in which the top-level summary of the power consumption information is written. You can include the directory name. You can convert a power report file to the gzip file format by specifying `-out_file` *filename.gz*. **Note:** If you do not specify this parameter, the software directs the power report to the command line, as well as the log file, and no power report is written to a separate file.
`-out_dir` *directory*	
	Specifies the name of the power analysis output directory that saves the power reports. The default is the current working directory.
`-output_cell_fit_file` *filename*	

	Specifies the name of the output file for the generated FIT report, which includes three columns: instance name, corresponding cell name, and computed FIT value. This parameter is specified with the `-cell_fit_file` parameter. The output file is generated in the power output directory.
	File Format Example for the FIT Output File:
	`#Instance Name Cell Name FIT`
	`############################################################`
	`inst_A/al_inst/g5 INV_X1A_A9TL 0.25`
	`inst_A/al_inst/g8 INV_X1B_A9TL 0.25`
	`....`
	`Total FIT: 115`

`-pg_net` {`all` | *pg_net_name_list*}	
	When set to `all`, reports the amount of power consumed by all power nets. If you specify a list, reports the power consumed by each instance that is connected to the specified list of power nets. **Note:** When using this parameter, the static power calculation engine always performs propagation and power calculation. *Default*: `all`
`-pg_pin`	
	Reports power consumed by all PG pins in a design.

`-power_db_directory` *directory*	
	Specifies the location of the power database that the Power Calculation engine uses for generating standalone power reports. **Note**: Before specifying the `-power_db_directory` parameter, ensure that a power database is created in the binary format by using the `set_db power_write_db true` attribute.
`-power_domain` {`all` | {*power_domain_list*}}	
	Specifies all power domains or lists power domains to be included in the report. It prints a summary of power break down for each domain before listing power for each instance in the domain.
`-power_density_tiles` *value*	
	Specifies the number of tiles used for power density calculation. *Default*: `100` **Note**: When dividing a design into equal-sized tiles, it might not always fit perfectly into a square grid. For example, if you want to create NxN tiles for the top design, the design's size might not align exactly with these dimensions. This means the tiles in the last row and the last column could be different in size to cover the remaining part of the design. All other tiles, however, in the (N-1)x(N-1) area, will be the same size.
`-power_density_tiles_row_column {`*value1 value2*`}`	

	Defines the number of tiles by rows * columns. *value1* is the number of tile rows and *value2* is the number of tile columns.
	The following example specifies to perform power density aware vector profiling using 2 tile rows and 2 column rows:
	`set_db power_method vector_profile power_vector_profile_mode power_density power_worst_step_size 10ns`
	`report_power -power_density_tiles_row_column {2 2} -time_based_report -out_file tiling1.rpt`
	The following is the vector profile report for this command:
	`Tile 0_0_0 5.06 5.33 14.95 14.555`
	`Tile 1_0_1 14.95 5.33 24.84 14.555`
	`Tile 2_1_0 5.06 14.555 14.95 23.78`
	`Tile 3_1_1 14.95 14.555 24.84 23.78`
	`...`
	`...`
	Note: When dividing a design into equal-sized tiles, it might not always fit perfectly into a square grid. For example, if you want to create NxN tiles for the top design, the design's size might not align exactly with these dimensions. This means the tiles in the last row and the last column could be different in size to cover the remaining part of the design. All other tiles, however, in the (N-1)x(N-1) area, will be the same size.

`-power_density_tiles_size {`*value1 value2*`}`	
	Defines the absolute tile size. *value1* is the horizontal width of a tile (in micron) and *value2* is the vertical height of a tile (in micron).
`-power_unit` *unit*	
	Sets the power unit to be used for power report generation. The default unit is mW (miliAmps).
`-print_memory_power`	

Specifies to report power values for the memory and macro cells under separate groups (`Memory` and `Macro`).

The following is a snapshot of the power report showing separate groups for memory and macro cells:

Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
Sequential	0.4031	0.1283	0.006583	0.538	18.38
Memory	0.1624	0.004345	0.011	0.1778	6.074
IO	0	0	0	0	0
Physical-Only	0	0	2.91e-06	2.91e-06	9.945e-05
Combinational	0.8711	1.05	0.01579	1.937	66.2
Clock (Combinational)	0.002302	0.01018	7.654e-05	0.01255	0.429
Clock (Sequential)	0.01177	0.008387	0.000347	0.0205	0.7005
Macro (non-mem)	0.2317	0.007259	0.001234	0.2402	8.208
Total	1.682	1.209	0.03503	2.926	100

`-group_type` *`user_defined_group_name`*

Specifies to report power (internal/switching/leakage/total) for a user-defined collection of cells that are stated under a power group in the power summary report. You can use the following Tempus commands to create power groups:

- `define_property` - define a property.
- `set_property` - assign the defined property (group type) and its value (power group) to a list of cells.

The use model of the `-group_type` parameter is:

```
report_power -group_type user_defined_group_type
```

Here, *`user_defined_group_type`* is the user-defined power group type applied to a list of cells in a design. When the `-group_type` parameter is specified, the command reports power for the power groups associated with the specified group type in the power summary report.

Example:

The following command defines the `poly_bias` property (group type) on library cells:

```
define_property poly_bias -obj_type lib_cell -type string
```

The following commands apply the property `poly_bias` with the group name `P0` and `P10` to the collection of cells:

```
set_property [get_lib_cells -of_objects [get_cells -hierarchical * -filter {ref_name=~*PB0*}]]
poly_bias  P0
set_property [get_lib_cells -of_objects [get_cells -hierarchical * -filter {ref_name=~*PB10*}]]
poly_bias  P10
```

The following command reports power for the user-defined power groups that are associated with the property `poly_bias`:

```
report_power -out_file basic_group_type.rpt -group_type poly_bias
```

The following is a snippet of the power report that shows the user-defined power groups `P0` and `P10` that are associated with the property `poly_bias`, in addition to the existing groups (sequential, macro, IO, and so on):

```
User Group                          Internal   Switching     Leakage       Total  Percentage
                                    Power      Power         Power         Power  (%)
-------------------------------------------------------------------------------------------
P0                                   0.01498   0.0002017     0.01765     0.03283       15.63%
P10                                   0.1268     0.04026     0.01011      0.1772       84.37%
-------------------------------------------------------------------------------------------
Total                                 0.1418     0.04047     0.02776        0.21         100%
-------------------------------------------------------------------------------------------
```

```
Group                          Internal   Switching     Leakage       Total  Percentage
                               Power      Power         Power         Power  (%)
------------------------------------------------------------------------------------
Sequential                      0.02958   0.0003768     0.01458     0.04453       16.62
Macro                                 0           0           0           0           0
IO                                    0           0           0           0           0
Combinational                  0.008217    0.002633     0.01025      0.0211       7.877
Clock (Combinational)             0.1239     0.04421    0.007419      0.1755       65.51
Clock (Sequential)              0.01773    0.008783   0.0002459     0.02676       9.988
------------------------------------------------------------------------------------
Total                            0.1794       0.056     0.03249      0.2679         100
------------------------------------------------------------------------------------
```

`-register_gating_efficiency`

	Generates a report that includes the Register Gating Efficiency (RGE) and Data Aware Gating Efficiency (DAGE) for all sequential cell instances as well as different hierarchies in the design. Average RGE and DAGE for different hierarchies is reported. It also displays a section on ICG cluster information, that is, information about registers at the fanout of each ICG instance along with their RGE metrics.
	Register Gating Efficiency is calculated as:
	`RGE = 1 - (toggles at register clock pin/root clock toggles)`
	When you generate an RGE report using the command `report_power -register_gating_efficiency -out_file` *`rge.rpt`*, the following information will be displayed:
	• **Register-gating Efficiency Report** - for each clock domain, it includes the toggle rate, number of registers, number of clock gates, average clock toggle at registers, average toggle savings at registers, and average toggle savings histogram.
	• **Register Gating Opportunities Report** - this report is sorted based on the Q/CLK toggle ratio. When a register's Q/CLK ratio is greater than .25 (25%), Q toggles every other clock cycle. However, when it is less than 25%, there is a gating opportunity to reduce power.
	Data Aware Gating Efficiency is calculated as:
	`DAGE = 1 - ((toggles at clock pin - 2*toggles at Q pin)/(root clock toggles))`

`-report_prefix` *`prefix`*	
	Specifies a prefix for the report in which the top-level summary of the power consumption information is written. All the new report names will be *`prefix`*`.*.rpt`, and the detailed summary will be *`prefix`*`.rpt`.
`-sort` {`internal	switching	leakage	total`}	
	Sorts the instance-based power consumption data by `internal`, `switching`, `leakage`, or `total` power. *Default*: `total`

`-stat`	Reports a set of statistics on the net status after propagation, for example, duty, transition density, or activity. This parameter is useful in viewing and debugging user-defined activity factors after the power run is complete.
	Example:
	`report_power -time_based_report -distribute -out_file power.rpt -stat`
	`----------`
	`-- Net status after propagation --`
	`----------`
	`Net  Transition Density  Duty  Activity  Ref. Freq`
	`----------`
	`VDDA1  0  1  0  1.25e+08`
	`VSSA1  0  0  0  1.25e+08`
	`...`

`-thermal_conductivity_inputs` *`file_name`*	

Specifies a file containing the thermal conductivity value for layers. You can use this parameter to include the thermal conductivity value of the following layers in the power map file:

- Silicon
- Metal
- Dielectric
- Micro Bump (optional for 3DIC design)
- Fill material in micro bump layer (optional for 3DIC design)

The thermal conductivity file format is:
Section 1 (layer type based thermal conductivity value)

```
#default
$layer_type1 $thermal_conductivity_parameter1
$layer_type2 $thermal_conductivity_parameter2
...
```

Section 2 (layer name based thermal conductivity value)

```
#perlayer
$layer1_name $thermal_conductivity_parameter1 $dielectric_thermal_conductivity_1
$layer2_name $thermal_conductivity_parameter2 $dielectric_thermal_conductivity_2
...
```

Here,

- You can either specify both Section 1 (layer type based) and Section 2 (layer name based), or specify one of the two sections. If both sections are specified, then layer name based section takes precedence over layer type based section.
- The layer name based section has thermal conductivity value and its dielectric layer's thermal conductivity value for each metal/via layer.
- If section 2 has no parameters defined, the software will use the default parameters defined in the section 1.

Example:
###default

```
Metal      430
Silicon    250
Dielectric 0.6
```

###Perlayer

```
Metal1    430    0.4
Via1      410    0.5
Metal2    390    0.5
Via2      410    0.5
```

...

For information on the thermal conductivity input file and power map file, refer to the "Static Power, IRdrop and EM Analysis" chapter in *Voltus User Guide*.

`-thermal_leakage_temperature {`*`temp1 temp2 temp3 …`*`}`

Defines the temperature to calculate leakage for the power map file.

`-thermal_leakage_temperature_scale_table {`*`$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3 .....`*`}`

	Specifies the global scale factor for a temperature value to enable scaling of the leakage power values coming from the input dotlib file. This parameter allows you to scale the power values only at certain specific temperatures. This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-thermal_leakage_temperature_scale_table` and `-thermal_leakage_temperature_scale_table_file` are mutually exclusive.
`-thermal_leakage_temperature_scale_table_file` *`filename`*	

	Specifies a file containing the user-defined temperature values and the corresponding scale factors for each library, cell, or instance. The format of the file is:
	`#Type Name temp1 temp2 temp3 temp4`
	`Lib` *`library_name scalefactor@temp1 scalefactor@temp2 scalefactor@temp3 scalefactor@temp4`*
	`Cell` *`cell_name scalefactor@temp1 scalefactor@temp2 scalefactor@temp3 scalefactor@temp4`*
	`inst` *`instance_name scalefactor@temp1 scalefactor@temp2 scalefactor@temp3 scalefactor@temp4`*
	This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-thermal_leakage_temperature_scale_table` and `-thermal_leakage_temperature_scale_table_file` are mutually exclusive.
	File Format Example for the Type Lib:
	`#Type Name 25 50 125 150`
	`Lib fast 1 2 10 100`
	`Lib bufao 1 2 10 100`
	`Lib header 1 2 10 100`
	`Lib pso_ring_wc 1 2 10 100`

`-thermal_material_file` *`filename`*	
	Specifies the name of a file containing the user-specified substrate layer thickness. The layer thickness value for the power map generation flow can either be a user-specified value or derived from the technology file. If both the values are available, the user-specified value takes precedence.
`-thermal_power_map_bottom_layer` *`layer_name`*	
	Writes zero metal density tables for layers lower than the user-specified layer in the power map file. This parameter prevents duplication of metal density tables when merging the top and block thermal power map files.
`-thermal_power_map_density_table_tiles {`*`Xint Yint`*`}`	
	Specifies the number of tiles in the X and Y directions for the metal density table.
`-thermal_power_map_file` *`file_name`*	
	Generates the power map file with the specified name in the output directory.
`-thermal_power_map_format {simple | stack }`	
	Generates a tile-based power map file in the simple or stack format to perform accurate thermal analysis. You must use the stack format to support multiple dies in a package for thermal analysis.

`-thermal_power_map_format_version {new|default}`	
	Determines whether to generate the default or the new power map file format. When set to `new`, the power map file format with non-uniform tile numbers for the power (`-thermal_power_map_power_table_tiles`) and metal density tables (`-thermal_power_map_density_table_tiles`) can be generated. The following are the features of the new report format: • version number added to the header "Version : 2020.0" • nx/ny under the "DIE_AREA" section has been removed • XCOR/YCOR for each layer has been removed • nx/ny has been added for the POWER and METAL_DENSITY sections
`-thermal_power_map_header_include_file` *filename*	
	Specifies a file that contains comments or additional information to be included in the header section of the power map file.
`-thermal_power_map_power_table_tiles {`*Xint Yint*`}`	
	Specifies the number of tiles in the X and Y directions for the power table.
`-thermal_power_map_si_unit {true | false}`	
	Specifies to change chip thermal power map units to SI in the thermal power map file. This parameter was introduced to support the thermal power map file format used by foundry. When this parameter is set to `true`, the values for all the power map parameters (length, power, and thermal conductivity) will be reported in the SI units. • **Length Unit:** default is `um` and SI is `m` • **Power Unit**: default is `mW` and SI is `W` • **Metal/Silicon Thermal Conductivity Unit**: default is `mW/(um * C)` and SI is `W/(m * C)` *Default* : `true`
`-thermal_power_map_tiles {`*xint yint*`}`	
	Defines the tile number in the X and Y direction. By default 10*10 will be used.
`-thermal_power_map_top_layer` *layer_name*	
	Writes zero metal density tables for layers higher than the user-specified layer in the power map file. This parameter prevents duplication of metal density tables when merging the top and block thermal power map files.
`-threshold` *value*	
	Filters out the instances whose total power dissipation is more than the specified value. Units are in milliwatts. *Default*: `0`
`-threshold_voltage_group { all |` *group_name*`}`	

	Reports leakage by voltage threshold group in the power report. This parameter supports the argument `all` (default value) or any other string, such as hvt or lvt. *Default*: `all`
`-time_based_report`	
	Writes out a detailed VCD profiling report into the file that you specify.
`-time_unit` *unit*	
	Sets the time unit to be used for power report generation. The default unit is ns (nanoseconds).
`-toggle_rate`	
	Specifies to dump out instance-based activity to the outfile. The toggle rate for each instance is based on the clock frequency to which it belongs. If multiple clocks are reaching the instance, then the fastest clock is used as a reference.
`-use_geometry_cell_size {true | false}`	
	Specifies whether to use the actual geometry or placement bounding box for macro PGV cell size during generation of tile-based power map file. This parameter is used when a macro's power is distributed across multiple tiles. When this parameter is set to `true`, the geometry-based bounding box will be used for power map generation and power will be distributed evenly across tiles. *Default* : `false`
`-view` *view_name*	
	Specifies the active timing view that was created using the multi-mode multi-corner (MMMC) `set_analysis_view -setup|-hold` command. Power analysis is performed for the specified view. The command prints the `MMMC` view name in the output file even if you do not specify the `-view` parameter. If `report_power -view` *view_name* is set, it will override the view set in the `power_view` attribute. However, if you specify the `set_analysis_view -dynamic` and `-leakage` parameters, then the view set using these `set_analysis_view` command parameters has higher priority over the `report_power -view` command. *Default* : Uses current view in Voltus.

Examples

- The following command reports the amount of power consumed by the clock nets clk and test_clk (including leaf flip-flop power) and writes the output information to a power report file named clk_pwr.rep:

```
report_power -clock_network{clk test_clk} \
-include_sequential_in_clock -out_file clk_pwr.rep
```

- The following command reports the amount of power consumed down to hierarchy level 3 from the top and writes the output information to a power report file named hier.rep:

```
report_power -hierarchy 3 -out_file hier.rep
```

- The following command compresses the power report file pg_net.rep with gzip:

```
report_power -out_file pg_net.rep.gz
```

- The following command reports the amount of power consumed by leaf-level instances that match ECO* and writes the

output information to a power report file named inst.rep:

```
report_power -insts ECO* -out_file inst.rep
```

- The following command reports the amount of power consumed by all cells that match buf*6 and writes the output information to a power report file named cell.rep:

  ```
  report_power -cell buf*6 -out_file cell.rep
  ```

- The following command reports the amount of power consumed by all I/O and sequential cells and writes the output information to a power report file named cell_type.rep:

  ```
  report_power -cell_type {io sequential} -out_file cell_type.rep
  ```

- The following command sorts the instance-based power consumption data by leakage power and writes the output information to a power report file named sort.rep:

  ```
  report_power -sort leakage -out_file sort.rep
  ```

- The following command reports the total power of all instances whose total power dissipation is less than 1 milliwatt and writes the output information to a power report file named threshold.rep:

  ```
  report_power -threshold 1 -out_file threshold.rep
  ```

- The following command reports the net switching power, load capacitance, toggle rate, and switching values for each net in the design, reports the top 100 nets with the highest net switching power, and writes the output information to a power report file named net.rep:

  ```
  report_power -net -nworst 100 -out_file net.rep
  ```

- The following command reports the amount of power consumed by each power domain and writes the output information to a power report file named pd_all.rep:

  ```
  report_power -power_domain all -out_file pd_all.rep
  ```

- The following command reports the amount of power consumed by each power domain PD2 and writes the output information to a power report file named pd.rep:

  ```
  report_power -power_domain PD2 -out_file pd.rep
  ```

- The following command reports the amount of power consumed by view view_max-hp-max-lp1 and writes the output information to a power report file named view.rep:

  ```
  report_power -view view_max-hp-max-lp1 -out_file view.rep
  ```

- The following command reports the amount of power consumed by each instance connected to power nets vdd and vdd1 and writes the output information to a power report file named pg_net.rep:

  ```
  report_power -pg_net {vdd vddl} -out_file pg_net.rep
  ```

- The following command reports the leakage for the threshold voltage group HVT:

  ```
  report_power -leakage -threshold_voltage_group { HVT }
  ```

- The following command writes out a detailed vector profiling report for the combinational and macro cell types in the `vectorprofile.rpt` file:

  ```
  report_power -time_based_report -out_file vectorprofile.rpt -cell_type {combinational macro}
  ```

- The following command is an example of generating multiple reports at the same time:

  ```
  report_power -out_dir static_power -report_prefix design -net {name1} -insts {inst_name}
  ```

  The following reports will be generated:

  ```
  static_power/design.rpt
  static_power/design.netpwr.rpt
  static_power/design.instpwr.rpt
  ```

- The following command generates a power map file in the simple format for the specified tile:
  `report_power -out_file powermeter.txt -thermal_power_map_file powermap.txt -thermal_power_map_tiles { 4 6 } -thermal_power_map_format simple`
- The following command specifies to report the power content for the `inst1` hierarchical instance in the `report.hierinstpwr.rpt` report:
  `report_power -output static -report_prefix report -clock_network {all} -instances * -no_wrap -hierarchy {all} -hierarchical_instances inst1 -cell_type {all}`
- Example of the output of the power report.

```
*------------------------------------------------------------------------
*        -  - Version  64-bit
*       Date & Time:    2008-Jan-27 15:49:51 (2008-Jan-27 23:49:51 GMT)
*------------------------------------------------------------------------

*       Design: dtmf_chip
*
*       Liberty Libraries used:
*               /sev//DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_1p08v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_0p8v_1p08v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_0p8v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_lvt_ss_1p08v_125c.lib
*               /sevDTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_lvt_ss_0p8v_1p08v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_lvt_ss_0p8v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_hvt_ss_1p08v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_hvt_ss_0p8v_1p08v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_hvt_ss_0p8v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_rvt_ss_1p08v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_rvt_ss_0p8v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_lvt_ss_1p08v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_lvt_ss_0p8v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_hvt_ss_1p08v_125c.lib
*     /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_hvt_ss_0p8v_125c.lib
*     /sev/TIMING/max/rom_512x16A_ss_1v08_125c_syn.lib
*     /sev/DTMF_CHIP/TIMING/max/pllclk_slow.lib
*               /sev/DTMF_CHIP/TIMING/max/iometroil_cmos10lp_hvt_ss_1p08v_2p3v_2p5v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/iometroil_cmos10lp_hvt_ss_0p8v_2p3v_2p5v_125c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_rvt_ff_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_rvt_ff_1p1v_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_lvt_ff_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_lvt_ff_1p1v_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_hvt_ff_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_hvt_ff_1p1v_1p32v_m40c.lib
*
```

```
/sev/DTMF_CHIP/TIMING/min/scmetro_cmos10lp_rvt_ff_1p32v_m40c.lib
*
/sev/DTMF_CHIP/TIMING/min/scmetro_cmos10lp_lvt_ff_1p32v_m40c.lib
*
/sev/DTMF_CHIP/TIMING/min/scmetro_cmos10lp_hvt_ff_1p32v_m40c.lib
*
/sev/DTMF_CHIP/TIMING/min/rom_512x16A_ff_1v32_m40c_syn.lib
*
/sev/DTMF_CHIP/TIMING/min/pllclk_fast.lib
*               /sev/DTMF_CHIP/TIMING/min/iometroil_cmos10lp_hvt_ff_1p32v_2p7v_2p5v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/iometroil_cmos10lp_hvt_ff_1p1v_2p7v_2p5v_m40c.lib
*
*       Power Domain used:
*               Rail:     VDD_AO        Voltage:       1.32
*               Rail: VDD_TDSPCore      Voltage:       1.32
*               Rail: VDD_TDSPCore_R    Voltage:       1.32
*
*       Parasitic Files used:
* /sev/DTMF_CHIP/SPEF/dtmf_chip.decoup.spef
*
*       DEF Files used:
* /sev/DTMF_CHIP/DEF/dtmf_chip.def
*
*       Power Units = 1mW
*       Time Units = 1e-09 secs
*       report_power
--------------------------------------------------------------------------------
  Total Power
--------------------------------------------------------------------------------
Total Internal Power:      0.01943(62.6774%)
Total Switching Power:     0.01057(34.0968%)
Total Leakage Power:       0.001(3.2258%)
Total Power:               0.031
--------------------------------------------------------------------------------
Group      Internal      Switching     Leakage       Total  Percentage
Power      Power         Power         Power  (%)
--------------------------------------------------------------------------------
Sequential    0.0006738   0.0002215   4.045e-05   0.0009358     3.019
Macro         0.0004916   1.515e-05   0.0002609   0.0007677     2.477
IO            0.01743     0.009085    0.0003172   0.02683      86.56
Combinational 0.0008324   0.001247    0.0003815   0.002461      7.941
--------------------------------------------------------------------------------
Total 0.01943     0.01057       0.001       0.031         100
```

```
--------------------------------------------------------------------------------
Rail                  Voltage   Internal   Switching   Leakage Total  Percentage
                     Power     Power      Power        Power  (%)
--------------------------------------------------------------------------------
VDD_AO                 1.32  0.0008238   0.0003404   0.0002694    0.001434       4.625
VDD_TDSPCore           1.32   0.001126    0.001022   0.0003931    0.002541       8.197
VDD_TDSPCore_R         1.32  5.833e-06   3.253e-07   8.371e-07   6.996e-06     0.02257


Clock                   Internal   Switching     Leakage       Total  Percentage
                            Power      Power         Power         Power  (%)
--------------------------------------------------------------------------------

m_digit_clk               0           0           0           0           0
m_ram_clk                 0           0           0           0           0
m_dsram_clk               0           0           0           0           0
m_spi_clk           1.018e-06    1.09e-06   2.572e-08   2.134e-06    0.006884
m_rcc_clk           1.282e-05   1.513e-05   2.672e-07   2.821e-05     0.09103
m_clk               5.039e-05   7.278e-05   7.658e-07   0.0001239      0.3998
refclk              1.11e-05   3.017e-05   2.488e-07   4.152e-05       0.134
--------------------------------------------------------------------------------
Total               7.533e-05   0.0001192   1.307e-06   0.0001958      0.6317
--------------------------------------------------------------------------------

*       Power Distribution Summary:
*  Highest Average Power:     IOPADS_INST/Prefclkip (PBCSCT2B):     0.003727
*  Highest Leakage Power: DTMF_INST/ROM_512x16_0_INST (rom_512x16A): 8.899e-05
*               Total Cap:      2.46221e-10 F
*               Total instances in design:  8525
*               Total instances in design with no power:    24
*                 Total instances in design with no activty:    24
*               Total Fillers and Decap:     8
--------------------------------------------------------------------------------
```

- Next command creates a leakage report (see example report below):

```
report_power -leakage
*--------------------------------------------------------------------------------
*    Tempus Timing Signoff Solution 08.10-b050_1 (64bit) 08/21/2008 19:23 (Linux 2.6)
*
*
*       Date & Time:    2008-Aug-24 21:34:17 (2008-Aug-25 04:34:17 GMT)
*
*--------------------------------------------------------------------------------
*
```

```
*       Design: dma_mac
*
*       Liberty Libraries used:
*               DATA/timing_libs//tcbn65lp_LVL_c060217wc0d90d9.lib
*               DATA/timing_libs//tcbn65lpcgwc.lib
*               DATA/timing_libs//tcbn65lplvtwc.lib
*               DATA/timing_libs//tcbn65lpwc.lib
*               DATA/timing_libs//tsdn65lpa1024x32m8f_100a_wc.lib
*               DATA/timing_libs//tsdn65lpa128x16m8f_100a_wc.lib
*               DATA/timing_libs//tcbn65lpcgwc0d72.lib
*               DATA/timing_libs//tcbn65lplvtwc0d72.lib
*               DATA/timing_libs//tcbn65lpwc0d72.lib
*               DATA/timing_libs//tsdn65lpa1024x32m8f_0p84v_wc.lib
*               DATA/timing_libs//tsdn65lpa128x16m8f_0p84v_wc.lib
*
*       Power Domain used:
*               Rail:      VDDlu        Voltage:       0.84
*               Rail:       VDDm        Voltage:       1.08
*               Rail:        VDD        Voltage:       1.08
*
*       Power Units = 1mW
*
*       Time Units = 1e-09 secs
*
*       report_power -out_file Leakage.rep -leakage
*
--------------------------------------------------------------------------------
Total Power
--------------------------------------------------------------------------------
Total Leakage Power:       0.1879
--------------------------------------------------------------------------------
Group                           Leakage       Percentage
                                Power         (%)
--------------------------------------------------------------------------------
Sequential                        0.0127       6.756
Macro                               0.1556        82.8
IO                                     0           0
Combinational                     0.01962       10.44
--------------------------------------------------------------------------------
Total                              0.1879         100
--------------------------------------------------------------------------------
Rail                  Voltage   Leakage       Percentage
                                Power         (%)
```

```
-------------------------------------------------------------------------
VDDau                         0          0           0
VDDlu                      0.84    0.06477       34.47
VDDm                       1.08   0.003587       1.909
VDD                        1.08     0.1195       63.62

Clock                             Leakage       Percentage
                                  Power         (%)
-------------------------------------------------------------------------
phy_rxclk_2                               0           0
phy_txclk_2                       0.0004786      0.2547
phy_rxclk_1                               0           0
phy_txclk_1                        3.39e-05     0.01804
clk                                0.001723      0.9171
-------------------------------------------------------------------------
Total                              0.002236        1.19
-------------------------------------------------------------------------



-------------------------------------------------------------------------

*       Power Distribution Summary:
*               Highest Average Power: ethernet_mac_2/tx_fifo_module_from_dma/fifo1/dual_port_
   ram_4_tx_fifo/ram2P1024x32/ram (TSDN65LPA1024X32M8F):      0.02424
*               Highest Leakage Power: ethernet_mac_2/tx_fifo_module_from_dma/fifo1/dual_port_
   ram_4_tx_fifo/ram2P1024x32/ram (TSDN65LPA1024X32M8F):      0.02424
*               Total Cap:      1.01625e-10 F
*               Total instances in design: 29235
*               Total instances in design with no power:  3596
*          Total instances in design with no activty:   115
*               Total Fillers and Decap:     0
-------------------------------------------------------------------------

Total leakage power = 0.187903 mW
  Cell usage statistics:
  Library tsdn65lpa128x16m8f_1v08, 1 cells ( 0.003900%) , 0.01016 mW (    5.407063%  )
  Library tsdn65lpa1024x32m8f_1v08, 6 cells ( 0.023402%), 0.145427 mW (    77.394568%  )
  Library tcbn65lp_LVL_c060217wc0d90d9 , 120 cells ( 0.468037%), 0.000175605 mW    ( 0.093455%  )

  Library unknown , 3881 cells ( 15.137096%) , 0.00630072 mW ( 3.353178%  )
  Library tcbn65lpcgwc , 118 cells ( 0.460236%) , 0.000625092 mW ( 0.332667%  )
  Library tcbn65lplvtwc , 3762 cells ( 14.672959%) , 0.0110529 mW ( 5.882233%  )
  Library tcbn65lpwc , 17751 cells ( 69.234370%) , 0.0141619 mW ( 7.536835%  )
```

report_thermal

```
report_thermal
-help
```

Specifies to invoke Celsius Thermal Solver to run die-only thermal analysis. The command generates the die temperature map file that is used by Voltus to run temperature-aware power/IR/EM analysis. Before specifying this command, use the `set_thermal_analysis_mode` command to set the parameters for performing chip-centric thermal analysis.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.

Examples

- The following command specifies to run die-only thermal analysis:
  ```
  report_thermal
  ```

Related Topics

- set_thermal_analysis_mode

report_unannotated_nets

```
report_unannotated_nets
[-help]
[-file filename]
-type annotationType
```

Shows the percentage of nets annotated by a specific type and also prints the names of all nets that are not annotated by that type. Annotation information is stored on a per-file basis for activity, VCD, TCF, and user commands to provide detailed annotation reports when requested.

`report_unannotated_nets` can be invoked multiple times with different report types (`-type`). You must specify this command before `report_power`.

Parameters

`-file filename`	Specifies the text file name that includes the annotation report information. If no file is specified, the report will be written to the default output file `annotation_type.log`, where *type* is taken from the `-type` argument specified.
`-help`	Outputs the command usage and a brief description about the command parameters.

`-type` *annotationType*	The type can be any of the following: • `all` Shows annotation reports for all types. • `activity` Shows annotation data for all types of activity specifications including TCF, VCD, SDC, TWF, and user commands that set activity. • `cap` Shows annotations from the SPEF files. • `fsdb` Shows annotations from the FSDB files. • `saif` Shows annotations from the SAIF files. • `tcf` Shows annotations from the TCF files. • `timing` Shows annotations from the TWF files. This type is applicable only to the dynamic power analysis flow. • `vcd` Shows annotation from the VCD files.

Example

- The following command dumps the nets in the netlist that are missing in the vcd file:
  ```
  report_unannotated_nets -type vcd -file vcd.log
  ```
  The net names are written in the file `vcd`.log.
- The following command dumps the nets in the netlist which do not have an activity annotated from any source (VCD/TCF/SDC/TWF/user defined) and writes the net names in the file `unannotatednets.log`:
  ```
  report_unannotated_nets -type all -file unannotatednets.log
  ```

reset_power_activity

```
reset_power_activity
[-help]
```

Resets all activity in the database. You can specify this command to reset all activity and power information in a given session.

> You can reset activity from an activity file using `read_activity_file -reset`, reset activity for nets, pins, and ports using `set_switching_activity -reset`, and reset the default switching activities using `set_default_switching_activity -reset`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_power_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: man `reset_power_activity`

set_default_switching_activity

```
set_default_switching_activity
[-help]
[-black_box_density value]
[-black_box_duty value]
[-block master_cell_name]
[-duty value ]
[-global_activity factor ]
[-hierarchy hierarchy_name ]
[-input_activity factor ]
[-period value ]
[-sequential_activity factor ]
[-reset]
[-reset_type { global_activity | seq_activity ... }]
[-clock_gates_enable { activity_factor }]
[-integrated_clock_gate_ratio num ]
[-combinational_clock_gate_ratio num ]
[-clock_gates_output_ratio num ]
[-clip_activity_to_domain_freq {true | false}]
[-clock_gates_output {activity_factor}]
[-name activity_name]
[-macro_activity factor]
```

Specifies the switching activity for all primary inputs, nets, and other devices in the design whose activity has not been previously defined through user attributes, the toggle count format (TCF) file, the value change dump (VCD) file, or the tracing of the clock network.

You can use this command to specify user-defined activities at the design level.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_default_switching_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_default_switching_activity`
`-black_box_density value`	
	Specifies to set the default transition density on all the output pins of all blackboxes in the design. The `-black_box_density` and `-black_box_duty` parameters must be used together. If one or both of these parameters are not specified, then the software will use the average transition density from all the input pins of the blackboxes.
`-black_box_duty value`	
	Specifies to set the default duty on all the output pins of all blackboxes in the design. The `-black_box_density` and `-black_box_duty` parameters must be used together. If one or both of these parameters are not specified, then the software will use the average transition density from all the input pins of the blackboxes.

`-block` *master_cell_name*	
	Specifies the name of the master cell for activity setting. The parameter allows you to define the user-defined switching activity for a block partition. This parameter is useful for assigning block-based activity. The `-block` parameter is applicable only to the block-based distributed power flow.
`-clip_activity_to_domain_freq {true | false}`	
	Specifies to perform domain-based activity clipping. This parameter clips data paths to 1x frequency whereas the clock networks are still at 2x frequency. This parameter allows you to have more control to determine the rate of clipping with respect to frequency.
`-clock_gates_enable {` *activity_factor* `}`	
	Specifies the activity at the enable pin(s) of the clock gating cells.
`-clock_gates_output {`*activity_factor*`}`	

	Specifies the activity at the output pin(s) of the clock gating cells. This works for both ICGs as well as combinational clock gating cells.
	Example:
	`set_default_switching_activity -global_activity 0.02 -clock_gates_output 2.0` (clock is transparent)
	Transition density at the output pin(s) of clock gating cells is calculated by:
	`transition_density = clock_domain_frequency * clock_gates_output_value`
	If the fastest clock frequency is 1 GHz for a clock gating cell, then:
	`transition_density = (1e9) * 2.0 = 2,000,000,000`

`-clock_gates_output_ratio` *num*	

	Specifies to control transition density at the output pin(s) of the integrated and combinational clock gating cells. This parameter is equivalent to specifying the parameters `-integrated_clock_gate_ratio` and `-combinational_clock_gate_ratio` together with the same value.
	
	For example, the `set_default_switching_activity - clock_gates_output_ratio 0.6` is equivalent to `set_default_switching_activity -integrated_clock_gate_ratio 0.6 -combinational_clock_gate_ratio 0.6`.
	
	Note: This parameter has a cascading effect when there are multiple levels of ICG cells on the clock path. For example, when `-clock_gates_output_ratio` is set to `0.5`, the first level ICG cell will have 0.5 ratio of the root clock's toggle density, the second level ICG cell will have 0.5*0.5=`0.25` ratio of the root clock's toggle density, and the third level ICG cell will have 0.5*0.5*0.5=`0.125` ratio of the root clock's toggle density, and so on.
	
	Note: For dynamic power analysis, fastest clock on the path is used for calculating output toggle density. However, for static power analysis, weighted averaging of input toggle densities is used.
	
	Example for static power analysis:
	`set_default_switching_activity -global_activity 0.02 -clock_gates_output_ratio 1.0`
	
	Transition density for either a combinational or an integrated clock gating cell with 'N' inputs is calculated by:
	`output_pin_transition_density = (summation) weighted_input_transition_density = clock_gates_output_ratio * [(duty_input_pin1 * Td_input_pin1) + (duty_input_pin2 * Td_input_pin2) + ... + (duty_input_pinN * Td_input_pinN)]`
	
	Given a 2:1 clock Mux and assuming duty of 0.5 for all input pins:
	Td is the toggle density for each input pin of the instance `(Td_pinSe1 , Td_pinI0 , Td_pinI1).`
	
	`output_pin_transition_density = 1.0 * [0.5 * Td_pinSe1 + 0.5 * Td_pinI0 + 0.5 * Td_pinI1]`

`-combinational_clock_gate_ratio` *num*	
	Specifies to set the propagation ratio for combinational clock gate cell outputs. When specified, this parameter sets the output activity of any instance that is identified as a combinational clock gating cell to *num* times the fastest transition density arriving on the instance's clock inputs. This parameter cannot be specified with the `-hierarchy` parameter. **Notes:** • This parameter does not apply to combinational clockgates with a constant data input, that is, a select pin tied low. • For dynamic power analysis, fastest clock on the path is used for calculating output toggle density. However, for static power analysis, weighted averaging of input toggle densities is used. For a detailed static power analysis example, refer to '`-clock_gates_output_ratio`', limiting calculations only to combinational clock gates.
`-duty` *value*	Specifies the duty cycle, which is the probability that the signal is a logical 1. **Note:** You must specify the `-period` parameter when assigning a duty. *Default*: `0.5` (equivalent to a 50 percent duty cycle)
`-global_activity` *factor*	

	Specifies the average number of times that all unset nodes switch in a clock cycle. If not specified the fastest domain frequency will be used unless one turns off clock domains. **Note**: Due to activity precedence, the `-global_activity` and `-sequential_activity` parameters of the `set_default_switching_activity` command are mutually exclusive. If both the parameters are specified, `-global_activity` takes precedence.
`-hierarchy` *`hierarchy_name`*	
	A hierarchical name that the global activity is being assigned to. If not provided, the whole design is assumed.
`-integrated_clock_gate_ratio` *`num`*	
	Specifies to set the propagation ratio for integrated clock gate (ICG) cell outputs. When specified, this parameter sets the output pin of all identified ICG instances to *`num`* times the fastest transition density arriving on clock inputs of the instance. This parameter cannot be specified with the `-hierarchy` parameter.
`-input_activity` *`factor`*	
	Specifies the average number of times that a primary input switches in a clock cycle. It can be any positive number. *Default*: `0.2` (Switches once every five clock cycles.)
`-macro_activity` *`factor`*	
	Specifies the activity factor for macros in a design. The allowed values for this parameter are between `0` to `1`. If you set the macro activity to `.30`, 30% of macros in the design would be selected for switching in a given cycle during dynamic simulation. When `-macro_activity` is specified for the static power analysis, all output pins of the macros are assigned the specified activity. *Default*: `0.2`
`-name` *`activity_name`*	

	Allows you to set the activity level for the specified jitter activity name. You can define a low/high activity level for jitter analysis.
	Examples:
	`set_default_switching_activity -name low -clock_gates_output 0 -global_activity 0.01`
	`set_default_switching_activity -name high -clock_gates_output 2 -global_activity 0.2`

`-period` *`value`*	Specifies the default operating period (1/frequency). It is the period that the activity is referenced to. If not specified, the software uses the *dominant* clock in the design. The specified period will only be applied to nets that do not have reference clock (ref_clock) associated with the net. If no clock is specified, the software generates an error and prompts you to enter one. Units in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). **Note:** If you enter the period as a negative number, the software reverts back to using the default frequency. *Default*: The time unit is defined in the first `.lib` file read during design import.
`-sequential_activity` *`factor`*	

	Specifies the activity of outputs of the sequential logic. This value will be used at the output ports of flops for propagating activity. If the sequential activity is not specified, propagation will be done for the full design starting from the input ports.
`-reset`	Resets all specified options back to default values.
`-reset_type { global_activity | seq_activity ... }`	
	Specifies to selectively reset the global specifications. **Note:** You must use the `-reset_type` parameter in conjunction with the `-reset` parameter.

Activity Precedence

See "Activity Precedence".

Examples

- The following command specifies that the primary inputs in the design will switch once every ten clock cycles, with a clock period of 7520 ns and a duty cycle of 50 percent:
```
set_default_switching_activity -input_activity 0.10 -period 7520ns -duty .5
```
- The following command specifies that all unswitched nodes in the design will switch once every ten clock cycles, with a clock period of 7520 ns and a duty cycle of 50 percent:
```
set_default_switching_activity -global_activity 0.10 -period 7520ns -duty .5
```
- The next example defines the switching activity, sequential activity, and the period:
```
set_default_switching_activity -input_activity 0.2 -period 10 \
    -sequential_activity 0.1
```
- The next example sets the default global activity to 0.5 for the hierarchy dma_dut:
```
set_default_switching_activity -global_activity 0.5 -hierarchy dma_dut
```
- The following command resets the sequential and global activity specification:
```
set_default_switching_activity -reset -reset_type
{sequential_activity,global_activity}
```
- The following command will define the input activity, sequential activity and macro activity for the design block A:
```
set_default_switching_activity -input_activity 0.1 -sequential_activity 0.1 -macro_activity 0.1 -block A
```

Related Commands

- `read_activity_file`
- `get_activity`
- `reset_power_activity`
- `set_switching_activity`

set_dynamic_power_simulation

```
set_dynamic_power_simulation
[-help]
[-period value]
[-resolution value]
[-reset]
[-activity_pattern {time1  activity_name1  time2  activity_name2 .... }]
```

Specifies the period and resolution for a dynamic power simulation. This command is used to specify the simulation period in the dynamic vectorbased/vectorless and power-up flows.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_dynamic_power_simulation` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_dynamic_power_simulation`
`-activity_pattern {`*`time1 activity_name1 time2 activity_name2 ....`*` }`	
	Divides the total simulation time based on the windows given in the `activity_pattern` parameter. For a particular window, it uses the defined activity specified with the `set_default_switching_activity` command. When the user-defined low cycle and high cycle time is specified for the input/sequential activity, all the clocks will be in the low or high cycle until completely past the cycle transition time. Following is the use model of this parameter: • `set_default_switching_activity` **`-name low -input_activity 0.05 -sequential_activity 0.1`** `-clock_gates_output 0.2` • `set_default_switching_activity` **`-name high -input_activity 0.1 -sequential_activity 0.2`** `-clock_gates_output 2.0` • `set_dynamic_power_simulation -period 15ns -resolution 50ps` **`-activity_pattern {0 high 5ns low 10ns high}`** Here, high activity is used for 0ns to 5ns, low activity is used for 5ns to 10ns, and high activity is used for 10ns to 15ns.
`-period` *`value`*	Specifies the simulation period that dynamic power analysis will use. The supported time units are `s`, `ms`, `us`, `ns`, and `ps`. If no explicit time unit is specified, the default `ns` (nanosecond) will be applied.
`-reset`	Resets all specified options back to default values.
`-resolution` *`value`*	Specifies the transient time step size or resolution. The supported time units are `s`, `ms`, `us`, `ns`, and `ps`. If no explicit time unit is specified, the default `ps` (picosecond) will be applied.

Examples

- The following command sets up the period and resolution for a dynamic simulation:
  `set_dynamic_power_simulation -period 5ns -resolution 100ps`

set_instance_temperature_file

```
set_instance_temperature_file
[-help]
file
[-reset]
```

Specifies the instance temperature file to support thermal-aware leakage. You can use this command to do thermal-aware power calculation. Based on the temperature specified in the file, the software will calculate temperature and leakage power.

In the presence of a trilib set, the internal and leakage power will be interpolated as the per the given temperature specification.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_instance_temperature_file` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_instance_temperature_file`
`file`	Specifies the name of the instance temperature file.
`-reset`	Resets the temperature file.

set_metal_density_options

```
set_metal_density_options
[-help]
[-metal_density_include_file filename]
-power_grid_library pgv_path
[-reuse_md_db_path MD_db_path]
-tech_file tech_file_path
-thermal_conductivity_file thermal_file
[-user_defined_metal_density_file user_defined_md_file}]
[-vtm_densitytable_tile {Xint Yint} ]
[-vtm_densitytable_tile_size {Xsize Ysize}]
[-vtm_header_include_file user_defined_header_content_file]
-vtm_path output_path
```

Sets the parameters for extracting the metal density information. This command must be specified before the `extract_metal_density` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_metal_density_options` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_metal_density_options`
`-metal_density_include_file filename`	
	Specifies the file for listing the additional variables that can be included when extracting metal density information. For example, you can specify "`setvar report_with_lef_layer_names true`" to output the LEF layer names.
`-power_grid_library pgv_path`	
	Specifies the path to the power cell libraries (`.cl`).
`-reuse_md_db_path MD_db_path`	
	Specifies to reuse the generated metal density database for `extract_metal_density`. When specified, the PGV/tech file specified with `set_metal_density_options` will be ignored.
`-tech_file tech_file_path`	
	Specifies the path to the extraction technology file to be used for top-level power-grid extraction.
`-thermal_conductivity_file thermal_file`	

Specifies a file containing the thermal conductivity value for layers. You can use this parameter to include the thermal conductivity value of the following layers in the power map file:

- Silicon
- Metal
- Dielectric
- Micro Bump (optional for 3DIC design)
- Fill material in micro bump layer (optional for 3DIC design)

The thermal conductivity file format is:
Section 1 (layer type based thermal conductivity value)

```
#default
$layer_type1 $thermal_conductivity_parameter1
$layer_type2 $thermal_conductivity_parameter2
...
```

Section 2 (layer name based thermal conductivity value)

```
#perlayer
$layer1_name $thermal_conductivity_parameter1 $dielectric_thermal_conductivity_1
$layer2_name $thermal_conductivity_parameter2 $dielectric_thermal_conductivity_2
...
```

Here,

- You can either specify both Section 1 (layer type based) and Section 2 (layer name based), or specify one of the two sections. If both sections are specified, then layer name based section takes precedence over layer type based section.
- The layer name based section has thermal conductivity value and its dielectric layer's thermal conductivity value for each metal/via layer.
- If section 2 has no parameters defined, the software will use the default parameters defined in the section 1.

Example:
###default

```
Metal      430
Silicon    250
Dielectric 0.6
```

###Perlayer

```
Metal1    430    0.4
Via1      410    0.5
Metal2    390    0.5
Via2      410    0.5
```

...

`-user_defined_metal_density_file` *user_defined_md_file*}

Allows you to define the metal density in the user-defined metal density flow.

The format of the user-defined metal density file is:

```
CELL <cell_name>
CELL_AREA [ xmin ymin xmax ymax | x1 y1 x2 y2 x3 y3 x4 y4 ... x1 y1]     # Must specify if cell PGV is not imported
LAYER <layer_name(s)>                          # Layer to be defined metal density; multiple layers are available (LEF layer name based)
ALL <metal_density_value>                      # To define metal density for whole cell area as 'ALL' only 'ALL' can be used now
END_LAYER
END_CELL
```

Example:

```
CELL Macro1
CELL_AREA 0 0 32 16
LAYER Metal1 Metal2 Metal3
ALL 0.2
END_LAYER
LAYER Via1 Via2
ALL 0.05
END_LAYER
END_CELL

CELL Macro2
CELL_AREA 0 0 32 16
LAYER Metal1 Metal2 Metal3
ALL 0.2
END_LAYER
LAYER Via1 Via2
ALL 0.05
END_LAYER
END_CELL
```

`-vtm_densitytable_tile {`*`Xint Yint`*`}`

Specifies the number of tiles in the X and Y directions for the metal density table.

`-vtm_densitytable_tile_size {`*`Xint Yint`*`}`

Specifies the size of tiles instead of tile number in the X and Y directions for the metal density table.

`-vtm_header_include_file` *`user_defined_header_content_file`*

Allows you to define extra header information like file location.

`-vtm_path` *`output_path`*

Specifies the directory path to the metal density file output folder.

Examples

- The following command specifies to set the metal density parameters:

```
set_metal_density_options \
-power_grid_library "$DesignDir/PGV/tech_pgv/techonly.cl" \
-tech_file $DesignDir/QRC/qrcTechFile \
-vtm_path ./vtm_MD_db \
-thermal_conductivity_file $DesignDir/MISC/thermal_conduct.txt \
-vtm_densitytable_tile {10 10} \
-vtm_header_include_file $DesignDir/MISC/user_header.txt
```

Related Topics

- extract_metal_density

set_power

```
set_power
[value]
[-cap value]
[-fanout_limit value]
[-reset]
[-leakage value ]
[-pg_pin {pg_pin}]
[-pg_net railname ]
[-pwl_waveform]
[-sticky]
[-dynamic_async_mode { mode_name }]
[-dynamic_switch_pattern pattern ]
[-dynamic_switch_event { switching_events }]
[-scale_factor value }
[-include_ddv_modes {pattern_list}]
[-internal]
[-switching value ]
[-clocks {all| clock_name }]
[-exclude_clocks]
[-exclude_ddv_modes {pattern_list}]
[-base_cells cell_name value ]
[-insts inst_list value ]
[-pin pin_name pin_power ]
[-force]
[-custom_macro_pwl_waveform filename]
[-repeat]
[-ascii_power_file filename ]
[-trigger_time_adjustment time]
[-no_propagation]
[-static_mode {  custom_label_name | mode_name }]
[-signal_net netname]
[-slew value]
[-default_dynamic_pgv_mode { mode_name }]
```

Specifies the amount of power for all or part of the design. This command is used to define user-specified power on cells and instances. You must specify this command before `report_power`.

The `set_power` command can accept a cell/instance list using the `get_cells`/`get_lib_cells` commands. You can use the `get*` commands to create a collection of cells/instances whose name matches the supplied pattern list. Therefore, providing you with the flexibility in providing lists through use of regular filter patterns. This feature is supported in both static and dynamic analysis engines.

Parameters

```
-ascii_power_file filename
```

	Specifies the name of an instance list file. This file contains the list of instances in the following format: `instance_name power_value <pin_name>` Here, • `instance_name` is the instance name to which power is to be distributed. • `power_value` is the power value for the instance in watts. • `pin_name` is an optional argument to specify the pin name. When specified, allows you to assign power to the specific pin of the instance. • The three fields should be in one line, and each instance should be in a new line.
`-base_cells cell_name value`	
	Specifies the cell name. If the cell or instance name is not specified, the power will be distributed to all rails. If you want to specify power for a group of cells/instances, use the `get_cells`/`get_lib_cells` commands. The `-base_cells` parameter can accept these commands to create a collection of cells/instances whose name matches the supplied pattern list. For example, `-base_cells[get_lib_cells NAND*] 2mw`
`-cap value`	
	Specifies the capacitance value to all instances with nets more than or equal to the fanout limit (`-fanout_limit`).
`-clocks {all| clock_name }`	
	Specifies the power value for clock networks. You can specify the power value for a specific clock network or the clock network power value for the whole design. **Note:** This parameter cannot be used with the `-base_cells`, `-insts` and `-pg_net` parameters.
`-custom_macro_pwl_waveform filename`	
	Specifies to use a custom trigger file to apply instance or cell PWL waveforms. Given one master cell PWL waveform, the software can automatically generate scaled PWL waveforms of the other companion cells defined in the trigger file proportional to the cell internal power. **Note**: This parameter must be used with `set_power -dynamic_switch_pattern` in the vectorless flow. For more information on the trigger file format, see Trigger File Format.
`-default_dynamic_pgv_mode { mode_name }`	
	Specifies the current characterization mode to be used during dynamic power and jitter analyses. This parameter can be specified when using a PGV with multiple functional modes. When this parameter is specified, the Power Calculation engine uses this preferred mode in scheduling the switching events for the macro blocks, and saves these events in the trigger file. The use model of this parameter is: `set_power -base_cells {} | -insts {} -default_dynamic_pgv_mode { mode_name }` This parameter is mutually exclusive to the `-dynamic_switch_pattern` parameter. If a cell or instance has both the `-dynamic_switch_pattern` and `-default_dynamic_pgv_mode` parameters defined in the same Tcl script, `-dynamic_switch_pattern` takes higher priority.
`-dynamic_async_mode { mode_name }`	

	Specifies to trigger asynchronous macros based on the length of the PWL waveform to create a single continuous waveform. This parameter allows you to specify the asynchronous mode for an instance or a cell. The start and end of the PWL saved in the PGV must have similar current value in order to avoid sudden jump in the current.
	Use model:
	`set_power -dynamic_async_mode {mode} [-insts {name} | -base_cells {name}]`
	Example:
	`set_power -base_cells pll -dynamic_asynchronous_mode {READ_C}`
	Here, power analysis will trigger the `READ_C` mode and automatically splice the waveform based on its length.

`-dynamic_switch_event { switching_events }`	
	Specifies the switching events for cells, instances, and pins along with their rise and fall time. `switching_events` is a string of toggle time followed by "r" or "f", where "r" is for rise toggle and "f" is for fall toggle. The `-dynamic_switch_event` parameter should not be specified with the `-repeat` parameter.
`-dynamic_switch_pattern pattern`	
	Specifies stimulus pattern on the pins of an instance and cell. Specify this parameter only in the dynamic mode for the type `instance` and `cell`. You can also use the `set_power -pin` parameter with the `-dynamic_switch_pattern` parameter to apply pin-specific dynamic switch pattern. That is, you can now apply different switching patterns on different pins, as shown in the following example: • `set_power -insts FF14 -dynamic_switch_pattern { r - - f - - }  -pin Q` The `-pin` parameter can be specified for both the `-insts` and `-base_cells` parameters. Pin-based dynamic switch pattern has the following limitations: • "`-insts *`" and "`-base_cells *`" are not supported. All other wildcards except "`*`" is supported. For example "`AND*`" or "`top*`" is supported. • The value of the `-no_propagation` parameter should be the same for all pins. The `pattern` is applied to all pins (input and output) on the instance/cell. It is a string. `pattern` specifies the switching pattern. If there is more than one clock domain with timing windows on a pin, only the windows on the fastest clock will be used, and the pattern will be applied to that clock period. It is a string of `0`, `1`, `r`, and `f` values. `0` for cycles not to switch and `1` for cycles to switch. `r` is for rise only and `f` for fall only. The pattern string should include enough digits to cover all of the clock cycles in the simulation period. If there are not enough digits, the pattern will be padded with 0 values at the end. If there are too many, the pattern will be truncated to the number of applicable cycles. Transitions will be scheduled in the center of each applicable timing window. **Notes** • `0` and `1` is supported only in the probability-based vectorless flow. It is not supported in the state-propagation-based vectorless flow and the vector-based flow. • In case of macros, you can specify the mode names, that is, which mode switches in which clock cycle `-dynamic_switch_pattern { mode1 - mode2 - }`.

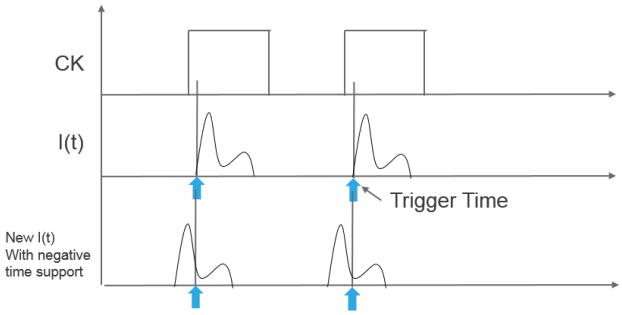
	• The dynamic switch pattern (DSP) events can be state propagated. Instance power is reported for instances directly annotated using DSP or those receiving activity through state propagation.
	• The dynamic switch pattern does not apply to clock network instances.
	You can specify "-" to indicate that instances will not switch in a particular clock cycle. If "-" is the first value in the string of values specified within {}, you must insert a space before "-", as shown below:
	• no switching: `-dynamic_switch_pattern { - - }`
	• no switching in the 1st clock period: `-dynamic_switch_pattern { - r }`
	Examples:
	In the following example, instance will not switch in the 4th clock cycle:
	`set_power -dynamic_switch_pattern { f r f - r }`
	In the following example, the mode `write` will switch in the 1st clock cycle, the mode `read` will switch in the 2nd and 4th clock cycle, and none of the modes will switch in the 3rd clock cycle:
	`set_power -dynamic_switch_pattern { write read - read }`
	Note: Wildcards are supported only for `-dynamic_switch_pattern` with `-pin`. When `-dynamic_switch_patter`n is applied for `-instance` or `-cell` without the `-pin` parameter, wildcards are not supported.

`-exclude_clocks`	Excludes clock networks when specifying a scale factor or power specification for the whole design. This parameter cannot be used with the `-base_cells`, `-insts` and `-pg_net` parameters.
`-exclude_ddv_modes {`*`pattern_list`*`}`	
	Excludes the current characterization (or Dynamic Detailed View) modes that match with the specified patterns. This parameter allows you to control the modes to be switched, instead of switching all the current characterization modes defined in the power-grid view. **Use Model** `set_power -insts|-base_cells` *`name`* `-exclude_ddv_modes {`*`pattern_list`*`}` Here: • *`pattern_list`* is a list of mode names, separated by space or comma • the instance, cell, and pattern names can contain wildcard characters
`-fanout_limit` *`value`*	
	Specifies the fanout limit for nets. If an instance has nets more than or equal to the fanout limit, its switching/internal/leakage power is taken to be zero. When this parameter is specified, the user-defined power value is applied to all the driver instances that have nets more than or equal to the fanout limit.
`-force`	
	Specifies to apply the `set_power` command even if there are Verilog mismatches. For dynamic or DEF-based static analysis, this parameter allows you to apply `set_power` to the DEF instances that do not exist in Verilog.
`-help`	Outputs the command usage.
`-include_ddv_modes {`*`pattern_list`*`}`	

	Includes only the current characterization (or DDV) modes that match with the specified patterns. This parameter allows you to control the modes to be switched, instead of switching all the current characterization modes defined in the power-grid view. **Use Model** `set_power -insts|-base_cells name -include_ddv_modes {pattern_list}` Here: • *pattern_list* is a list of mode names, separated by space or comma • the instance, cell, and pattern names can contain wildcard characters
`-internal`	Specifies the target internal power of the design, cell, or instance. To specify internal power for cells and instances, you can use the `-internal` parameter in conjunction with the `-base_cells` or `-insts` parameter.
`-insts` *inst_list value*	
	Specifies the instance name. If the cell or instance name is not specified, the power will be distributed to all rails.
`-leakage`	Specifies the target leakage power of the design, cell, instance, or power net. To specify leakage power for cells and instances you can use `-leakage` option in conjunction with `-base_cells` or `-insts` option. To specify leakage power of a certain power net in the design you can use `-pg_net` option along with `-leakage` option.
`-no_propagation`	
	Modifies the behavior of the `-dynamic_switch_pattern` parameter so that the user-defined switch pattern is applied only after state-propagation is complete. The use model of the command parameter is: `set_power -dynamic_switch_pattern {pattern} -no_propagation` The `-no_propagation` parameter is only applicable in conjunction with the `-dynamic_switch_pattern` parameter. When the `-no_propagation` parameter is added to the `-dynamic_switch_pattern` parameter, the dynamic power engine performs state propagation first, overrides the single instance switching event by the user-defined dynamic switch pattern, and does no further propagation.
`-pg_net` *railname*	
	Optional. The specified power or pwl for the cell, instance or design is applied to the specified rail. When not specified, the power or pwl is applied to all the rails associated with the instance. This option is generally used for MSMV cells. When this option is not specified the behavior is different for pwl and power options. When not specified and power option is used, the power is applied for the whole design, cell, or instance. When not specified and pwl option is used, the pwl is applied to every rail of the cell or instance.
`-pg_pin {`*pg_pin*`}`	
	Specifies the switch pattern for a specific PG pin of an instance. This parameter must be specified with the `-dynamic_switch_pattern` parameter.
`-pin` *pin_name pin_power*	

	Specifies pin-wise power numbers for a specific instance. This parameter allows you to control the power assigned to power ground pins of an instance. Following is the use model of this parameter:
	`set_power -insts <inst_name> -pin <VDDA> <pin_power>`
	`set_power -insts <inst_name> -pin <VDDB> <pin_power>`

`-pwl_waveform`	Specifies that power argument (`power | pwl`) is a waveform rather than a power number.
`-repeat`	Repeats dynamic switch pattern of all the clock cycles in the simulation period.
`-reset`	This parameter causes all previous `set_power` commands to be ignored. Note: You can selectively reset the instance/cell specifications. In addition, you can also selectively reset the cell and instance specifications of a list of objects through the `get_cells` and `get_lib_cells` commands.
`-static_mode { custom_label_name | mode_name }`	
	Specifies the mode name or the custom label name associated with a cell or instance for computing instance current from PGV with static multi-voltage multi-frequency (MvMf) modes. This current is used to compute the instance's internal power, and its switching and leakage power components would be set to zero. The use model of this parameter is given below: `set_power -insts -static_mode READ` If you specify custom label of a static MvMf mode, currents would be taken from it directly. Alternatively, if only mode name is specified for a cell/instance, the software snaps to the nearest custom label defined within this mode, based on the instance's voltage and frequency to get the currents. If you specify a dynamic mode with `set_power -static_mode`, average current from the mode will be used in the static MvMf flow.
`-scale_factor value`	
	Scales the power value by the specified scale factor. This parameter allows you to set a user-defined scale factor in the trigger file to scale the dynamic current of the PGV. To modify the scale factor, use the `-reset` parameter with the `-scale_factor` parameter. **Example**: `set_power -leakage -reset -scale_factor 5 -instance A`
`-signal_net netname`	
	Specifies the signal net name for power assignment. When this parameter is specified, the driver instances connected to the signal net will be assigned the user-defined power value.
`-slew value`	
	Specifies the slew value to all instances with nets more than or equal to the fanout limit (`-fanout_limit`).
`-sticky`	Applies the simulation based PWL waveform as is, to the specified cell/instance for dynamic power analysis. Use this option only if you have specified `-pwl_waveform`.
`-switching value`	

	Specifies the target switching power of the design, cell, or instance. To specify switching power for cells and instances, you can use the `-switching` parameter in conjunction with the `-base_cells` or `-insts` parameter.
`-trigger_time_adjustment` *time*	
	Specifies to adjust the trigger time of the current waveforms for the various functional modes (read/write/idle) of the custom macros. This parameter allows you to start the waveform before the trigger time (negative time support) to account for the setup time for the operation, as shown below:  The specified time will be applied to the computed trigger time for each mode in the trigger files generated by power analysis. For example, if the trigger time (from timing window/vector) is computed as 1ns, 2ns, and 3ns, and if you specify `set_power -trigger_time_adjustment -100ps`, the software will adjust the trigger time to .9ns, 1.9ns, and 2.9ns.

value	Specifies power in milliwatts during static power calculation or piecewise linear (`-pwl`) current waveform, during dynamic power calculation for cell or instance. The command accepts a unit specification (e.g. 10w).
	The format of the PWL waveform is:
	`{time1 current1 time2 current2 ...time_n current_n}`
	Time is in ns and current in mA.
	Example:
	`set_power -pwl_waveform -pg_net {} -base_cells BUFX2MTL {0 0 10 0 10.1 5 10.5 0}`
	Here, at `0ns` its value is `0mA`, at `10ns` its value is `0mA`, at `10.1ns` its value is `5mA`, and so on.

Examples

- The following command sets power for vdd rail to 10 milliwatts:
  ```
  set_power -pg_net vdd 10mw
  ```
- The following command sets a power value of .005 milliwatts on power net vdd for instance clkin_neg_L14_I1:
  ```
  set_power -pg_net vdd -insts Top/A/clkin_neg_L14_I1 .005
  ```
- The following command sets a power value of .005 milliwatts for any cell named INVX2 in the design:
  ```
  set_power -base_cells INVX2 .005mw
  ```
- The following command defines a piecewise linear waveform of {0 0 10 0 10.1 5 10.5 0} for the cell of type BUFX2MTL for all power rails.
  ```
  set_power -pwl_waveform -pg_net {} -base_cells BUFX2MTL {0 0 10 0 10.1 5 10.5 0}
  ```
- The following command specifies the leakage power for the cell DFFRX1.

```
set_power -leakage -base_cells DFFRX1 10mw
```

- The following commands specify the target leakage power for power nets VDD1 and VDD2.

```
set_power -leakage -pg_net VDD1 100mw
set_power -leakage -pg_net VDD2 200mw
```

- The following command specifies the PWL description of current for the instance SEHD130_1024X32X1CM8. The -sticky option tells the software to use the specified PWL simulation based waveforms as is.

```
set_power -pwl_waveform -insts SEHD130_1024X32X1CM8 -sticky {0 0 1 1 2 2 7 0}
```

- The following command scales the total power of the instance i_1 by a scale factor of 2:

```
set_power -scale_factor 2 -insts i_1
```

- The following command sets the total power of the design to 20mw while keeping the clock network power the same:

```
set_power 20mw -exclude_clocks {all}
```

- The following command assigns a switching power of 3mw to all instances of the cell AND1:

```
set_power -base_cells AND1 -switching 3mw
```

- The following command assigns 2mw power to all cells that match the specified pattern:

```
set_power -base_cells[get_lib_cells NAND*] 2mw
```

- The following command specifies that mode1 switches in the first cycle, mode2 switches in the third cycle and no switching in others:

```
set_power -dynamic_switch_pattern { mode1 0 mode2 0 }
```

- The following command resets power of all cells that match the specified pattern:

```
set_power -reset -base_cells[get_cells A/*]
```

- The following command does not schedule DDV mode `scan*`:

```
set_power -base_cells { * } -exclude_ddv_modes { scan* }
```

- The following command does not schedule DDV mode `scan*` for instance "`A1`":

```
set_power -insts { A1 } -exclude_ddv_modes { scan* }
```

- The following command only schedules DDV mode names with the `*read*` and `*write*` pattern:

```
set_power -base_cells { * } -include_ddv_modes { *read*, *write* }
```

- The following commands specify the toggle time for the pin names Q7 and Q6:

```
set_power -dynamic_switch_event { 1.1ns r 5.5ns f } -insts clk_b_i2 -pin Q7
set_power -dynamic_switch_event { 2.2ns r 6.6ns f } -insts clk_b_i2 -pin Q6
```

- The following command specifies the toggle time for the cell name BUFX8:

```
set_power -dynamic_switch_event { 2.22ns r 6.66ns f } -base_cells BUFX8
```

Trigger File Format

For specifying a custom trigger file, you must use the `-custom_macro_pwl_waveform` parameter. The format of the trigger file is:

Trigger File Format	Description
#Cell names	

MASTER CELL <*cellname1*>	Name of the reference cell.
COMPANION_CELLS <*cellname1 cellname2 … >*	Name of the cells that will use scaled PWL from the master cell.
#Mode and conditional details for trigger	
MODE_NAME <*mode1*> CONDITIONAL_INPUT <conditional input statement for cellname1>	Mode name and conditional input statement for the cell. Conditional input of current for the given mode.
CONDITIONAL_PIN <pin_name> (rise|fall|both)	Conditional pin statement for the cell. Signal pin for which the tool should rise or fall.
CONDITIONAL_STIMULUS_FILE <usim1.txt>	Simulation vector detail in the given mode.
USER_PWL_FILE <pwl1.txt>	User-specified PWL file that contains information about the PWL waveforms (power/ground currents). For an example of a PWL file, see PWL Format Example.
END_MODE	End mode section
#End cell section	
END	End cell section

Example of a Custom Macro Power Trigger File

```
MASTER_CELL        SRAM_A1
COMPANION_CELLS    { SRAM_A2, SRAM_A3, SRAM_A4, SRAM_A5 }
      MODE_NAME WRITE
               CONDITIONAL_INPUT = ( !SD & !SLP & !CEB & !WEB )
               CONDITIONAL_PIN = CLK { RISE }
               USER_PWL_FILE write_rise.txt
       END_MODE
      MODE_NAME READ
               CONDITIONAL_INPUT = ( !SD & !SLP & !CEB & WEB )
               CONDITIONAL_PIN = CLK { RISE }
               USER_PWL_FILE read_rise.txt
       END_MODE
      ...
END
```

Example of “USER_PWL_FILE” (Multiple regions)

```
UNIT CURRENT mA
UNIT TIME ns
       NET VSS
               REGION { 0 0 0.1 -1 0.2 -2 0.3 -3 0.4 -10 0.5 -50 0.6 -40 0.7 -30 0.8 -20 1 0 2 0 3 0 3.9 0 4 -
0.1 4.5 -0.2 5 -4 5.5 -3 6 -2 7 0 }
       NET VDD
               REGION { 0 0 0.1 1 0.2 2 0.3 3 0.4 10 0.5 50 0.6 40 0.7 30 0.8 20 1 0 2 0 3 0 3.9 0 4 0.1 4.5 0.2
5 4 5.5 3 6 2 7 0 }
```

```
      NET VDDM
              REGION { 0 0 0.1 1 0.2 2 0.3 3 0.4 10 0.5 50 0.6 40 0.7 30 0.8 20 1 0 2 0 3 0 3.9 0 4 0.1 4.5 0.2
5 4 5.5 3 6 2 7 0 }
```

- pwl_waveform

set_power_analysis_temperature

```
set_power_analysis_temperature
[-help]
temperature
```

Sets the temperature for static power calculation.

In the presence of a trilib set, the internal and leakage power will be interpolated as the per the given temperature specification.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_power_analysis_temperature` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_power_analysis_temperature`
`temperature`	Specifies the temperature for static power calculation. ⚠ The static power calculation software gets the temperature value of each instance from the following sources (in order of precedence): • Instance temperature file • `set_power_analysis_temperature` command • The current operating condition temperature

set_power_include_file

```
set_power_include_file
[-help]
file
[-block master_cell_name]
[-reset]
```

Specifies a power analysis include file. You can use the `set_power_include_file` command to specify certain commands of the dynamic engine which do not have a Voltus equivalent. For more information on these commands, refer to the Power Analysis Options chapter.

The `set_power_include_file` command is required only if the `power_static_netlist` attribute is set to `def`.

⚠ The `set_power_include_file` command is used to pass the legacy power analysis commands and `report_power` command options only in the dynamic mode (vectorbased and vectorless). If the `power_method` attribute is set to `static`, the `set_power_include_file` command is ignored.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_power_include_file` parameter. For a detailed description of the command and all of its parameters, use the man command: `set_power_include_file`
file	Name of the include file. This file will include all commands that cannot be translated to Voltus commands. In addition, you can also include the `report_power` command options to generate different power reports.
`-block` *master_cell_name*	
	Specifies the master cell name to which the powermeter options are to be applied. The parameter allows you to apply certain commands of the dynamic engine only to a specific block partition. This parameter allows you to apply the include options to a specific block instead of applying them globally to all blocks. The `-block` parameter is applicable only to the block-based distributed power flow.
`-reset`	Resets all options back to default values.

Examples

- The following command specifies a power include file of `power.inc`:

  ```
  set_power_include_file power.inc
  ```

- The following example shows the contents of the `power.inc` file.

  # power.inc file must use the pre-defined design and power objects as "design" # and "ChipPwr" respectively. The power output object is not supported.

  ```
  design calCellsToIgnore BUFX1
  ChipPwr calRailVoltage VDD 1.2

  report_power -net -outfile Net.rep
  ```

```
report_power -cell {INV*} -outfile Invpower.rep
```

set_power_output_dir

```
set_power_output_dir
[-help]
dir
[-reset]
```

Specifies the name of the directory that power information will be output to.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_power_output_dir` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_power_output_dir`
dir	Specifies the name of the directory in which power analysis output information is written.
`-reset`	Resets all options back to default values.

Example

The following command specifies the output directory called `static_power_AvallOn` in which the power information is written.

```
set_power_output_dir static_power_AVallOn
```

set_switching_activity

```
set_switching_activity
[-help]
[-reset]
[-activity factor | -transition_density transition_density ]
[-clock clock_name]
[-duty value]
[-inst instance_name]
[-net net_name | -port port_name | -pin pin_name | -input_port port_name |-output_port port_name | -
bidir_portport_name]
[-period value]
[-integrated_clock_gate_ratio num]
[-combinational_clock_gate_ratio num]
[-hierarchy hierarchy_name]
[-scale value]
[-cell cell_name]
[-force]
[-quiet]
[-unclocked]
```

Specifies activity for nets, pins, and ports. This command is used to specify user-defined activities at specific pins/nets/ports. You must specify this command before `report_power`.

The `set_switching_activity` command can accept a net/pin/port list using the `get_pins`/`get_ports`/`get_nets` commands. You can use the `get*` commands to create a collection of nets/pins/ports whose name matches the supplied pattern list. Therefore, providing you with the flexibility in providing lists through use of regular filter patterns. This feature is supported in both static and dynamic analysis engines.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_switching_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_switching_activity`
`-activity factor`	
	Specifies the activity for a net, pin, or top-level port. The *`activity_factor`* specifies the number of times the net, pin, or port switches in a clock cycle. Enter any positive number. If if the activity is set to 0.5, the activity happens once for every two clock cycles. If the activity is set to 2.0, the activity happens two times per clock cycle. There is no default value.
`-bidir_port port_name`	
	Specifies the name of the bidirectional port. The port names are in the following format: bidirectional`-`*`port`*. This parameter supports wildcards (`*`).
`-cell cell_name`	

	Specifies the activity on all output pins for all instances of the specified cell.
`-clock` *clock_name*	
	Assigns global activity factor to all nets in domain of *clock_name* if and only if *clock_name* is the fastest clock in the domain set. To use this option, `-clock` and `-activity` must be used without any other options.
`-combinational_clock_gate_ratio` *num*	
	Specifies to set the propagation ratio for combinational clock gate cell outputs. When specified, this parameter sets the output activity of any instance that is identified as a combinational clock gating cell to *num* times the fastest transition density arriving on the instance's clock inputs. You can specify this ratio for a net, pin, port, or clock. This parameter cannot be specified with the `-hierarchy` parameter.
`-force`	
	Specifies to apply user-defined activities for specific nets, pins, and ports, without having to load a verilog netlist. When a verilog netlist is not specified, the `set_switching_activity` command does not support the `-cell` parameter, and the net/pin/port list using the `get_pins/get_ports/get_nets` commands.
`-transition_density` *transition_density*	
	Specifies the transition density for a specific net, pin, or top-level port. Transition density specifies the number of transitions per second. it can be any positive number. There is no default value. **Note:** If you specify the `-transition_density` parameter in conjunction with the `-period` parameter, the software issues a warning and the `-period` parameter is ignored.
`-duty` *value*	
	Specifies the duty cycle, which is the probability that the signal is a logical 1. The `-period` parameter must be specified to add a duty. If not specified, the `set_switching_activity` command picks this value from the external TWF or SDC through the Common Timing Engine (CTE). *Default*: Uses the duty specified by the `set_default_switching_activity` command if the `-duty` parameter is not specified.
`-hierarchy` *hierarchy_name*	
	A hierarchical name that an activity is being assigned to.
`-integrated_clock_gate_ratio` *num*	
	Specifies to set the propagation ratio for integrated clock gate (ICG) cell outputs. When specified, this parameter sets the output pin of all identified ICG instances to *num* times the fastest transition density arriving on clock inputs of the instance. You can specify this ratio for a net, pin, port, or clock. This parameter cannot be specified with the `-hierarchy` parameter.

`-input_port` *`port_name`*	
	Specifies the name of the input port. The port names are in the following format: *`input-port`*. This parameter supports wildcards (`*`)
`-inst` *`instance_name`*	
	Specifies the activity on a specific instance.
`-net` *`net_name`*	
	Specifies the name of the net. This parameter supports wildcards (`*`).
`-output_port` *`port_name`*	
	Specifies the name of the output port. The port names are in the following format: *`output-port`*. This parameter supports wildcards (`*`)
`-period` *`value`*	
	Specifies the period that the activity is referenced to. If not specified, a default frequency is used. If a negative number is specified, the static power calculation engine reverts back to the default frequency. Units in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). **Note:** If you specify the `-period` parameter in conjunction with the `-transition_density` parameter, the software issues a warning and the `-period` parameter is ignored. *Default*: The time unit is defined in the first `.lib` file read during design import. If the time unit is not defined, the period specified in the `set_default_switching_activity` command is used.
`-pin` *`pin_name`*	
	Specifies the name of a pin. The pin names are in the following format: *`instance-name + default-hierarchyarchy-separator + pin`*. This parameter supports wildcards (`*`)
`-port` *`port_name`*	
	Specifies the name of a top-level port. The port names are in the following format: *`top-level-port`*. This parameter supports wildcards (`*`)
`-quiet`	Suppresses all error and warning messages generated when the `set_switching_activity` command is run.
`-reset`	Resets all specified options back to default values. The reset option should be the first option specified, otherwise all options set prior to this will be reset. **Note:** You can selectively reset the pin/port/net specifications. In addition, you can also selectively reset the pin/port/net specifications of a list of objects through the `get_pins`/`get_ports`/`get_nets` commands.
`-scale` *`value`*	
	Specifies to scale frequencies of the specified clock domains. This parameter allows you to scale transition densities of all instances associated with the clock domain by the specified scale factor to compute internal and switching power. This parameter must be specified with the `-clock` parameter. The `-scale` parameter is only applicable to static power and dynamic vectorless power analysis.

`-unclocked`	Applies *`activity`* (with reference *`period`*) to all nets that have no clock domain (`-unclocked`). To use this option, `-unclocked` must be used with `-activity` and `-period` with no other options.

Activity Precedence

See "Activity Precedence".

Examples

- The following command specifies that net `n189` will switch once every ten clock cycles, with a clock period of `7520ns` and a duty cycle of 50 percent:
  ```
  set_switching_activity -net n189 -activity 0.10 -period 7520ns -duty .5
  ```
- The following command sets the transition density of port `out_cpu_top` to `20000` transitions per second:
  ```
  set_switching_activity -port out_cpu_top -transition_density 20000
  ```
- The following command applies a transition density of `1e+06` to all input ports that match the specified pattern:
  ```
  set_switching_activity -port[get_ports "mode*" -filter {port_direction==in}] -transition_density 1e+06
  ```
- The following command resets the previously defined activity on all nets that match the specified pattern:
  ```
  set_switching_activity -reset -net[get_nets n23*]
  ```
- The following command specifies the clock gate ratio for the clock domain `sys_clk`:
  ```
  set_switching_activity -clock sys_clk -integrated_clock_gate_ratio 0.8 -combinational_clock_gate_ratio 0.5
  ```
- The following command specifies to set the default switching activity for all other clock domains of the design:
  ```
  set_default_switching_activity -integrated_clock_gate_ratio 1.0 -combinational_clock_gate_ratio 1.0
  ```

set_thermal_analysis_mode

```
set_thermal_analysis_mode
[-help]
[-boundary_condition_file filename]
[-power_map_file filename]
[-temperature_map_file filename]
[-thermal_output_dir directory_name]
[-tool_path path]
```

Sets the parameters for performing chip-centric thermal analysis after generation of the Voltus thermal model (VTM). This command must be specified before the `report_thermal` command.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-boundary_condition_file filename`	

	Specifies the name of the boundary condition file. The format of the boundary condition file is:
	`#Length Unit : um`
	`#Temperature Unit: C`
	`die_thickness=100`
	`front Tfix=25`
	`back Tfix=25`
	`top Tfix=50`
	`bottom Tfix=50`
	`left Tfix=25`
	`right Tfix=25`

`-power_map_file filename`	
	Specifies the name of the generated power map file.
`-temperature_map_file filename`	
	Specifies the name of the output temperature map file. Each analysis run would create a directory named “ThermalDir_<number>” automatically. Soft links to the power mapping file and boundary condition file would be created inside the directory. The analysis’s final result is a temperature map named as “`<DieName>_TemperatureMap.dat`”, and is saved in this directory.
`-thermal_output_dir directory_name`	
	Specifies a custom output directory for thermal analysis results instead of using the default `ThermalDir`. This parameter gives you better control over where your output files are stored, making it easier to manage results across multiple runs.
`-tool_path path`	

	Specifies the full path to the bin folder of the thermal analysis tool (Celsius).

Examples

- The following command specifies to set thermal analysis parameters:

```
set_thermal_analysis_mode -tool_path $CELSIUSTOOL \
-boundary_condition_file $bcDir/bc_100.txt \
-temperature_map_file temperatureMap_100 \
-thermal_output_dir thermal_dir_new \
-power_map_file vtm_output/top.vtm.txt
```

Related Topics

- report_thermal

set_twf_attribute

```
set_twf_attribute
[-help]
[-clock clock_name]
[-clock_edge {rise | fall}]
[-clock_frequency value]
[-fall_delay {min:max or single_value}]
[-fall_slack {min:max or single_value}]
[-fall_slew {min:max or single_value}]
[-net net_name]
[-pin pin_name]
[-reset]
[-rise_delay {min:max or single_value}]
[-rise_slack {min:max or single_value}]
[-rise_slew {min:max or single_value}]
[-type {data | clock}]
[-shift_arrival_time time]
[-clear_twf_attribute]
```

Specifies static timing analysis information. The command allows you to override timing parameters, such as slew and clock, on desired pins/nets from an existing timing windows file (TWF).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_twf_attribute` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_twf_attribute`
`-clear_twf_attribute`	
	Specifies to reset the TWF file values for a specific pin or net. The use model of this parameter is: `set_twf_attribute -clear_twf_attribute -pin/-net <name>`
`-clock clock_name`	
	Specifies the clock name. This indicates that the nets and pins that are timing constrained have arrival times (timing windows) caused by this clock.
`-clock_edge {rise | fall}`	
	Specifies the capturing clock edge direction: rise or fall.
`-clock_frequency value`	
	Specifies the clock frequency. You must specify the `-clock` parameter along with the `-clock_frequency` parameter to assign the clock for which the frequency needs to be updated.

`-fall_delay {min:max` or `single_value}`	
	Specifies the fall delay for the pin. This parameter is not supported in the static power analysis flow.
`-fall_slack {min:max or single_value}`	
	Specifies the fall slack for the pin.
`-fall_slew {min:max or single_value}`	
	Specifies the fall slew for the pin.
`-net net_name`	
	Specifies the net name. The `-net` and `-pin` parameters are mutually exclusive.
`-pin pin_name`	
	Specifies the pin name. The `-net` and `-pin` parameters are mutually exclusive.
`-reset`	Resets all specified options back to default values.
`-rise_delay {min:max` or `single_value}`	
	Specifies the rise delay for the pin. This parameter is not supported in the static power analysis flow.
`-rise_slack {min:max` or `single_value}`	
	Specifies the rise slack for the pin.
`-rise_slew {min:max` or `single_value}`	
	Specifies the rise slew for the pin.
`-shift_arrival_time time`	

	Specifies to filter the idle period of the current waveform in the dynamic power analysis flow. This parameter is useful in removing the idle period of the current waveform result with long idle period (small current) before the 1st high current peak because of TWF delay annotation. The parameter allows to offset idle period from the TWF delay values.
	The use model is given below:
	`set_twf_attribute -shift_arrival_time 3.0ns`
	
	The default value is 0ns. The supported time units are s, ms, us, ns, and ps. The default unit is ns. You must specify a positive value only. A positive value means shift right in the time line or add more delay to the original arrival time.
	
	You can specify the `-clock` parameter to limit the changes to a clock domain. In addition, you can further add the `-pin/-net` parameter to limit the changes to a block. The following are examples of shifting different delays to different clock domains because different clocks may have different clock insertion delays at the top level:
	
	`set_twf_attribute -shift_arrival_time 1.2ns -clock clk_a`
	`set_twf_attribute -shift_arrival_time 2.3ns -clock clk_b`
	
	If a clock domain is not specified, the delay will be applied to all the specified nets/pins.
	
	Note: The final rise/fall delays for the annotated nets/pins after the shift has been applied are reported under the "Shift Arrival Time Report" section of the detailed TWF Attributes report. This report is generated using the `power_report_twf_attributes detailed` attribute.
	
	The following example illustrates a sample command and report snippet:
	
	Command:
	
	`set_twf_attribute -shift_arrival_time 3.0ns`
	
	Report:
	
	`Clock domain annotated`
	`  - CK1_1`
	`  - CK1_2`
	`  - CK2_1`
	`  - CK2_2`
	
	`Rise Delay  Fall Delay  Net Name`
	`1e-09  1e-09  CK1_1`
	`1e-09  1e-09  CK1_2`
	`1e-09  1e-09  CK2_1`
	`...`

`-type {data | clock}`	
	Specifies whether the net or pin is part of the clock or data network.

Examples

- The following command specifies to use the following TWF attributes for the net `netB1`:

```
set_twf_attribute -net netB1 -rise_delay 2.1978:2.1978 -rise_slew 0.05:0.05 -rise_slack 0.2741:3.6961
-fall_delay 2.1819:2.1819 -fall_slew 0.05:0.05 -fall_slack 0.237:3.6948 -clock CK1_1 -clock_frequency 100 -
type data
```

- The following commands reset the TWF values for the "`DFF11/DFF/Q`" pin to default, and then sets the rise/fall delay:

```
set_twf_attribute -clear_twf_attribute -pin "DFF11/DFF/Q"
set_twf_attribute -pin "DFF11/DFF/Q" -rise_delay 0.35 -fall_delay 0.45 -clock CLK1
```

set_virtual_clock_network_parameters

```
set_virtual_clock_network_parameters
[-help]
[-cell cell_name]
[-clock list_of_clocks]
[-library library_name]
[-max_fanout value]
[-wire_load_model wireload_model]
[-reset]
```

Specifies to estimate power for a virtual clock tree. When the command is specified, the software reports the power of the clock network based on the buffers inserted in the clock tree. It also reports the number of buffers used for implementing the clock tree and the depth of the clock tree.

You can use this command for fast power estimation of clock network power of a pre-CTS netlist.

This command should be invoked before running the `report_power` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for `each` set_virtual_clock_network_parameters parameter. For a detailed description of the command and all of its parameters, use the man command man `set_virtual_clock_network_parameters`.
`-cell` *cell_name*	
	Specifies the name of the buffer or inverter cell that has to be used to build the clock tree.
`-clock` *list_of_clocks*	
	Specifies the list of clocks for which the clock tree needs to be implemented. The default is for all the clocks.
`-library` *library_name*	
	Specifies the name of the library which contains the cell.
`-max_fanout` *value*	
	Specifies the maximum fanout that the buffer cell can drive.
`-wire_load_model` *wireload_model*	
	Specifies the wire load model used to compute the wire load model for the nets in the clock network.
`-reset`	Resets all specified parameters back to default values.

Example

The following command specifies to use the cell `BUFX12`, with a maximum fanout value of `2`, to build a virtual clock tree:

```
set_virtual_clock_network_parameters -cell { BUFX12 } -library typical -max_fanout 2 -wire_load_model B0X0
```

write_power_constraints

```
write_power_constraints
[-help]
[-out_file filename]
```

Writes out all the power constraints such as the power analysis mode, activity file specifications, simulation period specifications, and so on, into a file. You can use this command at any time during the session to dump out the power commands/ options specified till that point.

You can source this file into other Voltus/Tempus/Innovus session in order to use the same settings.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_power_constraints` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_power_constraints`.
`-out_file` *`filename`*	Specifies the output file into which the power constraints are to be written.

Example

```
write_power_constraints -out_file test1.tcl
```

write_tcf

```
write_tcf file
[-help]
[-block block_name]
[-exclude_hier_insts list_of_instances]
[-primary_input]
[-sequential_output]
[-pin]
[-macro_output]
```

Writes out the propagated switching activity information to a toggle count format (TCF) file. The TCF file contains switching activity information, toggle count information, and the probability of the net or pin being in the logic1 state.

You must specify this command after the power attributes and before the `report_power` command.

Following is the behavior of the `write_tcf` command in different modes:

- **Command File Mode**: If the `write_tcf` command is specified multiple times within a command file, only the last instance of the command will be honored. This means that any earlier instances will be overridden by the final specification.
- **Interactive Mode**: In contrast, if the `write_tcf` command is specified multiple times during an interactive session, each instance of the command will be honored. This allows for multiple configurations or actions to be performed in sequence.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_tcf` parameter. For a detailed description of the command and all of its parameters, use the man command: man `write_tcf`
`-block` *`block_name`*	
	Specifies to generate the block-level TCF from the top-level design. When specified, it writes out the block-level duty and activity in the generated TCF file. The use model of this parameter is: `write_tcf -pin -block` *`blockname TCF_filename`*
`-exclude_hier_insts` *`list_of_instances`*	
	Excludes the specified hierarchical modules or instances when writing out a pin-based TCF. This parameter allows you to ignore the activity generated for the excluded instances. The use model of this parameter is: `write_tcf -pin -exclude_hier_insts {`*`list_of_instances`*`}`
`-macro_output`	Specifies to generate a TCF file that includes all the activities for the macro outputs in the design.
`-pin`	Specifies to write out pin-based TCF.
`-primary_input`	Generates a TCF file that includes only the primary inputs in the design.
`-sequential_output`	
	Generates a TCF file that includes all the activities for the sequential outputs in the design.
`file`	Specifies the name of the output TCF file.

Example

- The following command writes the propagated switching activity information to TCF file `mmm.tcf`:
  ```
  write_tcf mmm.tcf
  ```
- The following command writes a pin-based TCF for the whole design:
  ```
  write_tcf -pin pin_design.tcf
  ```
- The following command writes out a TCF which only includes activity information for the primary inputs and sequential outputs in the design:
  ```
  write_tcf -primary_input -sequential_output Power.tcf
  ```
- The following command writes out a pin-based TCF that excludes the specified hierarchical instances:
  ```
  write_tcf -pin -exclude_hier_insts {DTMF_INST/CORE_INST DTMF_INST/RAM_128x16 DTMF_INST/RESULTS_INST }
  ```

write_thermal_model

```
write_thermal_model
[-help]
[-leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3 .....}]
[-leakage_temperature_scale_table_file filename]
[-metal_density_file filename]
[-power_db path_to_power_database]
[-vtm_si_unit {true | false}]
[-vtm_conductivity_inputs filename]
[-vtm_densitytable_tile {Xint Yint}]
[-vtm_file filename]
[-vtm_format {simple | stack} ]
[-vtm_format_version {new | default} ]
[-vtm_header_include_file filename]
[-vtm_leakage_temperature {temp1 temp2 temp3} ]
[-vtm_output_dir directory]
[-vtm_powertable_tile {Xint Yint} ]
[-vtm_tile {Xint Yint}]
[-vtm_tile_size {$X_um $Y_um}]
```

Sets the parameters for extracting the metal density information. This command must be specified before the `extract_metal_density` command.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3 .....}`	
	Specifies the global scale factor for a temperature value to enable scaling of the leakage power values coming from the input dotlib file. This parameter allows you to scale the power values only at certain specific temperatures. This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-leakage_temperature_scale_table` and `-leakage_temperature_scale_table_file` are mutually exclusive.
`-leakage_temperature_scale_table_file filename`	

Specifies a file containing the user-defined temperature values and the corresponding scale factors for each library, cell, or instance. The format of the file is:

```
#Type Name temp1 temp2 temp3 temp4
Lib    library_name   scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
Cell   cell_name      scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
inst   instance_name  scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
```

This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-leakage_temperature_scale_table` and `-leakage_temperature_scale_table_file` are mutually exclusive.

File Format Example for the Type Lib:

```
#Type Name         25 50 125 150
Lib   fast         1 2 10 100
Lib   bufao        1 2 10 100
Lib   header       1 2 10 100
Lib   pso_ring_wc  1 2 10 100
```

`-metal_density_file` *filename*

Specifies the VTM metal density file generated by the `extract_metal_density` command.

`-power_db` *path_to_power_database*

Specifies the path to the power database with cell boundary box, location, and rotation information. This is generated by specifying the `set_power_analysis_mode -save_bbx true` parameter.

`-vtm_si_unit {true | false}`

Specifies the unit definition for power and length. The possible arguments are:

- `true`: unit will be in m and W.
- `false`: unit will be um and mW.

Default is `true` with unit m(meter) for length and W(Watt) for power.

`-vtm_conductivity_inputs` *filename*

This parameter is same as the `set_metal_density_options -thermal_conductivity_file` parameter. When you want to overwrite the thermal conductivity defined in the metal density file, you can provide a new thermal conductivity file using the `write_thermal_model -vtm_conductivity_inputs` command.

`-vtm_densitytable_tile {`*Xint Yint*`}`

Specifies the number of tiles in the X and Y directions for the metal density tables only.

`-vtm_file` *filename*

Specifies the name of the output Voltus thermal model file.

`-vtm_format {simple | stack}`

	Generates a tile-based power map file in the simple or stack format to perform accurate thermal analysis. The possible arguments are: • `simple`: generates only power tables without metal density tables. • `stack`: generates both power and metal density tables.
`-vtm_format_version {new | default}`	
	Determines whether to generate the default or the new power map file format. This parameter allows you to have different tile number between the power table and the metal density table. When set to `new`, the power map file format with non-uniform tile numbers for the power (`-vtm_powertable_tile`) and metal density tables (`-vtm_densitytable_tile`) can be generated.
`-vtm_header_include_file` *`filename`*	
	Allows you to define extra header information like file location.
`-vtm_leakage_temperature {`*`temp1 temp2 temp3`*`}`	
	Specifies the leakage temperature values used for trilib (single voltage but different temperatures dot libs) in the power calculation flow to output user-defined temperature points' leakage power tables.
`-vtm_output_dir` *`directory`*	
	Specifies the directory path to the Voltus thermal model output folder.
`-vtm_powertable_tile {`*`Xint Yint`*`}`	
	Specifies the number of tiles in the X and Y directions for the power tables only.
`-vtm_tile {`*`Xint Yint`*`}`	
	Specifies the number of tiles in the X and Y directions for the generated Voltus thermal model file. The default is 10x10.
`-vtm_tile_size {`*`$X_um $Y_um`*`}`	
	Specifies the unit tile size in the X and Y directions for the generated thermal model file.

Examples

- The following command is an example of the trilib Tcl command, that is, uses the temperature in `vtm_leakage_temperature` to calculate the leakage:

```
write_thermal_model \
-power_db powerdb \
-metal_density_file vtm_MD_db/my_chip.rpt \
-vtm_output_dir ./vtm_output \
-vtm_leakage_temperature {20 50 80}
```

- The following command is an example of the mix-usage Tcl command, that is, some instances have trilib data while other instances only have a single library:

```
write_thermal_model \
-power_db powerdb \
-metal_density_file vtm_MD_db/my_chip.rpt \
-vtm_output_dir ./vtm_output \
-vtm_leakage_temperature {20 50 80} \
-leakage_temperature_scale_table {0 1 20 2 50 3 100 10}
```

Related Topics

- `extract_metal_density`

14

Power Route Category Attributes

- route_special Category Attributes

route_special Category Attributes

The root attributes in the `route_special` category are:

- route_special_allow_non_preferred_direction_route
- route_special_avoid_over_core_row_layer
- route_special_block_pin_connect_ring_pin_corners
- route_special_block_pin_route_with_pin_width
- route_special_connect_broken_core_pin
- route_special_core_pin_ignore_obs
- route_special_core_pin_merge_limit
- route_special_core_pin_length
- route_special_core_pin_length_as_inst
- route_special_core_pin_max_via_scale
- route_special_core_pin_refer_to_follow_pin
- route_special_core_pin_reference_macro
- route_special_core_pin_site_rail_width
- route_special_core_pin_snap_to
- route_special_core_pin_stop_route
- route_special_extend_nearest_target
- route_special_jog_threshold_ratio
- route_special_layer_preferred_direction_cost

- route_special_layer_non_preferred_direction_cost
- route_special_pad_pin_min_via_size
- route_special_pad_pin_split
- route_special_pad_ring_use_lef
- route_special_secondary_pin_max_gap
- route_special_secondary_pin_rail_width
- route_special_signal_pin_as_pg
- route_special_split_long_via
- route_special_pg_pin_as_signal
- route_special_target_number
- route_special_target_search_distance
- route_special_time_limit
- route_special_endcap_as_core
- route_special_via_through_to_closest_ring
- route_special_via_connect_to_shape
- route_special_welltap_as_endcap

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `route_special` category, use the following command:

```
get_db -category route_special
```

route_special_allow_non_preferred_direction_route

```
route_special_allow_non_preferred_direction_route {1 | 0}
```

Default: `0`

Allows wrong way routing.

Note: If the metal1 preferred routing direction is vertical and standard cells have their VDD and VSS pins on layer metal1, sroute will route metal1 wires on horizontal direction to continue extending the followpin connections to core rings by this option

route_special_avoid_over_core_row_layer

```
route_special_avoid_over_core_row_layer layer_name
```

Default: `max layer of standard cell pin`

Specifies the number of layers to avoid routing of block pin and pad pin over the core row area.

route_special_block_pin_connect_ring_pin_corners

```
route_special_block_pin_connect_ring_pin_corners {1 | 0}
```

Default: `0`

Specifies that the block wire used for a connection is to have the same width as the pin from which it connects.

route_special_block_pin_route_with_pin_width

```
route_special_block_pin_route_with_pin_width {1 | 0}
```

Default: `0`

Specifies that the block wire used for a connection is to have the same width as the pin from which it connects.

route_special_connect_broken_core_pin

```
route_special_connect_broken_core_pin {1 | 0}
```

Default: `0`

When set to `1`, `sroute` checks the signal pin, second power pin, and obs, and then breaks the followpin to avoid violations. When set to `0`, `sroute` does not check internal geometries and does not break the followpin.

route_special_core_pin_ignore_obs

```
route_special_core_pin_ignore_obs {none | placement_blockage | overlap_obs
| block_halo}
```

Default: `none`

ignores the specified blockage while connecting the follow pin.

route_special_core_pin_merge_limit

```
route_special_core_pin_merge_limit value
```

Default: `0`

Joins follow pin rail gaps smaller than this variable, bigger gaps will remain open. The option is of type *float*.

route_special_core_pin_length

```
route_special_core_pin_length value
```

Default: `0`

Connects core pins of specified length. The option is of type *float*.

route_special_core_pin_length_as_inst

```
route_special_core_pin_length_as_inst {1 | 0}
```

Default: `0`

Connects core pins whose length equals or is greater than the length of instance.

route_special_core_pin_max_via_scale

```
route_special_core_pin_max_via_scale widthPct heightPct
```

Default: If you do not specify this parameter, the software uses a value of 100, which specifies a full-size via.

Controls the length and width of vias at the crossover of core pin rails and stripes. Specify this value as percentages of the width of the via at the top of the stack. Smaller vias leave room for more

routing tracks on layers between the vias.

route_special_core_pin_refer_to_follow_pin

```
route_special_core_pin_refer_to_follow_pin {1 | 0}
```

Default: `0`

If set to `1`, `sroute` generates m2 followpins based on metal1 standard cell pins, if there is no metal2 standard cell pin. This feature is limited in metal2 followpin generation.

route_special_core_pin_reference_macro

```
route_special_core_pin_reference_macro macro
```

Default: `""`

Uses specified macros as references for followpin creation. The option is of type *string*.

route_special_core_pin_site_rail_width

```
route_special_core_pin_site_rail_width {{site1 layer11 Width11 layer12 Width12 …}
{site2 layer21 Width21 layer22 Width22 …} …}
```

Default: `""`

Data_type: string

Defines the followpin layer width for the specified site rail for followpin generation. The followpin width defined by this attribute has higher priority than the width of the same layer defined in the macro LEF, but lower than that defined by the `route_special -core_pin_width` parameter.

Following are the values used with this attribute:

- `site`: Specifies the site name defined in the LEF file.
- `layer`: Specifies the name of the layer defined in the LEF file.
- `width`: Specifies the followpin width on the top and bottom sides of the defined site.

route_special_core_pin_snap_to

```
route_special_core_pin_snap_to {M1_pin | opt_routing_track | grid | half_grid}
```

Default: `M1_pin`

Snaps M2 followpins in the location to save routing tracks in a preventive manner. When set to:

- `M1_pin`: Generated metal2 followpins are snapped to metal1 stdcell pin center.
- `opt_routing_track`: Generated metal2 followpins are snapped to track or half track to save routing tracks.
- `grid`: Generated M2 followpins are snapped to the nearest routing track.
- `half_grid`: Generated M2 followpins are snapped to the nearest half routing track.

route_special_core_pin_stop_route

```
route_special_core_pin_stop_route {atRowEnd atCellEnd}
```

Default: `atRowEnd`

Extends the follow pin wire either to the row-end or cell-end.

route_special_extend_nearest_target

```
route_special_extend_nearest_target {1 | 0}
```

Default: `0`

Extends the nearest target so that it can be connected. The option is of type *bool*.

route_special_jog_threshold_ratio

```
route_special_jog_threshold_ratio value
```

Default: `10`

Defines the distance ratio between a jogging target and a straight target. The option is of type *float*.

route_special_layer_preferred_direction_cost

```
route_special_layer_preferred_direction_cost layer1 cost1 layer2 cost2...
```

Default: `""`

Specifies layer cost of preferred direction. The option is of type `string`.

route_special_layer_non_preferred_direction_cost

```
route_special_layer_non_preferred_direction_cost layer1 cost1 layer2 cost2...
```

Default: `""`

Specifies layer cost of non-preferred direction.

route_special_pad_pin_min_via_size

```
route_special_pad_pin_min_via_size viaSizePercent
```

Default: If you do not specify this parameter, the software uses a value of `20`, which permits vias that are 20% of the standard via size.

Permits the software to reduce the size of vias at pad pins when changing layers, if there is not enough room for a standard size via.

route_special_pad_pin_split

```
route_special_pad_pin_split width spacing
```

Default: `""`

Splits the width and spacing for wide pad pins.

route_special_pad_ring_use_lef

```
route_special_pad_ring_use_lef {1 | 0}
```

Default: `""`

Splits the width and spacing for wide pad pins.

route_special_secondary_pin_max_gap

```
route_special_secondary_pin_max_gap value
```

Default: `0`

Specifies variable value for gaps bigger than this value cause level shifter rail to break. The option type is *float*.

route_special_secondary_pin_rail_width

`route_special_secondary_pin_rail_width` *value*

Default: `0`

Specifies width of followpin rail. The option type is *float*.

route_special_signal_pin_as_pg

`route_special_signal_pin_as_pg {1 | 0}`

Default: `0`

Specifies signal pins as PG pins in pad ring creation. The option type is *bool*.

route_special_split_long_via

`route_special_split_long_via {`*threshold_value step_value offset_value height_value*`}`

Default: If you do not specify this parameter, the software does not split vias.

Splits vias longer than the specified length (`threshold_value`, in micrometers) into smaller vias with specified center-to-center spacing (`step_value`, in micrometers), left/bottom end offset (`offset_value`, in micrometers) and vertical/horizontal height (`height_value`, in micrometers) values. This is independent of the option `-split_via`.

Note: If the offset_value is given as negative, the vias will be placed symmetrically on both ends. The default value of `offset_value` is `-1`.

route_special_pg_pin_as_signal

`route_special_pg_pin_as_signal {`*cellName: pinName*`}`

Default: `""`

Lists the cell/pin information NOT extracted from CPF.

route_special_target_number

`route_special_target_number` *value*

Default: `0` (minimum). Maximum is `100000`.

Specify number of target to connect. The option type is *int*.

route_special_target_search_distance

```
route_special_target_search_distance dist
```

Default: 0

Specifies target search distance. The option type is *float*.

route_special_time_limit

```
route_special_time_limit value
```

Default: 0

Specify time limit (in seconds) that the router is allowed in routing each port. The option type is *float*.

route_special_endcap_as_core

```
route_special_endcap_as_core {1 | 0}
```

Default: 0

Specifies to treat Endcap macro as core macro. The option type is *bool*.

route_special_via_through_to_closest_ring

```
route_special_via_through_to_closest_ring {1 | 0}
```

Default: If you do not specify this parameter, vias connect to all overlapping rings.

Specifies that a via should connect only to the closest ring layer when multiple rings overlap.

route_special_via_connect_to_shape

```
route_special_via_connect_to_shape {padring|ring|stripe|blockring|blockpin|coverpin|nos
hape|blockwire|corewire|followpin|iowire}
```

Default: padring ring stripe blockring blockpin coverpin noshape blockwire corewire followpin iowire

Specifies which shapes vias can connect to.

The option type is *enum.*

route_special_welltap_as_endcap

`route_special_welltap_as_endcap` *string*

Default: ""

When used, `route_special` treats the `Welltap` cells as the `Endcap` cells. Additionally, the `ENDCAP`class cells are ignored by default, when connecting the `followpin` wires.

15

Power-Grid Library Commands

- check_pg_library
- check_pgv
- merge_pg_library
- rename_pg_library
- set_advanced_pg_library_mode
- set_pg_library_mode
- trim_pg_library
- write_pg_library
- write_tech_pg_lib
- write_xpgv

check_pg_library

```
check_pg_library
[-help]
power_library_paths
[-cell_list_file file]
[-check_compatibility]
[-check_parameters]
[-command_write]
[-integrity_check]
[-lef_consistency]
[-libgen_command_file filename]
[-list]
[-metal_density_map]
[-out_dir file]
[-report]
[-report_detail_multiple_voltage_cap file]
[-rule_file file]
[-skip_layer_list_file filename]
[-skip_signal_net_check {true | false}]
[-summary]
[-tap_node_text_write]
[-tech_layers]
[-total_currents]
```

Specifies to perform a comprehensive check of power-grid views (PGV) to be used in rail analysis. These checks include:

- Verify that all the PGV views exist: Early/IR/EM
- Report the dynamic PWL and static current
- Perform a compatibility check on version, layermap and resistivity parameters
- Check any missing information, such as temperature, qrcTechFile location, and so on.

Parameters

`-help`	Outputs the command usage.

power_library_path	
	Specifies the absolute path to the power library (`.cl`) file. You can run multiple power library files (.cl) at the same time.
`-cell_list_file` *file*	

	Specifies a file containing the list of cells for which the LEF consistency checks will be done. This parameter allows you to perform the consistency checks only for the user-defined cells.
	The format of the input file is:
	`CELL1`
	`CELL2`
	`CELL3`

`-check_compatibility`	
	Checks for compatibility between the specified cell libraries. It checks whether the following parameters are the same for the libraries that are to be merged: • Time Units • Capacitance Units • Layout Scale Factor • Manufacturing Grid If one or more of these parameters are not the same, a warning message appears stating that the libraries are not compatible and cannot be merged. It also specifies the parameters that are different. Based on the above electrical/physical parameter check, a compatibility report (compatibility.report) with pass/fail is generated. Fail implies that the library is not compatible and cannot be used in Voltus rail analysis. The use model of this parameter is: `check_pg_library -check_compatibility {`*lib1 lib2*`}`
`-check_parameters`	

	Checks for the following electrical parameters of the specified cell libraries: • `CINT_MIN and CINT_MAX`: Intrinsic device capacitance • `CG_MIN and CG_MAX`: Grid capacitance • `RON_MIN and RON_MAX`: On-resistance for powergates • `IDSAT_MIN and IDSAT_MAX`: Saturation current for powergates • `ILEAK_MIN and ILEAK_MAX`: Leakage current for powergates • `IAVG_MIN and IAVG_MAX`: Static current • `IPEAK_MIN and IPEAK_MAX`: Dynamic peak current • `ESR_MIN and ESR_MAX`: Equivalent series resistance For powergates, `-check_parameters` will check the Vds-Ids table for the correct trend. It checks whether: • for a given `Vgs`, `Ids` increases with `Vds` • for a given `Vds`, `Ids` increases with `Vgs` where; `Vgs` is voltage across gate and source `Vds` is voltage across drain and source `Ids` is current from drain to source (of a transistor) This parameter allows you to check the powergate parameters for all the voltages of a multi-voltage characterized powergate PGV.
`-command_write`	
	Specifies to print the settings used during PGV characterization to a text file called `command_list.txt`. This parameter allows you to determine the user-specified or default settings used during PGV generation for the specified power library (.cl).
`-integrity_check`	

	Specifies to load all the views from a power-grid library to ensure that the library contains the required data. This parameter checks if there are any missing files or views, and that the PGV is usable in rail analysis.
`-lef_consistency`	
	Checks for LEF consistency of the cells in the specified cell libraries. It reads the LEF and PGV files, and compares the given LEF file with the LEF file used in PGV generation.
`-libgen_command_file` *`filename`*	
	Specifies to enable the additional LibGen variables `copy_intersecting_obs_vias_to_ports` and `input_type check_lef_pgv_compatibility`, respectively, to verify the consistency between the generated PGV and the original LEF file.

`-list`	Specifies to list the PGVs and the cells in each PGV. The parameter allows you to determine the name of the PGV library that contains a specific cell and determine whether a cell is there in multiple libraries.
	The format of the output file is:
	`PGV Library: <Number of PGV libraries>`
	`<Lib_1> <TOOL VERSION> <PGV VERSION> <GENERATE_DATE> <Path>`
	`<Lib_2> <TOOL VERSION> <PGV VERSION> <GENERATE_DATE> <Path>`
	`...`
	`Duplicated Cells: <Number of cells in multiple libraries>`
	`<CELLNAME> <Lib_1 Lib_2 Lib_3>`
	`Cell List: <Number of cells in a library>`
	`<CELLNAME> <LIB#>`

`-metal_density_map`	

	Generates the metal density map report file (<*pgvname*>.metal_density) in the PGV output directory. This file contains information about the macro size, number of tiles in the x and y direction, metal layer ID, layer name, and metal density values for each tile. This parameter is used with the -calculate_metal_layer_density parameter of the set_pg_library_mode command.
-out_dir *dir_name*	
	Specifies the name of the output directory in which the power-grid library report is created. The default is the current working directory.
-report	Specifies to generate the power-grid library report.
-report_detail_multiple_voltage_cap *file*	
	Specifies to generate a voltage-dependent capacitance table that contains the detailed capacitance values of the given power-grid library. The capacitance table file is called <*pgvname*>.cap_table. This parameter is used with the -enable_multi_voltage_cap_generation parameter of the set_pg_library_mode command.
-rule_file *file*	

	Specifies the rule file name used for parameter checking. This file contains the list of electrical parameters to be checked.
	File Format:
	`CINT_MIN <min cap value>`
	`CINT_MAX <max cap value>`
	`RON_MIN <value>`
	`RON_MAX <value>`
	`IDSAT_MIN <value>`
	`IDSAT_MAX <value>`
	`ILEAK_MIN <value>`
	`ILEAK_MAX <value>`
	`IPEAK_MIN <value>`
	`IPEAK_MAX <value>`
	`IAVG_MIN <value>`
	`IAVG_MAX <value>`
	`ESR_MIN <value>`
	`ESR_MAX <value>`
	This parameter must be specified with the `-check_parameters` parameter.

`-skip_layer_list_file` *filename*	
	Specifies the name of a file containing a list of layers to ignore while checking PGV and LEF compatibility. Its format is one layer per line. This parameter must be specified with the `-check_compatibility` parameter.
`-skip_signal_net_check {true | false}`	
	Default: `false` Specifies to skip LEF consistency checks for signal nets. This forces skipping of the signal pin checks when performing a comprehensive check of power-grid views to be used in rail analysis. The `-skip_signal_net_check` parameter can be used if you want to check LEF consistency of only the power and ground pins.
`-summary`	
	Specifies to generate a detailed report of the library contents for the specified cells that can be used for debugging purpose.

-tap_node_text_write	
	Default: None Specifies to generate a detailed report of the tap current distribution for each interface node. The report name is `tapNodeDetailReport.txt`. It lists both the peak and average current values, capacitance value, and the number of taps for each interface node.
-tech_layers	
	Specifies to report technology layer information and layer mapping.
-total_currents	
	Specifies to report the static current for each power/ground net, and dynamic PWL for each mode. For each mode, it reports the peak and average current, and the number of points for power supply nets. When specified, the total current is reported in `<lib_name>.total_current` for each library.

Examples

- The following command creates a report for the library std_cells.cl:
  ```
  check_pg_library std_cells_fast/std_cells_fast.cl -report
  ```
- The following command checks for LEF consistency:
  ```
  read_lib -lef ddk.0.lef
  read_lib -lef ddk.1.lef
  check_pg_library -lef_consistency \
  "/../library1.cl\
  /../library2.cl"
  ```
- The following command displays the list of layers (defined in the technology file) on stdout:
  ```
  check_pg_library -tech_layers ./check_param/powergate/stdcells.cl
  ```
- The following command checks for compatibility between the specified cell libraries and generates a compatibility report (compatibility.report) in the output directory:
  ```
  check_pg_library -out_dir out -check_compatibility  {/../lib1.cl /../lib2.cl}
  ```

- The following command specifies to list the cells corresponding to the specified libraries, `pgv1.cl` and `pgv2.cl`:

  ```
  check_pg_library -list {/../pgv1.cl  /../pgv2.cl}
  ```

- The following command skips layer checks for the list of layers specified in the `sub_layers.txt` file:

  ```
  check_pg_library -check_compatibility -skip_layer_list_file sub_layers.txt

  ----sub_layers.txt----
  VNW
  VPW
  ```

- The following command checks for electrical parameters of the stdcells.cl library:

  ```
  check_pg_library -out_dir OUT -check_parameters ./stdcells.cl
  ```

  The following is a sample output of this command:

  ```
             PARAM        VALUE            RANGE         PASS/FAIL

  Cell: Cell1
     Net VDDG
          Device Cap    4.50e-14    (1.00e-18,1.00e-11)    PASS

     Net VSS
          Device Cap    2.61e-14    (1.00e-18,1.00e-11)    PASS

     Net VDD
          Device Cap    2.13e-14    (1.00e-18,1.00e-11)    PASS

     Power Gate SLEEP
               Ron      1.84e+01    (2.00e+01,1.00e+03)    FAIL
               Ileak    7.01e-08    (1.00e-15,1.00e-06)    PASS
               Idsat    3.36e-02    (1.00e-09,1.00e-02)    FAIL

     Trend Checks:
         Ron Trend   : PASS
         Idsat Trend  : PASS
         Ileak Trend : PASS
  ```

- The following command reports the static current for each power/ground net:

  ```
  check_pg_library -total_currents /../pg.cl
  ```

  The following is a sample output of this command:

  ```
  Net VSS
  Static current = -0.00021181

  Net VDD
  ```

```
Static current = 0.000211809

MODE1
Trigger Condition: ( !CEN & !WEN )
Net VDD
    Peak = 0.0138308
    Avg = 0.000276053
    Number of points 190

Net VSS
    Peak = -0.0166879
    Avg = -0.000300184
    Number of points 190

MODE2
...
```

- The following command generates a capacitance table (`powergate_stdcells.cap_table`) that contains the detailed capacitance values of the powergate_stdcells.cl library:

  ```
  check_pg_library -report_detail_multiple_voltage_cap ../powergate_stdcells.cl
  ```

  The following is a snippet of the capacitance table file (`powergate_stdcells.cap_table`):

  ```
  ========================================================================
  CellName    NetName      Voltage       Capacitance
  ------------------------------------------------------------------------
  powercell    vccm        0.1000        4.586e-14
  powercell    vccm        0.2825        4.102e-14
  ...
  ```

- The following command specifies to output the settings used during generation of `macros_pll.cl` to a text file called `command_list.txt`:

  ```
  check_pg_library -command_write macro_pgv/macros_pll.cl
  ```

  The following is a sample snippet of the `command_list.txt` file:

  ```
  temperature 25
  spice_subckts /../spice.sp
  spice_models /../spectre_load.sp
  extraction_tech_file /../tech.tch
  gds_files /../file1.gds
  stop@via VIA_1
  spice_subckts_xyscaling 1000
  ```

```
use_powergate_data_from_lib 0
default_frequency 2e+08
decap_frequency 1e+08
```

- The following command generates the power-grid library report:

```
check_pg_library -out_dir report -report /../pg.cl
```

The following is a sample output of this command:

```
================================================================================

Library path name:        /it/../library
Library name:            pg
Hierarchical library:    NO
User login:              user
Date:                    2015-Sep-03 03:58:45 (2015-Sep-03 10:58:45 GMT)
ToolIdentifier:          VOLTUS
Application version:     15.20-d127_1
Number of cells in library:    1

================================================================================
Cell                PowerNets       Capacitance      PowerGrid Views
--------------------------------------------------------------------------------
----------------------------------|

cell1

                    VSS(0.0000)     5.3e-12          EARLY(255 taps) EM(8827 taps)
IR(8827 taps)
                    VDD(0.9000)     6.8e-12          EARLY(193 taps) EM(7534 taps)
IR(7534 taps)

 Cell Type = MACRO                                     Cell_Status: PASS

INFO: Dynamic Tap Current views were created for this cell. MODE1 has Conditional
input = "( !CEN & !WEN ) " and Conditional pin = "CLK RISE".
INFO: Dynamic Tap Current views were created for this cell. MODE2 has Conditional
input = "( CEN & WEN ) " and Conditional pin = "CLK RISE".

--------------------------------------------------------------------------------
----------------------------------|
```

Related Topics

- "Power-Grid Library Generation" in the *Voltus User Guide*

check_pgv

```
check_pgv
[-help]
[-activity value ]
-cell cell_name
[-clock {clk1 clk2 ...}]
[-disable_rail_analysis {true | false}]
[-dynamic_analysis_duration value]
[-enable_xp {true | false}]
[-extraction_include_file filename]
-extraction_tech_file tech_file_path
[-frequency value]
[-input_pin_set_case_analysis_low {pin1 pin2 ...}]
[-input_pin_set_case_analysis_high {pin1 pin2 ...}]
[-liberty liberty_file ]
[-max_ir_drop value]
[-net_capacitance value]
[-output output_dir]
[-pgv_mode_dynamic_switch_pattern pattern]
[-pin_capacitance value]
-power_grid_library_path pgv_path
[-power_grid_library_view view_name]
[-power_pad_size value]
[-rail_analysis_type {static | dynamic} ]
[-rail_include_file_begin filename]
[-rail_include_file_end filename]
[-rail_trigger_switching_pattern pattern]
[-sdc_file filename]
[-set_power_switching_pattern trigger_file]
[-set_total_power value]
[-step_size value]
[-supply_voltage value]
-tech_lef_file tech_lef_file_path
[-temperature value]
[-transition_time time]
[-twf_path twf_location]
[-user_defined_power_pin_location_file filename]
```

Specifies to validate a macro PGV. The command takes a macro PGV and Liberty file as input, and performs IR Drop based validation by running static and dynamic power and rail analysis. In this

flow, Library Generation will read the input PGV and generate a dummy DEF file that has an instance of the macro (one-cell design). It will then run static/dynamic power and rail analysis on this design. You can analyze the power and rail results to check for hotspots, connectivity, and current distribution inside the PGV.

Parameters

`-help`	
	Outputs the command usage.
`-activity` *value*	
	Specifies the activity value for input pins. If not specified, a default activity of 0.2 is used. The activity specifies the number of times an input pin switches in a clock cycle.
`-cell` *cell_name*	
	Specifies the name of the cell to be validated.
`-clock {`*clk1 clk2 ...*`}`	
	Specifies the list of clock pins.
`-disable_rail_analysis {true | false}`	
	Specifies to disable rail analysis during macro PGV validation. The following is the behavior of `-disable_rail_analysis true` with the `-enable_xp` parameter: • if `-enable_xp` is set to `true`, the tool will run only power analysis in the XP mode • if `-enable_xp` is set to `false`, the tool will enable only power analysis in the non-XP or normal mode *Default*: `false`
`-dynamic_analysis_duration` *value*	
	Specifies the simulation period that dynamic power analysis will use. It is the duration for which a window is selected for calculation of IR drop during power analysis. The default value is 10ns.
`-enable_xp {true | false}`	

Specifies to validate the macro PGV in the extensively parallel (XP) mode. *Default*: `false`
`-extraction_include_file` *filename*
Specifies the extractor (ZX) commands and variables in the form of an include file. This parameter is applicable only for the standalone report generation flow in the XP mode (`check_pgv -enable_xp true`).
`-extraction_tech_file` *tech_file_path*
Specifies the path of the extraction technology file.
`-frequency` *value*
Specifies the clock frequency. The default value is 100Mhz.
`-input_pin_set_case_analysis_low {`*pin1 pin2 ...*`}`
Specifies the list of pins to be kept at a logical low. The `set_case_analysis` command in the SDC file will have the same effect as the set pin to constant value command described here.
`-input_pin_set_case_analysis_high {`*pin1 pin2 ...*`}`
Specifies the list of pins to be kept at a logical high. The `set_case_analysis` command in the SDC file will have the same effect as the set pin to constant value command described here.
`-liberty` *liberty_file*
Specifies the name of the Liberty file for power calculation. This parameter is optional for the static power analysis flow, and is mandatory for the dynamic power analysis flow. In the static flow, you need to specify either `-liberty` or `-set_total_power`. If both the parameters are specified, the `-set_total_power` parameter will be honored.
`-max_ir_drop` *value*
Specifies the maximum IR drop allowed. Default is 5 (in percentage).
`-net_capacitance` *value*

Specifies the value of the net capacitance. The default value is 1pF.
`-output` *`output_dir`*
Specifies the name of the output directory. This directory saves the power and rail reports, and the design output (such as .lef, .def, and so on).
`-pgv_mode_dynamic_switch_pattern` *`pattern`*
Specifies the switching patterns for the functional modes in the dynamic power calculation flow. The use model of this parameter is same as the `set_power -dynamic_switch_pattern` parameter. In the following example, the mode `write` will switch in the 1st clock cycle, the mode `read` will switch in the 2nd and 4th clock cycles, and none of the modes will switch in the 3rd clock cycle: `check_pgv -pgv_mode_dynamic_switch_pattern { write read - read }`
`-pin_capacitance` *`value`*
Specifies the value of the pin capacitance. The default value is 1pF.
`-power_grid_library_path` *`pgv_path`*
Specifies the path of the power-grid library file.
`-power_grid_library_view` *`view_name`*
Specifies the power-grid view name.
`-power_include_file` *`filename`*
Specifies a power analysis include file. This file specifies certain commands of the dynamic engine that do not have a Voltus equivalent. This parameter is applicable only to the dynamic power analysis flow.
`-power_pad_size` *`value`*
Specifies the distance between two adjacent DEF voltage sources. The default value is `50` (in Micron).
`-rail_analysis_type {static | dynamic}`
Specifies whether static or dynamic analysis will be done. The default value is `static`.

`-rail_include_file_begin` *`filename`*
Provides a file that includes additional rail options that will be run prior to the `analyze` command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.
`-rail_include_file_end` *`filename`*
Provides a file that includes additional rail options that will be run after the `analyze` command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.
`-rail_trigger_switching_pattern` *`pattern`*
Default: None Specifies a dynamic trigger file. When a Macro EM view with current signature is used in a design, it needs to be determined when this block/cell is triggered. The dynamic trigger file is used to determine this. The use model of this parameter is same as the `set_rail_analysis_config -dynamic_trigger_file` parameter.
`-sdc_file` *`filename`*
Specifies the name of the timing constraints file.
`-set_power_switching_pattern` *`trigger_file`*
Default: None Specifies the stimulus pattern on the pins of an instance and cell. Specify this parameter only in the dynamic mode for the type instance and cell. The use model of this parameter is same as the `set_power -dynamic_switch_pattern` parameter.
`-set_total_power` *`value`*
Specifies the total power for the design in Watts. This power is distributed based on cell area.
`-step_size` *`value`*
Specifies the simulation step size, in picoseconds. The default value is 20ps. This parameter is applicable only to the dynamic power analysis flow.
`-supply_voltage` *`value`*

Specifies the voltage of each power supply pin. The default value is 1V.
`-tech_lef_file` *`tech_lef_file_path`*
Specifies the path of the technology LEF file.
`-temperature` *`value`*
Sets the temperature for static power calculation. The default is 125 degrees.
`-transition_time` *`time`*
Specifies the transition time for specific pins or instances in the design. This parameter is applicable only to the dynamic power analysis flow.
`-twf_path` *`twf_location`*
Specifies the location of the Timing Window File (TWF) for noise and power calculation.
`-user_defined_power_pin_location_file` *`filename`*

Specifies the name of a file containing net names and corresponding user-defined ploc or power pad files (.pploc). The .pploc file includes power pin or voltage source locations for power and ground nets. The parameter gives you the flexibility to include either user-specified or tool-generated voltage sources.
 	
The following is a sample file with the list of nets and power pad files:
`----ploc.list----`
`#<NET_NAME> <Pointer to ploc file>`
 	
`VDD input/ploc/VDD.pploc`
`VDDA input/ploc/VDDA.pploc`
`VDDF input/ploc/VDDF.pploc`
`VSS input/ploc/VSS.pploc`
 	
When specified, the voltage source location files (.pploc) in the given *`ploc.list`* file will be used for library generation. If the pploc file is missing for a few nets in the *`ploc.list`* file, then library generation will automatically generate `.pploc` for those nets.

Examples

- The following command specifies to validate the cell `cell1` by running static power and rail analysis:

```
check_pgv -cell cell1 -liberty 125c.lib -clock {CLKW CLKR} -
power_grid_library_path macros_cell.cl -extraction_tech_file caps.tch -
tech_lef_file tech.lef -output validate_pg_output_static -rail_analysis_type
static
```

- The following command specifies to validate the cell `cell1` by running dynamic power and rail analysis

```
check_pgv -cell cell1 -liberty 125c.lib -clock {CLKW CLKR} -
power_grid_library_path macros_cell.cl -temperature 122 -extraction_tech_file
caps.tch -tech_lef_file tech.lef -output validate_pg_output_dynamic -
rail_analysis_type dynamic
```

merge_pg_library

```
merge_pg_library
[-help]
[-force_merge {true | false}]
-library_list_file file_name
[-library_prefix prefix]
[-merge_ddv_currents {true |false}]
[-merge_single_voltage_pgvs {true |false}]
[-out_dir directory_name]
[-remove_tech {true | false}]
[-validate_rail_merge {true | false}]
```

Specifies to merge power-grid libraries. Using this command, you can merge multiple same-type power-grid libraries into one consolidated power-grid library.

Parameters

`-help`	Outputs the command usage.
`-force_merge {true | false}`	
	Specifies to force merge PGVs with different resistivity and layers. It uses the technology information of the first library. If multiple tech-only PGVs are specified, it will take the first tech-only PGV definition and the first cell definition. If you are merging a tech-only PGV with other PGVs, it will keep the tech-only technology definition and the first cell definition. The default value of the parameter is `false`.
`-library_list_file file_name`	
	Specifies the file containing the list of libraries.
`-library_prefix prefix`	
	Specifies a prefix for the generated power-grid library.

`-merge_ddv_currents {true |false}`	
	Specifies to merge the tap currents from multiple DDV PGVs having different voltages. The parameter can be used during rail analysis to consider multiple single-voltage DDVs for a particular cell like a multi-voltage PGV for that cell. The default value of the parameter is `false`. **Usage Guidelines**: This parameter is a mandatory input for macro PGV generation (`-celltype macros`).
`-merge_single_voltage_pgvs {true |false}`	
	Specifies to merge all the single voltage PGVs into a multi-voltage PGV for better handling of the library characterization flow. This parameter is used in a scenario when you have generated multiple PGVs for different voltages, instead of generating a single multi-voltage PGV that has voltage-dependent capacitance. The default value of the parameter is `false`.
`-out_dir` *`directory_name`*	
	Specifies the name of the output directory in which power-grid libraries are created. The default is the current working directory. **Note**: If you specify both `-out_dir` *`output_dir`* and `-library_prefix` *`prefix`* parameters, the command generates the PGV in the *output_dir* directory with PGV name `prefix.cl`.
`-remove_tech {true | false}`	
	Specifies to remove technology dependency from the specified or merged library. The newly created library will not contain the technology information. The default value of the parameter is `false`.
`-validate_rail_merge {true | false}`	

	Specifies to validate the merged PGV in the same way as done when merging PGV during rail analysis. When this parameter is specified, the log file reports error, warning, and info messages if the tool finds any issues during PGV merging. The parameter allows you to check the PGV merging issues as reported during rail analysis, without having to run the full design. `-validate_rail_merge` is used only for validation purpose, and the merged PGV is not to be used for analysis.

Examples

- The following command merges the power-grid libraries specified in the `merge_lib.list` file into one consolidated power-grid library:

  ```
  merge_pg_library -library_list_file "merge_lib.list" -out_dir ./ -library_prefix "new_merged"
  ```

  The content of the `merge_lib.list` file is:

  ```
  library1.cl
  library2.cl
  library3.cl
  ```

- The following command merges the power-grid libraries specified in the `lib.txt` file into one consolidated power-grid library and validates the merged library:

  ```
  merge_pg_library -library_list_file lib.txt -force_merge true -validate_rail_merge true
  ```

  The content of the `lib.txt` file is:

  ```
  fast_allcells.cl
  pv_library.cl
  ```

Related Topics

- "Power-Grid Library Generation" in the *Voltus User Guide*

rename_pg_library

```
rename_pg_library
[-help]
[-cell_list_map_file filename]
[-merge_library_list_file filename]
[-out_dir directory_name]
[-output_library new_library_name]
```

Specifies to copy and rename the specified power-grid view library. The command also allows you to rename the cells of the specified input library.

Parameters

`-help`	Outputs the command usage.
`-cell_list_map_file` *filename*	
	Specifies a file containing the list of cells to be renamed. The format of the specified file is: *<old cell name1> <new cell name1>* *<old cell name2> <new cell name2>* ...
`-merge_library_list_file` *filename*	
	Specifies a file containing the input library to be renamed. The format of the specified file is: *<path to the input library>*
`-out_dir` *directory_name*	
	Specifies the name of the output directory in which the new power-grid library is created. The default is the current working directory.
`-output_library` *new_library_name*	

	Specifies the new name of the specified input library.

Examples

- The following command specifies to rename the input library to `new_lib`:

```
rename_pg_library -merge_library_list_file new.list -cell_list_map_file cell.list
-output_library new_lib -out_dir .
```

Related Topics

- "Power-Grid Library Generation" in the *Voltus User Guide*

set_advanced_pg_library_mode

```
set_advanced_pg_library_mode
[-help]
[-add_port_labels file_name]
[-assume_foreign {true | false}]
[-assume_foreign_mode [1 | 0]]
[-cell_electrical_data_file file]
[-cell_per_distributed_host value]
[-cell_pin_net_map_file file]
[-circuit_include_file thunder.inc]
[-cluster_via_rule {{via_layer1 number_of_equidistant_vias}...}]
[-cluster_via_sizevalue]
[-common_supply_pins {net_name+}]
[-create_static_view_from_dynamic_view {true | false}]
[-damping_decap_cells {cell1cell2 ..}]
[-damping_decap_frequency value]
[-decap_frequency value ]
[-default_frequency value]
[-default_power_voltage value]
[-delete_generated_fsdb_files {true|false} ]
[-disable_power_switch_ramp_up_simulation{true | false}]
[-distribute_current_to_switch_net {true | false}]
[-enable_ac_simulation_for_IO_characterization{true | false}]
[-enable_spectre_netlist_flow  {true | false}]
[-enable_subconductor_layers{true | false}]
[-enable_via_based_current_distribution {true | false}]
[-esd_cells {cell_list}]
[-esd_cells_list_filefilename]
[-esd_device_list{device1device2 ....deviceN}]
[-esd_parameters_file filename]
[-esd_pin_list{pin1pin2 ....pinN}]
[-exclude_tap_region_file filename]
[-extraction_command_file file]
[-followpins_interface_node_layer {lowest_lef_pin_layer | highest_lef_pin_layer |
all_lef_pin_layers}]
[-followpins_tap_layer {lowest_lef_pin_layer | highest_lef_pin_layer |
all_lef_pin_layers}]
[-force_tap_node_distance {true | false}]
[-generate_bulk_pin_cap_separately {true | false}]
[-generate_gray_box_data{true | false} ]
```

```
[-generate_interface_nodes_at_via_layer{true | false}]
[-generate_interface_node_x_direction {true | false}]
[-generate_interface_node_y_direction {true | false}]
[-generate_pin_intrinsic_cap {true | false}]
[-generate_pin_intrinsic_cap_list {netname1 netname2...}]
[-ignore_pg_nets {{cellnamenetname}+}]
[-input_port_value_list {pin1value1pin2value2 ....pinNvalueN}]
[-interface_node_location_file filename]
[-lef_layer_ignore_list {lef_layername+}]
[-lef_layer_ignore_list_file filename]
[-lef_pin_short {true | false}]
[-lef_pin_short_cell_file filename]
[-lef_pin_short_cell_list {cell1 cell2 ...}]
[-macro_parasitic_file filename]
[-marker_layer_mapfilename]
[-on_resistance_measure_threshold value]
[-pg_arc_file filename]
[-pg_library_generation_command_file file]
[-pgv_path path]
[-power_switch_characterization_voltages { val1val2val3 .... }]
[-power_switch_multi_enable_characterization_method {enable_all | enable_single} ]
[-process_bulk_diffusion_ports {true|false}]
[-quit_on_extraction_errors{true|false}]
[-redo_characterization_count count]
[-remove_em_view_dangling_resistor{true|false}]
[-retain_generated_pgv_on_error{true | false}]
[-reuse_electrical_data {true | false}]
[-schematic {true|false} ]
[-skip_switch_net_extraction {true | false}]
[-source_location_file {filename}]
[-spectre_path value]
[-stdcell_characterization_voltage_value {val1 val2val3 ...}]
[-strict_input_check {true|false}]
[-tap_node_distance value]
[-techgen_dir directory]
[-trim_metal_layer_map file]
[-thunder_command_file file]
[-use_embedded_spectre {true | false}]
[-use_physical_view_from_techpgv {true | false}]
[-use_peak_current_distribution {true | false}]
[-verbosity {true | false}]
[-via_based_current_config_file filename]
[-well_capacitance_file filename]
```

`[-xtc_command_file` *`filename`*`]`
`[-xtc_include_file` *`filename`*`]`

Specifies the advanced power-grid library generation features.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_advanced_pg_library_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_advanced_pg_library_mode`
`-add_port_labels` *`file_name`*	
	Specifies the name of a file containing details about the label of a switched rail. Using this parameter, you can add a label for the switched rail manually in the specified port file, if the switched rail is not labeled. The format of the port file is given below: MACRO *macro_name* PIN *pin_name*
`-assume_foreign {true | false}`	
	Enables the Library Characterization engine to process LEF files in which FOREIGN statements are not defined as part of the macro definition. When generating detailed power-grid views, the engine assumes that FOREIGN statements are defined as part of the macro definition in the LEF file. When this option is set to `true`, the engine processes the LEF file by using the name defined by the MACRO statement rather than requiring a FOREIGN statement. The `-assume_foreign` parameter ensures that the FOREIGN statement is placed at 0,0. Therefore, if the MACRO statement has an offset, you must provide a FOREIGN statement that contains an offset. This option works for offsets that are 0,0 or the inverse of the origin. *Default*: false

`-assume_foreign_mode [1 | 0]`	
	Determines how the Library Characterization engine processes cells for which FOREIGN statements are assumed. If set to 0, the engine assumes that the cell for which FOREIGN statements are assumed is placed at (0,0). When you set it to 1, the engine assumes that the FOREIGN cell is placed at the inverse of the macro origin. The default is 1. *Default*: 1
`-cell_electrical_data_file` *file*	
	Specifies the name of a file containing data about the electrical characteristics of cells. Voltus library simulator requires this data to generate currents during power-grid view generation.
`-cell_per_distributed_host` *value*	
	Specifies the number of cells to be submitted per host for distributed PGV generation. The default value is 50.
`-cell_pin_net_map_file` *file*	
	Specifies the mapping between the cell LEF pin names and the internal names of the net to which those pins are attached. It is used when multiple pins are attached to a single net.
`-circuit_include_file` *simulator.inc*	
	Specifies the name of the input file for Voltus library simulator in the Library Characterization command file.
`-cluster_via_rule {{`*via_layer1 number_of_equidistant_vias*`}...}`	

	Controls the number of vias to cluster on a layer basis. The VIA clustering rule specified using this parameter will override the default clustering rule for PGV generation. You can use this parameter when you need clustering at VIA1, or different clustering for different layers. The default cluster size for PGV generation is `64` for all via layers except VIA1. **Example**: The following command will cluster 100 equidistant VIA1 cuts, 200 equidistant VIA2 cuts, and 300 equidistant VIA7 cuts: `set_advanced_pg_library_mode -cluster_via_rule {{VIA1 100} {VIA2 200} {VIA7 300}}`
`-cluster_via_size` *value*	
	Specifies that power-grid extraction for library cell needs to perform 64 (8x8) via clustering inside PGV for all via layers above VIA1 between Metal1 and Metal2. The default value of this parameter is `64`. The following command specifies to perform 16 (4x4) via clustering: `set_advanced_pg_library_mode -cluster_via_size 16`
`-common_supply_pins{`*net_name+*`}`	
	Identifies the generic supply net or nets which could act as either a power or ground supply net. No wildcards are allowed for the specification of net names. Generic supply nets are used for pads or bump cells that could be connected to either a power or ground net. They will be connected to the correct power-grid based on the DEF or GDS file.
`-create_static_view_from_dynamic_view {true | false}`	
	Specifies to generate static power-grid view by averaging of dynamic currents in the multi-mode characterization flow. When this parameter is specified, the software performs averaging of the dynamic currents of all the modes and saves it as static current in the power-grid view. *Default*: `false`

`-damping_decap_cells {`*`cell1cell2`* `..}`	
	Specifies a list of damping decap cells.
`-damping_decap_frequency` *`value`*	
	Specifies the frequency of operation of the damping decap cells. The default value is 200MHz.
`-decap_frequency` *`value`*	
	Specifies the decoupling capacitance frequency (in hertz) used during PGV generation. Intrinsic capacitance of cells has a significant dependency on frequency used during PGV generation. PGVs characterized with higher frequency may lead to pessimism during dynamic analysis due to less impact of intrinsic capacitance of the cells characterized with higher frequency. Using this parameter, you can specify the dominant frequency to be used for PGV generation. The default value for decoupling capacitance frequency is 1GHz. For example, the following command specifies that the decap frequency is 200 MHz: `set_advanced_pg_library_mode -decap_frequency 2e+8`
`-default_frequency` *`value`*	
	Specifies the default frequency of signal nets in hertz (Hz); this is used to calculate the current and capacitance values during the creation of detailed power-grid views. For example, the following command specifies that the default switching frequency is 100 MHz: `set_advanced_pg_library_mode -default_frequency 100e+06`
`-default_power_voltage` *`value`*	
	Specifies the default voltage for power nets. This parameter assigns a default value to power nets for which voltage has not been defined using the `-power_pins` parameter.

`-delete_generated_fsdb_files {true|false}`	
	Specifies to delete the intermediate FSDB files generated in the Spice deck (user-specified) method of current characterization. This parameter allows you to retain or delete the huge FSDB files.
`-disable_power_switch_ramp_up_simulation {true | false}`	
	Specifies whether to run power gate ramp-up simulation during power-grid view generation. When this parameter is set to `true`, the Ids-Vds table will not be present in PGV, therefore, this PGV cannot be used for the power-up or rush current analysis flow. This parameter can be specified to reduce the run time when you require only the RON/Idsat/Ileak and Cap values in PGV. `-disable_power_switch_ramp_up_simulation` is used when power gate parameters are specified using the `set_pg_library_mode -power_switch_parameters_file` parameter. *Default*: `false`
`-distribute_current_to_switch_net {true | false}`	
	Distributes the PWL waveform, which is specified to the always-on power net, to the taps of both the always-on power net and the related switched power nets in the fine-grain memory characterization flow. By default, the current waveform is distributed only to the taps of the always-on power net when you specify the PWL to the always-on power net. *Default*: `false`
`-enable_ac_simulation_for_IO_characterization {true | false}`	

	Specifies to perform an AC analysis based coupled capacitance calculation for IO cells. When set to `false`, the software performs DC analysis based coupled capacitance calculation for IO cells. The default is `false`. The AC analysis based coupled capacitance calculation is slower than the DC analysis based coupled capacitance calculation.
`-enable_spectre_netlist_flow {true | false}`	
	Specifies to use the Spectre simulator to read the Spice/Spectre netlist for the standard cells characterization flow. This option should be used when the netlist is in Spectre format, or when the netlist is encrypted by Spectre. *Default*: `false`
`-enable_subconductor_layers {true | false}`	
	Allows to control the generation of the sub-conductor layers in the PGV. When this parameter is set to `false`, the sub-conductor layers are not generated in the PGV. The default value of this parameter is `true`.
`-enable_via_based_current_distribution {true | false}`	
	Default: `false` Specifies to enable location-based via current modelling during GDS-based macro power-grid view generation. When set to `true`, defines the current values only on the user-defined via location. This parameter allows you to define average and peak current values for specific-location vias. The `-enable_via_based_current_distribution` parameter must be specified with the `-via_based_current_config_file` parameter. If the parameter is set to `false`, current is uniformly distributed across all vias in the macro PGV.
`-esd_cells {`*`cell_list`*`}`	

	Specifies a list of Electrostatic Discharge (ESD) cells. You can use this parameter to characterize ESD cells. When you specify this parameter, the list of ESD cells are later used in the `check_esd` command while loading the power-grid view library. As a result, you do not need to specify the ESD cell names with the `-esd_cell_list` parameter of the `check_esd` command.
`-esd_cells_list_file` *`filename`*	

	Specifies a file containing a list of ESD cells that are later used in the `analyze_esd` command while loading the power-grid view library. As a result, you do not need to specify the ESD cell names with the `-esd_cell_list` parameter of the `analyze_esd` command.
	
	The `-esd_cells_list_file` parameter is the same as the existing `-esd_cells` parameter of the `set_advanced_pg_library_mode` command, except that the `-esd_cells_list_file` parameter allows you to specify a list of ESD cells in a file while the `-esd_cells` parameter allows you to specify the ESD cells in the command line itself.
	
	The following is an example of this parameter:
	`set_advanced_pg_library_mode -esd_cells_list_file {./Esd_Cell_list.txt}`
	
	The following is the ESD cell file format:
	
	`CELL_1 <type>`
	`CELL_2`
	`...`
	`CELL_n`
	
	Here,
	
	• the first column specifies the ESD cell name
	• the second column specifies the ESD cell type, such as DIODE and CLAMP. This is optional.
	
	The following is a snippet of the input file:
	`HEADER_SWITCH CLAMP`

`-esd_device_list {device1 device2 ....deviceN}`

Specifies the list of ESD devices. The ESD device information is used for accurate ESD analysis as it takes into account the internal grid from the ports to the ESD devices. The followings inputs are required for ESD tagging during macro PGV generation:

- an xDSPF file
- ESD device list

`-esd_parameters_file filename`

Specifies resistance information for the following types of ESD cells during ESD cell characterization:

- Diodes
- RC Clamp

When this parameter is specified, the software uses these electrical parameters from the generated PGV for ESD analysis.

The format of the input file is:

```
Cell <cell name>  <Ron> <Roff> <power_pin> <ground_pin>
<voltage>
Type <esd cell type> <Ron> <Roff> <power_pin>
<ground_pin> <voltage>
```

Here,

- `Cell` and `Type` are keywords
- `<cell name>` specifies the ESD cell name
- `<esd cell type>` specifies the ESD cell type, such as DIODE and CLAMP
- `RON` is the resistance of the complete cell when it is switched on.
- `ROFF` is the resistance of the complete cell when it is switched off.
- `power_pin` is the name of the power pin connected to the power net

	• *ground_pin* is the name of the ground pin connected to the ground net • *voltage* is the threshold value between the power and ground pin. • When you specify the cell name, the RON and ROFF would be applied only for that cell, and when you specify the cell type, the RON and ROFF would be applied for all the ESD cells of the particular type. The following is a snippet of the input file: `Cell HEADER_SWITCH 10 5 pwr_pin gnd_pin 0.4`
`-esd_pin_list {`*pin1 pin2 ....pinN*`}`	
	Specifies the list of ESD pins. This parameter tags the user-specified pin as the ESD pin before running the `report_esd` command, and ensures that the software traces only this ESD power pin to bump and extracts resistance. The ESD cell pins that are tagged using this parameter are listed in the `.report` file.
`-exclude_tap_region_file` *filename*	

	Specifies to exclude certain tap points during macro PGV generation. This parameter helps to avoid false IR-drop violations inside macros caused by inadvertently assigning evenly-distributed tap current to certain undesired tie high/low paths or shielding paths at certain via layers. The Spice netlist and models are not mandatory parameters for excluding the tap points in the macro PGV generation flow.
	The format of the region file is:
	`REGION X1 Y1 X2 Y2 <netname1>`
	`REGION X3 Y3 X4 Y4 <netname2>`
	where, `REGION` keyword specifies that the coordinates following it is a tap current region. The unit for coordinates is micron. <*netname*> is optional, and allows you to specify the exclude region for each net. If you do not specify <*netname*>, the tap points in the specified rectangular region are excluded for all power and ground nets.

`-extraction_command_file` *file*	
	Specifies the file name of an extraction command file.
`-followpins_tap_layer {lowest_lef_pin_layer | highest_lef_pin_layer | all_lef_pin_layers}`	

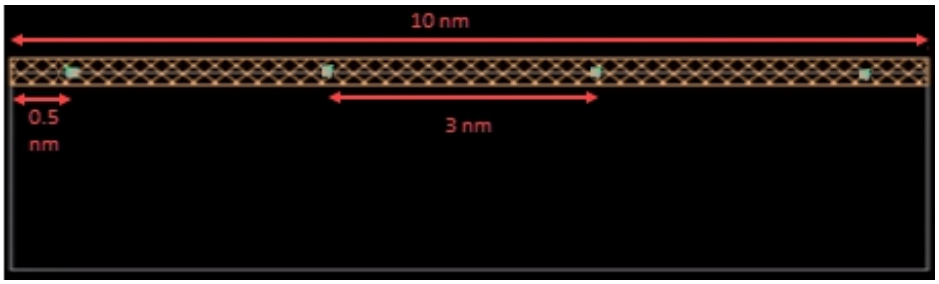
`-followpins_interface_node_layer {lowest_lef_pin_layer | highest_lef_pin_layer | all_lef_pin_layers}`	Specifies the LEF metal layer used to generate current taps. This parameter has two arguments: • `lowest_lef_pin_layer`: The tool generates current taps only on the lowest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool uses METAL1 layer as the tap layer. • `highest_lef_pin_layer`: The tool generates current taps only on the highest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool uses METAL2 layer as the tap layer. • `all_lef_pin_layers`: The tool generates current taps on all LEF metal layers. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool uses both METAL1 and METAL2 layers as the tap layer. This is the default value.

	Specifies the LEF metal layer used to generate interface nodes. This parameter has three arguments: • `lowest_lef_pin_layer`: The tool generates interface nodes only on the lowest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool will only generate the interface nodes on the METAL1 layer. • `highest_lef_pin_layer`: The tool generates interface nodes only on the highest LEF metal layer. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool will only generate the interface nodes on the METAL2 layer. • `all_lef_pin_layers`: The tool generates interface nodes on all LEF metal layers. For example, if both METAL1 and METAL2 metal layers exist for the power/ground pin in the cell LEF, the tool generates the interface nodes on both METAL1 and METAL2 layers. This is the default value.
`-force_tap_node_distance {true | false}`	

Specifies to create the interface nodes exactly at the user-specified distance. The default value of the parameter is `false`. The `-tap_node_distance` option of the `set_advanced_pg_library_mode` command uniformly places the interface nodes so the actual distance between the nodes might be less than the user-specified distance. Therefore, you can set the `-force_tap_node_distance` option to `true` to have maximum specified distance between the nodes.

Example:

For a LEF segment of length `10 nm`, if the user-specified distance between the nodes is `3 nm` and the `-force_tap_node_distance` option is set to `true` then there is maximum distance of `3 nm` between the nodes and `0.5 nm` is left on each side.



`-generate_bulk_pin_cap_separately {true | false}`

	Specifies to generate separate capacitance values for body bias pins in the library report. This parameter allows you to check the bulk pin capacitance based on the Spice simulation. The default value of this parameter is `false`. When set to `false`, the software would sum up the capacitance values for the body bias pins and power/ground pins.
	
	The following is a snippet of the library report:
	
	`cell_A`
	`VPP(0.9350) 2.3e-16 EARLY(1 taps) EM(1 taps) IR(1 taps)`
	`VBB(0.0000) 2.3e-16 EARLY(1 taps) EM(1 taps) IR(1 taps)`
	`VDD(0.9350) 8.1e-15 EARLY(1 taps) EM(1 taps) IR(1 taps)`
	`VSS(0.0000) 8.1e-15 EARLY(1 taps) EM(1 taps) IR(1 taps)`
	`Cell Type = STDCELL Cell_Status: PASS`
	
	Here, the blue-highlighted text shows the capacitance values for the `VPP` and `VBB` body bias pins.

`-generate_gray_box_data {true | false}`	
	Controls the generation of gray box data and storage of this data in the cell library. It is recommended to set this parameter to `false` to reduce the size of the PGV significantly (from 10MB to 1MB) without loss of accuracy. The default value of this parameter is `true`. This parameter is applicable only for macro PGV generation (`-cell_type macros`).
`-generate_interface_node_x_direction {true | false}`	
	When the LEF port geometry is bigger than the tap node distance (`set_advanced_pg_library_mode -tap_node_distance`), multiple interface nodes are generated. You can specify this parameter to generate the interface nodes along the X direction. The default value is `true`. When set to `false`, interface nodes are not generated along the specified direction.

`-generate_interface_node_y_direction {true | false}`
When the LEF port geometry is bigger than the tap node distance (`set_advanced_pg_library_mode -tap_node_distance`), multiple interface nodes are generated. You can specify this parameter to generate the interface nodes along the Y direction. The default value is `true`. When set to `false`, interface nodes are not generated along the specified direction.
`-generate_interface_nodes_at_via_layer {true | false}`
Specifies to generate interface nodes at via locations in scenarios where the tap current source is a VIA layer.
`-generate_pin_intrinsic_cap {true | false}`
Default: `false` Specifies to apply the intrinsic capacitance generated during library generation to all the nets in the datasheet-based current characterization flow. This parameter will override the value of capacitance specified using `DATASHEET_PARAMETER` in the trigger file.
`-generate_pin_intrinsic_cap_list {netname1 netname2...}`
Specifies the list of nets for which you want to apply the generated intrinsic capacitance in the datasheet-based current characterization flow. This parameter will override the value of capacitance specified using `DATASHEET_PARAMETER` in the trigger file. **Example**: `set_advanced_pg_library_mode -generate_pin_intrinsic_cap_list {VDDA VDD1 ...}`
`-ignore_pg_nets {{cellname netname}+}`
Skips the specified power or ground nets in PGV generation.
`-input_port_value_list {pin1 value1 pin2 value2 ....pinN valueN}`

Specifies the voltage conditions of the signal inputs during power-grid library characterization. A typical usage model of this parameter include setting the SET or RESET signal port to be the same voltage as supply or ground (tie high/low) to obtain the correct model of operation.

The following example illustrates the use model:

```
set_advanced_pg_library_mode -input_port_value_list {
RESETVDD 3.3 }
```

`-interface_node_location_file` *filename*

Specifies a file containing the location of interface nodes where tap currents are to be generated. The specified file includes the cell name, net name, layer name, and location of the nodes. This parameter allows to generate current taps at user-specified locations during PGV generation.

The format of the file is:

```
CELL <cell_name>
NET <net_name>
LAYER <layer_name> (x1,y1),(x2,y2)
```

Example:

```
CELL cellA
NET VDD
LAYER M0 (0,0.15),(0.585,0.177),(0.2,0.15),(0.3,0.177)

NET VSS
LAYER M0 (0.485,0.019)

CELL cellB
NET VDD
LAYER M0 (0.49,0.008),(0.24,0.008)
```

This parameter is applicable only for technology and standard cell PGV generation (`-cell_type techonly | stdcells`).

`-lef_layer_ignore_list {`*lef_layername+*`}`

Specifies the names of the LEF layers that power-grid library generation is to ignore when it creates the cell library.

`-lef_layer_ignore_list_file` *filename*

	Specifies the name of a file containing a list of the LEF layers that power-grid library generation is to ignore when it creates the cell library. Each line in the file must consist of a single LEF layer name. The format of the file is: *layer1* *layer2* *...* *layern*
`-lef_pin_short {true | false}`	
	Enables shorting of the unconnected LEF pins and ports to establish electrical connectivity. When set to `true`, shorts the interface nodes for all the cells that are being processed. The default value is `false`.
`-lef_pin_short_cell_file` *filename*	

	Specifies the name of a file containing a list of cells and the net name, metal layer name, and resistance value for each layer. It is used to short the LEF pins and ports by the user-specified resistance value. When this parameter is specified, only the given list of nets are shorted with the user-defined resistance values. If a resistance value is not specified for a layer, the default resistance value of 0.001 is used.
	The format of the file is:
	`CELL <cell_name>`
	`<net_name> <layer_name> <resistance_value>`
	Example:
	`CELL bump_70N_M4`
	`VDD Metal4 12`
	`CELL bump_70N_M5`
	`VDD Metal4 15`
	`VDD Metal5 12`

`-lef_pin_short_cell_list {`*cell1* `cell2 ...}`	

	Specifies a list of cells for which the the LEF pins and ports are shorted by the default resistance value of 0.001ohm. When this parameter is specified, all the nets of the given cells are shorted using the resistance value of 0.001ohm.
`-on_resistance_measure_threshold` *value*	
	Specifies the switched pin voltage during Ron simulation. The specified value will be multiplied by the VDD voltage (<value>*vdd_vol) and applied to the switched pin during Ron simulation. The default value is `0.99`. You can specify a value between 0 and 1.
`-pg_library_generation_command_file` *file*	
	Specifies the file name of a power-grid library generation command file.
`-macro_parasitic_file` *filename*	
	Specifies the name of the SPEF file that provides signal net parasitics while creating a detailed dynamic view in Library Characterization. The SPEF should be extracted in the decouple_rc mode. **Note:** A SPEF file is required only when you specify your own netlist in the Netlist-based flow. The SPEF file is a requirement when the netlist does not have signal caps. You should not specify the signal parasitics using the -ultasim_include_file option.
`-marker_layer_map` *filename*	

Specifies the GDS layer mapping file containing the list of marker GDS layers that are used for layer operations during XTC processing. Marker layers are layers which do not have corresponding tech layers.

This parameter is used to support extraction of Metal-Insulator-Metal Capacitor (MIMCAP) geometries during power-grid library characterization. The following is the format of the mapping file:

```
Layer Type    marker layername    GDS keyword
gds layer number
metal           CTM_O              gds             77 1
metal           CBM_O              gds             88 1
```

`-pg_arc_file` *filename*

Specifies the name of the user-defined PG arc file that contains the power-ground data to generate the power-ground correspondence view in PGV. This correspondence view is reported in the `PGV.summary` report.

File Format:

```
CELL cellname
{Power_pin_name1} {gnd_pin_name1}
{Power_pin_name2} {gnd_pin_name2}
…
CELL cellname2
```

Example:
The following is an example of the power-ground correspondence view that gets generated from the user-specified PG arc file:

User-specified PG arc file:

```
CELL          cell1


{VDDA}        {VSSA}
{VDDA1 VDD}   {VSS}
```

Power-ground correspondence view:

```
Pin Name      Correspondence Name


VDDA          VSSA
VDDA1         VSS
VDD           VSS
VSSA          VDDA
VSS           VDDA1
```

`-pgv_path` *path*

Specifies the path of the cell library from which the electrical data will be used to create a new PGV.

`-power_switch_characterization_voltages { val1 val2 val3 .... }`

	Supports characterization of power gates at multiple voltages. You can specify multiple values of voltages so that the library characterization engine will characterize the power gates at each of the voltages. The software will then be able to get the characterized cell from the cell library for a particular voltage.
`-power_switch_multi_enable_characterization_method {enable_all | enable_single}`	

	Specifies the Ron/Idsat characterization method for power gate cells with multi-enable pins. This parameter allows you to control the generation of RON and Id-Vd curve for each enable pin. The possible characterization methods are:
	• `enable_all`: All the enable pins are switched ON for Ron/Idsat characterization. The generated PGV model will contain a single Ron resistor with the total equivalent resistance value. The cell library report displays the equivalent Ron value of all the enable pins.
	• `enable_single`: Only one enable pin is switched ON at a time for Ron/Idsat characterization, and the other pins are switched OFF. The generated PGV model will contain total number of Ron resistors that is equal to the number of enable pins. The cell library report displays Ron values separately for each enable pin.
	This parameter is supported only for standard cells when the power gate parameters are specified using the `-power_switch_parameters_file` parameter. For multi-enable power gate cells, Ileak/Roff are always calculated by keeping all the enable pins OFF.
	Default: `enable_all`
	Example:
	`set_pg_library_mode ... -power_switch_parameters_file ./pg1.txt`
	`set_advanced_pg_library_mode -power_switch_multi_enable_characterization_method enable_all`

`-process_bulk_diffusion_ports {true|false}`	

	Specifies to create the `DIFF` layer ports of body bias pins in PGV. By default, library generation ignores ports on the diffusion layer. In order to process body bias pin's ports on the diffusion layer for electrical analysis on body bias network, set this parameter to `true` during library generation.
`-quit_on_extraction_errors {true|false}`	
	Specifies whether the software should exit with an error message or continue when it encounters extraction errors. The default is `true`.
`-remove_em_view_dangling_resistor {true|false}`	
	Specifies to restore or remove dangling resistors from power-grid views. It is recommended to minimize dangling resistors for optimal analysis of IR drops and accurate Blech length calculation at the top level in EM analysis. The default value is `false`.
`-retain_generated_pgv_on_error {true | false}`	
	Specifies to retain an incomplete cell library created during PGV generation even if the tool exits with a fatal error. When set to `false`, the tool deletes the incomplete cell library. This parameter is recommended for use during initial library creation and debugging. The default value is `false`.
`-reuse_electrical_data {true | false}`	
	Default: `false` Specifies to reuse the electrical data from the old PGV and only process the physical part in the new PGV.
`-redo_characterization_count` *count*	

	Specifies to rerun the cells that failed due to machine issues during PGV library generation. This parameter allows you to specify the number of times the failed cells can be resubmitted in the distributed processing mode. *Default*: `0`
`-schematic {true|false}`	
	Specifies whether the subcircuit netlist is a schematic netlist.
`-skip_switch_net_extraction {true | false}`	
	Specifies to skip extraction of switched nets inside the GDS file for a fine-grain memory cell. The software will add the capacitance from the switched net to the always-on net. The software will thus consider the cell as a regular memory without switched/power-gated nets. When this parameter is specified, you do not need to specify the port file (`set_advanced_pg_library_mode -add_port_labels`) as the switched nets are not traced. The final PGV will be created with the always-on and VSS nets. The power-gate library report file will contain information about the missing switched net, and the capacitance and current that was contributed by the switched net.
`-source_location_file {`*`filename`*`}`	
	Specifies the name of the file containing the locations of the voltage sources to be connected to the pad cells. For more information on the source location file, see Source Location File Format.
`-spectre_path` *`value`*	
	Specifies an external Spectre path for PGV generation. This parameter allows you to use a different Spectre version than the one embedded with the software. This parameter is applicable only for standard cell and macro PGV generation (`-celltype stdcells| macros`). **Note**: To specify an external Spectre path for PGV generation, the `-use_embedded_spectre` option should be set to `false`.

`-stdcell_characterization_voltage_value {val1 val2 val3 ...}`

Specifies capacitance characterization voltages when generating standard cell PGVs containing voltage-dependent capacitance table for the specified cell. This parameter allows you to compute capacitance based on the user-defined voltage values. The `-stdcell_characterization_voltage_value` parameter must be specified with the `set_pg_library_mode -enable_multi_voltage_cap_generation true` parameter.

If this parameter is not specified with `-enable_multi_voltage_cap_generation`, capacitance is calculated based on the software-derived voltage range.

When the `-stdcell_characterization_voltage_value` parameter is specified, the library generation engine prints the capacitance values and corresponding voltages in the report file, and the rail analysis engine prints the capacitance used for each instance.

The following example illustrates the use model of this parameter:

```
set_pg_library_mode  -ground_pins VSS -power_pins  {VDD 0.9 TVDD 0.9 } -cell_type stdcells \
...
-enable_multi_voltage_cap_generation true
set_advanced_pg_library_mode -stdcell_characterization_voltage_value {1.0 1.2 1.5 2.0 3.3}
```

`-strict_input_check {true|false}`

Specifies that the software should not create any view for the cell if the EM view is not generated. When set to `true`, the software displays an error message if the Spice netlist or GDS is not specified, and gives details about the cell name and the reason for not generating the view. The default value of this parameter is `false`.

`-tap_node_distance value`

	Specifies to set the maximum distance between interface nodes when they get generated in power-grid library generation. The default is `50` microns. Library generation creates interface nodes on port geometries in LEF and Extraction, uses them for top level connectivity tracing and also to connect the cell/block onto top level routing. The interface node generation is purely based on port geometries (tap points are placed one per port). For ports greater than `-tap_node_distance`, they are placed at `-tap_node_distance` apart. If the port pin geometry in LEF is bigger than the setting of the `-tap_node_distance` parameter, at least two nodes will get generated per port geometry.
`-techgen_dir` *directory*	
	Specifies the techgen directory.
`-thunder_command_file` *file*	
	Specifies the file name of a thunder command file.
`-trim_metal_layer_map` *file*	
	Specifies the name of the file containing the mapping between a metal layer and its corresponding trim layer. When this parameter is specified, the trim layer shapes are automatically removed from the routing layer shapes to correctly extract all the nets in the generated PGV. **File Format**: *metal_layer_name trim_layer_name* M0 CM0 M1 CM1 M2 CM2
`-use_embedded_spectre {true | false}`	
	When set to `false`, allows you to specify a different Spectre version than the one embedded with the software. Library Generation will automatically pick Spectre from users' path. *Default* : `true`

`-use_peak_current_distribution {true | false}`
Default: `false` Specifies to use the maximum currents of devices in the Spice netlist for current distribution in macros. When set to `true`, enables peak current distribution for faster PGV generation. The default (`false`) behavior is to enable activity propagation based current distribution.
`-use_physical_view_from_techpgv {true | false}`
Default: `false` Specifies to change the tap node distance of LEF based PGVs by only generating a new tech PGV, without regenerating the entire PGV. You can pass the new generated library with the previously generated PGVs during rail analysis, so that the updated tech PGV is used for tap node distance. The electrical parameters (currents, capacitances) are considered from original PGVs. **Usage Guidelines**: This parameter is applicable only for standard cell and macro PGV generation (`-celltype stdcells | macros`).
`-verbosity {true | false}`
Controls the amount of information displayed in the console and written to the log files during power-grid library generation. *Default*: false You can use this parameter for detailed troubleshooting and controlling the verbosity level in the log file.
`-via_based_current_config_file` *filename*
Specifies the name of the configuration file containing tap current details for location-based via current generation. If `-enable_via_based_current_distribution` is set to `true`, this parameter must be specified. **Configuration File Format**:

```
#-------------------------------------------------------
--------------
CURRENT_UNIT <default_value>
INPUT_SLEW <default_value>
OUTPUT_SLEW <default_value>
MAX_DISTANCE <Threshold distance for snapping via tap if
the exact xy location is not found as specified>
DELAY <default_value>
PERIOD <default_value>
INTRINSIC_CAP { power_net cap ground_net cap}
#-------------------------------------------------------
--------------
ID     PGPIN_NAME     X    Y     AVG_CURRENT
PEAK_CURRENT
#-------------------------------------------------------
--------------
```

Configuration File Example

```
#-------------------------------------------------------
--------------
CURRENT_UNIT mA
INPUT_SLEW 20ps
OUTPUT_SLEW 10ps
MAX_DISTANCE 2.5
DELAY 5ps
INTRINSIC_CAP { VDD 40nF VSS 5nF }
#-------------------------------------------------------
--------------
ID      PGPIN_NAME     X          Y     AVG_CURRENT
PEAK_CURRENT
#-------------------------------------------------------
--------------
PVDD_1   VDD       16.3090   7.6470     50.0373
100.037
PVDD_2   VDD       16.6900   7.6470     50.0373
100.037
PVDD_3   VDD       17.0700   7.6470     50.0373
100.037
...
```

When this feature is enabled, the tool by default generates a report "`dropped_via_location_taps.txt`" in the output directory to give a list of via IDs that were dropped because they were not

	within the snapping threshold limit. By default, the snapping threshold is zero. It is controlled by "`MAX DISTANCE`" in the configuration file.
`-well_capacitance_file` *`filename`*	
	Specifies the name of a file containing a list of cells and the well capacitance value for each.
`-xtc_command_file` *`filename`*	
	Specifies the name of the XTC command file.
`-xtc_include_file` *`filename`*	
	Specifies the file name of an XTC include file used during characterization. For example, you can include XTC commands to ignore GDS cells, to blackbox lower-level hierarchy cells, or to perform some basic GDS layer operations.

Examples

- The following command creates a power-grid library:

```
set_advanced_pg_library_mode \
-pg_library_generation_command_file library_inc.cmd
-thunder_command_file simulator.cmd
```

- In the following example, power-grid library generation ignores the LEF *metal1* and *metal3* layers when it creates the cell library:

```
set_advanced_pg_library_mode -lef_layer_ignore_list {metal1 metal3}
```

- The following line indicates that a file named *nolef.txt* contains a list of the LEF layer names that power-grid library generation should ignore:

```
set_advanced_pg_library_mode -lef_layer_ignore_list_file {nolef.txt}
```

- The following commands tags the `VDD` and `TVDD` pins as the ESD pins:

```
set_advanced_pg_library_mode -esd_cells {IOESD} -esd_pin_list {VDD TVDD}
```

- The following example specifies that FOREIGN statements are to be assumed for cells with no FOREIGN statements and that the FOREIGN cells should be placed at the inverse of the origin:

```
-assume_foreign true
-assume_foreign_mode 1
```

- The following example specifies that FOREIGN statements are to be assumed for cells with no FOREIGN statements, and the FOREIGN cells should be placed at (0,0):

```
-assume_foreign true
-assume_foreign_mode 0
```

- The following example enables the reuse of electrical data from an old PGV to create a new PGV:

```
set_advanced_pg_library_mode -pgv_path <path> -reuse_electrical_data true
```

Source Location File Format

The voltage source location file format is as follows:

```
<MIN_SPACING spacing_value>
<DENSITY density_value>
CELL cell_name
```

```
<[PORT|DETAILED] {>
NET net_name
<[PORT|DETAILED] {>
lef_layer_name <PORT|DETAILED> x y [x y]
lef_layer_name <PORT|DETAILED> x y [x y]
<}>
<}>
This source location file syntax includes the following parameters:
```

- NET net_name
  These specifications apply to the voltage source locations that follow until Library Generation encounters a new CELL or NET line. You can specify voltage sources as single points or as rectangles. If you specify them as rectangles, Library Generation generates a grid of voltage sources inside these rectangles.
- PORT | DETAILED
  The source location file can specify voltage sources to be used with port or detailed views.

lef_layer_name <PORT|DETAILED> x y [x y]
Lines beginning with a LEF layer name specify the actual location of the voltage sources.

You can specify this location as a single point by providing one set of values for x and y. These x and y coordinates are specified in microns and use the same coordinate system as that used for LEF ports in the LEF file.

Alternatively, you can specify the voltage source locations as a rectangle by giving two sets of x and y locations. In this case, Library Generation attempts to generate a 10-by-10 grid of voltage sources inside this area. From a rectangle specification, it does not create voltage sources closer than 1 micron apart, so if the rectangle is less than about 10 microns on a side, it generates fewer voltage sources.

In the source location file, the location of the voltage sources is specified by using a LEF layer name and location in LEF coordinate space. However, because of the way LibGen uses this data internally, no LEF geometries are actually required to exist at the location where the voltage sources are to be placed. Library Generation uses this data internally with the layout of the block (that is, the GDS data) during detailed power-grid view generation, so it only requires that there be layout (GDS) geometries at the location of the voltage sources. Similarly, the LEF layer name used in the source location file does not actually have to be included in any of the input LEF files; it must be defined only in the layer map used by Library Generation.

This is an example of the contents of the voltage source location file:

```
CELL BUMP
NET vcc
METAL1 11.0 16.0
METAL1 0.0 15.0 22.1 17.5
```

Here is an example file using MIN_SPACING and DENSITY to create a 10 x 10 grid of voltage sources on the VSS net in the detailed view of the NAND1 cell:

```
DENSITY 10
MIN_SPACING 1
CELL BUMP
DETAILED
NET VSS
```

Related Topics

- "Power-Grid Library Generation" in the *Voltus User Guide*

set_pg_library_mode

```
set_pg_library_mode
[-help]
[-add_power_pin_location_cells cell1 cell2 ...]
[-bulk_ground_pins { ground_pin_list }]
[-bulk_power_pins { pin1 voltage1 ... pinN voltageN }]
[-calculate_metal_layer_density {true | false}]
[-cell_power_pin_location_details filename]
-cell_type { techonly | stdcells | macros }
[-cells_file filename ]
[-current_distribution {propagation | dynamic_simulation trigger_file | current_region
region_file}]
[-custom_label label]
[-decap_cells cell_name_list]
[-decap_cells_file filename]
[-default_area_capacitance value]
[-default_power_voltage value]
[-design_qrc_layer_map file]
[-enable_device_size_current_distribution {true | false}]
[-enable_distributed_processing  {true|false} ]
[-enable_multi_voltage_cap_generation {true | false}]
[-exclude_cells_file filename]
[-extraction_tech_file file]
[-filler_cells cell_name_list]
[-fine_grain_memory_cell  {true | false} ]
[-generate_port_using_stream mbb_layer_name ]
[-ground_pins { ground_pin_list }]
[-ignore_stream_cells {gds_cell_name_list}]
[-internal_ground_pins ground_pin_list]
[-internal_power_pins power_pin_list]
[-layer_tap_node_distance { {LayerName1tap_node_distance} {LayerName2tap_node_distance}
... }]
[-layers_to_be_ignored_during_tap_creation {layername+}]
[-layers_to_have_ports_created {layername+}]
[-lef_layer_map file ]
[-liberty_files file_list]
[-macros_config_file filename]
[-max_metal_density_tile_dimension {maxlengthmaxwidth}]
[-non_zero_ground_pins { pin1voltage1 ... pinN voltageN }]
[-oasis_stream_files file_list ]
```

```
[-oasis_stream_layer_map file ]
[-pin_shape_current_distribution filename]
[-power_pins { pin1voltage1 ... pinN voltageN }]
[-power_switch_fine_grain_simulation {true|false} ]
[-power_switch_parameters  {{ cell supply switchedRon Idsat Ileak enablePinName
Control_Pin_Logic} + } ]
[-power_switch_parameters_file filename ]
[-spice_corners corner_list]
[-spice_models file_list]
[-spice_netlist_file filename]
[-spice_subckts file_list]
[-spice_subckts_xy_scaling value]
[-stop_via layername ]
[-stop_via_file filename]
[-stream_files file_list ]
[-stream_layer_map file ]
[-tap_node_distance_file filename]
[-tech_oasis_stream_layer_map filename]
[-tech_stream_layer_map filename]
[-temperature temp_value_in_degree_celcius ]
[-tsv_subckt_modelfile_list filename]
[-use_lefonly_for_detailedview_generation {true | false} ]
[-use_power_switch_data_from_lib {true|false}]
```

Specifies what kind of power-grid library will be created. This command is used to define the type of input and configuration for generating power-grid libraries. You must specify this command before running the `write_pg_library` command.

Parameters

`-help`
Outputs the command usage.
`-add_power_pin_location_cells cell1 cell2 ...`

Specifies the name of the low-dropout (LDO) cell for which a voltage source is to be added. You can specify a list of cells. This parameter is applicable to the LDO cells only, and must be used when creating voltage sources for these cells. When this parameter is specified, all the output pins will be made voltage sources using the highest voltage value of the cell.

The `-add_power_pin_location_cells` parameter is used with the `-cell_power_pin_location_details` parameter.

`-bulk_ground_pins { ground_pin_list }`

Specifies the names of nets that connect transistor bulk terminals to the ground net. This information is used to prevent any current from being assigned to these nets. If a net is connected to any transistor terminals other than a bulk terminal, you must specify that net with the `-ground_pins` parameter.

`-bulk_power_pins { pin1 voltage1 ...pinN voltageN }`

Specifies the names of nets that connect transistor bulk terminals to the power net. This information is used to prevent any current from being assigned to these nets. If a net is connected to any transistor terminals other than a bulk terminal, you must specify that net with the `-power_pins` parameter.

`-calculate_metal_layer_density {true | false}`

Enables metal layer density calculation for macros. The default value is `false`.

When this parameter is set to `true`, Library Generation calculates and stores the area density information for each metal layer in macros for accurate thermal analysis. Metal density for macros is calculated by dividing the macro cell into tiles, and computing the area of polygons in each tile. The calculated metal density values are used in the Power Analysis flow to generate the metal power density map. This power map file is later used as input by Celsius Thermal Solver, the thermal analysis tool for packages and boards.

This parameter is applicable only for macro PGV generation (`-cell_type macros`).

`-cells_file filename`

Specifies a file containing a list of cells that will be processed.

The `-cells_file` parameter is required when `set_pg_library_mode -cell_type macros` is selected. The program spawns the library generation job in serial mode for each macro in the cell list file. A separate library for each macro is generated. This facilitates debugging when library generation of any macro fails. It also helps in improving the overall performance. The following is a sample cell list file:

```
CELL1
CELL2
CELL3
```

`-cell_power_pin_location_details` *`filename`*

Specifies a file that contains all the information required to add a voltage source at the desired location for the power pad and LDO cells. This file includes the pad cell name, pin name, layer name, and domain name.

The format of the file is:

```
cell_name      pin_name    layer_name  domain_name
PADCELL           VDD       Metal3       VDD
```

- *`pin_name`* is the power/ground name for pad cells, and is the output pin name for LDO cells. The pin name could be represented by using the wildcard character "*" to indicate that voltage sources are to be added to all pins of a power pad/LDO cell.
- *`domain_name`* is the pin name for pad cells. For LDO cells, it is the power or ground name whose voltage needs to be used.

Example:

```
set_pg_library_mode \
    -ground_pins                {VSS 0 ExtVSS 0} \
    -power_pins                 {VDD 1.2 ExtVDD 1.2} \
    -cell_type                   techonly \
    ...
    -add_power_pin_location_cells             {PADVDD PADDI PADVSS} \
    -cell_power_pin_location_details          /VoltageSource/config.txt \
```

The following is a snippet of the config.txt file:

```
PADDO * VDD
PADVDD VDD Metal3 VDD
PADVSS VSS Metal2 VSS
PADVDD VDD Metal2 VDD
PADVSS VSS Metal3 VSS
```

`-cell_type { techonly | stdcells | macros}`

Specifies the type of library creation.

- `techonly` – Creates a technology Library that contains the tech file, area capacitance specification, decap/filler/powergate cells, and Tech view for all the cells. A technology library is a requirement for running rail analysis.
- `stdcells` – Generates a power-grid library for all the standard cells using the Spice netlist. The stdcells power-grid library does not contain the tech file. Power-grid is connected at the LEF pin, and all the capacitance are derived from simulation. The power-grid library contains three kind of power-grid views (PGVs), Early/IR/EM, and the views are selected based on the accuracy mode of rail analysis.
- `macros` – Generates a power-grid library for memory, I/Os and macros using the Spice and GDS netlist. GDS is mandatory for the cell type macros. The `macros` power-grid library does not contain the tech file. Power-grid is connected down to contact/specified via, and all the capacitance are derived from simulation. The power-grid library contains three kind of PGVs, Early/IR/EM, and the views are selected based on the accuracy mode of rail analysis.

`-current_distribution {propagation | dynamic_simulation` *`trigger_file`* `| current_region` *`region_file`*`}`

Specifies the method of current distribution inside the cell.

- **Propagation** uses Voltus library simulator to determine the propagation activity to distribute the current.
- **Dynamic simulation** uses a trigger file to run detailed analysis to determine current distribution. For more information on the format, see Trigger File Format.
- **Current region** specifies the amount of current to distribute within a cell region. For more information on the format, see Current Region File.

`-custom_label` *`label`*

Saves the specified label in the PGV for easy identification. This label can be used for revision control, label for Spice netlist/model, and internal keywords. It will be stored as “Custom Label” in VERSION.TXT in the .cl

`-decap_cells` *`cell_name_list`*

	Specifies the names of cells that have explicit decoupling capacitance for use in decap optimization. These cells are nearly equivalent to feedthrough cells, except that there is typically an active device between the power and ground nets to provide extra decoupling capacitance. Power-grid view library generation uses this list of cells to ensure that a capacitance value is associated with each cell. If no capacitance is associated with a cell, a warning message will be issued. This parameter supports wildcards (*).
`-decap_cells_file` *filename*	

Specifies the name of a file containing a list of cells and the decoupling capacitance value for each. You can also specify the series resistance value along with the decoupling capacitance value to accurately model the cell decoupling capacitance. The cell decap file format allows you to specify capacitance for multiple voltage values to support multi-voltage capacitance simulation for standard cell and macro PGV generation.

File Format:

```
UNIT <X>
CELL cell_name
PIN pin_name capacitance resistance

or
UNIT <X>
CELL cell_name
VOLTAGE value1
PIN pin_name capacitance resistance
...
....

VOLTAGE value2
PIN pin_name capacitance resistance
...
...
```

Unit is optional, default is `1e-15`. The *value* for each cell will be multiplied by the unit, except if engineering notation is provided.

Examples:

```
UNIT 1
CELL cell1

VOLTAGE 1.0
PIN VDD  1e-18
PIN VSS  1e-18

VOLTAGE 0.9
PIN VDD  1e-18
PIN VSS  1e-18
```

`-default_area_capacitance value`

Specifies the default amount of area based decoupling capacitance in a cell. The default area-cap is 0.0 (unit is fF per micron-square). You must provide a value for area-based decoupling capacitance.

`-default_power_voltage` *value*

Specifies voltage for the power pins which are present in LEF but not specified using the `-power_pins` parameter. As a result, the `-power_pins` parameter of the command is optional. The default voltage value specified using `-default_power_voltage` is added to all power pins of the cell.

If both the parameters `-power_pins` and `-default_power_voltage` are not specified, the default voltage of 1.2 will be used.

`-design_qrc_layer_map` *file*

Specifies a file that provides the mapping of the layer names in the LEF design file to the layer names in the Quantus technology file.
File Format:

```
lefdef_layername     technology_layername
M1                    METAL_1
VIA1                  VIA_1

M2                    METAL_2
VIA2                  VIA_2
...
```

`-enable_device_size_current_distribution {true | false}`

Specifies to compute the tap current distribution for a macro based on device sizes (Width, Length), without performing any simulation. This parameter is used for macro PGV generation in a scenario when the Spice netlist is available but the Spice models are not available.
Default: `false`

`-enable_distributed_processing {true|false}`

Specifies to enable parallel processing for the Library Generation flow.

`-enable_multi_voltage_cap_generation {true | false}`

Specifies to compute capacitance values over a range of input voltages, including the nominal voltage. It generates 10 values from 0.1 to 1.3 times the vdd/vss voltage. The default value is `false`. When set to `true`, it generates a PGV that contains the voltage-dependent capacitance table for the specified cell. This parameter allows you to use the accurate cell capacitance at different supply voltages in power-up analysis and in different voltage domains in dynamic analysis.
`-exclude_cells_file` *filename*
Specifies a file containing a list of cells (one per line) which should be excluded from power-grid library generation.
`-extraction_tech_file` *file*
Specifies the name of the technology file that will be used for extraction.
`-filler_cells` *cell_name_list*
List the names of the filler or feedthrough cells that have no GDS data. These cells can be swapped out for decaps during decap optimization. This parameter supports wildcards (*).
`-fine_grain_memory_cell {true | false}`
Specifies to tag the cell as the fine-grain cell in the PGV library when characterizing the cell, and ensures that the software automatically uses this information to identify the fine-grain memory cells in the rail analysis flow. The default value is `false`. This parameter is used with the `-internal_power_pins` and `-internal_ground_pins` parameters to specify the list of switch power/ground pins. It is recommended to use these parameters when characterizing fine-grain cells to avoid any potential cell type identification conflict in the rail analysis flow.
`-generate_port_using_stream` *mbb_layer_name*
Identifies the layer name that library generation uses to compute the placement bounding box for GDSII cells that are placed by the DEF file.
`-ground_pins {` *ground_pin_list* `}`
Specifies the names of the nets considered ground nets for all cells for which the ground nets are not specified.

`-ignore_stream_cells {gds_cell_name_list}`

Ignores the specified cells during macro PGV generation. When this parameter is specified, the Library Generation engine does not extract data for these cells. This parameter supports wildcards.

Example:

The following command does not extract data for the `cell3` cell:

```
set_pg_library_mode -ignore_stream_cells {cell3}
```

`-internal_power_pins power_pin_list`

Specifies the list of switch power pins.

`-internal_ground_pins ground_pin_list`

Specifies the list of switch ground pins.

`-layer_tap_node_distance { {LayerName1 tap_node_distance} {LayerName2 tap_node_distance} ... }`

Default: None

Specifies tap node distances for certain layers. Using this parameter, you can customize the tap node distance for each layer during power-grid library generation.

The tap node distance is in micron. If the tap node distance is specified for a single layer, double brackets {{}} are still required.

`-layers_to_be_ignored_during_tap_creation {layername+}`

Specifies the names of the layers that library generation is to ignore when it creates the current taps. This parameter can be used to suppress the generation of any nodes or taps on layers that should not be extracted. Some technologies include layers that are only used to influence the extraction of other layers.

`-layers_to_have_ports_created {layername+}`

Specifies the names of the layers for which ports are to be created. This parameter can be used to create ports for the poly and diffusion layers in the technology file. By default ports are only created on metal layers.

`-lef_layer_map file`

Specifies a file that provides the mapping of layers between the LEF and technology file. This parameter is optional. Library Generation automatically generates the layermap file using the technology LEF (containing all layer and via definitions) and QRC Tech (extraction library) files, therefore, a LEF layermap file is not a mandatory input. The automatically generated layermap file is located in the PGV temp directory.

The following is the format and sample content of the auto-generated file (`lefdefLayerMap_AutoGenerated.txt`):

```
Layer Type    tech layername    lefdef    lefdef layername
via               VIA_9         lefdef      VIA9
metal             METAL_1       lefdef      METAL1
```

You can specify this parameter if you want to provide your own layermap file.

`-liberty_files` *file_list*

Specifies to directly read decoupling capacitance and resistance from the specified Liberty file for standard cell and macro PGV generation.

The following is an example of the Liberty construct required for capacitance/resistance:

```
intrinsic_parasitic () {
  when : "I";

  intrinsic_capacitance (VDD) {
    value : 0.000248992;
  }

  intrinsic_resistance (VSS) {
    related_output : ZN;
    value : 1.89195;
  }

  intrinsic_capacitance (VSS) {
    value : 0.000639008;
  }
}
```

If there are multiple `when` conditions, the average intrinsic resistance/capacitance value is taken.

`-macros_config_file` *filename*

Specifies to provide configuration details for each macro cell per Library Generation run. This is an additional input for `macros` cell type wherein you can provide macro specific Spice and GDS. The format of the file is:

```
CELL <cell1>
GDS_FILE <gds_file1>
SUBCKT_FILE <subckt_file1>
DECAP_FILE <decap_file1>
TRIGGER_FILE <trigger_file1>
POWER_GATE_PARAMS supply <supplyName> switched <switchedName>
CELL <cell2>
GDS_FILE < gds_file1 >
SUBCKT_FILE < subckt_file1 >
```

`CELL` is the keyword to specify a new cell that is to be characterized. This is a mandatory keyword. This keyword is followed by optional keywords, like `GDS_FILE` and `SUBCKT_FILE`, that are specific to the cell.

`-max_metal_density_tile_dimension {maxlength maxwidth}`

Specifies the tile dimensions, wherein *`maxlength`* denotes maximum length and *`maxwidth`* denotes maximum width in µm. The default value for length or width is 50µm. The number of tiles in the X and Y direction is determined using the length and width dimension.

This parameter should only be used when the `-calculate_metal_layer_density` parameter is set to `true`.

`-non_zero_ground_pins { pin1voltage1 ... pinN voltageN }`

Default: None

Specifies the name and voltage of the nets considered ground nets for all cells for which the elevated ground nets need to be specified. This parameter allows you to provide a list of non-zero ground be it positive or negative for the ground nets. If the same name has been specified for both the parameters `-non_zero_ground_pins` and `-ground_pins`, then `-non_zero_ground_pins` will be given priority.

This parameter is applicable only for standard cell and macro PGV generation (`-celltype stdcells | macros`).

Example:

```
set_pg_library_mode -non_zero_ground_pins {VSS -1.05 VSSA 0.08 VSSA1 1.09}
```

`-oasis_stream_files` *`file_list`*

Specifies the names of the OASIS files used during power-grid library generation.

Usage Guidelines:

This parameter is a mandatory input for macro PGV generation (`-celltype macros`).

`-oasis_stream_layer_map` *`file`*

Specifies a file that provides the mapping between the library layers (qrcTechFile layers) and the OASIS layers.

File Format:

```
layer_type    layer_name    oasis    oasis_layer_no
via             VIA9        oasis        59
```

Usage Guidelines:

This parameter is a mandatory input for macro PGV generation (`-celltype macros`).

`-pin_shape_current_distribution` *`filename`*

Specifies a current distribution file that allows you to state how current should be distributed across multiple pin shapes of power and ground pins. Multiple pin shapes exist for VDD and VSS pins, and current drawn through each pin shape is non-uniform depending upon the output bit that is switching. This parameter allows you to define current distribution for multi-height multi-bit flip-flop (MBFF) characterization. The parameter allows you to accurately model the current through each pin shape.

File Format

```
CELL <cell name>
PGPIN VDD:(<LEF_layer_name>,<LEF_coordinates>):(<LEF_layer_name>,
<LEF_coordinates>)
PGPIN VSS:(<LEF_layer_name>,<LEF_coordinates>):(<LEF_layer_name>,
<LEF_coordinates>)
PIN <signal_pin_name> <current_ratio_for_VDD> <current_ratio_for_VSS>
```

Here,

- *<LEF_layer_name>,<LEF_coordinates>* specifies to assign the current ratio to the pin shape occupying the coordinates.
- *LEF_layer_name* the layer name to which the current taps are to be created

Example

```
CELL NAND3X8
PGPIN VDD:(Metal1,1,1):(Metal1,4,1)
PGPIN VSS:(Metal1,2,2):(Metal2,5,2):(Metal2,8,2)
PIN Q1 0.00:1.00 0.00:0.00:1.00
PIN Q2 0.90:0.10 0.95:0.05:0.00
```

`-power_pins { pin1 voltage1...pinN voltageN }`

Specifies the name and voltage of the nets considered power nets for all cells for which the power nets are not specified. If both the parameters `-power_pins` and `-default_power_voltage` are not specified, the default voltage of 1.2 will be used.

`-power_switch_fine_grain_simulation {true|false}`

Specifies to enable finegrain powergate simulation.

`-power_switch_parameters {{cell supply switched Ron Idsat Ileak enablePinName Control_Pin_Logic} + }`

Specifies the information that technology library generation needs to characterize a powergate cell. It includes the cell name, unswitched power pin, switched power pin, on resistance in ohms, saturation current in milliamps, leakage current in milliamps, and

control pin logic.

If ron, idsat, or ileak values are specified, they will be used to override the data extracted by Voltus library simulator.

`Control_Pin_Logic` can have the following values:

- `enableHigh` - control pin(s) of the control logic will be connected to the power net to turn on the power gate instances.
- `enableLow` - control pin(s) of the control logic will be connected to the ground net to turn on the power gate instances.

The control pin logic allows you to specify whether the enable pin of the specified power gate is active high or active low, thus ensuring that the appropriate voltages are applied to the enable pins.

You can specify multiple values for on-resistance, saturation current, and leakage current for the same cell based on different enable pin names.

This parameter also allows you to specify multiple functional modes to enable different combinations of control pins in each mode. For multiple functional modes, the tool calculates RON for each mode separately and generates a single PGV for the specified modes.

Format for Multiple Functional Modes:

```
set_pg_library_mode -power_switch_parameters { {cell_name mode_name1 {supply switch Ron Idsat Ileak Control_Pin_Logic} mode_name2 {supply switch Ron Idsat Ileak Control_Pin_Logic}... mode_nameN {supply switch Ron Idsat Ileak Control_Pin_Logic} }
```

Examples:

- For control pin logic:

  ```
  -power_switch_parameters {  {HEADER_SWITCH TVDD VDD enableHigh} }
  ```

- For Single Powergate:

  ```
  -power_switch_parameters{{HDRSID0 TVDD VDD}}
  ```

- For 2 Header Powergate:

  ```
  -power_switch_parameters{{HEAD16_A12TH_C35 VDDG VDD} {HEAD8_A12TH_C35 VDDG VDD}}
  ```

- For Multiple Footer Powergate

  ```
  -power_switch_parameters {{FOOTBUFVHSV16 VSSG VSS} {FOOTBUFVHSV32 VSSG VSS} {FOOTBUFVHSV64 VSSG VSS} {FOOTBUFVHSV8 VSSG VSS}}
  ```

- For Multiple Functional Modes:

  ```
  -power_switch_parameters { {HEADER_SWITCH S1 {TVDD VDD 750 .5 4.0e-8V} S3
  {TVDD1 VDD 350 .3 5.0e-8V} S2 {TVDD VDD 450 .5 4.0e-8V} } {RING_SWITCH TVDD
  VDD 750 .5 4.0e-8} }
  ```

- For Different Enable Pins:

  ```
  -power_switch_parameters {{Cell1 VDDG VDD 300 5e-8 1500 Enable1 } {Cell1
  VDDG VDD 280 4e-8 200 Enable2 }}
  ```

Usage Guidelines:

This parameter is applicable only for the technology PGV generation (`-celltype techonly`).

`-power_switch_parameters_file` *filename*

Specifies a file that contains details of power switches that have multiple control pins, input pins, and power nets. The control pins, input pins, and power nets can have either sweep voltages or user-defined voltages.

This parameter also allows you to specify multiple functional modes to enable different combinations of control pins in each mode. For multiple functional modes, the tool calculates RON for each mode separately and generates a single PGV for the specified modes.

File Format:

```
CELL <cell_name>
PGATE_PAIR  <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
PGATE_PAIR  <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
…
PWR_PIN <net_name> {voltage_values}
PWR_PIN <net_name> {voltage_values}
…
INPUT_PIN <pin_name> <Polarity|VoltageValues>
INPUT_PIN <pin_name> <Polarity|VoltageValues>
…

PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
…

CHARACTERIZATION_VOLTAGES { val1 val2 val3 .... }
```

```
END
```

File Format for Multiple Functional Modes:

```
CELL <cell_name>
MODE_NAME mode_name1
PGATE_PAIR <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
PWR_PIN <net_name> {voltage_values}
INPUT_PIN <pin_name> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
CHARACTERIZATION_VOLTAGES <VolageValues>
END_MODE

MODE_NAME mode_name2
PGATE_PAIR  <AON_PIN> <SW_PIN> <CTRL_PIN> <Polarity|VoltageValues>
PWR_PIN <net_name> {voltage_values}
INPUT_PIN <pin_name> <Polarity|VoltageValues>
PGATE_OFF_PIN <CTRL_PIN> <Polarity|VoltageValues>
CHARACTERIZATION_VOLTAGES <VolageValues>
END_MODE
...

END
```

- `PGATE_PAIR` and `CHARACTERIZATION_VOLTAGES` are mandatory keywords.
- `PGATE_PAIR` is the keyword to specify the always-on power pin, switched power pin, and control pin names, and the voltage values at which these pins are simulated. If you specify voltage values, then the software uses these user-defined values during power gate characterization. Alternatively, if you specify polarity (high/low), then the software uses the sweep voltage values (`CHARACTERIZATION_VOLTAGES`) during power gate characterization. If you do not specify both polarity or voltage values, then the software by default uses the sweep voltage values (`CHARACTERIZATION_VOLTAGES`) during power gate characterization.
- `CHARACTERIZATION_VOLTAGES` is the keyword to specify sweep voltage values. The number of voltage values specified with the `PGATE_PAIR` keyword must match with the number of voltage values specified with the `CHARACTERIZATION_VOLTAGES` keyword.
- `PWR_PIN` is the keyword to specify additional power pins of the cell other than the always-on and switched power pins.
- `INPUT_PIN` is the keyword to specify additional signal pins of the cell other than the control pins. If you specify voltage values, then the software uses these user-defined

values during power gate characterization. Alternatively, if you specify polarity (high/low), then the software uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization. If you do not specify both polarity or voltage values, then the software by default uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization.

- PGATE_OFF_PIN is the keyword to specify the control pin names and the voltage values at which the off-resistance is computed. If you specify voltage values, then the software uses these user-defined values during power gate characterization. Alternatively, if you specify polarity (high/low), then the software uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization. If you do not specify both polarity or voltage values, then the software by default uses the sweep voltage values (CHARACTERIZATION_VOLTAGES) during power gate characterization.

Examples:

Example 1: User-specified voltage values for control pins, input pins, and power nets

```
CELL cellA
PGATE_PAIR AONVDD SWVDD SLEEPN {0.600 0.660 0.700 0.750 0.800 }
PWR_PIN VDD_PS {0.7 0.8 0.9 1.0 1.1 }
INPUT_PIN PS_DISABLE {0.600 0.660 0.700 0.750 0.800 }
CHARACTERIZATION_VOLTAGES {0.600 0.660 0.700 0.750 0.800 }
END
```

Example 2: User-specified polarity

```
CELL cellb
PGATE_PAIR TVDD VDD SLEEPN
PWR_PIN VDD_SIDE
INPUT_PIN S1 HIGH
CHARACTERIZATION_VOLTAGES {0.600 0.660 0.700 0.750 0.800 }
END
```

Example 3: powergate_param_file.txt

```
CELL HEADER_SWITCH
MODE_NAME S1
PGATE_PAIR TVDD VDD NSLEEPIN LOW
CHARACTERIZATION_VOLTAGES {0.9 1.0 1.1}
END_MODE

MODE_NAME S2
```

```
PGATE_PAIR TVDD VDD NSLEEPIN HIGH
CHARACTERIZATION_VOLTAGES {0.9}
END_MODE

MODE_NAME S3
PGATE_PAIR TVDD1 VDD NSLEEPIN LOW
CHARACTERIZATION_VOLTAGES {0.9 1.1}
END_MODE

END
```

Usage Guidelines:

This parameter is applicable only for standard cell and macro PGV generation (`-celltype stdcells | macros`).

`-spice_corners` *`corner_list`*

Specifies a list of SPICE corners. This is an optional parameter. If you specify SPICE corner(s), the number of corner items should be equal to the number of SPICE model files specified in -spice_models *file_list* .

For example, if a SPICE model1 has three corners and model2 has two corners, you should specify:

```
-spice_models { spice_1.model spice_2.model }

-spice_corners { {s11 s12 s13} {s21 s22} }
```

Based on the above parameter description, the following information gets updated in the library generation file:

```
spice_model_file spice_1.model {s11 s12 s13}

spice_mode_file spice_2.model {s21 s22}
```

`-spice_models` *`file_list`*

Specifies the names of the SPICE model files used by the SPICE netlist file (`-spice_subckts`).

Some Spice process models use `wnom/Wn` and `lnom/Ln` instead of `w` and `l` (device model parameters names for length and width) for MOS models. Voltus recognizes and supports these alternate parameter names.

`-spice_netlist_file` *`filename`*

Specifies a file containing all the spice netlists to be used for library generation, the format is one netlist per line. These spice netlist files must contain the subcircuit definitions for the cells being processed by power-grid library generation.

The following example specifies a file called *spice_netlist_list_file* that contains the list of spice netlist files:

```
set_pg_library_mode -spice_netlist_file spice_netlist_list_file
```

`-spice_subckts` *file_list*

Specifies the names of the SPICE netlist files containing the subcircuits corresponding to the cells being processed during power-grid library generation. These subcircuit netlists will be used to calculate pin capacitances using Voltus library simulator.

`-spice_subckts_xy_scaling` *value*

Specifies the current region scale factor for the Netlist-based flow . The current region scale factor is the factor by which the netlist coordinates are scaled before finding the closest node to attach the tap currents.

The netlist-based flow fails when there is a mismatch in the units of the PGDB coordinates (in nm) and the coordinates in the SPICE netlist causing improper current tap creation. To match these coordinates, you need to specify the current region scale factor. The default scale factor is `1000` considering that the SPICE netlist coordinate unit is `um`.

`-stop_via` *layername*

Stops the extraction of the power-grid network inside a cell at the specified via layer.

`-stop_via_file` *filename*

Specifies the name of a file containing a list of cells and the corresponding via layers at which extraction of the power-grid network should be stopped. However, this functionality is applicable only for the cells that are mentioned in the cell list file as well.

File Format:

```
<cell_name> <stop@_layer_name>
```

Sample File Format:

```
Cell1 Via5
Cell2 Via9
```

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-stream_files` *`file_list`*

Specifies the names of the GDSII files used during power-grid library generation.

`-stream_layer_map` *`file`*

Specifies a file that provides the mapping between the library layers (qrcTechFile layers) and the GDS layers.

Sample File Format:

```
layer_type    layer_name    gds    gds_layer_no
via             VIA9        gds       59
```

Usage Guidelines:

This parameter is applicable only for macro PGV generation (`-celltype macros`).

`-tap_node_distance_file` *`filename`*

Specifies a file containing the distance between tap nodes for the given list of cells. The specified file includes the cell name and tap node distance in micron. The default unit for tap node distance is `micron`, and the default tap node distance is `50 microns`.

This parameter allows you to customize the tap node distance for each cell during power-grid library generation.

File Format:

`cell_name tap_node_distance`

Examples:

```
OA_cellA 1
OA_cellC 1
```

This parameter is applicable only for standard cell and technology PGV generation (`-cell_type stdcells|techonly`).

`-tech_oasis_stream_layer_map` *`filename`*

Specifies a file that provides the mapping of the layer names in the Quantus technology file to the layer names in the OASIS file.

File Format:

```
technology_layername   use     oasis_number  data_type
METAL_1                text      131          0
VIA_1                   x         51          0

METAL_2                text      132          0
VIA_2                   x         52          0
...
```

If `-design_qrc_layermap` is specified along with `-tech_oasis_stream_layer_map`, then the format of `-tech_oasis_stream_layer_map` is treated as:

```
lefdef_layername   use     oasis_number  data_type
      M1           text       131          0
```

`-tech_stream_layer_map` *`filename`*

Specifies a file that provides the mapping of the layer names in the Quantus technology file to the layer names in the GDS file.

File Format:

```
technology_layername    use      gds_number   data_type
METAL_1                 text       131          0
VIA_1                    x          51          0

METAL_2                 text       132          0
VIA_2                    x          52          0
...
```

If `-design_qrc_layer_map` is specified along with `-tech_stream_layer_map`, then the format of `-tech_stream_layer_map` is treated as:

```
lefdef_layername    use      gds_number   data_type
      M1            text       131          0
```

`-temperature` *`temp_value_in_degree_celcius`*

Specifies the temperature in degree celcius used in library characterization for thermal aware EM/IR analysis.

Default: 25

`-tsv_subckt_modelfile_list` *`filename`*

Specifies the name of external Through Silicon Via (TSV) subckt model files to be used for 3D-IC extraction. You can specify a list of files separated by space. Each file contains the subckt model information for TSV. The specified file has the Spice format.

A sample file is shown below:

```
.subckt TSV top bottom
R1 top n2 0.03075
R2 n2 bottom 0.03075
C1 n2 0 28f
.ends
```

`-use_lefonly_for_detailedview_generation {true | false}`

	Specifies to generate a detailed PGV with the bump LEF without loading a GDS file. You can specify this parameter to perform the RC extraction of a hybrid bump or via cell while skipping the extraction of its top and bottom routing metal layers. When set to `true`, the top and bottom metal layers are preserved as geometries in the library and promoted to the top level, whereas the via layer shapes are dropped and their resistance is used directly from the PGV. *Default*: `false`
`-use_power_switch_data_from_lib {true|false}`	
	Specifies to read powergate parameters ron, saturation current, and leakage current from the Liberty files specified using the existing `-liberty_files` parameter. This parameter allows you to obtain the powergate parameters from a Liberty file when a Spice netlist is not available, or when the Liberty file contains the powergate parameters.

Examples

- The following command sets the power library mode:

```
set_pg_library_mode
-extraction_tech_file RCgen.tch
-lef_layer_map lefdef.map
-cell_type techonly
-power_pins { VDD 1.08 VDDO 1.08 VDDG 1.08 }
-ground_pins { VSS GND VSSG }
-cells_file built_cells_list
-temperature -40
-spice_models models/v1d1_usage.l
-spice_corners { ss_lib }
-spice_subckts { typical_max_m40.spice }
-power_switch_parameters { { cell1 VDDG VDD } { cell2 VDDG VDD } }
-current_distribution propagation
```

- The following command generates a PGV that contains the voltage dependent capacitance table for the specified cells:

```
set_pg_library_mode -cell_type stdcells -cells_file cellfile -extraction_tech_file
./../qrcTechFile -lef_layermap ./../lefdef.layermap -temperature 125 -power_pins {
vcc 1.26 vccpg 1.26 vccr 1.26 } -ground_pins { vss }  -power_switch_parameters
{{powercell_pg1 vcc vccpg} {powercell_pg1 vcc vccr} {powercell_pg2 vcc vccpg}
{powercell_pg2 vcc vccr}} -spice_corners { tt } -spice_models spice_model/libspice
-spice_subckts { powercell_pg.net powercell_pg2.net } -
enable_multi_voltage_cap_generation true
```

  The PGV report includes the following information:

```
INFO: PGV contains voltage dependent cap table for this cell
```

Trigger File Format

For the current characterization flow, you must specify the `-current_distribution { dynamic_simulation` *`trigger_file`* `}` parameter. The format of the trigger file is:

Trigger File Format	Description
#Method of current characterization	
`CURRENT_CHARACTERIZATION_METHOD <method>` `<SIMULATION/DOTLIB/PWL/FSDB/DATASHEET>`	• SIMULATION -> Vector-based flow • DOTLIB -> Liberty based Flow • PWL - User-defined PWL flow • FSDB - Spice total current output captured in the FSDB format • DATASHEET - Datasheet details of any memory
#Memory cell name	
`CELL <cellname1>`	Name of the memory cell
#Mode and conditional details for trigger	

`LEAKAGE_CURRENT { <PGPin1> <value1> … <PGPinN> <valueN> }`	Specifies the leakage current for both power and ground pins, or just power pins. Dynamic Power Analysis will process `LEAKAGE_CURRENT` specified in the PGV trigger file, and use it when a macro cell is idle or dynamic switch pattern is not defined (defined as "-").
	
	In the following example, the second cycle will use `LEAKAGE_CURRENT` defined in the PGV trigger file:
	
	`set_power -base_cells x -dynamic_switch_pattern { READ - READ }`
	
	You must specify the leakage current for all power pins. If leakage current is not defined for some pins, then those pins will get 0 leakage current when processed by Power Analysis.
	
	`LEAKAGE_CURRENT` should be specified in the trigger file as a global, and specified outside the mode specification, as shown below:
	
	`CURRENT_CHARACTERIZATION_METHOD PWLCELL cell1`
	
	`LEAKAGE_CURRENT { VSS 2p VDD 17n }MODE_NAME READ static...`
	
	`END_MODEEND`

`MODE_NAME <mode1> CONDITIONAL_INPUT <conditional input statement for cellname1>`	Mode name and conditional input statement for the cell. Conditional input of current for the given mode.
`CONDITIONAL_PIN <pin_name> (rise|fall|both)`	Conditional pin statement for the cell. Signal pin for which the tool should rise or fall.
`CONDITIONAL_STIMULUS_FILE <usim1.txt>`	Simulation vector detail in the given mode.
`USER_PWL_FILE <pwl1.txt>`	User-specified PWL file that contains information about the PWL waveforms (power/ground currents). The current for ground net should be specified as a negative value.

`FSDB_START_TIME <time>` `FSDB_END_TIME <time>`	FSDB Simulation based mode details
`DATASHEET_PARAMETER < {pg_pin1 Capacitance Leakage} {pg_pin2 Capacitance Leakage}>`	Datasheet-based mode details
`END_MODE`	End mode section
#Datasheet clock required parameters	

`DATASHEET_INPUT_SLEW <time>`
`DATASHEET_OUTPUT_SLEW <time>`
`DATASHEET_DELAY <time>`
`DATASHEET_PARAMETER < { power_pin cap leakage_pwr } >`
	• `DATASHEET_INPUT_SLEW` - Clock input slew of the conditional pin
	• `DATASHEET_OUTPUT_SLEW` - Rising slew of the output pin Q
	• `DATASHEET_DELAY` - Mode delay as the read_access and write_access time
	• `DATASHEET_PARAMETER` - Datasheet parameters include power pin voltage, load capacitance, and leakage power. Load capacitance is mode dependent.

#FSDB required parameters	

`FSDB_PATH <path>`
`FSDB_NET_NAME <net> [<generic_name>]`
`FSDB_START_TIME <time>`
`FSDB_END_TIME <time>`
`FSDB_HIER_CHAR <hier_char, for hier DB>`
	• FSDB file path
	• Power net name and its FSDB probe name
	• FSDB start and end time
	• Hierarchical separator for hierarchical database. The default separator is "**.**"
	Single FSDB file having total current for ALL power/ground pins with all mode current in it

#<Other mode independent parameters>	

```
DOTLIB_LIST {a list of memory dotlibs
files}
MODE_COUNT
MODE_CONFIG_FILE <file_path>
SIM_START_TIME <time>
SIM_STOP_TIME <time>
SIM_STEP_SIZE <step_size>
STIMULUS_FILE <file_path>
INCLUDE_FILE <file_path>
```

- `DOTLIB_LIST` - Specifies a list of memory dotlibs.
- `MODE_COUNT` - Specifies the maximum number of modes to be generated. Valid values are either a positive number or `all'. `all' means stimulus for all power groups are to be generated. The default value is `4`. If you set this parameter to `all`, it may result in high runtime as it may generate too many modes.
- `MODE_CONFIG_FILE` - Specifies the file containing extra information for memory cell. This file contains the following information - signal slew, signal active value and memory write information.
- `SIM_START_TIME` - Specifies the simulation start time. The default unit is in nanoseconds (ns).
- `SIM_STOP_TIME` - Specifies the simulation stop time. The default unit is in nanoseconds (ns).
- `SIM_STEP_SIZE` - Specifies the step size for power-grid analysis. The default value of step size is 20 picoseconds (ps). This parameter supports other units such as, ns and fs. This parameter supports unit-prefixes (pico, femto, nano) combined with units of measure (ps, fs, ns), but does not support units itself ("second", "Ohm", "Farad") without unit-prefix.
- `STIMULUS_FILE` - Specifies the stimulus file containing the trigger information for all input ports.
- `INCLUDE_FILE` - Specifies the name of the simulation include file.

#End cell section	
`END`	End cell section

Current Region File Format

The following shows the current_region_file format:

```
 UNIT CURRENT default_unit_in_amps
UNIT CAPACITANCE default_unit_in_farads
CELL cell_name
<PROP [FLAT | HIER]>
NET net_name
LAYER layer_name
[ REGION x1 y1 x2 y2 current_value <cap_value> | REGION current_value <cap_value> ]
```

UNIT	Defines the default scale to be used with the current and capacitance values read from the current region file. These scales apply to any value that is not followed by an explicit unit specification. Valid current values are: A (Amps), mA (miliAmps), uA (microAmps), and nA (nanoAmps). Valid capacitance values are: F(Farads), mF (miliFarads), uF (microfarads), nF (nanofarads), pF (picoFarads), and fF (femtoFarads).
CELL *cell_name*	This command specifies the name of the cell. All the following NET, LAYER, and REGION commands will apply to this CELL until another CELL specification is encountered. Wildcards can be used in the cell_name.
<PROP [FLAT | HIER]>	If the keyword FLAT is applied, LibGen will not perform hierarchical tracing for the given net in this cell. If the keyword HIER is presented, LibGen will perform hierarchical tracing for the given net in this cell. This provides the ability for LibGen to automatically specify a current value to all instances of a cell in the design hierarchy. The assumption is that all of these instances of the cell will consume the same power. The default is HIER.

NET *net_name*	This command specifies that all the LAYER and REGION commands following it applies to the net_name. This stays in effect until another NET or CELL specification is encountered. Wildcards can be used in the net_name.
LAYER *layer_name*	This command specifies that all the REGION commands following it apply to the layer_name. This stays in effect until another LAYER, NET, or CELL command is encountered. The layer_name must be a library layer name. If the layer is not found, LibGen will issue an error message and exit. You must provide a valid layer name for LibGen to run. If you specified a valid layer, but LibGen does not find any via in the assigned current region, LibGen will issue a warning message, but the process will still continue and exit successfully. In this case, you may get a 0 current value for the specified cell.
REGION *x1 y1 x2 y2 current_value cap_value*	This command specifies the current value and optionally the capacitance value to be associated with the rectangular region specified by the bounding box (*x1 y1 x2 y2*). Current taps will be created at the locations of the vias or contacts specified by the most recent LAYER command. These current taps will be attached to the layer that is below the specified via or contact layer. Coordinate values are assumed to be in GDSII coordinate units and will be interpreted according to the scale and precision settings contained in the GDSII file (or spice netlist with device XY location) that contains the cell. The value of the current drawn by each current tap will be 1/n of the value specified for the current region, where n is the number of taps within the current region. The second variation of the REGION command without the bounding box is valid only if there is only one current region. In this case LibGen can automatically uses the bounding box of the cell for the region. You can specify a single region or multiple regions per cell.

Example of Current Region File

```
UNIT CURRENT uA
UNIT CAPACITANCE fF
CELL an4_80
NET vcc
LAYER CONT
REGION 0 14000 28900 17000 3.0 5.0
NET gnd
LAYER CONT
REGION 1.5e-2mA
```

Example of a Dynamic PWL Region File (Multiple Regions)

```
CELL Cell1
  NET VSS
    PROP HIER
    LAYER via1_r
      REGION 1 10 188 333   { 0 -1 1 -9  2 -8  3 -7  4 -6  5 -5  6 -4  7 -3  8 -2  9 -
1  } 1e-12
      REGION 121 10 185 80 { 0 -1 1 -90 2 -80 3 -70 4 -60 5 -50 6 -40 7 -30 8 -20 9 -10
} 4e-12

  NET VDDPR
    PROP HIER
    LAYER via1_r
      REGION 1 10 188 333 { 0 -1 1 9  2 8  3 7  4 6  5 5  6 4  7 3  8 2  9 1 } 5e-12

  NET VDDP
    PROP HIER
    LAYER via1_r
      REGION 1 10 188 333 { 0 -1 1 9  2 8  3 7  4 6  5 5  6 4  7 3  8 2  9 1 } 5e-12

  NET VDDAR
    PROP HIER
    LAYER via1_r
      REGION 1 10 188 333 { 0 -1 1 9  2 8  3 7  4 6  5 5  6 4  7 3  8 2  9 1 } 5e-12

  ...
```

Related Topics

- "Power-Grid Library Generation" in the *Voltus User Guide*

trim_pg_library

```
trim_pg_library
[-help]
[-delete_cells cell1 cell2 ...]
[-delete_cells_file file_name]
-library_list_file file_name
[-out_dir directory_name]
-output_library library_name
```

Specifies to remove a list of cells from the specified PGV. This command allows you to trim the size of a cell library by removing unwanted cells, which can reduce the overall time required for extraction.

Parameters

`-help`	Outputs the command usage.
`-delete_cells cell1 cell2 ...`	
	Default: `None` Specifies the name of a cell or a list of cells to be removed from the power-grid library.
`-delete_cells_file file_name`	

	Default: `None`
	Specifies the name of a file containing the list of cells (one per line) to be removed from the power-grid library.
	File Format:
	`cell1`
	`cell2`
	`cell3`

`-library_list_file file_name`	

	Default: None
	(**Mandatory**) Specifies the name of a file containing the library or libraries (one per line) that are to be trimmed. If multiple libraries are given, Library Generation merges the multiple same-type power-grid libraries into one consolidated power-grid library.
	File Format:
	`PGV1`
	`PGV2`
	`PGV3`

`-out_dir` *`directory_name`*	
	Default: current working directory Specifies the name of the output directory in which the trimmed power-grid library will be created.
`-output_library` *`library_name`*	
	Default: None (**Mandatory**) Specifies the name of the trimmed power-grid library created after removing the specified cells.

Examples

- The following command removes the cell `AND2X2` from the power-grid libraries specified in the `library_list.txt` file and generates one consolidated power-grid library `trimlibrary`:
  ```
  trim_pg_library -library_list_file library_list.txt -output_library trimlibrary -delete_cells AND2X2
  ```

Related Topics

- "Power-Grid Library Generation" in the *Voltus User Guide*

write_pg_library

```
write_pg_library
[-help]
[-out_dir dir_name]
[-library_prefix prefix]
```

Generates the output for power-grid library generation.

Parameters

`-help`	
	Outputs the command usage.
`-library_prefix prefix`	
	Specifies a prefix for the power-grid library. When this parameter is specified, the default PGV for each cell type is as follows: • techonly - `prefix_techonly.cl (default: techonly.cl)` • stdcells - `prefix_stdcells.cl (default: stdcells.cl)` • macros - `prefix_cellname.cl (default: macros_cellname.cl)` You can generate macro PGVs without the "`macro_`" string in the name and without any prefix by specifying an empty string for `-library_prefix`, as shown below: `write_pg_library -out_dir output_folder -library_prefix ""`
`-out_dir dir_name`	
	Specifies the name of the output directory in which power-grid libraries are created. The default is the current working directory. **Note**: If you specify both `-out_dir output_dir` and `-library_prefix prefix` parameters, the command generates the PGV in the *output_dir* directory with PGV name `prefix_<name>.cl`.

Examples

- The following command generates a power-grid library in the *temp* folder:

  ```
  write_pg_library -out_dir temp/
  ```

- The following command specifies to generate a technology library with the default PGV name `techonly` and prefix `ab`:

  ```
  set_pg_library_mode -celltype techonly ...
  write_pg_library -out_dir ./ -library_prefix ab
  ```

  The generated library name will be `ab_techonly.cl`

- The following command specifies to generate a macro library with the default PGV name `<cell_name>` and prefix `ab`:

  ```
  set_pg_library_mode -celltype macros ...
  write_pg_library -out_dir ./ -library_prefix ab
  ```

  The generated library name will be `ab_<cell_name>.cl`

write_tech_pg_lib

```
write_tech_pg_lib
[-help]
[-decap_cells cell_list]
[-default_power_voltage value]
[-esd_cells cell_list]
[-filler_cells cell_list]
[-keep {true | false}]
[-lef_layer_map filename]
[-output_dir dir_name]
[-power_switch_file filename]
```

-tech_file *filename*
[-tech_only {true | false}]

Generates technology power-grid library on the fly. The generated tech-PGV will be deleted after the session is over. If this command is specified, the Power Analysis and Rail Analysis engines will automatically use the on the fly generated tech-PGV. The purpose of generating the tech-only PGV on the fly is that you do not need to keep a track of the tech-only PGVs. Thus, it ensures that the tech-only PGV is generated from the correct Quantus technology file.

The tech-only PGV library will be generated under the working directory and will be called by Power/Rail Analysis on the fly, as shown in the following log file snippet:

```
Begin Loading PGV Libraries for Power Calculation
.//pgv/techonly.cl
Ended Loading PGV Libraries for Power Calculation:
(cpu:0:00:00,real=0:00:00,mem(process/total/peak)=275.25MB/2585.14MB/1194.52MB)

Begin Processing Cell Libraries for Rail Analysis

Merging libraries:
.//pgv/techonly.cl
../../LIBS/voltus_pgv/macro_pgv/macros_sram.cl
../../LIBS/voltus_pgv/stdcells_pgv/stdcells.cl

Output library: ./work/lib_merged.cl
```

Parameters

`-help`

Outputs the command usage.

`-decap_cells` *`cell_list`*

Default: None

Specifies the names of cells that have explicit decoupling capacitance for use in decap optimization. These cells are nearly equivalent to feedthrough cells, except that there is typically an active device between the power and ground nets to provide extra decoupling capacitance. Power-grid view library generation uses this list of cells to ensure that a capacitance value is associated with each cell. If no capacitance is associated with a cell, a warning message will be issued.

`-default_power_voltage` *`value`*

Default: 1.2

Specifies voltage for the power pins which are present in LEF.

`-esd_cells` *`cell_list`*

Default: None

Specifies a list of Electrostatic Discharge (ESD) cells. You can use this parameter to characterize ESD cells.

When you specify this parameter, the list of ESD cells are later used in the `analyze_esd_network` command while loading the power-grid view library.

`-filler_cells` *`cell_list`*

Default: None

List the names of the filler or feedthrough cells that have no GDS data. These cells can be swapped out for decaps during decap optimization.

`-keep {true | false}`

Default: `false`

Specifies that the generated library should be retained after the session completes.

`-lef_layer_map` *`filename`*

Default: None

Specifies a file that provides the mapping of layers between the LEF and technology file. This parameter is optional. Library Generation automatically generates the layermap file using the technology LEF (containing all layer and via definitions) and Quantus Tech (extraction library) files, therefore, a LEF layermap file is not a mandatory input. The automatically generated layermap file is located in the PGV temp directory.

The following is the format and sample content of the auto-generated file (`lefdefLayerMap_AutoGenerated.txt`):

```
Layer_Type    tech_layername    lefdef    lefdef_layername
via               VIA_9         lefdef      VIA9
metal             METAL_1       lefdef      METAL1
```

You can specify this parameter if you want to provide your own layermap file.

`-output_dir` *`dir_name`*

Default: current working directory

Specifies the name of the output directory in which tech-only power-grid library is created.

`-power_switch_file` *`filename`*

Default: None

Specifies a file with the information that technology library generation needs to characterize a powergate cell. Specifies the cell name, unswitched power pin, switched power pin, on resistance in ohms, saturation current in milliamps, and leakage current in milliamps.

File Format:

```
cell supply switched Ron Idsat Ileak
```

Examples:

The following is an example of multiple footer powergate:

```
FOOTBUFVHSV16 VSSG VSS 63 3.167 0.000128252
FOOTBUFVHSV32 VSSG VSS 63 3.167 0.000128252
FOOTBUFVHSV64 VSSG VSS 63 3.167 0.000128252
FOOTBUFVHSV8 VSSG VSS 63 3.167 0.000128252
```

`-tech_file` *`filename`*

Default: None

(**Mandatory**) Specifies the name of the technology file that will be used for extraction.

`-tech_only {true | false}`

Default: `false`

Specifies to generate the technology PGV without any cells.

Examples

- The following command generates a tech-only power-grid library:

```
write_tech_pg_lib -tech_file $tech_path/qrcTechFile -filler_cells *FILL* -
decap_cells *DCAP* -default_power_voltage 0.9
```

write_xpgv

```
write_xpgv
-cell cell_name
[-inst_list inst1 inst2 .. instn ]
[-libgen_include file_name]
[-multi_mode_config config_file]
[-output_dir directory_name]
[-rail_include file_name]
[-reuse_block_state_directory directory_name]
-state_directory directory_name
```

Creates power-grid view (xPGV) models for large digital blocks.The block xPGV can be generated either after the signoff dynamic IR drop analysis or after the dynamic Early Rail Analysis (ERA) of the block. In both cases, the xPGV is generated using the R-C grid of the block and currents captured at the via layer during the analysis. These block-level xPGVs can then be instantiated during the next level of the design or the top-level full-chip analysis. The next level rail analysis will run faster with the block-level xPGVs because these runs use the power-grid that is modeled inside the xPGV.

Parameters

`-help`	Outputs the command usage.
`-cell cell_name`	
	Default: None (**Mandatory**) Specifies the block name for which xPGV model is generated.
`-inst_list inst1 inst2 .. instn`	
	Default: None Specifies the block instances for which xPGV model is generated.
`-libgen_include file_name`	

Default: None

Specifies a file that includes additional library generation options that will be run for xPGV model generation.

`-multi_mode_config` *`config_file`*

Default: None

Specifies to generate a single xPGV model with different functional modes. Using this parameter, you can specify a file that includes the different modes and the mapping state directory for each mode. When `-multi_mode_config` is specified, the block-level currents are saved for multiple voltages and frequency (MvMf) per each functional mode in a single PGV.

During xPGV instantiation, the different modes are automatically selected based on the power domain to which each mode is connected.

The format of the *`config_file`* file is:

<modeName1> <stateDir_1>
<modeName2> <stateDir_2>
…
<modeNameN> <stateDir_N>

Before generating MvMf-xPGV, you need to perform multiple rail analysis runs for different corners to generate different state directories (stateDir_*) to be included in the configuration file.

`-output_dir` *`directory_name`*

Default: current working directory

Specifies the name of the output directory in which the power-grid library report is created.

`-rail_include` *`file_name`*

Default: None

Specifies a file that includes additional rail analysis options that will be run for xPGV model generation.

`-reuse_block_state_directory` *`directory_name`*

	Default: None Specifies an existing rail analysis state directory to skip block extraction. The software assumes that the design data (DEFs and GDS) and power-grid libraries are not changed, and will generate xPGV using the extraction data inside the specified state directory. If the state directory does not contain the necessary design and power-grid database files, it will issue an error message and provide information on the missing files. The `-reuse_block_state_directory` parameter is used if the entire design that is analyzed in the rail run should be used to create the xPGV. However, this parameter must not be used in case you require to generate xPGVs for each instance (sub-blocks) of a block. That is, for generating xPGVs for the sub-blocks B and C of the block A (rail analysis was performed on block A), you need to re-extract the block-level grids of B and C as part of xPGV generation for accurate current calculation. In this case, `-reuse_block_state_directory` must not be specified. For net based analysis, you should specify net-based state directory, as shown below: • `write_xpgv -reuse_block_state_directory` *`VDD_25C_avg_1`* For domain based analysis, you should specify the domain's state directory, as shown below: • `write_xpgv -reuse_block_state_directory` *`domain_25C_dynamic_1`* A new state directory will be generated for the new analysis with soft links to the old state directory files that were reused.
`-state_directory` *`directory_name`*	
	Default: None (**Mandatory**) Specifies the path to the rail analysis state directory.

Examples

- The following command creates a xPGV library `hier_block.cl`:
  ```
  write_xpgv -state_directory dynamic_rail/core_125C_dynamic_1/ -cell hier_block -output_dir ../xpgv/
  ```
- The following command creates an xPGV library `super_filter_A` with multiple functional modes specified in the `mode.config` file:
  ```
  write_xpgv -cell super_filter_A -multi_mode_config mode.config -output_dir ./mode_mvmf -rail_include rail.inc
  ```
  The following is a snippet from the mode.config file:
  ```
  label1 dynamic_rail/core_25C_dynamic_1
  lable2 dynamic_rail/core_25C_dynamic_1
  ```

Related Topics

- "Power-Grid Library Generation" in the *Voltus User Guide*

16

Placement Commands and Attributes

- add_fillers Category Attributes
- gui_clear_scan_chains
- gui_clear_spare_cells
- gui_show_scan_chains
- gui_show_spare_cells
- write_physical_context_data
- read_physical_context_data

add_fillers Category Attributes

The root attributes in the add_fillers category are:

- add_fillers_cells
- add_fillers_check_trim_rule
- add_filler_enable_restore_filler_flow
- add_fillers_prefix

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_fillers` category, use the following command:

```
get_db -category add_fillers
```

add_fillers_cells

```
add_fillers_cells {{list-of-cells1} {list-of-cells2} ...}
```

Default: `""`

Specifies the filler cells to add with add_fillers. Enclose the filler cell names in quotation marks (") or curly braces ({ }). You can specify multiple cell lists at one time, using curly braces ({ }) to separate each one.

add_fillers_check_trim_rule

```
add_fillers_check_trim_rule {true | false}
```

Default: `false`

Data_type: bool, read/write

Controls trim rule checking during the filler insertion. When set to true, the trim rule is checked, and runtime increases.

add_filler_enable_restore_filler_flow

```
add_filler_enable_restore_filler_flow {true | false}
```

Default: `true`

Data_type: bool, read/write

When this option is set to `false`, fillers are not saved in the memory (which is default i.e the save and restore filler insertion flow is default ON).

add_fillers_prefix

```
add_fillers_prefix prefix
```

Default: `""`

Specifies the prefix for the added filler cells.

gui_clear_scan_chains

```
gui_clear_scan_chains
[-help]
[scanPartitionName]
```

Removes scan chains from the display.

Parameters

scanPartitionName	Specifies the scan partition name to remove from the display.

gui_clear_spare_cells

`gui_clear_spare_cells`

Removes spare cell instances from the display.

Parameters

None

gui_show_scan_chains

```
gui_show_scan_chains
[-help]
[scan_chain_name]
```

Displays a scan chain. You must load the scan information, place the design, and view the design in the Placement view. Use this command after scan chains are traced.

Parameters

scan_chain_name	Specifies the scan chain. *Default:* If you do not specify this parameter, the software displays all scan chains.

Examples

- The following command displays all scan chains in the design:
  ```
  gui_show_scan_chains
  ```
- The following command displays the scan chain whose name is `chain0`.
  ```
  gui_show_scan_chains chain0
  ```

gui_show_spare_cells

```
gui_show_spare_cells
```

Displays the results of placing the spare instances. You must specify the spare cell types or spare instances before placing the design.

Parameters

None

write_physical_context_data

```
write_physical_context_data
[-help]
fileName
[-designation name]
[-type technologyName]
```

Writes the physical context data needed for timing analysis. The data that is being written is associated with each instance in the design, or module.

The command can be used, both in Innovus and Tempus, if Tempus is in physical-mode and has the physical data in DB to use for writing out the LDE data.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_physical_context_data` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_physical_context_data`.
`-designation` *`name`*	Specifies the Rx layer width map from micron to enum -um:nsw{:um:nsw}.
`fileName`	Specifies the name of the output file.
`-type` *`technologyName`*	Specifies the technology name.

Related Information

- `read_physical_context_data`

read_physical_context_data

```
read_physical_context_data
[-help]
fileName
[-clear]
[-type technologyName]
```

Reads the physical context data needed for timing analysis. The data that is being read is associated with each instance in the design, or module.

The command can be used in both Innovus and Tempus to restore the state of LDE as long as there is no netlist change. To read the data, there need not be physical information in the DB, as the data can be annotated directly to instances - to be used by STA.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_physical_context_data` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_physical_context_data`.
`-clear`	Clears physical context data before reading.
`fileName`	Specifies the names of the input files.
`-type` *`technologyName`*	Specifies the technology name.

Related Information

- `write_physical_context_data`

17

Rail Analysis Commands

- add_what_if_shapes
- analyze_jitter
- connect_die_package
- convert_stream_to_def
- extract_package
- generate_esd_rlrp_report
- get_ir_diagnostics
- get_voltus_diagnostics_mode
- map_dies
- report_differential_voltage
- report_irdrop_regions
- report_esd_network
- report_esd_voltage
- report_joule_heat
- report_noise_margin
- report_package
- report_rail
- report_resistance
- report_self_heat
- report_signal_resistance
- scale_what_if_capacitance
- scale_what_if_current
- scale_what_if_resistance
- set_advanced_package_options

- set_advanced_rail_options
- set_die_model
- set_dynamic_rail_simulation
- set_multi_die_analysis_mode
- set_net_group
- set_off_chip_package_trace
- set_package
- set_pg_nets
- set_power_data
- set_power_pads
- set_rail_analysis_config
- set_rail_analysis_domain
- set_self_heat_analysis_mode
- set_voltage_regulator_module
- set_voltus_diagnostics_mode
- view_dynamic_movie
- view_dynamic_waveform
- view_package_results
- write_current_region
- write_die_model
- write_hier_view
- write_power_pads
- write_power_up_report

add_what_if_shapes

```
add_what_if_shapes
[-help]
[-add  | -select | -deselect |
 -delete | -list  | -write filename | -read filename]
[-add -method {auto | exact}]
[-add -width width]
[-area {x1 y1 x2 y2}]
[-display]
[-direction {hor | ver}]
[-format {txt def}]
[-layer layerName]
[-nets {net1 net2 net3…}]
[-objects {selected | all} ]
[-offset {X Y}]
[-pitch pitch]
[-remove_existing_wires]
[-spacing value]
[-type {wire | via}]
[-write filename -append ]
```

Specifies to add, select, deselect, delete, list, and display the what if wires/vias to a design.

Parameters

`-help`	
	Outputs the command usage.
`-add`	
	Specifies to add what-if wires/vias.
`-append`	
	Specifies to append all what-if wires/vias to the existing file specified with the `-save filename` parameter.
`-area {x1 y1 x2 y2}`	
	Specifies the rectangular region in which the what if wires/vias are to be generated.

`-delete`
Specifies to delete all or selected what-if shapes by constraint `-net/-layer/-type`. You can use this parameter in conjunction with the `-objects` parameter.
`-deselect`
Specifies to deselect all what-if shapes by constraints `-net/-layer/-type`. Use this parameter in conjunction with `-area` to deselect what-if shapes overlapping with the specified area.
`-direction {hor | ver}`
Controls the direction of the wires added.
`-display`
Invokes the GUI to show what-if shapes and zooms to the selected shapes. It works only in the `-win` mode.
`-format {txt def}`
Specifies the output file format. The `def` format specifies to create incremental DEF to load into GUI using `read_def -top`. The default output format is `txt`.
`-layer` *`layerName`*
Specifies the LEF layer name for wires, or layer pair for via to be added.
`-list`
Specifies to list all or selected what-if shapes by constraint `-net/-layer/-type` . You can use this parameter in conjunction with the `-objects` parameter.
`-method {auto | exact}`
Specifies the method used to add vias: • `auto`: When area and layer pair are specified, the tool automatically adds vias on all intersecting points on the specified layer pairs in the given area. • `exact`: When an area and layer pair are specified, the tool adds a via of exact size of the specified area between the specified layer pair.
`-nets {`*`net1 net2 net3`*`…}`
Specifies the name of the nets for which what-if wires/vias is to be created. The what-if engine will create a shape based on the net pattern.

`-objects {selected | all}`
Specify `-objects` in conjunction with the `-delete` or `-list` parameters to delete or list the selected what-if shapes. You can also delete or list all what-if shapes. The default value is `all`. For example, you can use the following command to delete the selected objects: `add_what_if_shapes -delete -objects selected`
`-offset {X Y}`
Specifies to create metal/via from the area + offset. This parameter is useful when you run multiple what-if shape creation commands and want to keep the same area but have different offset.
`-pitch` *`pitch`*
Specifies the distance between the centre line of two adjacent what-if wires (pitch) to be drawn in the specified `-area` in the preferred direction of the specified `-layer`. The direction of created wires can also be controlled using the `-direction` parameter. If this parameter is not provided, a single rectangular shape of size {llx,lly,urx,ury} is created.
`-read` *`filename`*
Specifies to load the saved what-if shape file.
`-remove_existing_wires`
Specifies to remove a wire and connected vias from the specified layer and area before adding what-if shapes.
`-select`
Specifies to select all what-if shapes by constrains `-net/-layer/-type`. Use this parameter in conjunction with `-area` to select what-if shapes overlapping with the specified area.
`-spacing` *`value`*
Specify the spacing between each net. Use this parameter to control the spacing between each net. The `-pitch` value specified should be greater than number of nets * (width+spacing).
`-type {wire | via}`
Specifies what kind of shape is to be created.
`-width` *`value1 value2 ..valuen`*
Specifies the width of what-if wires. You can use the `-width` parameter to specify a separate width value for each net.

`-write` *filename*

Specifies to save all the in-memory what-if wires/vias to the specified file. The format of the file is:

```
type wire net netName area llx lly urx ury layer layer1
type via net netName area llx lly urx ury layer layer1 layer2
```

Following is an example of the file containing one wire and one via shape:

```
type wire net VDD area 191.000000 763.000000 207.000000 772.000000 layer METTOP
type via net VDD area 513.000000 497.700000 517.000000 499.100000 layer METG1 ME
```

Examples

- The following command adds a what-if wire for the net `VDD` and on the LEF layer `METTOP`:

```
add_what_if_shapes -type wire -net VDD -area {391.000000 663.000000 407.000000
672.000000} -layer METTOP -add
add_what_if_shapes -type wire -net VDD -area {191.000000 663.000000 207.000000
772.000000} -layer METTOP -add -pitch 9 -width 0.4
```

- The following command creates a what-if via for the net `VDD` and on the LEF layers `METG1 MET5`:

```
add_what_if_shapes -type via -net VDD -area {513.000000 497.700000 517.000000 499.100000}
-layer {METG1 MET5} -add -method exact
add_what_if_shapes -type via -area {0 0 1000 1000} -method auto -layer {METTOP METG1} -
add -net VDD
```

- The following command specifies to list selected what-if shapes:

```
add_what_if_shapes -list -objects selected
```

- The following command specifies to select and highlight all wires in the win mode:

```
add_what_if_shapes -select
```

- The following command specifies to select and highlight all what-if vias using area:

```
add_what_if_shapes -type via -area {500 490 530 500} -select -net VDD
```

- The following command specifies to save what-if shapes into the file `whatif.shapes`:

```
add_what_if_shapes -write whatif.shapes
```

- The following command specifies to delete all what-if shapes in memory:

```
add_what_if_shapes -delete -objects all
```

- In the following example, for nets "`VDD VDD1 VDD2 VSS`", three spacing is possible (between VDD and VDD1, between VDD1 and VDD2, and between VDD2 and VSS), that is, spacing is "0.5 0.6 0.7" and width of what-if wires for all nets is "0.5":

```
add_what_if_shapes -net {VDD VDD1 VDD2 VSS} -pitch 4 -width 0.5 -spacing {0.5 0.6 0.7} -
area {0 0 12 12} -type wire -add -layer M10 -direction hor
```

- The following command will remove all wires on `M7` in the `0,0` to `100,100` region as well as any vias connected to the deleted wires:

```
add_what_if_shapes -remove_existing_wires -layer {M7} -area { 0 0 100 100 }
```

analyze_jitter

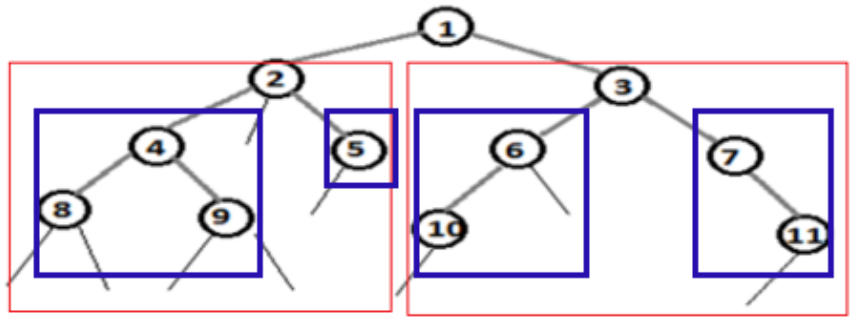
```
analyze_jitter
[-help]
[-eiv_end_time time]
[-eiv_file file_path]
[-eiv_start_time time]
[-eiv_power_ground net_names]
[-from pin_name]
[-from_fall pin_name]
[-from_rise pin_name]
[-level {true | false}]
[-levels_combine value]
[-lsf_job_string job]
[-max_spectre_lic value]
[-model_file filename]
[-out_dir directory_name]
[-run_simulation]
[-skip_stage_fanout value1 value2]
[-spectre spectre_path]
[-spectre_cmd_options options]
[-subckt_file file_path]
[-through through_points]
[-to pin_name]
[ [ > | >> ] ]
```

Analyzes jitter for the user-defined clocks and calculates clock jitter for all the clock end points or user-defined end points. The `analyze_jitter` command profiles the cycle-to-cycle EIV values from rail analysis, applies the two worst-case cycle-to-cycle EIV values for two simulations, measures the delta delays for each of the clock tree cells, and computes the effective jitter at the end point.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `analyze_jitter` parameter. For a detailed description of the command and all of its parameters, use the man `analyze_jitter` command.
`-eiv_end_time time`	

	Specifies the end time of the EIV timing window.
`-eiv_file` *file_path*	
	Specifies the path to the effective instance voltage files for accounting IR Drop in jitter computation.
`-eiv_start_time` *time*	
	Specifies the start time of the EIV timing window.
`-eiv_power_ground` *net_names*	
	Specifies the power and ground net name for each EIV file provided in the `-eiv_file` parameter.
`-from` *pin_name*	
	Creates the spice deck with mesh net starting from the specified from pin.
`-from_fall` *pin_name*	
	Creates the spice deck with mesh net starting from the falling transition of the specified from pin.
`-from_rise` *pin_name*	
	Creates the spice deck with mesh net starting from the rising transition of the specified from pin.
`-level {true | false}`	
	When set to `false` (default mode), the spice deck is generated only for traced paths starting from the `-from` pin and ending at the `-to` pin. It does not traverse through the fanout branches that do not lead to the `-to` specified pin. When set to `true`, traversal occurs through the whole fanout cone of the `-from` pin till it hits a clock pin of a flop, or any stop point specified with the `-to` parameter.
`-levels_combine` *value*	

	Specifies the number of levels above the leaf node from which sub trees are to be created. This parameter specifies the level at which the sub trees are to be combined. The default value is 0, which indicates that a separate tree will be created for each leaf node. After determining the nodes to be combined together, the tree is broken into smaller trees, and jitter analysis on these smaller trees can be done in parallel.
	Let us consider the following diagram:
	 
	
	Here,
	For levels_combine = 0:
	`T1: 1, 2, 4,8`
	`T2: 1,2,4,9`
	`T3: 1,2,5`
	`T4: 1,3,6,10`
	`T5: 1,3,7,11`
	For levels_combine (blue box) = 1:
	`T1: 1,2,4,8,9`
	`T2: 1,2,5`
	`T3: 1,3,6,10`
	`T4: 1,3,7,10`
	For levels_combine (red box) = 2:
	`T1: 1,2,4,5,8,9`
	`T2: 1,3,6,7,10,11`

`-lsf_job_string` *job*	
	Specifies a handle to the farm job for launching the simulation.
`-max_spectre_lic` *value*	
	Specifies the number of Spectre license available to run multiple Spectre instances simultaneously.

`-model_file` *`filename`*	
	Specifies the model file to be included in generated Spice.
`-out_dir` *`directory_name`*	
	Specifies the name of the output directory that will contain the generated jitter analysis reports: • `summary.jitter` • `clock_branch1.jitter` • `clock_branch2.jitter`
`-run_simulation`	
	Enables path simulation in addition to generating the spice deck.
`-skip_stage_fanout` *`value1 value2`*	
	Takes two positive integers in the format "N1 N2". When specified, scans up to the N1 stage in the fanin cone of each end point, and skips the segments where a stage is found to have fanout greater than N2.
`-spectre` *`spectre_path`*	
	Specifies the location of the Spectre version that should be used for simulation. If the path is not specified with this option, the software will check if you have the Spectre location defined in your path.
`-spectre_cmd_options` *`options`*	
	Specifies the command line options for Spectre.
`-subckt_file` *`file_path`*	
	Specifies the path of the Spice sub-circuit file that contains the Spice information. This information is necessary to get accurate Spice information.
`-through` *`through_points`*	
	Creates the spice deck with mesh net going by the -through points.
`-to` *`pin_name`*	
	Creates the Spice deck with mesh net ending at the specified to pins.
`>` *`filename`*	

	Redirects the `-run_simulation` output to the specified file.
`>> filename`	
	Appends the `-run_simulation` output to the specified file.

Examples

- The following command creates jitter reports for the clock path starting from the `i_REC_CLK` and places the results directories in the `jitter_output` directory:

```
analyze_jitter -eiv_power_ground {VDD VSS} -eiv_file
/IRDROP/PD_25C_dynamic_1/Reports/VDD_VSS.win_avg.rpt -model_file /../include.ss -
subckt_file /../worst.spi -spectre
/icd/flow/MMSIM/MMSIM161/ISR9/lnx86/tools.lnx86/bin/spectre -level true -out_dir
jitter_output -from i_REC_CLK -spectre_cmd_options {+aps +spice +mt=8  +postlayout
+dcopt  } -lsf_job_string {/grid/sfi/farm/bin/bsub -q rnd -P VOLTUS:16.1:PV:regTest -W
140:00 -R "OSNAME==Linux && OSBIT==64 && (OSREL==EE60||OSREL==EE50)" -Is } -
run_simulation -levels_combine 2 -max_spectre_lic 100
```

Related Topics

- "Static Power, IRdrop and EM Analysis" and "Dynamic Power and IRDrop Analysis" in the *Voltus User Guide*

connect_die_package

```
connect_die_package
[-help]
[-output_dir directory]
-package_model_file filename
{-die_mcp_header_file filename | -net_pad_file_pair_list {{<net1> <pad file1>} {<net2> <pad file2>}...}}
```

The command performs the following functions:

- launch the *MCP Editor* GUI to make manual connections between package and die pins.
- map the voltage source files with the package model to check and correct mapping information before rail analysis.
- generate a mapping file that can be used in previous versions of the software in case there is no mapping between die and package pins.

The command has two types of inputs:

- Voltage source files in the xy format and a package model file with the MCP header.
- A circuit file with the MCP header and a package model file with the MCP header.

The output will be the same for both types of inputs:

- A ckt mapping file containing only the MCP header, with the name `pkg_model_base_name + "_map"+suffix`.
- The file named '`un_mapped_bumps.txt`' will contain all the bumps which cannot be mapped.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-die_mcp_header_file` *filename*	
	Specifies a circuit file with the MCP header containing the die abstract.

`-net_pad_file_pair_list {{<net1> <pad file1>} {<net2> <pad file2>}...}`	
	Specifies the voltage source files mapping to each net.
`-output_dir directory`	
	Specifies the output directory containing the mapping results.
`-package_model_file filename`	
	Specifies the name of the package model file.

Examples

- The following command specifies a circuit file with MCP header and a package model file with the MCP header:
  ```
  connect_die_package -die_mcp_header_file super_filter_mcp_ploc.ckt -package_model_file
  C1_Voltus_4L_C1_Voltus_4L_Final_PinBaseSPICE.ckt -output_dir out_ploc
  ```
- The following command specifies voltage source files in the XY format and a package model file with the MCP header:
  ```
  connect_die_package -output_dir ./out_pad -net_pad_file_pair_list {{VDD VDD_AO_srcs.pp}
  {VDD VDD_external_srcs.pp} {VSS VSS_srcs.pp}} -package_model_file
  C1_Voltus_4L_C1_Voltus_4L_Final_PinBaseSPICE.ckt
  ```

Related Topics

- "Package Analysis" in the *Voltus User Guide*

convert_stream_to_def

```
convert_stream_to_def
[-help]
[-black_box_cell_file filename]
[-esd_cell_bbox_file filename]
[-esd_marker_layer_file filename]
[-extract_signal_net_logic_info_only]
[-ignore_stream_inst_name]
[-inst_name_file filename]
[-inst_name_prefix prefix]
-net_location_file filename
[-output_components {ALL | NONE | CONN}]
-output_def_file filename
[-output_lef_macros filename]
-power_grid_library libraryname
[-print_complete_short_path]
[-spef_file filename]
-stream_file filename
-stream_layer_map filename
-top_cell cellname
[-use_stream_label {ALL | TOP | <tech_layer_name> | <cell_name>:<tech_layer_name> |
<cell_name>:ALL}]
[-verilog_file filename]
[-zero_pin_size]
```

A GDS2DEF utility that converts a given GDSII file to a DEF file for power/rail signoff analysis. The utility can convert both cell-level GDS and full-chip level GDS to DEF. The generated DEF contains all the geometries for the P/G routing, the specified P/G pin information, component and logical connectivity information.

Parameters

-help	Outputs the command usage.
-black_box_cell_file *filename*	

Specifies the name of a file containing a list of black box cells that the utility is to ignore. You can specify both exact string matches and wildcard searches in the specified file.

For exact string matches, each cell name must reside on a separate line in the file, as shown below:

```
CellA
CellB
```

The wildcard search feature allows you to search cell names that contain wildcard characters, such as square brackets and underscore. When you want to perform wildcard search, type the keyword `wildcard` first, followed by the cell name. For example, if you want to ignore cell names which contains CLK, add one line `wildcard .*CLK.*`.

`-esd_marker_layer_file` *`filename`*

Specifies a file that includes a list of ESD cell/instance names and their corresponding GDS marker layers that represent the bounding box of the ESD cells. The format of the file is:
`<cell_name> <marker_layer> [top_layer] [bottom_layer]`

Here,

- `cell_name` is the ESD cell/instance name
- `marker_layer` could contain multiple GDS layers, separated by ";"
- The optional `top_layer` and `bottom_layer` specifies the top and bottom technology layers of the ESD cell pin shape. You can use these optional arguments to specify the top and bottom technology layers of the ESD cell pin shape. This provides you with the flexibility to specify the lowest layers to be used for creating pins in the LEF. In the following example, only the shapes between M3 and M1 layers are ESD cell pin shapes:

Marker file example:

```
ESD_diode 168;255:3 M3 M1
ESD_clamp 122 M3 M1
```

`-esd_cell_bbox_file` *`filename`*

Specifies a file that includes the bounding box for each ESD cell. The format of the file is:
`<cell_name> <x1 y1 x2 y2> [top_layer] [bottom_layer]`

Here,

- `cell_name` is the ESD cell/instance name
- `x1 y1 x2 y2` are the bounding box coordinates
- The optional `top_layer` and `bottom_layer` specifies the top and bottom technology layers of the ESD cell pin shape. You can use these optional arguments to specify the top and bottom technology layers of the ESD cell pin shape. This provides you with the flexibility to specify the lowest layers to be used for creating pins in the LEF. In the following example, only the shapes between AP and M11 layers are ESD cell pin shapes:

Bbox file example:

```
ESD_diode_0 1281296 860923 1296972 895445 AP M11
ESD_diode_1 1281296 898651 1296972 933173 AP M11
```

`-extract_signal_net_logic_info_only`

Specifies to print only the logical connectivity for the signal nets in the `NETS` section of the output DEF file. If this parameter is not specified, the software prints both the logical connectivity and routing shapes for the signal nets in the `SPECIALNETS` section of the DEF file.

`-stream_file` *filename*

Mandatory parameter. Specifies the name of the input GDS file. The top cell name in the GDSII file is the design name in the generated DEF.

`-stream_layer_map` *filename*

Mandatory parameter. The layer mapping file between GDS and tech layers. All the layers defined in a given mapping file is extracted. The layers not defined in the mapping file are ignored.

The format of the layer mapping file is:

`<layer_type> <tech layer name> <gds> <gds layer#> [gds layer data type] [PORT] [TEXT_ONLY] <mask keyword> [dummy]`

The format of the layer mapping file to identify the die area in the output DEF is:

`<layer> **<prBoundary>** <gds> <gds layer#> [gds layer data type]`

Format Description:

`<layer_type>` specifies the type of library layer. The various layer types are: `via`, `metal`, and `layer`.

<tech layer name> specifies the name of the library layer to map to the GDSII layer number. The library layers are technology layers which can be dumped using the `Techgen -techinfo` command.

<gds> specifies the keyword for the GDS layer mapping.

<gds layer#> specifies the layer number of the GDSII layer.

`[gds layer data type]` specifies the optional number of the subclass for the GDSII layer. If you do not specify a datatype, the software uses all datatypes for that GDSII layer. If GDS datatype is not defined, the software considers it as a regular layout routing layer where all the shapes will be extracted.

The GDS layer may have additional information/attribute assigned using the following keywords:

- `PORT`: Converts text labels in this layer to ports.
- `TEXT_ONLY`: Includes only the text in the named layer and ignores all geometries.

<mask keyword> keyword used to define the back-end metal layers with color masks (MASK A and Mask B).

`[dummy]` indicates that the layer is a metal fill layer. The `convert_stream_to_def` utility identifies these metal fill shapes and writes them to the `FILLS` section in the output DEF.

`<prBoundary>` specifies that the shapes on the `prBoundary` (Place-and-Route Boundaries) layer are used to determine the die area in the output DEF. This keyword can be used if you want to have the required die area in the output DEF file. If the `prBoundary` keyword is not specified, the die area is determined by the routing shapes specified in the GDS file, cell size, and the pin shapes of all the layers.

Example1:

```
metal METAL1 gds 15
via VIA1 gds 16
```

Example2:

The following is a sample layermap file with the mask keywords (`mask_a` and `mask_b`):

```
metal METAL1 gds 31 250
metal METAL1 gds 31 255 mask_a
metal METAL1 gds 31 256 mask_b
```

Here, the mask and main layers are defined using the same GDS layer number 31, but uses a different GDS data type (250, 255, and 256).

Example3:

The following is a sample layermap file with the `dummy` keyword:

```
metal  METAL5 gds 35 367 dummy
```

Here, the layer with the GDS layer ID `35` and data type `367` contains the metal fill shapes only.

Example4:
The following is a sample layermap file with the `prBoundary` keyword:

```
layer   prBoundary      gds     62 21
```

`-ignore_stream_inst_name`

Specifies to ignore the instance names from the specified GDS file, and the software will then automatically assign instance names to all the instances in the output DEF and Verilog files. This parameter can be used if instance names in the GDS file cannot be used, primarily in cases, such as duplicate instance names.

`-inst_name_file` *`filename`*

Specifies the file name containing instance names for the generated DEF in the following format:

```
<x coord> <y coord> <instance name>
```

The x y coordinates are in the unit of micron. The priority of getting an instance name is:

1. instance name file
2. GDS label
3. internally assigned name

The `-inst_name_file` parameter allows to provide user-specified instance names. The `convert_stream_to_def` utility reads the GDS label to decide the instance name. If there are no labels, the software internally assigns instance names.

`-inst_name_prefix` *`prefix`*

Specifies a prefix for all the instance names in the output DEF and Verilog files.

`-net_location_file` *`filename`*

Mandatory parameter. Specifies the name of the net location file containing a list of pin locations for nets that needed to be considered. All the geometries of the P/G nets as well as the instance pins connected to the specified P/G nets are included in the SPECIALNETS section of DEF. The instances connected to the P/G nets are included in the COMPONENTS section of DEF.

The format of the net location file is:

<net name> <x loc of pin> <y loc of pin> <layer> <POWER|GROUND|SIGNAL> [*pin name*]

Example:

```
VDD 60 73 METAL3 POWER
VSS 75 40 METAL2 GROUND
```

Here,

- The x/y location in the file has the unit of micron.
- The layer name is the tech layer name.
- The type of a given net can only be POWER, GROUND, or SIGNAL. If the net type is marked as SIGNAL, the net will be traced from the GDS file and included in the SPECIALNETS section with "USE SIGNAL" in the output DEF file. In addition, the signal pin connectivity is specified for the net.
- The pin name field is optional. If there is no pin name field, the pin name will be same as the net name.

If you do not provide pin locations, the utility uses the GDS text labels in either the top-level cell or cells throughout the hierarchy to get the location. In such a case, the format of net location file is:

<net name> <POWER|GROUND|SIGNAL>

Example:

```
VDD POWER
VSS GROUND
```

`-output_components {ALL | NONE | CONN}`

Specifies the components to be printed in the DEF file COMPONENTS section during GDS to DEF conversion. The default is ALL.

- ALL: Specifies to output all the instances in the DEF Components section.
- NONE: Specifies not to print instances.
- CONN: Specifies to output instances with logical connectivity only.

`-output_def_file` *filename*

	Mandatory parameter. Specifies the name of the output DEF file. If the given DEF name is specified with "`.gz`", the software will gzip the DEF file to reduce the file size.
`-output_lef_macros` *`filename`*	
	Specifies the name of the file containing a list of macros for which LEF files are to be generated. One LEF file is generated for each macro. The generated LEF pin shapes are all shapes that connect to top-level nets on all metal/via layer shapes. The cell boundary of the LEF macro is derived by the bounding box of all shapes within the GDS cell.
`-power_grid_library` *`libraryname`*	
	Mandatory parameter. Specifies the name of the merged power-grid library (.cl). You can specify only a single .cl library.
`-print_complete_short_path`	
	Specifies to print the complete shorting path when there is a short between two nets. When this parameter is specified, the entire shorting path is assigned to one of the shorted nets and the path is printed in the generated DEF file. The shapes of the shorting path will be assigned to the net that is specified first in the net location file. For example, if a tracing point of VDD is shorted with a tracing point of VSS, the software honors the net that is defined first in the net location file. If this parameter is not specified, the software assigns the shapes of the path to either of the shorted nets by default.
`-spef_file` *`filename`*	

	Specifies the name of the SPEF file that can be used to include the signal nets in the generated DEF file. The input SPEF file is required to have layer annotation in the `RES` statements to indicate the pin node layers. For the signal nets, both the `*P` and `*I` nodes are used as the starting tracing points, and the SPEF file pins `(*P)` are honored as the top-level pins and are reported in the `PINS` section in the output DEF file. For the PG nets, the `*P` nodes and the user-defined locations in the net location file are used as the starting tracing points. The SPEF file connections (`*I` statements) are honored when generating the DEF logic connectivity for both the signal and PG nets. That is, the `convert_stream_to_def` utility relies on the SPEF file for logic connection if a SPEF file is specified. The instance and pin names are from the SPEF `*I` statements. The output DEF file includes all the signal net pins from `*P` in the `PINS` section, the signal routing shapes in the `NETS` section, and the PG net routing shapes in the `SPECIALNETS` section. In addition to the SPEF file, you need to specify the following parameters for including the signal nets in the DEF file: • `-stream_file` • `-net_location_file` • `-power_grid_library` • `-stream_layer_map` • `-output_def_file` • `-top_cell` **Notes**: • The `-net_location_file` parameter is used to identify the power and ground nets, and therefore the remaining nets are considered as signal nets. • The `-power_grid_library` specifies the name of the power-grid library that includes the list of standard cells or macros that should be consistent with the SPEF file. That is, the cell of the instances in the SPEF file should be defined in the PGV.
`-top_cell` *cellname*	
	Mandatory parameter. Specifies the top cell name in the GDSII file.
`-use_stream_label {ALL | TOP | <`*tech_layer_name*`> | <`*cell_name*`>:<`*tech_layer_name*`> | <`*cell_name*`>:ALL>}`	

	Specifies the criteria for selecting text labels. • `ALL`: specifies to honor all the text labels from GDS. • `TOP`: specifies to honor just the top-level text labels. • *`<tech_layer_name>`*: specifies to honor all the text labels on the specified layer. • *`<cellname>`*`:`*`<tech_layer_name>`*: specifies to honor all the text labels of the specified GDS cell on the specified layer. • *`<cellname>`*`:ALL`: specifies to honor all the text labels of the specified GDS cell. The following example specifies to use the text labels of the cell "hierblock" on layer METAL10: `convert_stream_to_def -use_stream_label hierblock:METAL10`
`-verilog_file` *`filename`*	
	Specifies the name of the output verilog file. This parameter allows you to use the GUI to display the rail analysis results of the GDS-only flow.
`-zero_pin_size`	
	Specifies to set the pin shape to size zero "(0 0)(0 0)" in the output DEF file. You can use this parameter when the port shape is outside the drawn shape (metal geometry) at places where the port label is defined near the edge of the geometry.

Examples

- The following command converts the specified GDS file `new.gds` to a DEF file `top.def`.

```
convert_stream_to_def -net_location_file net.loc -stream_file new.gds -power_grid_library
../pv_library.cl -stream_layer_map lmap -top_cell cella -output_def_file top.def
```

- The following command converts the specified GDS file `chip.gds` to a DEF file `my.def`:

```
convert_stream_to_def -stream_file chip.gds.gz -power_grid_library
{pgv/techonly/techonly.cl   pgv/full.cl} \
                  -net_location_file netlocfile -stream_layer_map  layermap -
output_def_file my.def -top_cell chip -spef_file chip.spef.gz
```

  Here, the SPEF file `chip.spef` is specified to include the signal nets in the output DEF file.

- The following command specifies a marker layer file `esdcells` to enable ESD analysis using `convert_stream_to_def`:

```
convert_stream_to_def -stream_file gw16_500_usr_test.gds  -power_grid_library {
/../TECH/techonly.cl }  -net_location_file my.nlf -stream_layer_map  gds_layer.map \
    -output_def_file my.def  -top_cell gw16_500_usr -use_stream_label ALL -
esd_marker_layer_file esdcells
```

  `convert_stream_to_def` utility generates a LEF file for each ESD cell specified in the `esdcells` file to enable PGV generation. The output DEF includes the user-defined nets and the ESD instances based on the specified marker layer file.

Related Topics

- "GDS2DEF Utility" in the *Voltus User Guide*

extract_package

```
extract_package
[-help] <domain>
[-output_dir <directoryname>]
-package_board_name <bga_circuitname>
-package_die_name <die_circuitname>
[-reference_net <netname>]
[-spd_file filename]
-workspace <workspace_name>
```

Specifies to start package model extraction.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
domain	Specifies the rail analysis domain name.
`-output_dir <`*directoryname*`>`	
	By default, the package model will be created in the workspace. However, if you specify an output directory, then a copy of the result will be made in the new directory.
`-package_board_name <`*bga_circuitname*`>`	
	Defines the ball grid array (BGA) circuit name in the package model. It is the PCB component in the package model, which eventually connects to the PCB model.
`-package_die_name <`*die_circuitname*`>`	
	Defines the die circuit name in the package model. When specified, the software automatically creates a mapping between the die name in the package extraction tool and the die name in the software.
`-reference_net <`*netname*`>`	
	Specifies the ground net that is selected as reference net for pin-based model generation. This is the ground net that is being referred to by the power nets.
`-spd_file` *filename*	
	Specifies an existing Sigrity .spd file for package extraction. This parameter allows you to specify a .spd file instead of the workspace name (package model database) of the package extraction tool (XtractIM).
`-workspace <`*workspace_name*`>`	
	Specifies the workspace name (package model database) of the package extraction tool (XtractIM). The file name extension is `.ximx`.

Example

- The following example specifies the package models details and starts extraction:

```
extract_package ALL \
  -workspace ../rak_xim_pkg_model/C1_Voltus_4L.ximx \
  -package_board_name FPBGA144 \
  -package_die_name CD1 \
  -output_dir rak_xim_pkg_model_voltus_cpf \
  -reference_net VSS
```

generate_esd_rlrp_report

```
generate_esd_rlrp_report
[-help]
[-instance_pair_list {{inst1 inst2 [pin1 pin2]}+} | -instance_pair_list_file filename]
-net netname
[-number_threads value]
-state_directory directory
```

Reports the least-resistance path (RLRP) for the bump-to-clamp rule type in the ESD flow. This command helps to make debugging faster when a fail status is given for a bump-to-clamp pair. Using this command, you can:

- view the RLRP path for the selected bump-to-clamp pair in the GUI
- generate a point-to-point RLRP path report that displays the resistance between each node (resistor segment) pair along the RLRP path

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-instance_pair_list {{inst1 inst2 [pin1 pin2]}+}`	
	Specifies the instance pair list to enable effective resistance calculation between two instances in the design. *pin* is an optional argument. By default, the tool will use the instance pin associated with the net being analyzed. The *pin* argument can be used in special cases when two different pins of an instance are connected to the same net. *pin1* and *pin2* are pins of *inst2* respectively. If a pin name is not given, all pins associated to the specified instance are used.
`-instance_pair_list_file` *filename*	
	Specifies the name of the instance pair list file. This file contains the instance pair list in the same format as mentioned for the `-instance_pair_list` parameter. The file allows you to specify a large number of instance pairs. `-instance_pair_list_file` and `-instance_pair_list` are mutually exclusive.
`-net` *netname*	
	Specifies the name of the net to be analyzed.
`-number_threads` *value*	
	Specifies the maximum number of threads to use when reporting the least-resistance path (RLRP) for the bump-to-clamp rule type in the ESD flow. Generally, using more threads decreases the run time. There is no absolute limit to the number of threads you can use, however, it is recommended that you do not specify more threads than the number of CPUs available. The default value is `1`, and the maximum value that can be specified is the total number of available CPUs.
`-state_directory` *directory*	
	Specifies the name of the state directory in which the ESD analysis results are saved.

Examples

- The following commands specify to generate point-to-point RLRP path report for net `VDD_AO`:

```
generate_esd_rlrp_report \
-state_directory /rak_for_esd/ESD/ALL_125C_esd_1 \
-net VDD_AO \
-instance_pair_list { {VDD_AO_vsrc1 u0} {VDD_AO_vsrc2 u0 NA VDD} }
```

get_ir_diagnostics

```
get_ir_diagnostics
[-help]
[-cap_lower_limit value]
[-cap_upper_limit value]
-domain domainname
-domain_threshold value
[-eco_list list_of_instances ]
[-eco_mode {violation | aggressor | both}]
[-ir_drop_histogram_bin_size absolute_value_in_volts]
[-ir_drop_histogram_lower_limit absolute_value_in_volts]
[-ir_drop_histogram_upper_limit absolute_value_in_volts]
[-mode {report_by_region | predict_ir | report_aggressor | eco_report | report_only
|repair_directive_only |report_and_repair_directive}]
[-number_of_region value]
[-nworst_aggressor value]
[-nworst_instances value]
-output_directory directoryname
[-power_distribution_network_model directoryname]
[-report_mode {summary_only | detailed} ]
[-reset_predict]
-state_directory directoryname
[-watch_box filename]
[-watch_inst filename]
```

The `get_ir_diagnostics` command allows you do the following:

- IR drop distribution and summary reporting by the specified IR threshold - Worst Instance Voltage Drop (WIV), Total Instance Voltage Drop (TIV), and Number of Violation Instance Voltage Drop (NVIV)
- Aggressor IR impact analysis and reporting by the specified violation and aggressor limits
- Support various Tempus IR-ECO flows (violation or aggressor based flow with aggressor IR impact analysis and ECO IR estimation)
- IR drop prediction/estimation on input ECO changes
- IR violation reporting by regions

You must specify this command after running the `analyze_rail` command.

⚠ The `get_ir_diagnostics` command requires the `invs_vi` (Innovus Voltus InsightAI Option) license.

Parameters

`-help`
Outputs a brief description that includes the type and default information for each `get_ir_diagnostics` parameter. For a detailed description of the command and all of its parameters, use the man command man `get_ir_diagnostics`.
`-cap_lower_limit` *value*
Specifies the minimum capacitance value driven by aggressor candidates in the `eco_report` mode.
`-cap_upper_limit` *value*
Specifies the maximum capacitance value driven by aggressor candidates in the `eco_report` mode.
`-domain` *domainname*
Specifies the name of the power domain.
`-domain_threshold` *value*
Specifies the minimum allowed voltage for the domain being analyzed. The value will be in percentage. This parameter behavior is same as the `set_rail_analysis_domain -threshold` parameter.
`-eco_list` *list_of_instances*
Specifies the list of instances which were changed for `-mode predict_ir`. The format of the file is: *<original x>,<original y>,<original orientation>,<original cell>,<x>,<y>,<orientation>,<cell>, <instance>*
`-eco_mode {violation | aggressor | both}`

Default: `violation`

Specifies the mode to be used for generating the ECO report with the `-mode eco_report` parameter. The possible options are:

- `violation`: Generates a list of violations in the report.
- `aggressor`: Generates a list of aggressors in the report.
- `both`: Generates a report that includes both the violations and aggressors.

`-ir_drop_histogram_bin_size` *`absolute_value_in_volts`*

Allows you to control the EIV histogram bin size. The default value is 0.005.

In the following example, each histogram bin size is of 1mV:
`get_ir_diagnostics -eiv_histogram_bin_size 0.001`

`-ir_drop_histogram_lower_limit` *`absolute_value_in_volts`*

Specifies the lower limit of the EIV histogram range. This parameter must be used with `-report_mode summary_only`.

`-ir_drop_histogram_upper_limit` *`absolute_value_in_volts`*

Specifies the upper limit of the EIV histogram range. This parameter must be used with `-report_mode summary_only`.

`-mode {report_by_region | predict_ir | report_aggressor | eco_report | report_only |repair_directive_only |report_and_repair_directive}`

Default: `report_only`

Specifies the mode of operation for the command. The possible options are:

- `eco_report`: Generates a file (`eco.ird`) that is used by the Tempus-ECO flow to fix the IR drop violations in a design. The `eco.ird` file is located at: /*output_directory/eco.ird*. When this option is specified, it is recommended to specify the `-eco_mode` and `-power_distribution_network_model` parameters to support the intended ECO flow.
- `predict_ir`: Specifies to generate the IR prediction for the ECO changes specified with the `-eco_list` parameter. Based on the list of instances specified with `-eco_list`, the tool performs IR prediction. You can also choose to use the `-watch_box` and `-watch_inst` parameters to specify either the area or instance you want to observe and predict. You can specify multiple ECO change lists, and all the previous accumulated ECO changes will be saved until you reset them using the `-reset_predict` parameter. This option is associated with the `-eco_list`, `-watch_box`, `-`

`watch_inst`, `-reset_predict`, and `-power_distribution_network_model` parameters.

The format of prediction output is:

```
<instance_name1> <predicted_eiv_value1>
<instance_name2> <predicted_eiv_value2>
```

- `report_aggressor` : Reports the impact of the user-specified aggressor instances. The report file is located at: /*output_directory/IR_impact_<nworst_instances>_<nworst_aggressors>.rpt*. This option is combined with the `-nworst_instances`, `-nworst_aggressor`, and `-power_distribution_network_model` parameters.

  The following is the report format:

  ```
  VIOL:<viol name> <violating ir(domain drop-threshold)> <EIV time>
    -AGGR1:<aggr1 name> <impact1>
    -AGGR2:<aggr2 name> <impact2>
  ```

- `report_by_region`: Reports IR drop values by region in the report file (*output_directory/report.rpt* | *region_*.rpt*). This option is combined with the `-number_of_region` and `-nworst_instances` parameters.
- `report_only (default)`: Specifies to generate only the IR drop reports. When this argument is used, you can specify the type of report required using the `-report_mode` parameter.
- `repair_directive_only`: Specifies to generate the InsightAI database from the rail analysis results.
- `report_and_repair_directive`: Specifies to generate both the reports and InsightAI database from the rail analysis results.

`-number_of_region` *value*

Default: 3

Specifies the number of regions to be analyzed.

`-nworst_aggressor` *value*

Default: 50

Specifies the number of aggressors that should be reported for each violation.

`-nworst_instances` *value*

Default: 5

Specifies the number of worst instances to be reported.

`-output_directory` *`directoryname`*

Specifies the output directory in which the insight (IR drop and EM violation) results will be saved in.

`-power_distribution_network_model` *`directoryname`*

Specifies the rail state directory containing the sensitivity database. You can enable sensitivity database generation by specifying `set_rail_analysis_config -create_sensitivity_db true`. The `-power_distribution_network_model` parameter must be specified with the `-mode {report_aggressor | predict_ir}` and the `-eco_mode {aggressor | both}` parameters.

`-report_mode {summary_only | detailed}`

Controls the type of reports generated. The possible arguments are:

- `summary_only` (default): Includes the summary report only (saved in REPORTS/Design/Summary_Report).
- `detailed`: Includes both summary and detailed reports. The detailed reports instances based on domain, ranking, and range.

This parameter must be specified with the `-mode report_only | report_and_repair_directive` parameter. The parameter should not be specified with the `-mode repair_directive_only` parameter.

`-reset_predict`

Specifies to reset all the previous ECO changes. If this parameter is not specified, the previous changes in `-eco_list` will be retained for the `-mode predict_ir` runs.

`-state_directory` *`directoryname`*

Specifies the top-level state directory that is generated by the dynamic IR drop analysis. This state directory contains analysis of all nets associated with the power domain. This is the output directory in which the results of the `analyze_rail` command is stored (for example, core_25C_dynamic_1).

`-watch_box` *`filename`*

Specifies a file containing the list of regions, wherein only the instances inside these regions will be reported for prediction. You can write multiple regions in the file. This parameter is used with the `-mode predict_ir` parameter.

The format of the file is:

```
{{x1,y1}{x2,y2}}
{{x3,y3}{x4,y4}}
```

`-watch_inst` *`filename`*

	Specifies a file containing the list of instances to be reported for prediction. You can write multiple instances in the file. This parameter is used with the `-mode predict_ir` parameter.
	
	The format of the file is:
	`instance_1`
	`Instance_2`

Examples

- The following command specifies to perform IR prediction/estimation:

```
get_ir_diagnostics -mode predict_ir \
-state_directory Rail/latest \
-power_distribution_network_model Rail/latest  \
-eco_list eco_five \
-domain PD \
-domain_threshold 0.1 \
-watch_box region_watch\
-watch_inst instance_watch\
-reset_predict \
-output_directory predictIR
```

  The generated report will be saved at: *predict.out*

  The following is the sample report:

```
CTS_buf_01236 0.828523
CTS_buf_01248 0.829177
CTS_buf_02745 0.817085
```

- Specifies to generate the eco.ird file for Tempus IR-ECO consumption:

```
get_ir_diagnostics -mode eco_report \
-state_directory Rail/latest  \
-eco_mode both \
-domain PD \
-domain_threshold 0.1 \
-power_distribution_network_model Rail/latest  \
-output_directory debugIR
```

  The generated report will be saved at: *debugIR/eco.ird*

  The following is the sample report:

```
PGNET START
  NET   VDD  0.900
  NET   VSS  0.000
PGNET END

REGION START
```

```
INST ua/FE_n10786  1
  IRDROP VDD:VSS  8.000e-11  0.1307  0.000e+00  7.029e-04  3.315e-03
INST ua/FE_n6442  1
  IRDROP VDD:VSS  8.000e-11  0.1390  0.000e+00  7.452e-04  3.666e-03
```

- Specifies that the EIV histogram range is from 0.095 mV to 0.12 mV:

```
get_ir_diagnostics -domain_threshold 0.1 -output_directory debug_ir -state_directory
{rail_after/ALL_125C_dynamic_1} -domain ALL -mode summary_only -
ir_drop_histogram_bin_size 0.006 -ir_drop_histogram_lower_limit 0.095 -
ir_drop_histogram_upper_limit 0.12
```

Sample Summary Report

```
Found 8826 instances with IR-drop violations in design based on eiv_method: Worst and
eiv_eval_window: elapse.
```

Summary For All Cells :
Worst IR drop : 0.12337V
Specified IR drop threshold : 0.09V
Worst IR drop violation : 0.033327V
Total IR drop violation : 48.94V

IR drop violation distribution

Violation Range	Violation Count	Accumulated Violation count	Total Violation(V)	Accumulated Total Violation(V)
120mV ~ 125mV:	6	6	0.186857	0.186857
115mV ~ 120mV:	34	40	0.918714	1.10557
110mV ~ 115mV:	83	123	1.81664	2.92221
105mV ~ 110mV:	194	317	3.32192	6.24413
100mV ~ 105mV:	904	1221	10.5601	16.8043
95mV ~ 100mV:	2940	4161	21.3391	38.1433
90mV ~ 95mV:	4665	8826	10.7967	48.94

Related Topics

- *"Voltus InsightAI"* in *Innovus Common UI User Guide*

get_voltus_diagnostics_mode

```
get_voltus_diagnostics_mode
[-help]
[-config filename]
[-timing_protection_config filename]
```

Generates a configuration template after loading the design database and specifying the power/rail settings. This template is created based on the current rail settings.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_voltus_diagnostics_mode` parameter. For a detailed description of the command and all of its parameters, use the man command man `get_voltus_diagnostics_mode`.
`-config filename`	
	Specifies the name for the configuration file to be generated. This optional file includes details about the PG network definition, EIV method, EIV evaluation window, and a list of scenarios with their settings. By default, the configuration file is named `insight.config`. If a file with this name already exists, the suffix is automatically incremented (example, `insight.config1`, `insight.config2`, and so on).
`-timing_protection_config filename`	
	Specifies to automatically generate the timing protection configuration file. This file includes the timing degradation thresholds per timing view. The default file name is `insight_timing_protection.config`. This parameter allows you to specify the name of the file, where the timing protection configuration will be generated.

Examples

- The following command specifies to generate a configuration template for the Voltus InsightAI flow:
  `get_voltus_diagnostics_mode`

  The following message appears:
  `Config file insight.config will be generated`

  Following is a snippet from the generated `insight.config` file:

```
#### PG Network Definition ###Nets VDD VSS

Domain_1 ALL  VDD   VSS

#### Global Options ###eiv_eval_window {}eiv_method {}#### Scenario Definition
###SCENARIO_DEFAULT: SCENARIO_1  SCENARIO_2#### Scenarion Setting ###SCENARIO_1
 eiv_eval_window elapse   eiv_method Worst   Domain Domain_1        threshold_data 0.4
    threshold_clock 0.4   Net VSS        threshold_data 0.09        threshold_clock 0.09
 Net VDD        threshold_data 0.81        threshold_clock 0.81SCENARIO_2
 eiv_eval_window switching   eiv_method WorstAvg   Domain Domain_1        threshold_data
0.4        threshold_clock 0.4   Net VSS        threshold_data 0.09
threshold_clock 0.09   Net VDD        threshold_data 0.81        threshold_clock 0.81
```

Related Topics

- *"Voltus InsightAI"* in *Innovus Common UI User Guide*

map_dies

```
map_dies
[-help]
-3dic_design_stack_file filename
[-component_config_file filename]
[-die_pad_file_pair {{<die_name1> <pad file1>} {<die_name1> <pad file2>}...}]
[-map_search_distance distance]
[-mode sanity_check]
[-output directory]
[-stacked_die_mapping filename]
```

A utility to generate the inter die data mapping file based on the design stack configuration file and die ploc files. This mapping file is required if there is a direct connection between dies. The generated mapping file defines the connection between dies (between interposer and logic die). This mapping file is then specified with the `set_multi_die_analysis_mode -stacked_die_mapping_file` command for multi-die analysis.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-3dic_design_stack_file filename`	
	Specifies the name of the XML file containing the design stack information. This file is from the Integrity™ 3D-IC product. It includes each die's tier level (Z direction), die instance name, size (post shrink size if there is a scale factor), flip, rotation (counter, clockwise), placement location, and contact layer.
`-component_config_file filename`	

Specifies the name of the design configuration file. This file is used to define the power pad location files, and the power and ground net names.

File Format:

```
COMPONENT IDIE
Power_pad { {ldie_VDD.ploc VDD_AO} {ldie_VSS.ploc VSS} }
[Power_net { VDD_AO}]
[Ground_net {VSS}]
END
COMPONENT LDIE
Power_pad { {ldie_ALL.ploc all} }
[Power_net {VDD1}]
[Ground_net {VSS2}]
END
```

`-die_pad_file_pair {{<die_name1> <pad file1>} {<die_name1> <pad file2>}...}`

Specifies the mapping between die instance names and their corresponding power pad file for the interposer die and logic dies.

`-map_search_distance distance`

Specifies the legal mapping distance range between the interposer and legal dies. The unit for distance is micrometer (um).

`-mode sanity_check`

Specifies to perform sanity checks for the generated mapping file.

`-output_dir directory`

Specifies the output directory containing the mapping results. The output folder contains the generated mapping file (`die_die.map`).

The format of the 6-column mapping file is:

```
IDIE_INST_NAME(idie_inst_name) Inet_name Ibump_name
LDIE_INST_NAME(ldie_inst_name) Lnet_name Lbump_name
```

Here,

- The first 3 columns are for the interposer die instance name, net name, and bump name(vsrc name), and the last 3 columns are for the logic die instance name, net name, and bump name.
- The die instance names are written in upper case that is followed by the die instance names specified in the rail TCL for each die instance name.
- The file is split in two sections, POWER and GROUND.

`-stacked_die_mapping` *filename*	

	Specifies the mapping file used to define the connection between dies (between interposer and logic die). This file provides the information on bump-to-bump mapping (one bump from interposer die and the other bump from logic die) to provide logical connectivity.
	
	The format of the 6-column mapping file is:
	`IDIE VDD_AO BUMP_PWR_1 LDIE1 VDD_AO LDIE_PWR_BUMP7`
	`IDIE VDD_AO BUMP_PWR_2 LDIE1 VDD_AO LDIE_PWR_BUMP8`
	`...`
	Here, the first 3 columns are for the interposer die instance name, net name, and bump name, and the last 3 columns are for the logic die instance name, net name, and bump name.

Output

The following is the list of reports that are generated for sanity checking:

- `die_map.summary` - This report has 2 sections:
  - Interface mapping summary - Lists all the die_inst1/net to die_inst2/net mapping pairs, and their matched bump mappings in the overlap region.
  - Die connect-terminal summary - Reports matched and total terminals for each die instance/net/layer group.
- `map_diagnosis.log` - Reports a list of unmapped bumps/terminals for a die-to-die pair list and the reasons for the unmapped bumps (For example, unknown die, unknown net, undefined terminal, power to ground short, and floating terminal).
- `terminal_map.rpt` - Lists mapping pairs (`die1:net1:bump1 :: die2:net2:bump2`) that pass the mapping and alignment checks, therefore are considered as legal mapping pairs.
- `terminal_offset.rpt` - Reports all mapping pairs where offset is greater than the specified `map_search_distance`.

Examples

- The following command generates a mapping file that includes the connection between dies (between interposer die INTERPOSER and logic dies DIE0, DIE1, DIE2, DIE3):

```
map_dies \
-output_dir results \
-3dic_config_file ../data/design_stack.xml \
-die_pad_file_pair { {INTERPOSER ../data/Interposer_all.ploc} \
                     {DIE0 ../data/Die0_all.ploc} \
                     {DIE1 ../data/Die1_all.ploc} \
                     {DIE2 ../data/Die2_all.ploc} \
                     {DIE3 ../data/Die3_all.ploc} }
```

- The following command performs the sanity check for the generated mapping file:

```
map_dies \
-mode sanity_check \
-output Results \
-3dic_design_stack_file ./data/design_stack.xml \
-3dic_config_file ./data/design_stack.xml \
-component_config_file ./data/Components.config \
-stacked_die_mapping ./data/die2die_bump.map \
-map_search_distance 5 \
```

Related Topics

- "Package Analysis" in the *Voltus User Guide*

report_differential_voltage

```
report_differential_voltage
[-help]
-differential_voltage_limit value
-power_directory directory_name
-state_directory directory_name
```

Determines the differential voltage between a receiver/driver pair during ramp-up analysis. This command can be used to mitigate the risk of high short-circuit current for a driver-receiver instance pair due to slow/fast (differential) ramp-up voltage at the output of the power-switch cell connected to the driver/receiver cell.

The `report_differential_voltage` command reports the differential potential for those receiver/driver pairs that are greater than the specified differential voltage threshold, and the time interval at which the differential voltage remains greater than the threshold. You must specify the `power_create_driver_db true` attribute before specifying this command.

The following is the format of the output report:

```
# <driverName>   <receiverName>   <peak_diffv>  <minT>   <maxT>
```

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-differential_voltage_limit value`	
	Specifies to report differential voltage for the receiver/driver pairs that exceed the specified threshold limit.
`-power_directory directory_name`	
	Specifies the path to the power directory to access the driver-receiver pairs.
`-state_directory directory_name`	
	Specifies the path to the state directory generated by rail analysis to access the EIV or timing database.

Examples

- The following command lists all the driver-receiver pairs that exceed the differential voltage limit of 0.001:

  ```
  report_differential_voltage -differential_voltage_limit 0.001 -state_directory
  dynamic_rail/core_105C_dynamic_1 -power_directory dynamic_power
  ```

  The following is a snippet of the generated report:

  ```
  # <driverName>          <receiverName>       <peak_diffv>     <minT>      <maxT>
  column/MSV_I2         column/MULT8_reg_1_4    0.7382          5e-11     2.75e-09
  column/HIER_INST      column/MULT8_reg_1_4    0.7382          5e-11     2.75e-09
  ...
  ```

Related Topics

- "Static Power, IRdrop and EM Analysis" and "Dynamic Power and IRDrop Analysis" in the *Voltus User Guide*

report_irdrop_regions

```
report_irdrop_regions
[-help]
[-eco_report]
[-enable_xp {true | false}]
[-net netname | -domain domainname]
[-net_threshold value | -domain_threshold value]
[-non_violating_insts_limit value]
[-number_of_worst_instances value]
[-output_dir dir]
[-region {x1 y1 x2 y2 }| -number_of_region value]
-state_directory dir
[-tile_row_column {X Y}]
[-tile_size {X Y}]
```

Specifies to automatically identify the region with worst IRdrops and EM violations, and provide necessary information to determine and fix the root cause. The reports are consolidated for each region to find root-cause. Root cause analysis is done by identifying top 10 instances in each hotspot and report the following:

- Hotspot Region Coordinate
- Timing Library Analysis
- Power-Grid View Analysis
- Power-Grid Integrity Analysis
- IR-Drop Composition
- Root Cause Analysis

You must specify this command after running the `report_rail` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_irdrop_regions` parameter. For a detailed description of the command and all of its parameters, use the man command man `report_irdrop_regions`.
`-domain` *domainname*	

	Specifies the name of the power domain.
`-domain_threshold` *value*	
	Specifies the minimum allowed voltage for the domain being analyzed. The value will be in percentage. This parameter behavior is same as the `set_rail_analysis_domain -threshold` parameter.
`-eco_report`	
	Specifies to generate a file with IR drop information that can be used by Tempus ECO to fix IR voltage drop for all instances in a design. The report (`eco.ird`) is generated in the specified output directory (`report_irdrop_regions -output_dir`).
`-enable_xp {true | false}`	
	Specifies to identify the worst IR drop and EM violations regions in the Extensively Parallel (XP) mode wherein processing is distributed over a large number of machines. *Default* : `false`
`-net` *netname*	
	Specifies the name of the power or ground net that you want to analyze.
`-net_threshold` *value*	
	Specifies the minimum allowed voltage for the power net being analyzed. The value must be an absolute number. This parameter behavior is same as the `set_pg_nets -threshold` parameter.
`-non_violating_insts_limit` *value*	
	Reports non-violating instances within hotspots (violating tiles) that are close to the threshold value. This parameter allows you to define a limit (sub-threshold) that can be used to report the non-violating instances that are close to the threshold. For example, if the `-net_threshold` or `-domain_threshold` is `60mV`, and `-non_violating_insts_limit` is `3mV` close to the threshold, the `report_irdrop_regions` command will report all the violating instances above the domain/net threshold and also the non-violating instances which are in the range of 60mV-57mV. This information about the non-violating instances can be used for IR improvement in the Timing-aware IR Drop Fixing flow.
`-number_of_region` *value*	
	Specifies the number of regions to be analyzed. The default value is `3`.
`-number_of_worst_instances` *value*	

	Specifies the number of worst instances to be reported. The default value is 5.
`-output_dir` *dir*	
	Specifies the output directory in which the Irdrop and EM violation results will be saved in.
`-region {`*x1 y1 x2 y2* `}`	
	Specifies the region containing the worst instances. The default unit of *x1 y1 x2 y2* is in database units.
`-state_directory` *dir*	
	Specifies the top-level state directory that is generated by the dynamic IR drop analysis. This state directory contains analysis of all nets associated with the power domain. This is the output directory in which the results of the `report_rail` command is stored (for example, core_25C_dynamic_1).
`-tile_row_column {X Y}`	
	Specify the number of tiles in the x and y direction. The `-tile_row_column` and `-tile_size` parameters enable tile-based analysis of the hotspot regions. Tile-based analysis divides the design into user-defined tile size, and reports IR drop violations on each tile. The default value is `100um^2 (10x10 tiles)`.
`-tile_size {X Y}`	
	Specify the size of tiles in the x and y direction. The `-tile_row_column` and `-tile_size` parameters enable tile-based analysis of the hotspot regions. Tile-based analysis divides the design into user-defined tile size, and reports IR drop violations on each tile.

Hot-Spot Debugger Report

```
* Top Worst Instances
ID       | Instance Name
--------------------------------
       1| inst_I2
       2| inst_I3
...
#Power-Grid Integrity Analysis
Instance | LEF Pin:PG Net            | Floating PG Pins
-----------------------------------------------------------
       1| VDD:VDD ; VSS:VSS         | None
       2| VDD:VDD ; VSS:VSS         | None
...
* Number of Weakly Connected Wire Segments
        VDD |         VSS
-----------------------
          0 |           0
#Power-Grid View Library Status
Instance| Status | Power Pins(V)| Intrinsic Cap (fF)    | View Used  | #Current Taps | PGV Library Path
-------------------------------------------------------------------------------------------------
      1| PASS   | VDD:1.1      | VDD:13.480 VSS:13.480 | Early      | VDD:1 VSS:1 | /data/cl/stdcells.cl
...
#IR-Drop Composition
Instance| Total IR-drop (V)        | IR-drop on Power Net (V)| Ground-bounce on ground net (V)
-------------------------------------------------------------------------------------------
      1| 0.072 (TW) 0.072 (Worst) | 0.031 (TW) 0.031 (Worst)| 0.043 (TW) 0.043 (Worst)
...
* Layer Based IR-Drop for Net VDD
Instance| IR Across Package(mV) | IR Across Power-Gate(mV)| Layer Based IR(mV)
-------------------------------------------------------------------------
       |                       |                         |         M5L       0.683
       |                       |                         |       C4L5L       0.477
 1     |  0.035                |  0.000                  |         M4L       0.394
-------------------------------------------------------------------------
2 ...
-------------------------------------------------------------------------
* Layer Based IR-Drop for Net VSS
...
-------------------------------------------------------------------------
#Root Cause Analysis Report

* Net: VDD
Instance  | IR-Drop(V) | RLRP(Ohm) | Loading Cap(fF)| Current Density (mA/um2)| Transition Density| Frequency(Mhz) | Slew
-------------------------------------------------------------------------------------------------------------------
        1| 0.031*     | 66.5*     | 64.371*        | 0.022*                  | 615384576*        | 307*           | 0.100*
...
-------------------------------------------------------------------------------------------------------------------
Design Max| 0.058      | 159       | 140332.844     | 0.061                   | 615384640         | 307            | 0.984
Design Avg| 0.003      | 43.7      | 2.585          | 0.002                   | 93809752          | 300            | 0.076
Design Min| 0.000      | 0.0988    | 0.000          | 0.000                   | 0                 | 25             | 0.002
* Net: VSS
Instance  | IR-Drop(V) | RLRP(Ohm) | Loading Cap(fF)| Current Density (mA/um2)| Transition Density| Frequency(Mhz) | Slew
...
```

Examples

- The following command generates IR debug report for the `vss` net in the specified output directory:

```
report_irdrop_regions -state_directory dynamic_rail/core_25C_dynamic_1/VSS -net VSS -
output_dir IR_DEBUG_VSS -number_of_worst_instances 10
```

Related Topics

- *"Static Power, IRdrop and EM Analysis"* and *"Dynamic Power and IRDrop Analysis"* in *Voltus User Guide*

report_esd_network

```
report_esd_network
[-help]
<run_name>
[-clamp_lowest_layer_taps {true | false}]
[-clamp_pin_short_file pin_shorting_file]
-config_file rule_filename
[-output_dir directory]
[-remove_layers_below layer_name]
[-task_assistant]
-type {net | domain}
[-use_power_pad {true | false}]
```

Performs different types of effective resistance checks, such as bump to clamp, bump to bump, and clamp to clamp, and current density analysis during an ESD zap event. This command checks whether every power or ground bump has an ESD device, and the device is placed in such a way that it does not violate the effective resistance or current density limit.

The objective of ESD Checker is to easily identify the violations caused by high effective resistance or the current density above the specified threshold value.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`<run_name>`	Mandatory parameter. Specifies the domain or net name. The result directory is prefixed with `<run_name>`.
`-clamp_lowest_layer_taps {true | false}`	
	Specifies to keep only the lowest layer of clamp taps for effective resistance analysis. The default value is `false`. • When set to `true`, the ESD pin nodes on the lowest layer are shorted, and effective resistance checks are performed to the shorted node on the lowest layer. • When set to `false`, the ESD pin nodes on all layers are shorted, and effective resistance checks are performed to the shorted pin nodes of clamps on different layers.
`-clamp_pin_short_file pin_shorting_file`	

	Specifies the list of pin layers to be shorted for bump-to-clamp and clamp-to-clamp resistance checks. If multiple layers for the same pin are specified, then these layers are shorted with each other.
	The format of the file is:
	`#Cell specification`
	`CELL <cell name> <pin> <layer>`
	`#Instance specification`
	`INST <instance name> <pin> <layer>`
	In this file, multiple layers for the same cell or instance can be specified by adding multiple rows for the same cell or instance name corresponding to each layer.
	If the `-clamp_pin_short_file` parameter is not specified, the software shorts all pin nodes of the lowest ESD cell pin layer, and performs effective resistance checks to the shorted node on the lowest layer.
	Note: If the `-clamp_pin_short_file` parameter is used in conjunction with `-clamp_lowest_layer_taps`, the `-clamp_pin_short_file` parameter will override the settings of `-clamp_lowest_layer_taps`.

`-config_file` *`rule_filename`*	
	Mandatory parameter. Specifies the name of the rule file that includes all the ESD checks to be performed as a set of rules. It also specifies the effective resistance threshold limit. For more information, refer to the **ESD Rule Specification** section in the "ESD Analysis" chapter of *Voltus User Guide*.
`-output_dir` *`directory`*	
	Specifies the name of the output directory in which the ESD analysis results are saved.
`-remove_layers_below` *`layer_name`*	
	Specifies the layer name below which all layers are dropped from analysis. This parameter can be used in scenarios where the ESD cells connect to the PG grid at high layers, and therefore the impact of the lower PG layers is very small on effective resistance calculation and current density checks.
`-task_assistant`	
	Opens ESD Analysis Task Assistant, a task-based quick help functionality for the ESD analysis flow. The information in the Task Assistant is organized by topics relevant to the task you are performing. It answers frequently asked questions, such as format of the rule file, use model of the command, or supported ESD checks.
`-type {net | domain}`	

	Mandatory parameter. Specifies whether to run rail analysis on a domain or a net.
`-use_power_pad {true | false}`	
	Specifies that the x,y locations of the bump cells are to be obtained from the power pad file. This parameter can be used to support designs for which bump cells are not visible. The default value is `false`.

Examples

- The following commands specify to run ESD analysis on the domain `top`:

```
set_rail_analysis_config -method static -accuracy hd -power_grid_libraries ./techonly.cl
-ict_em_models ./Data/ESD_em.ict
report_esd_network top -config_file ./esd_new_rule.txt -type domain -output_dir
ESD_analysis -use_power_pad true
```

report_esd_voltage

```
report_esd_voltage
[-help]
-current_distribution_layer layer
[-detailed_report {true | false}]
-driver_receiver_file filename
-esd_pin_location_file filename
[-max_violations value]
-net NetName
[-report filename]
[-threshold value]
-total_current value
```

Performs IR Drop analysis for every driver/receiver pair and reports voltage drop above the given limit. Use this command to provide the current injection points (sinks), the current distribution layer (source), and the total ESD current to be distributed. The command generates a report containing all the violations where potential difference across driver/receiver is greater than the given threshold.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-current_distribution_layer layer`	
	Specifies to distribute current to all nodes of the given layer. This is the LEF/DEF layer name. This is a required parameter.
`-detailed_report {true | false}`	

Specifies to report the differential voltage for each zap point above the specified threshold. It gives the x,y location of the zap point and the zap point name. Using this parameter, you can fix all the differential voltage violations in a single run.
This parameter will generate the report file with the same name as specified using the `-report` parameter and with the `_detail.rpt` suffix.

Default : `false`

The following is an example of the parameter:

```
report_esd_voltage -net VSS -driver_receiver_file ./driverdb_railVSS  -
esd_pin_location_file VSS.pp -current_distribution_layer METAL5 -total_current 8 -
report FULL_new_gnd_esd_cdm.rpt -detailed_report true -threshold 2
```

`-driver_receiver_file` *`filename`*

Specifies the name of the driver/receiver file of the net. This file is created by the `-create_driver_db` parameter of the `set_power_analysis_mode`. This is a required parameter.

The file has multiple columns, wherein the first column is the receiver, and the remaining columns are drivers to the receiver instance. The following is an example of the file content:

`padring/I_RCV_7` `io_toplevel/padring/I_DECAP io_toplevel/padring/`

In this example:

`padring/I_RCV_7` is the receiver instance. `io_toplevel/padring/I_DECAP` and `io_toplevel/padring/` are driver instances.

`-esd_pin_location_file` *`filename`*

Specifies the name of the ESD pin location file. This is a required parameter.

The format of the file is:

```
#Pin_name    x coord    y coord    layer

gnd_pin_1      2284.6    -2898.05   m4
...
...
```

`-max_violations` *`value`*

Specifies the number of max worst violations to be reported. This is an optional parameter.

When this value is specified, only the specified number of top violations will be reported. If a value is not specified, voltage drop for all driver/receiver pairs will be reported.

`-net` *`NetName`*

	Specifies the name of the net to be analyzed. This is a required parameter.
`-report` *`filename`*	
	Specifies the name of the output report file in which details of the driver-receiver violation pair is written. This is an optional parameter. The report is filtered in descending order of differential voltage. The output file has four columns: • First column - `Differential voltage(V)` • Second column - `Driver name (node location(um) )` • Third column - `Receiver name (node location(um))` • Fourth column - `ESD pin name (location(nm))` The following is the sample ESD file format: **`14.1324 pmc/a_ip_pmc_chassis_c55fg ( 2050.875 -2760.085 ap ) io_toplevel/padring/I_ESDCLAMP_39 ( 1428.200 2884.250 m4 ) gnd_pin_22 ( 1458.200 2898.050 m4 )`**
`-threshold` *`value`*	
	Specifies the minimum allowed driver/receiver voltage for filtering the report. This is an optional parameter. When a threshold value is specified, driver/receiver pairs with differential voltage greater than the specified threshold value will be reported. If a threshold value is not specified, voltage drop for all driver/receiver pairs will be reported.
`-total_current` *`value`*	
	Specifies the total ESD current to be distributed. The default current unit is amperes. This is a required parameter.

Example

- The following example performs static IR Drop analysis to check the voltage drop between all driver and receiver pairs:

```
report_esd_voltage -net gnd -driver_receiver_file ./POWER/driverdb_railgnd  -
esd_pin_location_file ./epinfile -current_distribution_layer m1 -total_current 8 -report
FULL_new_gnd_esd_cdm.rpt
```

report_joule_heat

```
report_joule_heat
[-help]
[-detail_delta_temperature_file filename]
[-detail_delta_temperature_region {x1 y1 x2 y2}]
[-net pg_net_name]
[-output_directory   dir_name]
[-power_domain rail_domain_name]
[-report_connected_pin_wire filename]
[-scale_rms_limit value]
[-tile_delta_temperature_file filename]
[-tiles {n_tile_in_x m_tile_in_y}]
```

Specifies to perform joule heat analysis. This command will use the dynamic RMS current to compute delta-T RMS on the power and signal wires for all segments in the design. The `report_joule_heat` command is similar to the `report_self_heat` command, except that the `report_self_heat` command computes both delta-T FEOL (FEOL self heat) and delta-T RMS (BEOL self heat) whereas the `report_joule_heat` command computes only delta-T RMS.

> Joule heat analysis (`report_joule_heat`) should be performed for the entire design, that is, you must use domain analysis with all PG nets (`-power_domain ALL`).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_joule_heat` parameter. For a detailed description of the command and all of its parameters, use the man command `man report_joule_heat`.
`-detail_delta_temperature_file filename`	
	Specifies the name of the generated BEOL detailed delta-temperature file.
`-detail_delta_temperature_region {x1 y1 x2 y2}`	

	Specifies the region coordinates for generating detailed delta temperature file.
`-net` *`pg_net_name`*	
	Specifies a net for joule heat analysis.
`-output_directory` *`dir_name`*	
	Specifies the directory name for storing the joule heat analysis results. When you specify this parameter, the following result files are saved in the specified directory: • Rail analysis output directory • Detail delta temperature file If `-output_directory` is not specified, by default the joule heat analysis results are stored in the current working directory.
`-power_domain` *`rail_domain_name`*	
	Specifies the domain name of the rail.
`-report_connected_pin_wire` *`filename`*	
	Reports results on wires connecting to cell pins.
`-scale_rms_limit` *`value`*	
	Specifies the scale factor for the root-mean-square (rms) limit relaxation factor during Electromigration current limit analysis.
`-tile_delta_temperature_file` *`filename`*	
	Specifies the name of the generated BEOL tile-based delta-temperature file.
`-tiles` *`{n_tile_in_x m_tile_in_y}`*	
	Specifies the number of tiles in the x and y directions in the tile-based delta temperature file.

Examples

- The following command is an example of `report_joule_heat` command:

```
report_joule_heat -power_domain ALL -tile_delta_temperature_file joule_ttm.txt -tiles {10 10} -detail_delta_temperature_file joule_heat_detail.rpt -report_connected_pin_wire joule_heat.rpt
```

Related Topics

- "Self-Heating Effect Analysis" in the *Voltus Stylus Common UI User Guide*

report_noise_margin

```
report_noise_margin
[-help]
-noise_limit_file input_file
-power_directory dir
-state_directory dir
```

Specifies to calculate noise margins using the Effective Instance Voltage (EIV) method, that is, calculate noise margins with worst voltages seen on the driver-receiver pins. You need to specify the `power_create_driver_db true` attribute to generate the driver-receiver database.

To run this command, you need to specify the default or cell-specific noise limit, signal noise, guardband, rail directory, and power directory.

The command generates the noise margin output file in the state directory. The format of the EIV-based noise margin output file (*powernetname_groundnetname*`.noise_margin`) is:

Margin_High(V)	**Margin_Low(V)**	**NMH(V)**	**NMl(V)**	**Voh(V)**	**Vol(V)**	**Vih(V)**	**Vil(V)**	**Signal_Noise(V)**	**Guardband(V)**	**Drlnst/pin**	**Rxlnst/pin**
NMH - signal noise - guardband	NMI - signal noise - guardband	Voh-Vih	Vil-Vol	VDD_driver-VSS_receiver	VSS_driver-VSS_receiver	(l-noise_limit)*(VDD_receiv	noise_limit*(VDD_receiver-V	Signal noise value specified in the noise li	Guardband value specified in the noise li	Driver pin name	Receiver pin name

						er-VSS_receiver) noise_limit is the noise limit value specified in the noise limit file	SS_receiver) noise_limit is the noise limit value specified in the noise limit file	mit file	mit file			

Parameters

`-help`	Prints a brief description that includes the type and default information for each `report_noise_margin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_noise_margin`
`-noise_limit_file` *`input_file`*	

	Specifies the name of the input noise limit file that contains the default or cell-specific noise limit, signal noise, and guardband. The format of the noise limit file is:
	`default_noise_limit <value>`
	`default_signal_noise <value>`
	`default_guard_band <value>`
	`<cell_name> <noise_limit> <signal_noise> <guard_band>`
	
	The following is a snippet of the noise limit file:
	`default_noise_limit 0.46`
	`default_signal_noise 0.1`
	`default_guard_band 0.05`

`-power_directory` *`dir`*	
	Specifies the path to the power directory to access driver-receiver pairs.
`-state_directory` *`dir`*	
	Specifies the path to the state directory generated by rail analysis to access the EIV or timing database.

Examples

- The following command calculates noise margin:

```
report_noise_margin -noise_limit_file ./noiselimit -state_directory
dynamic_rail/core_105C_dynamic_1 -power_directory dynamic_power
```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

report_package

```
report_package
[-help] <domain>
[-computer_resources string]
[-output_dir <directoryname>]
-package_board_name <bga_circuitname>
-package_die_name <die_circuitname>
-package_pin_current_file <filename>
-result_name <filename>
[-spd_file filename]
-type <package_analysis_type>
-workspace <workspace_name>
```

Specifies how the package analysis will be performed, and to run package analysis.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
domain	Specifies the rail analysis domain name.
`-computer_resources` *string*	
	Specifies the computer resources for the package analysis run. This parameter allows you to customize the host distribution/CPU usage for package analysis. The default setting is "`-smt 0 -local -cn localhost -cpus 8`".
`-output_dir <`*directoryname*`>`	
	Specifies the output directory name of the package analysis results. By default, the package model will be created in the specified workspace.
`-package_board_name <`*bga_circuitname*`>`	
	Defines the ball grid array (BGA) circuit name in the package model.
`-package_die_name <`*die_circuitname*`>`	
	Defines the die circuit name in the package model.
`-package_pin_current_file <`*filename*`>`	

	Specifies the name of the generated pin current file from rail analysis that contains the package pin names and the current values. It is automatically generated after the package model is created.
`-result_name <`*`filename`*`>`	
	Specifies the name of the .xml result file.
`-spd_file` *`filename`*	
	Specifies an existing Sigrity .spd file for package analysis. This parameter allows you to specify a .spd file instead of the workspace name (package model database) of the package analysis (PowerDC) tool.
`-type <`*`package_analysis_type`*`>`	
	Specifies the type of package analysis. Currently, the software supports only DC package analysis.
`-workspace <`*`workspace_name`*`>`	
	Specifies the workspace name for package analysis (powerDC). The file name extension is `.pdcx`.

Example

- The following example specifies the package analysis details and starts package analysis:

```
report_package ALL \
-type dc \
-workspace /../sigrity/rak_pdc_irdrop/test.pdcx \
-package_board_name FPBGA144_Signal02 \
-package_die_name CD1_Signal01 \
-output_dir rak_pdc_irdrop_voltus_cpf \
-result_name rak_pdc_irdrop_voltus \
-package_pin_current_file /../spa_cpf/ALL_125C_avg_1/Reports/super_filter_pkg_iv.rpt
```

report_rail

```
report_rail
[-help]
name
[-output_dir directory_name]
[-type {domain | net}]
```

Runs rail analysis on a net or domain. This command must be specified after defining the rail analysis mode properties using the `set_rail_analysis_config` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_rail` parameter. For a detailed description of the command and all of its parameters, use the man command: man `report_rail`
name	Specifies the name of the domain or net that you want to analyze.
`-output_dir` *directory_name*	
	Specifies the name of the directory in which to store the output directories created during the analysis. *Default*: Current working directory.
`-type {domain | net}`	
	Specifies whether to run rail analysis on a domain or a net. A domain is a list of power or ground nets to analyze together. See `set_power_pads` for more details. You can specify the variable "ALL" to analyze all domains. The domain name or the net name must be specified at the end of the command line, as shown below: `report_rail -type net -output_dir ./` **`vdd`**

Examples

- The following command performs an analysis for the vdd net and places the results directories in the analysis_out directory:

  ```
  report_rail -type net -output_dir analysis_out vdd
  ```

- The next command performs rail analysis on TDSP power domain and writes the results in current working directory.

  ```
  report_rail -type domain -output_dir ./ TDSP
  ```

- The following command performs rail analysis on all power domains and writes the results in the current working directory:

  ```
  report_rail -type domain -output_dir ./ ALL
  ```

Related Topics

- "Static Power, IRdrop and EM Analysis" and "Dynamic Power and IRDrop Analysis" in the *Voltus User Guide*

report_resistance

```
report_resistance
[-help]
[-cell_name master_cell_name]
[-die_mode {3dic}]
[-force_reextraction {true | false}]
[-instance_list {instance1 pin1 instance2 pin2...}]
[-instance_list_file filename]
[-instance_pair_list {instance1 instance2 pin1  pin2 ...}]
[-instance_pair_list_file filename]
-net_name netName | -domain_name domainName
[-node_list {{nodeX nodeY METAL_LAYER [LABEL]}...}]
[-node_list_file filename]
[-node_pair_list {node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2...LABEL}]
[-node_pair_list_file filename]
[-output_file filename]
[-output_dir directory_name]
[-region {x1 y1 x2 y2 layer number_of_nodes region_name}]
[-report_limit value]
[-threshold value]
[-write_cell_name]
[-domain_threshold value]
[-net_threshold  value]
[-exclude_list_file filename]
```

Performs effective resistance analysis. Effective resistance computes effective resistance between two nodes on the power-grid, or from a node to all the voltage sources in the design, or from an instance to all the voltage sources in the design.

You can run this command without performing power calculation and IRdrop analysis. The flow requires you to load the design, set power nets, set power pads, and run resistance analysis. This command enables the tool to consider the entire power structure, including embedded ring structure within IO cells if any.

Note: The `report_resistance` full chip analysis now supports running the `set_rail_analysis_config -enable_rlrp_analysis` command to perform least resistance path analysis (RLRP) in conjunction with effective resistance analysis (REFF) in the DP mode. However, RLRP and REFF analysis should be done separately in the XP mode. When performing the point-to-point effective resistance analysis (`report_resistance`) for the power/signal nets in the XP mode, disable the RLRP analysis (`set_rail_analysis_config -enable_rlrp_analysis false`).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_resistance` parameter. For a detailed description of the command and all of its parameters, use the man command `man report_resistance`.
`-cell_name` *`master_cell_name`*	
	Reports effective resistance for all instances of the specified master cell name. The `-cell_name` parameter improves the usability of effective resistance calculation by allowing you to specify only the cell name of the instances you want to analyze. You can specify multiple cells for performing effective resistance analysis of multiple cells simultaneously.
`-die_mode {3dic}`	
	Specifies to perform multi-die effective resistance analysis.
`-domain_name` *`domainName`*	
	Specifies the domain name that is to be analyzed.
`-domain_threshold` *`value`*	
	Specifies the minimum allowed effective resistance (effective resistance sum of each net) to instance. For specific instance list analysis, the instances below the threshold value will be reported as “PASS”, while the instances above the threshold value will be reported as “FAIL”. For full chip analysis, the instances below the threshold value will not be reported in the domain report. The default value is 0.
`-exclude_list_file` *`filename`*	

	Specifies the name of a file containing a list of cells or instances that will not be processed during the resistance analysis flow. The specified cells/instances will be ignored from report generation.
	The use model of this parameter is:
	`report_resistance -exclude_list_file <`*`file_name`*`>`
	File Format:
	`<CELL | INST> <`*`name`*`>`
	Example:
	`CELL BUMP_MASTER`
	`CELL ESD_MASTER`
	`INST INSTA`

`-force_reextraction {true | false}`	
	Specifies to force a re-extract of resistances.
`-instance_list {`*`instance1 pin1 instance2 pin2...`*`}`	
	Specifies the list of instances for which effective resistance is to be calculated. *pin* is an optional argument. By default, the tool will use the instance pin associated with the net being analyzed. The *pin* argument can be used in special cases when two different pins of an instance are connected to the same net. **Note**: The `-instance_list` and `-instance_list_file` parameters are mutually exclusive to the `-instance_pair_list` and `-instance_pair_list_file` parameters. This means that you can specify `-instance_list` and `-instance_list_file` together but you cannot specify `-instance_list` and `-instance_list_pair` together.
`-instance_list_file` *`filename`*	
	Specifies the name of the instance list file. This file contains the list of instances in the same format as mentioned for the `-instance_list` parameter. The file allows you to specify a large number of instances. `-instance_list_file` and `-instance_list` are mutually exclusive. **Note**: The `-instance_list` and `-instance_list_file` parameters are mutually exclusive to the `-instance_pair_list` and `-instance_pair_list_file` parameters. This means that you can specify `-instance_list` and `-instance_list_file` together but you cannot specify `-instance_list` and `-instance_list_pair` together.
`-instance_pair_list {`*`instance1 instance2 pin1 pin2`* `... }`	
	Specifies the instance pair list to enable effective resistance calculation between two instances in the design. *`pin1`* and *`pin2`* are pins of *`instance1`* and *`instance2`*, respectively. **Note**: The `-instance_list` and `-instance_list_file` parameters are mutually exclusive to the `-instance_pair_list` and `-instance_pair_list_file` parameters. This means that you can specify `-instance_list` and `-instance_list_file` together but you cannot specify `-instance_list` and `-instance_list_pair` together.
`-instance_pair_list_file` *`filename`*	

	Specifies the name of the instance pair list file. This file contains the instance pair list in the same format as mentioned for the `-instance_pair_list` parameter. The file allows you to specify a large number of instance pairs. `-instance_pair_list_file` and `-instance_pair_list` are mutually exclusive. **Note**: The `-instance_list` and `-instance_list_file` parameters are mutually exclusive to the `-instance_pair_list` and `-instance_pair_list_file` parameters. This means that you can specify `-instance_list` and `-instance_list_file` together but you cannot specify `-instance_list` and `-instance_list_pair` together.
`-net_name` *netName*	
	Specifies the net name that is to be analyzed.
`-net_threshold` *value*	

	The effective resistance requirements for different power nets vary depending on the design. For example, an array or periphery supply to an SRAM needs to be signed off at a lower effective resistance threshold compared to the core supplies. This parameter allows you to specify different threshold values for different nets.
	The use model of this parameter is given below:
	`set_rail_analysis_domain -domain_name ALL -power_nets {vdd vddar vdda vdd1 vdd2} -ground_nets vss`
	`report_resistance -threshold 50 -net_threshold {{vdd 50} {vddar 20} {vdda 35} {vss 40}} -domain_name ALL -output_dir Reffective`
	In this example, the threshold value for different nets are:
	vdd -> 50
	vddar →20
	vdda -> 35
	vss -> 40
	vdd1 -> 50
	vdd2 -> 50
	Here, for all the remaining nets for which a value is not specified using the `-net_threshold` parameter, the value specified with the `-threshold` parameter is applied.
	The different threshold values will have an impact on the list of instances reported in the `<state_dir>/Reports/<net>/<net>.effr` report.

`-node_list` *{{nodeX nodeY METAL_LAYER [LABEL]}...}*	

	Specifies the list of user-defined nodes for effective resistance calculation between the specified user-defined nodes and all voltage sources. You can add a list of nodes based on the region on a given layer. `LABEL` is any name chosen by you to identify node entries in the list. This is optional but helps during debugging. **Note**: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This means that you can specify `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.
`-node_list_file` *filename*	

	Specifies the name of the node list file. This file contains the list of user-defined nodes in the same format as mentioned for the `-node_list` parameter. The file allows you to specify a large number of nodes.
	You can specify multiple nets for node list based resistance analysis. The use model for multiple nets is:
	`report_resistance -domain <`*domain name*`> -node_list_file <`*filename*`>`
	Here,
	• the `-domain` parameter will specify the domain name, that is, the domain with nets that is required to be analyzed.
	• the `-node_list_file` parameter will include the multiple net names
	The format for specifying multiple nets is:
	`NET <netname1>`
	`node1X node1Y METAL_LAYER [LABEL]`
	`node2X node2Y METAL_LAYER [LABEL]`
	`NET <netname2>`
	`node1X node1Y METAL_LAYER [LABEL]`
	`node2X node2Y METAL_LAYER [LABEL]`
	Note: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This means that you can specify `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.

`-node_pair_list {`*node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2...LABEL*`}`	

	Specifies the node pair list for effective resistance calculation between two nodes on the power-grid. `You can add a list of nodes based on the region on a given layer. LABEL` is any name chosen by you to identify node entries in the list. This is optional but helps during debugging. **Note**: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This means that you can specify `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.
`-node_pair_list_file` *`filename`*	

	Specifies the name of the node pair list file. This file contains the node pair list in the same format as mentioned for the `-node_pair_list` parameter. The file allows you to specify a large number of node pairs.
	You can specify multiple nets for node pair list based resistance analysis. The use model for multiple nets is:
	`report_resistance -domain <domain name> -node_pair_list_file <filename>`
	Here,
	• the `-domain` parameter will specify the domain name, that is, the domain with nets that is required to be analyzed.
	• the `-node_pair_list_file` parameter will include the multiple net names
	The format for specifying multiple nets is:
	`NET <netname1>`
	`node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2 [LABEL]`
	`node3X node3Y METAL_LAYER3 node4X node4Y METAL_LAYER4 [LABEL]`
	`NET <netname2>`
	`node1X node1Y METAL_LAYER1 node2X node2Y METAL_LAYER2 [LABEL]`
	`node3X node3Y METAL_LAYER1 node4X node4Y METAL_LAYER4 [LABEL]`
	Note: The `-node_list` and `-node_list_file` parameters are mutually exclusive to the `-node_pair_list` and `-node_pair_list_file` parameters. This means that you can specify `-node_list` and `-node_list_file` together but you cannot specify `-node_list` and `-node_pair_list` together.

`-output_file` *`filename`*	
	Specifies the name of the output report file that contains the report header and the sorted list for resistance values. **Note**: This parameter should not be specified in the XP flow as the XP reports are consolidated under the state directory by default.

`-output_dir` *`directory_name`*	
	Specifies the name of the directory in which the output report file is saved.
`-region {`*`x1 y1 x2 y2 layer number_of_nodes region_name`*`}`	
	Analyzes resistance in the specified region. You can draw a box on a specific layer, and specify the number of nodes to be selected (maximum limit of 100 nodes). This parameter allows you to compute effective resistance on a given layer across the chip and determine how the resistance is changing for the design from top to bottom layer. You can assign a name for the region.
`-report_limit` *`value`*	
	Specifies the number of instances printed in the report to save disk space.
`-threshold value`	
	Specifies the minimum allowed effective resistance to display node/node-pair/instance/instance-pair that have a resistance value above this threshold value. If you specify threshold as a non zero value, then instances with resistance value above that threshold will be tagged as FAIL and the rest will be tagged as PASS.
`-write_cell_name`	
	Specifies to print cell names in the effective resistance report. This parameter allows you to sort resistance values by cell names.

Examples

- The following command performs effective resistance calculation for the list of nodes and instances:
  ```
  report_resistance -net_name VSS -node_list { 231 1061.6 M1 } -instance_list_file
  instance.list
  ```
- The following command specifying a list of user-defined nodes and a node list file for effective resistance calculation:
  ```
  report_resistance -net_name VDD -node_list {{ 213.556 133.469 M1  I123 }} -node_list_file
  /../node_list.txt.1 -output_file reff_rpt
  ```
- The following is an example of specifying a list of instances for effective resistance calculation:
  ```
  report_resistance -instance_list {{INV11 VDD} {INV21 VDD}} -net_name VDD
  ```
- The following is an example of specifying a list of user-defined nodes for effective resistance calculation:
  ```
  report_resistance -net_name VDD -node_list {{800 11.88 M4} {999.768 11.88 M4} {1000.296
  11.880 M4}} -output_file out.txt
  ```

- The following is an example of specifying node pair list for effective resistance calculation:
  ```
  report_resistance -net_name VDD -node_pair_list {195.728 133.369 M1 213.556 133.469 M1} -
  output_file out_nodelist.txt
  ```
- The following is an example of specifying instance pair list for effective resistance calculation:
  ```
  report_resistance -net_name GND -instance_pair_list {inst1 inst2 GND GND} -output_file
  outfile -threshold 100
  ```
- The following command performs effective resistance calculation for full chip analysis:
  ```
  report_resistance -threshold 13.0  -domain_threshold 25.0 -domain_name ALL
  ```

  Sample Report:

  ```
  NET: VSS VDD
  # Threshold:   25.000
  # Total Effective resistance for instances sorted from high to low
  # PASS/FAIL         REFF  INSTANCE                  PIN...

    FAIL            25.664  clk__L1_I1       VDD(51.52%)      VSS(48.48%)
    FAIL             25.35  clk__L2_I1       VDD(52.16%)      VSS(47.84%)
    FAIL            25.037  clk__L3_I1       VDD(51.56%)      VSS(48.44%)
  ```

- The following command performs effective resistance calculation for instance list analysis:
  ```
  report_resistance -domain_name ALL -threshold 13.0 -domain_threshold 25.0 -
  instance_list_file inst.list
  ```

  Sample Report:

  ```
  NET: VSS VDD
  # Threshold:   25.000
  # Total Effective resistance for instances sorted from high to low
  # PASS/FAIL         REFF  INSTANCE             PIN...
    FAIL            25.664  clk__L1_I1       VDD(51.52%)      VSS(48.48%)
    FAIL            25.35   clk__L2_I1       VDD(52.16%)      VSS(47.84%)
    FAIL            25.037  clk__L3_I1       VDD(51.56%)      VSS(48.44%)
    PASS            24.724  clk__L4_I1       VDD(52.21%)      VSS(47.79%)
    PASS            23.047  inst_pll         VDD(50.55%)      VSS(49.45%)
  ```

- The following command reports reff for all instances of the cell type CKBD8:
  ```
  report_resistance -net_name VDD -cell_name CKBD8
  ```
  The following is a sample reff report generated using the `-cell_name` parameter:

  ```
  *****************

  # Layer Representation: LEF
  # Coordinate Unit: um
  # Resistance Unit: Ohm
  ```

```
NET: VDD
# Threshold:    0.000
# Effective resistance for instances sorted from low to high
# PASS/FAIL       REFF         INSTANCE          PIN
  -              3.5752    add_94_37_I2/g695    VDD
  -              3.7521    add_94_37_I2/g694    VDD
```

Related Topics

- "Static Power, IRdrop and EM Analysis" in *Voltus User Guide*

report_self_heat

```
report_self_heat
[-help]
[-output_directory   dir_name]
[-parameter_file filename]
[-power_domain rail_domain_name]
```

Runs Self-Heating Effect (SHE) analysis on a single die or multiple dies. This command must be specified after defining the SHE properties using the `set_self_heat_analysis_mode` command.

The `report_self_heat` command will use the static power data to compute delta T FEOL for all instances in the design. This command:

- computes the RMS currents on signal wires
- computes the RMS currents on power network wires
- generates Detail Delta Temperature File (ddt), Instance Delta Temperature File (idt), and Tile Delta Temperature File (tdt)

> Self-heat analysis (`report_self_heat`) should be performed for the entire design, that is, you must use domain analysis with all PG nets (`-power_domain ALL`).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_self_heat` parameter. For a detailed description of the command and all of its parameters, use the man command `man report_self_heat`.
`-output_directory   dir_name`	

Specifies the directory name for storing the self-heat analysis results. When you specify this parameter, the following result files are saved in the specified directory:

- Rail analysis output directory
- Detail delta temperature file
- Instance delta temperature file
- Tile delta temperature file

If `-output_directory` is not specified, by default the self-heat analysis results are stored in the current working directory.

`-parameter_file` *`filename`*

Specifies the self-heating effect (SHE) analysis parameter file name (param.sh) required for SHE calculations. This file, provided by the foundry, contains essential alpha and beta coupling factors that the tool uses for coupledT calculations when modeling heat transfer from devices to metal layers.

Sample File:

```
HEADER {
FILE := Thermal_Coupling.ircx
}

BETA_C1_OD := 0.0055
BETA_C2_OD := -0.00065
BETA_C3_OD := 0.85000

* SELFHEAT_TABLE 17 5 (LAYER:V, ALPHA_CONNECT_OD:V, ALPHA_OVERLAP_OD:V,
ALPHA_HIGHR:V, BETA_HIGHR:V)
FIELD          ALPHA_CONNECT_OD   ALPHA_OVERLAP_OD     ALPHA_HIGHR    BETA_HIGHR
LAYER
AP                    0.90              0.30              0.35             0.85
M15                    .90              0.30              0.35
 0.85
M14                   0.90              0.30              0.35             0.85
```

`-power_domain` *`rail_domain_name`*

Specifies the domain name of the rail.

Examples

- The following command is an example of `report_self_heat` command in the multi-die analysis flow:

```
set_self_heat_analysis_mode \
-die_instance_name die0 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} ....} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die0.rpt

set_self_heat_analysis_mode \
-die_instance_name die1 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} ...} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die1.rpt

set_self_heat_analysis_mode \
-die_instance_name die2 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} ...} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die2.rpt

report_self_heat -power_domain core
```

Related Topics

- "Self-Heating Effect Analysis" in the *Voltus User Guide*

report_signal_resistance

```
report_signal_resistance
[-help]
[-instance_pair_list_file filename]
-net net_name
[-output_dir directory]
[-report filename]
[-threshold value]
```

Computes effective resistance for signal nets from a given instance1/pin1 to instance2/pin2 accounting all applicable manufacturing effects. You can use the effective resistance output file to determine whether the critical routes are within the specified resistance limits.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-instance_pair_list_file` *`filename`*	
	Specifies the file name containing the signal net instance-pin pairs for which effective resistance needs to be computed. The format of the file is: `INST1 PIN1 INST2 PIN2`
`-net` *`net_name`*	
	Specifies the name of the signal net.
`-output_dir` *`directory`*	
	Specifies the name of the output directory.
`-report` *`filename`*	
	Specifies the name of the report file in which details of effective resistance between instance-pin pairs are written.
`-threshold` *`value`*	
	Specifies the minimum allowed effective resistance between instance-pin pairs.

Examples

- The following command computes effective resistance for the signal net `n_13190`:

  ```
  report_signal_resistance -net n_13190
  ```

- The following command computes effective resistance for the signal net `n_13190`, and writes the effective resistance details in the report `n_13190.rpt`:

  ```
  report_signal_resistance -net n_13190 -report n_13190.rpt
  ```

- The following command computes effective resistance for signal net instance-pin pairs specified in the file `instance.list`:

  ```
  report_signal_resistance -net n_0 -instance_pair_list_file instance.list
  ```

scale_what_if_capacitance

```
scale_what_if_capacitance
[-help]
[-add]
-reset | {{-global | -region x1 y1 x2 y2 | -cell cell_name | -instance inst_name | -file
filename}
[-grid_capacitance value]
[-intrinsic_capacitance value]
[-layer {layername | all} ]
[-loading_capacitance value]
[-net net_name | all]
}
```

Scales the intrinsic cell capacitance value to determine how much additional capacitance can be added in a region to reduce dynamic IR drop.

Parameters

`-add`	
	Specifies to add capacitance (farads) on the power-grid on a specific layer and region. The parameter works in conjunction with the existing parameters, such as `-intrinsic_capacitance`, `-grid_capacitance`, and `-loading_capacitance`. When this parameter is specified, the value provided with the `-intrinsic_capacitance`, `-grid_capacitance`, and `-loading_capacitance` parameters will be treated as absolute value instead of a scale factor value, which is the default behavior.
`-cell cell_name`	
	Specifies to scale the capacitance value for the specified cell.
`-file filename`	

	Specifies a file containing the list of instances and cells that use the same scaling factor. The format of the file is:
	`CELL cellname`
	`INSTANCE instancename`

`-global | -region x1 y1 x2 y2`	

	You must specify either `-global` or `-region x1 y1 x2 y2`. The `-global` option scales the specified capacitances for the entire design. The `-region x1 y1 x2 y2` scales the specified capacitances in the region. The default unit of `x1 y1 x2 y2` is in database units.
`-instance inst_name`	
	Specifies to scale the capacitance value for the specified instance.
`-layer layername | all`	
	Specifies the LEF layer on which capacitance scaling is performed. Multiple commands can be issued to scale capacitance on multiple layers. You can specify the `all` argument to scale all the layers.
`-net net_name | all`	
	Specifies to scale the capacitance value for the specified net. Alternatively, you can specify the `all` argument to apply the scaling values across all nets simultaneously. With this argument, you can apply what-if capacitance data to all nets at once, rather than being limited to applying scaling values to a single net at a time.
`-help`	Outputs a brief description that includes the type and default information for each `scale_what_if_capacitance` parameter. For a detailed description of the command and all of its parameters, use the man command `man scale_what_if_capacitance`.
`-reset`	Resets all the previous `scale_what_if_capacitance` command settings.
`-grid_capacitance value`	
	Scales power-grid capacitance of the design to the specified scale factor. The default scale value is 1. ⚠ The `-grid_capacitance` parameter can be specified either with the `-global` or the `-region` parameter. This parameter is not supported with the `-cell` and `-instance` parameters.
`-intrinsic_capacitance value`	

	Scales instance internal gate capacitances to the specified scale factor. The default scale value is 1.
`-loading_capacitance` *value*	
	Scales loading capacitance of the instance to the specified scale factor. The default scale value is 1.

Examples

- **Cell-based capacitance scaling** - The following command scales intrinsic capacitance on all instances of the cell *cellA* by 1e-10 (<*intrinsic_cap*> * 1e-10):
  ```
  scale_what_if_capacitance -cell cellA  -intrinsic_capacitance 1e-10
  ```
- **Pin-based capacitance scaling** - The following command scales intrinsic and loading capacitance on all instances of the cell *cellA* connected to the pin *vdd* by 1e-10 (<*intrinsic_cap*> * 1e-10 and <*loading_cap*> *1e-10):
  ```
  scale_what_if_capacitance -cell cellA -net vdd -intrinsic_capacitance 1e-10  -add -loading_capacitance 1e-10
  ```
- **Region-based capacitance scaling** - The following command scales capacitance in the specified region by 1e-10 (<*intrinsic_cap*> * 1e-10, <*loading_cap*> *1e-10, and <*grid_cap*> * 1e-10):
  ```
  scale_what_if_capacitance -region {896.877  937.661  1276.9365  1615.358} -intrinsic_capacitance 1e-10  -loading_capacitance 1e-10  -grid_capacitance 1e-10
  ```
- The following command scales grid capacitance in the specified layer by 1e-10:
  ```
  scale_what_if_capacitance -net VDD -region { 0 0 100 100 } -layer M2 -grid_capacitance 1e-10
  ```
- The following command scales intrinsic capacitance in all layers by 1e-4 for all nets:
  ```
  scale_what_if_capacitance -global -net all -layer all -intrinsic_capacitance 2e-3 -loading_capacitance 1e-4
  ```

Related Topics

- "What-If Rail Analysis" in the *Voltus User Guide*

scale_what_if_current

```
scale_what_if_current
[-help]
[-hierarchy hier_name | -region x1 y1 x2 y2 | -global | -instance_list filename]
[-net net_name]
[-reset]
[-scale_clock_network]
[-scale value | -current value]
```

Scales current to do what-if analysis for a hierarchical partition and assess its effect on IR drop.

Parameters

`-current` *value*	
	Specifies current value in Amperes. This is used as target peak current for the given hierarchy and is then scaled appropriately. You cannot specify this option with -region *x1 y1 x2 y2* or -global options.
`-global`	Specifies current scaling to be applied to entire design.
`-help`	Outputs a brief description that includes the type and default information for each `scale_what_if_current` parameter. For a detailed description of the command and all of its parameters, use the man command man `scale_what_if_current`.
`-hierarchy` *hier_name*	
	Specifies the name of the hierarchical block. For example, -hierarchy top/blockA. To specify global scaling for the design use "/". For example, -hierarchy / `-hierarchy` is a required parameter. This parameter allows instance-based current scaling by the specified scale factor. The parameter can be used to scale down peak current values. The use model of instance current scaling is: `scale_what_if_current -hierarchy <`*instance_name*`> -scale <`*value*`>`
`-instance_list` *filename*	

	Specifies to scale current for the specified nets. This parameter allows you to specify a file containing the list of instances and their scale factor. **Use Model** `scale_what_if_current -instance_list` *`filename`* `-scale` *`val`* The format of *`filename`* is: *`instanceName scaleFactor`* *`instanceName scaleFactor`* Here, • *`val`* means the global scale factor. This value will be used if the local or instance-specific scale factor is not specified. • *`scaleFactor`* means the local scale factor. If both global and local scale factors are specified, the local scale factor takes precedence. Default scale factor value is `1`.
`-net` *`net_name`*	
	Scales current of the specified net.
`-region` *`x1 y1 x2 y2`*	
	Specifies a region where current scaling is to be applied. The default unit of x and y coordinates is in microns (um).
`-reset`	Resets all the previous `scale_what_if_current` command settings.
`-scale` *`value`*	
	Specifies any value greater than 0. For example, a value of 0.5 will scale the current in half. This is a required parameter.
`-scale_clock_network`	
	When specified, scales current for all instances within the given hierarchy. When not specified, the software does not scale the instances that belong to the clock network. It scales current for the rest of the instances within the given hierarchy to the specified value. You cannot specify this option with -region *x1 y1 x2 y2* or -global options. This is an optional parameter.

Examples

- The following command scales current for the hierarchical block `BLOCK_C`:
  ```
  scale_what_if_current -hierarchy top/BLOCK_C -current 0.5
  ```

Related Topics

- "What-If Rail Analysis" in the *Voltus User Guide*

scale_what_if_resistance

```
scale_what_if_resistance
[-help]
-reset | {-net netname
{-global | -region x1 y1 x2 y2 | -instance instancename | -cell cellname | -file filename}
[-auto_scale_adjacent_via_layers {true | false}]
[-layer {layername | all}]
[-scale value | -scale_ron {R_ON_scale_factor | R_OFF}]
}
```

Scales grid resistance globally or by region, or scales power switch resistance. In designs with power switches, the scaling of power switch resistance will be applied to cells in the always-on net as well as its switched nets in a given region.

Parameters

`-auto_scale_adjacent_via_layers {true|false}`	
	Specifies to use auto via scaling for what-if resistance analysis. When scaling resistance on a specific metal layer, via resistance on the adjacent via layers are automatically scaled with the same scaling factor. The default value is `true`. Use `-auto_scale_adjacent_via_layers false` to keep via resistances un-scaled.
`-global | -region x1 y1 x2 y2 | -instance instancename | -cell cellname | -file filename`	

	You must specify either `-global`, `-region`, `-cell`, `-instance`, or `-file`. • When the `-global` parameter is specified with the `-scale` parameter, it scales resistance by the specified value globally for a given layer. If the `-global` parameter is specified with the `-scale_ron` parameter, it scales the resistance value of all power switch cells globally for a given layer. This is the default. • When the `-region` parameter is specified with the `-scale` parameter, it scales the grid resistance in the specified region. If the `region` parameter is specified with the `-scale_ron` parameter, it scales the resistance of all power switch cells in the specified region. The default unit of x1 y1 x2 y2 is in database units. • The `-instance` *`instancename`* or `-cell` *`cellname`* parameter specifies to scale the internal resistors from the PGV for the specified instance or cell. • The `-file` *`filename`* parameter specifies a file containing the list of instances and cells that use the same scaling factor. The format of the file is: `CELL` *`cellname`* `INSTANCE` *`instancename`* This is a required parameter. Default is `-global`. The precedence for resistance scaling is as follows: 1. Instance 2. Cell 3. Region 4. Global
`-help`	Outputs a brief description that includes the type and default information for each `scale_what_if_resistance` parameter. For a detailed description of the command and all of its parameters, use the man command man `scale_what_if_resistance`.
`-layer` *`layername`* `|` `all`	
	Specifies the layer name (LEF layer or technology layer) on which the scaling is performed. Multiple commands can be issued to scale resistance on multiple layers. You can specify the `all` argument to scale all the layers. When the `all` argument is used, the `-auto_scale_adjacent_via_layer` parameter is disabled. This is a required parameter.

`-net` *`netname`*	
	Scales resistance of the specified net in the domain which is being analyzed. This is a required parameter. ⚠ Specify always-on net name when scaling powergate on resistance.
`-reset`	Resets all the previous `scale_what_if_resistance` command settings.
`-scale` *`value`*	
	Specifies any value greater than 0. For example, a value of 0.5 will scale the resistance in half. This is a required parameter.
`-scale_ron {`*`R_ON_scale_factor`* `| R_OFF}`	
	Specifies the value used to scale the RON of the power-gate instance/cell. The default value is `1`. You can also specify the "`R_OFF`"(case insensitive) keyword to apply the off state power-gate resistance for the specified instance/cell.

Related Topics

- "What-If Rail Analysis" in the *Voltus User Guide*

set_advanced_package_options

```
set_advanced_package_options
[-help]
-reset  |
{[-tool_path <binary_path>] [-net_mapping_file <filename>]}
```

Specifies the path to the bin folder of package extraction and analysis tools. Also, a net mapping file needs to be specified. The mapping file is a simple two column format containing the die net name and the corresponding package net names. After running this command, use the `extract_package` command to start package model extraction.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-tool_path <binary_path>`	
	Specifies the full path to the bin folder of the package extraction and analysis tools (XtractIM and PowerDC).
`-net_mapping_file <filename>`	
	Specifies the name of the two column mapping file that contains the mapping between the net names of package with the net names of die.
`-reset`	
	Resets all specified options back to default values.

Example

- The following example specifies the path to the bin folder of package extraction and analysis tools:

```
set_advanced_package_options \
   -tool_path /home/sigrity/tools/bin/ \
   -net_mapping_file net_mapping.txt
```

set_advanced_rail_options

```
set_advanced_rail_options
[-help][-reset] | [[-voltus_rail_include_file_begin file_name1]
[-voltus_rail_include_file_end file_name2] ]
```

Provides the ability to include a file that provides a list of additional rail options that should be used for rail analysis. If you do not specify values to the set_advanced_rail_options command parameters, the parameter values are set to the default value, that is, empty file name.

For more information on these options, refer to appendix B: Rail Analysis Options of the *Voltus Text Command Reference*.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-reset`	Resets parameters to their default values. This means that an extra include file is not required.
`-voltus_rail_include_file_begin` *file_name*	
	Provides the ability to provide a file that includes additional rail options that will be run prior to the `analyze` command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.
`-voltus_rail_include_file_end` *file_name*	
	Provides the ability to provide a file that includes additional rail options that will be run after the `analyze` command. This can be used for those legacy rail analysis commands that do not have a Voltus equivalent.

Examples

- The following command needs to be set to override the default precision digits:

  ```
  set_advanced_rail_options -voltus_rail_include_file_begin vstorm.file


  <vstorm.file>
  setvar precision_digits 10
  ```

  With this variable, you will get IR with 10 precision digits (10 includes all digits before and after the decimal point).

set_die_model

```
set_die_model
[-help]
-spice_model_file model_file
[-static_current {initial | average}]
[-sub_circuit_name   subcircuitname]
{-reset  | [[-die_instance_name dieinstname [-spice_model_file model_file [-mapping_file
mapping_file]]] ]
}
```

Specifies to import a die model on die Interposer or on package netlist.

This command gives information about the die model that will be used in the rail analysis of another die, wherein the two die are connected through die Interposer die or package netlist. For example, if you are performing IR drop analysis for a die (**die2**) on an Interposer/package design that is connected to another die (**die1**), and if the two die share the power/ground network, it is imperative to import the die model of **die1** during IR drop analysis of **die2**.

You can import multiple die models by specifying multiple `set_die_model` commands.

Parameters

`-die_instance_name` *dieinstname*	
	Mandatory parameter. Specifies the die instance name for die model. This is the die model of the die that needs to be imported.
`-help`	Outputs a brief description that includes the type and default information for each `set_die_model` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_die_model`.
`-mapping_file` *mapping_file*	

	Mandatory parameter. Specifies the name of the mapping file that contains the port connection mapping between the die model and voltage source of the chip to be analyzed.
	The two mapping file formats are:
	• **Mapping when die model on LEF/DEF-based Interposer Die**
	`#port_of_die_model cell die_name/VSRC_pad_name`
	`VDD_Bump_VDD_9_0 cell Interposer/Bump_DIE0_VDD_0`
	`VDD_Bump_VDD_9_0 cell Interposer/Bump_DIE0_VDD_1`
	...
	• **Mapping when die model on Package Design**
	`#port_of_die_model pkg port_of_pkg_netlist`
	`VDD_Bump_VDD_9_0 pkg VDD_3`
	`VDD_Bump_VDD_9_0 pkg VDD_4`

`-reset`	Resets all the previous `set_die_model` command settings to the default settings.
`-spice_model_file` *`model_file`*	
	Mandatory parameter. Specifies a file containing a SPICE subcircuit describing the die model.
`-static_current {initial | average}`	
	Default: `initial` Specifies to use the average of PWL waveform as the static current for static rail analysis. By default, only the current value at `t=0` from the PWL waveform is considered for static run. Example: `set_die_model -static_current average`
`-sub_circuit_name` *`subcircuitname`*	
	Defines the top-level subcircuit in package files. When this parameter is not specified, the first circuit in the Spice file is used as the default top circuit.

Examples

- The following command imports two die models `Die1` and `Die2`:

```
set_die_model -die_instance_name Die1 -spice_model_file die1_model.ckt -mapping_file
die1_port.txt
set_die_model -die_instance_name Die2 -spice_model_file die2_model.ckt -mapping_file
die2_port.txt
```

Related Topics

- "Package Analysis" in the *Voltus User Guide*

set_dynamic_rail_simulation

```
set_dynamic_rail_simulation
[help]
[-auto_repeat]
[-reset]
[-resolution value]
[-start_time value]
[-stop_time value]
```

An optional command to set the parameters for dynamic rail analysis. By default, dynamic rail analysis uses the start time, stop time, and resolution from current data file. To analyze fewer clock cycles, you can choose different start and stop times using the `-start_time` and `-stop_time` options. It is recommended that at least one or two full cycles are simulated to fully charge and discharge the on-chip capacitance.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_dynamic_rail_simulation` parameter. For a detailed description of the command and all of its parameters, use the man command man `set_dynamic_rail_simulation`.
`-auto_repeat`	

	Specifies to repeat the dynamic current waveform till the `-stop` time specified. If `-stop` is not specified, all the waveforms will repeat till the longest waveform duration. The `-auto_repeat` parameter can be used when multiple dynamic current files have different simulation periods or when IR drop analysis needs to be run for longer duration.
	The following command sets the dynamic simulation time from 20ns to 100ns in which the current data will be repeated until 100ns is reached:
	`set_dynamic_rail_simulation -start 20ns -auto_repeat -stop 100ns`
	The `-repeat` parameter of the `set_power_data` command has higher priority than `set_dynamic_rail_simulation -auto_repeat`, as shown in the following example:
	`set_power_data  VDD1.ptiavg -repeat 50`
	`set_power_data  VDD2.ptiavg`
	`set_dynamic_rail_simulation -auto_repeat -stop 100`
	In this example, VDD1 waveform will repeat till 50ns, while VDD2 waveform will repeat till 100ns.

`-reset`	Resets all the previous `set_dynamic_rail_simulation` command settings to the default settings.
`-resolution` *value*	
	By default rail analysis picks up the resolution (or step size) saved in the dynamic current files. To achieve better rail analysis performance, you can increase the step size using this option. However, using larger than two times the step size of current data file is not recommended as it can impact accuracy of dynamic IR drop analysis. You can specify the units with the value. If you do not specify a unit, the software uses the default unit. *Default*: Picoseconds (ps).
`-start_time` *value*	
	Defines the start time to be used for dynamic rail analysis. This option is used with the `-stop_time` option and enables you to perform a more detailed analysis in specific time ranges. You can specify the units with the value. If you do not specify a unit, the software uses the default unit. *Default*: The default unit is nanoseconds (ns). The default value is the start time from current data file. The start time in the current data file is usually 0ns.
`-stop_time` *value*	

	Defines the stop time to be used for dynamic rail analysis. This option is used with the `-start_time` option and enables you to perform a more detailed analysis in specific time ranges. You can specify the units with the value. If you do not specify a unit, the software uses the default unit. *Default*: The default unit is nanoseconds (ns). The default value is the end of the current data. If multiple current files are specified and they all have different simulation periods, the largest simulation period is used to decide the stop time.

Examples

- The following command sets the dynamic simulation time from 2ns to 12ns with a resolution of 100ps:
  ```
  set_dynamic_rail_simulation -start_time 2ns -stop_time 12ns -resolution 100ps
  ```

set_multi_die_analysis_mode

```
set_multi_die_analysis_mode
[-help]
-analysis_name analysis_name
-die_list {die_instance_name1 die_instance_name2}
[-report_rlrp_path_instance_list filename]
[-stacked_die_mapping_file Innovus_map_file]
```

Specifies how the multi-die analysis will be performed.

Parameters

-help	Outputs a brief description that includes the type and default information for each set_multi_die_analysis_mode parameter. For a detailed description of the command and all of its parameters, use the man command: man set_multi_die_analysis_mode
-die_list *die_instance_name1 die_instance_name2*	
	Specifies the die instance names.
-analysis_name *analysis_name*	
	Specifies the name of the analysis.
-report_rlrp_path_instance_list *filename*	

Specifies a file containing a list of die instances (one per line) for which the RLRP path should be reported. This parameter is used to generate the RLRP path across multiple die.

The format of the specified input file is:
`<die_name> <net_name>`

The `-report_rlrp_path_instance_list` parameter generates a report `mdie_inst_rlrp_path.rpt` at the following location:
`<output_dir>/<state_dir>/results/rail/.`

The format of the report is:

```
Die Name: <die_name>
Net Name: <net_name>
Pin Name: <pin_name>
Instance Name: <inst_name>
Die   Layer/Via   Resistance(cumulative)   Ohm Length   Node1#(X,Y)
Node2#(X,Y)   Net
```

Example of an Output File:

```
Die Name: LDIE1
Net Name: VDD_AO
Pin Name: TVDD
Instance Name: ring/PSO_RING_78

Die Layer/Via    Resistance(cumulative) Ohm Length Node1#(X,Y) Node2#
(X,Y) Net
LDIE1   METAL_1   0.1238(3.4182)           2.210   169.28500,233.27500
167.07500,233.27500 VDD_AO
LDIE1   METAL_1   0.1302(2.7137)           2.035   169.28500,221.89000
169.28500,223.92500 VDD_AO
LDIE1   VIA_1     0.0269(2.5835)           0.140   169.28500,221.89000
169.28500,221.89000 VDD_AO
LDIE1   METAL_2   0.0070(2.5565)           0.145   169.28500,221.74500
169.28500,221.89000 VDD_AO
LDIE1   VIA_2     0.0875(2.5496)           0.140   169.28500,221.74500
169.28500,221.74500 VDD_AO
LDIE1   VIA_3     0.0875(2.4621)           0.140   169.28500,221.74500
169.28500,221.74500 VDD_AO
...
```

`-stacked_die_mapping_file Innovus_map_file`

Specifies the name of the inter die data mapping file from Innovus. The mapping file is a virtual mapping that provides the connections between any 2 stacked die with communication directly between the die (master/slave configuration).

Examples

- The following command specifies an analysis name called `tsv`, two die instances called `mother_die` and `daughter_die`,and a stacked mapping file `pwr.mapping`:

```
set_multi_die_analysis_mode -analysis_name tsv -die_list {mother_die daughter_die}
-stacked_die_mapping_file ../data/pwr.mapping
```

Related Topics

- "Through-Silicon Via and System-in-Package" in the *Voltus User Guide*

set_net_group

```
set_net_group
[-help]
-reset | {-name group_name
-type [power | ground]
-nets {net_list}
}
```

Specifies the list of either power or ground nets that are isolated but need to be analyzed together as a single net. The results are generated for the new grouped net. The new grouped net name must be one of the net names in the group. To analyze the grouped net, you must also specify power pads for the grouped net name.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_net_group` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_net_group`
`-name group_name`	
	Name of the net group. **Note:** The name of the net group must be one of the net names in the group. Using a net name that is not in the net group will trigger error when displaying results in the flow.
`-nets {net_list}`	
	Specifies a list of power or ground nets.
`-reset`	Resets all previously created net name(s) with the new grouped net name(s).
`-type [power | ground]`	
	Specifies whether it is a power or ground net.

Examples

- The following example groups the power nets that will be analyzed together as a group called VDD.

```
set_net_group -name VDD -type power -nets { VDD VDD1 VDD2 VDD3 }
set_power_pads -net VDD -format xy -file VDD.pp
report_rail -type net VDD
```

set_off_chip_package_trace

```
set_off_chip_package_trace
[-help]
-mapping_file mapping_file
-name switched_name
-pad_location_files list_of_bump_location_files
[-reset]
-spice_model_file spice_model_name
[-subckt subcircuitname]
```

Specifies to include RDL0 (off-chip package trace) resistance effects during the static and dynamic rail analysis. This flow allows you to analyze switched net bumps that are marked as RDL0 bumps. When this command is specified, the software automatically generates switched net bump current and bump voltage waveforms in the `voltus_rail_pad.tran.ptiavg` file created in the new switch net directory located under the rail state directory.

You can specify multiple `set_off_chip_package_trace` commands to define connectivity of different bump nets.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_off_chip_package_trace` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_off_chip_package_trace`.
`-mapping_file` *`mapping_file`*	
	Specifies the name of the mapping file that contains the mapping between the package port and switched net bump names. The format of the file is: `<spice_circuit_port_name> pin <vsrcName_in_switch_net's_ploc>`
`-name` *`trace_model_name`*	
	Specifies the trace model name or switched net bump of RDL0.
`-pad_location_files` *`list_of_bump_location_files`*	

	Specifies the name of a file containing switched net names and associated ploc/power pad files. The format of the file is:
	`SWITCHED_NET1 plocfile1`
	`SWITCHED_NET2 plocfile2`
	`…..`
	The format of plocfile (power pad) is:
	`<voltage_source_name> <x> <y> <metal_layer>`
	The format of this file is same as specified for the `set_power_pads -format xy` format.

`-reset`	Resets all the previous `set_off_chip_package_trace` command settings to the default settings.
`-spice_model_file` *`model_file`*	
	Specifies a file containing a SPICE subcircuit (RDL0 netlist or full-chip package netlist) for off-chip package trace.
`-subckt` *`subcircuitname`*	
	Defines the top-level subcircuit for off-chip package tracing.

Examples

- The following command specifies to analyze the switched net `VDDau`:

```
set_off_chip_package_trace -name VDDau  -pad_location_files  pad_location_files.txt -
spice_model_file pkg.ckt -mapping_file pkg_mapping.file
```

Related Topics

- "Package Analysis" in the *Voltus User Guide*

set_package

```
set_package
[-help]
-reset | {-spice_model_file model_file
[-die_instance_name dieinstname]
[-flip {true | false}]
[-mapping_file mapping_file]
[-mapping_type { distributed | lumped }]
[-offset {x y}]
[-rotate_angle rotation_angle_in_degrees]
[-scale scaling_factor_number]
[-sub_circuit_name subcircuitname]
}
```

Specifies the package model information to be used in power rail analysis.

Parameters

`-die_instance_name` *dieinstname*	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode multi-die`).
`-flip {true | false}`	
	Specifies to flip the die along the Y axis. Default value is `false`.
`-help`	Outputs a brief description that includes the type and default information for each `set_package` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_package`.
`-mapping_file` *mapping_file*	

Specifies the name of the package terminal mapping file, which contains the mapping of the terminals of the SPICE subcircuit in the package model file (Specified by the `-spice_model_file` *`model_file`* parameter) to the locations at the grid or top-level pins.

The mapping file formats supported by the software are:

- Model Connection Protocol (MCP)
- Voltus-specific Format

For more information on the mapping file formats, refer to the *Package Model and Mapping File* section in the "File Formats" chapter of *Voltus User Guide*.

`-mapping_type { distributed | lumped }`

Specifies the type of mapping to be used in the Voltus-specific terminal mapping file that will be added in the rail analysis flow. The possible arguments are:

- `distributed` – A distributed mapping file lists out the mapping for each bump on the die to its corresponding package port. The mapping file is specified with the `set_package -mapping_file` parameter.
- `lumped` - A lumped mapping file allows for auto-grouping of the die bumps to the lumped package ports. When `lumped` is specified, the software implements a lumped package by internally creating a lumped package model and its mapping file.

  The format of the lumped mapping file is:

  ```
  <terminal_name>   <die | vsrc>    <net_name>
     vddcore_in         die            vddcore
     vddcore_out        vsrc           vddmpu
  ```

  In this format:

  - the software connects "`<terminal_name> die <net_name>`" to the bumps belonging to `<net_name>` on the die
  - the software connects "`<terminal_name> vsrc <net_name>`" to the voltage source of `<net_name>`
  - The number of lines in the lumped mapping file are few and are equal to the number of terminals on the package model

Default: `distributed`

`-offset {x y}`	
	Specifies to adjust and shift the X Y coordinates in the MCP header for each pin. When performing package analysis, the MCP is used to map the terminals of the SPICE subcircuit in the package model file to the locations at the grid or top-level pins in DEF. The die origin specified in MCP must be same as specified in DEF. If the die origin is not the same, you can use the `-offset` parameter to modify the XY location specified in the MCP file.
`-reset`	Resets all the previous `set_package` command settings to the default settings.
`-rotate_angle` *`rotation_angle_in_degrees`*	
	Specifies the rotation angle of the die. The possible values are: `0/45/90/135/180/225/270/315`. Rotation is always in the counter clockwise direction from the original point. Default value is `0`. This information is required to do coordinate conversion for xy based pin mapping.
`-scale` *`scaling_factor_number`*	
	Specifies to scale the package model.
`-spice_model_file` *`model_file`*	
	Specifies a file containing a SPICE subcircuit describing the package model.
`-sub_circuit_name` *`subcircuitname`*	
	Defines the top-level subcircuit in package files.

Related Topics

- "Package Analysis" in the *Voltus User Guide*
- "File Formats" in the *Voltus User Guide*

set_pg_nets

```
set_pg_nets
-reset |
[-die_instance_name dieinstname]
-force
[-lib_binding_voltage value]
-net net_name
[-package_net_name package_net_name]
[-package_voltage_drop_limit value]
[-threshold value]
-tolerance value
-voltage value
```

Specifies the power grid nets. Using this command, you can specify power/ground net voltages in the static and dynamic power analysis flow, and the rail analysis flow.

> For the power analysis flow, you must specify the `set_pg_nets` command before `report_power`.

Note: When specifying a ground net with a non-zero nominal operating voltage value, a threshold greater than the ground value must be specified in order for the net to be treated as the ground net.

Parameters

`-die_instance_name dieinstname`	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode multi-die`).
`-force`	Skips the validity check on power nets.
`-help`	Outputs a brief description that includes the type and default information for each `set_pg_nets` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_pg_nets`.

`-lib_binding_voltage` *value*	
	Specifies the voltage to be used for library binding in power analysis. This parameter binds the instances to the library that is closest to the user-specified library binding voltage.
`-net` *net_name*	
	Specifies the name of the power or ground net that you want to analyze.
`-package_net_name` *package_net_name*	
	Specifies the net name on the package side. By default, the package net name is same as the net name.
`-reset`	
	Specifies to clear all previously specified values for this command. This parameter can be used to honor the power and ground voltages defined in the Innovus database without impacting the regular EM-IR analysis.
`-threshold` *value*	
	Specifies the minimum allowed voltage on the power net or the maximum rise on the ground net, depending on the setting of the `-voltage` parameter. The `-threshold` parameter is used to verify that the power grid passes the limit check and to set the filters for an IR plot.
`-tolerance` *value*	
	Specifies the tolerance range of the voltage value. Specify a value from 0 to 1 (not including 1). A value of 0.2 results in a range of "voltage +20%" to "voltage -20%." *Default*: 0.3.
`-voltage` *value*	
	Specifies the voltage on the power rail to be analyzed.
`-package_voltage_drop_limit` *value*	
	Specifies the voltage drop limit for a net. This parameter allows you to specify the voltage threshold value for each net separately. The specified threshold value is used for the rail reports, histograms, and plots. If the threshold value is not specified for some of the nets being analyzed, the default range will be applied to those nets. *Default Range*: 0mV to max VU

Examples

- The following command specifies a VDD_TDSPCore value of 0.9 volts with a threshold of 0.89V and a tolerance of 0.3:

  ```
  set_pg_nets -net VDD_TDSPCore -voltage 0.9 -threshold 0.890 -tolerance 0.3
  ```

- The following command specifies to use the library binding voltage for binding the libraries to the instances instead of "`-voltage`":

  ```
  set_pg_nets -net vdd -voltage 0.90 -lib_binding_voltage 0.75
  ```

set_power_data

```
set_power_data
-reset | {-format {current | ascii | area | ascii_current}
[-bias_voltage value]
[-cell cellname]
[-die_instance_name dieinstname]
[filename]
[-instance_name instance_name]
[-offset offset_value]
[-power value]
[-power_directory dir]
[-power_output_directory dir]
[-repeat_time time]
[-repeat_window {start end}]
[-scale factor]
[-strip_hierarchy_instance_name instance]
}
```

Specifies the static and dynamic power or current files for IRdrop and EM analysis. The power data for the design can be generated using the `report_power` command. This command also has ability to read external ASCII static power files to do static rail analysis. For dynamic analysis, you must use the `report_power` command to generate dynamic current files.

This command should be specified prior to the `report_rail` command. You must specify the `set_power_data` command after the `report_power` command in the flow script if power data is not available.

Parameters

`-bias_voltage value`	
	Specifies the value of the bias voltage. This parameter must be set if -format is set to ascii or area.
`-cell cellname`	
	Specifies power from the block-level run to multiple instances of the specified block, so you need not to mention power for each instance separately. For example, `set_power_data -cell BlockA -format current {file1, file2, file3 ...}`

`-die_instance_name` *dieinstname*	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode multi-die`).
filename	Specifies the name of the power data files. The file is optional if -format area is specified.
`-format {current | ascii | area | ascii_current}`	
	Specifies the format of the power data. The power data format can be `current, ascii`, or `area`. The parameter `-bias_voltage` must be set if `area` or `ascii` is specified. • When using the current format option, specify the current file (*.ptiavg) that is generated by the power analysis software within Voltus. For example, `set_power_data -format current static_VDD.ptiavg`

	• When using the `ascii` format option, specify the ascii instance based power file, generated by the Power Calculation engine within Voltus. For example,
	`set_power_data -format ascii -bias_voltage 1.08 powermeter.pwr`
	Note: The command accepts an ASCII power file in three-column format consisting of *instance/cell name*, *power* in W,and *power pin name*. Lines starting with "#" or "*" are treated as comment lines.
	The format of the three-column input file is as follows:
	`<instance/cell name> <power in Watts> <power pin name>`
	Example:
	`top/inst1 0.001 vdd`
	`top/inst2 0.001 vdd1`
	`top/inst3 0.003 vdd3`
	where vdd, vdd1, and vdd3 are power pin names of the instance
	Although `-bias_voltage` is a required argument with `-format ascii`, when 3-column power file format is specified, the program will ignore `-bias_voltage` and will rely on associated net voltages to determine current for the pins.

	• When using the `area` format option, specify the total power of the chip in Watts. This power is distributed based on cell area. Optionally, you can specify the known power for macro cells using a file, for example, `set_power_data -format area -bias_voltage 1.08 -power 10 cell_power.file` where, `-bias_voltage` is the voltage at which power was characterized; If the analysis voltage is different, this power will be scaled accordingly. `cell_power.file` is in two column format with *`cell_name`**power_in_watts*.
	• When using the `ascii_current` format option, specify an ASCII instance current file for static analysis. The format of the three-column input file is as follows: *`<instance/cell name> <current in Ampere> <power pin name>`*
`-help`	Outputs a brief description that includes the type and default information for each `set_power_data` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_power_data`.
`-instance_name` *`instance_name`*	
	Specifies the hierarchical instance name for which the current files are included. This option is useful when power is calculated for hierarchical partitions of the design, enabling you to read power for the hierarchical partitions at the top-level. For example, `set_power_data -format current -instance_name xy {dynamic_VDD.ptiavg dynamic_VSS.ptiavg}`
`-offset` *`offset_value`*	

	Specifies the start time offset for current waveforms saved inside the current files. The default unit is in seconds; However, you can also specify the unit with the offset time value. This option is useful in the hierarchical flow to align current waveforms of various partitions. It can be also used to shift power-up rush current waveforms generated by the UltraSim software. For example, `set_power_data -format current -offset 1ns -instance_name xy {dynamic_VDD.ptiavg dynamic_VSS.ptiavg}`
`-power` *value*	
	Specifies the total power of the chip in Watts and is used with -format area option.
`-power_directory` *dir*	
	Specifies to read the power data from the specified directory for cell power. You can use this parameter to read `power_results.json` and automatically identify all the PTI files. If `power_results.json` is not found in the specified directory, then the power data could be retrieved from `dir/voltus_power_output.latest`. When specified, the `-power_directory` parameter takes precedence and bypasses the `-format` parameter. For example, `set_power_data -cell top_design -power_directory ./power`
`-power_output_directory` *dir*	
	Specifies the name of the directory that contains the current files required to run power analysis in parallel with rail analysis.
`-repeat_time` *time*	
	Repeats the dynamic current waveform for the specified time. You must ensure that `set_dynamic_rail_simulation -stop` is also specified and it matches the repeat duration. This option is only available for current format files during dynamic analysis. The repeat option can be used when multiple current files have different simulation periods or when IR drop analysis needs to be run for longer duration. See example below. *Default units*: ns.
`-repeat_window {`*start end*`}`	

	Specifies to extract the given window from the current waveform and repeat it. You can pick a window such that the current at the `start_time` and `end_time` are similar. This parameter allows you to identify and repeat windows where the start and end time have similar values to ensure that the repeated waveform is continuous and that there are no high current values. The unit for the start and end time is ns. The following command specifies that the pattern between {20ns 80ns} will be repeated until simulation stops: `set_power_data -repeat_window {20ns 80ns}`
`-reset`	Overwrites previously specified current data.
`-scale` *factor*	
	Specifies a scale factor to apply to the current data. You can use this parameter to scale currents to a more optimistic or pessimistic power-consumption estimate. This parameter is only for `-format current` and `-format ascii`.
`-strip_hierarchy_instance_name` *instance*	
	Specifies to use power generated from the top-level rail analysis for block-level rail analysis. This parameter allows you to specify the top-level hierarchical instance name for which the current files are to be included. When the parameter is specified, the tool removes the prefix from the top-level and reads the top-level current (ptiavg) files during the block-level analysis. The parameter is useful when top-level power data is already available, therefore, you do not need to re-run block-level power analysis.

Examples

- The following command specifies a format of type current and a scale of 1 for the three given files.:

```
set_power_data -format current -scale 1 {static_VDD_TDSPCore.ptiavg \
static_VDD_TDSPCore_R.ptiavg static_VSS.ptiavg}
```

- The next example specifies two current data files. The first file specifies vdd_vdd.ptipeak. The second one specifies dynamic_VDD.ptiavg and this current data will be repeated every 100ns, which is the stop time specified for the dynamic analysis.

```
set_power_data -format current vdd_vdd.ptipeak
set_power_data -format current -repeat_time 100ns dynamic_VDD.ptiavg
set_dynamic_rail_simulation -stop 100ns
report_rail ...
```

set_power_pads

```
set_power_pads
[-help]
-reset | {
[-auto_voltage_source_creation {true|false}]
[-consider_partial_small_tiles_for_vsrc {true | false}]
[-def_shape_only {true | false}]
[-die_instance_name dieinstname]
[-enable_switchnet_ploc_generation {true | false}]
[-format {defpin | padcell | xy | boundary | xyiv}]
[-layer vialayername]
[-net net_name | all]
[-number_of_voltage_source value]
[-file file]
[-region {x1 y1 x2 y2}]
[-short_pin_nodes {true | false}]
[-tile {x y}]
}
```

Specifies power pad information.

You can specify multiple power pad formats (`xy` coordinates, pad cell, or power pin) for the same power net, in both, net-based and domain-based rail analysis.

⚠ All voltage source names in the power pad location file must be unique for all nets in the domain, e.g. if voltage source name vsrc1 is used in pad location file of net VDD as well as VSS in domain based analysis, program will issue error and exit. In addition, the names of voltage sources in the power pad location file must be different from the names in the DEF pin section.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_power_pads` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_power_pads`	
`-auto_voltage_source_creation {true|false}`		

	Specifies to automatically create voltage sources either for all nets (`-net ALL`) or for a specific net (`-net net_name`). The voltage sources are created on the top metal of all via cuts located either on the user-specified metal layer or the design top metal layer (default). Default: `false`.	
`-consider_partial_small_tiles_for_vsrc {true | false}`		
	Specifies to honor voltage source generation on small tiles. Small tiles refer to tile areas less than 1/3 of the pitch*pitch area size. This parameter is used with the automatic voltage source creation feature (`-auto_voltage_source_creation`). The default value of this parameter is `false`. **Example**: `set_power_pads -net VDD_AO -auto_voltage_source_creation true -layer VIA_5 -consider_partial_small_tiles_for_vsrc true -tile {90 90}`	
`-def_shape_only {true | false}`		
	Specifies to generate voltage sources only on the DEF routing shapes. This parameter controls the generation of voltage sources on the cell instance shapes. When this parameter is set to `false`, voltage sources are created for both the DEF routing shapes and shapes inside cell instances. This parameter is specified with the `-auto_voltage_source_creation` parameter. Default: `true`	
`-die_instance_name` *dieinstname*		
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode multi-die`).	
`-enable_switchnet_ploc_generation {true | false}`		
	Specifies to generate voltage sources on all switched nets. This parameter is specified with the `-auto_voltage_source_creation` parameter. Default: `false`	
`-file` *file*		
	Specifies the name of the file containing the voltage source information. Not required for `-format defpin` option.	
`-format {defpin | padcell | xy | boundary | xyiv}`		

	Specifies how the voltage source is determined. Required for `xy`, `padcell`, or `boundary`.
	`defpin` - Specifies that voltage sources will be determined by power pins.
	`padcell` - Specifies that voltage sources will be determined by pad cells.

	`xy` - Specifies that voltage sources will be specified by x y coordinates. You can also specify a PWL function on the X,Y location for a voltage source. When specified, Rail Analysis will read the power pad file and use the PWL waveform for the voltage source.
	Example:
	`set_power_pads -format xy -file <xy_plocFile>`
	
	Add the PWL option in the xy file:
	`*vsrc_name x y layer_name net_name [pwl=(t1,v1,t2,v2,t3,v3,...]`
	`vsrc3 8.39 370.35 Metal5 VDD pwl=(0,0.6,20e-9,0.7,25e-9,0.9,30e-9`
	`0.9,40e-9,0.7,50e-9,0.6)`

	`boundary` - Specifies that voltage source locations will be used for instances in the design.
	`xyiv -` Specifies whether the voltage sources specified in the xy power pad location file are current source or reference voltage source. The format is the same as the xy file, except the additional first and last columns to indicate the current/voltage source and current/voltage value.
`-layer` *`vialayername`*	
	Specifies the via layer on which the voltage sources are to be generated. If this parameter is not specified, the voltage sources will be generated on the top via layer of the design. This parameter is specified with the `-auto_voltage_source_creation` parameter.
`-net` *`net_name`* `|` `all`	
	Specifies the name of the power net. You can use `all` to set voltage source information for all nets in the domain or specify the net name for which the voltage source is to be specified.
`-number_of_voltage_source` *`value`*	

	Specifies the number of voltage sources to be generated. You can adjust either `-tile` or `-number_of_voltage_source` to reduce the number of voltage sources created. This parameter is specified with the `-auto_voltage_source_creation` parameter. Default: `1 million`.	
`-region {x1 y1 x2 y2}`		
	Generates the voltage sources in the specified region. If this parameter is not specified, the software uses the entire design/layout size as the region. This parameter is specified with the `-auto_voltage_source_creation` parameter.	
`-reset`	Resets all the previous `set_power_pads` commands	
`-short_pin_nodes {true | false}`		
	Specifies to short all interface nodes on a pad cell pin to a single node and create voltage source for that node. Default: `false` **Example**: `set_power_pads -net vdd -format padcell -short_pin_nodes -file pad_cell.file`	
`-tile {x y}`		
	Defines the number of tiles in the X and Y direction for automatic creation of voltage sources. When `-tile` is specified, the area (`-region` or default design area) will be tiled vertically and horizontally per the given tile size (x*y). For each tile, one voltage source will be created on the top metal of the biggest via array. If `-tile` is not specified, the software creates voltage sources on the top metal of all via cuts located either on the user-specified via layer or the top via layer of the design. You can adjust either `-tile` or `-number_of_voltage_source` to reduce the number of voltage sources created. The `-tile` parameter is specified with the `-auto_voltage_source_creation` parameter.	

Examples

- The following command specifies the power pad information for the `VDD_TDSPCore_R` net in the file `dtmf_chip.VDD_TDSPCore_R.vsr`c. The voltages sources are specified by x y coordinates.

  ```
  set_power_pads -net VDD_TDSPCore_R -format xy -file \ dtmf_chip.VDD_TDSPCore_R.vsrc
  ```

- The following command specifies `xy`, `padcell`, and `defpin` formats for a single net (`VDD`) as follows:

```
set_power_pads -net VDD -format xy -file VDD.pp
set_power_pads -net VDD -format padcell -file VDD.padcell
set_power_pads -net VDD -format defpin
```

- The following command specifies power pad information using a combination of user-specified and automatic voltage source creation methods:
  # Specific setting for net VSS using automatic voltage source creation

```
set_power_pads -net VSS -auto_voltage_source_creation true -layer VIA_6
```

  # Global setting for all nets using automatic voltage source creation

```
set_power_pads -auto_voltage_source_creation true -net ALL -layer VIA_5
```

  # User-specified power pad file for net VDD_AO, therefore automatic voltage source creation feature will not be enabled for this net

```
set_power_pads -net VDD_AO -format padcell -file padcell.file -short_pin_nodes true
```

Example of power pad location file with XY coordinates for net VDD

```
*-----------------------------------------------------------------------------------------------
-------------------------------------------------

* XY power pad location file

* vsrc name X(um) Y(um) LEF/Tech Layer <R=value ohm> <C=value F> <L=value H>

* Package R,L and C parameters are optional

*-----------------------------------------------------------------------------------------------
-------------------------------------------------

VDD100 5.000 793.500 M7 VDD_AO

VDD101 1187.835 293.500 M7 VDD_External

VDD102 695.000 5.000 M6  VDD

VDD103 1187.835 493.500 M7  VDD

VDD104 595.000 1293.800 M6 VDD_AO

VDD105 595.000 5.000 M6 VDD_External
```

Example of pad cell file for VDD net,

```
*-----------------------------------------------------------------------------------------------
-------------------------------------------------

* Pad cell file for Voltus

* cellname <r=value> <l=value> <c=value>

* Package parameters are optional

*-----------------------------------------------------------------------------------------------
-------------------------------------------------

PVDD1 r=2.5e-3 l=2.5e-11 c=120e-12

PVDD2 r=2.5e-3 l=2.5e-11 c=120e-12
```

set_rail_analysis_config

```
set_rail_analysis_config
[-help]
[-accumulate_overlapping_pgv_pwl_waveforms{true | false}]
-accuracy {xd | hd }
[-analysis_view view ]
[-avg_em_analysis {true | false}]
[-block_power_up_net_name netname | -powering_up_net net_name1... netname_n ]
[-cell_ignore_file filename ]
[-check_current_balanced_power_grid_em {true | false}]
[-check_thermal_aware_em {true | false}]
[-check_voltage_source_placement_on_switched_net {true | false}]
[-cluster_via1_ports {true | false}]
[-cluster_via_rule { {via_layer1 number_of_equidistant_vias}… }]
[-cluster_via_size value]
[-common_res_inst_pair_file_name file]
[-compress_power_grid_db {true | false}]
[-decap_cell_list { cell1 cell2 ... celln }]
[-def_based_hierarchical_reports {true | false}]
[-default_package_capacitor value ]
[-default_package_inductor value ]
[-default_package_resistor value ]
[-die_delimiter delimiter]
[-die_mode {single|multi-die|3dic} -die_instance_name dieinstname [-design designname ]]
[-disable_analysis_types { list of analysis types }]
[-disable_em_split_ac_dc_rules {true | false}]
[-disable_parallel_extraction]
[-dynamic_trigger_file filename ]
[-eiv_average_per_window_list filename]
[-eiv_eval_ground_window {true | false}]
[-eiv_eval_layers {layerlist}]
[-eiv_eval_layer_file filename]
[-eiv_eval_nodes {tap | port}]
[-eiv_eval_window {switching | timing | both | elapse}]
[-eiv_histogram_max value]
[-eiv_histogram_min value]
[-eiv_histogram_number_of_buckets value]
[-eiv_max_instance value]
[-eiv_method {worst best avg worstavg bestavg}]
[-eiv_pin_based_report {true | false}]
[-eiv_pin_location  {true | false}]
[-eiv_print_time {true | false}]
[-eiv_report {auto | netonly | all}]
```

```
[-eiv_threshold value]
[-em_cdf_percentage value]
[-em_ignore_pgv_resistors {true | false}]
[-em_limit_scale_factor {{avg value} {rms value} {peak value}}]
[-em_models file ]
[-em_peak_analysis {true | false}]
[-em_report_line_threshold value]
[-em_rms_delta_temperature temp]
[-em_rms_delta_temperature_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}]
[-em_temperature string ]
[-em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}]
[-em_threshold value]
[-enable_2d_partition_extraction  {true | false}]
[-enable_ccs_analysis {true | false}]
[-enable_distributed_processing_in_solver {true | false}]
[-enable_instance_power_gate_report {true | false}]
[-enable_mfg_effects {true|false}]
[-enable_multi_die_connectivity_check {true | false}]
[-enable_package_battery_current_probing {true | false}]
[-enable_rc_analysis {true | false}]
[-enable_reff_analysis {true | false}]
[-enable_rlrp_analysis {true|false}]
[-enable_scheduler {true | false}]
[-enable_seb {true | false}]
[-enable_sensitivity_analysis {true | false}]
[-enable_voltage_across_vias {true|false}]
[-enable_voltage_source_in_gif {true | false}]
[-enable_xp {true | false}]
[-enable_xpgv_scaling { true | false }]
[-env_temperature temperature]
[-era_add_virtual_followpin { standard | extended | none }]
[-era_add_virtual_followpin_for_io {true | false}]
[-era_add_virtual_via_on_layers  value ]
[-era_check_wires_for_generated_current_regions {true | false}]
[-era_current_distribution { unplaced | placed | all | none }]
[-era_current_distribution_factor_for_placed value]
[-era_current_distribution_layer layer_name]
[-era_current_distribution_nets {net1 net2 ...}]
[-era_current_distribution_unplaced_area {instance | diearea}]
[-era_current_region_file filename]
[-era_instance_dynamic_current_file filename]
[-era_lef_layer_map filename]
[-era_power_switch_file filename]
[-era_skip_virtual_via_by_type {whatif | def | all | none}]
[-era_skip_virtual_via_on_layers {{ layer1 layer2 } { layer3 layer4 } ... }]
[-era_tech_lib_generation {true | false}]
```

```
[-exclude_nets net_name_list ]
[-extend_bump_resolution_in_gif {true | false}]
[-extract_subconductor_layers {true | false}]
[-extraction_tech_file filename ]
[-feol_scale_factor non_negative_integer_number]
[-filler_cell_list { cell1 cell2 ... celln }]
[-fine_grain_power_switch_r_on {min avg max}]
[-fine_grain_power_switch_r_on_list filename]
[-fine_pre_simulation_period value]
[-flatten_xpgv_block_instances  filename]
[-force_extraction  {true | false} ]
[-force_library_merging {true | false}]
[-generate_multi_voltage_library {true | false}]
[-gif_iv_threshold value]
[-gif_new_color_scale {true | false}]
[-gif_resolution {low | medium | high}]
[-gif_zoom_area { x1 y1 x2 y2 }]
[-gif_zoom_top_cell_die_area {true | false}]
[-hier_delimiter character]
[-hier_pgv_block_lefs list_of_files]
[-hier_pgv_write_view {ir | em | all}]
[-honor_negative_static_current {true | false}]
[-ict_em_models file]
[-ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}]
[-ignore_decaps {true | false}]
[-ignore_duplicate_voltage_source {true | false}]
[-ignore_fillers {true | false}]
[-ignore_from_reporting filename]
[-ignore_incomplete_net {true | false}]
[-ignore_nets_without_voltage_sources {true | false}]
[-ignore_shorts {true | false}]
[-import_what_if_shapes {true | false}]
[-include_extraction_tool filename ]
[-inst_ignore_file filename]
[-lef_map lefMappingFile ]
[-lifetime value ]
[-lifetime_rating_factor_file filename]
[-limit_number_of_steps {true | false}]
[-lowest_layer_for_em_check layer_name]
[-mcp_model_mapping {{<mcp model name1> <die DEF name1>} {<mcp model name2> <die DEF name2>}
...}]
-method {static | dynamic | era_static | era_dynamic}
[-off_state_powergate_instance_file filename]
[-override_set_power_data {true | false}]
[-package_trace_connectivity {true | false}]
[-peak_em_analysis {true | false}]
```

```
-power_grid_libraries dir_list
[-power_up_fast_mode {true | false}]
[-power_up_sequence_file filename ]
[-powering_down_net net_name1... netname_n ]
[-pre_chain_power_up_sequence_file filename]
[-pre_simulation_period value]
[-pre_simulation_resolution value]
[-print_gif_range_percentage  {true | false}]
[-probe_instance_pin_waveforms {true | false}}
[-probe_instance_tap_waveforms {true | false}]
[-probe_package_interface_ports {true | false}]
[-probe_pin_voltage_list filename]
[-probe_waveform_list filename]
[-probing_node_file file_name]
[-process_bulk_pins_for_body_bias {true | false}]
[-process_techgen_em_rules {true | false}]
[-promote_pin_shapes_only {filler | decap | both} ]
[-r_effective_detail_report {true|false}]
[-r_effective_domain_report_format {1 | 2 | 3 | 4}]
[-r_effective_eval_nodes {port | tap}]
[-r_effective_pin_report_layer { top | bottom | all }]
[-r_effective_pin_report_method { best | worst } ]
[-r_effective_report_all {true | false}]
[-rdl_def def_file]
[-rdl_def_list {{rdl_filename1  x1  y1  orient1} {rdl_filename2  x2  y2  orient2} ...}]
[-rdl_orient { N|S|W|E|FN|FS|FE|FW }]
[-rdl_placement {X Y}]
[-read_thermal_map thermal_map_file ]
[-record_inst_peak_current {true | false}]
[-record_results_start_time time ]
[-remove_duplicate_inst {true | false}]
[-report_compress_level {0 | 1-9}]
[-report_instances_missing_current_data {true | false}]
[-report_layer_based_ir_average {true | false}]
[-report_layers_above_pin_for_instance_ir {true | false}]
[-report_power_domain {true | false}]
[-report_power_in_parallel {true | false}]
[-report_power_options option_names]
[-report_shorts {true | false}]
[-report_voltage_drop {true | false}]
[-reset]
[-reuse_state_directory dir_name]
[-rlrp_cell_report_layer filename]
[-rlrp_detail_report {true | false}]
[-rlrp_eval_nodes {port | tap}]
[-rlrp_percentage_threshold value]
```

```
[-rlrp_pin_based_report {true | false}]
[-rlrp_pin_report_layer { top | bottom | all }]
[-rlrp_pin_report_method { best | worst | eiv_best | eiv_worst} ]
[-rlrp_threshold value]
[-rms_em_analysis {true | false}]
[-scale_initial_condition_current filename]
[-seb_lifetime value]
[-seb_table table_filename]
[-seb_temperature temperature]
[-set_analyze_bbox {xmin ymin xmax ymax}]
[-shorting_resistance value]
[-single_partition_extraction_layer {METAL_LAYER}]
[-single_partition_extraction_layer_list {METAL_LAYER_1 METAL_LAYER_2 ……}]
[-skip_extraction  {true | false} ]
[-snap_layer_for_current_taps file_name]
[-static_multi_mode_analysis {true | false}]
[-static_trigger_file filename]
[-stream_file file [-stream_offset {x y}]]
[-stream_map file ]
[-stream_purpose {metalFill | flipChip | fullChip}]
[-stream_top_cell cell_name ]
[-suppress_message { message_id + } ]
[-switching_bit_file_for_eiv_calculation filename]
[-temperature value ]
[-tmp_directory_name directory ]
[-top_cell_placement {X Y}]
[-top_cell_orient { N|S|W|E|FN|FS|FE|FW }]
[-tsv_subckt_model_file_list filename]
[-unconnected_die_package_pins {ignore | error | edit}]
[-unconnected_cell_ignore_file filename]
[-unified_power_switch_flow {true | false}]
[-use_early_view_list filename]
[-use_em_view_list filename]
[-use_ir_view_list filename]
[-use_rms_delta_t {true | false}]
[-verbosity  {true | false}]
[-verify_logical_connectivity {true | false}]
[-voltage_source_search_distance value ]
[-watch_location_waveform { {layerName xCoord yCoord}+ }]
[-what_if_shapes_file filename]
[-work_directory_name directory ]
[-write_combinational_inst_voltage_drop_gif {true | false}]
[-write_demand_current_region_file filename]
[-write_hier_block_boundary_voltage {true | false}]
[-write_instance_ir_report file]
[-write_instance_pin_ir_report file]
```

```
[-write_instance_reff_report file]
[-write_package_pin_current_voltage {true | false}]
[-write_voltage_waveforms {true | false}]
[-xp_cpu_per_job_power value]
[-xp_cpu_per_job_simulation value]
[-xp_host_allocation_method {on_demand | at_startup}]
[-xp_purge {full}]
[-xp_resume {true | false}]
[-xp_reuse_extraction_directory directory_name]
[-xp_reuse_hpt_directory state directory]
[-xp_reuse_parse_def_directory directory_name]
[-xp_simulation_cpu_timeout value]
[-xp_simulation_min_cpu value]
[-xpgv_config_file filename]
```

Specifies how the rail analysis will be performed.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_rail_analysis_config` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_rail_analysis_config`
`-accumulate_overlapping_pgv_pwl_waveforms{true | false}`	
	Default: `false` Specifies to accumulate the current values of Dynamic Detailed View (DDV) PWL waveforms from different trigger events when the PWL waveforms are overlapping with each other. If set to `true`, the current values of the overlapping PWL waveforms are added together for rail analysis.
`-accuracy {xd | hd }`	
	Default: `hd` Specifies the accuracy mode of analysis.

	• `xd`: xd or accelerated definition accuracy mode is used for early implementation stage IR/EM analysis.
	• `hd`: hd or high definition accuracy mode is used for final verification stage IR/EM analysis.
`-analysis_view` *view*	
	Default: None Specifies analysis view created in a CPF (Common Power Format) file. Optional.
`-avg_em_analysis {true | false}`	
	Default: `false` Specifies to calculate the average EM in the dynamic rail analysis run. This parameter generates the `<net_name>.rj.avg.rpt` report.
`-block_power_up_net_name` *netname*	
	Default: None Specifies a net for block power-up analysis. You can specify only one net for block power-up analysis at a time.
`-cell_ignore_file` *filename*	
	Default: None If specified, excludes all references to the named cells in the specified file, from the design database.
`-check_current_balanced_power_grid_em {true | false}`	
	Default: `false` Specifies to check the power-grid EM rules (power-rail terminal-via EM rule). This parameter allows EM comparison with/without this EM rule. This parameter is only applicable in TSMC N20 process nodes where the EM limit may be relaxed if certain constraints are met. Refer to TSMC DRM for more details.
`-check_thermal_aware_em {true | false}`	

	Default: `false` Specifies to use the thermal map for EM checks. The thermal map file is specified using the `-read_thermal_map` parameter. The Rail Analysis engine uses the temperature specified in the thermal map file for EM analysis. EM limit has a strong temperature dependency. For a chip that has substantial temperature variation across the chip, the Rail Analysis engine can use the temperature specified in the thermal map file to perform accurate EM analysis.
`-check_voltage_source_placement_on_switched_net {true | false}`	
	Default: `false` Specifies to check voltage source placement on switched nets. When this parameter is set to `true`, the software checks whether the specified voltage source location file places any voltage sources on the switched net of the power-gated domain, and disables voltage source placement on the switched net. It also detects shorts and generates an error message.
`-cluster_via1_ports {true | false}`	
	Default: `true` Designs with follow-pin routing on M1 and M2 layers have standard cells library LEF with M1, M2, and VIA1 ports. This parameter specifies to cluster VIA1 ports for such cells in order to improve overall extraction and rail analysis performance. This parameter is set to `true` by default to improve VIA1 port handling in designs with M1 and M2 follow-pin routing. This parameter gives significantly better performance by clustering equidistant VIA1 ports defined in cell LEF without significant loss of accuracy. However, if you specify a clustering rule, the user specification takes precedence over automated VIA1 clustering.
`-cluster_via_rule { {`*`via_layer1 number_of_equidistant_vias`*`}...}`	
	Default: None Controls the number of vias to cluster on a layer basis. The VIA clustering rule specified using this parameter will override the default clustering rule for a given accuracy mode. The following command will cluster 100 equidistant VIA1 cuts, 200 equidistant VIA2 cuts, and 300 equidistant VIA7 cuts: `set_rail_analysis_config -cluster_via_rule { {VIA1 100} {VIA2 200} {VIA7 300}}` The rest of the VIAs will be clustered using the default clustering rule depending upon the rail analysis accuracy mode.

`-cluster_via_size` *value*	
	Default: `625` Specifies the size of the via array to cluster all via layers except VIA1 (layer between Metal1 and Metal2). The default value is `625` (25x25 via array), except when running static analysis using HD mode. The default for static HD mode is `16` (4x4), primarily to preserve more accuracy for EM analysis.
`-common_res_inst_pair_file_name` *file*	

	Default: None
	Specifies to calculate common resistance along the least resistance path (RLRP) between a pair of cells having the same voltage source. This parameter allows you to check RLRP along cell instance pairs from multiple voltage source locations.
	The format of the specified file is:
	`inst_name1 inst_name2`
	`inst_name3 inst_name4`
	`...`
	The `-common_res_inst_pair_file_name_name` parameter generates a report `<`*net_name*`>.common_res` in the following location:
	`<`*output_dir*`>/<`*state_dir*`>/Reports/<`*net_name*`>/`
	The format of the report is:
	`inst_name1 inst_name2 common_res1 pad1`
	`inst_name3 inst_name4 common_res2 pad2`
	`...`
	The following is a snippet from the report:
	`inst_name1 inst_name2` 13.4315 PAD_VDD2

`-compress_power_grid_db {true | false}`	
	Default: `false` Specifies to compress the output files generated by extraction and rail analysis to save disk space usage. This parameter reduces disk space requirement from the extraction and rail analysis stages at approximately 5-10% performance penalty.
`-decap_cell_list {` *cell1 cell2 ... celln* `}`	

	Default: None Specifies a list of decap cells. This option is used when the decap cells are not tagged in the power-grid library. A cell characterized as a decap cell during power-grid view generation with LibGen does not need to be specified. For dynamic only.
`-def_based_hierarchical_reports {true | false}`	

	Default: `false`
	Specifies to generate separate rail reports for each DEF instance. The net-based effective instance voltage, RLRP, and effective resistance reports are generated hierarchically.
	Reports will be generated for every net in each DEF hierarchy using the following directory structure:
	<*output_dir*>/<*state_dir*>/Reports/HIER/hier_<*HIER_ID*>/<*domain_name* | *net_name*>.<report_type>
	In the `Reports` folder, the Rail Analysis engine will generate a hierarchical map (`def_hierarchy.map` file) during DEF parsing to associate a hierarchical ID with each DEF instance, as shown below:
	`#hier_id  hier_name  #def_name`
	`0  chip  chipCore`

`-default_package_inductor` *value*	
	Default: None Specifies the default inductance value of the package resistance-inductance-capacitance (RLC) model to use when you specify voltage source locations. If the power-grid views contain area-based voltage source locations, the following equation is used to define the inductance applied to the voltage sources: `Li = L * number_of_sources` where: `Li` is the inductance on an individual voltage source. `L` is the default inductance set by `-default_package_inductor`. `Number_of_sources` is the number of voltage sources in the power-grid view. The default inductance value is specified in `henrys`.
`-default_package_resistor` *value*	

	Default: None Specifies the default resistance value of the package resistance-inductance-capacitance (RLC) model to use when you specify voltage source locations. If the power-grid views contain area-based voltage source locations, the following equation is used to define the resistance applied to the voltage sources: `Ri = R * number_of_sources` where: `Ri` is the resistance on an individual voltage source. `R` is the default resistance set by `-default_package_resistor`. `Number_of_sources` is the number of voltage sources in the power-grid view. The default resistance value is specified in `ohms`.
`-default_package_capacitor` *value*	
	Default: None Specifies the default capacitance value of the package resistance-inductance-capacitance (RLC) model to use when you specify voltage source locations. If the power-grid views contain area-based voltage source locations, the following equation is used to define the capacitance applied to the voltage sources: `Ci = C/number_of_sources` where: `Ci` is the effective capacitance on an individual voltage source. `C` is the default capacitance set by `-default_package_capacitor`. `Number_of_sources` is the number of voltage sources in the power-grid view. The default capacitance value is specified in `farads`.
`-design` *designname*	
	Default: None Specifies the DEF or GDS design name for multi-die analysis. This option is only available in multi-die analysis (`-die_mode multi-die`).
`-die_delimiter` *delimiter*	

	Default: `/` Specifies the character `:` as delimiter to separate the die instance name from the other element (net, bump, ground) names. You can specify the parameter when there is `/` character in the die instance name. **Use Model**: `set_rail_analysis_mode -die_delimiter :` **Example**: • When there is no `/` character in the die instance name, the `/` character is used as a delimiter in the die net name. Die instance name – `Die1` Net name – `Net1` Die net name – `Die1/Net1` • When there is `/` character in the die instance name, you can specify the `-die_delimiter` parameter to use `:` character as a delimiter in the die net name: Die instance name – `Die/Die1` Net name – `Net1` Die net name – `Die/Die1:Net1`
`-die_mode {single|multi-die|3dic}`	
	Default: `single` Specifies whether single or multi-die analysis will be performed. The `3dic` option is used for performing multi-die rlrp analysis.
`-die_instance_name` *dieinstname*	
	Default: None Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`-die_mode multi-die`).
`-disable_analysis_types` { *list of analysis types* }	

	Default: By default, the disable list is empty. Disables specific analysis types during rail analysis. For example, `set_rail_analysis_config -disable_analysis_types { rlrp vu pi pv }` disables `rlrp, vu`, `pi`, and `pv` analysis types during `report_rail`. By default, the disable list is empty. The following analysis types can be disabled: `ir tc iv vc dd pi pv javg rj jrms rc node_effr vu`.
`-disable_em_split_ac_dc_rules {true | false}`	
	Default: `false` Specifies to use the same AC/DC rules for the EM analysis of the signal and PG nets. When this parameter is set to `true`, the software disables separate handling of the AC/DC rules. By default, the software uses separate EM rules for the signal and PG nets.
`-disable_parallel_extraction`	
	Default: Uses parallel processors for extraction Specifies to use a single processor for extracting resistance or capacitance. If this parameter is not specified, the software uses parallel processors for extraction.
`-dynamic_trigger_file` *`filename`*	
	Default: None Specifies a dynamic trigger file. When a Macro EM view with current signature is used in a design, it needs to be determine when this block/cell is triggered. The dynamic trigger file is used to determine this. At the trigger time, the dynamic current waveforms of the EM view block will be applied. **Note**: If you specify trigger for cells or instances using this parameter, the tool would give higher precedence to this user-defined trigger file over the auto-generated trigger files in the power analysis directory (`power_output_dir/trigger*.txt`). For more information on the trigger file format, refer to the "Dynamic Trigger File During Rail Analysis" section in the *"**File Formats**"* chapter in *Voltus Stylus Common UI User Guide*.
`-eiv_average_per_window_list` *`filename`*	

Default: None

Specifies to generate a report containing different sets of Effective Instance Voltage (EIV) for each timing/switching window for a list of cells/instances. You can use the `ALL` keyword to report EIV for every window for all cells and instances.

The format of the specified file is:

```
INST inst_name
CELL cell_name
ALL
```

The report contains a set of average EIVs for each instance based on the timing/switching window in the current files. The report file name is:

`name.win_avg.rpt.`

Example:

```
set_rail_analysis_config -eiv_average_per_window_list ./clock_network.insts
```

File format:

```
INST clk__L2_I1
INST clk__L11_I2
INST clk__L13_I4
INST clk__L13_I3
INST clk__L13_I2
```

Output EIV report format:

```
#INST_NAME START_TIME:END_TIME EIV_AVG
clk__L15_I4 1.32e-09:1.78e-09 1.06354 6.38e-09:6.8e-09 1.06217
clk__L15_I3 1.32e-09:1.68e-09 1.06273 6.38e-09:6.72e-09 1.06854
...
```

`-eiv_eval_ground_window {true | false}`

Default: `false`

Specifies that the ground net switching window should also be considered for the EIV computation. The switching windows from the power and ground nets will be merged into a single switching window.

`-eiv_eval_layers {`*`layerlist`*`}`

	Specifies to report the EIV of nodes only on the specified layers. When this parameter is not specified, the software reports the EIV values on the worst nodes across all layers. The parameter allows you to analyze the EIV values for the user-defined layers. **Example**: `set_rail_analysis_config -eiv_eval_layers B_M0 -eiv_eval_nodes port` This command reports only the EIV values of nodes on the `B_M0` layer.
`-eiv_eval_layer_file` *filename*	

	Specifies a file containing a list of user-specified cells and corresponding layers for EIV reporting. The parameter allows you to analyze the EIV nodes on the user-defined cells and layers. This parameter is used for dual-sided rails (B_M0/M0) with only back-side interface/port nodes, and without the interface/tap nodes.
	The format of the file is:
	<cellName> <layerList>
	Example:
	The following command will report the EIV values of the instance nodes on the B_M0 layer associated with the H338 cell and report the EIV values on the instance nodes on the M0 layer:
	`set_rail_analysis_config -eiv_eval_layer_file file.rpt -eiv_eval_layers{M0}`
	The following is the content of `file.rpt`:
	`H338* B_M0`
	When the `-eiv_eval_layer_file` parameter is not specified, the generated report is based on `-eiv_eval_layers`. If no nodes are found on the specified layer B_M0/M0, the software reports the EIV values on the worst nodes across all layers.

`-eiv_eval_nodes {tap | port}`	
	Default: `tap` Specifies to report EIV based on the instance nodes (tap or port). By default, the software uses the tap nodes for EIV reporting.
`-eiv_eval_window {switching | timing | both | elapse}`	

	Default: `both` Controls the window where EIV is evaluated. EIV is obtained by processing the voltage waveforms and finding the worse-case effective voltage between the power and ground pins during a specific window. There are four possible windows to evaluate EIV: • `switching` – any switching activity (current is non-leakage) reported in power analysis • `timing` – timing window information from TWF or embedded timing engine • `both` – both switching or timing window • `elapse` – entire rail simulation not filtered to a specific window **Note**: Although power analysis uses timing window information to generate switching activities, it is possible that switching window will be slightly outside timing window if the instance is scheduled to switch at the very beginning or the very end of the timing window. Current waveform may start prior or end later than the time of switching. For additional information on EIV window, see "effective instance voltage" in the *"**Glossary**"* chapter of *Voltus Stylus Common UI User Guide*.
`-eiv_histogram_max` *value*	
	Default: `0` Specifies the maximum voltage value of the user-defined voltage range for EIV histogram generation.
`-eiv_histogram_min` *value*	
	Default: `0` Specifies the minimum voltage value of the user-defined voltage range for EIV histogram generation.
`-eiv_histogram_number_of_buckets` *value*	
	Default: `8` Specifies the number of buckets/ranges to be used for EIV histogram generation. The data points are divided into buckets based on the range specified.
`-eiv_max_instance` *value*	

	Default: `2e-9` Specifies the maximum number of instances to be included in the .iv files.
`-eiv_method {worst best avg worstavg bestavg}`	
	Default: `worst` Allows different methods of computing EIV during rail analysis. Using this parameter, you can write out the best, worst, or average effective instance voltage between power and ground nets. The `worstavg` EIV method computes the average instance voltage during each evaluation window defined by the `-eiv_eval_window` parameter, and reports the worst average instance voltage out of all the windows. Similarly, the `bestavg` EIV method computes the average instance voltage during each evaluation window defined by the `-eiv_eval_window` parameter, and reports the best average instance voltage out of all the windows. If `-eiv_method worstavg/bestavg` and `-eiv_eval_window elapse` (i.e. 1 evaluation window) are specified, then this method will report average voltage across all time steps in simulation. For additional information on EIV methods, see "effective instance voltage (EIV) method" in the ***"Glossary"*** chapter of *Voltus Stylus Common UI User Guide*.
`-eiv_pin_based_report {true | false}`	

	Default: `false`
	Specifies to report an individual pin's instance voltage for each instance. When enabled, the format of the instance voltage file (.iv file) will be changed to one instance pin per line with pin name added to the last column. If an instance has multiple pins connected to the same net, it will have multiple entries in the .iv file.
	The format of the report is:
	<Instance Name> <Instance Voltage> <Cell Name> <Pin Names>
	The following is a snippet of the report from the .iv file:
	`NET`
	`INSTANCESUPPLY 18118 VDD 1.08 POWER`
	`BEGIN`
	`- inst1  1.0798 CKND12 VDD`
	`- inst2  1.0799 INVD6LVT VDD`
	`DOMAIN`
	`INSTANCESUPPLY 18118 VDD 1.08 DIFFERENTIATE`
	`- clk__L13_I3  1.0767 CKND12 VDD VSS`
	`- clk__L13_I4  1.0767 CKND12 VDD VSS`

`-eiv_pin_location {true | false}`	
	Default: `false` Specifies to report an individual pin's location for each instance. This parameter works only with the `-eiv_pin_based_report` parameter. When enabled, the format of the instance voltage file (.iv file) will be changed to one instance pin per line with the pin location added to the last column. This parameter only works for `ivdn` (net-based instance voltage drop). It is not supported for `ivdd` (domain-based instance voltage drop). That is, the pin location will be displayed only in the net-based iv report file (for example, VDD.iv file) and not in the domain-based iv report file (for example, VDD-VSS_div.iv). The format of the report is: *<Instance_Name> <PWR_IV> <Cell_Name> <Pin_Names> <X Y Coordinates>* The following is a snippet of the report from the .iv file: `- INST1 1.13831 MUX2_F8_75LL VDD met1` **`<6152370, 5708640>`** `- INST1 1.13831 ENOR4_F4_75LL VDD met1` **`<6153420, 5714520>`** `- INST2 1.13831 BUF_F1_1SR_75LL VDD met1` **`<6147330, 5708640>`**
`-eiv_print_time {true | false}`	
	Default: `false` Prints the time stamp for the worst voltage in the switching window of the instance, and for the whole (elapse) window. The time stamp values are printed in the detailed EIV report. Using the time stamp information, you can determine the current signature related to the worst EIV for easy debugging. The default unit for time stamp is `ns`.
`-eiv_report {auto | netonly | all}`	
	Default: `auto` Specifies the type of EIV report to be generated. The possible options are: • `auto` - generates domain-based .iv report for domain analysis. • `netonly` - generates net-based .iv report. • `all` - generates both net and domain .iv reports, if available.
`-eiv_threshold` *value*	

	Default: `1e-9` Specifies the EIV drop threshold in mV for reporting. Only voltage drop above the threshold will be saved into the report.
`-em_cdf_percentage` *value*	
	Default: `1e-9` Specifies to calculate the Cumulative Distribution Factor (CDF) derating factor based on the EM rules specified in the ICT EM file, given by the `-ict_em_models` parameter. This parameter is used to calculate the fail rate and reliability of a design. The `-em_cdf_percentage` parameter is only applicable to the foundry-specific ICT EM file with the `cdf_percentage` keyword. The value of the CDF percentage should be between 0 and 100. The use model of the new parameter is: `set_rail_analysis_config -em_cdf_percentage 5` When the CDF related rule exits in the ICT EM file, the software uses the default value `1e-09` for `-em_cdf_percentage` if it is not specified.
`-em_ignore_pgv_resistors {true | false}`	
	Default: `false` Specifies to filter out all the EM violations of resistors inside a macro (resistors coming from PGV).
`-em_limit_scale_factor {{avg` *value*`} {rms` *value*`} {peak` *value*`}}`	
	Default: None Scales the calculated EM limits by the respective factor for avg, rms or peak current limit specified by `{avg` *value*`} {rms` *value*`} {peak` *value*`}`. The software compares the current through the net against the scaled EM limit to report violations in power EM.
`-em_models` *file*	**Default:** None Specifies the name of a file that includes the electromigration (em) models. The `-process_techgen_em_rules` and `-em_models` parameters are mutually exclusive. In the static mode, if `-em_models` is not specified, or if `-process_techgen_em_rules` is not set to `true`, current density (RJ) analysis will be disabled due to lack of EM models.
`-em_peak_analysis {true | false}`	

	Default: `false` Specifies to perform peak EM analysis. When this parameter is specified with `-process_techgen_em_rules true` or `-em_models`, both RMS and Peak EM will be reported in the same run.
`-em_report_line_threshold` *value*	
	Default: `INT_MAX` Controls the size of the EM (`rj.avg.rpt`) text report by limiting the number of lines written to the report file. This feature is useful in limiting the report size when you specify a very low EM threshold. **Example**: The following command reports only the top 18 lines: `set_rail_analysis_config -em_report_line_threshold 18` The truncated EM report file has the following note: # NOTE: Aborting EM Report printing as total number of lines has exceeded the Line Threshold "em_report_line_threshold = 18"
`-em_rms_delta_temperature` *temp*	
	Default: `5C` Specifies the delta temperature for the EM RMS current limit analysis. Default is 5C.
`-em_rms_delta_temperature_layer_list {{<`*LEF_layer_name*`> <`*delta_T_in_C*`>}+}`	
	Default: None Specifies the delta temperature for specific LEF layers during power EM RMS current limit analysis.
`-em_temperature <string>`	**Default:** If `-em_temperature` is not specified and `em_tref` is not included in the EM rule file, the tool uses the value set by the `-temperature` parameter. Sets a different temperature for EM analysis. As a result, you can use a different but higher t temperature for conservative EM analysis.
`-em_temperature_layer_list {{<`*LEF_layer_name*`> <`*em_temperature_in_C*`>}+}`	
	Default: None Sets a different temperature for specific LEF layers during power EM AVG current limit analysis.

`-em_threshold` *value*	
	Default: `0.9` Specifies to set EM analysis threshold value for the resistors being reported. Using this parameter, you can control the reporting threshold. By default, rail analysis generates EM report for any resistor having current above `0.9` of its limit. Changing the value to `0` will show every resistor, and `1` will limit to the failed resistors only.
`-enable_2d_partition_extraction {true | false}`	
	Default: `false` Specifies to enable the upgraded extraction algorithm that allows 2D partitioning. The 2D extraction algorithm uses an updated capacitance model from the qrcTechFile that provides better accuracy for capacitance calculation. The resistance accuracy level of the 2D extraction algorithm is the same as 1D extraction algorithm. It is recommended to use 2D partitioning as it improves the parallelism of the extraction jobs for large designs. When the `-enable_2d_partition_extraction` parameter is set to `true`, you must provide the complete qrcTechFile using the `set_rail_analysis_config -extraction_tech_file` parameter. This parameter does not support the 3DIC, inductance extraction, and full-chip GDS flows.
`-enable_ccs_analysis {true | false}`	
	Default: `false` Specifies to enable constant current source analysis.
`-enable_distributed_processing_in_solver {true | false}`	
	Default: `true` Specifies to enable distributed processing for Matrix Solver. When set to `true`, Matrix Solver will be able to use multiple hosts across the network to analyze power-grid. You can specify this parameter when peak memory of the analysis exceeds machine memory in the local multi-threaded mode. In the distributed mode, peak memory on the local machine will be reduced as child processes will be executed on network hosts. However, the master thread on the local machine will continue to consume majority of peak memory even in the distributed mode.
`-enable_instance_power_gate_report {true | false}`	

	Default: `false` Specifies to generate a report (`powergate.inst.rpt`) that summarizes information about each power gate instance. The format of the report is: `<inst_name> <cell_name> <Inst X> <Inst Y> <V_Ref> <Ion(total)> <PI(Ipeak/Iavg)> PV(IR)>` Here, • <`inst_name`> is the power gate instance name • <`cell_name`> is the name of the power gate cell • <`Inst X`> is the x coordinate of the power gate instance • <`Inst Y`> is the y coordinate of the power gate instance • <`V_Ref`> is the rail voltage • <`Ion(total)`> is the total current for enabled powergate instance • <`PI(Ipeak/Iavg)`> is the worst Idsat ratio out of all the Ron values • <`PV(IR)`> is the voltage across the power switch using the combination for the worst V(AON Pin)-V(SW Pin)
`-enable_mfg_effects {true | false}`	
	Default: `false` Specifies to honor manufacturing effects during static or dynamic analysis. The manufacturing effects are enabled by default in the hd accuracy mode. That is, if you specify `set_rail_analysis_config -accuracy hd`, the `-enable_mfg_effects` parameter is set to `true` by default.
`-enable_multi_die_connectivity_check {true | false}`	

	Default: `false` Specifies to check if voltage sources with `type=nonvsrc` have resistance path to any voltage source with `type=vsrc` inside each die. This parameter allows you to determine if the non-real voltage source (type=`nonvsrc`) is floating or does not have any resistance path to an ideal voltage source (type=`vsrc`), and to ignore floating voltage sources during rail analysis. The "`vsrc`" and "`nonvsrc`" types are given in the .ploc file specified with the `set_power_pads` command. When the `-enable_multi_die_connectivity_check` parameter is specified, the log file displays the list of disconnected terminals for each net. This parameter can be used in both the single-die and multiple-die analysis flows.
`-enable_package_battery_current_probing {true | false}`	
	Default: `false` Specifies to include the battery current waveform data in the `tran_<net>_pad.ptiavg` file. This file includes the voltage of all battery side power_pin and ground_pin mapped pins.
`-enable_rc_analysis {true | false}`	
	Default: `false` Enables the Resistor Current (RC) analysis. This parameter supports designs without EM rules/analysis to generate the RC plot for IR drop debugging.
`-enable_reff_analysis {true | false}`	

	Default: `false`
	Specifies to run full-chip effective resistance analysis as part of the static or dynamic IR drop analysis. Effective resistance analysis will be performed on the domain specified with `report_rail`. This parameter allows you to run both resistance and IR drop analyses in a single session.
	The use model of this parameter is:
	`set_rail_analysis_config -enable_reff_analysis true`
	`report_rail -output_dir <dir_name> -type domain <domain_name>`
	Usage Guidelines:
	• Supports both the static and dynamic rail analyses.
	• Supports both the net-based and domain-based analyses.

`-enable_rlrp_analysis {true|false}`	

	Default: `false` Specifies to enable least resistance path analysis for all the instances in the design. The RLRP path can be displayed in the GUI using the `gui_set_power_rail_display -enable_rlrp` command.
`-enable_scheduler {true | false}`	
	Default: `true` Controls the Voltus advanced scheduler that improves data handling and multi-CPU scalability of rail analysis . When set to `true`, the scheduler can significantly improve performance and memory consumption, especially when running on large number of processors. The default value is `true`. **Note**: It is recommended to set `-enable_scheduler` to `true`. If set to `false`, some of the advanced extraction features will not be supported.
`-enable_seb {true | false}`	
	Default: `false` Specifies to enable the Statistical ElectroMigration Budgeting (SEB) analysis.
`-enable_sensitivity_analysis {true | false}`	
	Default: `false` Only available in the static mode. When set to `true`, the software performs sensitivity analysis on all power-grid segments and highlights those which can be optimized to improve the overall IRdrop profile of the design. The power-grid segments can be optimized using the `scale_what_if_resistance` command.
`-enable_voltage_across_vias {true|false}`	
	Default: `false` Specifies to report voltage across vias.
`-enable_voltage_source_in_gif {true|false}`	

	Default: `true` Specifies to enable or disable the display of voltage sources in the generated IR drop plot GIFs. When set to `false`, disables voltage sources (power pads) from the *Layout* tab during IR drop analysis. This parameter allows you to control the display of voltage sources in the generated IR drop plot GIFs so that the plot is easily interpreted.
`-enable_xp {true | false}`	
	Default: `false` Specifies to run rail analysis in the Extensively Parallel (XP) mode. When this parameter is set to `true`, the rail analysis engine distributes processing over a large number of machines in every stage of the flow (for example, extraction, reporting, GUI, and simulation) to reduce the processing time.
`-enable_xpgv_scaling { true | false }`	
	Default: `false` Specifies to enable xPGV current scaling that is calculated as a ratio of voltages. If PGV voltage is V1 and analysis voltage is V2, then the scaling factor would be V2/V1. This factor is applied to each power net. In case there are multiple power nets and a single ground net, the tool will use the worst scaling factor from the power rails and apply it to the ground net. This parameter is used for scaling of currents stored in xPGV. The parameter allows you to use the same xPGV in different designs, irrespective of the voltage at which the xPGV is generated. `-enable_xpgv_scaling` and `-static_trigger_file` are mutually exclusive.
`-env_temperature <`*`temperature in °C`*`>`	
	Default: `25C` Specifies the ambient temperature used to calculate S_{DC} for SEB analysis.
`-extend_bump_resolution_in_gif {true | false}`	
	Default: `false` Specifies to increase the size of voltage sources represented for the voltage source current (vc) and voltage drop across package (vu) plots to improve visibility.
`-era_check_wires_for_generated_current_regions {true | false}`	

	Default: `false` Specifies to generate unplaced current regions only in areas where actual net wires are present. The current regions are generated from the power domain information present in the design.
`-era_current_distribution_factor_for_placed` *value*	
	Default: 1 When you use `-era_current_distribution` and set to distribute current for placed blocks/macros, the ratio of power allocated to each placed instance is calculated based on its total area. Using this parameter, you can control the current distribution factors for the placed blocks/macros, hence power allocated for area-based power calculation. For example, if you specify 0.5, the tool will assume all placed blocks/macros to be 50% of its actual size and distribute current accordingly.
`-era_current_region_file` *filename*	

	Default: None
	Specifies a file that includes a list of regions and the amount of current to be distributed within them for the power-grid. Default units in file are `mA`. You can also specify rectilinear current regions in the file.
	#Format:
	`LABEL` *`name`* `NET` *`netName`* `AREA` *`x1 y1 x2 y2`* `LAYER` *`layername`* `<CURRENT` *`value`* `| PWL (`*`t1 i1 t2 i2 …`*`)> INTRINSIC_CAP <`*`value`*`> LOADING_CAP <`*`value`*`> [ADD_OVERLAP | SUBTRACT_OVERLAP]`
	#Unit:
	current mA, cap pf, time ns, coordinate um
	• `ADD_OVERLAP`: Honors all overlapping regions, and adds current in the overlapped regions.
	• `SUBTRACT_OVERLAP`: Specifies that if two XY regions overlap, the first specified region will be honored, and the subsequent region will subtract the overlapped area to apply the region current to the remainder area.
	Static Example :
	`label test1 net VDD area 100 200 400 500 layer M1 current 10`
	Dynamic Example :
	`label test1 net VDD area 100 200 400 500 layer M1 pwl (0ns 0mA 1ns 0mA 1.9ns 0mA 2ns 10mA) intrinsic_cap 10 loading_cap 60`
	`label test1 net VDD_AO area 0 0 150.93 0 150.93 175.14 354.93 175.14 354.93 379.12 0 379.12 0 0 layer Metal1 PWL (0 1 10 1 20 025 1 100 1) INTRINSIC_CAP 1 LOADING_CAP 1`

`-era_current_distribution_layer` *`layer_name`*	
	Default: None Specifies the layer name for distributing unplaced current in the early rail analysis mode.
`-era_current_distribution { unplaced | placed | all | none }`	

	Default: None This parameter controls the behavior of era current distribution. `set_power_data` area based power, or `current_region_file` need to be specified for ERA current distribution to work. Placed instanced without uti or ascii based power can also be considered for era current distribution. • Unplaced: Enable current distribution only for unplaced instances. If `set_power_data` area based power is specified, `-era_current_distribution_layer` will be required. • Placed: Enable current distribution for placed instances without any power specified. • All: Both unplaced and placed instances without power specified; will have ERA current. • None: Disable ERA current distribution.
`-era_current_distribution_nets {`*`net1 net2 ...`*`}`	
	Default: None Specifies to distribute current for unplaced instances by total die area on specified nets only. The default behavior is to distribute current for all nets. This parameter will be required when `-era_current_distribution_unplaced_area diearea` is used. For single net analysis, you can either specify a single net (for example, `-era_current_distribution_nets {VDD_AO}`) or perform the analysis without this parameter.
`-era_current_distribution_unplaced_area {instance | diearea}`	
	Default: `instance` Specifies to distribute current for unplaced instances by total die area or actual instance area. The default value is `instance`. When `diearea` is specified, the entire unplaced die area is considered for computation of current distribution. The die area considered excludes user-specified current regions, placed instances, and placement blockages.
`-era_add_virtual_followpin { standard | extended | none }`	
	Default: `none` Specifies to generate virtual followpins. The `extended` followpins will create followpins that extend from one stripe to another. The `standard` followpins may extend to previous stripe but does not reach the next stripe.

`-era_add_virtual_followpin_for_io {true | false}`	
	Default: `false` By default, ERA virtual follow pin insertion only works in the core area. When this parameter is set to `true`, ERA follow pin routing is extended to the IO area as well.
`-era_add_virtual_via_on_layers` *value*	
	Default: None Enables you to control on which layers virtual via can be inserted in early rail analysis. Using this parameter, virtual via can only be inserted between each pair of layer names provided in the design. Alternatively, you can provide wildcard, '*', as the second layer name and virtual via will be allowed to be inserted from the first layer to any layers in the design. For example, `-era_add_virtual_via_on_layers { {M1 *} }` will allow virtual via to be inserted from M1 to any layers in the design. This parameter does not have a default value but ERA will insert virtual via for all layers if -era_skip_virtual_via parameters are not provided.
`-era_instance_dynamic_current_file` *filename*	

Default: None

Specifies an instance/cell based PWL file in the dynamic ERA flow (`set_rail_analysis_config -era_dynamic`). This feature allows you to perform dynamic ERA without having to specify the dynamic .ptiavg/.ptiavk current files (`set_power_data`).

The specified PWL should cover the entire simulation period. If the simulation period is longer than the specified PWL, the software will assume 0 current for the remaining period (`set_dynamic_rail_simulation -stop <time>`). You can also use the `set_dynamic_rail_simulation -auto_repeat` parameter to repeat the dynamic current waveform till the `-stop` time specified, instead of assuming 0 current for the remaining period.

The format of the file is:

```
INSTANCE | CELL <NAME> PWL { <time> <current> } pwrnet|gndnet <net_name>
[ADD_OVERLAP|SUBTRACT_OVERLAP]
```

- Default time unit = `ns`.
- Default current unit = `mA`
- `pwrnet|gndnet` keyword indicates if the specified net is a power or ground net.
- `ADD_OVERLAP` keyword indicates to honor all overlapping instance currents, that is, instance to instance overlap and instance to region overlap.
- `SUBTRACT_OVERLAP` keyword indicates to honor instance current, and subtract its area from the overlapped X-Y region to apply the X-Y region current to the remainder area. `SUBTRACT_OVERLAP` is default. This keyword is not applicable for instance to instance overlap.

The following is a snippet of the specified file:

```
INSTANCE  top/instA PWL { 0ns 0mA 0.1ns 1mA 0.2ns 0.5mA } pwrnet VDD
INSTANCE top/instA PWL { 0ns 0mA 0.1ns -1mA 0.2ns -0.5mA } gndnet VSS
CELL macro_A { 0 0 0.1ns 1 0.2 0.5 }
...
```

If you have specified both the `-era_instance_dynamic_current_file` and `-era_current_region_file` parameters, and there are overlapping instances with the specified current region, the `SUBTRACT_OVERLAP` behavior is implemented by default.

`-era_lef_layer_map` *filename*

Default: None

If a technology library or an extraction techfile is not provided, ERA can automatically create a technology library and the layermap needed to generate the library. If the automatic layer map generation fails, you can use this parameter to manually specify a layer map file.

The format of the layer map file is:

File Format:

```
Layer Type tech_layername lefdef lefdef_layername
     metal     METAL_1    lefdef      M1
     via       VIA_1      lefdef      VIA1
     metal     METAL_2    lefdef      M2
     via       VIA_2      lefdef      VIA2
     metal     METAL_3    lefdef      M3
```

`-era_power_switch_file` *filename*

Default: None

If a power gate PGV is not provided, this parameter specifies the name of the power gate (power switch) file that will be used to perform power gate steady-state analysis.

File format:

```
CELL <CELLNAME> SUPPLY <SUPPLY_PIN> SWITCHED <SWITCHNET_PIN> RON <VALUE>
IDSAT <VALUE> ILEAK <VALUE>

CELL HDRSID0 SUPPLY TVDD SWITCHED VDD RON 500 IDSAT 1 ILEAK 0.001
```

`-era_skip_virtual_via_by_type {whatif | def | all | none}`

Default: `all`

Specifies to skip a given via type. By default, ERA generates all virtual via layer types.

- `whatif` vias are virtual vias that have connectivity to user-defined what if shapes.
- `def` vias are virtual vias between two metal shapes defined in DEF.
- `all` will skip all virtual via generation.
- `none` will insert both what-if and DEF vias. It will insert vias on all layers, unless the layers are controlled by other parameters.

`-era_skip_virtual_via_on_layers {{` *layer1 layer2* `} { layer3` *layer4* `} ... }`

	Default: None Skips via insertion between stripes and non-stripes, on the specified LEF layer pairs.
	{ *layer1 layer2* } { *layer3 layer4* } *...:* Specifies the LEF layer names.
`-era_tech_lib_generation {true | false}`	
	Default: `true` Specifies to skip technology PGV generation in the early rail analysis mode. This parameter skips technology PGV generation even if the `-extraction_tech_file` parameter is specified.
`-extract_subconductor_layers {true | false}`	
	Default: `false` Specifies to perform extraction of the sub-conductor layers.
`-extraction_tech_file` *filename*	
	Default: None Specifies the extraction technology file to be used for top-level power-grid extraction. If you do not specify this parameter, the software will use the extraction technology file stored inside the power-grid view library.
`-feol_scale_factor` *non_negative_integer_number*	
	Default: None Specifies to scale the front-end-of-line (FEOL) self-heat value for all instances in the self-heating analysis flow. The delta T FEOL will be calculated based on the static power, thermal resistance, and FEOL scale factor.
`-filler_cell_list {` *cell1 cell2 ... celln* `}`	
	Default: None Specifies a list of filler cells. This option is used when the filler cells are not tagged in the power-grid library.
`-fine_grain_power_switch_r_on {min avg max}`	
	Default: `max` Specifies whether the minimum, maximum, or average Ron will be used for a powergate cell in finegrain memory characterization with multiple Ron values. The default value is `max`.

`-fine_grain_power_switch_r_on_list` *filename*	

	Default: None
	Specifies the name of a file containing the powergate cell names and their corresponding Ron values (minimum, maximum, or average) to be used in finegrain memory characterization with multiple Ron values. The format of the file is:
	`CELL cell1 min/max/avg`
	`CELL cell2 min/max/avg`
	`INST inst1 min/max/avg`
	Note: The `-fine_grain_power_switch_r_on` and `-fine_grain_power_switch_r_on_list` parameters allow you to specify multiple Ron values in a scenario where the Ron value can change with respect to different modes of simulation.

`-fine_pre_simulation_period` *value*	
	Default: `0` Specifies the fine-grain pre-simulation period that rail analysis will use. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns (nanosecond) will be applied. The time step in the fine-grain pre-simulation period will be the same as the simulation period. Pre-simulation allows you to determine the current profile and analyze IRdrop of the circuit when power supply is in steady state. **Note**: When both the coarse-grain (`-pre_simulation_period`) and fine-grain pre-simulation periods are specified, the fine-grain pre-simulation period will start after the coarse-grain pre-simulation period. The simulation phase starts after the fine-grain pre-simulation period. Both fine-grain pre-simulation and coarse-grain pre-simulation can be specified just by themselves without requiring the other one to be specified. In the case where fine-grain pre-simulation is not specified, the simulation will start right after the coarse-grain period.
`-flatten_xpgv_block_instances` *filename*	

	Default: None
	
	Specifies a file that includes a list of block instances that use DEF instead of xPGV for rail analysis, that is, specify the instances that are not to be used as xPGV. This parameter allows you to specify a few instances of the same block that will use DEF, and the others that will use xPGV at the top-level. If you do not specify this parameter, and if both DEF and xPGV are given for a block, xPGV takes precedence over DEF.
	
	Instance List File Format:
	`<block_name>  <block_instance_name|All|None>`
	
	In this file, you can specify if a block is using a DEF by specifying the block instance name, or with the `All` or `None` keywords. `All` indicates that all instances are using DEF, and `None` indicates that none of the instances are using DEF.
	
	Example:
	`CORE  /top/x1/x12/instA`
	
	For a design with three blocks, if you want to use the DEF and xPGV as following:
	
	Block_1 - > A1 (xPGV) & A2 (DEF)
	Block_2 - > B1 & B2 (xPGV for both instances)
	Block_3 - > C1 & C2 (DEF for both instances)
	
	The specified file will contain the following information:
	`Block_1 A2`
	`Block_2 None`
	`Block_3 All`

`-force_extraction {true | false}`	
	Default: `false` Specifies whether the existing work directory extracted data should be used, or the design needs to be re-extracted during rail analysis.
`-force_library_merging {true | false}`	
	Default: `false` Specifies to force merge PGVs with different resistivity and layers during rail analysis, wherein the first library definition will be used.
`-generate_multi_voltage_library {true | false}`	

	Default: `false`
	
	When this parameter is specified, all the PGVs for each cell will be considered, and voltage-dependent capacitance information will be available for use in rail analysis, similar to the multi-voltage PGVs.
	
	The `-generate_multi_voltage_library` parameter is used in a scenario when you have generated multiple PGVs for different voltages, instead of generating a single multi-voltage PGV that has voltage-dependent capacitance. The default value of the parameter is `false`.
	
	The following example shows the voltage-dependent capacitance table for the cell INV:
	`==========`
	
	`CellName Net Name Voltage Capacitance`
	`INV VDD 1.0 1.1e-15`
	`INV VDD 0.9 1.0e-15`
	
	Here, there are two characterization voltage values and their corresponding capacitance values. The rail analysis engine uses both the voltage-dependent capacitance values to perform interpolation based on the two voltage values.

`-gif_iv_threshold` *value*	
	Default: `1e-9` Specifies the instance voltage threshold for the combined instance voltage drop GIF for all the nets specified in the domain. The threshold is specified in voltage drop value. Any instance voltage drop value greater than the threshold will be put into the red bucket range in GIF, and the rest of the range will be linearly distributed down to the minimum instance voltage drop.
`-gif_new_color_scale {true | false}`	
	Default: `false` Specifies to change the color scheme of the GIF file generated in the state directory during the static and dynamic rail analysis flow to match the GUI color scale.
`-gif_resolution {low | medium | high}`	
	Default: `medium` Controls the resolution of the generated GIFs after rail analysis. This option can have the following values: `low`, `medium`, and `high`.
`-gif_zoom_area {` *x1 y1 x2 y2* `}`	

	Default: None Specifies to set the GIF generation bounding box to the given area (in microns).
`-gif_zoom_top_cell_die_area {true | false}`	
	Default: `false` Specifies to set the GIF generation bounding box to the topcell DIEAREA listed in the DEF file, instead of the total design size including RDL. If RDL is not used, this parameter has no effect. This parameter will change the bounding box of all the GIFs generated in rail analysis. **Note:** The `-gif_zoom_area` and `-gif_zoom_top_cell_die_area` parameters are mutually exclusive. If both the parameters are specified, the `-gif_zoom_area` parameter takes precedence.
`-hier_delimiter` *character*	
	Default: / Specifies the hierarchical delimiter in the DEF file. The default hierarchical delimiter is a forward slash (/) but can be changed by setting the `-hier_delimiter` parameter. You can use this parameter to override the existing hierarchical delimiter.
`-hier_pgv_block_lefs` *list_of_files*	
	Default: None Specifies the block LEF file for all hierarchical DEF blocks in the design. This will trigger automatic hierarchical PGV generation for the design hierarchy. The software will error out if any of the LEF files are missing for a DEF hierarchy.
`-hier_pgv_write_view {ir | em | all}`	
	Default: `all` Specifies the view type to generate for the hierarchical PGV. The allowed values are “`ir`”, “`em`” and “`all`”. “`all`” means both `ir` and `em` views will be created. If you choose to only generate `ir` or `em` view, disk consumption can be reduced since only one view will be generated.
`-honor_negative_static_current {true | false}`	

	Default: `false` Specifies to honor negative current value in the current file (`.ptiavg`) during static rail analysis.
`-ict_em_models` *file*	
	Default: None Specifies the ICT-EM file which only contains the EM rules and excludes the RC extraction data in ICT. The software will use the EM rules in the specified ICT-EM file to calculate EM limits for power net EM analysis even if there are embedded EM rules in the Quantus QRC tech file.
`-import_what_if_shapes {true | false}`	
	Default: `false` Specifies to import the what-if shapes created by the `add_what_if_shapes` command for rail analysis. **Note**: If both the `-import_what_if_shapes` and `-what_if_shapes_file` parameters are specified, union of two sets of what-if shapes is considered for analysis.
`-include_extraction_tool` *filename*	
	Default: None Specifies extractor (ZX) commands and variables in the form of an include file.
`-ignore_decaps {true | false}`	
	Default: `true` for static, `false` for dynamic Ignores decap cells during rail analysis. It ignores cells that are tagged as DECAP cells during library characterization or set as decap cells using the `set_rail_analysis_config -decap_cell_list` command parameter. If decaps are used to preserve connectivity on the follow-pin routing, you must set this parameter to `false`. Ignoring decaps during dynamic analysis is not recommended.
`-ignore_duplicate_voltage_source {true | false}`	

	Default: `true` Specifies to check for multiple voltage sources with the same name in the power pad (.ploc) file. When set to `true`, the duplicate voltage sources are ignored. When this parameter is set to `false` and duplicate voltage sources are present in the .ploc file, an error will be generated. If there are duplicate voltage sources, rail analysis will report an error message like the following: `** ERROR: (VOLTUS_RAIL-3647): Failed to run rail analysis due to redefined voltage source name in power pin location file(s) - /vsrc_same_name/PADs/VDD_AO.pp.`
`-ignore_fillers {true | false}`	
	Default: `true` Ignores filler cells during rail analysis if the list of filler cells are defined in PGV generation, or specified as fillers using the `set_rail_analysis_config -filler_cell_list` command parameter. During rail analysis, ignore library cells that are tagged as FILLER cells and have no capacitance associated with the interface nodes. If fillers are used to preserve connectivity on the follow-pin routing, you must set this parameter to `false`.
`-ignore_from_reporting` *`filename`*	

	Specifies the name of a file containing a list of cells, instances, and cell types (filler and decap) that are not to be included in the EIV, RLRP, and REFF reports.
	
	A sample file is given below:
	`INST AO2/MULT16_reg_2_6`
	`CELL BUFX*`
	`CELL_TYPE FILLER`

`-ignore_incomplete_net {true | false}`	
	Default: `false` Specifies to skip power and ground nets that are not defined in the design, or have been defined but with no instance connection.
`-ignore_nets_without_voltage_sources {true | false}`	

	Default: `false` Specifies if Rail Analysis should continue or exit with an error message if there are nets with no voltage sources. The possible arguments are: • `true`: prints an error message only but will allow the Rail Analysis run to continue. • `false`: exits immediately and returns to the Tcl prompt. If all the VDD/VSS nets have no voltage sources attached, the Rail Analysis run will error out and exit even if `-ignore_nets_without_voltage_sources` is set to `true`.
`-ignore_shorts {true | false}`	
	Default: `false` if it is not signoff mode This parameter allows you to ignore shorts that are found during rail extraction and continue rail analysis. However, `-ignore_shorts` cannot be applied if the design contains GDS.
`-inst_ignore_file` *filename*	

Specifies to exclude the LEAF instances mentioned in the specified file during rail extraction.

File Format:

```
Inst1
Inst2
.
.
.
InstN
```

If the instance name contains any special characters (such as ?, (, [, and so on) that are not treated as delimiters by default, prefix them with a backslash (\) to ensure correct processing.

Instances to be Ignored	Pattern to be provided
inst_B1/inst_1	*inst_B1/inst_1*
inst_B1/inst[11]	*inst_B1/inst\[11\]*
*inst_B1/inst_3**	*inst_B1/inst_3**
inst_B1/inst\/[4]	*inst_B1/inst\\/\[4\]*
inst_B1/inst\[5\]	*inst_B1/inst\\\[5\\\]*

Data_type: string, optional

`-ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}`

Default: None

Specifies an encrypted or unencrypted RC/EM technology (.ircx) file. The foundry-specific IRCX file, which contains the process information for extraction and EM modeling, can be used when the ICT-EM file is not available for electromigration analysis.

You can either specify two IRCX files, one file containing the RC process data (`RC.ircx`) and the other file containing EM rules (`EM.ircx`), or a single IRCX file that includes both the RC and EM sections (`RC.ircx`). A single IRCX file is used for the mature nodes, and separate RC and EM IRCX files are used for the advanced process nodes.

`-lef_map` *`lefMappingFile`*

	Default: None Specifies the LEF layermap file, which contains the mapping of the LEF layer and the technology layer. The LEF layermap file format is as follows: `layer map lef_layer_name tech_layer_name`
`-lifetime` *`value`*	
	Default: 87,600 hours (= 10 years) Specifies the lifetime value in hours.
`-lifetime_rating_factor_file` *`filename`*	

	Specifies a lifetime rating factor file that contains `jmax_life` values for each layer. These values are defined based on temperature and lifetime pairs. When you provide this file, EM analysis will automatically select the appropriate `jmax_life` factor for each layer according to the specific temperature-lifetime combinations defined within the file. The lifetime and EM temperature can be specified using the `-lifetime` and `-em_temperature` parameters.
	Following is a snippet of the lifetime rating factor file:
	`*Rating factor of M0/V0/M1/V1/M2/V3/M3...`
	`TEMPERATURE 90 95 100 105 110 ...`
	`LifeTime`
	`---`
	`0.5 2.578 2.242 2.024 1.774 1.547`
	`1 2.520 2.191 1.979 1.734 1.512`
	`2 2.430 2.113 1.908 1.672 1.458`
	`3 2.382 2.072 1.871 1.639 1.430`
	`...`
	`---`

`-limit_number_of_steps {true | false}`	

	Default: `true` Specifies to limit the number of steps (simulation period, and resolution or step size) during dynamic IRdrop analysis or power-up analysis. If the number of steps during dynamic IRdrop analysis or power-up analysis exceeds `1000`, the tool gives an error message and exits analysis. This parameter ensures that you are aware that there could be a problem in how the analysis is set up. If the performance risks are understood, you can set this parameter to `false` and re-run the analysis. The step-size and simulation period for the analysis is derived from current files generated by power analysis. You can explicitly set the step size and simulation period during power analysis using the `set_dynamic_power_simulation` command. During power-up analysis, the simulation period of the analysis is derived based on the turn-on time of the last power-switch in the power-up domain. If this time is too large, you may want to check if the turn-on times of the power-gates are correctly captured in the timing window file. Optionally, you can reduce the number of steps by explicitly specifying the stop time during rail analysis using the `set_dynamic_rail_simulation` command.
`-lowest_layer_for_em_check` *layer_name*	
	Default: None Specifies the layers that should not be displayed in the EM plots (such as, rj, jrms, and javg) and reports. *<layer_name>* is the layer below which the EM plots and reports will not be generated.
`-mcp_model_mapping {{`*<mcp model name1>* *<die DEF name1>*`} {`*<mcp model name2>* *<die DEF name2>*`} ...}`	
	Default: None Specifies to set the mapping between the package model and die name.
`-method {static | dynamic | era_static | era_dynamic}`	
	Default: `static` Specifies whether static or dynamic rail analysis, or static or dynamic early rail analysis will be performed.
`-exclude_nets` *net_name_list*	

	Default: None Specifies a list of nets to exclude from analysis. Used in designs with power switches.
`-off_state_powergate_instance_file` *filename*	
	Specifies the list of the off-state powergates. When specified, the powergate report will show the Roff (resistance of the cell when it is switched off) value and set the status of those powergate cells to Off. The specified file lists one powergate instance name in each line. Wildcard (*) is supported in the file.
`-override_set_power_data {true | false}`	
	Default: `false` Specifies to override the previous settings of the `set_power_data` command and use the last specified setting of `set_power_data` when targeting the same design object using the same current file. This feature is convenient when many `set_power_data` commands have been set and you want to override one of them with a new scaling factor or other options. **Example**: In the following example, the first command will be discarded and the second command will be honored: • `set_power_data -format current -instance_name XYZ dynamic_VDD.ptiavg` • `set_power_data -format current -instance_name XYZ dynamic_VDD.ptiavg -scale 2.0`
`-package_trace_connectivity {true | false}`	
	Default: `false` When `-package_trace_connectivity` is set to `true`, connectivity tracing is enabled from the die pins to the board pins of all the elements in the package model file. When this parameter is set to `false`, connectivity tracing is disabled.
`-peak_em_analysis {true | false}`	
	Default: `false` Specifies to calculate the peak EM in the dynamic rail analysis run. This parameter generates the `<net_name>.rj.peak.rpt` report.

`-power_grid_libraries` *dir_list*	
	Default: None Specifies the name of the power-grid view library directories.
`-power_up_fast_mode {true | false}`	
	Default: `true` Specifies to enable fast mode simulation for native power-up analysis. When the fast mode is enabled, the software uses lumped parasitic model for all instances connected to the powering up net. The fast mode native power-up analysis allows you to quickly estimate turn-on time and rush current injected in the global power-grid as the power-gated net is powering up. The fast-mode power-up analysis gives faster turn-around time and similar accuracy as accurate power-up analysis.
`-power_up_sequence_file` *filename*	

	Default: None
	Specifies the name of the file containing the firing time for each power gate instance. The power-up sequence file format is as follows:
	`<power gate instance name> <pin name> <always- on net> <switched net> <rise_time> <rise_slew> <fall_time> <fall_slew> <control pin voltage>`
	`<control pin voltage>` is optional.
	A sample power-up sequence file is given below:
	`inst_name E VCC_NORMAL VCC_SWITCHED 5.50E-09 2.90E-10 5.50E-09 2.90E-10 0.84`
	`inst_name E VCC_NORMAL VCC_SWITCHED 5.93E-09 7.23E-10 5.93E-09 7.23E-10 0.84`

`-powering_up_net` *net_name1* **...** *netname_n*	
	Default: None Specifies a list of powering up nets.
`-powering_down_net` *net_name1... netname_n*	
	Default: None Specifies a list of powering down nets.
`-pre_chain_power_up_sequence_file` *filename*	

	Default: None
	Specifies a power-up sequence file that contains the pre-chain start/end and mother (data) chain start/end timing information. In the power-up flow, you can specify this pre-chain sequence file instead of a complete power-up sequence file to modify the ramp-up timing of different power switches using the timing constraint in the specified file. The timing information for the other power-gate instances is calculated using the pre-chain sequence file.
	The format of the file is:
	`Prechain_start <pggate_instance_name> <pin_name> <d_time> <d_delay> <d_slew>`
	`Prechain_end <pggate_instance_name>`
	`Datachain_start <pgagate_instance_name> <pin_name><m_time> <m_delay> <m_slew>`
	`Datachain_end pggate_instance_name`

`-pre_simulation_period` *value*	
	Default: `0` Specifies the pre-simulation period that rail analysis will use. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns (nanosecond) will be applied. The pre-simulation period include the coarse-grain and fine-grain pre-simulation periods. If `-fine_pre_simulation_period` is not specified, then the entire pre-simulation period will do coarse pre-simulation.
`-pre_simulation_resolution` *value*	
	Default: `1e-10s` Specifies the transient time step size or resolution. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ps (picosecond) will be applied.
`-print_gif_range_percentage {true | false}`	

Default: `false`

`Drop%`

- **IR** (File Location: `<rail_dir>/Reports/<power_net>/`)

  `ir_linear.gif`

  `ir_limit.gif`

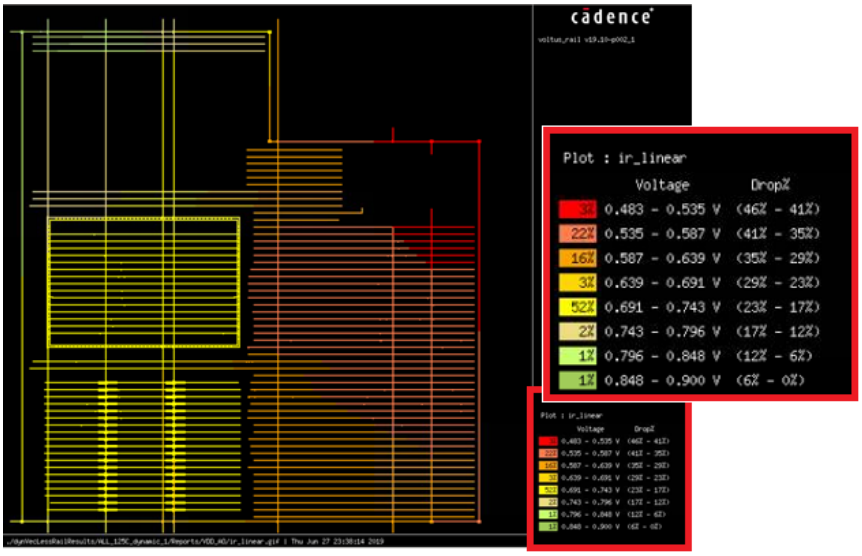
- **EIV** (File Location: `<rail_dir>/Reports/`)

  `<pg_pair>.<eiv_method>_eiv.gif`

  `<pg_pair>.<eiv_method>_limit_eiv.gif`

`IR Drop%` is calculated by 1-(voltage/ideal_voltage), and `EIV Drop%` is calculated by 1-(effective_voltage/ideal_effective_voltage)

The following image shows the voltage drop percentage for the `ir_linear` plot:



`-probe_instance_pin_waveforms {true | false}`

	Default: `false` Specifies to probe current and voltage waveforms for all the pins belonging to probing instances. When `-probe_instance_pin_waveforms` is set to `true`, it writes all the voltages and currents of the pins of probing instances to the waveform files. The use model of this parameter is: `set_rail_analysis_config -method dynamic -probe_instance_pin_waveforms true` The watched instance has the pin names in the waveform file, as shown below: Voltage waveform name: `<instance_name>:ir` Current waveform name: `TC_<instance_name>:ir`
`-probe_instance_tap_waveforms {true | false}`	
	Default: `false` Specifies to probe current and voltage waveforms for all tap nodes belonging to probing instances. When `-probe_instance_tap_waveforms` is set to `true`, it writes all the tap node voltages and currents of the instances from `-probe_waveform_list` to the waveform files. The use model of this parameter is: `set_rail_analysis_config -method dynamic -probe_instance_tap_waveforms true -probe_waveform_list` *`instance_list_file`* The instance's tap nodes are named based on instance name, and its node ID and location in the waveform file, as shown below: Voltage waveform name: `<instance_name>:N_...:X_..:Y_:ir` Current waveform name: `TC_<instance_name>:N_...:X_..:Y_...`
`-probe_package_interface_ports {true | false}`	
	Default: `false` Specifies to probe voltages for all the ports of the top-level subcircuit and the interfaces within the package. When this parameter is specified, the rail analysis engine retrieves the package interface ports from the package model and writes them into a probing node file to watch their voltage waveforms.
`-probe_pin_voltage_list` *`filename`*	

	Default: None Writes the voltage of the interface nodes for the specified instance/cell to a block boundary voltage file. The software generates a block boundary voltage file with file name based on the block instance name, `<instance_name>.pp` for every instance/cell in the list.
`-probe_waveform_list` *filename*	

	Default: None
	Specifies the name of a file containing the instance and cell name(s) for which you want to capture voltage and current waveforms. This parameter allows you to control whether to probe voltage or current waveforms or both.
	You can specify the optional parameter `PROBE <VOLTAGE|CURRENT|ALL>` in the file. Default is `ALL`. When `VOLTAGE` or `CURRENT` is specified, only the voltage or current waveforms will be probed.
	The format of the file is:
	`CELL/INST <`*name*`>`
	Following is an example of the file:
	`INST I1_1`
	`INST I2_1`

`-probing_node_file` *file_name*	

Default: None

Specifies the probe node file containing the probe point on a die. The file format for specifying probing nodes is as follows:

```
<user defined unique probe name> <location = x> <location = y> <layer>
<Power/ground net name> type=rc/ir
```

When type is `rc`, probes resistor current, and when type is `ir`, probes node voltage. If type is not given, the tool probes node voltage by default.

Following is an example of the probing node file:

```
probe1 8.39 239.37 Metal_6 VDD_AO type=ir
probe1 8.39 234.37 Metal_6 VDD_AO type=rc
```

The generated waveform is stored in tran_<NET>.ptiavg.

- Voltage waveform name: N_<x>_<y>:ir:<probe name>
- Current waveform name: TC_NodeRes:x_..._y_...:x_..._y_...:<probe name>

Example
Output directory:
<output_dir>/ALL_125C_dynamic_1/reports/rail/waveforms/tran_VDD_AO.ptiavg

Waveform name in ptiavg:

- N_83900_2393700_6:ir:probe1
- TC_NodeRes:x_83900_y_2393700:x_83900_y_1483800:probe1

You can also specify the package node voltages and resistor currents that are to be probed in the following format:

```
<probe name> pkg ir <pkg circuit node name>
<probe name> pkg rc <pkg circuit resistor name>
```

Example

```
probe_1 18 16 METAL_1 VDD

# pkg node voltage
probe_pkg_1 pkg ir GND.in0
probe_pkg_6 pkg ir VDD.in2

# pkg resistor current
probe_pkg_cur_1 pkg rc R_1_1
probe_pkg_cur_2 pkg rc R_5_2
```

Note: The units of x and y should be in microns.

`-process_techgen_em_rules {true | false}`	
	Default: `false` Processes the EM rules defined in the extraction technology file. When `-process_techgen_em_rules` is specified with `-extraction_tech_file`, the extraction technology file will be processed. If `-process_techgen_em_rules` is specified without `-extraction_tech_file`, the .cl model will be processed. The `-process_techgen_em_rules` and `-em_models` parameters are mutually exclusive. In the static mode, if `-em_models` is not specified, or if `-process_techgen_em_rules` is not set to `true`, current density (RJ) analysis will be disabled due to lack of EM models.
`-process_bulk_pins_for_body_bias {true | false}`	
	Default: `false` When set to `true`, rail analysis will process current sinks on the cell's bulk pin connections. This parameter is used during body bias analysis, where a body bias net is connected to bulk pins of the cell. The bulk pins for a cell is defined in the cell library using the `set_pg_library_mode -bulk_power_pins -bulk_ground_pins` command parameter. If bulk pins are defined using `set_pg_library_mode -power_pins -ground_pins`, then the `-process_bulk_pins_for_body_bias` parameter must not be used.
`-promote_pin_shapes_only {filler | decap | both}`	
	Default: None Specifies to promote either only filler shapes, only decap shapes, or both types of pin shapes during rail analysis. When this parameter is specified, the interface nodes for the filler and decap cells are ignored. The `-promote_pin_shapes_only` parameter helps to reduce the resistor and node count for big designs without affecting connectivity. This parameter must be specified with `set_rail_analysis_config -enable_2d_partition_extraction true` (enabled by default for the XP mode).
`-rdl_def` *`def_file`*	

	Default: None
	Specifies to instantiate the RDL instance/DEF at the top level and merge it with the DEF of the top level block being analyzed.
	The following is the usage model:
	`set_rail_analysis_config -rdl_def <def_file> -rdl_placement { X Y }`
	For GUI display, the same parameters also work for `read_def`:
	`read_def -rdl_def <def_file> -rdl_placement { X Y }`

`-rdl_def_list {{rdl_filename1 x1 y1 orient1} {rdl_filename2 x2 y2 orient2} ...}`	
	Default: None Specifies to import the list of the RDL DEF files for the top-level rail analysis. This parameter allows to import multiple RDL DEFs with their location and rotation information in the top level. Example: `-rdl_def_list { {RDL1.def 0 0 N} {RDL2.def 100 200 N}}`
`-rdl_placement {X Y}`	
	Default: None Specifies to place the RDL at a given X,Y location (in micron).
`-rdl_orient { N|S|W|E|FN|FS|FE|FW }`	
	Default: None Specifies the orientation of the RDL DEF file.
`-read_thermal_map thermal_map_file`	
	Default: None Specifies to process the thermal map file to calculate resistance based on uniform temperature. Resistor scaling is supported for the tile-based power map file. Using the tile-base thermal map from Sigrity PowerDC, the software will scale the resistance value based on the temperature of the resistor's physical location, which provides more accurate IR and EM results with thermal impact. For more information on the thermal map file format, refer to the Thermal Map File section in the "File Formats" chapter of the *Voltus User Guide*.
`-record_inst_peak_current {true | false}`	

	Default: `false` Specifies to save the instance peak current during rail analysis in the rail state directory. This parameter allows you to check and debug occurrences of abnormal instance peak current, which may result in high IR drop. The default value of this parameter is `false`.
`-record_results_start_time` *time*	
	Default: `0` Records rail analysis results based on the specified start time. The default unit of this parameter is `ns`. The software will use the specified time for all types of analysis, for example, instance voltage calculation, dynamic waveforms, and power-gate optimization.
`-r_effective_detail_report {true|false}`	
	Default: `false` Specifies to include the following additional columns for instance pins in the effective resistance (`effr.rpt`) report: • `X` and `Y`: location (x/y coordinates) • `LAYER`: layer name • `PIN_NODES(FLOAT/TOTAL)`: number of floating nodes per total number of nodes of the instance By default, these columns are not included in `effr.rpt`. **Usage Guidelines**: • Applicable only to the resistance analysis flow (`report_resistance`). • Supports both the static and dynamic rail analyses. • Supports both the net-based and domain-based analyses. • Does not affect `domain_effr.rpt`.
`-r_effective_domain_report_format {1 | 2 | 3 | 4}`	

Default: `1`

Controls the format of the effective resistance analysis domain report (`domain_effr.rpt`). This parameter allows you to specify the level of detail to be displayed in the domain-based analysis report.

The following table gives the report format for the four format values:

Format Value	Format

1
 	
(1 is the default format)
	`# PASS/FAIL REFF INSTANCE PIN(Pin REFF Percentage)`
	
	Example:
	`FAIL 10.0 instanceName VDD1(50.00%) VDD2(30%) VSS1( 10.00%) VSS2(10%)`

2	`# PASS/FAIL REFF INSTANCE POWER-PIN GROUND-PIN (Pin REFF Percentage)`
	
	Example:
	`FAIL 6.0 instanceName VDD1(83.33%) VSS1(16.67%)`

3	`# PASS/FAIL REFF INSTANCE PIN (Pin Absolute REFF Value, Pin REFF Percentage)`
	
	Example:
	`FAIL 10.0 instanceName VDD1(5.0, 50.00%) VDD2(3.0, 30%) VSS1( 1.0, 10.00%) VSS2(1.0 10%)`

4	`# PASS/FAIL REFF INSTANCE PWRNET PIN Layer X Y % GNDNET PIN Layer X Y %`
	
	Example:
	`FAIL 22.709 instanceName VDD pinName layerName 571.340 473.060 value(37.49%) VSS pinName Layername 571.340 471.800 value(62.51%)`

Usage Guidelines:

- Applicable only to the resistance analysis flow (`report_resistance`).
- Supports both the static and dynamic rail analyses.
- Applicable only to the domain-based analysis. It is not supported in the net-based analysis flow.

`-r_effective_eval_nodes {port | tap}`

Default: `port`

Reports effective resistance based on the port (pin) or tap nodes. By default, the software uses the port nodes for REFF reporting.

`-r_effective_pin_report_layer { top | bottom | all }`

	Default: `all` Specifies which layer of the pin shapes are to be reported in the EFFR report. This parameter allows you specify which cell boundary pin layers will be used for the resistance report. The possible arguments of this parameter are: • `top`: Specifies to report only the pin shapes located on the highest pin layer. • `bottom`: Specifies to report only the pin shapes located on the lowest pin layer. • `all`: Specifies to report the pin shapes located on every layer. **Usage Guidelines**: • Applicable only to the resistance analysis flow (`report_resistance`). • Supports both the static and dynamic rail analyses. • Supports both the net-based and domain-based analyses.
`-r_effective_pin_report_method { best | worst }`	
	Default: `worst` Specifies whether to report the best or worst effective resistance value of an instance pin with multiple nodes. The possible arguments of these parameters are: • `best`: Reports the smallest resistance value among the nodes of instance pin shapes for REFF. • `worst`: Reports the largest resistance value among the nodes of instance pin shapes for REFF. **Usage Guidelines**: • Applicable only to the resistance analysis flow (`report_resistance`). • Supports both the static and dynamic rail analyses. • Supports both the net-based and domain-based analyses.
`-r_effective_report_all {true | false}`	
	Default: `false` Reports both fail and pass instance pins in the REFF report. By default, only the "FAIL" instance pins are reported.
`-remove_duplicate_inst {true | false}`	

	Default: `false` Specifies to remove duplicate cell instances during the DEF parsing stage of the rail analysis run. These duplicate cell instance names are from the same DEF file, and are also printed in a warning message so that you are not required to debug these disconnected instances. When this parameter is set to `true`, the duplicate cell instances within the same DEF file will not be reported as disconnected instances in the *Physically Disconnected Instances* section of the power-grid integrity report.
`-report_compress_level {0 | 1-9}`	
	Specifies to compress the rail output files in the GNU Zip format. The compressed files will have the suffix `.gz`. • `0`: default, no compression • `1-9`: the compression levels vary from 1 to 9. 1 provides the fastest compression speed but at a lower ratio, so the file size will be larger. 9 provides the highest compression ratio but at a lower speed.
`-report_instances_missing_current_data {true | false}`	
	Default: `false` Specifies to generate a report that includes a list of all instances with missing current data. This parameter is supported in both the net-based and domain-based analyses. When this parameter is specified, the `<net_name>.nocurrentdata.inst.rpt` file is created in the `<state directory>/<net_name>/` directory. In addition, the log file includes the annotation statistics for instances with current data available on a per-net basis, as shown below: `#Instances missing current data: #count/#total_instances (%#percent value)`
`-report_layer_based_ir_average {true | false}`	
	Default: `false` Specifies to add the average IR drop value to the layer-based IR drop report and log file.
`-report_layers_above_pin_for_instance_ir {true | false}`	
	Default: `false` Specifies to report nodes above the pin layer. With this parameter set to `true`, the report will also contain layers above the pin nodes to the top most routing layer. The reporting will be based on the largest rectangular bounding box of the instance.

`-report_power_options` *option_names*	
	Default: None This parameter is used when `-report_power_in_parallel` is set to `true`. When the `-report_power_in_parallel` parameter is specified, you will not run the `report_power` command, therefore the reporting parameters cannot be specified. Using the `-report_power_options` parameter, you can specify the required `report_power` command parameters that are to be executed in parallel with rail analysis.
`-report_power_domain {true | false}`	
	Default: `true` When set to `true`, the software generates an instance voltage file for each MSMV power domain in the design, e.g. VDD_VSS.iv and VDDm_VSS.iv where VDD and VDDm are MSMV power nets with common VSS net.
`-report_shorts {true | false}`	

	Default: `false`
	
	Specifies to check for power and ground shorts between two geometries belonging to different nets. The parameter performs power and ground short check for the following:
	• PG and PG nets
	• PG and signal nets
	• All extracted nets
	
	When the parameter is set to `true`, if shorts are detected, the file "`voltus_short.report`" will be generated in the extraction work directory. The format of the report file is:
	
	LayerName X-coord Y-coord NetName1 NetName2
	
	If one of the net is not an extracted net (that is, signal net or non-extracted PG net), it will be marked as N/A in the report.
	
	The following is the use model of the parameter:
	
	`set_rail_analysis_config -report_shorts true`
	
	The following is a snippet from `voltus_short.report`:
	
	`# Layer X Y NET NET`
	
	`AP 98.500 211.000 VSS N/A`
	`AP 98.500 219.550 VSS N/A`
	`AP 98.500 301.000 VDD25_LVTL N/A`
	`AP 98.500 309.550 VDD25_LVTL N/A`
	`M8 13007.800 2267.200 VSS VDD`
	`...`

`-report_power_in_parallel {true | false}`	

	Default: `false` Enables power analysis in parallel with rail analysis to improve the overall turnaround time. **Note**: When using the Voltus-XP flow with the `-report_power_in_parallel true` parameter, if there are any parameters specified using both `power` category attributes and `set_rail_analysis_config`, the tool will honor the settings from `set_rail_analysis_config`. For example, the power grid library defined in `set_rail_analysis_config` will be used for power analysis in Voltus-XP. For information on the parallel flow setup, refer to the "**Run Dynamic Power Analysis and Rail Analysis in Parallel**" section in the *Dynamic Power and IRDrop Analysis chapter* of the *Voltus User Guide*.
`-report_voltage_drop {true | false}`	
	Default: `false` When set to `true`, instead of reporting the actual (effective) instance voltage, all the values will be reported in terms of voltage drop/bounce. For example, if the full pwr-gnd is 1V, pwr is 0.9 and gnd is 0.1, the IV file will report 0.8 (0.9-0.1) if this parameter is set to `false`, or 0.2 (0.1drop+ 0.1bounce) if the parameter is set to `true`.
`-reset`	Specifies to reset all the `set_rail_analysis_config` command parameters to their default values.
`-reuse_state_directory` *dir_name*	
	Default: None Specifies an existing state directory. The software assumes that the design data (DEFs and GDS) and power-grid libraries are not changed, and will re-start analysis of the design using power-grid database inside the specified state directory. If the state directory does not contain the necessary power-grid database files, it will issue an error message and provide information on the missing files. For net based analysis, you should specify net-based state directory, as shown below: • `set_rail_analysis_config -reuse_state_directory` *VDD_25C_avg_1* For domain based analysis, you should specify the domain's state directory, as shown below: • `set_rail_analysis_config -reuse_state_directory` *domain_25C_dynamic_1* A new state directory will be generated for the new analysis with soft links to the old state directory files that were reused.

`-rlrp_cell_report_layer` *filename*	

	Default: None
	Specifies the cell names and layer of the pin shapes that are to be reported in the RLRP report. This parameter allows you to perform cell-based layer reporting for instance RLRP report.
	The format of the specified input file is:
	`#cell_name #netname #pinlayer`
	`netname` is either a netname or "`ALL`". If you specify `ALL` for a net name, all the nets in the analysis domain will be considered for RLRP reporting.
	Example of an Input File
	`PDVDD_18_18_NT_DR_V ALL {M9,M1}`
	`PDVDD_18_18_NT_DR_V DVDD_D1 {M9,M1}`
	`PDVDD_18_18_NT_DR_V DVDD_D1 {M9,M1} VSS_D1 {M2}`

`-rlrp_detail_report {true | false}`	
	Default: `false` Specifies to trace and report the total power gate resistance along the RLRP path for each instance. The RLRP report (*<netName>*`.rlrp_inst`) has the following additional columns for each instance: • `RLRP_TOTAL` - Total RLRP resistance including the power gate path (from the always-on and switched nets) • `NET_RES` - Total RLRP resistance excluding the power gate path • `PGATE_RES` - RLRP resistance for the power gate path • `PGATE_RON` - Number of power gate RON resistors The report allows you to trace the Ron resistors in each power gate instance and generate the RLRP report based on power gate instances. The default value of this parameter is `false`.
`-rlrp_eval_nodes {port | tap}`	
	Default: `port` Reports RLRP based on the port (pin) or tap nodes. By default, the software uses the port nodes for RLRP reporting.
-rlrp_percentage_threshold *value*	

	Default: `100` Specifies to output top x% of the instances with large RLRP.
`-rlrp_pin_based_report` {true | false}	
	Default: `false` Specifies to generate a pin-based RLRP report. When specified, a new column "*Pin*" is included in the RLRP report.
`-rlrp_pin_report_layer { top | bottom | all }`	
	Default: `all` Specifies which layer of the pin shapes are to be reported in the RLRP report. This parameter allows you specify which cell boundary pin layers will be used for the resistance report. The possible arguments of this parameter are: • `top`: Specifies to report only the pin shapes located on the highest pin layer. • `bottom`: Specifies to report only the pin shapes located on the lowest pin layer. • `all`: Specifies to report the pin shapes with best or worst resistance value among all layers.
`-rlrp_pin_report_method { best | worst | eiv_best | eiv_worst}`	
	Default: `worst` Specifies whether to report the best or worst RLRP to an instance pin with multiple nodes. The possible arguments of these parameters are: • `best`: Reports the smallest resistance value among the nodes of instance pin shapes for RLRP. • `worst`: Reports the largest resistance value among the nodes of instance pin shapes for RLRP. • `eiv_worst`: Reports the RLRP of the node that has the worst EIV value among the tap nodes of instance pin shapes. • `eiv_best`: Reports the RLRP of the node that has the best EIV value among the tap nodes of instance pin shapes.
-rlrp_threshold *value*	

	Default: `0.0` Specifies to ignore instances with RLRP less than the specified threshold value.
`-rms_em_analysis {true | false}`	
	Default: `false` Specifies to calculate the RMS EM in the dynamic rail analysis run. When not specified, it will be enabled automatically if the EM model files are specified. You can set this parameter to `false` to skip the RMS EM analysis even if the EM models are provided. This parameter generates the `<net_name>.rj.rms.rpt` report.
`-write_voltage_waveforms {true | false}`	
	Default: `false` When set to `true`, instance voltage waveforms are saved in the state directory (.ptiavg). This waveform file can be loaded in the `view_dynamic_waveform` command to probe voltage waveform for desired instances. This waveform file can be fed to Tempus to perform SPICE critical path analysis with IRdrop waveforms. It can be used in Substrate Noise Analysis as well.
`-scale_initial_condition_current` *filename*	

	Default: None
	Specifies a file containing the current scale factor for the initial condition of the specified nets. The parameter allows you to increase/decrease the initial current at time 0 to minimize oscillation in the package and to shorten the settling time.
	The format of the file is:
	`default <default scale factor for all nets>`
	`<net name> <scale factor for this net>`
	`...`
	The use model of this parameter is:
	`set_rail_analysis_config -method dynamic -scale_initial_condition_current` *scale_factor.file*
	The content of *scale_factor.file* is:
	`default 1.0`
	`VDD 0.8`
	`VSS 0.7`

`-seb_lifetime` *value*	

	Default: `43800` hours (that is, 5 years) Specifies the life time of product in hours.
`-seb_table` *`table_filename`*	
	Default: None Specifies the file name of the Failures in Time (FIT) table. This file is provided by the foundry. The file has the following columns: • Layer • MTF • Sigma • Activate energy • Current density exponent
`-seb_temperature` *`temperature`*	
	Default: If not specified, the internally calculated junction temperature is used Specifies the temperature used to update the median time to failure (MTF) with respect to the ambient temperature. • If this temperature is specified, it would be used globally for all the resistors. • If this temperature is not specified, the seb_temperature may take the following possible values depending on the specific circumstances: ◦ If detailed delta temperature file is not used for SEB, the `seb_temperature` always takes the same value as that used for EM analysis. ◦ If detailed delta temperature file is used for SEB, the seb_temperature is calculated by the following equation: `seb_temperature = env_temperature + delta_T` Where delta_T is the maximum temperature rise, including joule heat and coupling heat from FEOL, in the resistor's bounding box with zero extension.
`-set_analyze_bbox {`*`xmin ymin xmax ymax`*`}`	

	Specifies the X, Y, min and max coordinates of the bounding box in which the instances can be analyzed. The values of the coordinates are specified in microns (um). Based on physical partitioning, the bounding box coordinates analyzed by the tool could be same or greater than the user-specified coordinates. You can check the info message in the main Voltus log file to view the coordinates of the actual analyzed bounding box. The following is an example of info message that appears in the main Voltus log file: INFO: The analyzed bounding box co-ordinates of the design are { `0.0`, `0.0`, `355.0`, `380.0` }
`-shorting_resistance` *value*	
	Default: `0` Specifies the minimum shorting resistance value (in ohm) assigned to resistors. When specified, those resistors with resistance equal or less than the specified value are merged to reduce the size of the R network.
`-single_partition_extraction_layer {`*METAL_LAYER*`}`	
	Specifies to extract RC using a single partition on a top metal layer. This helps to minimize extraction fracturing at partition boundaries, leading to better RC grid quality. Example: `set_rail_analysis_config -single_partition_extraction_layer {METAL_8}`
`-single_partition_extraction_layer_list {`*METAL_LAYER_1 METAL_LAYER_2 …… }*	
	Specifies to extract RC using a single partition on multiple layers. This helps to minimize extraction fracturing at partition boundaries, leading to better RC grid quality. Example: `set_rail_analysis_config -single_partition_extraction_layer_list {METAL_5 METAL_8}`
`-skip_extraction {true | false}`	
	Default: `false` Specifies to reuse a work directory. It is recommended to use this parameter when you want to reuse the previous extraction work directory during the net-by-net rail analysis flow.

`-snap_layer_for_current_taps` *file_name*	

	Default: None
	
	Specifies the layer file name for adaptive current modeling. It snaps current taps from lower via layers to higher via layers for better performance. This feature is supported for EM view of both macro PGV and hierarchical PGV. The current taps from lower layers will be processed based on the specified tile size, and redistributed equally across all the vias within the tile at the higher via layers. The default tile size is `5umx5um (25 um2)` for macro PGV, and `25umx25um (625 um2)` for hierarchical PGV.
	
	For example, when running full chip analysis, you already have detailed analysis on the sub-blocks or hierarchical blocks so you might consider speeding up the analysis by merging the current taps from lower layer to spread it across higher via layers. The current drawn from within the PGV will still remain the same so the accuracy at the top level will not be affected.
	
	The format of the file is:
	
	#Instance Based
	`INST <inst_name> <via_layer_name> <tile_size>`
	
	#Cell Based
	`CELL <cell_name> <via_layer_name> <tile_size>`
	
	`via_layer_name` is the layer name used in the LEF file.
	`tile_size` is optional.

`-static_multi_mode_analysis {true | false}`	

Default: `false`

Specifies to analyze the worst-case static IR drop across multiple functional modes of operation for macros. When this parameter is specified, Rail Analysis will use the multi-mode static current file (`power_static_multimode_scenario_file`) to iterate over multiple scenarios for all macro instances, and output the worst-case IR and EM report. The generated report identifies the worst Differential Instance Voltage (DIV) and the scenario that produces worst DIV across all instances.

This parameter generates the following report in the Reports folder:

State_dir/Reports/powerNetName_gndNetName`.worst_scenario_div.rpt` - this file reports each instance's worst scenario. It lists the instances based on worst DIV in the ascending order to determine the worst among all worst instance DIVs.

The following is a snippet from the worst scenario report:

```
#INST_NAME      SCENARIO_ID   DIV
block19/ICG4/ICG    12     -0.107583
block19/ICG3/ICG    12     -0.107557
...
```

Here, `SCENARIO_ID` is the worst time in the current file.

`-static_trigger_file` *filename*

Default: None

Specifies a file containing user-defined xPGV current scaling information on a per PG net basis for static analysis. This parameter is used for scaling of currents stored in xPGV.

You must provide scaling for all PG nets associated with the xPGV in the trigger file. The parameter allows you to use the same xPGV in different designs, irrespective of the voltage at which the xPGV is generated.

The format of the user-specified trigger file for a net is:

`<INSTANCE|CELL>` *inst_name/cell_name time scale_factor mode ref net*

Instance-based scale factors get higher precedence than the cell- based scale factors.

Example of a Trigger File:

```
INSTANCE    inst_C1    0   3   IPPGV::DEFAULT::super_filter   0   VDDD1
INSTANCE    inst_C1    0   2   IPPGV::DEFAULT::super_filter   0   VDDC1
INSTANCE    inst_C1    0   3   IPPGV::DEFAULT::super_filter   0   VSSA1
```

`-enable_xpgv_scaling` and `-static_trigger_file` are mutually exclusive.

`-stream_file` *file*	**Default:** None Specifies a GDS file. Partial (RDL GDS in flip-chip) or full-chip GDS can be specified. You can view extracted parasitic network for GDS, but, in order to view GDS layout it needs to be imported as an OA database in Voltus.

`-stream_map` *file*	**Default:** None
	Specifies the GDS layermap for the GDS file specified using `-stream_file` parameter. The GDS layermap file format is as follows:
	#layer_type layer_name gds *gds_layer_number* <*gds_data_type*> <[port|text]>
	`layer via8 gds 85`
	`layer metal9 gds 74`
	`layer metal9 gds 126 1 text`

`-stream_offset` { *x y* }	
	Default: None Specifies the offset value used to adjust and shift the GDS of a Redistribution Layer (RDL) cell in the mixed DEF+GDS flow. This parameter is available only when the `-stream_file` parameter is specified.
`-stream_purpose {metalFill | flipChip | fullChip}`	
	Default: None Specifies the data type of the GDS provided. • `metalFill`: The GDS data will be used as fill metals for manufacturing effects calculation but the physical shapes will not be extracted. • `flipChip`: The GDS data contains only the RDL layers, and the extracted physical shapes will be merged with the DEF netlist. • `fullChip`: The GDS data contains physical shapes for the entire design.
`-stream_top_cell` *cell_name*	
	Specifies the name of the GDS top cell.
`-suppress_message` { *message_id* + }	

	Default: No messages are suppressed. The messages with the specified message ids will not be output.
`-switching_bit_file_for_eiv_calculation` *filename*	

	Default: None
	Specifies the file that includes the cell or instance based switch-bit information. This parameter allows you to override the switch-bit in a PGV, or specify the switch-bit for a specific cell or instance.
	Note: The switch-bit information is defined for memory cells to distinguish between the pre-charge and functional mode (read/write) operation. During pre-charge, the memory cells may have large IRdrops due to large demand current in a short period of time. However, these cells can tolerate the large IRdrops because word-lines are shut-off. To accurately measure the effect of IRdrop on the operation of a memory, the instance voltage should be calculated during the functional mode. The switch-bit signal is used by the software to calculate the instance voltage when memory is in the functional mode.
	The format of the specified file is:
	`CELL | INST <cell_name | instance_name>`
	`<MODE_NAME> <Tmin1> <Tmax1> <Tmin2> <Tmax2>`
	where,
	• *cell_name*/*instance_name* must have a PGV with the dynamic current waveform.
	• `MODE_NAME` must match one of the mode names in the PGV for the cell.
	• `Tmin` and `Tmax` is the window when the switch-bit is ON, and when EIV will be calculated. The default time unit is ns.
	The Rail Analysis engine reads this switch-bit information, and reports the elapsed and worst instance voltage drop (IVD) when the switch-bit is 1. Rail Analysis will generate two IVD values for a given instance in the EIV report:
	• Worst IVD across simulation (`ELAPSE_EIV`)
	• Worst IVD in switching window (`WIN_EIV`)

`-tmp_directory_name` *directory*	

	Default: None Specifies a directory to use for temporary files. You can also use the global environment variable `TMPDIR` to specify an alternative location for the temporary data. If both are specified, the directory specified using the `-tmp_directory_name` parameter will take precedence. Following is an example for setting the TMPDIR environment variable: `setenv TMPDIR ./temp_directory`
`-temperature` *value*	
	Default: `25C` Specifies the temperature in degree celcius during rail analysis.
`-top_cell_placement {X Y}`	

	Default: None
	Specifies to place the top DEF block at a given X,Y location (in micron) with respect to virtual top (0,0).
	The following is the usage model:
	`set_rail_analysis_config -top_cell_placement { X Y } -top_cell_orient { N|S|W|E|FN|FS|FE|FW }`
	For GUI display, the same parameters also work for `read_def`:
	`read_def -design_offset { X Y } -design_orient { N|S|W|E|FN|FS|FE|FW }`

`-top_cell_orient { N|S|W|E|FN|FS|FE|FW }`	
	Default: None Specifies the orientation of the top DEF block.
`-tsv_subckt_model_file_list` *filename*	

Default: None

Specifies the name of the external Through Silicon Via (TSV) subckt model files to be used for 3D-IC extraction. You can specify a list of files separated by space. Each file contains the subckt model information for TSV. The specified file has the Spice format.

A sample file is shown below.

```
.subckt TSV top bottom
R1 top n2 0.03075
R2 n2 bottom 0.03075
C1 n2 0 28f
.ends
```

`-unconnected_cell_ignore_file` *`filename`*

Default: None

Specifies the name of a file containing the list of cells and instances to be ignored while reporting connectivity issues in the .rpt files, log file, and GUI. This parameter allows you to filter out the cells/instances that are known to have connectivity related issues and are not to be fixed.

The format of the file is:

```
CELL/INST    Cell Name/Instance Name    Pin Name
```

Here:

- `CELL` and `INST` are keywords
- Pin names are not mandatory. If pin names are not given, the corresponding cell/instance is ignored.

Example:

```
CELL  AND                => ignore all AND cell(s)
INST  INST1              => ignore instance INST1
CELL  BLOCK2*            => Ignore all blocks with the name starting with BLOCK2
CELL    *                => ignore all cells
INST    *                => ignore all instances
CELL   AN1  VDD*         => ignore AN1 cells's VDD* pins
INST  INST2 {VDD1 VDD2}  => ignore multiple pins of instance INST2
INST   INS*     VDD      => ignore VDD pin of all instances with the name starting with INS
```

`-unconnected_die_package_pins {ignore | error | edit}`

	Default: `ignore` Specifies whether or not to ignore unconnected pins between die and package (if any), and consequently if the simulation should be paused, stopped, or continued. This option enables the manual connection between package and die through the *MCP Editor* utility if any disconnects are present.
`-unified_power_switch_flow {true | false}`	
	Specifies to enable the unified analysis of the always-on and switch nets. When set to `true`, the power-up and always-on analyses are performed in a single flow, wherein the switch nets and always-on nets are simulated together resulting in improved performance and accuracy. **Use Model**: • Always-on Flow: `set_rail_analysis_config -unified_power_switch_flow true` • Power-up Flow: `set_rail_analysis_config -unified_power_switch_flow true` and `-powering_up_net <powerup_rail_list>`
`-use_early_view_list filename`	
	Default: None Specify cells in the file for which you want to use Early power-grid views. The file format accepts cell name in each line. You can specify wildcards (*, ?, -) in cell names.
`-use_em_view_list filename`	
	Default: None Specify cells in the file for which you want to use EM power-grid views. The file format accepts cell name in each line. You can specify wildcards (*, ?, -) in cell names.
`-use_ir_view_list filename`	
	Default: None Specify cells in the file for which you want to use IR power-grid views. The file format accepts cell name in each line. You can specify wildcards (*, ?, -) in cell names.
`-use_rms_delta_t {true | false}`	

	Default: `false` When this parameter is set to `true`, the `delta_T` used to calculate `seb_temperature` would be based on the RMS delta T (joule heat only). If this parameter is set to `false`, the `delta_T` used to calculate `seb_temperature` would be based on the sum of self-heat delta T and RMS delta T.
`-verbosity {true | false}`	
	Default: `false` Specifies to get side logs for extraction and rail analysis.
`-verify_logical_connectivity {true | false}`	
	Default: `false` Specifies to report the list power and ground pins of each cell instance that do not have logical connectivity. The list of pins with missing logical connectivity are saved in a report file called `cell.unconnected_pins` under the extraction work directory.
`-voltage_source_search_distance` *value*	
	Default: `50` Specifies the search distance in microns for adding voltage sources at power pin locations.
`-watch_location_waveform { {`*layerName xCoord yCoord*`}+ }`	
	Default: None Specifies to perform node based voltage probing for rail analysis. You can use this parameter to observe voltage waveform at a given location, specified by x,y coordinates and the metal layer. The unit of x,y coordinates is um, and the layer name should be a LEF/DEF layer.
`-what_if_shapes_file` *filename*	
	Default: None Specifies to take the what-if shape specification from a file. This file is the file created with the `-write` parameter of the `add_what_if_shapes` command. **Note**: If both the `-import_what_if_shapes` and `-what_if_shapes_file` parameters are specified, union of two sets of what-if shapes is considered for analysis.
`-work_directory_name` *directory*	

	Default: None Specifies the work directory location for extraction data. When you specify this parameter, the software generates extraction temporary files in the specified directory.
`-write_hier_block_boundary_voltage {true | false}`	

Default: `false`

Specifies to control the generation of block boundary voltage (BBV) for all hierarchical block instances that use xPGVs. The xPGVs for the blocks can be generated using the xPGV flow (Refer to "Hierarchical Power Integrity Analysis with Extreme Modeling" in the *Voltus User Guide*). The default value of this parameter is `false`. The BBV file contains realistic static or dynamic voltages for all nodes in the xPGVs starting where the top-level power-grid connects to the xPGVs. Since the BBV file is generated in context of full-chip analysis, running block-level analysis using this file gives insight to IR drop problems on the lower layers not included in the xPGV. This top-down flow can be used to identify and fix IR drop problems at the partition level without having to re-run the expensive fullchip analysis.

`-write_hier_block_boundary_voltage` is a global parameter so all instances of xPGV will be generated. The output is written to the state directory for each power/ground net analyzed as '`instance_name.pp`'. The file contains '`putvsrc`' commands. You can output on a per-instance basis using the '`write block_boundary_voltage`' command in an include file, as shown below:

```
set_advanced_rail_options -voltus_rail_include_file_begin voltus_begin.inc
```

voltus_begin.inc:

```
write block_boundary_voltage instance_name output_file_name
```

For dynamic analysis, add `-watch`:

```
write block_boundary_voltage instance_name output_file_name -watch
```

The dynamic flow is recommended since transient information is not saved by default.

To use the output BBV file at the block level, return to the block-level execution script and change `set_power_pads -format` to `boundary` and include the BBV file:

```
set_power_pads -net net_name -format boundary -file bbv_file
```

Example of a BBV file for dynamic analysis:

```
* Boundary conditions for instance sample_instance
# putvsrc vsrc_# IR/Bounce x y PWLFILE pwl_file_for_node_location
putvsrc K4 Vbc_VSS_1 1.210359e-01 247.296 179.712 PWLFILE
sample_instance.pp.ptiavg 1
putvsrc K3 Vbc_VSS_2 1.210360e-01 247.296 179.712 PWLFILE
sample_instance.pp.ptiavg 2
putvsrc K4 Vbc_VSS_3 1.210097e-01 247.296 175.104 PWLFILE
sample_instance.pp.ptiavg 3
…
```

`-write_instance_reff_report` *file*

	Default: None
	Generates a layer-based instance effective resistance report for the specified cells/instances. The specified file contains the list of cells or instances for which the Reff report is to be generated. The format of the input file is:
	`CELL cell1`
	`CELL cell2`
	`INST inst1`
	For each cell or instance specified in the input file, the report has information such as layer-based effective resistance ranges, worst node location, average statistics, and interface node resistances. The output file will be stored under each net's report folder `<net>.instance_effr.rpt`. For example,
	`Reports/VDD/VDD.instance_effr.rpt`.
	Usage Guidelines:
	• Applicable only to the resistance analysis flow (`report_resistance`).
	• Supports both the static and dynamic rail analyses.
	• Supports both the net-based and domain-based analyses.

`-write_instance_ir_report` *`file`*	

	Default: None
	Specifies to generate a layer-based IR drop report for a set of cells/instances. The *`file`* contains the list of cells or instances for which the IR drop report is to be generated. The format of the file:
	`CELL cell1`
	`CELL cell2`
	`INST inst1`
	The output file will be stored under each net's report folder `<net>.instance_ir.rpt`.
	This feature is applicable for both net-based and domain-based analysis.

`-write_instance_pin_ir_report` *`file`*	

	Default: None
	Specifies to generate a detailed report containing boundary node IR information for a set of cells/instances.
	The file contains the list of cells or instances for which the IR drop report is to be generated. The format of the file is:
	`INST <instance_name>`
	`CELL <cell_name>`
	The instance pin IR report will list all the pins' node information (layer, location) and the worst IR drop value.
	The output file will be stored under each net's report folder `NET/<net>.instance_pin_ir.rpt`. The following is a snippet of the IR drop report:
	`instance: <NAME>`
	`cell: <CELL>`
	`pin name: <PIN>`
	`# pin nodes: <total count>`
	`Min/Avg/Max: <min/avg/max interface node IR>`
	`For each of the interface node:`
	`IR X Y Layer`
	`......`
	The interface node output is sorted in the descending order of IR drop (that is, worst IR drop first).

`-write_combinational_inst_voltage_drop_gif {true | false}`	
	Default: `false` Specifies to generate a combined instance voltage drop GIF file for all the nets specified in the domain. This GIF allows you to see the entire design's instance voltage plot.
`-write_demand_current_region_file` *filename*	

	Default: None
	Specifies a file that contains regions for custom demand current generation. This parameter allows to compute the total current within each given region, and sum up all tap currents that are outside the specified regions as a global port.
	The format of the region file is:
	`<REGION_PORT_LABEL> <XL YL XR YR> <NET>`
	here,
	• `REGION_PORT_LABEL` specifies a name for the region.
	• `XL YL XR YR` specifies the enclosing region coordinates in microns.
	• `NET` specifies the power/ground net name for analysis.
	Example file:
	`A 0 0 100 100 VDD_AO`
	When specified, one demand current waveform "`tran_<netname>_region.ptiavg`" will be generated per net in the rail output directory at:
	• For DP: `./Reports/Waveforms/tran_<netname>_region.ptiavg`
	• For XP: `./reports/rail/waveforms/tran_<netname>_region.ptiavg`

`-write_package_pin_current_voltage {true | false}`	

	Default: `false` Specifies to generate a file containing the transient current or static current and voltage information of the package pins for the package analysis flow. This file is generated after rail analysis. This parameter is used in both the Sigrity Package Analysis (SPA) and non-SPA package analysis flows: • SPA flow - used in Sigrity tools to do chip-aware package analysis. This parameter is enabled by default (set to `true`) in the SPA flow. • Non-SPA flow - allows you to check the current on package pins. The format of the generated report in the static analysis mode (3 column format) is: *pin_name1 current1 voltage1* *pin_name2 current2 voltage2* ... The format of the generated report in the dynamic analysis mode is: .SUBCKT *Subcircuit_Name <list of pins for power and ground nets>* *MCP header with package pin name IA1 A1 0 PWL t1 i1 t2 i2 t3 i3 t4 i4 t5 i5 +t6 i6 … .ENDS *Subcircuit_Name* Here, • `IA1` (first item) is the current source element name • `A1` (second item) and `0` (third item) are the power and ground nodes • `PWL` keyword is followed by the time and current values
`-xpgv_config_file` *filename*	

	Default: None
	Specifies block details, such as, block name for which xPGV is generated, via cut layer name that indicates to preserve the layers about the cut layer, and block LEF file names. This parameter is specified during block-level analysis.
	Configuration File Format:
	`CELL <block_name>`
	`CUT_LAYER <cut_via_layer_name>`
	`Lef <tech_lef_name  block_lef_name>`
	`END`
	Note: This parameter is not supported with the `report_resistance` command.

`-xp_cpu_per_job_power` *value*	
	Default: If this parameter is not specified, the value will be determined automatically based on the number of host resources. Specifies the number of CPUs per power analysis job in the XP mode (`-enable_xp`). The specified value must be greater than `0`. This value should not exceed the value specified with `set_multi_cpu_usage -cpu_per_remote_host`.
`-xp_cpu_per_job_simulation` *value*	
	Default: If this parameter is not specified, the value will be determined automatically based on the number of host resources. Specifies the number of CPUs per simulation job in the XP mode (`-enable_xp`). The specified value must be greater than `0`. This value should not exceed the value specified with `set_multi_cpu_usage -cpu_per_remote_host`.
`-xp_host_allocation_method {on_demand | at_startup}`	
	Default: `on_demand` Specifies the method for remote host allocation during distributed processing. The possible arguments are: • `at_startup` – Specifies to wait for all available remote hosts before starting rail analysis. This parameter allocates all available remote hosts specified with `set_multi_cpu_usage` at startup. • `on_demand` - Allocates the available remote hosts based on demand. *Default* : `on_demand`

`-xp_purge {full}`	
	Default: None Specifies to clean up disk after run completion to delete temporary directories created for each analysis step (data generated in the voltus_data directory). If this parameter is specified, the reuse parameters (`-reuse_state_directory` and `-xp_reuse_extraction_directory`) will not work for the current state directory in the future runs. This parameter is applicable only in the XP mode (`-enable_xp`).
`-xp_resume {true | false}`	
	Default: `false` Specifies to resume from the failed step, and skip the previously successful run steps. This parameter is applicable only in the XP mode (`-enable_xp`).
`-xp_reuse_extraction_directory` *`directory_name`*	
	Default: None Specifies the path to the state directory that contains the extraction power-grid database. This parameter allows you to skip extraction and reuse the extraction data from a previous run in the XP mode.
`-xp_reuse_hpt_directory` *`state directory`*	
	Default: None Specifies to reuse the hierarchical path tracing (HPT) data from the HPT directory of a previous run with the modified extraction options if there are no DEF changes in the component section or logical connectivity. However, you can still reuse the HPT data using this parameter if the DEF changes are related to routing shape.
`-xp_reuse_parse_def_directory` *`directory_name`*	
	Default: None Specifies to reuse the data of the parsed-DEF file from the specified directory. This parameter significantly reduces the runtime required for reading the large DEF files. **Example**: `set_rail_analysis_config -xp_reuse_parse_def_directory /DefHierReport/rail_dynamic/core_125C_dynamic_1`
`-xp_simulation_cpu_timeout` *`value`*	

	Default: `7200` Specifies the time limit to either acquire the number of CPUs requested at the start of the run or acquire the number of CPUs specified with the `-xp_simulation_min_cpu` parameter in the XP mode. When this parameter is specified, the tool will wait for the given time limit to acquire the CPUs to run simulation before exiting. The unit for timeout is seconds. If a timeout value is not specified, the tool will wait for 7200 seconds (two hours) to acquire the CPUs.
`-xp_simulation_min_cpu` *value*	
	Default: If this parameter is not specified, the tool performs simulation with the available number of CPUs. Specifies the minimum number of CPUs required for the rail simulation stage in the XP mode. The specified value must be greater than 0. When this parameter is specified, the tool waits to acquire the specified number of CPUs before performing simulation. This parameter can be used if you want to perform simulation with the minimum number of CPUs required to proceed with simulation instead of waiting for all the CPUs needed during the run.

Examples

- The following command sets the rail analysis mode to static and xd accuracy, along with some other parameters:

```
set_rail_analysis_config \
-method static \
-accuracy xd \
-power_grid_library fast_library/fast_allcells.cl \
-voltage_source_search_distance 50
```

- The following command is an example of setting up rail analysis in the XP mode:

```
#resume previously failed XP run
set_rail_analysis_config -enable_xp true -xp_resume true

set_rail_analysis_config -method dynamic -accuracy hd \
...
-enable_xp true \
-xp_host_allocation_method at_startup \
-xp_reuse_extraction_directory dynamic_rail/core_125C_dynamic_1 \
-xp_cpu_per_job_power 2 \
-xp_cpu_per_job_simulation 8 \
-xp_purge full
```

- The following is an example of net-based early rail static analysis on an unplaced or partial placed design where virtual via/followpin would be created. The power would be distributed to placed instances on the basis of area. For unplaced instances, current region would be created on the M1 layer.

```
set_pg_nets -net vdd -voltage 0.998 -threshold 0.99301
set_power_pads -net vdd -format xy -file vdd.pp
set_power_data -format area -bias_voltage 0.998 -power 2.15
set_rail_analysis_config -method era_static -accuracy xd -extraction_tech_file
./qrcTechFile -era_current_distribution_layer M1 -era_current_distribution unplaced -
era_skip_virtual_via_by_type none -era_insert_virtual_followpin extended
report_rail -type net vdd
```

- The following is an example of domain-based early rail static analysis with virtual via/followpin. Here, user-specified instance current files are used.

```
set_pg_nets -net VSS -voltage 0 -threshold 0.18
set_pg_nets -net VDDm -voltage 1 -threshold 0.856
set_pg_nets -net VDD -voltage 0.84 -threshold 0.756
set_power_pads -net VDD -format xy -file vdd.pp
```

```
set_power_pads -net VDDm -format xy -file vddm.pp
set_power_pads -net VSS -format xy -file vss.pp
set_power_data -format current {instance_current_files/static_VSS.ptiavg
instance_current_files/static_VDD.ptiavg instance_current_files/static_VDDm.ptiavg
instance_current_files /static_VDDlu.ptiavg instance_current_files/static_VDDau.ptiavg}
set_rail_analysis_config -method era_static -accuracy xd -extraction_tech_file
typical.tch -era_power_switch_file pwrgates_port.txt -era_skip_virtual_via_by_type none -
era_insert_virtual_followpin extended
set_rail_analysis_domain -name PD -pwrnets {VDD VDDm} -gndnets VSS
report_rail -type domain PD
```

- The following is an example of net-based early rail analysis with user specified ASCII instance and current region file. Here, all virtual vias and followpin creation have been skipped.

```
set_pg_nets -net vss -voltage 0.0 -threshold 0.01
set_power_pads -net vss -format xy -file vss.pp
set_power_data -format ascii -bias_voltage 1 -power 1.5 instance_power_file.txt
set_power_data -bias_voltage 1 -power 1.5
set_rail_analysis_config -method era_static -accuracy xd -extraction_tech_file
./qrcTechFile -era_current_region_file ./cur_region_file -era_skip_virtual_via_on_layers
MI M2
report_rail -type net vdd
```

- The following is an example of static early rail analysis with current distribution for unplaced instances by total die area:

```
set_rail_analysis_config -power_grid_library fast_library/fast_allcells.cl -
extraction_tech_file typical.tch -accuracy hd -ignore_shorts true  -method era_static -
enable_rlrp_analysis true -voltage_source_search_distance 50  -era_current_distribution
unplaced -era_current_distribution_layer Metal1 -era_current_distribution_unplaced_area
diearea -era_tech_lib_generation false -era_check_wires_for_generated_current_regions
true
```

- Let us consider a design with cell type A that has the M1 and M2 PG pin shapes, and cell type B that has the M3 and M4 PG pin shapes.
  The following command calculates the reff values of the interface nodes on the M2 pin shapes for the cell type A and on the M4 pin shapes for the cell type B, and reports the worst (biggest) value among them. Similarly, it calculates the rlrp values of the interface nodes on the M1 pin shapes for the cell type A and on the M3 pin shapes for the cell type B, and reports the best (smallest) value among them.

```
set_rail_analysis_config \
      -r_effective_pin_report_layer top \
```

```
      -r_effective_pin_report_method worst \
      -enable_rlrp_analysis true \
      -rlrp_pin_report_layer bottom \
      -rlrp_pin_report_method best \
…
report_resistance
```

- The following command reads the switch-bit information in the specified memory cells:

```
set_rail_analysis_config \
  -power_grid_library "${Libgen_Golden_Pgv}" \
  -accuracy hd \
  -ignore_shorts true \
  -method dynamic \
  -enable_rlrp_analysis true \
  -eiv_eval_window switching  \
  -limit_number_of_steps false \
  -watch_location_waveform u0 \
  -switching_bit_file_for_eiv_calculation  sw.list
```

  The following is a snippet of the sw.list file:

```
CELL MEM128Kx32
READ 0.1ns 0.13ns
WRITE 0.1ns 0.15ns
CELL MEM64Kx32
READWRITE  0.1ns 0.2ns 0.3ns 0.4ns
```

  The following is a snippet of the report:

```
INST_NAME WIN_EIV PWR_WIN_IV GND_WIN_IV WIN_TIME ELAPSE_EIV PWR_ELAPSE_IV GND_ELAPSE_IV
ELAPSE_TIME CELL_NAME
*"NA" means the instance is disconnected from the net or the instance does not have
timing/switching window
BEGIN
- external/MULT8_reg_3_6 0.75841 0.85220 0.09379 48.470 0.75841 0.85220 0.09379 48.470
SDFFQXL
```

- The following command specifies the instance-based PWL current file in the dynamic ERA flow:

```
set_rail_analysis_config -method era_dynamic -accuracy hd -vsrc_search_distance 500 -
era_instance_dynamic_current_file ./inst_cell_pwl_file -power_grid_library
./../techonly.cl
```

  The following is a snippet of the PWL file:

```
instance ram/ram3 net VDD pwl (0ns 0mA 0.1ns 1mA 0.2nA 0.5mA) intrinsic_cap 10pf
loading_cap 60pf
instance ram/ram2 net VDD pwl (0ns 0mA 0.1ns 1mA 0.2nA 0.5mA) intrinsic_cap 10pf
```

```
loading_cap 60pf
...
```

- The following command specifies different temperature values to the layers M1, M2, M3, VIA1, VIA2, and VIA3 for average current limit analysis:

  ```
  set_rail_analysis_config
  ...
     -em_temperature 110 \ #temperature for the remaining layers
     -em_temperature_layer_list {M1 125 M2 115 M3 105 VIA1 115 VIA2 105 VIA3 120} #layer-based temperature
  ```

  The log file will display the following info message:

  ```
  Layer specific EM temperature will be used for AVG check for layer M1(125C), M2(115C), M3(105C), VIA1(115C), VIA2(105C), VIA3(120C).
  ```

  The header of the power EM report displays the layer-based temperature, as shown below:

  ```
  # Temperature: 110.0 (C)
  # Jmax_factor:
  # ……
  #   M2  0.981000 @ 115.0 (C)
  #  ……
  ```

- The following command specifies to perform unified power switch flow:

  ```
  set_rail_analysis_config -method dynamic -accuracy hd \
  ....
  -powering_up_net "VDD_sw1 VDD_sw2" \
  -enable_xp true \
  -unified_power_switch_flow true
  ```

Related Topics

- File Formats
- Glossary
- Static Power, IRdrop and EM Analysis
- Dynamic Power and IRDrop Analysis

set_rail_analysis_domain

```
set_rail_analysis_domain
[-help]
-domain_name domain_name
-ground_nets ground_net_list
-power_nets power_net_list
-signal_nets signal_nets_list
[-threshold value]
```

Specifies a rail analysis power domain.

⚠ For a power-gated domain, you should only specify the "always on" net. The switched net should not be specified as part of power domain definition, because it is traced by the program automatically.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_rail_analysis_domain` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_rail_analysis_domain`
`-ground_nets` *`ground_net_list`*	
	Specifies a list of ground nets that are in the domain.
`-domain_name` *`domain_name`*	
	Specifies the name of the power domain.
`-power_nets` *`power_net_list`*	
	Specifies a list of power nets that are in the domain.
`-signal_nets` *`signal_nets_list`*	
	Specifies a list of signal nets.
`-threshold` *`value`*	
	Specifies the minimum allowed voltage on the power net or the maximum rise on the ground net, depending on the setting of the `-voltage` parameter. The `-threshold` parameter is used to verify that the power grid passes the limit check and to set the filters for an IR plot. **Note:** This parameter does not support domains with MSMV power nets. If you have specified both net (`set_pg_nets -threshold`) and domain-based (`set_rail_analysis_domain -threshold`) thresholds, the net-based threshold will be honored. The valid value for threshold is between 0 and 0.5 (50%). For example, `set_rail_analysis_domain -threshold 0.1` indicates that 0.1 is 10% IRdrop. If a power net in the domain has nominal voltage of 1v, 10% of threshold will translate to 100mV of maximum effective IRdrop between the power and ground net.

Examples

- The following command sets the power domain `TDSP` to include `VDD_TDSPCore_R` and `VSS` nets:

  ```
  set_rail_analysis_domain -domain_name TDSP -power_nets VDD_TDSPCore_R -ground_nets VSS
  ```

set_self_heat_analysis_mode

```
set_self_heat_analysis_mode
[-help]
[-alpha_parameters {{<layer_name> < α_overlapping > < α_connecting >}+}]
[-beta_parameters {C1 C2 C3}]
[-cell_thermal_resistance_file filename]
[-detail_delta_temperature_file filename]
[-detail_delta_temperature_region {x1 y1 x2 y2}]
[-die_instance_input_signal_em filename]
[-die_instance_name dieinstname]
[-instance_delta_temperature_file filename]
[-instance_power_file filename]
[-parameter_file filename]
[-report_connected_pin_wire filename]
[-scale_rms_limit value]
[-tile_delta_temperature_file filename]
[-tiles {n_tile_in_x m_tile_in_y}]
```

Specifies the self-heating related options for each die instance. The command supports both single-die and multi-die flows.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `set_self_heat_analysis_mode` parameter. For a detailed description of the command and all of its parameters, use the man command `man set_self_heat_analysis_mode`.
`-alpha_parameters {{<layer_name> < αoverlapping > < αconnecting >}+}`	
	Specifies the alpha parameters for all layers. It is the metal layer effect.
`-beta_parameters {C1 C2 C3}`	
	Specifies the thermal coupling effect coefficient Value.
`-cell_thermal_resistance_file filename`	

	Specifies the name of the cell's equivalent thermal resistance file. The specified file contains the list of cells and their corresponding thermal resistance value. The unit of thermal resistance is in °C/W (temperature rise per unit rate of heat dissipation). The format of the file is: `<cell_name> <thermal_resistance_in_C/W>` To get instance deltaT, use half of the instance power multiplied by the cell thermal resistance.
`-detail_delta_temperature_file` *`filename`*	
	Specifies the name of the generated BEOL detailed delta-temperature file.
`-detail_delta_temperature_region {`*`x1 y1 x2 y2`*`}`	
	Specifies the region coordinates for generating detailed delta temperature file.
`-die_instance_input_signal_em` *`filename`*	
	Specifies the setup Tcl script for each die instance.
`-die_instance_name` *`dieinstname`*	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis (`set_rail_analysis_mode -die_mode multi-die`).
`-instance_delta_temperature_file` *`filename`*	
	Specifies the name of the generated FEOL instance-based delta-temperature file.
`-instance_power_file` *`filename`*	
	Specifies a file for instances and their corresponding static power. It is a two column format of <*`instance_name`*> and <*`total_power`*>.
`-parameter_file` *`filename`*	

Specifies the self-heating effect (SHE) analysis parameter file name (param.sh) required for SHE calculations. This file, provided by the foundry, contains essential alpha and beta coupling factors that the tool uses for coupledT calculations when modeling heat transfer from devices to metal layers.

Sample File:

```
HEADER {
FILE := Thermal_Coupling.ircx
}

BETA_C1_OD := 0.0055
BETA_C2_OD := -0.00065
BETA_C3_OD := 0.85000

* SELFHEAT_TABLE 17 5 (LAYER:V, ALPHA_CONNECT_OD:V, ALPHA_OVERLAP_OD:V,
ALPHA_HIGHR:V, BETA_HIGHR:V)
FIELD          ALPHA_CONNECT_OD   ALPHA_OVERLAP_OD     ALPHA_HIGHR    BETA_HIGHR
LAYER
AP                    0.90              0.30              0.35             0.85
M15                    .90              0.30              0.35
 0.85
M14                   0.90              0.30              0.35             0.85
```

`-report_connected_pin_wire` *filename*

Reports results on wires connecting to cell pins.

`-scale_rms_limit` *value*

Specifies the scale factor for the root-mean-square (rms) limit relaxation factor during Electromigration current limit analysis.

`-tile_delta_temperature_file` *filename*

Specifies the name of the generated FEOL/BEOL tile-based delta-temperature file.

`-tiles` *{n_tile_in_x m_tile_in_y}*

Specifies the number of tiles in the x and y directions in the tile-based delta temperature file.

Examples

- The following command is an example of `set_self_heat_analysis_mode` command:

```
set_self_heat_analysis_mode \
-die_instance_name die0 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} {M4 0.42 0.85} {M5 0.32
0.85} {M6 0.26 0.85} {M7 0.26 0.85} {M8 0.26 0.85} {M9 0.26 0.85} {M10 0.26 0.85} {M11
0.26 0.85} {M12 0.26 0.85}} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die0.rpt

set_self_heat_analysis_mode \
-die_instance_name die1 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} {M4 0.42 0.85} {M5 0.32
0.85} {M6 0.26 0.85} {M7 0.26 0.85} {M8 0.26 0.85} {M9 0.26 0.85} {M10 0.26 0.85} {M11
0.26 0.85} {M12 0.26 0.85}} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die1.rpt

set_self_heat_analysis_mode \
-die_instance_name die2 \
-alpha_parameters {{M1 0.74 0.85} {M2 0.62 0.85} {M3 0.53 0.85} {M4 0.42 0.85} {M5 0.32
0.85} {M6 0.26 0.85} {M7 0.26 0.85} {M8 0.26 0.85} {M9 0.26 0.85} {M10 0.26 0.85} {M11
0.26 0.85} {M12 0.26 0.85}} \
-beta_parameters {0.00423 -0.00045 0.81534}\
-instance_power_file ./dynamic_power.rpt \
-cell_thermal_resistance_file TRF.txt \
-instance_delta_temperature_file sh_itm.txt \
-tile_delta_temperature_file sh_ttm.txt \
-tiles {10 10} \
-detail_delta_temperature_file power_detail_die2.rpt
```

Related Topics

- "Self-Heating Effect Analysis" in the *Voltus User Guide*

set_voltage_regulator_module

```
set_voltage_regulator_module
[-help]
-component_list {VRM_DEF_component_list}
-ground_pin_mapping {{spice_pin_name[DEF_net_name]}+}
-input_power_pin_mapping {{spice_pin_name[DEF_net_name]}+}
-name vrm_name
-netlist vrm_spice_netlist_file_name
-output_power_pin_mapping {{spice_pin_name[DEF_net_name]}+}
-reset
-sub_circuit_name vrm_subckt_name
```

Analyzes voltage regulator effects. Use this command to enable the Vreg flow.

You must specify this command before running the `analyze_rail` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_voltage_regulator_module` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_voltage_regulator_module`
`-component_list {VRM_DEF_component_list}`	
	Specifies the names of the DEF instances which comprise the VRM. The component name must be the instance name used for the specified VRM. If this parameter is not specified, it is assumed that VRM is represented as a single MACRO on the die.
`-ground_pin_mapping {{spice_pin_name[DEF_net_name]}+}`	
	Specifies the mapping between the ground pin name in the SPICE netlist and the corresponding DEF net name for the same power pin.
`-input_power_pin_mapping {{spice_pin_name[DEF_net_name]}+}`	
	Specifies the mapping between the input power pin name in the SPICE netlist and the corresponding DEF net name for the same power pin.

`-name` *vrm_name*	
	Specifies the VRM name assigned to a voltage regulator. You need to specify this parameter for each voltage regulator in the design.
`-netlist` *vrm_spice_netlist_file_name*	
	Specifies the VRM netlist file in the SPICE format that is encapsulated in a single sub-circuit. This should also include SPICE models and testbench for each voltage regulator.
`-output_power_pin_mapping {{`*spice_pin_name*`[`*DEF_net_name*`]}+}`	
	Specifies the mapping between the output power pin name in the SPICE netlist and the corresponding DEF net name for the same power pin.
`-reset`	
	Resets all specified parameters back to default values.
`-sub_circuit_name` *vrm_subckt_name*	
	Specifies the subcircuit that corresponds to the VRM. This parameter is specified when the specified SPICE netlist has multiple subcircuit definitions. If this optional parameter is not specified, the first subcircuit found in the netlist file will be used.

Examples

- The following command performs Vreg analysis for the VRM `vrm1`:

```
set_voltage_regulator_module -name vrm1 -netlist ./TEST.spi -input_power_pin_mapping
{{VCC VCC}} -output_power_pin_mapping {{VDD VDD}} -ground_pin_mapping {{ VSS VSS }} -
sub_circuit_name LDO_MODEL -component_list { vmesh_reg_1 }
```

Related Topics

- "Static Power, IRdrop and EM Analysis" and "Dynamic Power and IRDrop Analysis" in the *Voltus User Guide*

set_voltus_diagnostics_mode

```
set_voltus_diagnostics_mode
[-help]
[-all_scenario_ranking {wiv | tiv}]
-config filename
[-db_mapping filename]
-domain  name
[-disable_layer filename]
[-ground_reff_threshold value]
[-ground_rlrp_threshold value]
[-hier_block_instance  names]
[-hierarchical {true | false}]
[-initialize]
[-input_insight_db database_path]
[-instance_mapping filename]
[-ir_effort {high||medium||low}]
[-master_list {list_of_master_blocks}]
[-master_mapping filename]
[-number_of_parallel value ]
[-output_insight_db database_path]
[-path_group {all | reg2reg}]
[-power_reff_threshold value]
[-power_rlrp_threshold value]
[-scenario_ranking {wiv | tiv | powerrlrp | powerreff | groundrlrp | groundreff}]
[-scenario_default list]
[-skip]
[-timing_effort {high||medium||low}]
[-timing_protection_config filename]
[-violation_input filename]
[-watch_box  ]
[-watch_inst ]
```

Specifies the user-defined settings related to the Voltus InsightAI flow. The command allows you to configure flows for various scenarios, such as differing EIV method, EIV evaluation window, and thresholds for each instance type. Using this command, you can tailor your configurations precisely, ensuring optimal performance for every scenario.

Parameters

```
-all_scenario_ranking {wiv | tiv}
```

	Specifies that all scenarios are ranked either based on the total number of violations (TIV) or worst violations (WIV).
`-help`	Outputs a brief description that includes the type and default information for each `set_voltus_diagnostics_mode` parameter. For a detailed description of the command and all of its parameters, use the man command man `set_voltus_diagnostics_mode`.
`-config` *`filename`*	
	(**Mandatory**) Specifies the name of the configuration file that includes details about the PG network definition, EIV method, EIV evaluation window, list of scenarios, and their setting.
`-db_mapping` *`filename`*	

	Specifies the name of the file that contains the mapping between a scenario and its corresponding rail directories. Each scenario can be mapped to multiple rail directories.
	Following is a snippet of the rail database mapping file (`db_mapping.txt`):
	`SCENARIO_1 {RailDB1 RailDB2}`
	`SCENARIO_2 {RailDB1}`

`-domain` *`name`*	
	(**Mandatory**) Specifies the name of the power domain.
`-disable_layer` *`filename`*	

	Specifies the name of the file that contains the list of layers to be disabled or excluded during pre-route and post-route RPG.
	The format of the file is:
	`Layer1`
	`Layer2`
	`Layer3`

`-ground_reff_threshold` *`value`*	
	Specifies that the violation report contains only instances that have values less than the specified ground REFF threshold.
`-ground_rlrp_threshold` *`value`*	
	Specifies that the violation report contains only instances that have values less than the specified ground RLRP threshold.

`-hier_block_instance` *names*	
	Specifies the list of instances of the master block that are to be repaired or reported. This parameter is specified with the `-hierarchical` parameter. When the instances are specified using this parameter, only the specified instances are repaired or reported.
`-hierarchical {true | false}`	
	Specifies to run the hierarchical repair flows (reinforce_pg, IR-aware placement, and ECO). When this parameter is set to `true`, you must specify at least one state directory in the configuration file. **Default**: `false`
`-initialize`	Saves the current power settings before running the Insight repair commands using internal power/rail values. After InsightAI-based analysis, this parameter enables restoration of the original power settings.
`-input_insight_db` *database_path*	

	Specifies the path of the input Voltus InsightAI database.
	After generating the output database using the `-output_insight_db` parameter (in the <output db>/master directory), the input database specified with `-input_insight_db` will be one of the master from the output database generation. You can then run `add_reinforce_pg` without loading the entire rail database, enabling you to fix issues without reloading the rail result.
	Example:
	1st Step: Output Database Generation
	`set_voltus_diagnostics_mode -config <insight_config> -db_mapping <db_mapping_file>` **`-output_insight_db`** `<output_db>`
	The database is generated under the directory <*output_db*>
	2nd Step: Insight Database Fixing
	`set_voltus_diagnostics_mode -config` **`-input_insight_db`** `<input_db>`
	`add_reinforce_pg -auto_ir_fix postroute`

`-instance_mapping` *filename*	

	Specifies the name of the file that contains the mapping between a scenario and its corresponding instance files. Each scenario has its own set of instance files. Each line in an instance file defines either an instance name or a block instance name.
	Following is a snippet of the instance mapping file (`instance_mapping.txt`):
	`SCENARIO_1 {instance1.txt instance2.txt}`
	`SCENARIO_2 {instance1.txt}`
	The content of the 2 instance files is shown below:
	`instance1.txt`
	`------------`
	`Inst_A*`
	`Inst_B*`
	`instance2.txt`
	`-------------`
	`Inst*`

`-ir_effort {high||medium||low}`	
	Specifies the effort level for running IR repair. The effort level for IR repair can be set to `high`, `medium`, or `low`. By default, the level is set to `high`. If there is an impact on Quality of Results (QOR), you may choose to lower the effort level. **Default**: `high` ⚠ To achieve optimal results, a balance between IR and timing efforts is crucial. The default setting is high IR and low timing effort, which generally provides the best IR fix rate. However, if there is a degradation in timing, you should increase the timing effort to safeguard it. If the timing continues to degrade even with a high timing effort, then you should reduce the IR effort.
`-master_list {`*`list_of_master_blocks`*`}`	
	Specifies the list of master blocks to be reported for IR repair.
`-master_mapping` *`filename`*	

	Specifies the name of the file that contains the mapping between a scenario and its corresponding master files. Each scenario has its own set of master files. Each line in a master file defines one master block.
	Following is a snippet of the master mapping file (master_mapping.txt):
	`SCENARIO_1 {master1.txt master2.txt}`
	`SCENARIO_2 {master1.txt}`
	The content of the 2 master files is shown below:
	`master1.txt`
	`-------------`
	`Top`
	`master2.txt`
	`-------------`
	`Super_filter`

`-number_of_parallel` *value*	
	Specifies the number of scenarios or databases that are to be analyzed.
`-output_insight_db` *database_path*	

	Specifies the path of the output Voltus InsightAI database.
	After generating the output database using this parameter (in the <output db>/master directory), the input database specified with `-input_insight_db` will be one of the master from the output database generation. You can then run `add_reinforce_pg` without loading the entire rail database, enabling you to fix issues without reloading the rail result.
	Example:
	1st Step: Output Database Generation
	`set_voltus_diagnostics_mode -config <`*insight_config*`> -db_mapping <`*db_mapping_file*`>` **`-output_insight_db`** `<`*output_db*`>`
	The database is generated under the directory <*output_db*>
	2nd Step: Insight Database Fixing
	`set_voltus_diagnostics_mode -config` **`-input_insight_db`** `<`*input_db*`>`
	`add_reinforce_pg -auto_ir_fix postroute`

`-path_group {all | reg2reg}`	

	Specifies which timing paths to protect during `reinforce_pg`. • ALL: Protects all timing paths • reg2reg: Protects only register-to-register paths
`-power_reff_threshold` *value*	
	Specifies that the violation report contains only instances that have values less than the specified power REFF threshold.
`-power_rlrp_threshold` *value*	
	Specifies that the violation report contains only instances that have values less than the specified ground RLRP threshold.
`-scenario_default` *list*	
	Specifies the list of default scenarios or databases.
`-scenario_ranking {wiv | tiv | powerrlrp | powerreff | groundrlrp | groundreff}`	
	Specifies that the scenario ranking is done for a single scenario. When specified, the ranking is done based on the given parameter.
`-skip`	Skips user-defined instances specified with `-watch_box` and `-watch_instances`.
`-timing_effort {high||medium||low}`	

	Specifies the effort level the software uses for timing optimization. The effort level for timing can be set to `high`, `medium`, or `low`. By default, the level is set to `low` to ensure best IR fix. If timing degrades, you may choose to increase the effort level. ⚠ To achieve optimal results, a balance between IR and timing efforts is crucial. The default setting is high IR and low timing effort, which generally provides the best IR fix rate. However, if there is a degradation in timing, you should increase the timing effort to safeguard it. If the timing continues to degrade even with a high timing effort, then you should reduce the IR effort.
`-timing_protection_config` *filename*	

	Defines timing degradation thresholds per timing view using a configuration file. A template of the configuration file for a design can be generated with `get_voltus_diagnostics_mode -timing_protection_config` after loading the design database in Innovus.
	
	The following is a snippet from the configuration file:
	
	`Global 0.0001`
	`View1 0.0002`
	
	This sets a global threshold of 0.0001 ps, with a specific threshold of 0.0002 ps for View1.

`-violation_input` *filename*	

	Enables repair operations based on violation specifications that align with an external signoff tool. This feature allows Innovus to replicate violation magnitudes for individual instances, ensuring consistent repair behavior between the external signoff tool and Innovus while eliminating the need for global threshold adjustments.
	
	You can specify a text file containing a per-instance violation specification in the following format:
	`instance_name threshold(mV) violating_amount(mV)`
	`Example:`
	`inst_1 100 35`
	`inst_2 80 20`
	
	For instance 1 (inst_1), an external signoff tool detected a 35mV violation against a 100mV threshold. This violation data allows Innovus to perform repairs using the same violation magnitude values reported by the external tool, ensuring consistency between the tools.

`-watch_box` *filename*	

	Specifies a file containing the list of regions, wherein only the instances inside these regions will be reported. You can write multiple regions in the file.
	
	The format of the file is:
	`{{x1,y1}{x2,y2}}`
	`{{x3,y3}{x4,y4}}`

`-watch_inst` *filename*	

	Specifies a file containing the list of instances to be reported for IR repair. You can write multiple instances in the file.
	
	The format of the file is:
	`instance_1`
	`Instance_2`

Examples

- The following command runs 2 scenarios with different eiv method, eiv_eval_window, and thresholds for each instance type (clock and data):

  ```
  set_voltus_diagnostics_mode -config insight.config -domain ALL -db_mapping
  db_mapping.txt
  ```

  Following is a snippet from the insight.config file:

  ```
  #### PG Network Definition ###Nets VDD VSSDomain_1 ALL  VDD   VSS#### Global Options
  ###eiv_eval_window {}eiv_method {}#### Scenario Definition ###SCENARIO_DEFAULT:
  SCENARIO_1  SCENARIO_2#### Scenarion Setting ###SCENARIO_1   eiv_eval_window elapse
   eiv_method Worst   Domain Domain_1        threshold_data 0.4        threshold_clock 0.4
   Net VSS        threshold_data 0.09        threshold_clock 0.09   Net VDD
  threshold_data 0.81        threshold_clock 0.81SCENARIO_2   eiv_eval_window switching
   eiv_method WorstAvg   Domain Domain_1        threshold_data 0.4        threshold_clock
  0.4   Net VSS        threshold_data 0.09        threshold_clock 0.09   Net VDD
  threshold_data 0.81        threshold_clock 0.81
  ```

Related Topics

- *"Voltus InsightAI"* in *Innovus Common UI User Guide*

view_dynamic_movie

```
view_dynamic_movie
[-type [ir|tc] ]
[-movies_directory dir]
```

Provides the ability to view movies created during dynamic analysis. Movies are sequences of analysis plots displayed over a specific simulation time.

Parameters

`-movies_directory` *dir*		
	Specifies the movie directory that was generated during dynamic rail analysis (that is, `state_directory/movie_analysis`)	
`-type [ir | tc]`		
	Specifies the movie type.	
	`ir`	IRdrop
	`tc`	Tap current

Examples

- The below command views the dynamic movie of type `ir` in the specified state directory:

```
view_dynamic_movie -type ir -movies_directory \
dynamic_rail/TDSP_25C_dynamic_1/VDD_TDSPCore_R/movie_analysis
```

Related Topics

- "Dynamic Power and IRDrop Analysis" in the *Voltus User Guide*

view_dynamic_waveform

```
view_dynamic_waveform
[-help]
[-clear_hier]
[-composite_waveform_name [hierarchy_name | clock_name]]
[-composite_waveform_type [hierarchy | clock | clock_with_seq
    | total_current]
[-effective_voltage_waveform]
[-free_memory]
[-hier_block_file filename]
[-instance_name instance_name]
[-power_db [powermeter.db]]
[-state_directory directory_name]
[-type [current | voltage | profile]]
[-waveform_files {file1 file2 ... fileN}]
```

Provides the ability to view dynamic current or voltage waveforms generated during analysis.

Parameters

`-clear_hier`	Specifies to clear the hierarchical block data that was loaded using the `-hier_block_file` parameter.
`-composite_waveform_name [hierarchy_name | clock_name]`	
	Only available for `-type current`. If `hierarchy_name` is specified, plots total current waveform for the specified hierarchy. If `clock_name` is specified, plots current waveform for the selected or all clocks. If `-composite_waveform_type clock_with_seq` is specified, plots total current waveform for all sequential cells. If `-composite_waveform_type total_current` option is specified, plots total current waveform for all instances in the current file.
`-composite_waveform_type [hierarchy | clock | clock_with_seq | total_current]`	

	You can specify to view a waveform for a specific hierarchy, a clock, clock with sequential cells, or total current.
`-effective_voltage_waveform`	
	Plots the effective voltage waveform (i.e. VDD - VSS). You must specify the voltage for two nets to generate the effective waveform.
`-free_memory`	Frees the memory but leaves the menu entries.
`-help`	Outputs a brief description that includes the type and default information for each `view_dynamic_waveform` parameter. For a detailed description of the command and all of its parameters, use the man command `man view_dynamic_waveform`.
`-hier_block_file` *`filename`*	

	Specifies to analyze the multiple (top/block-level) current (.ptiavg) files of a hierarchical design in the SimVision waveform viewer interface. This parameter can be used to easily find an instance of a specific block when there are multiple blocks with the same instance name.
	`-hier_block_file` specifies a text file that contains the list of blocks. For example, if the block name `BLOCK_A` is specified in the text file, you can match the instance named `BLOCK_A/Instance_A` and `Instance_A` in the `BLOCK_A`'s .ptiavg file.
	When specifying multiple block names in the text file, one block name should be given per line.
	The following is a snippet from the `block_hier` file:
	`u_top/vcore3`
	`u_top/vcore2`
	`u_top/vcore1`

`-instance_name` *`instance_name`*	
	Plot the waveform for the specified instance.
`-power_db [`*`powermeter`*`.db]`	
	Specifies the power database. Must be specified if `-composite_waveform_type` is `clock` or `clock_with_seq`.
`-state_directory` *`directory_name`*	

	Specifies to load the state directory where the current or voltage waveform files have been saved. The current and voltage waveforms are saved in the state directory generated during rail analysis. If you specify this state directory when viewing dynamic waveforms, the generated waveform file is automatically populated in the GUI.
`-type [current | voltage | profile]`	
	Specifies to display either current (power) or voltage waveform. The `profile` option displays the profiling db and the SimVision interface to view the histograms.
`-waveform_files {`*`file1 file2 ... fileN`*`}`	
	Specifies one or more current or voltage waveform files generated with a `.ptiavg` extension.

Examples

- The following command views dynamic composite total current waveforms from the file vddm_vddm.ptipeak:

```
view_dynamic_waveform -type current -composite_waveform_type total_current \
    -waveform_files vddm_vddm.ptipeak
```

- The following command views the dynamic current waveform for instance inst1:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -instance_name inst1
```

- The following command is for viewing the total current waveform:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -composite_waveform_type total_current
```

- The following command plots a clock domain composite waveform:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
```

```
    -power_db power.db \
    -composite_waveform_type clock \
    -composite_waveform_name clk
```

- The following command plots a hierarchical composite waveform:

```
view_dynamic_waveform \
    -type current \
    -waveform_files dynamic_vss.ptiavg \
    -composite_waveform_type hierarchy \
    -composite_waveform_name ethernet_mac_2
```

- The following command specifies to load multiple block-level current files:

```
view_dynamic_waveform -hier_block_file block_hier -waveform_files {PTIAVG1 PTIAVG2
PTIAVG3 …}
```

Related Topics

- "Dynamic Power and IRDrop Analysis" in the *Voltus User Guide*

view_package_results

```
view_package_results
[-help]
[-result_file <filename>]
-type <package_analysis_type>
[-workspace <workspacename>]
```

Specifies to view the package IR drop analysis results in the PowerDC GUI which will be invoked from within Voltus/Innovus.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-result_file <filename>`	
	Specifies the name of the .xml result file.
`-type <package_analysis_type>`	
	Specifies the type of package analysis. Currently, the software supports only the `dc` package analysis type.
`-workspace <workspace_name>`	
	Specifies the workspace name for package analysis (powerDC). The file name extension is `.pdcx`.

Example

- The following example specifies to view the package analysis results:

  ```
  view_package_results -type dc
  ```

write_current_region

```
write_current_region
[-help]
-reset | {-region x1 y1 x2 y2
[-current value | -current_pwl_file filename]
[-die_instance_name dieinstname]
[-file filename]
[-ground_net net_name]
[-intrinsic_capacitance value]
[-label name]
[-layer layername]
[-loading_capacitance value]
[-net net_name]
[-on_interdie_bump {true | false}]
[-polygon {x1 y1 x2 y2 x3 y3 ..x1 y1}]
[-power_net net_name]
[-series_resistance value]
}
```

Specifies static or dynamic current regions on global power-grids over the placed macro to accurately capture high and low current consuming regions during rail analysis.

You can use this command prior to rail analysis to specify current regions or power distribution over the top of power-grid on a user-defined layer. These current regions are generally specified for a placed macro for which power-grid view library is not available, and power/current distribution inside this macro could possibly impact IRdrop in neighboring regions. These current regions can also be defined over top-level power-grid for an unplaced macro during early stages of power-planning.

⚠ When you specify PWL currents over a region that includes instances, the current is distributed on nodes in the specified layer and not to each instance. The number of nodes to which current is distributed are determined by the tool so as to preserve the shape of the waveform.

Parameters

`-current value`

	In the static mode (set_rail_analysis_mode -method static), the parameter specifies a single current value in mAmp. In the dynamic mode (set_rail_analysis_mode -method dynamic) the parameter specifies a PWL current waveform in time (in ns) and current (in mA) format. For example, 0ns 0mA 0.5ns 1mA 1ns 2mA 3ns 1mA 4ns 0.
`-current_pwl_file` *filename*	

	Specifies a PWL current file. The file has two columns, time (in ns) and current value (in mA), as shown below:
	`time1 value1`
	`time2 value2`
	The `-current_pwl_file` and `-current` parameters are mutually exclusive.

`-die_instance_name` *dieinstname*	
	Specifies the name of the die to which you need to apply a current region. This parameter is only applicable to the multi-die analysis flow (`set_rail_analysis_config -die_mode multi-die`). The parameter allows you to accurately assign current to power and ground nets in specific geometric regions for individual dies during rail analysis in 3D-IC designs.
`-file` *filename*	

	Specifies the name of a file containing the list of current regions for each die. `-file` can be used only with the `-die_instance_name` parameter. The other parameters of the `write_current_region` command can be included within the specified file.
	Example:
	`write_current_region -file write_current_region_die0.tcl -die_instance_name Die0`
	`write_current_region -file write_current_region_die1.tcl -die_instance_name Die1`
	`###File content in write_current_region_die0.tcl`
	`-net VDD -current  50mA -layer M2 -region "10 10 100 100"`
	`-net VDD -current  50mA -layer M2 -region "100 100 200 200"`
	`-net VSS -current  100mA -layer M1 -region "10 10 200 200"`
	`###File content in write_current_region_die1.tcl`
	`-net VDD -current  150mA -layer M2 -region "10 10 50 50"`
	`-net VDD -current  50mA -layer M2 -region "100 100 200 200"`
	`-net VSS -current  200mA -layer M1 -region "10 10 200 200"`

`-help`	Outputs a brief description that includes the type and default information for each write_current_region parameter. For a detailed description of the command and all of its parameters, use the man command man write_current_region.
`-ground_net` *net_name*	
	Specifies a ground net name on which the specified current is to be applied.
`-intrinsic_capacitance` *value*	
	Specifies the cell intrinsic capacitance in pico farads (pf). This is required parameter when set in the dynamic mode (set_rail_analysis_mode -method dynamic). This parameter is disabled in static mode.
`-label` *name*	
	Specifies a name to a specific current region. The parameter allows to assign a user-specific name to the region, instead of using the default name generated by the tool. By assigning a unique name, you can easily identify and manage the regions. For instance, instead of having generic names like "Region 1," using specific names like "VDD_area1" or "VSS_area2" can provide immediate context.
`-layer` *layername*	
	Specifies a layer name on which the scaling will be performed. Multiple commands can be issued to scale resistance on multiple layers. This is a required parameter.
`-loading_capacitance value`	
	Specifies the cell loading capacitance in pico farads (pf). This is required parameter when set in the dynamic mode (`set_rail_analysis_mode -method dynamic`). This parameter is disabled in static mode.
`-net` *net_name*	
	Specifies a net name on which the specified current is to be applied.
`-on_interdie_bump {true | false}`	
	Specifies to create the tap nodes only on non-vsrc PLOC nodes. When the option is set to `true`, the number of total tap changes to include the tap nodes only on non-vsrc PLOC. You can check the change in total taps in the “Grid Statistics of Net XXX” section of the log file at `<state directory>/logs/design_report/dmg/ext_report.log`
`-polygon {`*x1 y1 x2 y2 x3 y3 ..x1 y1*`}`	

	Specifies a rectilinear current region where the current will be distributed within. The rectilinear region enables you to specify multiple x,y points to add current regions in the areas that are not rectangular in shape. When specified, the last xy coordinate location will be the same as the first xy location. The xy coordinates will be in the counterclockwise sequence.
`-power_net` *`net_name`*	
	Specifies a power net name on which the specified current is to be applied.
`-region` *`x1 y1 x2 y2`*	
	Specifies the region on the layer across which the specified current and capacitance (in dynamic mode) is distributed by the rail analysis engine. The units of x and y should be in microns. This is a required parameter.
`-reset`	Resets all the previous write_current_region command settings.
`-series_resistance` *`value`*	
	Specifies the series resistance (ESR) value attached to each tap point in the current region definition.

Examples

- The following command specifies static current region:
  ```
  write_current_region -current 2mA -layer METAL_1 -region "100 100 200 200"
  ```
- The following command specifies dynamic current region:
  ```
  write_current_region -current {0ns 0mA 0.5ns 1mA 1ns 2mA 3ns 1mA 4ns 0} -layer METAL_1 -region "200 200 400 400" -intrinsic_capacitance 40 -loading_capacitance 20
  ```
- The following command resets all write_current_region parameters to their default values:
  ```
  write_current_region -reset
  ```
- The following command specifies rectilinear current regions for die instance `Die1`:
  ```
  write_current_region -net VDD -current 50mA -layer Metal5 -polygon "10 10 100 10 100 100 10 100 10 10 " -die_instance_name Die1
  write_current_region -net VDD -current 50mA -layer Metal5 -polygon "100 100 200 100 200 200 100 200 100 100" -die_instance_name Die1
  ```
- The following command specifies the static current region with the region name `VDD_area1`:

```
write_current_region -label VDD_area1 -region "0.0 0.0 87.3 107.2" -layer M1 -current 1
mA -net VDD
```

Related Topics

- "What-If Rail Analysis" in the *Voltus User Guide*

write_die_model

```
write_die_model
[-help]
[-accuracy {xd | hd }]
[-disable_current_source_node_zero {true | false}]
[-disable_global_node_zero { true|false }]
[-enable_package_pin_map { true|false }]
[-enable_reference_node]
[-enable_xp {true | false}]
[-generate_sipi_port_definition_file {true | false}]
[-grid_reduction_cut_layer layer_name]
[-ignore_port_grouping_conflict]
[-local_reference_ground {true | false}]
[-max_ports_per_net integer]
[-multi_die {true | false}]
[-output_dir directory_name]
[-port_grouping_by_ploc_files { true|false }]
[-port_grouping_file filename]
[-port_grouping_tiles_filefilename]
[-probing_node_file file_name]
[-probing_region_filefilename]
[-rail_analysis_domain domain_name | -net net_name]
[-repeat_time time]
[-report_effective_rc]
-state_directory directory_name
[-suppress_negative_rc { true|false }]
[-tmp_directory_name directory]
[-use_sigrity_repeat]
[-write_effective_rc_deck]
```

Specifies to generate a reduced n-port die model in the form of equivalent die parasitic and current profile after rail analysis to perform system-level analysis with extracted package and board models. Resonance and electro-magnetic interference like system-level analyses can be performed using this die-model in package designing software like Allegro Package Design and Sigrity.

> This command requires a VTS-XL license.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_die_model` parameter. For a detailed description of the command and all of its parameters, use the man command: man `write_die_model`
`-accuracy {xd | hd }`	
	Specifies the accuracy mode for die-model generation. *Default* : `hd` • `xd`: xd or accelerated definition accuracy mode is used for faster die model generation with slight reduction in RC accuracy. • `hd`: hd or high definition accuracy mode is used for most accurate die model generation.
`-disable_current_source_node_zero {true | false}`	
	Specifies to disable node zero for current sources in a generated die-model. The current source node zero is used to connect decoupling capacitance and the current source in the generated die-model. This parameter is added to generate a die model without current source node zero. By default, a virtual ground is used to connect decoupling caps and the current source in the generated die-model. When set to `true`, generates die model without the virtual ground and uses the ground network for connecting decoupling caps and current sources. It also creates the <BLOCK>_nportModel.sp.isrc_balanced file in the die-model directory. This file has information on the current sources and their port connections. The .subckt file generated for Spice simulation points to the .isrc_balanced file. *Default*: `false`
`-disable_global_node_zero { true|false }`	

	Specifies to disable the global node zero in a generated die model. • When set to `true`, generates die models without the virtual ground and uses the ground network for connecting decoupling caps and current sources. • When set to `false`, a virtual ground is used to connect decoupling caps and the current source in the generated die-model. *Default*: `true`
`-enable_package_pin_map { true|false }`	

	Specifies to support the 6-column style MCP format to map package pins to circuit ports. The following are the old and new MCP file formats:
	The original 5-column format is:
	<vsrc_name> <Ckt_port> <Net_name> <X coord> <Y coord>
	The new 6-column format is:
	<package_pin> <Ckt_port> <Net_name> <X coord> <Y coord> <vsrc_name>
	Example:
	Original format:
	`chip1/bmp_100_6_12 D1_bmp_100_6_12 VDDE 350.000 147.000`
	`chip2/bmp_122_3_15 D2_bmp_122_3_15 VDD2 321.000 983.800`
	New format:
	`bmp_100_6_12 D1_bmp_100_6_12 VDDE 350.000 147.000 chip1/bmp_100_6_12`
	`bmp_122_3_15 D2_bmp_122_3_15 VDD2 321.000 983.800 chip2/bmp_122_3_15`

`-enable_reference_node`	
	Specifies to replace the global node zero with the reference port in the generated die model.
`-enable_xp {true | false}`	

	Specifies to generate the die model in the Extensively Parallel (XP) mode in which processing is distributed over a large number of machines. *Default* : `false`
`-generate_sipi_port_definition_file {true | false}`	
	Generates a ports definition file during die-model creation. When set to `true`, the port definitions are output to the `port_defintion.txt` file, which is later used by Sigrity SystemPI for connecting the die model to the package design. *Default* : `false`
`-grid_reduction_cut_layer` *layer_name*	
	Specifies a cut via layer below which all the layers will be reduced. This feature minimizes data and enables efficient die model generation.
`-ignore_port_grouping_conflict`	
	Specifies to not generate an error message in case there is a conflict between the user-defined port grouping and the default port groups (voltage sources that are grouped by mapping to the same package terminal). For example, vsrcA and vsrcB are both mapped to the package terminal PKG_T, but in the user-specified grouping file, vsrcA and vsrcB belong to two groups. When this parameter is specified, the tool will not generate an error message if there is a conflict, but instead will group the voltage sources according to the user-specified port grouping file. If this parameter is not specified and there is a conflict in port grouping, an error will be generated.
`-local_reference_ground {true | false}`	

	Specifies to assign a ground source as reference ground locally for sources in the current file `.isrc`. You can specify the reference node in the PLOC file as "`RefGnd=PortName`" to compute the isrc of port by considering reference ground locally during die-model generation.
	Default: `false`
	Example:
	`write_die_model \`
	`**-local_reference_ground true** \`
	`-port_grouping_by_ploc_files true \`
	`-generate_SIPI_port_definition_file true \`
	`-disable_isrc_node_zero true \ (make isrc with ref ground)`
	`-disable_global_node_zero false (keep global node zero in ckt)`

`-max_ports_per_net` *integer*	
	Specifies the maximum number of ports allowed for each net in a generated die model. If you do not specify this parameter, the software by default enables port grouping with number of ports set to `200`. If the number specified through this parameter is `0`, port grouping will be disabled. The maximum number of ports per net should be `0` or a positive value. A negative value will be treated as 0. **Note:** The `-max_ports_per_net` parameter is only meaningful for N-port die model creation. The 2-port die model creation always groups all ports together.
`-multi_die {true | false}`	
	Specifies to generate a complete die model for a TSV/SiP design. *Default*: `false`
`-output_dir` *directory_name*	
	Specifies the directory in which you save the die-model files.
`-port_grouping_by_ploc_files {true | false}`	

	Specifies to read the PLOC group information from the user-defined PLOC files. You can specify the port information in the PLOC file, as shown below: `Vsource_5_VDD 21985.00 23785.00 AP VDD p18` # with net and group `Vsource_7_VDD 22385.00 23785.00 AP VDD p18 C=1e-09 L=1e-11` # with capacitance and inductance When `-port_grouping_by_ploc_files` is set to `true`, the die model flow will read the user-specified port grouping information and generate a reduced die model. This parameter is mutually exclusive with the `-port_grouping_file` parameter.
`-port_grouping_file` *filename*	

	Specifies a text file containing the user-defined pin grouping details on a per power/ground net basis. When this parameter is specified, a reduced die model is generated that honors the user-specified port grouping. This die model has a reduced RC netlist and `.isrc` when compared with the original un-grouped die model. The degree of reduction will depend on how many pins are being grouped.
	The format of the pin grouping file is as follows:
	`NET <netname>`
	`<grouped pin name> <vsrc names which have to be grouped>`
	`<grouped pin name> <vsrc names which have to be grouped>`
	`NET <netname>`
	`<grouped pin name> <vsrc names to be grouped>`
	`<grouped pin name> <vsrc names to be grouped>`
	`END`
	where,
	• `NET` is a keyword followed by the power or ground net name
	• *`grouped pin name`* is the user defined port name
	• *`vsrc names`* are the actual design voltage sources that need to be grouped

`-port_grouping_tiles_file` *filename*	

Specifies to group power ground ports by a user-defined tile array. All the ports in a specific tile are grouped together. The use model of the parameter is:

```
create_die_model -port_grouping_tiles_file tiles.txt
```

The following is the format of the port grouping tiles format is:

```
NET <net>
BBOX <x1 y1 x2 y2>
ROWS <num rows>
COLUMNS <num columns>
```

Here,

- the keywords are highlighted in blue.
- *<net>* is the power supply net name
- *<x1 y1 x2 y2>* are the coordinates of the bounding box lower and upper corners. This is optional. If the bounding box is not specified, the default is the chip bounding box.
- *<num rows>* is the number of rows in the tile array
- *<num columns>* is the number of columns in the tile array

This parameter allows you to partition the voltage sources of the die model in to several port groups. If the same grouping pattern is used for the voltage sources of the die model and the package pins, it will help in matching the die ports and package terminals, and connecting the die model to the package netlist.

The grouping information is available in the MCP header of the generated die model file, as shown below:

```
* [REM]
* [Power Nets]
* vsrc_821 VDD2_GROUP_BUMP_tile01 VDD2 6316.25 20
* vsrc_820 VDD2_GROUP_BUMP_tile01 VDD2 623.75 20
* vsrc_822 VDD2_GROUP_BUMP_tile00 VDD2 5000 40
* vsrc_823 VDD2_GROUP_BUMP_tile00 VDD2 5002 40
……

*[Ground Net]
* vsrc_100 VSS_GROUP_BUMP_tile01 VSS 6300 25
* vsrc_101 VSS_GROUP_BUMP_tile01 VSS 6250 25
* vsrc_102 VSS_GROUP_BUMP_tile00 VSS 4000 40
* vsrc_103 VSS_GROUP_BUMP_tile00 VSS 4002 40
```

Here

	• the first column is the voltage source name. • the second column is the port name under which the voltage sources are grouped. It also displays the tile number to check a specific tile's power and group port during simulation. • the third column is the net name. • the fourth and fifth columns indicate the X and Y coordinates, respectively. The unit of x/y coordinate location is micrometer (µm).
`-probing_node_file` *`file_name`*	

	Specifies the probe node file containing the probe point on a die. The file format for specifying probing nodes is as follows:
	`<user defined unique probe name> <location = x> <location = y> <layer> <Power/ground net name>`
	You can also specify the package node voltages and resistor currents that are to be probed in the following format:
	`<probe name> pkg ir <pkg circuit node name>`
	`<probe name> pkg rc <pkg circuit resistor name>`
	Example
	`probe_1 18 16 METAL_1 VDD`
	`# pkg node voltage`
	`probe_pkg_1 pkg ir GND.in0`
	`probe_pkg_6 pkg ir VDD.in2`
	`# pkg resistor current`
	`probe_pkg_cur_1 pkg rc R_1_1`
	`probe_pkg_cur_2 pkg rc R_5_2`
	Note: The units of x and y should be in microns.

`-probing_region_file` *`filename`*	
	Specifies a probe node region file to define a probe region and use all points in the region as probe contact points on the die. The file format for specifying a probing region is as follows: *`region_label x1 y1 x2 y2 layer_name net_name`*

-rail_analysis_domain *domain_name* | -net *net_name*	
	Specifies the domain name or net name for which the die-model is to be generated. When the domain name is specified, coupled die-model is generated and it includes all power and ground nets associated with the specified domain. When net name is specified, decoupled die-model is generated for the specified net. Before specifying this command parameter, the power net name must be specified using the `set_pg_nets` command. This is a required option and one of them must be specified.
`-repeat_time` *time*	Specifies the repeat time in ns. When this parameter is specified without the `-use_sigrity_repeat` parameter, the generated die model has the dynamic current profile of the die repeated to the specified time. By default, the die model consists of the default simulation period saved in dynamic current files during IR drop anlaysis. When this parameter is specified along with the `-use_sigrity_repeat` parameter, the software forces to write out repeating waveforms in the Sigrity specific format. If the `-use_sigrity_repeat` parameter is specified, the value specified through the `-repeat_time` parameter indicates the starting point of the repeating waveform.

`-report_effective_rc`	Generates an additional report file (.txt) containing the effective resistance and capacitance information. The parameter is used for generation of a table for effective resistance and capacitance across multiple operating frequencies.
	The following is a sample report format:
	`# Net:VDD_AO, Net:VSS`
	`# frequency  # rdie  # cdie`
	`1.0000000e+07  4.73148443  2.47034747e-11`
	`1.2589254e+07  4.90209173  2.46916698e-11`
	`....`
	`....`
	`# Net:VDD_external, Net:VSS`
	`# frequency  # rdie  # cdie`
	`1.0000000e+07  -183.231085  6.65466673e-12`
	`1.2589254e+07  -164.007129  6.55112569e-12`
	`....`
	`....`

`-state_directory` *`directory_name`*	
	Specifies the top-level state directory that is generated by the static or dynamic IR drop analysis. This state directory contains analysis of all nets associated with the power domain; For example, `core_25C_dynamic_1`. This is a required parameter.
`-suppress_negative_rc {true | false}`	
	Specifies to suppress the negative resistor and capacitor (RC) values during die-model generation. When this parameter is set to `true`, the generated netlist contains positive RC values with voltage-controlled (VC), current-controlled(CC), and function-controlled (FC) sources to ensure accuracy and stability in the die models.
`-tmp_directory_name` *`directory`*	
	Specifies a directory to save temporary files for die-model generation.

`-use_sigrity_repeat`	When this parameter is specified along with the `-repeat_time` parameter of the `write_die_model` command, the software forces to write out repeating waveforms in the Sigrity specific format. If the `-use_sigrity_repeat` parameter is specified, the value specified through the `-repeat_time` parameter indicates the starting point of the repeating waveform. If `-use_sigrity_repeat` is specified without specifying `-repeat_time`, time `0` is used for the starting point. The default unit for the value is "ns". **Note:** When the `-use_sigrity_repeat` parameter is specified, the software will write the stimulus current in the specific Sigrity format, which cannot be used by other Spice like simulators.
`-write_effective_rc_deck`	

	Specifies to include the Rdie/Cdie (effective resistance/capacitance) calculation for the die-model frequency domain simulation setup in the Spice (.sp) format.
	The following is a snippet of the .sp file with the frequency-domain setup for Rdie/Cdie calculation:
	`**********`
	`* Frequency domain analysis to capture input impedance.`
	`.AC DEC 10 10Meg 5G`
	`.PRINT AC RDIE=PAR('VR(VSS_in,VDD_AO_in)')`
	`CDIE=PAR('1/(6.283185*freq*(VI(VDD_AO_in)-VI(VSS_in)))')`
	`**********`

Examples

- The following command creates an `n-port` die model having the dynamic current profile of the die repeated upto `50ns`, using the state directory `allDomains_25C_dynamic_1` and saves the generated n-port die-model file in the `n_port_diemodel` output directory:

```
write_die_model
-state_directory irdrop_allOn/allDomains_25C_dynamic_1
-rail_analysis_domain allDomains -output_dir n_port_diemodel -repeat_time 50
```

- The following example shows the use model of the `-report_effective_rc` parameter to create a report file containing the effective resistance and capacitance information:

```
write_die_model
  -report_effective_rc -state_directory staticRailResults/ALL_125C_avg_1/
  -output_dir static_die_model -rail_analysis_domain ALL
```

Related Topics

- "Package Analysis" in the *Voltus User Guide*

write_hier_view

```
write_hier_view
[-help]
[-block_lef lef_file | -block_stream gds_file]
[-block_lef lef_file | -stream_port_file gds_port_file]
[-cell_name name]
[-cluster_via1_ports {true | false}]
[-cluster_via_rule { {via_layer1 number_of_equidistant_vias}...}]
[-cluster_via_size value]
[-compress_power_grid_db {true | false}]
[-def list_of_DEF_files]
[-extraction_mode {fast | accurate}]
[-extraction_work_directory path_to_extraction_work_directory]
[-force_library_merge {true | false}]
[-ground_pins {pin1 pin2 ...pinN}]
[-ignore_decaps {true | false}]
[-ignore_fillers {true | false}]
[-ignore_shorts {true | false}]
[-libgen_command_file file]
-library_name name
[-max_via_cluster_mode {true | false}]
[-oa_reference_libs OA_reference_libs]
[-output_dir dir]
[-parasitic_extractor_command_file filename]
[-power_grid_view_libraries primitive_cells.cl]
[-power_pins {pin1 value1 pin2 value2 ...pinN valueN}]
```

Creates a hierarchical power-grid view. The hierarchical PGV generation of a block is part of the top-level Hierarchical Rail Analysis.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_hier_view` parameter. For a detailed description of the command and all of its parameters, use the man command: man `write_hier_view`
`-block_lef` *`lef_file`*	
	Specifies the name of the block LEF file. The *`lef_file`* must have all power pin port geometries defined for the hierarchical block.
`-block_stream` *`gds_file`*	
	In absence of block LEF, you can specify block GDS and port label file to generate hierarchical power-grid view.
`-cell_name` *`name`*	Specifies the block cell name as specified in the block DEF file.
`-cluster_via1_ports {true | false}`	
	Designs with follow-pin routing on M1 and M2 layers have standard cells library LEF with M1, M2, and VIA1 ports. This parameter specifies to cluster VIA1 ports for such cells in order to improve overall extraction and rail analysis performance. *Default*: `true` This parameter is set to `true` by default to improve VIA1 port handling in designs with M1 and M2 follow-pin routing. This parameter gives significantly better performance by clustering equidistant VIA1 ports defined in cell LEF without significant loss of accuracy. However, if you specify a clustering rule, the user specification takes precedence over automated VIA1 clustering.
`-cluster_via_rule { {`*`via_layer1 number_of_equidistant_vias`*`}...}`	
	Controls the number of vias to cluster on a layer basis. The VIA clustering rule specified using this parameter will override the default clustering rule for a given accuracy mode. The following command will cluster 100 equidistant VIA1 cuts, 200 equidistant VIA2 cuts, and 300 equidistant VIA7 cuts: `write_hier_view -cluster_via_rule { {VIA1 100}} {VIA2 200} {VIA7 300}}` The rest of the VIAs will be clustered using the default clustering rule depending upon the rail analysis accuracy mode.
`-cluster_via_size` *`value`*	

	Specifies the size of the via array to cluster all via layers except VIA1 (layer between Metal1 and Metal2). The default value is `625` (25x25 via array), except when running static analysis using HD mode. The default for static HD mode is `16` (4x4), primarily to preserve more accuracy for EM analysis. *Default*: `625`
`-compress_power_grid_db {true | false}`	
	Specifies to compress the output files generated by extraction and rail analysis to save disk space usage. This parameter reduces disk space requirement from the extraction and rail analysis stages at approximately 5-10% performance penalty. *Default*: `false`
`-def` *`list_of_DEF_files`*	
	Specifies a list of block DEF files for hierarchical analysis.
`-extraction_mode {fast | accurate}`	
	Determines what power-grid views will be created, dependent on whether fast or accurate extraction is to be done. By default, hierarchical PGV generates both detailed and reduced views.
`-extraction_work_directory` *`path_to_extraction_work_directory`*	
	Specifies to re-use the extracted power-grid database to generate HPGV. This parameter is only supported to generate hierarchical extraction view of a block in the design. The specified extraction work directory must contain the extracted data for all the nets for which the hierarchical extraction view needs to be generated. This parameter is mutually exclusive with the `-def` parameter of the `write_hier_view` command.
`-force_library_merge {true | false}`	
	Specifies to force merge PGVs with different resistivity and layers. It uses the technology information of the first library. If multiple tech-only PGVs are specified, it will take the first tech-only PGV definition and the first cell definition. If you are merging a tech-only PGV with other PGVs, it will keep the tech-only technology definition and the first cell definition.
`-ground_pins {`*`pin1pin2...pinN`*`}`	

	Specifies a list of ground nets of the block.
`-ignore_decaps {true | false}`	
	Ignores decap cells during rail analysis. It ignores cells that are tagged as DECAP cells during library characterization or set as decap cells using the `set_rail_analysis_mode -decap_cell_list` command parameter. If decaps are used to preserve connectivity on the follow-pin routing, you must set this parameter to `false`. Ignoring decaps during dynamic analysis is not recommended. *Default*: `true` for static, `false` for dynamic
`-ignore_fillers {true | false}`	
	Ignores filler cells during rail analysis if the list of filler cells are defined in PGV generation, or specified as fillers using the `set_rail_analysis_mode -filler_cell_list` command parameter. During rail analysis, ignore library cells that are tagged as FILLER cells and have no capacitance associated with the interface nodes. If fillers are used to preserve connectivity on the follow-pin routing, you must set this parameter to `false`. *Default*: `true`
`-ignore_shorts {true | false}`	
	Ignores shorts that are found during extraction and continues hierarchical PGV generation. However, `-ignore_shorts` cannot be applied if the design contains GDS. *Default*: `false`
`-libgen_command_file` *file*	
	Specifies the file name of a power-grid library generation (libgen) command file. When specified, it will be included in the auto-generated `libgen.cmd` file at the end as `include <file>`. This parameter allows you to provide a file that includes specific library generation options.
`-library_name` *name*	
	Specifies the name of the library that the block power-grid view will be placed in. A `.cl` extension will automatically be added to the name.
`-max_via_cluster_mode {true | false}`	

	Specifies the extractor to cluster vias automatically to reduce the RC elements by clustering vias into one node, therefore reducing the IR drop analysis runtime in dynamic mode. It is based on the notion that current evenly distributes through all vias of an array via. *Default*: `true`
`-oa_reference_libs` *OA_Reference_libs*	
	Reads OpenAccess reference libraries.
`-output_dir` *dir*	Specifies the output directory that the results will be saved in.
`-parasitic_extractor_command_file` *filename*	
	Specifies the name of the file input to the extractor invoked by the Library Generation engine. This file contains information about the cells in the design and is used for resistance and capacitance extraction. For variables and functions that can be placed in this command file, see *Extractor Options* in *Voltus Text Command Reference*. In general, this parameter is used for debugging purposes.
`-power_grid_view_libraries` *primitive_cells.cl*	
	Specifies a list of primitive cell libraries for hierarchical PGV generation.
`-power_pins {`*pin1value1pin2value2 ...pinNvalueN*`}`	
	Specifies a list of power nets and the nominal voltage of those nets.
`-stream_port_file` *gds_port_file*	
	In absence of block LEF, you can specify block GDS and port label file to generate hierarchical power-grid view.

Examples

- The following command generates a hierarchical power-grid view for the hierarchical block using LEF and extraction directories for VDD and VSS nets:

```
write_hier_view -block_lef blockA.lef -extraction_work_directory work -library_name
blockA  -power_grid_view_libraries ./pgv/stdcells.cl  -power_nets {vdd 1.2} -ground_nets
{gnd} -cell_name blockA
```

  The design.lef must have all power pin port geometries defined for the hierarchical block. The generated hierarchical PGV library is blockA.cl. The library is in binary format and it stores the block's connectivity to the top-level based on the provided LEF data. It stores the extracted power-grid network and current distribution from the provided directories.

- The following command specifies to force merge PGVs with different resistivity and layers:

```
write_hier_view  \
  -force_library_merge true \
  -library_name lib1  \
  -view hier_extraction_view \
  -power_grid_view_libraries {../accurate_stdcells_A.cl ../accurate_stdcells_A.cl ...
../accurate_stdcells_AN.cl } \
  -def {../design.def.gz} \
  -cell_name  CELLA \
  -power_pins {VDD 1.32} \
  -ground_pins {VSS} \
  -block_lef {./design.lef}  \
  -extraction_mode accurate \
write_pg_library -output pgv
```

Related Topics

- "Hierarchical Rail Analysis" in the *Voltus User Guide*

write_power_pads

```
write_power_pads
[[-add | -delete] -location {x y}]
[-append_to_voltage_source_file]
[-auto_fetch]
[-cell_list {cell_list} ]
[-cell_pin {{cellname1 pinname1}...{cellnamen pinnamen}+} ]
[-clear]
[-display]
[-format {padcell xy tsv}]
[-honor_pin_connection]
[-instance_list {inst_list} ]
[-instance_pin {{instname1 pinname1}...{instnamen pinnamen}+} ]
[-layer {layername}]
[-list]
[-net netname]
[-package_capacitance value ]
[-package_inductance value ]
[-package_resistance value ]
[-region {x1 y1 x2 y2}]
[-region_pitch {xpitch ypitch}]
[-snap_distance { true|false }]
[-voltage_source_file vsrc_filename]
```

Returns a list of DC sources from your design. DC sources are identified in the LEF file as CLASS PAD POWER, or in the DEF file as PIN with +USE POWER/GROUND. If a single port shape is connected to N number of wires, N number of DC sources are reported for that port shape.

You can use one of the following three methods to fetch voltage sources into the database:

- fetch all voltage sources
- fetch the voltage sources within the specified region of the specified layer controlled by the specified xPitch/yPitch
  If the voltage sources with the specified xPitch/yPitch are not on the stripe, you can use the `-snap_distance` parameter to snap the voltage sources to the stripe. As a result, the voltage sources are generated only on the stripes and the xPitch/yPitch is honored. Use this feature to generate mesh type voltage sources on any process layer.
- fetch voltage source for the specified cell/cell pin/instance/instance pin

You can edit the fetched voltage sources before saving them to a voltage source file. To save the fetched

list to a file, use the `-voltage_source_file` parameter. By default, voltage sources will not be displayed on GUI after they are saved to file.

> The `write_power_pads` command supports hierarchical IR drop analysis for a TSV design. When you specify the `write_power_pads` command to fetch the voltage sources from the LEF file, the command not only searches for the P/G pin of the pad connected to the specified P/G net, but also fetches for the embedded bump TSV property definition for the P/G pin connected to the P/G net. The port that contains the point defined in the embedded bump property is considered as the embedded bump.

Parameters

`-add`	Adds the location of a new voltage source to memory.
`-append_to_voltage_source_file`	Appends the fetched voltage sources to the existing voltage source file. You can use this parameter to fetch voltage sources in the incremental mode.
`-auto_fetch`	Specifies to fetch all voltage sources.
`-cell_list {`*`cell_list`*`}`	
	Specifies the cells from which voltage sources are determined.
`-cell_pin {{`*`cellname1pinname1`*`}...{`*`cellnamenpinnamen`*`}+}`	
	Specifies the cells and their pins from which voltage sources are determined.
`-clear`	Clears the display of the DC sources that were specified in the *Edit Pad Location* form.
`-delete`	Deletes the location of a voltage source from memory.
`-display`	Loads and displays the pad location data from the specified file.
`-format {padcell | xy | tsv}`	

	The format of the saved pad location file. The choices are Pad File (VoltageStorm), XY File (xy coordinates), and TSV. You can create and save the voltage sources location file with a unique name for the voltage sources (both VDD and VSS). *Default*: `xy`
`-help`	Outputs the command usage and a brief description about the command parameters.
`-honor_pin_connection`	
	Honors pin wire connection when fetching voltage source location from a power pad. When you specify this parameter, only pins with wire connection are output as voltage sources. You have the choice to specify whether real wire connection is to be honored or not. Typically, during the sign-off stage, power pads are already fully routed so the unconnected power pads should not be counted in as voltage sources. You can specify this using the -honor_pin_connection parameter. Alternatively, in the early stage, power pads may not have been fully routed so you do not need to set this parameter. **Note:** For a port with shapes at multiple layers, all layer connected shapes are fetched as voltage sources.
`-instance_list {`*`inst_list`*`}`	
	Specifies the instances from which voltage sources are determined.
`-instance_pin {{`*`instname1pinname1`*`}...{`*`instnamenpinnamen`*`}+}`	
	Specifies the instances and their pins from which voltage sources are determined.
`-layer` *`layerName`*	

	Specifies the process layer in LEF on which the voltage sources are to be generated. When the `-layer` parameter is specified in conjunction with the `-region {x1 y1 x2 y2}` and `-region_pitch {xpitch ypitch}` parameters, the layer set here is the layer for which voltage sources are to be fetched. If the `-layer` parameter is not used together with the `-region {x1 y1 x2 y2}` and `-region_pitch {xpitch ypitch}` parameters, the layer set here is for the lowest layer to snap (the default value is `M5`).
`-list`	Loads and displays all voltage sources from memory.
`-location {x y}`	Specifies the location at which the specified voltage source is to be added or deleted.
`-net` *`netname`*	Specifies the name of a power net for which to list DC sources. *Default:* If you do not specify a net name, the power analyzer automatically lists DC sources for all nets.
`-package_capacitance` *`value`*	
	Specifies the capacitance value of the package resistance-inductance-capacitance (RLC) model in Farad (F).
`-package_inductance` *`value`*	
	Specifies the inductance value of the package resistance-inductance-capacitance (RLC) model in Henry (H).
`-package_resistance` *`value`*	
	Specifies the resistance value of the package resistance-inductance-capacitance (RLC) model in Ohm.
`-region {`*`x1 y1 x2 y2`*`}`	
	Generates the voltage sources in the specified region.
`-region_pitch {`*`xpitch ypitch`*`}`	
	Specifies the region pitch in the x and y direction for the voltage sources.
`-snap_distance` *`value`*	

	Specifies the snapping distance constraint from IO/pad pins. If this parameter is not specified with a value, no snapping will be done. *Default:* 0 Specifies whether to snap DC sources from I/O pad pins to some location on the power structure. When set to true, the software performs autofetch with snapping. For I/O pad pins that are connected to core ring, the software selects the highest layer and center of the pin as the DC source location. When I/O pad pins are not connected to core ring, the software snaps the location of the I/O pad to somewhere on the core ring. The DC source location should be within the distance of snapping distance constraint from I/O pad pins, and the layer should not be lower than the lowest layer snap constraint.
`-voltage_source_file` *`vsrc_filename`*	
	Specifies the name of the voltage source file that is required to save the auto fetched result.

Examples

- The following command fetches all voltage sources of the net `VDD`:

  ```
  write_power_pads -net VDD -auto_fetch
  ```

- The following command loads and displays all voltage sources from memory:

  ```
  write_power_pads -list
  ```

- The following command specifies the location at which the specified voltage source is to be added:

  ```
  write_power_pads -add -net VDD -layer METTOP -location {500 500}
  ```

- The following command specifies the location from which the specified voltage source is to be deleted:

  ```
  write_power_pads -delete -location {500 500} -layer METTOP -net VDD
  ```

- The following command fetches the voltage sources within the specified region of the layer `METAL6` controlled by the specified xpitch/ypitch, and saves the sources to the file `report2` in the xy format:

  ```
  write_power_pads -net VDD -region 100 100 3000 3000 -region_pitch 555 444 -layer METAL6 -format xy -snap_distance 100 -voltage_source_file report2.pp
  ```

  > ⚠ In the first example, the output was not saved but it remains in the memory till you specify the second example. Before loading and saving the output of the second example, the first example output in the memory is saved to an auto generated dc source file incrementally, like pp.src, pp1.src, pp2.src, and so on.

- The following command fetches voltage source for the cell BUMPCELL in the xy format:

  ```
  write_power_pads -net VDD -cell_list BUMPCELL -format xy
  ```

- The following command clears the voltage sources displayed in the GUI and the fetched voltage sources in the database:

  ```
  write_power_pads -clear
  ```

- The following command displays the fetched and saved voltage sources for the net `VDD` in the file `report2` :

  ```
  write_power_pads -net VDD -auto_fetch -voltage_source_file report2.pp -display
  ```

- The following command displays voltage sources in the GUI by loading the saved voltage source file `report1`:

  ```
  write_power_pads -voltage_source_file report1.pp -display
  ```

- The following command saves the fetched voltage sources to the voltage source file `report1.pp`:

  ```
  write_power_pads -voltage_source_file report1.pp
  ```

write_power_up_report

```
write_power_up_report
[-help]
-net net_name
-output_directory directory_name
-state_directory directory_name
```

Specifies to generate power-up statistics report to view the ramp up/inrush current, worst power switch voltage, and additional instance voltage data (average, median, max, and standard deviation voltage data versus time) for the powering-up rail. The following files are generated in the GIF and text formats:

- average voltage of instances (.avgV)
- max voltage of instances (.maxV)
- min voltage of instances (.minV)
- median of instance voltages (.medianV)
- standard deviation of instance voltages (.deltaV)
- total inrush current (.inrushI)
- average voltage of power gates (.avgV)
- max voltage of power gates (.maxV)
- min voltage of power gates (.minV)
- median of power gate voltages (.medianV)
- standard deviation of power gate voltages (.deltaV)

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_power_up_report` parameter. For a detailed description of the command and all of its parameters, use the man command: man `write_power_up_report`

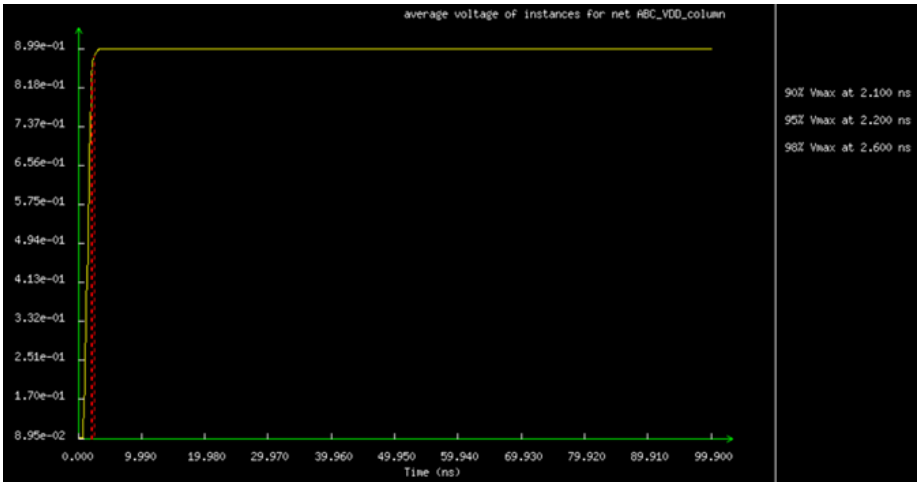
`-net` *net_name*	
	Specifies the name of the powering up rail for which you want to generate detailed power-up statistics.
`-output_directory` *directory_name*	
	Specifies the output directory in which the power-up statistics results will be saved in.
`-state_directory` *directory_name*	
	Specifies the top-level state directory that is generated by IR drop analysis. This state directory contains analysis of all nets associated with the power domain; for example, core_25C_dynamic_1.

Examples

- The following command generates power-up statistics for the `ABC_VDD_column` net:
  `write_power_up_report -net ABC_VDD_column -state_directory core_25C_dynamic_1 -output_directory rpt`

  The following is a snippet of the text and GIF formats with information on average voltage of instances:

Text Format (`ABC_VDD_column.avgV`)	**GIF Format (`ABC_VDD_column.avgV`)**

`#List average voltage of instances for net ABC_VDD_column`
`# 90% of nominal voltage at 2.1 ns`
`# 95% of nominal voltage at 2.2 ns`
`# 98% of nominal voltage at 2.6 ns`
 	
`#XLABEL# Time (ns)`
`#YLABEL# Voltage (V)`
`0 0.0894674`
`0.1 0.0894675`
`0.2 0.0894676`
`...`
	

18

RC Extraction Commands and Attributes

- create_parasitic_coordinate_transform
- extract_parasitics
- extract_rc Category Attributes
- read_parasitics
- read_spef
- report_net_parasitics
- report_rcdb
- report_unit_parasitics
- write_parasitics
- write_rcdb

create_parasitic_coordinate_transform

```
create_parasitic_coordinate_transform
[-help]
{-inst inst_name
[-orient {r0 r90 r180 r270 my mx90 mx my90}]
[-x_block_origin value]
[-x_offset value]
[-y_block_origin value]
[-y_offset value]
[-relative_to_parent]} | {-reset}
```

Determines accurate location information of nodes during SPEF reading.

In case of hierarchical designs, there can be multiple blocks in a design that are implemented separately. During Top level STA analysis, SPEFs of these blocks are stitched together with top for flat STA analysis. This works well if pin location of blocks is not required. In case of AOCV analysis, when spatial analysis is enabled, accurate location of all nodes (pin/net) in the design is required

for calculating accurate derates in the design. This makes it necessary to determine accurate nodes location during SPEF reading. This can be achieved by transforming SPEF coordinates as per their physical locations.

Note:

- This command works with all possible SPEF and RCDB input combinations with `read_parasitics` command, which means for only RCDB(s) as input, for a mix of SPEF(s) and RCDB(s) as input, and for only SPEF(s) as input.
- If this command is not specified, the `read_spef` command tries to automatically find the orientation and origin of the hierarchical SPEF blocks and applies those identified transforms to the parasitic co-ordinates.
- The `create_parasitic_coordinate_transform` command settings remain in effect for all the subsequent `read_spef` and `read_parasitics` until they are reset using the `-reset` option.
- Whenever the block's origin (in block SPEF/DEF) is non-zero, the x and y coordinates need to be specified for applying the transform. The last two parameters, `-x_block_origin` and `-y_block_origin` in the following command use-model are only required to be specified for the case of non-zero block origin.

  ```
  create_parasitic_coordinate_transform -inst inst_name
  -x_offset x-offset of block origin in top-level DEF/SPEF
  -y_offset y-offset of block origin in top-level DEF/SPEF
  -orient orientation
  -x_block_origin x-coord of non-zero block origin
  -y_block_origin y-coord of non-zero block origin
  ```

  For example:

  - **For Non-Zero Origin Block**: Specify the block origin as shown below:

    ```
    create_parasitic_coordinate_transform -inst eco_inst1
    -x_offset 421.6
    -y_offset 485
    -orient R180
    -x_block_origin 3.4
    -y_block_origin 9.7
    ```

  - **For zero origin block**: No need to specify the block origin as shown below:

    ```
    create_parasitic_coordinate_transform
    -inst controller
    ```

```
-x_offset 16.8
-y_offset 205
-orient R0
```

or

```
create_parasitic_coordinate_transform
-inst controller
-x_offset 16.8
-y_offset 205
-orient R0
-x_block_origin 0
-y_block_origin 0
```

Both the above commands are equivalent.

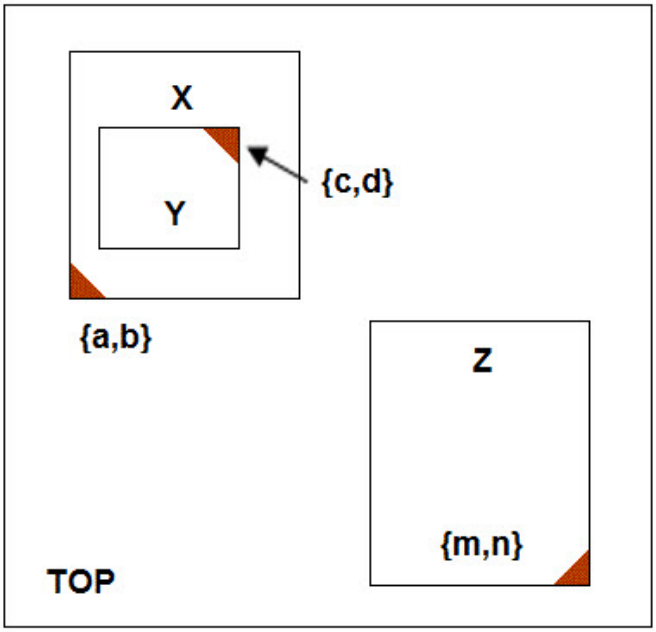
- The impact of this command is valid till first `read_spef` command is read (in case of MMMC, till all corners SPEFs have been read). If you want to read SPEF again, this command needs to be applied once again. These transformations will be applied during `read_spef` command.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man create_parasitic_coordinate_transform`.
`-inst` *`inst_name`*	Indicates the hierarchical name of the instance for which SPEF is being read. *Data_type*: string, required
`-orient {R0 R90 R180 R270 MY MX90 MX MY90}`	Indicates orientation change of instances. The valid values are `R0`, `R90`, `R180`, `R270`, `MX`,`MX90`, `MY`, and `MY90`. *Data_type*: enum, required
`-relative_to_parent`	When this parameter is specified, the software considers block offsets for the specified instance relative to the immediate parent. By default, the software considers the offset value relative to the top hinst. *Data_type*: bool, required
`-reset`	Resets SPEF transform values. *Data_type*: bool, required
`-x_block_origin` *`value`*	Specifies that in case of non-origin block, offset x-coordinate in microns. *Data_type*: float, optional
`-x_offset` *`value`*	Indicates the actual x-coordinate as exists in the physical floor plan, corresponding to the specified instance. *Data_type*: float, required
`-y_block_origin` *`value`*	Specifies that in case of non-origin block, offset y-coordinate in microns *Data_type*: float, optional
`-y_offset` *`value`*	Indicates the actual y-coordinate as exists in the physical floor plan, corresponding to specified instance. *Data_type*: float, required

Examples

Consider the following hierarchical design where X and Z are two blocks inside TOP, and Y is another hierarchical block in X:



Here, X,Y, and Z have the following physical locations in the floor plan:

```
X - (a,b)
Y- (c,d)
Z- (m,n)
```

You can apply transformations to all blocks with respect to TOP, by using the following commands:

```
create_parasitic_coordinate_transform -inst X -x_offset a -y_offset b

create_parasitic_coordinate_transform -inst Z -x_offset m -y_offset n -orient MY

create_parasitic_coordinate_transform -inst X/Y -x_offset c -y_offset d -orient R180
```

These transformations will be applied when `read_spef` command is issued.

Related Commands

- `redirect`
- `read_parasitics`
- `read_spef`

extract_parasitics

```
extract_parasitics
[-help]
```

Extracts resistance and capacitance for the interconnects and stores the results in an RC database. Optionally, an ASCII report file can be generated. Use this command after performing trial routing or final routing, and after setting the extract_rc attributes.

Parameter

`-help`	Outputs a brief description of the `extract_parasitics` command. For a detailed description of the command, use the man command: `man extract_parasitics`.

Example

The following command extracts interconnect resistance and capacitance:

```
extract_parasitics
```

RC database is created.

The `extract_parasitics` command can be followed by the `write_parasitics` command to extract the parasitics from the RC database, and to generate an ASCII report containing the database information. You can find all the extraction reports in the `extLogDir`.

extract_rc Category Attributes

The root attributes in the `extract_rc` category are:

- extract_rc_cap_filter_mode
- extract_rc_compress_rcdb
- extract_rc_coupled
- extract_rc_coupling_cap_threshold
- extract_rc_lef_tech_file_map
- extract_rc_local_cpu
- extract_rc_pvs_fill
- extract_rc_qrc_cmd_file
- extract_rc_qrc_cmd_type
- extract_rc_qrc_stream_map_file
- extract_rc_qrc_run_mode
- extract_rc_relative_cap_threshold
- extract_rc_shrink_factor
- extract_rc_signoff_stream_layer_map
- extract_rc_total_cap_threshold
- extract_rc_turbo_reduce

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `extract_rc` category, use the following command:

```
get_db -category extract_rc
```

extract_rc_cap_filter_mode

```
extract_rc_cap_filter_mode { relative_only | relative_and_coupling |
relative_or_coupling }
```

Default: For standalone Quantus extraction, if the process node is 130nm or below, the default value is `rel_and_coup`. If the process node is above 130nm, the default value is `rel_only`.

Specifies the capacitance filtering mode. The filtering modes are conditions which define how to use the `extract_rc_coupled` and `extract_rc_coupling_cap_threshold` attribute values, or their combination. Based on the filtering mode specified, the software determines the nets in the design, which will have their coupling capacitance lumped to the ground.

Note: The behavior of the extract_rc_total_cap_threshold attribute is independent of the `extract_rc_cap_filter_mode` attribute setting. Nets that have a total capacitance value lower than the value specified with the `extract_rc_total_cap_threshold` attribute will have their coupling capacitances grounded regardless of the `extract_rc_cap_filter_mode` attribute setting.

You can use one of the following modes:

- `relative_only`: Grounds the coupling capacitance of nets based on the threshold value set by the `extract_rc_relative_cap_threshold` attribute.
- `relative_and_coupling`: Grounds the coupling capacitance of nets only when it is lower than the threshold value specified with the `extract_rc_relative_cap_threshold` attribute as well as the `extract_rc_coupling_cap_threshold` attribute. In this mode, you enforce maximum restriction on grounding the coupling capacitances.
- `relative_or_coupling`: Grounds the coupling capacitance of nets if it is lower than the threshold value specified with one of the `extract_rc_relative_cap_threshold` or `extract_rc_coupling_cap_threshold` attributes.

extract_rc_compress_rcdb

```
extract_rc_compress_rcdb {0 | 1}
```

Default: 0

Generates compressed RCDB data.

Note: This functionality reduces the disk usage but leads to marginal increase in run time.

extract_rc_coupled

```
extract_rc_coupled {0 | 1}
```

Default: 1

Separately outputs grounded and coupling capacitance. The 0 value of this attribute is typically used for static timing analysis. Signal Integrity analysis requires the coupled mode.

extract_rc_coupling_cap_threshold

```
extract_rc_coupling_cap_threshold file_name
```

Default: 0.1fF

Specifies the threshold value that determines when the extractor lumps a net's coupling capacitance to ground. This attribute is applicable only when the `extract_rc_coupled` attribute is set to 1. The software decouples the coupling capacitance of nets when the total coupling capacitance between the pair of nets is lower than the threshold specified with this attribute. The actual decoupling also depends on the setting of the `extract_rc_cap_filter_mode` attribute.

extract_rc_lef_tech_file_map

```
extract_rc_lef_tech_file_map filename
```

Default: "" (empty string)

Specifies the location of the optional layer map file for the Standalone Quantus engine.

Note: If used, the layer map file must be in the Common Command Language (CCL) format. The following is an example of the layer map file format:

```
extraction_setup \
-technology_layer_map \
# <LEF/DEF Name> #<Tech File Name>
M1 METAL_1 \
VIA1 VIA_1 \
M2 METAL_2 \
```

Note: For cell-level Quantus layermap, the DEF/LEF layer names are case sensitive, but ICT layer names are case-insensitive.

extract_rc_local_cpu

```
extract_rc_local_cpu number_of_cpus
```

Default: As specified in the `set_multi_cpu_usage -local_cpu` setting.

Specifies the number of CPUs to be used for IQuantus extraction. This attribute is only applicable for the IQuantus extraction engine. When specified, it overrides the `set_multi_cpu_usage -local_cpu` option for IQuantus.

Related Command

- `extract_rc`

extract_rc_pvs_fill

```
extract_rc_pvs_fill {true | false}
```

Default: `false`

When this attribute is set to true, the PVS fill data attached to the Innovus database is sent to Quantus so that the effects of the fill data are extracted. This attribute only affects signoff Quantus extraction. It has no effect on tQuantus or iQuantus extraction done inside Stylus. This atttribute requires that either the `extract_rc_qrc_stream_map_file` attribute or the `extract_rc_signoff_stream_layer_map` attribute is specified.

extract_rc_qrc_cmd_file

```
extract_rc_qrc_cmd_file filename
```

Default: "" (empty string)

Specifies the user-created Quantus command file to be used for running Standalone Quantus. This command file should be in the new CCL syntax.

Note: This attribute is only valid if `extract_rc_qrc_cmd_type partial` or `custom` is selected.

extract_rc_qrc_cmd_type

```
extract_rc_qrc_cmd_type {auto | partial | custom}
```

Default: `auto`

Specifies whether to derive the settings for running Quantus from the software settings, or from user-created command file using the `extract_rc_qrc_cmd_file` attribute.

This attribute supports the following values:

- `auto`: Derives the Quantus settings automatically from the software parameter settings.
  Note: In this mode, the `extract_rc_lef_tech_file_map` attribute is required to be specified.
- `partial`: Appends user commands from the `qrcCmdFile` to the settings derived from the software parameter settings. You must provide the partial mode command file in CCL syntax. Use the partial command file only when you add extraction directives for those options that cannot be set in software. Following are some important restrictions:
  - Do not specify the following options in the partial mode command file:
    - Threshold setting
    - Coupled/decoupled mode
    - Quantus Techfile (s)
    - Operating Temperature
  - Do not specify "`-directory_name`" and "`-temporary_directory_name`" and "`-file_name`" in `output_setup` section of the command file
  - Do not specify output file using `output_setup -file_name` command in partial command file
    There is no checking for syntax or content of this file, and it will be appended as is to the automatically created Quantus command file.
- `custom`: Gets the complete Quantus settings from the user-created command file. Except for input and output, no settings from software parameters are used. You must provide the complete Quantus command file in CCL syntax.
  Following are some important restrictions:
  - Do not specify an input DEF/OA DB in the custom mode command file
  - Do not specify "`-directory_name`" and "`-temporary_directory_name`" in `output_setup` section of the command file.
  - Specify output SPEF with the following convention for the name:`<design.spef>`.
    In MMMC, the output SPEF files will be named: `<design>_<corner>_<Temperature>.spef`. Here "design" refers to the top cell name.

Note: Do not prefix or postfix the output file name.

- Specify the Quantus Techfile(s) to be used. In MMMC, the corner names should match the names used in the `create_rc_corner` command
- Provide Quantus Techfile layer mapping information in custom command file
- If GDS is provided for cell contents, provide a Quantus Techfile to GDS layer number matching

Note: There is no checking for syntax or content of the command file prior to running the extraction. The command file will be used as is. If the output file is not specified correctly, the resulting parasitic file will not be read into the Tempus RC database.

extract_rc_qrc_stream_map_file

`extract_rc_qrc_stream_map_file` *filename*

Default: `""` (empty string)

This attribute is specified only when `-pvs_fill` parameter is set to true. When the `extract_rc_pvs_fill` attribute is set to `true`, a Quantus stream/oasis map Tcl command is required to map the PVS layer/dtype data to the corresponding Quantus layers.

If the `pvs_fill.data` is generated based on (GDS), the map file should be in the following format:

```
#layer gds_layer gds_dtype
<layer_name> <layer_number> <data_type>
```

…

If the `pvs_fill.data` is generated based on OASIS, the map file should be in the following format:

```
#layer oasis_layer oasis_dtype
<layer_name> <layer_number> <data_type>
```

…

Where,

- `layer_name` is the layername in the design
- `layer_number` is the layer ID in the GDS/OASIS file

- `data_type` is the datatype specified in the GDS/OASIS file

For example,

```
M1  31  0

M1  31  1

M2  32  0

M2  32  1
```

extract_rc_qrc_run_mode

```
extract_rc_qrc_run_mode {concurrent | sequential}
```

Default : `concurrent`

Specifies the Quantus run mode in MMMC designs.

Note : For MMMC designs, in Quantus partial command type, do not specify the output SPEF file name and RC corner name in the command file to run Quantus in concurrent or sequential mode. For MMMC designs, in Quantus custom command type, do not specify the output SPEF file name and RC corner name together in the command file. Quantus sequential mode is not supported in custom command type.

This attribute supports the following values:

- `concurrent` : Runs Quantus for all active corners concurrently.
- `sequential` : Runs Quantus for all active corners sequentially.

extract_rc_relative_cap_threshold

```
extract_rc_relative_cap_threshold value
```

Default: `1.0`

Sets a ratio threshold value that determines when the extractor lumps a net's coupling capacitance to ground. This attribute is applicable only when the `extract_rc_coupled` attribute is set to 1. If the total coupling capacitance between a pair of nets is less than the percentage (specified with this attribute) of the total capacitance of the net with the smaller total capacitance in the pair, the coupling capacitance between these two nets will be considered for grounding. The actual decoupling also depends on the setting of the `extract_rc_cap_filter_mode` attribute.

The range of this value is 0 to 1.

extract_rc_shrink_factor

`extract_rc_shrink_factor` *`shrink_factor_value`*

Default: `1.0`

Sets the value by which the software shrinks the design before extraction. The shrink factor is used only for extraction calculations. It does not change the actual design. This attribute must only be used when restoring design.

Note: This attribute is not recommended for newer technologies where shrink factor is defined in the Quantus techfiles or capacitance tables.

extract_rc_signoff_stream_layer_map

`extract_rc_signoff_stream_layer_map` *file_name*

Default: "" (empty string)

This atrribute is specified only when the `extract_rc_pvs_fill` atrribute is set to `true`. When the `extract_rc_pvs_fill` atrribute is set to `true`, a Quantus stream/oasis map Tcl command is required to map the PVS layer/dtype data to the corresponding Quantus layers.

If the `pvs_fill.data` is generated based on stream (GDS/OASIS), the map file should be in the following format:

```
#layer gds_layer gds_dtype use-type color-ID
<layer_name> <layer_number> <data_type> <use-type> <color-ID>
…
```

Where,

- `layer_name` is the LEF layername in the design
- `layer_number` is the layer ID in the GDS/OASIS file
- `data_type` is the datatype of the layer in the GDS/OASIS file
- `use_type` is the keyword for identifying the use of the data. The valid values are:
  - `gray`: library data in the GDS/OASIS file
  - `fill`: regular metal fill data in the GDS/OASIS file
  - `active`: active metal fill data in the GDS/OASIS file
- `color-ID` is the color ID. Use “`1`” or “`2`” if the layer is colored, or use hyphen “`-`” if the layer is

not colored

For example,

```
M1  31  0 gray -
M1  31  1 fill -
M2  32  0 gray -
M2  32  1 fill -
```

extract_rc_total_cap_threshold

```
extract_rc_total_cap_threshold value_in_fF
```

Default: `0fF`

Specifies the threshold value (femtoFarads) that determines when the extractor lumps a net's coupling capacitance to ground. This attribute is applicable only when the `extract_rc_coupled` attribute is set to 1. The software grounds the coupling capacitances for the nets which have a total capacitance value less than the value specified with this attribute.

extract_rc_turbo_reduce

```
extract_rc_turbo_reduce {0 | 1 | auto}
```

Default: `auto`

Controls the turbo reduction feature for Standalone Quantus extraction. You can set the following options:

- `auto`: When set to `auto` , the software automatically turns `on` turbo reduction and proceeds when the required Quantus QRC license is available. However, if the required Quantus QRC license is not installed in the license server, the software automatically turns off Turbo reduction and proceeds with a warning message. If the license is installed in the license server but temporarily not available, then two scenarios are possible. First, if the license wait mode is not turned on, then Quantus will look for the license right away and error out because the license is not available. Second, if the license wait mode is turned on, then Quantus will wait and check for the license periodically until timeout. If the license is available before the timeout, turbo reduction will be performed but if the license is not available before timeout, Quantus will error out.

- 1: When set to `true` , the turbo reduction engine is `on` . However, if the required Quantus QRC license is not installed in the license server, then standalone Quantus will error out with a message and the run will stop. It is because setting the option to true means the user really wants to perform turbo reduction.
- `0`: The turbo reduction engine is switched `off` .

For details, see the table below.

Behavior of Turbo Reduction

	Availability of Quantus QRC AA License		
Option Selected – extract_rc_turbo_reduce {1 | 0 | auto}	Quantus QRC AA Installed and Available (IQRC gets the license right away even in the wait mode because sufficient licenses are available in the license server)	Quantus QRC AA Installed but not Available (If license wait mode is not turned on, then Quantus looks for the license . If the wait mode is turned on and if the license is temporarily not available, then IQRC waits and checks for the license periodically until timeout.)	Quantus QRC AA Not Installed on License Server
auto	perform turbo reduction	1. error out if wait mode is not on or if the specified wait time is exceeded 2. perform turbo reduction if wait mode is on and the license is available within the specified time.	print warning and do not perform turbo reduction
1	perform turbo reduction	error out	error out because the user explicitly wants to use turbo reduction
0	do not perform turbo reduction	do not perform turbo reduction	do not perform turbo reduction

read_parasitics

```
read_parasitics
[-help]
[-force]
[-keep_star_node_locations]
[-rc_corner_map rc_corner1:spef_field1 rc_corner2:spef_field2 ...]
[-rc_corner string]}}}]}
[-via_variation_file viaVariationFile]
```

Reads SPEF and RCDB-based RC parasitics information and already created RCDBs in the

software. For hierarchical designs, use this command to read top-level as well as block-level parasitic information.

Note: The following are important aspects to be considered for using this command:

- The RCDB parasitic file must have the `.rcdb.d` extension.
- The RCDB-based parasitic data that is provided as input to this command must be the one that was saved in the same major release of the software. The command will not read the RCDB data saved in a previous release. For example, if you provide RCDB saved in the 10.1.x release of the software as input to the `read_parasitic`s command in the 11 software release, then it cannot be read.
- Only the RCDB parasitic data on systems that are same as those used for saving the data can be read. For example, if the data is saved on Linux in 64-bit mode, you can only restore it on Linux 64-bit mode.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_parasitics` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_parasitics`.
`-force`	Allows reading of parasitic data even when block level RCDB/ SPEF is missing for a corner. **Note:** This parameter is only applicable for hierarchical designs. It is required that the top cell should have parasitic data specified for all the active RC corners.
`-keep_star_node_locations`	Specifies whether or not to read the *N statements in the SPEF file. The *N statements define the location of the RC nodes. When this parameter is specified, the *N statements are read and when it is not specified, the *N statements are ignored. *Default*: `true`
`-rc_corner_map` *rc_corner1:spef_field1 rc_corner2:spef_field2 ...*	
	Provides a mapping of the RC corners to the SPEF fields of multi-value SPEF files.

`-rc_corner` *list of RCcorner specific RCDBs/SPEFs for hierarchical design|RCcorner specific RCDB/SPEF*	
	Specifies the RC corner name along with a corner-specific RCDB or SPEF for both top and block levels. For hierarchical designs, this parameter lists the corner-specific SPEFs/RCDBs for both top and block levels. **Note:** A corner-specific RCDB parasitic file can only be provided as input to the RC corner for which it was generated. It cannot be provided as input to other RC corners.
`-via_variation_file` *viaVariationFile*	
	Specifies the name of the file for modeling via variation. The contents of the file include all the via layer names with the corresponding MEAN_SHIFT and STDDEV so that all the VIA resistance in SPEF can be mapped to their own variation multiplier. This option is used to specify the via_variation file only when the design has a single corner. For example, `read_parasitics` -rc_corner rcCornerName \ `-via_variation_file` *viaVariationFileName*

Examples

- The example below is for hierarchical design with Top (T), Block 1(B1) and Block 2(B2), and two active RC corners, C1 and C2.
    - The following command specifies the RC corner names along with corner-specific RCDB and SPEF for both top and block levels:

```
read_parasitics \
-rc_corner C1 Top_C1.spef Block1_C1.spef Block2_C1.spef.gz \
-rc_corner C2 Top_C2.rcdb.d Block1_C2.spef Block2_C2.rcdb.d
```

read_spef

```
read_spef
[-help]
file_name_list fileName
[-decoupled]
[-keep_star_node_locations]
[-extended]
[-rc_corner rcCornerName(s)]
[-scale_nets_file netListFile]
[-scale_rc]
[-skip_star_node_locations]
[-spef_field integer_list]
[-via_variation_file viaVariationFileName]
```

Loads resistors and capacitors for the interconnects in SPEF into the Voltus database to calculate delays or build a timing graph. Use this command to read in a SPEF file generated by a third-party extraction tool.

Note: You can use this command to load a SPEF file, which is compressed (`.gzip`) format.

Note: This command supports multi-threaded SPEF reading scalable upto a maximum of four CPUs. The number of CPUs can be specified using the standard use model of Voltus.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_spef` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_spef`.
`-decoupled`	Grounds the coupling capacitances in the SPEF file and merges the capacitances on the same node. This parameter can be used only for static timing analysis and cannot be used for SI analysis.

`-extended`	Enables the reading of "`$llx=<value1> $lly=<value2> $urx=<value3> $ury=<value4> $LAYER=<number>`" to set the layer number for nodes of a resistance from Quantus extended SPEF file. It also reads in extended `*LAYER_MAP section`, if it is present. This parameter can also read in "`$l=<length> $w=<width> $si_w=<swidth> $lvl=<number>`" to set the width, silicon width, or layer for nodes of a resistance from StarRC extended SPEF file.
`file_name_list fileName`	Specifies a SPEF file or list of SPEF files as input for flat chip or hierarchical SPEF stitching respectively.
`-keep_star_node_locations`	Enables the reading of the `*N` statements in the SPEF file. The `*N` statements define the location of the RC nodes.
`-rc_corner` *`rcCornerName(s)`*	Annotates the parasitics to the predefined RC corners for multi-corner analysis. You can specify a list of RC corners to be annotated. If you want to use SPEF information when calculating delays in multi-mode multi-corner analysis (MMMC) mode, you must annotate the parasitics for all RC corners in the design. A few examples are provided below to explain different scenarios of specifying rc corners are their corresponding corner indices and SPEF files. • `read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field {1 2 3 4 5} rc-spef.gz` This means that SPEF file can contain parasitics from N RC corners and names of RC corners in the RC corner list map to the index fields specified using the `-spef_field` option as shown above. Assuming that the SPEF file, `rc-spef.gz` has RC fields from the specified five RC corners, the above command would be interpreted as follows: 1st field in SPEF maps to RC corner, `rcCorner1` 2nd field in SPEF maps to RC corner, `rcCorner2` 3rd field in SPEF maps to RC corner, `rcCorner3` 4th field in SPEF maps to RC corner, `rcCorner4` , and 5th field in SPEF maps to RC corner, `rcCorner5`

	• `read_spef -rc_corner {rc_max rc_min} -spef_field {2 5} rc-spef.gz` Assuming `rc-spef.gz` in above example has RC fields from "N" RC corners, the above command would read SPEF data for two RC corners only and would be interpreted as follows: 2nd field in SPEF maps to RC corner, rc_max 5th field in SPEF maps to RC corner, `rc_min` • `read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3} -spef_field {1 1 3} rc-spef.gz` Assuming `rc-spef.gz` in the above example has RC fields from N RC corners, the above command would read SPEF data for 3 RC corners only and would be interpreted as follows: 1st field in SPEF maps to RC corner, `rcCorner1` 1st field in SPEF also maps to RC corner, `rcCorner2` 3rd field in SPEF maps to RC corner, `rcCorner3` • `read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field {1 2 3 4 5} {top.spef.gz block1.spef.gz block2.spef.gz}` In the above example, hierarchical design SPEF files are read in. It reads top level multi-field SPEF file followed by the multi-field SPEF files from `block1`and `block2` . **Note:** SPEF reading will not be triggered until SPEF files for all active RC corners are specified.
`-scale_rc`	Scales the SPEF file read in for the specified corner as per the `postRoute_cap`, `postRoute_xcap`, and `postRoute_res` scale factors.
`-scale_nets_file` *`netListFile`*	

	Enables the scaling of resistances and capacitances for user-specified nets as per the specified scale factors. The list of nets is specified in a file. **Note**: This parameter can only be used if the `-scale_rc` parameter is specified. Otherwise, it will be ignored. For example: `read_spef -scale_rc -scale_nets netFileName -rc_corner r1 detail.spef`
`-skip_star_node_locations`	Ignores the `*N` statements in the SPEF file. The `*N` statements define the location of the RC nodes. **Note**: By default, the software ignores the *N statements while reading SPEF.
`-spef_field` *`integer_list`*	Specifies the list of RC corner indices in SPEF to be loaded. This parameter provides the flexibility to perform order-independent and field-independent mapping to RC corners. For example, SPEF may contain data for 10 corner fields but you can choose to read data only for 6 corners. You can also choose to use data from one particular field to map to one or more RC corners. A few examples to illustrate different scenarios are given in the Examples section. **Note:** This parameter is relevant only when the `-rc_corner` parameter is specified. The `-spef_field` is an optional parameter, and if not specified, the software assumes the default fields of {1 2 3 … N} where N is equal to the number of RC corners provided in the `-rc_corner` list.
`-via_variation_file` *`viaVariationFileName`*	
	Specifies the name of the file for modeling via variation. The contents of the file include all the via layer names with corresponding MEAN_SHIFT and STDDEV so that all the VIA resistance in SPEF can be mapped to their own variation multiplier. This parameter is used to specify the via_variation file only when the design has a single corner. For example, `read_spef`

```
    -rc_corner rcCornerName \
-via_variation_file viaVariationFileName
```

A sample via_variation file is provided below:

```
// corner_name  : rcMax

// wildcard to cover default
object_type     : design
delay_type      : via

layer_name      : *

area            : 0.20 0.40 0.80

// no nominal for wildcard default
table_min_sigma : 0.80 0.90 0.95

table_max_sigma : 1.60 1.30 1.15

table_meanshift : 0.05 0.03 0.02

table_stdev     : 0.10 0.05 0.02

// per via layer data
object_type     : design
delay_type      : via
layer_name      : VIA1

Area            : 0.0004 0.0008 0.0016

table_min_sigma : 0.80 0.90 0.95

table_max_sigma : 1.60 1.30 1.15

table_meanshift : 0.015 0.015 0.012

table_stdev     : 0.214 0.150 0.117

object_type     : design

delay_type      : via

layer_name      : VIA2

area            : 0.0004 0.0008 0.0016

table_min_sigma : 0.80 0.90 0.95

table_max_sigma : 1.60 1.30 1.15

table_meanshift : 0.015 0.012 0.014
```

	….. ……. `table_stdev : 0.223 0.183 0.140` **Note**: Each line in the file lists the basic components of the statistical variation approach. The values in the "area" line list all possible areas. In the above example, the area line lists three cases that can happen in the design. It can list different number of cases, for example one or four. The subsequent lines list the corresponding values matching to the areas defined in the area line.

Examples

- The following command loads the SPEF file `TOPCHIP_SP.spef` for a flat chip methodology for RC corner, `corner1`:

```
read_spef -rc_corner corner1 TOPCHIP_SP.spef
```

- The following command loads multiple SPEF files for hierarchical SPEF stitching:

```
read_spef -rc_corner corner1 { PTN/results_conv/results_conv.spef
tdsp_core/tdsp_core/tdsp_core.spef PTN/TOPCHIP_SP/TOPCHIP_SP.spef }
```

- The following commands a) setup the appropriate scale factors for the corners `RCmax` and `RCmin` b) scale the SPEF read for the defined corners `RCmax` and `RCmin`:

```
=== MMMC setup ====
create_rc_corner -name RCmax -postRoute_cap 1.4 -post_route_xcap 1.4
-postRoute_res 1.4
create_rc_corner -name RCmin -postRoute_cap 0.8 -post_route_xcap 0.8
-postRoute_res 0.8
...
=== run script===
...
read_spef -rc_corner RCmax typSPEF.spef -scale_rc
read_spef -rc_corner RCmin typSPEF.spef -scale_rc
```

- The following command loads a SPEF file `pnta-spef.gz` that has RC fields from 5 RC corners:

```
read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -
```

```
spef_field {1 2 3 4 5} pnta-spef.gz
```

The above command will be interpreted as follows:

```
1st field in SPEF maps to RC corner rcCorner1
2nd field in SPEF maps to RC corner rcCorner2
3rd field in SPEF maps to RC corner rcCorner3
4th field in SPEF maps to RC corner rcCorner4
5th field in SPEF maps to RC corner rcCorner5
```

- The following command loads a SPEF file `pnta-spef.gz` that has RC fields from N RC corners:

  ```
  read_spef -rc_corner {rc_corner1 rc_corner2} -spef_field {2 5} pnta-spef.gz
  ```

  The above command will read SPEF data for 2 RC corners only and will be interpreted as follows:

  ```
  2nd field in SPEF maps to RC corner rc_corner1
  5th field in SPEF maps to RC corner rc_corner2
  ```

- The following command loads a SPEF file `pnta-spef.gz` that has RC fields from N RC corners:

  ```
  read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3} -spef_field {1 1 3} pnta-spef.gz
  ```

  The above command will read SPEF data for 3 RC corners only and will be interpreted as follows:

  ```
  1st field in SPEF maps to RC corner rcCorner1
  1st field in SPEF maps to RC corner rcCorner2 as well.
  3rd field in SPEF maps to RC corner rcCorner3
  ```

- The following command loads hierarchical design SPEF files. The software reads the top level multi-field SPEF file followed by the multi-field SPEF file from `block1` and `block2`:

  ```
  read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field {1 2 3 4 5} {top.spef.gz block1.spef.gz block2.spef.gz}
  ```

report_net_parasitics

```
report_net_parasitics
[-help]
net_name
-rc_corner rc_corner_name
[-out_file output_file_name]
```

Reports the RC parasitics of the specified net.

Note: This command is not supported for preRoute extraction.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_net_parasitics` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_net_parasitics`.
`-out_file` *`output_file_name`*	Specifies the name of the output file.
`-rc_corner` *`rc_corner_name`*	Specifies the RC corner name for which the RC parasitics are to be reported.
`net_name`	Specifies the name of the net for which the RC parasitics are to be reported.

Example

- The following command reports the RC parasitics for net, NET1 for RC corner, `rc_best`:

```
report_net_parasitics -rc_corner rc_best NET1

Net: NET1
RC Corner: rc_best
Total Cap = 0.473082 ff
Total Res = 2.104041 ohms
Total XCap = 0.334035 ff
Cap Distribution:

Total Ground Cap = 0.139046 ff
XCap With Nets:

NET3 = 0.059964 ff
NET4 = 0.040277 ff
NET5 = 0.233794 ff
```

report_rcdb

```
report_rcdb
[-help]
dir_name
```

Displays the contents of the RCDB being read.

Note:

1. This command is supported only for reading flat single RCDB. Multiple RCDB's cannot be read simultaneously using this command.
2. The RCDB being read must be generated on the same platform and architecture on which it is being read. For example, if you have generated the RCDB on Linux platform, then the `report_rcdb` command cannot report its content on the IBM AIX platform. Similarly, an RCDB generated on a 64-bit system cannot be reported on a 32-bit system.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_rcdb` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_rcdb`.
`dir_name`	Specifies the location of the RCDB directory.

Example

- The following command reports the contents of the RCDB being read in the `test.rcdb.d` directory:

```
report_rcdb test.rcdb.d
Opening parasitic data file 'test.rcdb.d/header.da' for reading.
RCDB VERSION           : 11.1
OS-bit                 : 64
OS-endianness          : little-endian
RCDB Compression       : on
RCDB Source            : QRC
DESIGN                 : test
RC-Corners:            : typical
Coupled                : yes
Node-loc               : yes
Statistical            : yes
```

report_unit_parasitics

```
report_unit_parasitics
[-help]
-layer metal_layer_name
-rc_corner rc_corner_name
[-space wire_space_in_um]
[-via_res via_resistance]
[-width wire_width_in_um]
```

Reports unit parasitics for individual metal layers and vias. The report prints the per unit resistance and capacitance for the specified layer. The resistance values are reported in ohms per micron for min width and min spacing of the layer. The capacitance values are reported in femtofarad (fF) per micron for min width and min spacing of the layer.

Via parasitics are printed considering the layer mentioned in the command as the lower layer of the via. The default via specified in the LEF file will be considered in this case.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_unit_parasitics` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_unit_parasitics`.
`-layer` *metal_layer_name*	Specifies the name of the metal layer for which the unit parasitics are to be reported.
`-rc_corner` *rc_corner_name*	Specifies the name of the RC corner for which the unit parasitics are to be reported.
`-space` *wire_space_in_um*	Specifies the wiring space in microns. *Default*: minimum spacing
`-via_res` *via_resistance*	When this parameter is specified, the resistances of all vias, whose upper layer is specified using the `-layer` parameter, are reported.
`-width` *wire_width_in_um*	Specifies the wiring width in microns. *Default*: minimum width

Examples

- The following command reports the unit parasitics for metal layer, `Metal5`, for the specified RC corner, `rc_worst`:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)
      RC-Corner = rc_worst
        Cap = 0.213204
        Res = 0.900873
  Parasitics for default Via "VIA5_0_VH"
        Via Cap = 0.017901
        Via Res = 8.859576
  ```

- The following commands reports the unit parasitics for `Metal5` with spacing two times the minimum spacing:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst -space [expr 2 * [dbGet
  [dbGet head.layers.extName Metal5 -p].minSpacing]]
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)

      RC-Corner = rc_worst

          Cap = 0.188657
          Res = 0.900873

  Parasitics for default Via "VIA5_0_VH"

          Via Cap = 0.017901
          Via Res = 8.859576
  ```

- The following commands reports the unit parasitics for Metal5 with width two times the minimum width:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst -width [expr 2 * [dbGet
  [dbGet head.layers.extName Metal5 -p].minWidth]]
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)

      RC-Corner = rc_worst
      Cap = 0.198657
      Res = 0.555333
  ```

```
Parasitics for default Via "VIA5_0_VH"

    Via Cap = 0.017901
    Via Res = 8.859576
```

- The following commands reports the unit parasitics for vias with top layer of Metal5:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst -via_res
  ```

  The software gives the following information:

```
Unit parasitics for layer Metal5 (in fF,ohm)

    RC-Corner = rc_worst
    Cap = 0.213204
    Res = 0.900873

Parasitics for default Via "VIA5_0_VH"

    Via Cap = 0.017901
    Via Res = 8.859576
    Other Via res
    VIA45_1C 8.859576
    VIA45_1C_H 8.859576
    VIA45_1C_V 8.859576
    VIA45_1ST_N 8.859576
    VIA45_1ST_S 8.859576
    VIA45_2C_E 4.429788
    VIA45_2C_W 4.429788
    VIA45_2C_N 4.429788
    VIA45_2C_S 4.429788
    VIA45_2ST_N 4.429788
    VIA45_2ST_S 4.429788
    VIA45_4C 2.214894
```

write_parasitics

```
write_parasitics
[-help]
[-exclude_ilm]
[-no_resistances]
{{{-spef_file file_name} [-exclude_nets_file file_name | -include_nets_file file_name |
-net_name net_name]}}
[-view view_name | -rc_corner rc_corner_name]
[-cap_unit {pf ff}]
```

Reads the parasitic database and outputs the parasitics in the Standard Parasitic Exchange Format (SPEF).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_parasitics` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_parasitics`.
`-cap_unit {pf ff}`	Specifies the unit of capacitance as `fF` or `pF` in the output SPEF file. *Default:* `pf`
`-exclude_ilm`	Writes out only the top-level SPEF while excluding the ILM parasitics. For example, `write_parasitics -exclude_ilm -spef abc.spef -rc_corner corner1`
`-exclude_nets_file` *file_name*	Specifies the name of the file that contains nets to be excluded from the output file.
`-include_nets_file` *file_name*	Specifies the name of the input file that contains the net names to be extracted. *Default:* Extracts all net names
`-net_name` *net_name*	Specifies the name of the net to be written in the SPEF file. *Default:* Writes all nets
`-no_resistances`	Generates a SPEF file without the resistance information. The SPEF file generated by this parameter can only be used to improve the performance of third-party simulator tools in early stages of the design. **Caution**: Most tools expect the resistance information to be present in the SPEF file for performing extraction. Therefore, SPEF files with no resistance information should not be used with ostrich for RC correlation.
`-rc_corner` *rc_corner_name*	Specifies the RC corner that is used to generate the parasitic output file in the multi-corner flow.
`-spef_file` *file_name*	Specifies the name of the SPEF output file.
`-view` *view_name*	Specifies that the RC corner associated with the specified view is used to generate the output SPEF file in the multi-corner flow.

Examples

- Reads the parasitic database created during RC extraction for RC corner, `corner1`:

  ```
  write_parasitics -rc_corner corner1
  ```

- Reads the parasitic database created during RC extraction for the view, `view1`:

  ```
  write_parasitics -view view1
  ```

write_rcdb

```
write_rcdb
[-help]
out_dir
```

Saves the RC extraction data in the specified directory. Use this command after running `read_spef` to ensure that the parasitic data is available in the Tempus database.

Note: You can restore the parasitic data on systems that are same as those used for saving the data. For example, if the data was saved on Linux in 32-bit mode, you can restore it on Linux 32-bit mode only.

Parameters

`-help`	Outputs a brief description that includes type and default information for the `write_rcdb` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_rcdb`.
`out_dir`	Specifies the location of the RCDB directory. The directory name should follow the `*.rcdb.d` format.

19

Signal ElectroMigration Commands

- create_cell_signal_em_model
- create_top_scope
- report_freq_violation
- set_max_cap_per_freq
- set_max_transition_per_freq

create_cell_signal_em_model

```
create_cell_signal_em_model
[-help]
[-cell cell_name]
-dir directory_name
[-power_net_voltages voltage]
[-sdc list_of_SDC_files]
[-spef list_of_SPEF_files]
[-tech_file filename]
[-tech_file_map mapping_file]
[-lef list_of_LEF_files]
[-verilog list_of_Verilog_files]
[-overwrite  | -reuse_existing ]
[-def filename]
[-include_def]
[-log_dir directory_name]
[-spef_options { list_of_options }]
[-def_options { list_of_options }]
[-use_verilog]
```

Specifies to create the cell boundary model for the Top Scope Signal EM analysis flow. After the boundary model is generated, it is automatically loaded to the higher-level module.

You can specify the `create_cell_signal_em_model` command before or after running the `init_design` command. The details of the two use models are given below:

- If the `create_cell_signal_em_model` command is specified before `init_design`, the command will generate the boundary model for a sub-cell and load it. For this use model, the `-cell`, `-dir`, `-spef`, and `-verilog` parameters must be specified.
- If the `create_cell_signal_em_model` command is specified after `init_design`, the command will generate the model for the top cell for use in a subsequent top-level run. In this case, the command is issued in a block-level run, and the SPEF should be loaded prior to the `create_cell_signal_em_model` command. For this use model, the `-dir` parameter must be specified. Since the design is already loaded, the relevant information is obtained from the current session.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_cell_signal_em_model` parameter. For a detailed description of the command and all of its parameters, use the man command man `create_cell_signal_em_model`.
`-cell` *`cell_name`*	
	Specifies the name of the cell that is being modeled. The `-cell` parameter takes a single cell name. If the `-cell` parameter is used with the command after specifying the `init_design` command, the cell name must be the top cell name.
`-def` *`filename`*	

	Specifies to read the list of DEF files during the boundary model generation session so that the physical information is present when generating the model. When this parameter is specified, the Tech LEF file must be specified with the `-lef` parameter. **Note**: The `-def` and `-include_def` options must be used together. These parameters are specified if you want the boundary model to include a DEF that corresponds to the boundary model logic. When the boundary model is loaded into the top-level session, this DEF will override any user-supplied DEF that was specified to the `read_def` command for the model block. If you do not specify a DEF for the boundary model block, then the boundary model DEF will be added to the list of DEFs loaded by `read_def`.
`-def_options { list_of_options }`	
	Specifies additional options that are to be used with the `read_def` command run in the boundary model generation session. The options specified with the `-def_options` parameter are appended to the `read_def` command. The `-def_options` parameter must be specified with the `-def` parameter. For example, if you want to preserve the wire segments in the user-supplied DEF that was specified with the `read_def` command for the model block, use the following command: `create_cell_signal_em_model … -def files -def_options {-preserve_shape}`
`-dir directory_name`	
	Reads the list of specified DEF files during the boundary model generation session so that the boundary model physical information is available.
`-include_def`	Specifies to generate the boundary DEF file for the netlist in the model. When specified, the boundary model directory will contain a subset of the input DEF for just the parts of the netlist that are included in the model netlist. **Note**: The `-def` and `-include_def` options must be used together. These parameters are specified if you want the boundary model to include a subset DEF that corresponds to the boundary model logic. When the boundary model is loaded into the top-level session, the subset DEF will override any user-supplied DEF which was specified to the `read_def` command for the model block. If you do not specify a subset DEF for the boundary model block, then the boundary model DEF will be added to the list of DEFs loaded by `read_def`.
`-lef list_of_LEF_files`	

	Specifies the list of LEF files to be loaded. A technology LEF with the layer definition is required if layer information is to be preserved in the SPEF generated for the model.
`-log_dir` *`directory_name`*	
	Specifies the directory where the log files for the boundary model generation run will be saved. If the directory does not exist, a new directory will be created. This directory contains the `voltus_sigem.log`, `voltus_sigem.logv`, and `voltus_sigem.cmd` files.
`-overwrite`	
	Allows overwrite of the existing directory.
`-power_net_voltages` *`voltage`*	
	Specifies the voltage values for the power nets. The format of the `-power_net_voltages` argument is a list of net names and voltage values with the @ character separating the power net name from the voltage. Example: `{VDD1@1.0 VDD2@0.9}`
`-reuse_existing`	
	Specifies to load an existing boundary model directory.
`-sdc` *`list_of_SDC_files`*	
	Specifies the list of SDC files to be loaded. If SDC files are specified, then the fanin to the clock definition pins will be included in the generated model.
`-spef` *`list_of_SPEF_files`*	
	Specifies the list of SPEF files to be loaded.
`-spef_options { list_of_options }`	

	Specifies additional options that are to be used with the `read_spef` command run in the boundary model generation session. The options specified with the `-spef_options` parameter are appended to the `read_spef` command. For example, if the boundary model is to be generated from a session that uses a view definition, the `-spef_options` parameter can be used to specify the rc_corner, as shown in the following command: `create_cell_signal_em_model … -spef` *`files`* `-spef_options {-rc_corner corner1}` The `-spef_options` parameter must be specified with the `-spef` parameter.
`-tech_file` *`filename`*	
	Specifies the extraction technology file to be used with the RC corner. The technology file is required if layer information is to be preserved in the SPEF generated for the model.
`-tech_file_map` *`mapping_file`*	

	Specifies the mapping file that includes the mapping between the SPEF layers to the LEF layers. The mapping file is required if layer information is to be preserved in the SPEF generated for the model.
	
	The format of the mapping file is:
	`# <LEF/DEF Layer Name> #<SPEF Layer Name>`
	`M1 METAL_1 \`
	`VIA1 VIA_1 \`
	`M2 METAL_2 \`

`-use_verilog`	
	Specifies to generate boundary models that use Verilog for the model netlist. When this parameter is not specified, the models are generated using a non-readable binary database. The `-use_verilog` parameter can be specified if you want to debug the models and prefer a readable format for the model netlist.
`-verilog` *`list_of_Verilog_files`*	
	Specifies the list of Verilog files to be loaded.

Examples

- The following is the command for creating a boundary model and loading it into the current session:
  ```
  create_cell_signal_em_model -dir bm_dir  -cell cella  -verilog cella.v  -spef cella.spef.gz  -overwrite -lef tech.lef  -tech_file 65lp_typical.tch -tech_file_map  leflayermap
  ```
- The following is the command for creating a boundary model after running `init_design` in a block-level session:
  ```
  create_cell_signal_em_model -dir bm_dir -overwrite
  ```
- The following command can be used prior to `init_design` (or `set_top_module`) to import physical design data:
  ```
  create_cell_signal_em_model -dir bm_dir -overwrite -verilog block.v -spef block.spef -def block.def -include_def
  ```
  Note: It is not possible to import physical design data after `init_design` (or `set_top_module`) as the data has already been loaded. A warning will be issued but will not prevent boundary model generation.
- The following command generates a boundary model, and saves the log files produced by the boundary model generation sub-process into a different directory:
  ```
  create_boundary_mode -dir  bm_dir -overwrite -verilog block.v -spef block.spef -log_dir ./bm_logs
  ```

Related Topics

- "Signal ElectroMigration" in the *Voltus User Guide*

create_top_scope

```
create_top_scope
[-help]
[-cell_list list]
[-fanout_limit value]
[-ignore_nets netname]
[-include_side_loads]
[-inst_list list]
[-summary]
[-thread_count value]
[-include_pins pin_names]
```

Specifies to to perform abstraction of the top-level hierarchical instances for analysis of only the top-level logic, resulting in faster turnaround time. The command allows you to perform abstraction of all the hierarchical instances in the top-level, or specify the top-level hierarchical instances that should be abstracted. When a hierarchical instance is abstracted, all the internal logic is ignored while retaining the interface paths.

The `create_top_scope` command must be specified before the `read_sdc` and `init_design` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_top_scope` parameter. For a detailed description of the command and all of its parameters, use the man command man `create_top_scope`.
`-cell_list list`	
	Specifies to include the interface paths of only the specified cells.
`-fanout_limit value`	
	Specifies to unmark the boundary nets with fanout greater than the given limit. The default limit is 1000.

`-ignore_nets` *`netname`*	
	Skips unmarking the fanin/fanout of the specified boundary nets.
`-include_side_loads`	
	Specifies to include all side receivers in the scope.
`-inst_list` *`list`*	
	Specifies to include the interface paths of only the specified instances.
`-include_pins` *`pin_names`*	
	Specifies a list of user-defined pins which should be included that may not be on the boundary paths in the Top Scope Signal EM Analysis flow. The top scope will include the full fanin cone to all the specified pins as part of the analysis. This parameter allows you to handle scenarios where there are some clock constraints defined on pins in sub-blocks which are not part of the boundary logic. In such a case, the constraint would fail and display an error message, and any constraints for clocks generated from the failing clock would fail as well. With this parameter, you can include all the root pins for the clock constraints as part of the top scope, thus resulting in no errors.
`-summary`	
	Specifies to print statistics about the included instances or nets.
`-thread_count` *`value`*	
	Specifies the number of threads to be used.

Examples

- The following command specifies the top-level hierarchical instances that should be abstracted:

```
create_top_scope -summary -inst_list {InstanceName1, InstanceName2..}
```

report_freq_violation

```
report_freq_violation
[-help]
[-detailed]
[-em_table_scale float]
[-exclude_insts_file string]
[-include_insts_file string]
[-out_file string]
[-report_format integer]
[-sort {name | freq | mindiff | maxdiff | minfreq | maxfreq}]
[-transition {min | max}]
[-view <string>]
[-em_type {avg peak rms all}]
```

Runs electromigration (EM) analysis based on the frequency table defined in the Liberty library, and generates a frequency violation report.

The `report_freq_violation` command uses default activity (1.0 or 100% activity on data network and 200% on clock network) or user-defined activity (TCF/VCD) to compute effective frequency on each instance pin. It then determines frequency violation at the output pin from the electromigration templates defined in Liberty, as shown in the following figure:

```
library (name) {
   em_lut_template(EM_Tran) {
      variable_1: input_transition_time ;
      index_1("transtion_list");
   }
   em_lut_template(EM_Cap) {
      variable_1: total_output_net_capacitance ;
      index_1("capacitance_list");
   }
   em_lut_template(EM_Tran_Cap) {
      variable_1: input_transition_time ;
      variable_2: total_output_net_capacitance ;
      index_1("transtion_list");
      index_2("capacitance_list");
   }
```

The frequency limit on the output pin is derived based on the slew on the related input pin and output load.

The activity for the design can be specified as follows:

- `set_default_switching_activity`
- `propagate_activity`

- `write_tcf`
- `read_activity_file -format TCF -set_net_freq true <TCF file>`

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_freq_violaltion` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_freq_violation`.
`-detailed`	Generates a detailed report containing information for all instances, including those without violations.
`-em_table_scale` *`float`*	Scale factor for EM table value. *Default*: 1.0
`-em_type {avg peak rms all}`	
	Specifies to report frequency violations for the rms/peak/avg current waveform in the frequency violation report. When this parameter is specified, the software reads the `em_max_toggle_rate` table in the Liberty library for the specified EM current type (rms/peak/avg). You can specify `all` to perform all three types of analyses.
`-exclude_insts_file` *`string`*	Specifies a file that lists the names of instances to exclude from the frequency violation report.
`-include_insts_file` *`string`*	Specifies the selected instance file name.
`-out_file` *`string`*	Specifies the report file (default: `freqViolations.rpt`) for the violation data.

`-report_format` *integer*	Specifies a format of the violation report. The report details vary according to the selected report format. You can specify report formats 0, 1, or 2. *Default*: 0	
`-sort` {`name` | `freq` | `minDiff` | `maxDiff` | `minFreq` | `maxFreq`}	Sorts the instance-based frequency data by `name`, `freq`, `minDiff`, `maxDiff`, `minFreq`, and `maxFreq`. By default, the violation report is not sorted.	
	`name`	Name of a pin or arc.
	`freq`	Frequency value of a pin or arc.
	`min_freq`	Minimum frequency value of a pin or arc as defined in the frequency table.
	`max_freq`	Maximum frequency value of a pin or arc as defined in the frequency table.
	`min_diff`	Difference between `freq` and `minFreq` of a pin or arc.
	`max_diff`	Difference between `freq` and `minFreq` of a pin or arc.
-transition {min | max}	Specify the slew used for the report. *Default:* `min`	
`-view` *string*	Specifies the view for which to generate the frequency violation report. Use this parameter when working with multi-mode multi-corner (MMMC) designs.	

Example

The following command reports instances that violate frequency constraints and lists them in the `freqViol.rpt` report file:

```
report_freq_violation -out_file freqViol.rpt
```

Instance File Format

The instance file is a text file that contains a list of instances. Each instance name appears in one line in the file. The comments begin with "`#`". Leading or ending spaces in instance names are ignored.

Report Format

The report formats are described below.

Format 0

The following command generates a report providing details of `minFreq`, `Freq`, `Tran`, `Cap`, `Remark`, and `Arc/Pin` and directs the output to report file `summaryViol.rpt`:

```
report_freq_violation -out_file summaryViol.rpt -report_format 0
```

For example,

```
minFreq    Freq    Tran     Cap         Remark    Arc/Pin
5.985      1.009   0.0356   0.003103              tt0/data/r3/add_27/U1_1_6/Y
```

Format 1

When `reportFreqViolation -report_format` is set to `1`, the report provides details as shown in the following example:

```
Status   Remark   freq   minFreq   maxFreq   Arc/Pin
[P]      *        1.009  5.985     5.995     tt0/data/r3/add_27/U1_1_6/Y
```

Format 2

A detailed report is generated when `reportFreqViolation -report_format` is set to `2`, as shown in the following example:

```
Status  Remark  freq   minFreq  maxFreq    minTran    maxTran    minCap         max_cap
     Arc/Pin
[P]     *       1.009  5.985    5.995      3.560e-02  3.790e-02  3.103e-03
3.103e-03  tt0/data/r3/add_27/U1_1_6/Y
```

Where:

`Status`	Arc or pin status - `F` (fail), `V` (violation), `P` (pass), or `?` (unknown).
`Remark`	Indicates whether the pin is a clock.
`freq`	Working frequency of the arc or pin.
`minFreq`	Minimum frequency value taken from the frequency table.
`maxFreq`	Maximum frequency value taken from the frequency table.
`minTran`	Minimum slew value.
`maxTran`	Maximum slew value.
`minCap`	Minimum capacitance load.
`maxCap`	Maximum capacitance load.
`Arc/Pin`	Name of the arc or pin.

set_max_cap_per_freq

```
set_max_cap_per_freq
-cap {cap1 cap2 …}
[-force]
-freq {freq1 freq2 …}
[-lib libName]
-pins {cell pin}
```

Adds a maximum capacitance per frequency table to the existing `max_capacitance` statement in the timing library.

Within-cell, DC electromigration limits vary with varying frequency of transition. The software manages DC electromigration using a maximum capacitance limit based on switching frequency. This is driven by maxCapPerFreq data in the liberty. These constraints can be set for cell pins manually using this command.

Timing analysis uses this table to detect violations.

Parameters

`-cap {cap1cap2...}`	
	Specifies a list of capacitance values that must be greater than or equal to 0. You must specify the same number of `-freq` and `-cap` values. The software derives the maximum capacitance from a linear interpolation (or extrapolation) for a given frequency value using two frequency values in the table. For interpolation, the software uses the closest value above the frequency, and the closest value below the frequency. For extrapolation, the software uses the two closest values to the frequency. *Type:* Float, specified in units of pF

`-force`	Computes the maximum capacitance from the interpolated `-freq` and `-cap` values only, and ignores any previously defined `max_capacitance` values for these pins in the library. *Default:* Computes the maximum capacitance using the worst case (minimum value) between the library `max_capacitance` values and the interpolated `-cap` values.
`-freq {`*`freq1freq2...`*`}`	
	Specifies a list of at least two frequency values that must be greater than or equal to 0. You must specify the same number of `-freq` and `-cap` values, and the frequency values must be specified in ascending order. *Type:* Float, specified in units of Hz
`-lib` *`libName`*	
	Specifies the library name inside liberty. For example, library (`slowlvt`) inside slow.lib. The library name should be specified as `slowlvt`. Modifies only the matching cell-pins in the specified library. *Default:* Modifies the matching cell-pins in all libraries
`-pins {`*`cellpin`*`}`	
	Adds the new capacitance constraints to the specified set of cells and pins. You can specify output or inout pins. The asterisk (`*`) wildcard character can be used in a cell or pin name. If a table associated with the pin already exists, it is replaced with the new table. For analysis and optimization, the software uses the most restrictive (the lowest capacitance) value from either the table or the `.lib max_capacitance`. If you also specify the `-force` parameter, the `max_capacitance` value is ignored. The final value can be further modified using `.sdc set_max_capacitance` statements.

Example

Assume all output pins of a library have a `max_capacitance` of 3.5 pF from the library file.

- The following command sets the frequency dependent `max_capacitance` to 3.0 pF at a frequency of 100 Mhz, and to 1.5 pF at a frequency of 200 Mhz, for all output or inout pins for

all cells in the library named `fast`:

```
set_max_cap_per_freq -lib fast -freq {100e6 200e6} -cap {3.0 1.5} -pins {* *}
```

As a result, for a net at 300 Mhz, an and2 cell output pin has an extrapolated `max_capacitance` value of 0 pF (which means the cell cannot be used at that frequency). An extrapolation to 50 Mhz results in a `max_capacitance` value of 3.75 pF. However, the 3.75 pF value is greater than the existing `.lib max_capacitance` limit of 3.5 pF; therefore timing analysis uses the 3.5 pF value.

- The following command applies the same constraints to all libraries with cell names matching `dff*` that have a pin named `Qbar`:

  ```
  set_max_cap_per_freq -force -freq {100e6 200e6} -cap {3.0 1.5} -pins {dff* Qbar}
  ```

  The original `.lib max_capacitance` limit is ignored because the `-force` parameter is specified; therefore extrapolation to 50 Mhz results in a final `max_capacitance` constraint value of 3.75 pF.

set_max_transition_per_freq

```
set_max_transition_per_freq
[-help]
[-force]
-freq {freq1 freq2 ...}
[-lib lib_name]
-pins {cell | pin}
-transitions string
```

Adds a maximum transition per frequency table to the existing max_transition statement in the timing library. Timing analysis uses this table to detect violations, and the software attempts to repair those violations.

Use this command any time after loading the timing library files (that is, after importing or restoring the design).

The timing engine derives the frequency of an instance from the clock domain to which it is connected (the fastest clock domain, if there is more than one). If the instance is a clock instance (that is, part of the clock tree), the actual clock frequency is used. If the instance is part of the combinational logic (that is, a flip-flop, or the logic connected to the output of a flip-flop), the frequency used equals one-half of the associated clock domain frequency because a flip-flop output can only toggle at one-half the rate of the input clock. For example, a clock of 1.0 Mhz has 1,000,000 rise and fall transitions per second; however the output of a flip-flop connected to a 1.0 Mhz clock can only have 500,000 rise and fall transitions per second.

Timing analysis computes the frequency directly from the timing graph. Use the `check_ac_limits -toggle` command to write the effective frequency onto the associated net in the database.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_max_transition_per_freq` parameter. For a detailed description of the command and all of its parameters, use `manset_max_transition_per_freq`.

`-force`	Computes the maximum transition from the interpolated `-freq` and `-transition`values only and ignores any previously defined `max_transition` values for these pins in the library. *Default:* Uses the worst case (minimum value) between the `.libmax_transition` values and the interpolated `-transition` values.
`-freq {`*`freq1 freq2 …`*`}`	Specifies a list of at least two frequency values that must be greater than or equal to 0. You must specify the same number of `-freq` and `-transition` values, and the frequency values must be specified in ascending order. *Type:* Float, specified in units of Hz
`-lib` *`lib_name`*	Modifies only the matching cell-pins in the specified library. *Default:* Modifies the matching cell-pins in all libraries.
`-pins {`*`cell | pin`*`}`	Adds the new transition constraints to the specified set of cells and pins. You can specify input, output, or inout pins. The asterisk (`*`) wildcard character can be used in a cell or pin name. If a table associated with the pin already exists, it is replaced with the new table. For analysis and optimization, the software uses the most restrictive (the lowest transition) value from either the table or the `.lib max_transition`. If you also specify the -force parameter, the `max_transition` value is ignored. The final value can be further modified using `.sdc set_max_transition` statements.
`-transitions` *`string`*	Specifies a list of transition values that must be greater than or equal to 0. You must specify the same number of `-freq` and `-transition` values. The software derives the maximum transition limit from a linear interpolation (or extrapolation) for a given frequency value using two frequency values in the table. For interpolation, the software uses the closest value above the frequency, and the closest value below the frequency. For extrapolation, the software uses the two closest values to the frequency. *Type:* Float, specified in units of ns

Examples

Assume all output pins of a library have a `max_transition` value of 0.9 ns from the library file.

- The following command sets the frequency dependent `max_transition` to 1.0 ns at a frequency of 100 Mhz, and to 0.7 ns at a frequency of 200 Mhz, for pins matching `in*`of cells matching `*` (that is, all cells) in the library named `slow`:

  ```
  set_max_transition_per_freq -lib slow -freq {100e6 200e6} -transitions {1.0 0.7} -
  pins {* in*}
  ```

  As a result, at 300 Mhz, an `and2` cell input pin named `in1` has an extrapolated `max_transition` value of 0.4ns. An extrapolation to 50 Mhz results in a `max_transition` value of 1.15 ns. However, the 1.15 ns value is greater than the existing `.lib max_transition` value of 0.9 ns; therefore timing analysis uses the 0.9 value.

- The following command applies the same constraints to all libraries with cell names matching `dff*` that have a pin named `clk`:

  ```
  set_max_transition_per_freq -force -freq {100e6 200e6} -transitions {1.0 0.7} -
  pins {dff* clk}
  ```

  Note: The original `.lib max_transition` value is ignored because the `-force`parameter is specified; therefore an extrapolation to 50 Mhz results in a final `max_transition` limit of 1.15 ns.

20

Unix Commands

- flowtool
- lef2oa
- oa2lef
- voltus

flowtool

```
  flowtool
[-flow <>] [-from <> [-ref_run <>]] [-to <>]
[l][-files <>][/l]
[-overwrite]
[-display {all | trunk}]
[-dist <>]
[-dir <>] [-db <>]
[-enabled <>] [-inject_tcl <>]
[-isolate]
[-legacy_ui | -common_ui]
[-predict {script | summary | verbose | tcl}]
[-run_tag run_tag_value]
[-metrics_file]
[-status]
[-version]
```

`flowtool` provides a way to run flows from the command line. In addition to providing another interface to existing Flowkit functionality, it enables multi-session, multi-tool and branching flows.

Note: TCL 8.5 or higher is required to run flowtool.

Parameters

`-common_ui`	Runs the command in common user interface (default).

`-display`	Filters the output to various types of flows. Possible values are `all` and `trunk`.
`-flow`	Specifies that the flow has started running.
`-from`	Specifies the step (inclusive) to start running the flow accepting a full dotted, comma-separated, and case-insensitive canonical path to a flow step object.
`-ref_run`	If `-from` is specified, the `-ref_run` option points to a working directory/database of a previous run.
`-to`	Specifies the step (inclusive) to stop running the flow accepting a full dotted, comma-separated, and case-insensitive canonical path to a flow step object.
-files	Specifies the tcl scripts describing the flow and flow steps Default: *`flow.scripts/run_flow.tcl`*
-overwrite	Ignores any existing steps that have been run and runs those again.
`-dist`	Specifies the distribution script and arguments to be prepended to the command line when executing tools.
`-dir`	Changes to the named directory before doing anything else (including looking for starting scripts); analogous make's `-C` argument.
`-db`	Specifies the database to be used to start the run.
`-isolate`	Runs each step in its own tool session.
`-legacy_ui`	Runs the command in legacy user interface.
`-predict`	Specifies the prediction of the flow that will run (before executing the flow). Possible values are `script`, `summary`, `verbose`, and `tcl`.
`-run_tag` *`run_tag_value`*	Sets the `-dir` option to the same as the *`run_tag_value`*, if the `-dir` option is not specified. Additionally, it updates the tool startup code to include: `set_db flow_run_tag <value>`
`-enabled`	Enables the specified features.
`-inject_tcl`	Injects the specified TCL into every step.
`-metrics_file`	Overrides the location of the `flow.metrics` file (unless the file is provided), and keeps any existing metrics.

`-status`	Prints the execution status (completed steps) of a run.
`-version`	Displays the version number and exits from the tool.

See also:

Flowkit chapter of the *Stylus Common UI Migration Guide* document.

lef2oa

```
lef2oa
-lef file
-lib library
[-commentChar char]
[-dataModel dataModel_value]
[-defaultRuleName name]
[-DMAttributes attributeList]
[-DMSystem sysName]
[-h | -help]
[-layerMap filelist]
[-libDefFile fileList]
[-logFile file]
[-mapConflicts]
[-noInfo msgIds]
[-noWarning msgIds]
[-overwrite]
[-pinLabels]
[-pnrLibDataOnly]
[-shared]
[-techDMAttributes attrList]
[-techDMSystem sysName]
[-techLib library]
[-techLibPath path]
[-techRefs parentTechs]
[-templateFile file]
[-textHeight value]
[-textLayer value]
[-view viewName]
[-v]
[-version]
```

The LEF to OpenAccess (`lef2oa`) translator can be used to convert LEF files into equivalent OpenAccess data. It supports three main styles of use:

1. Converting the technology information, such as layers, DRC rules, via rules, vias, and so on, into an OA tech file. This OA techfile is sufficient for the software use but is not really usable for Virtuoso because it does not contain information about custom layers below `metal1`, pcells, and so on. This techfile can be read by `techLoadDump` to create an ASCII techfile and then users can cut-and-paste the necessary information, such as DRC rules, into a Virtuoso OA PDK ASCII techfile to create a merged OA techfile.

2. Creating an incremental techfile using the `-pnrLibDataOnly` option that only has technology data that is dependent on the standard cell library, such as default routing rule (`LEFDefaultRouteSpec`), non-default rules (NDRs), sites, and so on.
3. Converting the LEF macros into OA abstract cellviews.

The software uses a Cadence version of the LEF to OpenAccess translator that does more than the public domain `lef2oa` executable. The main difference is that many DRC rules that are expressed as properties that begin with `LEF58_` (for example, `PROPERTY LEF58_SPACING "..." ;`) are understood and mapped directly to the correct OA foundry constraints. In general, the rules needed for 28nm nodes and older are supported, while newer rules for 20nm and smaller are not supported. If you have any unsupported LEF property rules, they can be translated using Voltus commands instead. To do so, read the LEF into Voltus and use the `write_oa_techfile` command.

Parameters

`-commentChar` *`char`*	Identifies the comment character
`-dataModel` *`dataModel value`*	
	Specifies a data model value of 0, 1, 2, or 3, which determines the data model for the OpenAccess database and determines how properties are mapped to the database. If this option is omitted, the default data model value of 1 is used unless the OA_DATAMODEL environment variable is set or a file named OA.DATAMODEL is found in the data directory of your OpenAccess installation hierarchy.
`-defaultRuleName` *`name`*	
	Specifies the name of the `LEFDefaultRouteSpec` (LDRS) to generate in the technology database. If a constraint group named `LEFDefaultRouteSpec` already exists in a technology database being modified, the translation fails unless the -overwrite option is used.
`-DMAttributes` *`attributeList`*	

	Specifies a list of design management (DM) attributes that are associated with every library that this application creates. Attributes are specified as `name=value` pairs. There must be no space on either side of the equal sign, two or more pairs must be space-separated, and the list must be enclosed in quotes.
`-DMSystem` *`sysName`*	Specifies the DM system to be used for the library, which can be any DM plug-in available to the user. The DM plug-ins included with OpenAccess are `oaDMTurbo` or `oaDMFileSys`. The default DM system is `oaDMSystem`, which automatically selects between `oaDMTurbo` or `oaDMFileSys`. Use of the `oaDMTurbo` plug-in is not recommended.
`-lef` *`file`*	Specifies the LEF file to import into the OpenAccess database.
`-lib` *`library`*	Specifies the name of the destination library. If the library exists, the library must contain a technology database or refer to a library with a technology database. If the specified library exists, new designs are created for macro definitions. If you redefine a macro in the file, a new design is created by overwriting the existing design. If the library does not exist, a library is created in the current working directory and will contain a technology database.
`-mapConflicts`	Maps certain conflicts between incremental and base technologies to constraints and constraint parameter definitions.
`-h | -help`	Displays usage information.
`-layerMap` *`filelist`*	Specifies a quoted, space-separated list of one or more layer mapping files to be used during translation. You must specify the complete path for each layer map file.
`-libDefFile` *`fileList`*	
	Specifies the `lib.defs` file to load. The path to the `lib.defs` file can be a full or relative path. If this option is omitted, the default search mechanism of the DM system is used to find and load the `lib.defs` file.
`-logFile` *`file`*	Specifies the log filename. If this option is omitted, the log filename defaults to `lef2oaCDS.log`.

`-noInfo` *msgIds*	Suppresses the specified INFO messages. *msgIds* is a quoted, space separated list of numbers. Each number in the list represents the numerical portion of the ID for the message you want to suppress. None of the numbers in the list may be zero. Suppressed messages do not appear on the terminal or in the log file.
`-noWarning` *msgIds*	Suppresses the specified WARNING messages. *msgIds* is a quoted, space separated list of numbers. Each number in the list represents the numerical portion of the ID for the message you want to suppress. None of the numbers in the list may be zero. Suppressed messages do not appear on the terminal or in the log file and are not included in the total of WARNING messages displayed in the summary.
`-overwrite`	Replaces MACROS in an existing design with the translated data. If this option is omitted, the default behavior is to update the existing library. This option does not affect the DBU value in an existing technology database.
`-pinLabels`	Creates pin labels in designs by using OpenAccess `oaAttrDisplay` objects on `oaTerms`. Labels are added on the `oaTerm` for each pin figure of the terminal at the location of the pin figure. See also the `-textLayer` and `-textHeight` options.
`-pnrLibDataOnly`	Maps only library-specific place and route information to an incremental tech, ignoring foundry rules. Recommended for converting a LEF file to an incremental technology database that references a technology database that has complete foundry information. LEF constructs `DIRECTION` and `PITCH`, along with `WIDTH`, `OFFSET`, and `WIREEXTENSION` (if specified) are mapped to constraints in the `LEFDefaultRouteSpec` type constraint group. `VIAS` and `VIARULES` defined are added to the `validRoutingVias` constraint in the `LDRS`. `NONDEFAULTRULES` are mapped to OA constraint groups, `SITES` are mapped to `oaSiteDefs`, and `MACROS` are mapped to abstract `oaDesigns`.
`-shared`	Allows other applications to access the output library while this application is running.
`-techDMAttributes` *attrList*	Specifies a list of DM attributes to be associated with every technology library created.
`-techDMSystem` *sysName*	Specifies the DM system to be used for the technology library, which is either `oaDMTurbo` or `oaDMFileSys`. The default is `oaDMTurbo`.

`-techLib` *library*	Specifies the name of a technology library to use when creating a new cell library. If the specified technology library does not exist, the library is created using the technology information in the LEF file.
`-techLibPath` *path*	Specifies the path to be used for creating a technology library specified by the `-techLib` argument. The is only used when the library specified by the `-techLib` argument does not exist.
`-techRefs` *parentTechs*	Creates an incremental technology database that derives from one or more of the specified parent technology databases.
`-templateFile` *file*	Specifies a file containing arguments to `lef2oa`. You can specify a template file instead of entering a string of arguments on the command line.
`-textHeight` *value*	Specifies an integer value for the pin label text height in database units (DBU). The DBU value is set by `lef2oa` by using the DATABASE UNITS value in LEF. If `-pinLabels` is used, and this option is omitted, the pin label text height defaults to the DBU value in the technology database for maskLayout views.
`-textLayer` *value*	Specifies the layer used for pin label text. If `-pinLabels` is used, and this option is not specified, the pin label text layer defaults to text. If the specified layer does not exist in the technology database, it is added.
`-view` *viewName*	Specifies the output view name. The default view name is `abstract`.
`-v`	Prints tool, format and library version information
`-version`	Prints tool and format version information

Related Commands

- `oa2lef`

oa2lef

```
oa2lef
-lib library
-lef file
[-cell Cells]
[-libDefFile filename]
[-logFile file]
[-noInfo msgIds]
[-noTech]
[-noWarning msgIds]
[-templateFile file]
[-h | -help]
[-views viewName]
[-v]
[-ver version]
[-version]
```

This translator can be used to convert OpenAccess data into equivalent LEF data.

The software uses a Cadence version of `oa2lef` that does more than the public domain `oa2lef` executable. The main difference is that many DRC rules that are expressed as properties that begin with `LEF58_` (for example, `PROPERTY LEF58_SPACING "..." ;`) are understood and mapped to the correct LEF syntax. In general, the rules needed for 28nm nodes and older are supported, while newer rules for 20nm and smaller are not supported. If you have any unsupported LEF property rules, they can be translated into LEF using Voltus commands instead. To do so, read the OA techfile into Voltus, and use the `write_lef_library` command.

Parameters

`-cell Cells`	Specifies a space-separated list of cells to translate.
`-lef file`	Specifies the output LEF file name.
`-lib library`	Specifies the name of the source library, containing a technology database.
`-libDefFile filename`	Specifies the `lib.defs` file to load. The path to the `lib.defs` file can be a full or relative path. If this option is omitted, the default search mechanism of the DM system is used to find and load the `lib.defs` file.

`-logFile` *file*	Specifies the log filename. If this option is omitted, the log filename defaults to `oa2lef.log`.
`-noInfo` *msgIds*	Suppresses the specified INFO messages. msgIds is a quoted, space separated list of numbers. Each number in the list represents the numerical portion of the ID for the message you want to suppress. None of the numbers in the list may be zero. Suppressed messages do not appear on the terminal or in the log file.
`-noTech`	Specifies that the technology information is not output.
`-noWarning` *msgIds*	Suppresses the specified WARNING messages. *msgIds* is a quoted, space separated list of numbers. Each number in the list represents the numerical portion of the ID for the message you want to suppress. None of the numbers in the list may be zero. Suppressed messages do not appear on the terminal or in the log file and are not included in the total of WARNING messages displayed in the summary.
`-templateFile` *file*	Specifies a file containing arguments to `oa2lef`. You can specify a template file instead of entering a string of arguments on the command line.
`-help or -h`	Displays usage information.
`-views` *viewName*	Specifies a space-separated list of views to translate. Only the first view name for a particular cell is used; all others are ignored.
`-v`	Prints tool, format, and library version information.
`-ver` *version*	Specifies the LEF version to write, which can be 5.3, 5.4, 5.5, 5.6, 5.7 or 5.8. The default version is 5.7.
`-version`	Prints tool and format version information

Related Commands

- `lef2oa`

voltus

```
voltus
[-abort_on_error]
[-batch]
[-cds_lib_file in_file]
[-cpus integer]
[-disable_user_startup]
[-execute list_of_Tcl_commands]
[-files in_file_list]
[-help]
[-lic_multi_cpu lic_list]
[-lic_options lic_list]
[-lic_startup lic_list]
[-lic_startup_options lic_list]
[-lic_startup_extra lic_list]
[-log prefix]
[-no_logv]
[-no_gui]
[-overwrite]
[-stylus]
[-version]
[-wait minutes]
```

Launches a Voltus session and starts one of the following products:

- Voltus IC Power Integrity Solution L
- Voltus IC Power Integrity Solution XL

To start a Voltus session, type the `voltus` command with the appropriate parameters on the UNIX/Linux command line. If you type a command without parameters, the software starts in GUI mode and creates a log file and a command file. The system attempts to check out the license with the most functionality, then the license with the next most functionality, and so on.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man voltus`
`-abort_on_error`	Exits if an error is detected at any point. If any Tcl command returns `TCL_ERROR` to the top level, or any `Error (XXX): …` message occurs, the process will exit.
`-batch`	Runs without interactive prompt. After all the options, including `-execute` and `-file`, are processed, the job exits.
`-cds_lib_file` *`in_file`*	Loads the specified file instead of the default cds.lib file in the current directory to obtain the names and locations of OpenAccess libraries. The cds.lib file is used to list the OpenAccess library names, and their physical locations.
`-cpus` *`integer`*	Specifies the number of CPUs to be used for multi-threading on current host. This calls the appropriate Tcl command to set the local CPU count. This happens before any `-execute` or `-file` commands are processed. This may require extra multi-CPU licenses to be checked out (see `-lic_multi_cpu`).
`-disable_user_startup`	Reads only the initialization file in the installation hierarchy. Does not read the initialization files in the home or current directory. This option suppresses the default loading from the home and current directory, which can be used to guarantee there are no local over-rides to various project settings. For more information, see "Setting Preferences" in the ***"Getting Started"*** chapter of the *Voltus Stylus Common UI User Guide* for init files that are loaded by default.
`-execute` *`list_of_Tcl_commands`*	Specifies the Tcl commands to be executed as a quoted string at startup.
`-files` *`in_file_list`*	Specifies the Tcl file to be sourced at startup.
`-lic_multi_cpu` *`lic_list`*	Controls which licenses are to be used for higher multi-CPU usage in priority order from left to right. The allowed license names are: • `vtsmp`: Voltus IC Power Integrity Solution MP • `vtsxl`: Voltus IC Power Integrity Solution XL

`-lic_options` *`lic_list`*	Specifies the order that extra option licenses should be checked-out if needed. In some cases more than one option license can be used to enable a given feature. You can control which options are allowed to be checked-out, and in what priority (from left to right) when required during the run. The allowed license names are: • `vtsaa`: Voltus IC Power Integrity Solution Advanced Analysis GXL Option • `vtsesd`: Voltus IC Power Integrity Solution - ESD (Electrostatic Discharge Analysis) Option • `vtsxm`: Voltus IC Power Integrity Solution - XM (Extreme Modeling) Option • `tpsxl`: Tempus Timing Signoff Solution XL • `tpsl`: Tempus Timing Signoff Solution L • `tpstso`: Tempus Timing Signoff Solution TSO • `tpspi`: Tempus Timing Signoff Solution - PI (Power Integrity) Option • `invs_ms`: Innovus Mixed Signal Option • `vtsgaabsp`: Voltus GAA and Back Side Power Option • `vts_vi`: Voltus InsightAI Technology Option Option licenses can be checked-out dynamically when a specific Tcl command requires it. This may cause the Tcl script to stall and wait for a license, or to exit if the wait period expires. If you specify more than one product option, begin and end the list with double quotation marks or braces. For example, `-lic_options "vtsaa vtsesd"` will only allow these two options to be checked-out dynamically, and in the given priority order. **Note:** For information, see "***Product and Licensing Information***".

`-lic_startup` *`lic_list`*	Requests one of these licenses at startup in priority order from left to right. The supported licenses are: `vtsxl`: Voltus IC Power Integrity Solution XL `vtsl`: Voltus IC Power Integrity Solution L `vpsxl`: Virtuoso Power System XL `vdsp`: Virtuoso Digital Signoff Power Solution For example, `-lic_startup "vtsxl vtsl"` will try to check-out `vtsxl` first and then `vtsl`. The check-outs will be attempted repeatedly until it succeeds or the specified wait (`-wait`) period expires.
`-lic_startup_extra` *`lic_list`*	Checks out the specified Innovus license to enable saving of physical changes. By default, Voltus cannot save physical changes to the database, but if you check out the correct extra Innovus license at startup, you can save the physical changes. The supported licenses are: • `invs` - Innovus Implementation System • `invsb` - Innovus Implementation System Basic • `vdixl` - Virtuoso® Digital Implementation XL **Note**: The `vdixl` license can only be used with the Voltus base license `vdsp`.

`-lic_startup_options` *`lic_list`*	Specifies additional option licenses needed at startup. If not provided, they will be checked out when needed during the run. The allowed license names are: • `vtsaa`: Voltus IC Power Integrity Solution Advanced Analysis GXL Option • `vtsesd`: Voltus IC Power Integrity Solution - ESD (Electrostatic Discharge Analysis) Option • `vtsxm`: Voltus IC Power Integrity Solution - XM (Extreme Modeling) Option • `tpsxl`: Tempus Timing Signoff Solution XL • `tpsl`: Tempus Timing Signoff Solution L • `tpstso`: Tempus Timing Signoff Solution TSO • `tpspi`: Tempus Timing Signoff Solution - PI (Power Integrity) Option • `invs_ms`: Innovus Mixed Signal Option • `vtsgaabsp`: Voltus GAA and Back Side Power Option • `vts_vi`: Voltus InsightAI Technology Option For example, `-lic_startup_options "vtsaa, vtsesd"` will try to check-out both `vtsaa` and `vtsesd` when Voltus starts up. The check-outs will be attempted repeatedly until it succeeds or the `-wait` period expires.
`-log` *`prefix`*	Specifies a name for the log file. By default, the software saves log files in the run directory and increments them, for example, `voltus.log`, `voltus.log1`, `voltus .log2`, `voltus .log3`, and so on. *Default:* `voltus.log`
`-no_gui`	Disables GUI and Tk support. GUI will not be available later in the session.
`-no_logv`	Specifies that verbose log file will not be created.
`-overwrite`	Overwrites an existing .log file (and associated .cmd and .logv files) instead of adding an integer to make the files unique.

`-stylus`	Launches the Voltus Stylus Common UI interface.
`-version`	Returns the program version information.
`-wait` *minutes*	Specifies the number of minutes the system waits for a license to become available before exiting. *Default*: 0 (no wait time) *Value Range*: 0 to 10,000

Initializing Common User Interface

By default, various initialization files are loaded at startup, if they exist. They can be used to configure the GUI, load utility Tcl files, or configure Voltus settings. The initialization files are read in the following sequence:

1. `~/{.enc.pref.tcl, .enc.color.tcl}` in the home directory if the GUI is enabled. These files are intended to be created by *View -> Set Preference* and V*iew -> All Colors -> Save* options for a specific user.
2. `./{enc.pref.tcl, enc.color.tcl}` in the current directory if the GUI is enabled. These files are intended to be created by *View -> Set Preference* and V*iew -> All Colors -> Save* options for any user working in this directory.
3. `<`*`install_dir`*`>/etc/voltus/enc.tcl` in the installation directory. This is intended for CAD teams that want to set Tcl parameters, customize the GUI, or add user-defined Tcl commands that affect all users of this release.
4. `~/enc.tcl` in the home directory. This is intended for user-specific setting that should affect only one user.
5. `./enc.tcl` in the current directory. This is intended for design-specific setting that should affect only designs in this directory.

If the `-stylus` parameter is specified, the initialization files are loaded in the following order instead:

1. `~/.cadence/voltus/{gui.pref.tcl, gui.color.tcl, workspaces}` in the home directory if the GUI is enabled.
2. `.cadence/voltus/{gui.pref.tcl, gui.color.tcl, workspaces/}` in the current directory if the GUI is enabled.

3. `<install_dir>/etc/voltus/voltus.tcl` in the installation directory.
4. `~/.cadence/voltus/voltus.tcl` in the home directory.
5. `./.cadence/voltus/voltus.tcl` in the current directory.
6. `./voltus.tcl` in the current directory.

Setting Up the Run-Time Environment

If *install_dir* is the location of your Voltus installation, you should set up your run-time environment like this:

- Add the *install_dir*`/bin` directory to your path. The `bin` directory has links to all the public executables in the install hierarchy.
- If you want the Voltus Stylus Common UI man pages to be available from the Unix `man` command, you can add *install_dir*`/share/voltus/stylus/man` to your `MANPATH` envar.
- If you want the Tcl man pages to be available from the Unix `man` command, you can add *install_dir*`/share/tcltools/man` to your `MANPATH` envar.

For example, you might add this to your startup shell script:

```
set install_dir = /tools/voltus16.2/lnx86

set path = ($install_dir/bin $path)

setenv MANPATH
$install_dir/share/voltus/stylus/man:$install_dir/share/tcltools/man:$MANPATH
```

Note: When Voltus launches, it automatically adds either the legacy or the Voltus Stylus Common UI man pages (depending on the `-stylus` option) and the Tcl man pages to the beginning of the current `MANPATH` inside Voltus. Therefore, from within Voltus, the `man` command will see both sets of man pages before any other man pages.

Temporary File Locations

Each Voltus session creates its own temporary directory to store temporary files at the beginning of the run.

By default, `tmp_dir` is created in `/tmp`. If the Unix envar `TMPDIR` is set, then the `tmp_dir` directory is created inside `$TMPDIR`.

The name of the `tmp_dir` directory will look like:

```
voltus_temp_[pid]_[hostname]_[user]_xxxxxx
```

Where `_xxxxxx` is a string added to make the directory unique. For example:

```
voltus_temp_10233_farm254_bob_nfp9ez
```

The temporary directory is automatically removed on exit or if the run terminated with a catchable signal (e.g. `SIGSEGV`).

See also:

- Product and Licensing Information
- Getting Started

Index

A B C D E F G H I L M O P Q R S T U V W Z

A

add_filler_enable_restore_filler_flow
add_fillers_cells
add_fillers_check_trim_rule
add_fillers_prefix
add_route_via_defs
add_route_vias_auto
add_route_vias_ndr_only
add_thru_substrate_insts
add_what_if_shapes
analyze_jitter
analyze_resistance
analyze_resistance
analyze_resistance
analyze_resistance
auto_file_dir
auto_file_prefix

B

bindKey

C

check_ac_limit_additional_individual_violation_report
check_ac_limit_avg_recovery
check_ac_limit_check_thermal_aware_em
check_ac_limit_current_file
check_ac_limit_current_scale_factor
check_ac_limit_current_scale_table
check_ac_limit_default_freq_for_unconstrained_nets
check_ac_limit_delta_temperature
check_ac_limit_delta_temperature_layer_list
check_ac_limit_detailed
check_ac_limit_effort_level
check_ac_limit_em_cdf_percentage
check_ac_limit_em_limit_scale_factor
check_ac_limit_em_limit_scale_table
check_ac_limit_em_res_width
check_ac_limit_em_temperature
check_ac_limit_em_temperature_layer_list
check_ac_limit_em_threshold
check_ac_limit_enable_seb
check_ac_limit_env_temperature
check_ac_limit_extraction_tech_file
check_ac_limit_force_hold_view
check_ac_limit_handle_pin_obs_via
check_ac_limit_ict_em_models
check_ac_limit_ircx_models
check_ac_limit_lifetime
check_ac_limit_lifetime_rating_factor_file
check_ac_limit_max_error
check_ac_limit_method
check_ac_limit_min_peak_duty_ratio
check_ac_limit_min_peak_freq
check_ac_limit_net_file
check_ac_limit_nets
check_ac_limit_out_file
check_ac_limit_peak_td_method
check_ac_limit_read_detail_delta_temperature_file
check_ac_limit_report_db
check_ac_limit_report_unannotated_shapes
check_ac_limit_seb_lifetime
check_ac_limit_seb_table
check_ac_limit_seb_temperature
check_ac_limit_selected
check_ac_limit_simulation_net
check_ac_limit_simulation_net_file
check_ac_limit_skip_category_mode
check_ac_limit_skip_net
check_ac_limit_skip_net_file
check_ac_limit_toggle

check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path
check_ac_limit_use_db_freq
check_ac_limit_use_qrc_tech
check_ac_limit_use_rms_delta_t
check_ac_limit_view
check_ac_limits
check_connectivity
check_design
check_flow
check_multi_cpu_usage
check_netlist
check_pg_library
check_pg_shorts
check_pgv
check_power_domains
check_power_vias
clear_all_rulers
clone_msv_gate
cmd_file
commit_power_intent
compare_oa_cellview
compress_special
connect
connect_die_package
connect_hpin
connect_pin
convert_dbu_to_um
convert_stream_to_def
convert_um_to_dbu
create_cell_signal_em_model
create_flow
create_flow_step
create_gui_shape
create_hier_design
create_hinst
create_hnet
create_hport
create_inst
create_marker
create_metric_page
create_metric_table
create_metric_table_heading
create_module
create_oa_lib
create_oa_lib_property
create_parasitic_coordinate_transform
create_port
create_power_pads
create_ruler
create_snapshot
create_top_scope
cut_power_domain_by_overlaps

D

decompress_special
decrypt
define_attribute
define_feature
define_metric
define_proc
define_variables
delete_cell_obs
delete_dangling_ports
delete_floating_constants
delete_floating_nets
delete_flow_config
delete_inst
delete_markers
delete_metric
delete_metric_page
delete_metric_run
delete_metric_table
delete_metric_table_heading
delete_notch_fill
delete_obj
delete_power_intent
delete_thru_substrate_insts
deselect_obj
design_backside_bottom_routing_layer
design_backside_top_routing_layer
design_bottom_routing_layer
design_compressed_pg_db
design_early_clock_flow
design_high_freq_interposer_flow
design_ideal_hold_fixing
design_merge_trim_shapes
design_optimization_density_screen_margin
design_pessimistic_mode
design_process_node
design_tech_node
design_top_routing_layer
design_trim_grid_group
disconnect
disconnect_hpin
disconnect_pin
distributed_mmmc_disable_reports_auto_redirection

E

edit_flow
encrypt
encrypt_thermal_file
end_flow
etric_capture_design_image_route_drc
eval_legacy
extract_metal_density
extract_package
extract_parasitics
extract_rc_cap_filter_mode
extract_rc_compress_rcdb
extract_rc_coupled
extract_rc_coupling_cap_threshold
extract_rc_lef_tech_file_map
extract_rc_local_cpu
extract_rc_pvs_fill
extract_rc_qrc_cmd_file
extract_rc_qrc_cmd_type
extract_rc_qrc_run_mode
extract_rc_qrc_stream_map_file
extract_rc_relative_cap_threshold
extract_rc_shrink_factor
extract_rc_signoff_stream_layer_map
extract_rc_total_cap_threshold
extract_rc_turbo_reduce

F

flow_branch
flow_caller_data
flow_db_directory
flow_error_errorinfo
flow_error_message
flow_error_write_db
flow_exclude_time_for_init_flow
flow_exit_when_done
flow_feature_values
flow_features
flow_footer_tcl
flow_header_tcl
flow_hier_path
flow_history
flow_log_directory
flow_log_prefix_generator
flow_mail_on_error
flow_mail_to
flow_metrics_file
flow_metrics_snapshot_parent_uuid
flow_metrics_snapshot_uuid
flow_overwrite_db
flow_plugin_names
flow_plugin_steps
flow_post_db_overwrite
flow_remark
flow_report_directory
flow_run_tag
flow_schedule
flow_starting_db
flow_startup_directory
flow_status_file
flow_step_canonical_current
flow_step_check_tcl
flow_step_current
flow_step_last
flow_step_last_msg
flow_step_last_status
flow_step_next
flow_summary_tcl
flow_template_feature_definition
flow_template_tools
flow_template_type
flow_template_version
flow_top
flow_user_templates
flow_verbose
flow_working_directory
flow_write_db_snapshot
flow_write_db_snapshot_exclude_time
flow_yamllint_exec
flowtool
flowtool_exit_timeout
flowtool_extra_arguments
flowtool_metrics_qor_excel
flowtool_metrics_qor_html
flowtool_metrics_qor_text
flowtool_metrics_qor_vivid
flowtool_predict_full_names
flowtool_summary_tcl

G

generate_esd_rlrp_report
get_all_layers

get_cells
get_common_ui_map
get_computed_shapes
get_cpf_user_attributes
get_db
get_db_display_limit
get_default_switching_activity
get_distribute_host
get_dynamic_power_simulation
get_edge
get_feature
get_flow_config
get_ir_diagnostics
get_legacy
get_lib_cells
get_message
get_metric
get_metric_alias
get_metric_config
get_metric_definition
get_metric_header
get_nets
get_oa_default_rule_lib
get_oa_design_data
get_obj_in_area
get_pins
get_ports
get_power
get_rect
get_transform_shapes
get_via_pillars
get_visible_nets
get_voltus_diagnostics_mode
gui_add_marker
gui_add_ui
gui_attach_to_cursor
gui_bind_key
gui_clear_highlight
gui_clear_scan_chains
gui_clear_spare_cells
gui_create_user_bundle_net
gui_delete_objs
gui_delete_ui
gui_delete_user_bundle_net
gui_deselect
gui_dim_foreground
gui_fit
gui_get_ui
gui_hide
gui_highlight
gui_highlight_pin
gui_list_marker
gui_move_cursor
gui_open_schematic
gui_pan
gui_pan_center
gui_pan_page
gui_redraw
gui_remove_marker
gui_select
gui_select_highlighted
gui_set_power_rail_display
gui_set_power_rail_layers_nets
gui_set_tool
gui_set_ui
gui_set_visible_nets
gui_show
gui_show_buffer_tree
gui_show_edge_number
gui_show_markers
gui_show_scan_chains
gui_show_spare_cells
gui_show_user_bundle_nets
gui_verbose
gui_view_box
gui_write_picture
gui_zoom

H

help

I

init_check_netlist
init_delete_floating_hnets
init_design
init_flow
init_hier_cell
init_hier_design
init_keep_empty_modules
init_oa_tech_lib
init_power_intent_files
init_read_netlist_allow_verilog_error
init_sync_relative_path
init_timing_enabled

is_attribute
is_common_ui_mode
is_feature
is_flow

L

log_file
log_verbose_type
logv_file

M

man
map_dies
merge_vtm
metric_advanced_url_endpoint
metric_capture_3d_hotspots
metric_capture_depth
metric_capture_design_image
metric_capture_design_image_blockages_threshold
metric_capture_design_image_power_intent
metric_capture_max_drc_markers
metric_capture_min_count
metric_capture_overwrite
metric_capture_pba_tns_histogram
metric_capture_per_view
metric_capture_reg2reg_metrics
metric_capture_timing_path_groups
metric_capture_timing_paths
metric_capture_tns_histogram
metric_capture_tns_histogram_buckets
metric_capture_tns_histogram_max_slack
metric_capture_tns_histogram_paths
metric_capture_vth_metrics
metric_capture_vth_per_power_domain
metric_category_default
metric_current_run_id
metric_enable
metric_summary_metrics
monitor_distributed_hosts

O

oa2lef
oa_allow_analysis_only
oa_allow_bit_connection
oa_allow_tech_update
oa_bindkey_file
oa_cell_view_dir
oa_convert_diagonal_path_to
oa_cut_rows
oa_display_resource_file
oa_display_resource_file_in_library
oa_enclose_quick_abstract_pins
oa_full_layer_list
oa_full_path
oa_inst_placed_if_none
oa_lib_create_mode
oa_lock
oa_logic_only_import
oa_new_lib_compress_level
oa_pin_purpose
oa_push_pin_constraint
oa_silently_ignore_unsupported_vias
oa_text_purpose
oa_tie_net
oa_update_mode
oa_use_virtuoso_bindkey
oa_use_virtuoso_color
oa_view_sub_type
oa_write_mask_data_locked
oa_write_net_voltage
oa_write_relative_path

P

pop_snapshot_stack
power-grid library commands
power_add_simulation
power_adjust_input_activity_in_iterations
power_adjust_macro_activity_in_iterations
power_analysis_temperature
power_annotation_detail_report
power_auto_twf_delay_annotation
power_average_rise_fall_cap
power_block_independent_peakpower
power_boundary_gate_leakage_file
power_boundary_gate_leakage_report
power_boundary_leakage_cell_property_file
power_boundary_leakage_edge_annotation_file
power_boundary_leakage_multi_pgpin_support
power_boundary_leakage_pessimism_removal
power_boundary_leakage_pxe_support
power_bulk_pins

power_capacity
power_case_insensitive_mapping
power_clock_source_as_clock
power_comprehensive_automapping
power_compress_reports
power_constant_override
power_corner
power_create_driver_db
power_current_generation_method
power_db_name
power_decap_cell_list
power_default_frequency
power_default_slew
power_default_supply_voltage
power_detailed_view_current_only
power_disable_clock_gate_clipping
power_disable_leakage_scaling
power_disable_static
power_distribute_switching_power
power_distributed_combine_report_format
power_distributed_setup
power_domain_based_clipping
power_dynamic_glitch_filter
power_dynamic_scale_clock_by_frequency
power_dynamic_scale_clock_by_name
power_dynamic_vectorless_ranking_methods
power_effective_load_cap_slew_threshold_limit
power_enable_auto_mapping
power_enable_auto_queue
power_enable_disk_mapping
power_enable_duty_propagation_with_global_activity
power_enable_dynamic_current_slew_load_interpolation
power_enable_dynamic_scaling
power_enable_flop_state_propagation
power_enable_force_propagation
power_enable_generated_clock
power_enable_input_net_power
power_enable_interactive_reports
power_enable_mt_state_propagation
power_enable_multithread_reports
power_enable_pba_for_tempus_pi
power_enable_power_target_flow
power_enable_rtl_dynamic_vector_based
power_enable_scan_report
power_enable_single_cycle_scheduling
power_enable_slew_based_ccs_pin_cap
power_enable_stable_clock_gating
power_enable_stable_flop_scheduling
power_enable_state_propagation
power_enable_tempus_pi
power_enable_xp
power_enhanced_blackbox_avg
power_enhanced_blackbox_max
power_equivalent_annotation
power_event_based_leakage_power
power_external_load_config_file
power_extraction_tech_file
power_extractor_include
power_fanout_limit
power_force_library_merging
power_from_x_transition_factor
power_from_z_transition_factor
power_generate_activity_mapping_report
power_generate_current_for_rail
power_generate_flop_ranking_data
power_generate_leakage_power_map_based_on_calculated_leakage
power_generate_static_report_from_state_propagation
power_grid_libraries
power_handle_glitch
power_handle_transport_glitch
power_handle_tristate
power_hier_delimiter
power_honor_combinational_logic_on_clock_net
power_honor_negative_energy
power_honor_net_activity
power_honor_non_mission_leakage
power_hybrid_analysis
power_ignore_control_signals
power_ignore_data_phase_for_clock
power_ignore_end_toggles_in_profile
power_ignore_glitches_at_same_time_stamp
power_ignore_inout_pin_cap
power_ignore_macro_leakage_scale_for_temperature
power_ignore_unconstraint_net_with_timing_data
power_include_file
power_include_initial_x_transitions
power_include_sequential_clock_pin_power
power_include_timing_in_current_file
power_intent_do_not_use_top_domain_for_port_voltage
power_ir_derated_timing_view
power_keep_clock_gate_ratio_in_iterations
power_leakage_scale_factor_for_temperature
power_lib_files
power_library_preference
power_macro_pgv_leakage
power_macro_toggle_pin_percentage
power_match_state_for_logic_x
power_memory_current_scaling_method
power_merge_switched_net_currents
power_method

power_min_leaf_count
power_multibit_flop_toggle_behavior
power_off_pg_bulk_leakage
power_output_current_data_prefix
power_partition_count
power_pin_based_twf
power_pre_simulation_period
power_precision
power_read_rcdb
power_report_black_boxes
power_report_idle_instances
power_report_instance_switching_list
power_report_missing_bulk_connectivity
power_report_missing_nets
power_report_statistics
power_report_twf_attributes
power_reuse_flop_ranking_data_hier
power_scan_chain_activity
power_scan_control_file
power_settling_buffer
power_smart_window
power_start_time_alignment
power_stateprop_ignore_unannot_pins
power_static_netlist
power_thermal_leakage_temperature_scale_table_file
power_to_z_transition_factor
power_transition_time_method
power_twf_load_cap
power_use_cell_leakage_power_density
power_use_fastest_clock_for_dynamic_scheduling
power_use_only_ddv_current
power_use_twf_stable_randomized_arrival_delay
power_vector_based_multithread
power_worst_case_vector_activity
power_worst_window_count
power_worst_window_reports
power_worst_window_type
power_write_db
power_write_dynamic_currents
power_write_profiling_db
power_write_static_currents
power_x_transition_factor
power_zero_delay_vector_toggle_shift
program_major_version
program_short_name
push_snapshot_stack
power_multi_scenario_simulation
power_multibit_flop_toggle_pin_percentage
power_off_pg_nets
power_output_dir
power_partition_twf
power_pre_simulation_empty_period
power_pre_simulation_power_exclude_period
power_quit_on_activity_coverage_threshold
power_relax_arc_match
power_report_clock_instance_switching_info
power_report_instance_switching_info
power_report_library_usage
power_report_missing_input
power_report_scan_chain_statistics
power_report_time_display_fraction_digits
power_reuse_flop_ranking_data
power_scale_to_sdc_clock_frequency
power_scan_chain_name_pattern
power_scan_multi_bit_flop_chain_type
power_show_cell_usage_stats
power_split_bus_power
power_state_dependent_leakage
power_static_multi_mode_scenario_file
power_thermal_input_file
power_to_x_transition_factor
power_transition_factor_based_duty
power_twf_delay_annotation
power_unified_power_switch_flow
power_use_effective_load_cap_method
power_use_lef_for_missing_cells
power_use_physical_partition
power_use_zero_delay_vector_file
power_vector_profile_mode
power_worst_step_size
power_worst_window_coverage
power_worst_window_size
power_write_bbox
power_write_default_pti_files
power_write_gui_db
power_write_simulation_db
power_x_count_transition_using_3_states
power_z_transition_factor
print_message
program_name
program_version
puts

Q

query_power_data

R

Related Documents
read_db
read_db_file_check
read_db_version
read_flow
read_libs
read_metric
read_metric_html_package
read_netlist
read_physical
read_physical_check_mask_shifts
read_physical_extend_cell_obs_shapes_under_trim
read_physical_merge_multi_port_to_single_port
read_power_intent
read_sdc
read_temperature_map_file
redirect
relink_db_files
rename_pg_library
report_annotated_parasitics
report_dangling_ports
report_esd_network
report_floating_nets
report_freq_violation
report_inst_power
report_isolation
report_messages
report_metric
report_netlist_statistics
report_obj
report_obj_display_limit
report_partition_boundary_ports
report_power_rail_results
report_rcdb
report_signal_resistance
report_unannotated_nets
report_via_def
reset_db
reset_flow
reset_metric_run_id
resume
route_special_avoid_over_core_row_layer
route_special_block_pin_route_with_pin_width
route_special_core_pin_ignore_obs
route_special_core_pin_length_as_inst
route_special_core_pin_merge_limit
route_special_core_pin_reference_macro
route_special_core_pin_snap_to
read_activity_map_file
read_db_directory
read_db_tool_name
read_def
read_hier_db
read_markers
read_metric_config
read_mmmc
read_oa
read_physical_check_colored_shape
read_physical_context_data
read_physical_extend_polygon_cell_shapes
read_physical_must_join_all_ports
read_power_rail_results
read_spef
read_twf
register_trigger
rename_obj
rename_snapshot
report_area
report_differential_voltage
report_esd_voltage
report_flow
report_hidden_usage
report_irdrop_regions
report_markers
report_metal_stack
report_net_parasitics
report_noise_margin
report_obj
report_package
report_power_domains
report_rail
report_shifters
report_thermal
report_unit_parasitics
report_voltage
reset_design
reset_metric_config
reset_power_activity
route_special_allow_non_preferred_direction_route
route_special_block_pin_connect_ring_pin_corners
route_special_connect_broken_core_pin
route_special_core_pin_length
route_special_core_pin_max_via_scale
route_special_core_pin_refer_to_follow_pin
route_special_core_pin_site_rail_width
route_special_core_pin_stop_route

route_special_endcap_as_core
route_special_extend_nearest_target
route_special_jog_threshold_ratio
route_special_layer_non_preferred_direction_cost
route_special_layer_preferred_direction_cost
route_special_pad_pin_min_via_size
route_special_pad_pin_split
route_special_pad_ring_use_lef
route_special_pg_pin_as_signal
route_special_secondary_pin_max_gap
route_special_secondary_pin_rail_width
route_special_signal_pin_as_pg
route_special_split_long_via
route_special_target_number
route_special_target_search_distance
route_special_time_limit
route_special_via_connect_to_shape
route_special_via_through_to_closest_ring
route_special_welltap_as_endcap
run_flow
run_metric_category

S

scale_what_if_capacitance
scale_what_if_current
scale_what_if_resistance
schedule_flow
select_obj
selected
set_advanced_package_options
set_advanced_pg_library_mode
set_advanced_rail_options
set_cell_power_domain
set_compress_level
set_compress_level
set_db
set_db_verbose
set_db_verbose
set_die_model
set_distributed_hosts
set_dynamic_power_simulation
set_dynamic_rail_simulation
set_feature
set_flow_config
set_flowkit_read_db_args
set_flowkit_write_db_args
set_instance_temperature_file
set_layer_preference
set_license_check
set_limited_access_feature
set_max_transition_per_freq
set_message
set_metal_density_options
set_metric
set_metric_alias
set_metric_header
set_multi_cpu_usage
set_multi_die_analysis_mode
set_net_group
set_object_color
set_off_chip_package_trace
set_package
set_pg_library_mode
set_pg_nets
set_power_analysis_temperature
set_power_data
set_power_def_files
set_power_include_file
set_power_lib_files
set_power_output_dir
set_power_pads
set_preference
set_proc_verbose
set_rail_analysis_config
set_thermal_analysis_mode
set_twf_attribute
set_via_pillars
set_virtual_clock_network_parameters
set_voltage_regulator_module
set_voltus_diagnostics_mode
soft_stack_size_limit
sort_db
source
source_echo_filename
source_verbose
source_verbose_line_length_limit
specify_oa
summaryReport
suspend
switching_power_on_rise_only

T

tcl_error_info
tech_db_units

tech_finfet_grid_direction
tech_finfet_grid_offset
tech_finfet_grid_pitch
tech_inst_mask_shift_layers
tech_mfg_grid

U

uiFind
ui_precision
ui_precision_capacitance
ui_precision_derating
ui_precision_power
ui_precision_sensitivities
ui_precision_timing
uniquify
uniquify_filename
unlock_oa_design
update_inst
update_inst_name
update_metric
update_metric_definition
update_oa_lib
update_power_domain

V

view_dynamic_movie
view_dynamic_waveform
view_last
view_log
view_next
view_package_results
voltus

W

write_current_region
write_db
write_db_auto_save_user_globals
write_db_create_read_file
write_db_include_metal_fill_rules
write_def
write_def_lef_out_version
write_die_model
write_flow
write_flow_template
write_hier_db
write_hier_view
write_lef_library
write_metric
write_metric_config
write_mmmc
write_netlist_full_pin_out
write_netlist_port_association_style
write_oa_techfile
write_parasitics
write_pg_library
write_physical_context_data
write_power_constraints
write_power_intent
write_power_up_report
write_preserves
write_rcdb
write_tcf
write_tech_pg_lib
write_thermal_model
write_to_gif
write_twf
write_via_defs
write_xpgv

Z

zoomTo

21

RTL-to-Gate Rule Mapping

Rule Type	RTL	Gate	Remarks
Reg Pattern	Hier/flop	Hier/flop_reg	
	Hier/flop_reg	Hier/flop_reg_reg	
	Hier/flop[1]	Hier/flop_1	
		Hier/flop_x1x	
		Hier/flop 1	flop {space} 1
		Hier/flop\[1\]	
		Hier/flop_x1	
Reg + Index Match	Hier/flop[1]	Hier/flop_reg_1	
		Hier/flop_regx1x	
Multi Index Match	Hier/flop[1][2]	Hier/flop_1_2	
		Hier/flop 1 2	
		Hier/flop\[1\]\[2\]	flop {space} 1 {space} 2
		Hier/flop\[1\]_2	
		Hier/flop_1_\[2\]	
		Hier/flop_1\[2\]	
		Hier/flop.1.2	
		Hier/flop.1_2	
		Hier/flop\[1\] 2	combination of '.' '[' and ' ' for index separation
		Hier/flop\[1\].2	
		Hier/flop/1_2	
		Hier/flop/1/2	
		Hier/flop_x1xx2x	

		Hier/flop_1_xxx_2	_xxx_ is treated as a separator
Multi Bit Flip Flop	Hier/dff_Q_reg[1]	Hier/CDN_MBIT_dff_Q_reg\[1\]_MB_dff_Q_reg\[2\]_MB_dff_Q_reg\[3\]\Q1	
	Hier/dff_Q_reg[2]	Hier/CDN_MBIT_dff_Q_reg\[1\]_MB_dff_Q_reg\[2\]_MB_dff_Q_reg\[3\]\Q2	
	Hier/dff_Q_reg[3]	Hier/CDN_MBIT_dff_Q_reg\[1\]_MB_dff_Q_reg\[2\]_MB_dff_Q_reg\[3\]\Q3	
	Hier/dff_Q_reg[1]	Hier/CDN_MBIT_dff_Q_reg\[1\]___dff_Q_reg\[2\]___dff_Q_reg\[3\]\Q1	
	Hier/dff_Q_reg[2]	Hier/CDN_MBIT_dff_Q_reg\[1\]___dff_Q_reg\[2\]___dff_Q_reg\[3\]\Q2	
	Hier/dff_Q_reg[3]	Hier/CDN_MBIT_dff_Q_reg\[1\]___dff_Q_reg\[2\]___dff_Q_reg\[3\]\Q3	
	Hier/Q_reg[1]	Hier.CDN_MBIT_Q_reg\[1\]_MB_Q_reg\[2\]_MB_Q_reg\[3\]\Q1	
	Hier/Q_reg[2]	Hier.CDN_MBIT_Q_reg\[1\]_MB_Q_reg\[2\]_MB_Q_reg\[3\]\Q2	
	Hier/Q_reg[3]	Hier.CDN_MBIT_Q_reg\[1\]_MB_Q_reg\[2\]_MB_Q_reg\[3\]\Q3	
	Hier/q_reg[1]	Hier/CDN_MBIT_q_reg_CDN_CPX_\[1\]_MB_\[2\]_MB_\[3\]\Q1	
	Hier/q_reg[2]	Hier/CDN_MBIT_q_reg_CDN_CPX_\[1\]_MB_\[2\]_MB_\[3\]\Q2	
	Hier/q_reg[3]	Hier/CDN_MBIT_q_reg_CDN_CPX_\[1\]_MB_\[2\]_MB_\[3\]\Q3	
	Hier/prefix_Q_reg[1]	Hier/MBFF_prefix_Q_reg_[1]_MB_Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/MBFF_prefix_Q_reg_[1]_MB_Q_reg_[2]/Q2	
	Hier/prefix_Q_reg[1]	Hier/MBFF_prefix_Q_reg_CDN_CPX_[1]_MB_Q_reg_[2]\Q1	
	Hier/prefix_Q_reg[2]	Hier/MBFF_prefix_Q_reg_CDN_CPX_[1]_MB_Q_reg_[2]\Q2	
	Hier/prefix_Q_reg[1]	Hier/MBFF_prefix_Q_reg_[1]___Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/MBFF_prefix_Q_reg_[1]___Q_reg_[2]/Q2	
	Hier/prefix_Q_reg[1]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_[1]_MB_Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_[1]_MB_Q_reg_[2]/Q2	
	Hier/prefix_Q_reg[1]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_CDN_CPX_[1]_MB_Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_CDN_CPX_[1]_MB_Q_reg_[2]/Q2	
	Hier/prefix_Q_reg[1]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_CDN_CPX_[1]___Q_reg_[2]/Q1	
	Hier/prefix_Q_reg[2]	Hier/CDN_MBIT_MBFF_prefix_Q_reg_CDN_CPX_[1]___Q_reg_[2]/Q2	

Hierarchy Merge	A/B	A_B	
Mixed Hierarchy	A/B/C	A/B_C	
		A_B/C	
Flatten Array	A[0]	A_0_	
	A[1]	A_1_	
Flatten Double Array	A[0][1]	A_0_1	
Flatten Array with Hierarchy Merge	A[0]/B	A_0_B	
Dot Separator	A/B	A.B	
Mixed Dot Separator	A/B/C	A/B.C	
		A.B/C	
Regex			
Continuous Index/Double Index	A/B/C[0]	A/B/C[0:0]	
Multiple __	A/B/C	A__B/C	
xxx	fifoctr[1]	fifoctr_reg_xxx_1	